# DevOps, Cloud & Platform Engineering

Complete Notes v2.0: from zero to job-ready

Networking · Linux · Git · Bash · Python · AWS · Docker
Terraform · Ansible · CI/CD · Kubernetes · Helm · EKS
Monitoring · SRE · DevSecOps · GitOps · Platform Engineering · Interview Q&A

**Thirupathi Keloth**

100-Day Learning Program · 2026

# Contents

1. Computer & IT Basics

2. Networking Basics for DevOps

3. Linux

4. Git & GitHub

5. Bash Scripting

6. Python for DevOps Automation

7. AWS Cloud

8. Docker

9. Terraform (Infrastructure as Code)

10. Ansible (Configuration Management)

11. CI/CD

12. Kubernetes

13. Monitoring & SRE

14. DevSecOps

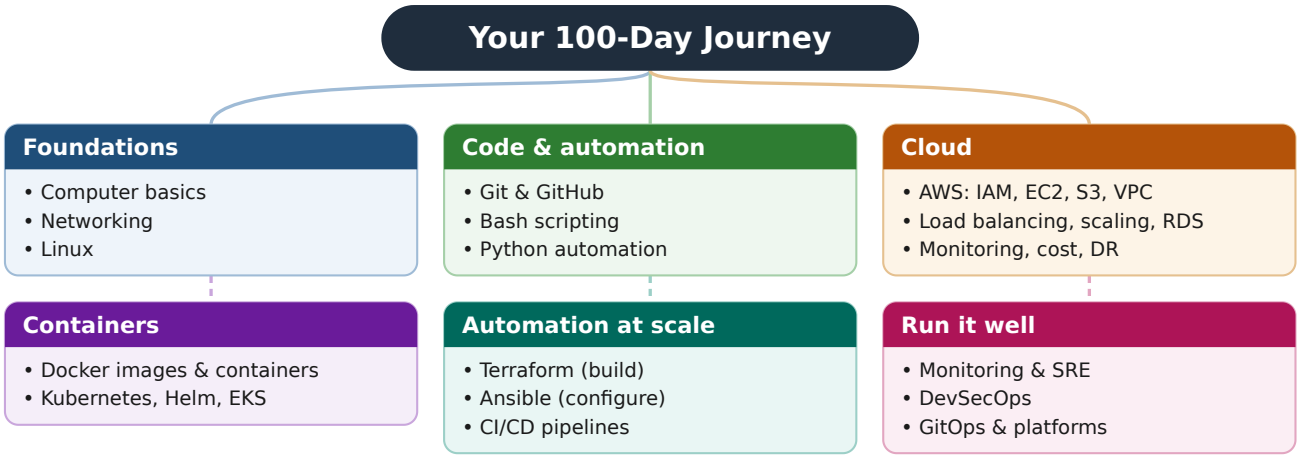
15. GitOps & Argo CD

16. Platform Engineering

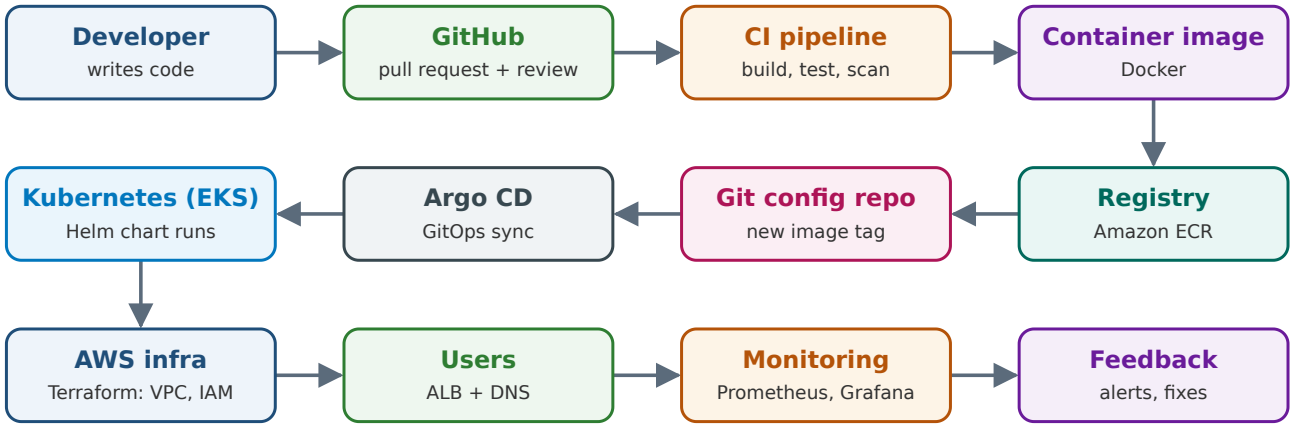
17. Troubleshooting

18. Interview Preparation

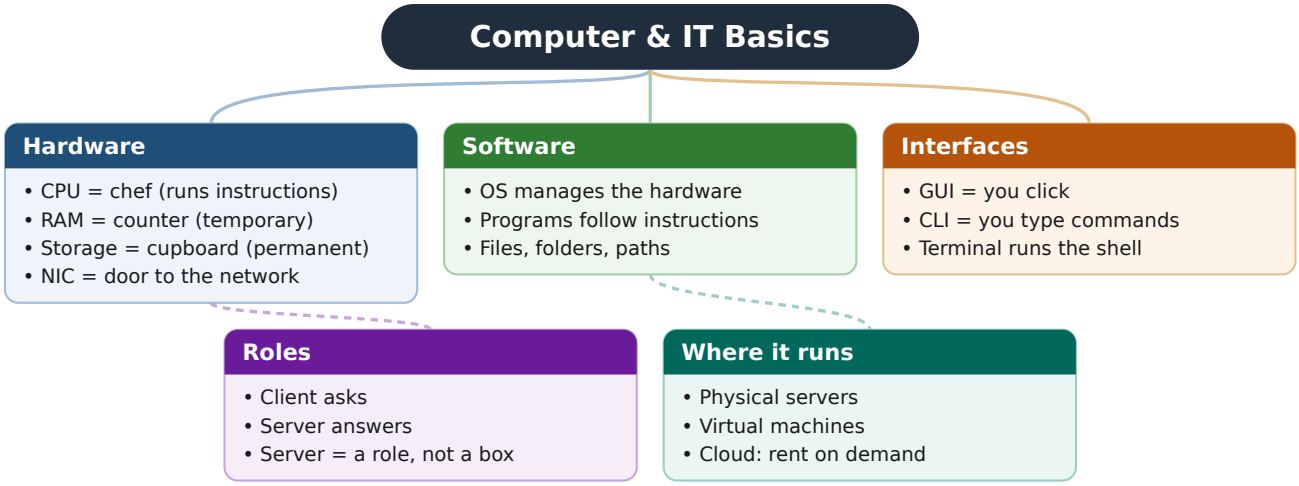
# The Big Picture




# How it all connects (your final project)



# 1. Computer & IT Basics




## 1.0 Why this topic matters

Everything in DevOps and cloud runs on computers: physical servers in data centers, virtual machines, containers. When you choose an EC2 instance type, size a Kubernetes pod, debug a slow server or explain why a disk is full, you use the concepts in this topic. Learn them well once; every later topic builds on them.

## 1.1 What a computer does

```
INPUT ──► PROCESSING ──► OUTPUT
(keyboard,    (CPU + RAM)      (screen, network,
 network,          │             disk, speakers)
 disk)             ▼
              STORAGE (keeps data when power is off)
```

A computer runs **programs** (lists of instructions) on **data**. The CPU executes instructions; RAM holds the program and data being worked on; storage keeps everything permanently; the network connects it to other computers.

## 1.2 Hardware components in depth

### CPU (Central Processing Unit)

| Term | Meaning | Why it matters in DevOps/cloud |
|---|---|---|
| Core | An independent processing unit inside the CPU | More cores = more work in parallel (web servers, builds) |
| Thread / hyper-threading (SMT) | One core runs two instruction streams | Cloud **vCPU** = usually one hardware thread, not a full core |
| Clock speed (GHz) | Instruction cycles per second | Affects single-threaded performance |
| Cache (L1/L2/L3) | Very fast memory inside the CPU | Big caches help databases and compute-heavy work |

| Term | Meaning | Why it matters in DevOps/cloud |
|---|---|---|
| Architecture | **x86-64** (Intel/AMD) or **ARM64** (AWS Graviton, Apple M-series, phones) | Software and container images must be built for the right architecture ( `amd64` vs `arm64` ) |

**CPU usage** (%) shows how busy the cores are. **Load average** (Linux) shows how many tasks are running or waiting.

## Memory (RAM)

- Fast, temporary working space; contents are lost when power goes off.
- Programs must be loaded into RAM to run.
- When RAM is full, the OS uses **swap** (disk as slow overflow memory) or kills processes (Linux **OOM killer**).
- Measured in GB/GiB. In the cloud you pick RAM per instance type (e.g. 1 GiB on `t3.micro`, 8 GiB on `m7i.large`) or per container/pod (memory requests/limits).

## Storage

| Type | How it works | Speed | Use |
|---|---|---|---|
| HDD (hard disk drive) | Spinning magnetic platters + moving head | Slow random access, cheap | Archives, backups, bulk data |
| SSD (SATA) | Flash memory, no moving parts | Fast | General use |
| NVMe SSD | Flash on the PCIe bus | Very fast | Databases, modern servers, cloud instance storage |
| Network/block storage | Disk provided over the network (e.g. AWS EBS) | Depends on configuration (IOPS, throughput) | Cloud servers |
| Object storage | Files stored via HTTP APIs (e.g. S3) | High throughput, higher latency per request | Backups, static files, data lakes |

**Performance terms: IOPS** (input/output operations per second: many small reads/writes, e.g. databases), **throughput** (MB/s: large sequential reads/writes, e.g. video, logs), **latency** (time per operation, ms).

## Other components

| Component | Role |
|---|---|
| Motherboard | Connects everything (CPU socket, RAM slots, PCIe, storage ports) |
| GPU | Massively parallel processor: graphics, machine learning, video encoding (cloud: GPU instances) |
| NIC (network interface card) | Connects to the network; has a MAC address; speeds 1/10/25/100 Gbps |
| PSU | Power supply; servers have **redundant** PSUs |
| BIOS/UEFI firmware | First code that runs at power-on; tests hardware and starts the bootloader |

## Servers vs personal computers

Servers are built for **reliability and remote management**: redundant power supplies and disks (RAID), ECC memory (detects/corrects memory errors), many cores and lots of RAM, out-of-band

management (iLO/iDRAC/IPMI: manage the machine even when the OS is down), rack-mounted (1U/2U), no monitor/keyboard. They usually run Linux without a graphical interface.

## 1.3 Units and number systems

### Data units

| Unit | Size |
|---|---|
| Bit (b) | 0 or 1 |
| Byte (B) | 8 bits |
| Kilobyte KB / Kibibyte KiB | 1,000 bytes / 1,024 bytes |
| MB / MiB | $1{,}000^2$ / $1{,}024^2$ bytes |
| GB / GiB | $1{,}000^3$ / $1{,}024^3$ bytes |
| TB / TiB | $1{,}000^4$ / $1{,}024^4$ bytes |

Disk makers use decimal (GB); operating systems and Kubernetes often use binary (GiB, `Mi`, `Gi`). A "500 GB" disk shows as ~465 GiB.

**Network speeds are in bits** per second (Mbps, Gbps); **file sizes in bytes**. 100 Mbps ≈ 12.5 MB/s.

### Number systems

| System | Base | Digits | Where you see it |
|---|---|---|---|
| Decimal | 10 | 0–9 | Everyday numbers, IPv4 written form |
| Binary | 2 | 0–1 | How computers store everything; subnet masks |
| Hexadecimal | 16 | 0–9, A–F | MAC addresses, IPv6, memory addresses, colors, hashes |
| Octal | 8 | 0–7 | Linux permissions ( `chmod 755` ) |

Conversion methods are covered step by step in the Networking notes (binary and subnetting chapter).

### Text encoding

Computers store text as numbers. **ASCII** covers English letters in 7 bits; **UTF-8** covers every language and emoji and is the standard everywhere (files, web, JSON). Mismatched encodings produce garbled characters; Windows vs Linux **line endings** ( `\r\n` vs `\n` ) break shell scripts edited on Windows ( `dos2unix script.sh` fixes it).

## 1.4 Operating systems

An **operating system (OS)** manages hardware and gives programs a safe, shared way to use it.

| OS responsibility | What it means |
|---|---|
| Process management | Starts/stops programs, shares CPU time between them (scheduling) |
| Memory management | Gives each process its own **virtual memory**, protects processes from each other, uses swap |
| File systems | Organizes data on disks into files and directories with permissions |
| Device management | Drivers let the OS talk to hardware |

| OS responsibility | What it means |
|---|---|
| Networking | TCP/IP stack, sockets |
| Security | Users, permissions, isolation |
| User interface | Shell (CLI) and/or graphical desktop (GUI) |

### Kernel and user mode

The **kernel** runs with full hardware access (**kernel mode**). Normal programs run in **user mode** and must ask the kernel for anything privileged via **system calls** (read a file, open a network connection, start a process). A crash in a program doesn't take down the whole system; a crash in the kernel does.

### Processes, threads and memory

| Concept | Meaning |
|---|---|
| Program | Instructions stored in a file |
| Process | A running program with its own memory space and an ID (PID) |
| Thread | A unit of execution inside a process; threads share the process memory |
| Virtual memory | Each process sees its own address space; the OS maps it to physical RAM or swap |
| Context switch | The OS pausing one task and running another, many times per second |

### Common operating systems

| OS | Where |
|---|---|
| Linux (Ubuntu, RHEL, Amazon Linux, Debian, Alpine) | Servers, cloud, containers, Kubernetes, embedded, Android |
| Windows / Windows Server | Office PCs, Active Directory, .NET apps, some enterprise servers |
| macOS | Developer laptops |
| Unix (AIX, Solaris) | Legacy enterprise systems |

### Boot process (any computer)

Power on → firmware (BIOS/UEFI) checks hardware (POST) → bootloader loads the OS kernel → kernel initializes hardware and starts the first process → system services start → login screen/prompt.

## 1.5 Software

| Type | Examples |
|---|---|
| System software | OS, drivers, firmware |
| Application software | Browsers, databases, web servers, your company's app |
| Middleware | Message queues, application servers (Tomcat), API gateways |
| Utilities / tools | Git, Docker, Terraform, kubectl |

### How programs are built and run

| Model | How | Examples |
|---|---|---|
| Compiled | Source code → compiler → machine-code binary for one OS/architecture | C, C++, Go, Rust |
| Interpreted | An interpreter reads and runs the code | Python, Bash, Ruby, JavaScript (Node.js) |
| Bytecode + VM | Compiled to bytecode, run by a virtual machine | Java (JVM), C# (.NET) |

**Dependencies** are libraries a program needs. Managing them (versions, conflicts) is a big part of DevOps; containers solve much of this by packaging the app with its dependencies.

### Open source and licenses

Open-source software has publicly available source code (Linux, Kubernetes, Terraform/OpenTofu, Python). Licenses decide how you may use it: **permissive** (MIT, Apache 2.0, BSD: use freely, keep notices) vs **copyleft** (GPL: modified versions you distribute must stay open). Companies check licenses before using libraries.

### Versions

**Semantic versioning** `MAJOR.MINOR.PATCH` (e.g. `2.7.1`): major = breaking changes, minor = new features, patch = bug fixes. **LTS** = long-term support release.

## 1.6 Files, folders and paths

| | Windows | Linux / macOS |
|---|---|---|
| Root | Drive letters `C:\` | One tree starting at `/` |
| Separator | `\` | `/` |
| Home | `C:\Users\thirupathi` | `/home/thirupathi` (`~`) |
| Case sensitive | No | **Yes** (`File.txt` ≠ `file.txt`) |
| Hidden files | Attribute | Name starts with `.` |
| Executable | By extension (`.exe`, `.bat`) | By **permission** (`chmod +x`) |

**Absolute path** = full path from the root. **Relative path** = from the current folder. **File extension** is only a hint (`file` command shows the real type on Linux).

## 1.7 The command line

A **CLI (command-line interface)** lets you control a computer by typing commands; a **GUI** uses windows and clicks.

**Why professionals use the CLI:** servers have no GUI; commands can be **scripted, automated, repeated and version-controlled**; it's faster and precise; remote access over SSH is text-based; every DevOps tool (git, docker, kubectl, terraform, aws) is a CLI.

| Term | Meaning |
|---|---|
| Terminal | The window/app showing a text session (Windows Terminal, GNOME Terminal) |

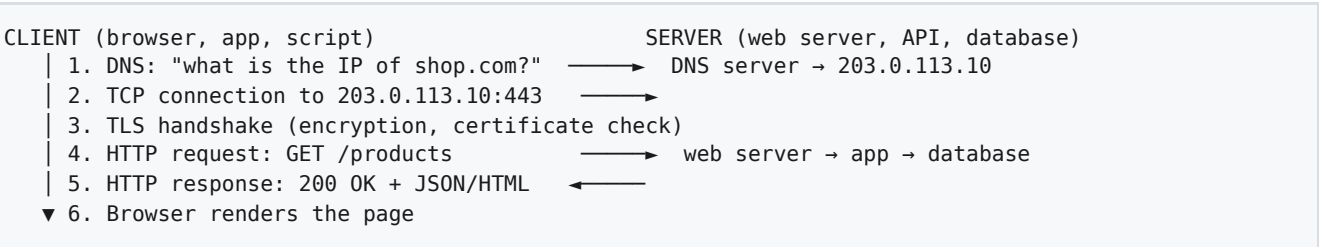
| Term | Meaning |
|---|---|
| Shell | The program interpreting commands (PowerShell, cmd, bash, zsh) |
| Prompt | Text showing the shell is ready ( `PS C:\Users\me>` , `user@host:~$` ) |
| Command / options / arguments | `ls -l /var/log` → command `ls` , option `-l` , argument `/var/log` |

### Windows PowerShell essentials

| Task | PowerShell | Linux equivalent |
|---|---|---|
| Where am I | `Get-Location` ( `pwd` ) | `pwd` |
| List files | `Get-ChildItem` ( `ls` , `dir` ) | `ls` |
| Change folder | `Set-Location` ( `cd` ) | `cd` |
| Make folder | `New-Item -ItemType Directory name` ( `mkdir` ) | `mkdir` |
| Read file | `Get-Content file` ( `cat` ) | `cat` |
| Copy / move / delete | `Copy-Item` , `Move-Item` , `Remove-Item` | `cp` , `mv` , `rm` |
| Computer name / user | `hostname` , `whoami` | `hostname` , `whoami` |
| Network config | `ipconfig /all` , `Get-NetIPAddress` | `ip a` |
| Test reachability / port | `ping host` , `Test-NetConnection host -Port 443` | `ping` , `nc -zv` |
| DNS lookup | `Resolve-DnsName name` , `nslookup` | `dig` , `nslookup` |
| Processes | `Get-Process` , `Stop-Process -Id 123` | `ps` , `kill` |
| Services | `Get-Service` , `Restart-Service name` | `systemctl` |
| Help | `Get-Help Get-Process -Examples` | `man` |

**WSL2 (Windows Subsystem for Linux)** runs a real Linux kernel inside Windows: `wsl --install` (admin PowerShell), then use Ubuntu like a Linux server. This is the recommended setup for learning DevOps on Windows.

## 1.8 Client–server model and how the web works




| Term | Meaning |
|---|---|
| Client | Requests a service |
| Server | Provides a service; "server" is a **role** (software listening on a port), usually on a dedicated machine |
| Request / response | The basic exchange (HTTP, database queries, API calls) |

| Term | Meaning |
|---|---|
| Port | Number identifying the service on a machine (443 HTTPS, 22 SSH, 5432 PostgreSQL) |
| API | A defined way for programs to talk to each other (often HTTP + JSON) |
| Stateless vs stateful | Stateless servers keep no session data between requests (easy to scale); stateful ones do (databases) |

### Common architectures

| Architecture | Description |
|---|---|
| Monolith | One application containing all features; simple to start, harder to scale teams |
| 3-tier | Presentation (web) → application (logic) → data (database) |
| Microservices | Many small services communicating over APIs; independent deployment and scaling, more operational complexity |
| Serverless | Functions run on demand by a cloud provider (AWS Lambda) |

## 1.9 Networks and the internet (overview)

| Term | Meaning |
|---|---|
| Network | Connected devices sharing data |
| LAN / WAN | Local network (home, office) / wide area network (connects sites; the internet is the biggest) |
| IP address | Logical address of a device ( `192.168.1.10` ) |
| MAC address | Hardware address of a network card |
| Router / switch | Connects networks / connects devices inside one network |
| DNS | Translates names to IP addresses |
| ISP | Internet service provider |
| Bandwidth / latency | Capacity (Mbps) / delay (ms) |

Full detail: **Networking Basics** notes (topic 2) and the **Computer Networking** book.

## 1.10 Data centers, virtualization and cloud

### Data centers

Buildings full of **racks** of servers, storage and network equipment, with redundant power (UPS, generators), cooling, physical security and multiple internet connections. Companies either run their own (**on-premises**), rent space (**colocation**), or use **cloud** providers who run massive data centers for them.

### Virtualization

```
Without virtualization:   1 physical server → 1 OS → apps (often 10–20% used)
With virtualization:      1 physical server → HYPERVISOR → many VMs, each with its own OS
```

| Term | Meaning |
|---|---|
| Hypervisor | Software that runs virtual machines. **Type 1** runs directly on hardware (VMware ESXi, KVM, Hyper-V, AWS Nitro); **Type 2** runs on a normal OS (VirtualBox, VMware Workstation) |
| Virtual machine (VM) | A complete simulated computer with its own OS |
| Host / guest | Physical machine / VM running on it |
| Snapshot | Saved state of a VM |

**Benefits:** better hardware utilization, isolation, fast provisioning, snapshots, live migration. Every cloud instance (EC2) is a VM.

### Containers

Containers package an app with its libraries and share the **host's kernel**: lighter and faster than VMs (MBs vs GBs, seconds vs minutes). Docker builds and runs them; Kubernetes runs them at scale. Covered fully in the Docker and Kubernetes notes.

| | VM | Container |
|---|---|---|
| Includes | Full guest OS | App + dependencies |
| Isolation | Strong (own kernel) | Process-level (shared kernel) |
| Size / start | GBs / minutes | MBs / seconds |

### Cloud computing

Renting compute, storage, networking and services on demand over the internet, paying per use. Models: **IaaS** (VMs, networks), **PaaS** (managed platforms/databases), **SaaS** (finished software). Providers: AWS, Microsoft Azure, Google Cloud. Covered fully in the AWS notes.

## 1.11 Security fundamentals

| Concept | Meaning |
|---|---|
| CIA triad | **Confidentiality** (only authorized people see data), **Integrity** (data isn't altered), **Availability** (systems are usable when needed) |
| Authentication | Proving who you are (password, key, certificate) |
| Authorization | What you are allowed to do |
| MFA | Two or more factors: something you know (password), have (phone/key), are (fingerprint) |
| Least privilege | Give only the access needed, nothing more |
| Principle of defense in depth | Multiple layers of protection |
| Patch | Update that fixes bugs/vulnerabilities; patching is a core ops task |
| Vulnerability / CVE | A weakness / a public ID for a known vulnerability (CVE-2024-xxxx) |

### Encryption basics

| Type | How | Used in |
|---|---|---|
| Symmetric | Same secret key encrypts and decrypts (AES) | Disk encryption, bulk data, inside TLS sessions |
| Asymmetric | Key **pair**: public key encrypts/verifies, private key decrypts/signs (RSA, ECDSA, Ed25519) | SSH keys, TLS certificates, digital signatures |
| Hashing | One-way fingerprint of data (SHA-256); can't be reversed | Password storage (with salt, bcrypt/argon2), file integrity checksums, Git commit IDs |
| Encoding (not security!) | Reversible format change (Base64) | Transport of binary data; Kubernetes Secrets are only Base64 by default |

**Golden rules:** never share private keys or passwords, never commit secrets to Git, use a password manager and MFA, keep systems patched.

## 1.12 IT roles and how they work together

| Role | Focus |
|---|---|
| Software developer | Builds application features |
| QA / test engineer | Tests quality, automates tests |
| System administrator | Manages servers and OS |
| Network engineer | Designs and runs networks |
| Database administrator (DBA) | Databases: performance, backups, security |
| Security engineer | Protects systems and data |
| **DevOps engineer** | Automates build, test, deploy and infrastructure; bridges dev and ops |
| **Cloud engineer** | Designs and operates cloud infrastructure |
| **SRE** | Reliability through engineering: SLOs, monitoring, incident response |
| **Platform engineer** | Builds internal platforms and self-service tools for developers |
| Cloud support engineer | Helps customers troubleshoot cloud issues |

**Software delivery lifecycle (SDLC):** plan → code → build → test → release → deploy → operate → monitor → feedback → plan. DevOps automates and connects these steps.

## 1.13 Basic troubleshooting mindset

1. **Reproduce and define:** what exactly fails, for whom, since when? Get the exact error message.
2. **What changed?** Updates, deployments, config changes, network changes.
3. **Check the simple things first:** power, cable/Wi-Fi, disk space, is the service running, correct credentials.
4. **Divide and conquer:** split the system into layers (network → OS → service → app → data) and test each.
5. **One change at a time,** then test again.
6. **Read logs and error messages carefully:** search the exact message.
7. **Document** what you found and fixed.

## 1.14 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | RAM vs storage? | RAM is fast temporary working memory, cleared at power off; storage is slower and permanent |
| 2 | What is a vCPU in the cloud? | Usually one hardware thread of a physical core |
| 3 | HDD vs SSD vs NVMe? | Spinning disk (slow, cheap) / flash (fast) / flash over PCIe (fastest) |
| 4 | IOPS vs throughput? | Operations per second (small random I/O) vs MB/s (large sequential I/O) |
| 5 | What does an operating system do? | Manages processes, memory, files, devices, networking, security; gives programs an interface to hardware |
| 6 | Kernel mode vs user mode? | Kernel has full hardware access; programs run restricted and use system calls |
| 7 | Process vs thread? | A process has its own memory; threads inside a process share memory |
| 8 | What is virtual memory / swap? | Each process gets its own address space; swap uses disk when RAM is full |
| 9 | Compiled vs interpreted languages? | Compiled to machine code ahead of time vs executed by an interpreter at runtime |
| 10 | What is a hypervisor? Type 1 vs 2? | Software running VMs; type 1 on bare metal, type 2 on a host OS |
| 11 | VM vs container? | VM has its own OS kernel; container shares the host kernel and is lighter |
| 12 | What is the client-server model? | Clients send requests; servers listen on ports and respond |
| 13 | Why use the CLI? | Automation, remote access, precision, speed; all DevOps tools are CLIs |
| 14 | Explain the CIA triad | Confidentiality, integrity, availability: the three goals of security |
| 15 | Symmetric vs asymmetric encryption? | One shared key vs public/private key pair |
| 16 | Hashing vs encryption vs encoding? | One-way fingerprint / reversible with a key / reversible format change with no security |
| 17 | What is MFA? | Authentication with two or more different factor types |
| 18 | What is semantic versioning? | MAJOR.MINOR.PATCH: breaking / feature / fix |
| 19 | Why are Linux paths case-sensitive problems common? | `Config.yaml` and `config.yaml` are different files on Linux but not on Windows |
| 20 | What is x86-64 vs ARM64 and why does it matter? | Different CPU architectures; binaries and container images must match the architecture |

## 1.15 Self-check: I can…

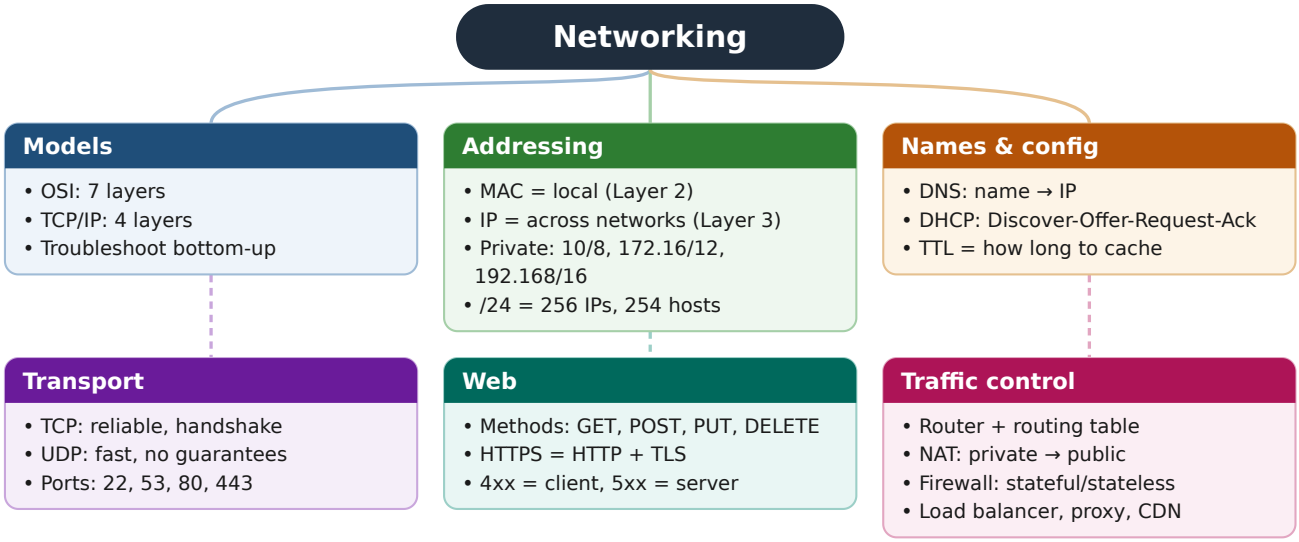
- [ ] Explain CPU (cores, threads, architecture), RAM, storage types and their performance terms.
- [ ] Convert between bits and bytes and explain GB vs GiB and Mbps vs MB/s.
- [ ] Explain what an OS does, kernel vs user mode, processes, threads, virtual memory.
- [ ] Explain compiled vs interpreted software, dependencies, open-source licenses and versioning.
- [ ] Use basic PowerShell commands and explain why WSL2 is useful.

- [ ] Describe the client-server model and the steps of loading a website.
- [ ] Explain data centers, virtualization (hypervisors, VMs), containers and cloud models.
- [ ] Explain the CIA triad, authentication vs authorization, MFA, encryption, hashing and encoding.
- [ ] Name the main IT roles and how DevOps, Cloud, SRE and Platform engineers fit in.
- [ ] Apply a structured troubleshooting approach.

**REMEMBER**

- Every computer: **Input → Process → Store → Output**.
- RAM is fast but forgets at power-off; storage keeps data.
- Servers usually run **Linux**, have no screen, and are managed through a **terminal** over a network.

# 2. Networking Basics for DevOps




## 2.0 Why networking matters for DevOps

Most production incidents involve the network somewhere: DNS not resolving, a security group blocking a port, a certificate expiring, a load balancer marking targets unhealthy, a pod that can't reach a database. Cloud (VPCs), containers (bridges, port publishing) and Kubernetes (Services, Ingress, NetworkPolicies) are all networking. These notes cover what a DevOps/Cloud/SRE engineer uses daily. The separate **Computer Networking** book goes deeper into switching, routing protocols and CCNA topics.

## 2.1 Network fundamentals

| Term | Meaning |
|---|---|
| Network | Devices connected to exchange data |
| Host / node | Any device with an address (server, laptop, container, router) |
| LAN / WAN | Local network (one site) / wide area network (connects sites; the internet) |
| Packet | A unit of data with headers (addresses) and payload |
| Protocol | Agreed rules for communication (IP, TCP, HTTP, DNS) |
| Bandwidth / throughput | Maximum capacity / actually achieved rate |
| Latency / RTT | One-way delay / round-trip time (ms) |
| Jitter / packet loss | Variation in delay / packets that never arrive |

| Device | Layer | Job |
|---|---|---|
| Switch | 2 | Connects devices in a LAN using MAC addresses |
| Router | 3 | Connects networks; forwards packets by IP address |
| Firewall | 3–7 | Allows/blocks traffic by rules |

| Device | Layer | Job |
|---|---|---|
| Load balancer | 4–7 | Spreads traffic across servers |
| Access point | 1–2 | Wi-Fi |
| Proxy / reverse proxy | 7 | Forwards requests on behalf of clients / servers |

## 2.2 OSI and TCP/IP models

| OSI layer | TCP/IP layer | Data unit | Addressing / examples | DevOps view: what breaks here |
|---|---|---|---|---|
| 7 Application | Application | Data | HTTP, DNS, SSH, TLS-protected apps | Wrong URL, 4xx/5xx, bad API response |
| 6 Presentation | Application | Data | TLS, encoding, compression | Certificate errors, encoding issues |
| 5 Session | Application | Data | Sessions | Session timeouts |
| 4 Transport | Transport | Segment / datagram | **Ports**, TCP, UDP | Port closed, firewall, connection refused/timeouts |
| 3 Network | Internet | Packet | **IP addresses**, routing, ICMP | Wrong IP/route, no gateway, NAT issues |
| 2 Data link | Link | Frame | **MAC addresses**, Ethernet, ARP, VLANs | ARP/VLAN problems (rare in cloud) |
| 1 Physical | Link | Bits | Cables, fibre, radio | Cable/Wi-Fi down |

Mnemonic (7→1): **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.

### Encapsulation

```
App data                                    "GET /index.html"
+ TCP header (src port 51514, dst port 443)       → segment
+ IP header  (src 10.0.1.5, dst 93.184.216.34)    → packet
+ Ethernet header (src MAC, dst MAC = gateway)    → frame → bits on the wire
```

Each router hop **removes and rebuilds** the Ethernet header (new MACs) but keeps the **IP addresses** (unless NAT changes them) and the TCP ports.

## 2.3 Ethernet, MAC and ARP

- **MAC address:** 48-bit hardware address ( `02:5e:3a:1b:4c:7d` ), used only inside the local network.
- **Switch logic:** learn source MACs per port, forward to the known port, flood unknown/broadcast.
- **ARP:** finds the MAC for an IPv4 address in the local network ("Who has 10.0.1.1? Tell 10.0.1.5"). For destinations outside the subnet, the host ARPs for its **default gateway**.
- **VLANs** split one physical switch into separate logical networks (one VLAN = one subnet). In the cloud, you work with subnets and security groups instead of VLANs.

Commands: `ip neigh` (Linux), `arp -a` (Windows).

## 2.4 IPv4 addressing

- 32 bits written as four decimal octets: `192.168.10.25` .

- Split into **network** and **host** parts by the subnet mask / prefix length ( `/24` ).

| Range | Purpose |
|---|---|
| `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` | **Private** (RFC 1918): not routed on the internet; used in LANs, VPCs, Docker |
| `127.0.0.0/8` ( `127.0.0.1` ) | Loopback: "this host" ( `localhost` ) |
| `169.254.0.0/16` | Link-local / APIPA (no DHCP answer); in AWS also the **instance metadata** service `169.254.169.254` |
| `100.64.0.0/10` | Carrier-grade NAT; sometimes used in Kubernetes/cloud for pod networks |
| `0.0.0.0` | "Any address" (a server listening on `0.0.0.0` accepts on all interfaces); `0.0.0.0/0` = default route |
| `255.255.255.255` | Limited broadcast |
| Everything else (mostly) | **Public** addresses routable on the internet |

**Public vs private in practice:** your laptop has a private IP behind your home router; the router has a public IP from the ISP and uses **NAT** so many private devices share it. Cloud instances have private IPs in the VPC and optionally a public IP mapped by the cloud provider.

**Listening addresses matter:** a service bound to `127.0.0.1:8000` is reachable only from the same machine; bound to `0.0.0.0:8000` it is reachable from the network (if firewalls allow). This is a very common cause of "works on the server, not from outside" and "container port not reachable".

## 2.5 Binary and subnetting (complete method)

### 2.5.1 Number systems

| System | Base | Digits | Used for |
|---|---|---|---|
| Decimal | 10 | 0–9 | How people write IPv4: `192.168.1.10` |
| Binary | 2 | 0, 1 | How computers actually store IPs and masks |
| Hexadecimal (hex) | 16 | 0–9, A–F | MAC addresses, IPv6 |

| Unit | Size |
|---|---|
| Bit | One 0 or 1 |
| Nibble | 4 bits (= one hex digit) |
| Byte / **octet** | 8 bits. An IPv4 address = 4 octets = 32 bits |

### 2.5.2 Place values (the key to everything)

Each position in an 8-bit octet has a value. **Memorize this row:**

| Position | 8th | 7th | 6th | 5th | 4th | 3rd | 2nd | 1st |
|---|---|---|---|---|---|---|---|---|
| Power of 2 | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| **Value** | **128** | **64** | **32** | **16** | **8** | **4** | **2** | **1** |

All 8 bits on: 128+64+32+16+8+4+2+1 = **255** → that's why an octet goes from 0 to 255.

**Powers of 2 (for counting addresses):**

| $2^1$ | $2^2$ | $2^3$ | $2^4$ | $2^5$ | $2^6$ | $2^7$ | $2^8$ | $2^9$ | $2^{10}$ | $2^{11}$ | $2^{12}$ | $2^{16}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 | 2048 | 4096 | 65536 |

### 2.5.3 Binary → decimal

Write the place values above the bits and **add the values where the bit is 1**.

| Binary | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | Sum |
|---|---|---|---|---|---|---|---|---|---|
| 11000000 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 128+64 = **192** |
| 10101000 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 128+32+8 = **168** |
| 00001010 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 8+2 = **10** |
| 01001101 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 64+8+4+1 = **77** |

### 2.5.4 Decimal → binary (subtraction method)

Go left to right. **If the number is ≥ the place value: write 1 and subtract. Otherwise write 0.**

**Example: 172**

| Place value | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|
| Fits? | 172 ≥ 128 ✓ | 44 < 64 ✗ | 44 ≥ 32 ✓ | 12 < 16 ✗ | 12 ≥ 8 ✓ | 4 ≥ 4 ✓ | 0 ✗ | 0 ✗ |
| Remaining | 44 | 44 | 12 | 12 | 4 | 0 | 0 | 0 |
| **Bit** | **1** | **0** | **1** | **0** | **1** | **1** | **0** | **0** |

172 = **10101100**. Check: 128+32+8+4 = 172 ✓

**Example: 200** → 128 ✓ (72 left) → 64 ✓ (8 left) → 32 ✗ → 16 ✗ → 8 ✓ (0 left) → 4 ✗ → 2 ✗ → 1 ✗ = **11001000**

**Full address:**

```
192      .168      .10       .77
11000000 .10101000 .00001010 .01001101
```

### 2.5.5 Hexadecimal

One hex digit = 4 bits. A=10, B=11, C=12, D=13, E=14, F=15.

| Hex | Binary | Hex | Binary | Hex | Binary | Hex | Binary |
|---|---|---|---|---|---|---|---|
| 0 | 0000 | 4 | 0100 | 8 | 1000 | C | 1100 |
| 1 | 0001 | 5 | 0101 | 9 | 1001 | D | 1101 |
| 2 | 0010 | 6 | 0110 | A | 1010 | E | 1110 |
| 3 | 0011 | 7 | 0111 | B | 1011 | F | 1111 |

- **Hex → decimal:** `1A` = 1×16 + 10 = **26**. `FF` = 15×16 + 15 = **255**. `2F` = 2×16 + 15 = **47**.
- **Hex → binary:** convert each digit separately: `0DB8` = 0000 1101 1011 1000.
- Used in MAC addresses ( `00:1A:2B:...` ) and IPv6 ( `2001:db8::1` ).

## 2.5.6 The subnet mask in binary

An IPv4 address has a **network part** (which network) and a **host part** (which device). The **subnet mask** marks them: **1 = network bit, 0 = host bit**. The 1s are always continuous from the left.

```
/24  = 11111111.11111111.11111111.00000000 = 255.255.255.0     (24 network bits, 8 host bits)
/27  = 11111111.11111111.11111111.11100000 = 255.255.255.224   (27 network bits, 5 host bits)
```

**Prefix length** ( `/27` ) = number of 1s. **Host bits (h)** = 32 − prefix.

**Only these values can appear in a mask octet:**

| Mask octet | Binary | Network bits in octet | Block size (256 − value) |
|---|---|---|---|
| 0 | 00000000 | 0 | 256 |
| 128 | 10000000 | 1 | 128 |
| 192 | 11000000 | 2 | 64 |
| 224 | 11100000 | 3 | 32 |
| 240 | 11110000 | 4 | 16 |
| 248 | 11111000 | 5 | 8 |
| 252 | 11111100 | 6 | 4 |
| 254 | 11111110 | 7 | 2 |
| 255 | 11111111 | 8 | 1 |

**The formulas:**

| Want to know | Formula | Example /27 (h = 5) |
|---|---|---|
| Total addresses in a subnet | $2^h$ | $2^5 = 32$ |
| Usable hosts | $2^h - 2$ (network + broadcast reserved) | $32 - 2 = 30$ |
| Number of subnets when borrowing s bits | $2^s$ | /24 → /27 borrows 3 bits → $2^3 = 8$ subnets |

## 2.5.7 How the network address is found: the AND operation

Routers find the network by doing a bitwise **AND** of the IP and the mask. **AND rule: 1 AND 1 = 1, anything else = 0.**

**Example: 192.168.10.77/27** (only the last octet changes, because the first three mask octets are 255)

```
IP   last octet   77 = 0 1 0 | 0 1 1 0 1
Mask last octet  224 = 1 1 1 | 0 0 0 0 0
                       network | host bits
AND result           = 0 1 0 | 0 0 0 0 0 = 64   → Network   192.168.10.64

Set all host bits to 1 = 0 1 0 | 1 1 1 1 1 = 95   → Broadcast 192.168.10.95
```

| Result | Value |
|---|---|
| Network address (all host bits 0) | 192.168.10.64 |

| Result | Value |
|---|---|
| First usable host (network + 1) | 192.168.10.65 |
| Last usable host (broadcast − 1) | 192.168.10.94 |
| Broadcast address (all host bits 1) | 192.168.10.95 |
| Usable hosts | $2^5 - 2 = 30$ |

### 2.5.8 The fast method: block size

Use this in exams and real work. No binary needed once you understand 6.7.

1. **Find the interesting octet:** the octet where the mask is **not** 255 or 0.

| Prefix | /1–/8 | /9–/16 | /17–/24 | /25–/32 |
|---|---|---|---|---|
| Interesting octet | 1st | 2nd | 3rd | 4th |

2. **Block size** = 256 − mask value in that octet.
3. Subnets start at **multiples of the block size**: 0, block, 2×block...
4. **Network** = the largest multiple that is ≤ the IP's value in that octet. Octets to the right become 0.
5. **Broadcast** = next network − 1 (octets to the right become 255).

**Example 1: 10.20.30.200/26**

- /26 → 4th octet, mask 192 → block **64** → subnets .0, .64, .128, **.192**
- 200 is in the .192 block → **Network 10.20.30.192**, broadcast **10.20.30.255**, hosts .193 – .254 (62 hosts)

**Example 2: 172.16.45.10/20**

- /20 → 3rd octet, mask 255.255.**240**.0 → block **16** → 0, 16, **32**, 48...
- 45 is in the 32 block → **Network 172.16.32.0**, broadcast **172.16.47.255**
- Hosts 172.16.32.1 – 172.16.47.254 → $2^{12} - 2$ = **4,094**

**Example 3: 192.168.5.130/30**

- block 4 → .128, .132 → **Network .128**, hosts **.129, .130**, broadcast **.131** (typical router-to-router link)

**Example 4: 10.5.200.9/14**

- /14 → 2nd octet, mask 255.**252**.0.0 → block **4** → 0, **4**, 8...
- 5 is in the 4 block → **Network 10.4.0.0**, broadcast **10.7.255.255**, $2^{18} - 2$ = 262,142 hosts

### 2.5.9 Designing subnets

**A) "I need N subnets"** → find the smallest s where $2^s \geq N$, add s to the prefix.

Need 6 subnets from 192.168.1.0/24 → $2^3 = 8 \geq 6$ → borrow 3 bits → **/27** → block 32, 30 hosts each:

| # | Network | Usable range | Broadcast |
|---|---|---|---|
| 1 | 192.168.1.0/27 | .1 – .30 | .31 |
| 2 | 192.168.1.32/27 | .33 – .62 | .63 |
| 3 | 192.168.1.64/27 | .65 – .94 | .95 |
| 4 | 192.168.1.96/27 | .97 – .126 | .127 |

| # | Network | Usable range | Broadcast |
|---|---|---|---|
| 5 | 192.168.1.128/27 | .129 – .158 | .159 |
| 6 | 192.168.1.160/27 | .161 – .190 | .191 |
| 7–8 | .192/27, .224/27 | spare | |

**B) "I need H hosts per subnet"** → find the smallest h where $2^h - 2 \geq H$, prefix = 32 − h.

| Hosts needed (up to) | 2 | 6 | 14 | 30 | 62 | 126 | 254 | 510 | 1022 | 2046 | 4094 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Prefix | /30 | /29 | /28 | /27 | /26 | /25 | /24 | /23 | /22 | /21 | /20 |

Need 500 hosts → $2^9 - 2 = 510 \geq 500$ → h = 9 → **/23**.

### 2.5.10 Prefix reference table

| Prefix | Mask | Addresses | Usable hosts |
|---|---|---|---|
| /8 | 255.0.0.0 | 16,777,216 | 16,777,214 |
| /16 | 255.255.0.0 | 65,536 | 65,534 |
| /20 | 255.255.240.0 | 4,096 | 4,094 |
| /22 | 255.255.252.0 | 1,024 | 1,022 |
| /23 | 255.255.254.0 | 512 | 510 |
| /24 | 255.255.255.0 | 256 | 254 |
| /25 | 255.255.255.128 | 128 | 126 |
| /26 | 255.255.255.192 | 64 | 62 |
| /27 | 255.255.255.224 | 32 | 30 |
| /28 | 255.255.255.240 | 16 | 14 |
| /29 | 255.255.255.248 | 8 | 6 |
| /30 | 255.255.255.252 | 4 | 2 (point-to-point links) |
| /31 | 255.255.255.254 | 2 | 2 (point-to-point, RFC 3021) |
| /32 | 255.255.255.255 | 1 | Single host / loopback |

### 2.5.11 VLSM (Variable Length Subnet Masking)

Give each subnet only the size it needs. **Allocate the largest first**, so the blocks line up.

Example from `192.168.1.0/24`:

| Need | Hosts | Prefix | Subnet | Range | Broadcast |
|---|---|---|---|---|---|
| Sales | 100 | /25 | 192.168.1.0 | .1 – .126 | .127 |
| IT | 50 | /26 | 192.168.1.128 | .129 – .190 | .191 |
| Mgmt | 20 | /27 | 192.168.1.192 | .193 – .222 | .223 |
| Link R1–R2 | 2 | /30 | 192.168.1.224 | .225 – .226 | .227 |

### 2.5.12 Route summarization (in binary)

Combine contiguous networks into one route. Write the changing octet in binary and **count the bits that are the same in all of them**.

```
10.1.0.0  → 3rd octet 0 = 000000|00
10.1.1.0  → 3rd octet 1 = 000000|01
10.1.2.0  → 3rd octet 2 = 000000|10
10.1.3.0  → 3rd octet 3 = 000000|11
Common bits: 8 + 8 + 6 = 22  →  summary 10.1.0.0/22
```

Smaller routing tables and more stability.

### 2.5.13 Practice questions (answers below)

1. Convert 150 to binary.
2. Convert 11110000 to decimal.
3. Convert hex 2F to decimal.
4. Network, broadcast and host range of 192.168.50.100/28?
5. How many usable hosts in a /22?
6. Smallest subnet for 50 hosts?
7. Are 10.0.0.70/26 and 10.0.0.130/26 in the same subnet?
8. How many /24 subnets fit in 172.16.0.0/16?
9. Summarize 192.168.16.0/24 to 192.168.23.0/24 into one route.
10. Last usable host of 10.10.10.0/23?

**Answers:**

| # | Answer | How |
|---|---|---|
| 1 | 10010110 | 128 + 16 + 4 + 2 = 150 |
| 2 | 240 | 128 + 64 + 32 + 16 |
| 3 | 47 | 2×16 + 15 |
| 4 | Network .96, hosts .97 – .110, broadcast .111 | /28 → block 16 → 96 ≤ 100 < 112 |
| 5 | 1,022 | h = 10 → $2^{10} - 2$ |
| 6 | /26 (62 hosts) | /27 gives only 30 |
| 7 | No | /26 blocks: 64–127 and 128–191 |
| 8 | 256 | Borrow 8 bits → $2^8$ |
| 9 | 192.168.16.0/21 | 16 = 00010000, 23 = 00010111 → 5 common bits → 16 + 5 = 21 |
| 10 | 10.10.11.254 | /23 → block 2 in 3rd octet → network 10.10.10.0, broadcast 10.10.11.255 |

### 2.5.14 Subnetting for cloud networks

- AWS reserves **5 addresses per subnet** (network, VPC router `.1`, DNS `.2`, future `.3`, broadcast): a `/24` gives **251** usable IPs, a `/28` only 11.
- Typical VPC plan: VPC `10.0.0.0/16` → public `/24`s (`10.0.1.0`, `10.0.2.0`) → private app `/24`s (`10.0.11.0`, `10.0.12.0`) → data `/24`s (`10.0.21.0`, `10.0.22.0`), one per AZ.

- Kubernetes on AWS (VPC CNI) gives every pod a VPC IP: size node subnets generously (`/20` or larger) or use secondary CIDRs.
- **Never overlap** CIDRs between VPCs, offices and VPNs you might connect; Docker's default `172.17.0.0/16` can also clash with corporate networks.

## 2.6 IPv6 essentials

- 128 bits, 8 groups of hex: `2001:0db8:0000:0000:0000:ff00:0042:8329` → shortened `2001:db8::ff00:42:8329` (drop leading zeros; one `::` replaces consecutive zero groups).
- LANs use **/64** subnets. No broadcast (multicast instead), no NAT needed (huge address space), so **firewalls/security groups** are essential.

| Type | Prefix | Use |
|---|---|---|
| Global unicast | `2000::/3` | Public addresses |
| Unique local | `fd00::/8` | Private |
| Link-local | `fe80::/10` | Local link only; every interface has one |
| Loopback | `::1` | This host |
| Multicast | `ff00::/8` | Group delivery (replaces broadcast) |

Assignment: static, **SLAAC** (host builds its own address from the router's prefix), DHCPv6. **NDP** replaces ARP. In AWS, VPCs can be dual-stack; an **egress-only internet gateway** gives IPv6 outbound-only access.

## 2.7 Routing, default gateway and NAT

### How a host decides where to send a packet

1. Is the destination in **my subnet** (compare with my IP + mask)? → send directly (ARP for its MAC).
2. Otherwise → send to the **default gateway** (router), which forwards using its routing table.

```
$ ip route
default via 10.0.1.1 dev ens5           ← everything not matched below goes to the gateway
10.0.1.0/24 dev ens5 proto kernel       ← my own subnet: deliver directly
172.17.0.0/16 dev docker0               ← Docker bridge network
```

**Routing table rules:** most specific match wins (**longest prefix**); `0.0.0.0/0` is the default route; each hop decreases the IP **TTL** by 1 (packet dropped at 0 → prevents endless loops; `traceroute` uses this).

| Routing type | Where |
|---|---|
| Static routes | Configured manually (small networks, cloud route tables) |
| Dynamic routing (OSPF, BGP) | Routers learn routes from each other; BGP runs the internet and cloud hybrid links (VPN/Direct Connect) |

### NAT (Network Address Translation)

| Type | What | Example |
|---|---|---|
| SNAT / PAT (masquerade) | Many private IPs share one public IP, distinguished by **source ports** | Home router; AWS **NAT Gateway**; Docker containers going out |

| Type | What | Example |
|---|---|---|
| DNAT / port forwarding | Traffic to a public IP:port is forwarded to a private IP:port | `docker run -p 8080:80`; Kubernetes NodePort; router port forward |
| 1:1 NAT | One private ↔ one public IP | AWS public IP / Elastic IP on an instance |

**Consequences:** inbound connections to private hosts need DNAT or a load balancer; servers see the NAT's public IP, not the client's private IP; NAT keeps connection state (tables can fill up under heavy load).

## 2.8 TCP and UDP in depth

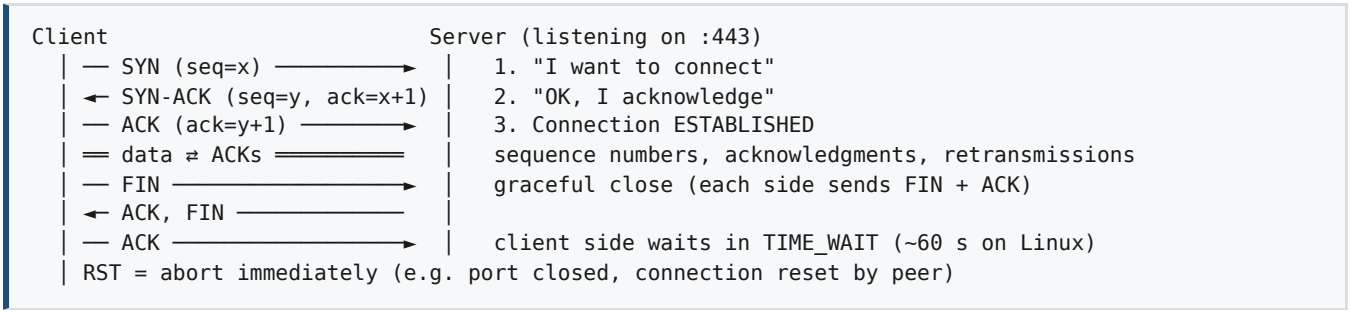
### Ports and sockets

- **Port:** 16-bit number (0–65535) identifying an application on a host. **Well-known** 0–1023, **registered** 1024–49151, **ephemeral/dynamic** (client side, Linux default 32768–60999).
- **Socket:** IP + port. A **connection** is identified by the 5-tuple: protocol, source IP, source port, destination IP, destination port.

| Port | Service | Port | Service |
|---|---|---|---|
| 20/21 | FTP | 443 | HTTPS (TCP; HTTP/3 uses UDP 443) |
| 22 | SSH / SFTP | 465/587 | SMTP submission (TLS) |
| 25 | SMTP | 853 | DNS over TLS |
| 53 | DNS (UDP + TCP) | 1433 | Microsoft SQL Server |
| 67/68 | DHCP (UDP) | 2049 | NFS (AWS EFS) |
| 80 | HTTP | 2379-2380 | etcd |
| 110/143 | POP3 / IMAP | 3000 | Grafana (default) |
| 123 | NTP (UDP) | 3306 | MySQL / MariaDB |
| 161/162 | SNMP | 3389 | RDP |
| 389/636 | LDAP / LDAPS | 5432 | PostgreSQL |
| 6379 | Redis | 6443 | Kubernetes API server |
| 8080 | Alt HTTP (apps, Jenkins) | 9090 | Prometheus |
| 9092 | Kafka | 10250 | kubelet API |
| 27017 | MongoDB | 30000-32767 | Kubernetes NodePort range |

### TCP (Transmission Control Protocol)

Reliable, ordered, connection-oriented byte stream.



| Mechanism | Purpose |
|---|---|
| Sequence numbers & ACKs | Order and confirm delivery |
| Retransmission | Resend lost segments (causes latency spikes on lossy networks) |
| Flow control (window) | Receiver tells sender how much it can accept |
| Congestion control | Sender slows down when the network is congested (CUBIC, BBR) |

**TCP states you'll see in `ss -tan`:** `LISTEN` (server waiting), `SYN-SENT` (client waiting for answer: firewall dropping?), `ESTAB` (established), `TIME-WAIT` (recently closed by this side: normal; huge numbers mean many short connections → use keep-alive/pooling), `CLOSE-WAIT` (peer closed but the app hasn't: often an **application bug** leaking connections).

**What errors mean:**

| Error | Meaning |
|---|---|
| `Connection refused` | Host reachable, but nothing listening on that port (or RST from a firewall) |
| `Connection timed out` | No answer at all: packets dropped (firewall/security group/NACL), wrong IP/route, host down |
| `Connection reset by peer` | The other side aborted the connection (crash, idle timeout, proxy) |
| `No route to host` | Routing problem or ICMP unreachable from a firewall |

## UDP (User Datagram Protocol)

Connectionless, no handshake, no delivery guarantee, low overhead. Used by DNS, DHCP, NTP, syslog, VoIP/video, gaming, QUIC/HTTP/3, VXLAN overlays. Testing UDP is harder: no "connection" to confirm (`nc -u`, application-level checks).

| | TCP | UDP |
|---|---|---|
| Connection | Yes (handshake) | No |
| Reliability / ordering | Yes | No (application handles it) |
| Header | 20+ bytes | 8 bytes |
| Speed / overhead | More overhead | Minimal |
| Examples | HTTP/1.1, HTTP/2, SSH, databases | DNS, NTP, VoIP, QUIC |

## ICMP

Control messages for IP: **echo request/reply** (`ping`), **destination unreachable**, **time exceeded** (used by `traceroute`), "fragmentation needed" (Path MTU discovery). Blocking all ICMP can break PMTU discovery and makes troubleshooting harder; many clouds block ping by default in security groups.

## MTU

Maximum packet size on a link (Ethernet 1500 bytes; AWS within a VPC supports 9001 "jumbo frames"; VPNs/overlays reduce it). MTU mismatches cause "small requests work, large ones hang". Test: `ping -M do -s 1472 host` (Linux).

## 2.9 DNS in depth

**DNS (Domain Name System)** is the internet's distributed phone book: it maps names to IPs (and more).

### Name hierarchy

```
                    . (root)
          ┌─────────┼──────────┐
        com         org        in      ← TLDs (top-level domains)
         │
      example                           ← domain (registered by you)
     ┌───┴────┐
    www      api                        ← subdomains / hostnames
FQDN: api.example.com.
```

### Resolution step by step

```
App → OS stub resolver (checks /etc/hosts, local cache)
    → Recursive resolver (ISP, 8.8.8.8, 1.1.1.1, VPC resolver 10.0.0.2)
        ├─ asks a ROOT server:          "who handles .com?"         → .com TLD servers
        ├─ asks the .com TLD server:    "who handles example.com?"  → NS ns1.example-dns.net
        └─ asks the AUTHORITATIVE server: "A record for api.example.com?" → 203.0.113.10 (TTL 300)
    ← answer cached by the resolver and the client for TTL seconds
```

| Server role | Job |
|---|---|
| Recursive resolver | Does the lookups for clients and caches results |
| Root servers | Know the TLD servers |
| TLD servers | Know the authoritative servers for each domain |
| Authoritative server | Holds the actual records for a zone (e.g. Route 53, Cloudflare, your DNS provider) |

### Record types

| Record | Purpose | Example |
|---|---|---|
| A | Name → IPv4 | `api.example.com. 300 IN A 203.0.113.10` |
| AAAA | Name → IPv6 | `api AAAA 2001:db8::10` |
| CNAME | Alias to another name (not allowed at the zone apex) | `www CNAME example.com.` |
| ALIAS / ANAME | Provider-specific apex alias (Route 53 Alias) | `example.com → ALB` |
| MX | Mail servers with priority | `MX 10 mail.example.com.` |
| TXT | Text: SPF, DKIM, DMARC, domain verification | `"v=spf1 include:_spf.google.com ~all"` |
| NS | Authoritative name servers of a zone | `NS ns-123.awsdns-15.com.` |
| SOA | Zone metadata (primary NS, serial, timers) | Created automatically |
| PTR | Reverse lookup IP → name | `10.113.0.203.in-addr.arpa PTR api.example.com.` |
| SRV | Service location with port | `_sip._tcp SRV 10 60 5060 sip.example.com.` |

| Record | Purpose | Example |
|---|---|---|
| CAA | Which CAs may issue certificates | `CAA 0 issue "letsencrypt.org"` |

### TTL and caching

**TTL** (seconds) tells resolvers how long to cache an answer. High TTL = fewer queries, slow changes; low TTL = fast changes, more queries. **Before a migration, lower the TTL** (e.g. to 60) a day in advance, switch, then raise it again. **Negative caching:** "does not exist" (NXDOMAIN) answers are cached too.

### DNS in different environments

| Environment | Detail |
|---|---|
| Linux | `/etc/hosts`, `/etc/resolv.conf` (often `127.0.0.53` = systemd-resolved), `resolvectl status` |
| Windows | `ipconfig /displaydns`, `ipconfig /flushdns`, `Resolve-DnsName` |
| AWS VPC | Route 53 Resolver at VPC base + 2; private hosted zones; Resolver endpoints for hybrid DNS |
| Docker | Embedded DNS ( `127.0.0.11` ) resolves container names on user-defined networks |
| Kubernetes | **CoreDNS**: `service.namespace.svc.cluster.local`; pods' `/etc/resolv.conf` has search domains and `ndots:5` |
| Split-horizon DNS | Same name resolves differently inside vs outside (internal IP inside the VPC, public IP outside) |

**Security:** DNSSEC signs records (authenticity); DNS over HTTPS/TLS encrypts queries; SPF/DKIM/DMARC TXT records protect email from spoofing.

### DNS commands

```
dig api.example.com                # status, ANSWER section, TTL, which server answered
dig +short api.example.com         # just the IPs
dig @8.8.8.8 api.example.com       # ask a specific resolver
dig example.com NS / MX / TXT      # other record types
dig +trace api.example.com         # follow the delegation from the root
dig -x 203.0.113.10                # reverse lookup
nslookup api.example.com           # simple lookup (Windows too)
getent hosts api.example.com       # resolve like applications do (includes /etc/hosts)
```

**Response codes:** `NOERROR` (exists; check the ANSWER), `NXDOMAIN` (name doesn't exist), `SERVFAIL` (resolver couldn't get an answer: DNSSEC failure, broken authoritative servers), `REFUSED` (server won't answer you).

## 2.10 DHCP

Gives devices their network configuration automatically: IP address, subnet mask, default gateway, DNS servers, lease time.

**DORA: D**iscover (client broadcast: "any DHCP server?") → **O**ffer (server proposes an IP) → **R**equest (client accepts) → **A**cknowledge (server confirms). Leases are renewed before they expire. Routers forward DHCP to another subnet with a **DHCP relay** ( `ip helper-address` ).

A host with `169.254.x.x` means DHCP failed. In AWS, DHCP options sets define the DNS servers and domain name for a VPC.

## 2.11 HTTP in depth

### Request and response structure

```
REQUEST                                  RESPONSE
GET /api/products?page=2 HTTP/1.1        HTTP/1.1 200 OK
Host: shop.example.com                   Content-Type: application/json
User-Agent: curl/8.5.0                   Content-Length: 512
Accept: application/json                 Cache-Control: max-age=60
Authorization: Bearer eyJhbGciOi...      Set-Cookie: session=abc123; HttpOnly; Secure

(optional body, e.g. JSON for POST)      {"items": [...], "page": 2}
```

A request = **method + path + version**, **headers**, optional **body**. A response = **status code**, headers, body.

### Methods

| Method | Purpose | Safe (no change) | Idempotent (repeat = same result) |
|---|---|---|---|
| GET | Read | Yes | Yes |
| HEAD | Like GET, headers only | Yes | Yes |
| POST | Create / trigger action | No | **No** |
| PUT | Replace a resource | No | Yes |
| PATCH | Partial update | No | Not necessarily |
| DELETE | Delete | No | Yes |
| OPTIONS | Allowed methods / CORS preflight | Yes | Yes |

Idempotency matters for **retries**: retrying a GET or PUT is safe; retrying a POST may create duplicates (use idempotency keys).

### Status codes

| Code | Meaning | Typical cause |
|---|---|---|
| 200 OK / 201 Created / 204 No Content | Success | |
| 301 / 308 | Permanent redirect | HTTP → HTTPS, moved URL |
| 302 / 307 | Temporary redirect | Login redirects |
| 304 Not Modified | Use your cached copy | Conditional request with ETag |
| 400 Bad Request | Malformed request | Invalid JSON, missing parameter |
| 401 Unauthorized | Not authenticated | Missing/expired token |
| 403 Forbidden | Authenticated but not allowed | Permissions, WAF block, S3 policy |
| 404 Not Found | No such path/resource | Wrong URL, wrong Ingress path |
| 405 Method Not Allowed | Wrong method | POST to a GET-only endpoint |

| Code | Meaning | Typical cause |
|---|---|---|
| 408 / 429 | Request timeout / Too many requests | Slow client / rate limiting |
| 500 Internal Server Error | Unhandled error in the app | Bug, exception: check app logs |
| 502 Bad Gateway | Proxy/LB got an invalid response from the backend | Backend crashed, wrong port |
| 503 Service Unavailable | Backend unavailable/overloaded | No healthy targets, maintenance |
| 504 Gateway Timeout | Backend didn't respond in time | Slow app/database, timeouts |

**Rule of thumb:** 4xx = the client (request) is wrong; 5xx = the server side is wrong. 502/503/504 point at the **path between proxy/LB and backend**.

## Important headers

| Header | Purpose |
|---|---|
| `Host` | Which site (one IP can host many sites: virtual hosting, Ingress host rules) |
| `Content-Type` / `Accept` | Body format sent / wanted ( `application/json` , `text/html` ) |
| `Authorization` | Credentials ( `Bearer <token>` , `Basic <base64>` ) |
| `Cookie` / `Set-Cookie` | Session state in the browser ( `HttpOnly` , `Secure` , `SameSite` flags) |
| `Cache-Control` , `ETag` , `Last-Modified` , `Expires` | Caching by browsers and CDNs |
| `X-Forwarded-For` , `X-Forwarded-Proto` , `Forwarded` | Original client IP and scheme when behind proxies/load balancers |
| `Location` | Target of a redirect |
| `Strict-Transport-Security` (HSTS) | Force HTTPS in browsers |
| `Access-Control-Allow-Origin` | CORS: which other websites may call this API from a browser |
| `User-Agent` | Client software |

## Statelessness, sessions and tokens

HTTP is **stateless**: each request stands alone. State is added with **cookies + server-side sessions** (store sessions in Redis/DB, not in one server's memory, so any server can handle the next request) or **tokens** (e.g. JWT carried in the `Authorization` header).

## REST APIs and JSON

**REST** style: resources as URLs ( `/orders/42` ), methods as actions, JSON bodies, status codes for results, versioning ( `/v1/` ). Other API styles: **GraphQL** (one endpoint, client chooses fields), **gRPC** (binary Protobuf over HTTP/2, fast service-to-service), **WebSockets** (two-way persistent connection for real-time apps).

### HTTP versions

| Version | Key change |
|---|---|
| HTTP/1.1 | Text protocol, keep-alive connections, one request at a time per connection |
| HTTP/2 | Binary, **multiplexing** many requests over one TCP connection, header compression |
| HTTP/3 | Runs over **QUIC (UDP)**: faster setup, no head-of-line blocking, better on mobile networks |

### curl for HTTP debugging

```
curl -v https://shop.example.com/health               # full request/response incl. TLS
curl -I https://shop.example.com                      # headers only
curl -L http://shop.example.com                       # follow redirects
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://shop.example.com
curl -H "Host: shop.example.com" http://10.0.1.25/    # test a virtual host directly on an IP
curl --resolve shop.example.com:443:10.0.1.25 https://shop.example.com/  # keeps SNI right
curl -X POST -H "Content-Type: application/json" -d '{"qty":2}' https://api.example.com/orders
# where is the time spent?
curl -o /dev/null -s https://shop.example.com -w \
  "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} total:%{time_total}\n"
```

## 2.12 TLS and HTTPS in depth

**TLS (Transport Layer Security)** provides **encryption** (no eavesdropping), **integrity** (no tampering) and **authentication** (you're talking to the real server). **HTTPS = HTTP over TLS** (port 443). SSL is the obsolete predecessor; use **TLS 1.2 or 1.3** only.

### TLS 1.3 handshake (simplified)

```
Client                                              Server
  │ ClientHello: TLS versions, cipher suites, SNI (hostname), key share ─►│
  │◄─ ServerHello: chosen cipher, key share; Certificate; signature; Finished
  │  client verifies the certificate chain and hostname, derives session keys
  │ Finished ──────────────────────────────────────────────────────────────►│
  │ ═══════════ encrypted application data (symmetric keys, e.g. AES-GCM) ═══════════
```

Asymmetric cryptography (certificates, key exchange) is used only to agree on **symmetric session keys**; bulk data uses fast symmetric encryption.

### Certificates and the chain of trust

```
Root CA certificate (pre-installed in OS/browser trust store)
   └── signs Intermediate CA certificate
          └── signs Server (leaf) certificate: CN/SAN = shop.example.com, valid from/to, public key
```

| Term | Meaning |
|---|---|
| CA (Certificate Authority) | Trusted organization that signs certificates (Let's Encrypt, DigiCert, AWS ACM's CA) |
| SAN (Subject Alternative Name) | The hostnames the certificate is valid for ( `shop.example.com` , `*.example.com` ) |
| Wildcard | `*.example.com` covers one level of subdomains |
| Chain / bundle | Server must send its certificate **plus intermediates** |
| Private key | Kept secret on the server/load balancer; never shared or committed |

| Term | Meaning |
|---|---|
| CSR | Certificate Signing Request sent to a CA |
| SNI | Hostname sent in ClientHello so one IP can serve many certificates |
| Validity | Public certificates are short-lived (Let's Encrypt 90 days): **automate renewal** (certbot, ACM, cert-manager) |
| mTLS (mutual TLS) | **Both** sides present certificates: service-to-service auth, service meshes, partner APIs |
| Self-signed | Not trusted by default: fine for labs, not for public services |

### TLS commands

```
# handshake + certificate chain
openssl s_client -connect shop.example.com:443 -servername shop.example.com </dev/null
echo | openssl s_client -connect shop.example.com:443 -servername shop.example.com 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName   # cert details
openssl x509 -in cert.pem -noout -text                       # inspect a certificate file
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 30 \
  -subj "/CN=lab.local"                                       # self-signed cert for labs
```

### Common TLS errors

| Error | Cause |
|---|---|
| Certificate has expired | Renewal failed: automate and monitor expiry |
| Hostname mismatch | Name not in the certificate SAN; wrong cert served (SNI) |
| Unable to get local issuer / incomplete chain | Server not sending intermediate certificates |
| Self-signed certificate | Untrusted CA (labs, internal CAs not installed in the trust store) |
| Handshake failure / protocol version | Client and server share no TLS version/cipher (old clients) |
| Certificate not yet valid | Clock wrong on the client/server (check NTP) |

**TLS termination:** decrypting at the load balancer/ingress (simpler certificate management) vs **end-to-end** (re-encrypt to backends) for stricter security.

## 2.13 SSH (protocol view)

SSH (TCP 22) gives encrypted remote shells, file transfer (SCP/SFTP) and tunnels. Server proves identity with a **host key** (stored in `known_hosts`); users authenticate with **key pairs** (preferred) or passwords. Full practical coverage is in the Linux notes (keys, config, jump hosts, port forwarding, hardening).

## 2.14 Firewalls and network security

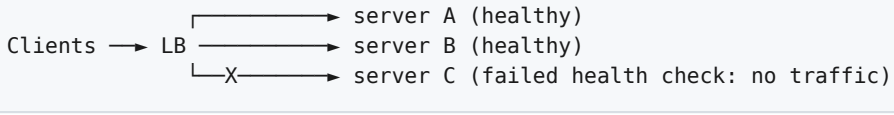
| Type | How it works | Examples |
|---|---|---|
| Packet filter (stateless) | Checks each packet against rules (IP, port, protocol); return traffic needs explicit rules | AWS NACLs, simple ACLs |
| **Stateful firewall** | Tracks connections; replies to allowed connections pass automatically | AWS Security Groups, iptables/nftables conntrack, ufw, firewalld |
| NGFW (next-generation) | Adds application awareness, user identity, IPS, TLS inspection | Palo Alto, Fortinet, AWS Network Firewall |

| Type | How it works | Examples |
|---|---|---|
| WAF | Inspects HTTP requests for attacks (SQL injection, XSS, bots, rate limits) | AWS WAF, Cloudflare, ModSecurity |
| Host firewall | On the server itself | ufw, firewalld, Windows Firewall |

**Design principles:** default deny inbound, allow only what's needed (least privilege), segment networks (public / app / data tiers), restrict admin ports (SSH/RDP) to VPN/bastion or remove them (SSM), log denied traffic, defense in depth (cloud SG + host firewall + app auth).

**Zones:** internet (untrusted) → **DMZ** (public-facing load balancers/proxies) → internal app network → data network.

## 2.15 Load balancing




| Type | Works on | Can route by | Examples |
|---|---|---|---|
| Layer 4 | IP + TCP/UDP port | Connection | AWS NLB, HAProxy (TCP mode), kube-proxy |
| Layer 7 | HTTP(S) content | Host, path, headers, cookies, methods | AWS ALB, nginx, HAProxy (HTTP), Envoy, Kubernetes Ingress controllers |
| DNS / global | DNS answers | Geography, latency, health | Route 53, Cloudflare, Global Accelerator |

| Algorithm | Behaviour |
|---|---|
| Round robin | Each server in turn |
| Weighted round robin | Bigger servers get more |
| Least connections / least outstanding requests | Server with the fewest active requests |
| IP hash / consistent hashing | Same client → same server (affinity, caches) |
| Random with two choices | Pick two at random, choose the less loaded |

| Feature | Detail |
|---|---|
| Health checks | Active probes ( `GET /health` every N seconds, healthy/unhealthy thresholds); passive (observe failures) |
| Sticky sessions | Keep a user on one server (cookie); avoid when possible: store sessions centrally |
| TLS termination / offload | LB handles certificates and decryption |
| Connection draining | Let in-flight requests finish before removing a server (deployments) |
| Timeouts | Idle/backend timeouts must be consistent with the app (cause of 502/504) |
| Client IP | Lost behind L7 proxies → use `X-Forwarded-For` or PROXY protocol |

## 2.16 Proxies, reverse proxies, API gateways and CDNs

| Component | Sits in front of | Purpose | Examples |
|---|---|---|---|
| Forward proxy | **Clients** | Outbound filtering, caching, hiding client IPs; corporate internet access ( `HTTP_PROXY` / `HTTPS_PROXY` / `NO_PROXY` env variables) | Squid, Zscaler |
| **Reverse proxy** | **Servers** | Single entry point, TLS termination, load balancing, caching, compression, routing by path/host | nginx, HAProxy, Envoy, Traefik, ALB |
| API gateway | APIs | Reverse proxy + authentication, rate limiting, API keys, request transformation, analytics | AWS API Gateway, Kong, Apigee |
| Service mesh | Service-to-service traffic | mTLS, retries, timeouts, traffic splitting, telemetry via sidecar/ambient proxies | Istio, Linkerd |
| **CDN** | Websites/APIs | Caches content at edge locations near users; DDoS absorption | CloudFront, Cloudflare, Akamai |

**Caching basics:** cache **hit** (served from cache) vs **miss** (fetched from origin); controlled by `Cache-Control` headers and TTLs; **invalidation** clears cached objects; use **versioned file names** ( `app.3f9a1c.js` ) for static assets so you rarely need invalidation.

## 2.17 VPNs and private connectivity

| Type | Use | Technology |
|---|---|---|
| Site-to-site VPN | Connect an office/data center to a cloud VPC or another office | IPsec tunnels (IKE for keys, ESP for encryption), often with BGP |
| Remote-access VPN | Employees reach internal networks | OpenVPN, WireGuard, IPsec clients, TLS VPNs |
| Zero-trust access (ZTNA) | Per-application access based on identity and device, not network location | Cloudflare Access, Zscaler, AWS Verified Access |
| Dedicated links | Private, consistent bandwidth to cloud | AWS Direct Connect, Azure ExpressRoute |
| Overlay networks | Virtual networks over existing IP networks | VXLAN, WireGuard meshes (Tailscale) |

## 2.18 Networking in cloud, containers and Kubernetes

### Cloud (AWS mapping)

| Traditional | AWS |
|---|---|
| Data center network | VPC |
| Subnet / VLAN | Subnet (one AZ) |
| Router + routing table | Route tables |
| Internet edge | Internet Gateway |
| NAT router | NAT Gateway |

| Traditional | AWS |
|---|---|
| Host firewall / stateful firewall | Security Group |
| Subnet ACL | Network ACL |
| Load balancer | ALB / NLB |
| DNS server | Route 53 / VPC Resolver |
| VPN / leased line | Site-to-Site VPN / Direct Connect |

Details: AWS notes, VPC chapter.

### Docker networking

```
Host eth0 10.0.1.25 ── docker0 bridge 172.17.0.1 ─┬─── container A 172.17.0.2
                                                  └─── container B 172.17.0.3
docker run -p 8080:80 nginx   →  host:8080 ──(DNAT iptables)──► container:80
Containers → internet          →  source NAT (masquerade) to the host's IP
```

- **Bridge** (default): private network per host; publish ports with `-p host:container`.
- **User-defined bridge:** containers resolve each other **by name** (embedded DNS).
- **Host network:** container shares the host's network stack (no isolation, no `-p`).
- Inside a container, `localhost` is the **container itself**, not the host.

### Kubernetes networking model

| Rule / object | Meaning |
|---|---|
| Every **pod** gets its own IP | Pods talk to each other directly across nodes without NAT (implemented by a **CNI** plugin: AWS VPC CNI, Calico, Cilium, Flannel) |
| **Service** (ClusterIP) | Stable virtual IP + DNS name in front of changing pods; kube-proxy (iptables/IPVS) or eBPF load-balances to pod IPs |
| NodePort / LoadBalancer | Expose a Service on every node's port (30000–32767) / through a cloud load balancer |
| **Ingress / Gateway API** | Layer 7 HTTP(S) routing by host/path through an ingress controller |
| **CoreDNS** | `my-svc.my-namespace.svc.cluster.local` names |
| **NetworkPolicy** | Firewall rules between pods (needs a CNI that supports it) |

Details: Docker and Kubernetes notes.

## 2.19 Networking tools reference

| Tool | Purpose | Example |
|---|---|---|
| `ip a`, `ip r`, `ip neigh` | Addresses, routes, ARP (Linux) | `ip route get 8.8.8.8` |
| `ipconfig /all`, `route print`, `arp -a` | Same on Windows | |
| `ping` | Reachability + latency (ICMP) | `ping -c 4 10.0.11.5` |
| `traceroute` / `tracert` / `mtr` | Path, hop where it fails or slows | `mtr -rwc 50 api.example.com` |

| Tool | Purpose | Example |
|---|---|---|
| `dig` / `nslookup` / `Resolve-DnsName` | DNS | `dig +short api.example.com` |
| `curl` / `wget` | HTTP(S) testing | `curl -v https://…` |
| `nc` / `Test-NetConnection` | TCP port test | `nc -zv db 5432` |
| `ss` / `netstat` | Listening ports, connections | `ss -tulpn`, `ss -tan state established` |
| `tcpdump` | Packet capture on the CLI | `sudo tcpdump -i any -nn host 10.0.1.5 and port 443` |
| Wireshark | GUI packet analysis (filters: `dns`, `tcp.port == 443`, `http`, `tcp.flags.reset == 1`) | Open a `.pcap` |
| `openssl s_client` | TLS/certificate checks | see 2.12 |
| `iperf3` | Bandwidth test between two hosts | `iperf3 -s` / `iperf3 -c server` |
| `ethtool` | NIC speed/duplex/errors | `ethtool eth0` |

## 2.20 Network troubleshooting

### Layered method

```
1. Name    → does DNS resolve correctly?          dig / nslookup / getent hosts
2. Route   → can I reach the IP?                  ping, traceroute/mtr, ip route
3. Port    → is the port open and allowed?        nc -zv / Test-NetConnection, ss -tulpn on server
4. TLS     → is the certificate valid?            openssl s_client, curl -v
5. App     → does the application answer right?   curl -v, app logs, status codes
At each step: firewalls / security groups / NACLs / proxies / NAT on the path?
```

### Scenarios

| Symptom | Likely causes and checks |
|---|---|
| Website down for everyone | DNS record changed/expired domain? LB health (no healthy targets → 503)? Certificate expired? App crashed (502)? Recent deploy? |
| Works by IP, not by name | DNS: wrong record, not propagated (TTL), wrong resolver, `/etc/hosts` override |
| `Connection timed out` | Security group/NACL/firewall dropping, wrong IP, no route, host down |
| `Connection refused` | Service not running or listening on `127.0.0.1`/different port; host firewall REJECT |
| Works inside the server, not from outside | Bound to `127.0.0.1`; host firewall; security group; LB/target port mismatch |
| Intermittent failures | One bad backend behind a LB, DNS round-robin to a dead IP, packet loss, connection pool exhaustion, NAT port exhaustion |
| Slow responses | Check timing (`curl -w`): slow DNS? slow TCP connect (network/latency)? slow TLS? slow server (app/DB)? |
| Large requests hang, small work | MTU problem across VPN/overlay |

| Symptom | Likely causes and checks |
|---|---|
| Private server can't reach the internet | No NAT/route, no DNS, outbound rules, proxy settings |
| Container can't reach another container | Not on the same user-defined network, using `localhost`, wrong port |
| Pod can't reach a Service | Service selector doesn't match pod labels (no endpoints), wrong port/targetPort, NetworkPolicy, CoreDNS issues |

## 2.21 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | Explain the OSI model and where TCP, IP, HTTP sit | 7 layers; HTTP = 7, TCP = 4, IP = 3, Ethernet = 2 |
| 2 | What happens when you type a URL in a browser? | DNS resolution → TCP handshake → TLS handshake → HTTP request → server/app/DB → response → render; caches at several steps |
| 3 | TCP vs UDP? | Reliable ordered connection with handshake vs connectionless best-effort; HTTP/SSH vs DNS/VoIP/QUIC |
| 4 | Explain the TCP three-way handshake | SYN → SYN-ACK → ACK |
| 5 | Connection refused vs timed out? | Refused = reached the host but no listener/REJECT; timeout = no answer (dropped by firewall, routing, host down) |
| 6 | What is a subnet? How many hosts in /26? | A range of addresses sharing a network prefix; 64 addresses, 62 usable (AWS: 59) |
| 7 | Private vs public IP? | Private RFC 1918 ranges are not internet-routable and need NAT |
| 8 | What is NAT and where is it used? | Address translation: home routers, NAT Gateway, Docker port publishing, LBs |
| 9 | What is a default gateway? | The router a host sends traffic to for destinations outside its subnet |
| 10 | Explain DNS resolution | Stub → recursive resolver → root → TLD → authoritative; cached per TTL |
| 11 | A vs CNAME vs Alias? | A = name → IP; CNAME = name → name (not at apex); Alias = provider feature for apex → AWS resource |
| 12 | Why lower TTL before a migration? | So resolvers cache the old answer briefly and the switch propagates quickly |
| 13 | What does DHCP do? | Assigns IP, mask, gateway, DNS automatically (DORA) |
| 14 | HTTP 401 vs 403? | Not authenticated vs authenticated but not permitted |
| 15 | HTTP 502 vs 503 vs 504? | Bad response from backend / no available backend / backend timed out |
| 16 | What is idempotency and why does it matter? | Repeating a request has the same effect; makes retries safe (GET/PUT/DELETE yes, POST no) |
| 17 | How does HTTPS work? | TLS handshake authenticates the server with a CA-signed certificate and agrees on symmetric keys; data is then encrypted |
| 18 | What is a certificate chain? | Leaf cert signed by an intermediate signed by a trusted root; servers must send intermediates |
| 19 | What is SNI? | Hostname in the TLS ClientHello so one IP can serve multiple certificates |

| # | Question | Answer |
|---|---|---|
| 20 | What is mTLS? | Both client and server authenticate with certificates |
| 21 | L4 vs L7 load balancer? | Transport (IP/port) vs application (HTTP host/path/headers) routing |
| 22 | Forward vs reverse proxy? | In front of clients (outbound) vs in front of servers (inbound) |
| 23 | What does a CDN do? | Caches content at edge locations near users: faster, less origin load, DDoS absorption |
| 24 | Stateful vs stateless firewall? | Tracks connections and allows replies vs evaluates each packet independently |
| 25 | How do you find what's listening on a port? | `ss -tulpn` / `lsof -i :port` (Linux), `netstat -ano` (Windows) |
| 26 | How do you troubleshoot "cannot reach service X"? | DNS → ping/route → port test → firewall/SG → service listening address → TLS → app logs |
| 27 | What is MTU and what breaks when it's wrong? | Max packet size; large transfers hang while small ones work |
| 28 | How do Docker containers talk to each other? | Same user-defined network, by container name via embedded DNS |
| 29 | How does a Kubernetes Service work? | Stable virtual IP/DNS name; kube-proxy/eBPF forwards to the pods matching its selector |
| 30 | What is `0.0.0.0` vs `127.0.0.1` for a server? | Listen on all interfaces vs only local loopback |

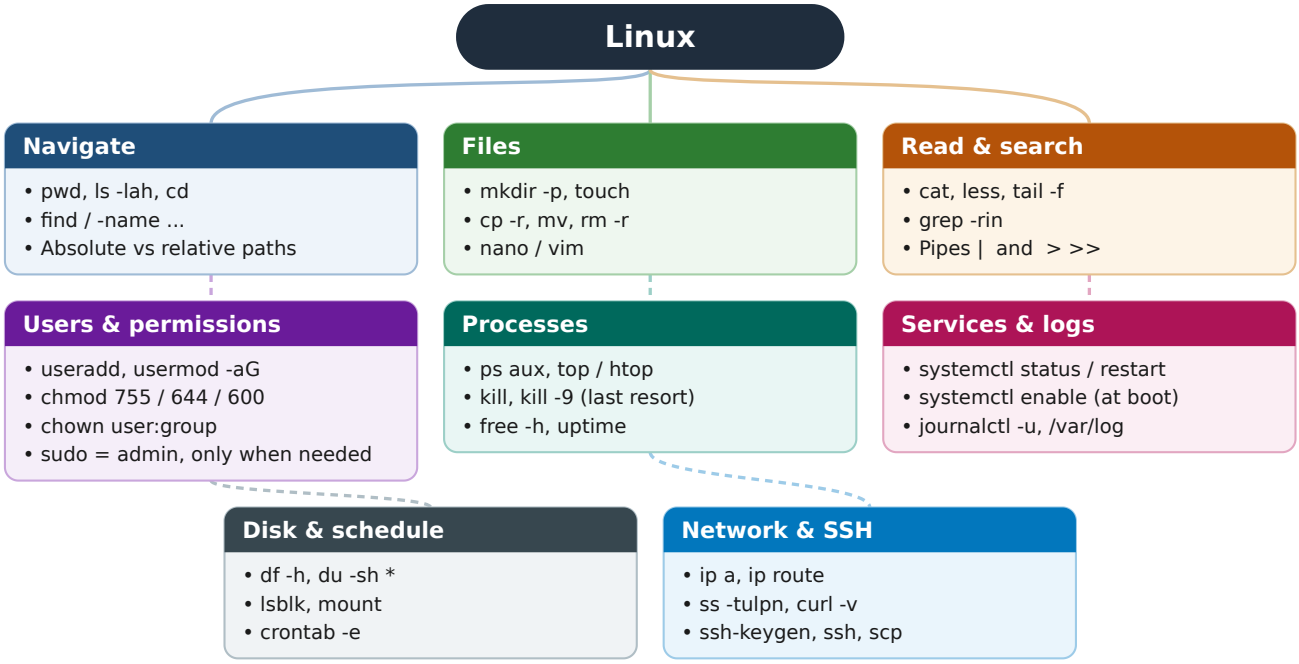
## 2.22 Self-check: I can…

- [ ] Explain the OSI/TCP-IP layers and map common problems to layers.
- [ ] Explain MAC, ARP, IPv4 classes of addresses, private vs public and special ranges.
- [ ] Convert binary/decimal/hex and subnet any address in under a minute; design VLSM and VPC plans.
- [ ] Explain IPv6 basics, routing, default gateways, longest-prefix match, NAT types.
- [ ] Describe TCP handshake, states, errors (refused/timeout/reset) and UDP use cases.
- [ ] Explain DNS resolution, record types, TTL, and debug with `dig`.
- [ ] Read HTTP requests/responses; explain methods, status codes, headers, caching, REST, HTTP/2/3.
- [ ] Explain TLS handshake, certificates, chain of trust, SNI, mTLS; inspect certs with `openssl`.
- [ ] Compare stateful/stateless firewalls, WAFs and network segmentation.
- [ ] Explain L4/L7 load balancing, algorithms, health checks, draining and timeouts.
- [ ] Explain proxies, reverse proxies, API gateways, service meshes and CDNs.
- [ ] Describe VPNs, Direct Connect-style links and zero-trust access.
- [ ] Explain Docker and Kubernetes networking basics.
- [ ] Troubleshoot connectivity step by step with the right tool at each layer.

**REMEMBER**

- OSI layers 7 → 1: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.
- Status codes: **2xx** OK · **3xx** moved · **4xx** you made a mistake · **5xx** the server made a mistake.
- Troubleshoot: link → IP → gateway → internet (8.8.8.8) → DNS → port → application.
- Subnet hosts = $2^{\text{host bits}} - 2$. Block size = 256 − mask octet.

# 3. Linux




## 3.0 How to use these notes

- Practise every command in a terminal while you read: WSL2 Ubuntu on Windows, a VirtualBox/VMware VM, or a cloud instance.
- Commands starting with `sudo` change the system; run them only on a lab machine until you are confident.
- `$` at the start of an example line means "run as a normal user"; `#` means "run as root". Don't type the `$` / `#`.
- When a command is new, read its help: `man <command>` or `<command> --help`.

## 3.1 What Linux is and where it is used

**Linux** is an open-source, Unix-like operating system. Strictly speaking, "Linux" is the **kernel** (written by Linus Torvalds in 1991); a complete system adds GNU tools, a shell, libraries and software. Together they are a **distribution (distro)**.

**Where it runs:** most web servers and cloud instances, almost every container image, Kubernetes nodes, network devices, Android phones, embedded systems/IoT, supercomputers. As a DevOps/Cloud engineer you will spend most of your working day on Linux servers, usually without a graphical interface.

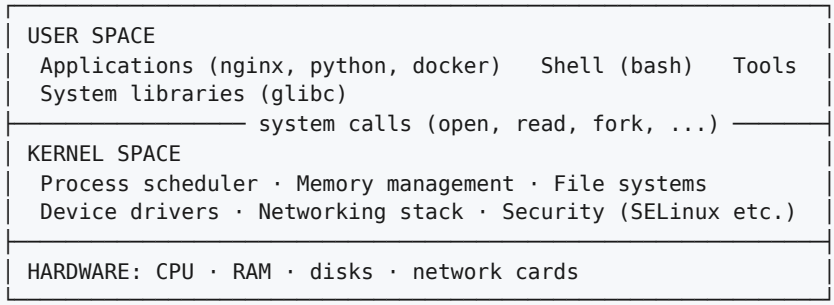
### Distribution families

| Family | Distributions | Package format / tool | Where you meet it |
|---|---|---|---|
| Debian | Debian, **Ubuntu**, Linux Mint | `.deb` / `apt` | Very common on cloud servers, WSL, CI runners |

| Family | Distributions | Package format / tool | Where you meet it |
|---|---|---|---|
| Red Hat | **RHEL**, Rocky Linux, AlmaLinux, Fedora, CentOS Stream, **Amazon Linux** | `.rpm` / `dnf` (older: `yum`) | Enterprises, banks, AWS |
| SUSE | SLES, openSUSE | `.rpm` / `zypper` | European enterprises, SAP |
| Alpine | Alpine Linux | `apk` | Tiny container images |
| Arch | Arch Linux | `pacman` | Desktop enthusiasts |

**LTS (Long-Term Support)** releases (e.g. Ubuntu 24.04 LTS, RHEL 9) get security updates for many years; servers should run LTS versions. Check the version: `cat /etc/os-release`, kernel: `uname -r`.

## 3.2 Linux architecture and boot process




- **Kernel:** the core. Only it talks to hardware. Programs ask it for services through **system calls**.
- **Shell:** a program that reads your commands and asks the kernel to run programs (`bash`, `zsh`, `sh`).
- **User space:** everything outside the kernel.

### Boot sequence

```
Power on → Firmware (BIOS or UEFI) checks hardware, finds the boot disk
        → Bootloader (GRUB2) shows the menu, loads the kernel + initramfs
        → Kernel starts, detects hardware, mounts the root file system
        → systemd starts as PID 1 (the first process)
        → systemd starts services to reach the default target (multi-user / graphical)
        → login prompt / SSH available
```

| Piece | Location / command |
|---|---|
| GRUB config | `/etc/default/grub`, apply with `sudo update-grub` (Ubuntu) or `grub2-mkconfig` (RHEL) |
| Kernel and initramfs | `/boot/vmlinuz-*`, `/boot/initrd.img-*` / `initramfs-*` |
| Default target | `systemctl get-default` (usually `multi-user.target` on servers) |
| Boot messages | `journalctl -b` (this boot), `dmesg` (kernel ring buffer) |
| Boot time analysis | `systemd-analyze`, `systemd-analyze blame` |

**Rescue options:** `rescue.target` (single-user, minimal services) and `emergency.target` (root shell only) are chosen from GRUB by adding `systemd.unit=rescue.target` to the kernel line. On cloud instances use the **serial console** or attach the disk to another instance.

## 3.3 The shell and the command line

### The prompt

```
thirupathi@web01:~/projects$
|          |     |          └ $ = normal user   (# = root)
|          |     └ current directory (~ = your home)
|          └ hostname
└ username
```

### Command anatomy

```
ls  -l -a  --human-readable  /var/log
|   └─┬─┘  └──────┬───────┘   └──┬──┘
|   short options long option   argument (what to act on)
command
```

- Short options combine: `ls -la` = `ls -l -a`.
- Many commands accept `--` to mark the end of options (useful for file names starting with `-`).
- Commands are **case-sensitive**: `ls` works, `LS` does not.

### Getting help

| Command | What it gives |
|---|---|
| `man ls` | Full manual (`/word` search, `n` next, `q` quit). Sections: 1 commands, 5 file formats, 8 admin commands (`man 5 crontab`) |
| `ls --help` | Short usage summary |
| `apropos network` / `man -k network` | Search manual descriptions |
| `whatis ls` | One-line description |
| `type cd` | Is it a builtin, alias, function or file? |
| `which python3` / `command -v python3` | Path of the program that would run |
| `tldr tar` | Community examples (install `tldr`) |

### Keyboard shortcuts (bash)

| Keys | Action |
|---|---|
| `Tab` / `Tab Tab` | Complete / show all completions |
| `↑` `↓` | Previous / next command |
| `Ctrl+R` | Search history (type part of a command) |
| `Ctrl+C` | Interrupt (stop) the running command |
| `Ctrl+D` | End of input / log out of the shell |
| `Ctrl+Z` | Suspend the running command (then `bg` or `fg`) |
| `Ctrl+L` | Clear the screen |
| `Ctrl+A` / `Ctrl+E` | Start / end of line |

| Keys | Action |
|---|---|
| `Ctrl+U` / `Ctrl+K` | Delete to start / end of line |
| `Ctrl+W` | Delete previous word |
| `!!` / `sudo !!` | Repeat last command (with sudo) |
| `!$` | Last argument of the previous command |
| `history` / `!123` | Show history / run entry 123 |

### Shells

`bash` (default on most distros), `sh` / `dash` (minimal, used for system scripts), `zsh` (macOS default, popular with plugins), `fish` (user-friendly). Current shell: `echo $SHELL`; change: `chsh -s /bin/zsh`.

## 3.4 The file system

### Filesystem Hierarchy Standard (FHS)

```
/                   root of everything (one tree; no C:\ D:\ drives)
├── bin, sbin       essential commands (today usually links into /usr)
├── boot            kernel, initramfs, GRUB files
├── dev             device files (disks /dev/sda, /dev/nvme0n1, /dev/null)
├── etc             system-wide configuration (text files)
├── home            users' home directories (/home/thirupathi)
├── lib, lib64      shared libraries
├── media, mnt      mount points for removable/temporary file systems
├── opt             optional third-party software
├── proc            virtual: live process and kernel info (/proc/cpuinfo, /proc/<PID>)
├── root            home directory of the root user
├── run             runtime data since boot (PID files, sockets)
├── srv             data served by this system (web/ftp)
├── sys             virtual: devices and kernel settings
├── tmp             temporary files (often cleared on reboot)
├── usr             installed software (/usr/bin, /usr/lib, /usr/share, /usr/local)
└── var             variable data: logs (/var/log), caches, mail, web roots, databases
```

**Most important for DevOps:** `/etc` (config), `/var/log` (logs), `/home` (users), `/tmp`, `/opt` (apps), `/var/lib` (data of services like Docker, databases), `/proc` and `/sys` (live system info).

### Everything is a file

| Type (first character in `ls -l`) | Meaning |
|---|---|
| `-` | Regular file |
| `d` | Directory |
| `l` | Symbolic link |
| `b` | Block device (disk) |
| `c` | Character device (terminal, `/dev/null`) |
| `s` | Socket |
| `p` | Named pipe (FIFO) |

### Paths

| Path | Meaning |
|---|---|
| `/etc/nginx/nginx.conf` | **Absolute**: starts at `/` |
| `logs/app.log` | **Relative**: from the current directory |
| `.` / `..` | Current / parent directory |
| `~` / `~alice` | Your home / alice's home |
| `-` (with `cd`) | Previous directory |
| `.bashrc` | Files starting with `.` are **hidden** (`ls -a` shows them) |

File names are case-sensitive, can contain spaces (quote them: `"my file.txt"`), and have no required extension (`file` tells you the real type).

## 3.5 Working with files and directories

| Command | Purpose | Useful options |
|---|---|---|
| `pwd` | Print working directory | |
| `ls` | List | `-l` long, `-a` hidden, `-h` human sizes, `-t` by time, `-S` by size, `-r` reverse, `-R` recursive, `-d` directory itself, `-i` inode numbers |
| `cd dir` | Change directory | `cd`, `cd ~`, `cd ..`, `cd -` |
| `mkdir dir` | Make directory | `-p` create parents, no error if exists |
| `touch file` | Create empty file / update timestamp | |
| `cp src dst` | Copy | `-r` recursive, `-p` keep permissions/times, `-a` archive (everything), `-i` ask before overwrite, `-v` verbose |
| `mv src dst` | Move or rename | `-i`, `-n` never overwrite |
| `rm file` | Remove | `-r` recursive, `-f` force, `-i` ask. **No recycle bin** |
| `rmdir dir` | Remove an empty directory | |
| `ln -s target link` | Symbolic link | |
| `file name` | Detect file type | |
| `stat name` | Size, permissions, inode, timestamps | |
| `tree -L 2` | Directory tree, 2 levels | |

**Safe deletion habits:** run `ls` with the same pattern before `rm`; use `rm -i` for important files; never `rm -rf /` or `rm -rf $VAR/` with an unset variable (use `set -u` in scripts).

### Links and inodes

An **inode** stores a file's metadata (owner, permissions, size, data block locations); the file name is just a directory entry pointing to an inode.

| | Hard link ( `ln a b` ) | Symbolic link ( `ln -s a b` ) |
|---|---|---|
| Points to | The same inode | A path (name) |
| If original deleted | Data still exists | Link breaks ("dangling") |
| Across file systems | No | Yes |
| For directories | No | Yes |
| Typical use | Rare (backups, dedupe) | Very common: versions ( `/opt/app/current → /opt/app/v2` ), config switches |

### Wildcards (globbing) and expansion

| Pattern | Matches | Example |
|---|---|---|
| `*` | Any characters | `ls *.log` |
| `?` | Exactly one character | `ls file?.txt` |
| `[abc]` , `[0-9]` | One character from a set/range | `ls app[12].conf` |
| `[!a]` | Any character except | `ls [!t]*` |
| `{a,b,c}` | Brace expansion (generates words) | `mkdir -p proj/{src,tests,docs}` |
| `{1..5}` | Sequence | `touch log{1..5}.txt` |

The **shell** expands wildcards before the command runs. Quote patterns to stop expansion: `find . -name "*.log"` .

### Quoting

| Quote | Effect | Example |
|---|---|---|
| `'single'` | Everything literal | `echo '$HOME'` → `$HOME` |
| `"double"` | Variables and `$(...)` expand; spaces kept | `echo "Home: $HOME"` |
| `\` | Escape one character | `echo \$HOME` |

## 3.6 Viewing files

| Command | Use |
|---|---|
| `cat file` | Print whole file (small files) |
| `cat -n file` | With line numbers |
| `tac file` | Print in reverse line order |
| `less file` | Page through: `Space` / `b` page, `/text` search, `n` / `N` next/prev, `G` end, `g` start, `F` follow, `q` quit |
| `head -n 20 file` | First 20 lines |
| `tail -n 50 file` | Last 50 lines |
| `tail -f file` | Follow new lines (logs); `-F` also follows across log rotation |
| `wc -l file` | Count lines ( `-w` words, `-c` bytes) |

| Command | Use |
|---|---|
| `diff -u old new` | Differences between files (unified format) |
| `cmp a b` | Byte-by-byte comparison |
| `md5sum` / `sha256sum file` | Checksums to verify files |
| `xxd file` / `hexdump -C file` | Hex view of binary files |

## 3.7 Searching

### find

```
find /var/log -name "*.log"                # by name (case-sensitive; -iname ignores case)
find . -type f -size +100M                 # files larger than 100 MB (-type d = directories)
find /tmp -type f -mtime +7                # modified more than 7 days ago (-mmin for minutes)
find /home -user alice                     # owned by alice
find / -perm -4000 -type f 2>/dev/null     # SUID files (security audit)
find . -maxdepth 1 -type f -empty          # empty files in this directory only
find /var/log -name "*.gz" -mtime +30 -delete              # delete old compressed logs
find . -name "*.sh" -exec chmod +x {} \;    # run a command on each result
find . -name "*.txt" -print0 | xargs -0 grep -l "TODO"     # safe with spaces in names
```

`locate name` searches a prebuilt database (fast; refresh with `sudo updatedb`).

### grep

| Option | Meaning |
|---|---|
| `grep "error" file` | Lines containing "error" |
| `-i` | Ignore case |
| `-r` / `-R` | Recursive through directories |
| `-n` | Show line numbers |
| `-v` | Invert: lines **not** matching |
| `-c` | Count matching lines |
| `-l` | Only file names with matches |
| `-w` | Whole words only |
| `-o` | Print only the matched part |
| `-E` | Extended regex, e.g. `grep -E "error\|fail"` (error or fail) |
| `-A 3` / `-B 3` / `-C 3` | Lines after / before / around each match |
| `--include="*.py"` | Limit recursive search to file patterns |

### Regular expressions (regex) essentials

| Pattern | Matches |
|---|---|
| `^start` | Line starting with "start" |
| `end$` | Line ending with "end" |

| Pattern | Matches |
|---|---|
| `.` | Any single character |
| `*` | Previous item 0 or more times |
| `+` | 1 or more (extended regex) |
| `?` | 0 or 1 (extended regex) |
| `[0-9]`, `[a-z]` | Character ranges |
| `[^0-9]` | Not a digit |
| `{3}`, `{2,4}` | Exact / range repetitions (extended) |
| `(a\|b)` | Alternatives (extended) |
| `\b` | Word boundary |

```
grep -E "^[0-9]{1,3}(\.[0-9]{1,3}){3}" access.log     # lines starting with an IPv4 address
grep -E " (4|5)[0-9]{2} " access.log                   # HTTP 4xx/5xx status codes
grep -rn --include="*.yaml" "image:" k8s/              # find image lines in YAML files
```

## 3.8 Text processing

Linux tools are small and combine through pipes. Learn these and you can analyze any log file.

| Command | Purpose | Example |
|---|---|---|
| `cut` | Pick columns by delimiter or position | `cut -d: -f1,7 /etc/passwd` (user and shell) |
| `sort` | Sort lines | `-n` numeric, `-r` reverse, `-h` human sizes ( `10K` , `2G` ), `-k2` by column 2, `-t,` delimiter, `-u` unique |
| `uniq` | Collapse **adjacent** duplicates (sort first) | `-c` count, `-d` only duplicates |
| `tr` | Translate/delete characters | `tr 'a-z' 'A-Z'` , `tr -d '\r'` , `tr -s ' '` (squeeze spaces) |
| `paste` / `join` / `column -t` | Merge files side by side / join on a key / align columns | `column -t -s, data.csv` |
| `tee` | Write to a file **and** pass output on | `cmd \| tee out.log` |
| `xargs` | Build commands from input | `cat ids.txt \| xargs -n1 echo` |

### sed (stream editor)

```
sed 's/old/new/' file              # replace first match on each line (prints result)
sed 's/old/new/g' file             # replace all matches on each line
sed -i 's/8080/80/g' app.conf      # edit the file in place (-i.bak keeps a backup)
sed -n '10,20p' file               # print only lines 10-20
sed '/^#/d' file                   # delete comment lines
sed '/^$/d' file                   # delete empty lines
sed -i '3i new line' file          # insert a line before line 3
sed 's#/var/www#/srv/www#g' file   # any delimiter works (useful with paths)
```

### awk (field processing)

awk splits each line into fields: `$1`, `$2`, ... (`$0` = whole line, `NF` = number of fields, `NR` = line number).

```
awk '{print $1}' access.log                          # first field
awk -F: '{print $1, $3}' /etc/passwd                 # custom separator
awk '$9 >= 500 {print $1, $7}' access.log            # condition: status code >= 500
awk '{sum += $10} END {print sum/1024/1024 " MB"}' access.log   # total bytes sent
awk 'NR==1 || /ERROR/' app.log                       # header line + error lines
awk '{count[$1]++} END {for (ip in count) print count[ip], ip}' access.log | sort -rn | head
printf "%-10s %5d\n" web 42                          # formatted output (also inside awk)
```

### Log analysis one-liners

```
# Top 10 client IPs in an nginx access log
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10

# Count responses per status code
awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn

# Most requested URLs that returned 404
awk '$9 == 404 {print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head

# Errors per hour in a syslog-style log
grep -i error /var/log/syslog | awk '{print $3}' | cut -d: -f1 | sort | uniq -c

# Failed SSH logins by source IP
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn
```

## 3.9 Text editors

### nano (simplest)

`nano file` · shortcuts are shown at the bottom (`^` = Ctrl): `Ctrl+O` save, `Ctrl+X` exit, `Ctrl+W` search, `Ctrl+K` cut line, `Ctrl+U` paste, `Alt+U` undo.

### vim (on every server, worth learning)

Vim has **modes**: **Normal** (move and run commands, default), **Insert** (type text), **Visual** (select), **Command-line** (`:` commands).

| Keys (Normal mode) | Action |
|---|---|
| `i` / `a` / `o` | Insert before cursor / after cursor / new line below |
| `Esc` | Back to Normal mode |
| `h j k l` | Left, down, up, right |
| `w` / `b` | Next / previous word |
| `0` / `$` | Start / end of line |
| `gg` / `G` / `:42` | First line / last line / line 42 |
| `x` / `dd` / `5dd` | Delete char / line / 5 lines |
| `yy` / `p` | Copy (yank) line / paste |
| `u` / `Ctrl+R` | Undo / redo |
| `/text` then `n` | Search forward, next match |

| Keys (Normal mode) | Action |
|---|---|
| `:%s/old/new/g` | Replace in the whole file |
| `v` / `V` | Visual select characters / lines |
| `:w` / `:q` / `:wq` / `:q!` | Save / quit / save and quit / quit without saving |
| `:set number` | Show line numbers |

**Survival rule:** stuck in vim? Press `Esc`, type `:q!`, press Enter.

## 3.10 Redirection, pipes and exit codes

Every process has three standard **file descriptors**:

| FD | Name | Default |
|---|---|---|
| 0 | stdin (input) | Keyboard |
| 1 | stdout (normal output) | Screen |
| 2 | stderr (errors) | Screen |

| Syntax | Meaning |
|---|---|
| `cmd > file` | stdout to file (overwrite) |
| `cmd >> file` | stdout appended |
| `cmd 2> err.log` | stderr to file |
| `cmd > all.log 2>&1` | stdout and stderr to the same file (order matters) |
| `cmd &> all.log` | Same, bash shortcut |
| `cmd > /dev/null 2>&1` | Discard everything |
| `cmd < file` | File as stdin |
| `cmd << EOF ... EOF` | Here-document (multi-line input) |
| `cmd <<< "text"` | Here-string |
| `cmd1 \| cmd2` | Pipe stdout of cmd1 into stdin of cmd2 |
| `cmd1 \|& cmd2` | Pipe stdout and stderr |
| `$(cmd)` | Command substitution: replaced by the output |
| `<(cmd)` | Process substitution: output as a temporary file ( `diff <(ls a) <(ls b)` ) |

**Exit codes:** every command returns a number: **0 = success**, anything else = failure. See it with `echo $?`.

| Operator | Meaning |
|---|---|
| `cmd1 ; cmd2` | Run both, always |
| `cmd1 && cmd2` | Run cmd2 only if cmd1 succeeded |
| `cmd1 \|\| cmd2` | Run cmd2 only if cmd1 failed |
| `cmd &` | Run in the background |

## 3.11 Users and groups

### Account files

```
/etc/passwd   alice:x:1001:1001:Alice Kumar:/home/alice:/bin/bash
              |     | |    |    |           |           └ login shell (/usr/sbin/nologin = no login)
              |     | |    |    |           └ home directory
              |     | |    |    └ comment (full name)
              |     | |    └ primary group ID (GID)
              |     | └ user ID (UID)
              |     └ x = password is in /etc/shadow
              └ username

/etc/shadow   alice:$6$...hash...:19700:0:90:7:::      (password hash + aging; root-only)
/etc/group    devops:x:1002:alice,bob                  (group name, GID, members)
```

| UID range | Meaning |
|---|---|
| 0 | root (superuser) |
| 1–999 | System/service accounts (nginx, www-data, sshd) |
| 1000+ | Normal users |

### Commands

| Command | Purpose |
|---|---|
| `id`, `id alice` | UID, GID, groups |
| `whoami`, `who`, `w`, `last`, `lastlog` | Current user / logged-in users / what they do / login history |
| `sudo useradd -m -s /bin/bash -c "Alice Kumar" alice` | Create user with home dir (`-m`), shell, comment |
| `sudo adduser alice` | Interactive friendly version (Debian/Ubuntu) |
| `sudo passwd alice` | Set password |
| `sudo usermod -aG docker alice` | **Append** to a supplementary group (without `-a` you replace all groups!) |
| `sudo usermod -L alice` / `-U` | Lock / unlock account |
| `sudo usermod -s /usr/sbin/nologin svc` | Disable interactive login for a service account |
| `sudo userdel -r alice` | Delete user and home |
| `sudo chage -l alice` / `chage -M 90 alice` | Show / set password aging |
| `sudo groupadd devops`, `groupdel`, `gpasswd -d alice devops` | Manage groups / remove member |
| `groups alice` | Group memberships |
| `/etc/skel/` | Template files copied into new home directories |

Group changes apply at the **next login** (or run `newgrp docker`).

### root, su and sudo

| | `su - alice` | `sudo command` |
|---|---|---|
| Does | Switch to another user's shell (needs **their** password) | Run one command as root (needs **your** password) |
| Audit | Weak | Every command logged ( `/var/log/auth.log` , `journalctl` ) |
| Best practice | Avoid sharing root password | Grant specific users/groups specific rights |

**sudoers** (edit only with `sudo visudo` , or files in `/etc/sudoers.d/` ):

```
# who      where=(as whom)   which commands
%admins    ALL=(ALL:ALL)     ALL
deploy     ALL=(root)        NOPASSWD: /usr/bin/systemctl restart myapp
alice      ALL=(ALL)         /usr/bin/journalctl, /usr/bin/systemctl status *
```

`sudo -l` lists what you may run. `sudo -i` opens a root login shell (use sparingly).

## 3.12 Permissions in depth

```
-rwxr-x---  1  alice  devops  4096  Oct 17 10:00  deploy.sh
 |└┘└┘└┘     |      |
 | |  |  |      owner  group
 | |  |  └ others: ---
 | |  └ group:  r-x
 | └ owner (user): rwx
 └ type
```

| Permission | On a file | On a directory |
|---|---|---|
| `r` (4) | Read contents | List names ( `ls` ) |
| `w` (2) | Modify contents | Create, delete, rename entries inside |
| `x` (1) | Execute as a program | Enter it ( `cd` ) and access files inside |

**Important:** deleting a file depends on **write permission on the directory**, not on the file.

### chmod

| Numeric | Symbolic | Typical use |
|---|---|---|
| `755` | `rwxr-xr-x` | Scripts, programs, directories |
| `644` | `rw-r--r--` | Normal files, config |
| `600` | `rw-------` | Private keys, secrets |
| `700` | `rwx------` | Private directories ( `~/.ssh` ) |
| `640` | `rw-r-----` | Config readable by a service group |
| `775` | `rwxrwxr-x` | Shared team directories |

```
chmod 640 app.conf
chmod u+x deploy.sh          # add execute for the owner (u=user, g=group, o=others, a=all)
chmod g-w,o= report.txt      # remove group write; others nothing
chmod -R 750 /srv/app        # recursive
chown alice:devops file      # owner and group
chown -R www-data: /var/www  # owner www-data, group = its primary group
chgrp devops file
```

### umask

The **umask** removes permissions from new files/directories. Default files start at 666, directories at 777.

| umask | New file | New directory |
|---|---|---|
| `022` (common default) | 644 | 755 |
| `027` (more secure) | 640 | 750 |
| `077` (private) | 600 | 700 |

Show with `umask`; set in `~/.bashrc` or `/etc/login.defs`.

### Special permission bits

| Bit | Number | On a file | On a directory | Example |
|---|---|---|---|---|
| SUID | 4000 | Runs with the **owner's** privileges | — | `/usr/bin/passwd` (`-rwsr-xr-x`) |
| SGID | 2000 | Runs with the group's privileges | New files inherit the **directory's group** | Shared team folders: `chmod 2775 /srv/team` |
| Sticky | 1000 | — | Only the file owner can delete their files | `/tmp` (`drwxrwxrwt`) |

Audit SUID files regularly: `find / -perm -4000 -type f 2>/dev/null`.

### ACLs and attributes

- **ACLs** give permissions to extra users/groups: `setfacl -m u:bob:rx /srv/reports`, view with `getfacl /srv/reports` (a `+` appears in `ls -l`).
- **Attributes:** `sudo chattr +i file` makes a file immutable (even root can't change it until `chattr -i`); `lsattr` shows attributes.
- **Capabilities:** give a program one root power without full root, e.g. bind to port 80: `sudo setcap 'cap_net_bind_service=+ep' /usr/local/bin/app`; view with `getcap`.

### Permission troubleshooting

`Permission denied` → check **every directory in the path** (`namei -l /srv/app/config/app.yaml` shows permissions along the path), the file mode, the owner/group, the process user (`ps -o user= -p <pid>`), ACLs, and SELinux/AppArmor (3.25).

## 3.13 Processes and jobs

A **process** is a running program. Each has a **PID**, a parent (**PPID**), an owner, priority, memory and open files. `systemd` is PID 1, the ancestor of all processes.

## Process states

| State (in `ps` / `top`) | Meaning |
|---|---|
| `R` | Running or ready to run |
| `S` | Sleeping (waiting for an event): normal for idle services |
| `D` | Uninterruptible sleep (waiting for disk/network I/O). Many `D` processes = storage problem |
| `T` | Stopped (Ctrl+Z or SIGSTOP) |
| `Z` | Zombie: finished but parent hasn't collected its exit status |

**Zombie:** already dead, uses no CPU/RAM, only a process-table entry; fix the **parent** (restart it).
**Orphan:** parent died; adopted by systemd.

## Viewing processes

| Command | Use |
|---|---|
| `ps aux` | All processes (BSD style): USER, PID, %CPU, %MEM, VSZ, RSS, STAT, START, TIME, COMMAND |
| `ps -ef` | All processes (System V style) with PPID |
| `ps -ef --forest` / `pstree -p` | Parent/child tree |
| `ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%cpu \| head` | Top CPU users, custom columns |
| `pgrep -a nginx` | Find PIDs by name |
| `top` | Live view: `P` sort CPU, `M` memory, `k` kill, `1` per-CPU, `q` quit |
| `htop` | Friendlier `top` (install it) |
| `/proc/<PID>/` | Live info: `cmdline`, `status`, `environ`, `fd/` |
| `lsof -p <PID>` | Files and sockets a process has open |

**RSS** = physical memory used. **VSZ** = virtual memory reserved (often much larger; don't panic about it).

## Signals

| Signal | Number | Meaning | Send with |
|---|---|---|---|
| SIGTERM | 15 | Polite stop: process can clean up. **Default for `kill`** | `kill PID` |
| SIGKILL | 9 | Immediate kill by the kernel, no cleanup. **Last resort** | `kill -9 PID` |
| SIGINT | 2 | Interrupt (what Ctrl+C sends) | `kill -2 PID` |
| SIGHUP | 1 | Hang up; many daemons reload config on it | `kill -HUP PID` |
| SIGSTOP / SIGCONT | 19 / 18 | Pause / resume | `kill -STOP PID` |

`pkill -f "python app.py"` (by command line), `killall nginx` (by name).

### Foreground, background and jobs

| Command | Use |
|---|---|
| `long_task &` | Start in the background |
| `Ctrl+Z` then `bg` | Suspend, then continue in the background |
| `jobs`, `fg %1` | List jobs / bring job 1 to the foreground |
| `nohup cmd > out.log 2>&1 &` | Keep running after logout |
| `disown` | Detach a job from the shell |
| `tmux` / `screen` | Persistent terminal sessions: survive SSH disconnects (`tmux new -s work`, detach `Ctrl+B d`, `tmux attach -t work`) |

**For real services use systemd (3.16), not `nohup`.**

### Priority

Nice values range from **-20** (highest priority) to **19** (lowest); default 0. Normal users can only lower priority. `nice -n 10 ./backup.sh` · `sudo renice -n 5 -p 1234` · for disk: `ionice -c3 cmd` (idle I/O class).

### strace (advanced debugging)

`strace -p <PID>` or `strace -f -e trace=open,connect cmd` shows the system calls a program makes: which files it fails to open, which host it connects to. Powerful when logs say nothing.

## 3.14 System resources and performance

### CPU

| Command | What to look at |
|---|---|
| `nproc`, `lscpu` | Number of cores, CPU model |
| `uptime` | **Load average** for 1, 5, 15 minutes |
| `top` header `%Cpu(s)` | `us` user, `sy` kernel, `ni` niced, `id` idle, `wa` **I/O wait**, `st` stolen (by the hypervisor) |
| `mpstat -P ALL 1` | Per-core usage (package `sysstat`) |
| `pidstat 1` | Per-process CPU over time |

**Load average** = average number of processes running **or waiting** (CPU or uninterruptible I/O). Compare with the core count: load 4 on a 4-core machine = fully busy; load 12 on 4 cores = overloaded. High load with low CPU usage usually means **I/O wait** (disk/network storage problem).

High `st` (steal) on a cloud VM = the host is overcommitted or a burstable instance is out of CPU credits.

### Memory

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       2.1Gi       1.2Gi        45Mi       4.4Gi       5.3Gi
Swap:          2.0Gi          0B       2.0Gi
```

- **available** is the number that matters: memory that can be given to programs (includes reclaimable cache).

- **buff/cache** is used by the kernel to speed up disk access and is released when programs need it. "Free" being low is normal.
- **Swap** used heavily + slow system = not enough RAM (`vmstat 1`: watch `si` / `so` swap-in/out columns).
- **OOM killer:** when memory runs out, the kernel kills a process. Evidence: `dmesg -T | grep -i -E "killed process|out of memory"` or `journalctl -k | grep -i oom`.

### Disk I/O

`iostat -xz 1` (`%util` near 100% and high `await` = saturated disk) · `iotop` (per-process I/O) · `vmstat 1` (`wa` column, `b` = blocked processes).

### Network

`ss -s` (socket summary) · `ss -tan state established | wc -l` (number of connections) · `iftop` / `nload` (live bandwidth) · `sar -n DEV 1` · `ip -s link` (errors/drops).

### Performance method (USE)

For every resource (CPU, memory, disk, network) check **U**tilization, **S**aturation (queues, waiting), **E**rrors. Then find **which process** is responsible, then **why** (logs, recent change, traffic).

| Quick 60-second checklist | |
|---|---|
| `uptime` | Load trend |
| `dmesg -T \| tail` | Kernel errors, OOM kills, disk errors |
| `vmstat 1 5` | Run queue, memory, swap, I/O wait |
| `mpstat -P ALL 1 3` | CPU balance per core |
| `pidstat 1 3` | Which processes use CPU |
| `iostat -xz 1 3` | Disk saturation |
| `free -h` | Memory |
| `sar -n DEV 1 3` | Network throughput |
| `top` | Overall view |

## 3.15 Package management

**Concepts:** a **package** is software + metadata (version, dependencies, install scripts). **Repositories** are servers with packages and signed indexes. The package manager resolves **dependencies** and verifies **GPG signatures**.

### Debian/Ubuntu: apt and dpkg

| Command | Purpose |
|---|---|
| `sudo apt update` | Refresh package lists (do this first) |
| `sudo apt upgrade` | Upgrade installed packages |
| `sudo apt full-upgrade` | Upgrade, allowing removals/new dependencies |
| `sudo apt install nginx curl` | Install |
| `sudo apt install nginx=1.24.0-2ubuntu7` | Install a specific version |
| `sudo apt remove nginx` / `purge nginx` | Remove (purge also deletes config) |

| Command | Purpose |
|---|---|
| `sudo apt autoremove` | Remove unneeded dependencies |
| `apt search redis`, `apt show redis-server` | Search / details |
| `apt list --installed`, `apt list --upgradable` | Installed / upgradable |
| `apt-cache policy nginx` | Installed vs candidate versions and their repos |
| `sudo apt-mark hold nginx` | Prevent upgrades of a package |
| `dpkg -l \| grep nginx` | Installed package list |
| `dpkg -L nginx` | Files installed by a package |
| `dpkg -S /usr/sbin/nginx` | Which package owns a file |
| `sudo dpkg -i package.deb` | Install a local .deb (then `sudo apt -f install` for dependencies) |

**Adding a third-party repository (modern, keyring-based):**

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
```

Repositories are configured in `/etc/apt/sources.list` and `/etc/apt/sources.list.d/`.

## RHEL/Amazon Linux/Rocky: dnf and rpm

| Command | Purpose |
|---|---|
| `sudo dnf check-update` / `sudo dnf upgrade` | List / install updates |
| `sudo dnf install nginx` | Install |
| `sudo dnf remove nginx` | Remove |
| `dnf search`, `dnf info`, `dnf list installed` | Search / details / installed |
| `dnf provides /usr/bin/dig` | Which package provides a file |
| `dnf history`, `sudo dnf history undo <id>` | Transaction history / roll back |
| `dnf repolist` | Enabled repositories (`/etc/yum.repos.d/`) |
| `rpm -qa`, `rpm -qi nginx`, `rpm -ql nginx`, `rpm -qf /path` | Query installed packages, info, files, owner |

## Other ways software is installed

| Method | Note |
|---|---|
| Snap / Flatpak | Self-contained packages (`snap install`) |
| Language package managers | `pip`, `npm`, `gem`, `go install` (use virtual environments / per-project installs) |
| Single binaries | Many DevOps tools (Terraform, kubectl, Helm) are one binary: download, verify checksum, put in `/usr/local/bin` |

| Method | Note |
|---|---|
| Build from source | `./configure && make && sudo make install` (rare today) |
| Containers | Increasingly, software is run as a container image instead of being installed |

**Security:** install only from trusted, signed repositories; keep systems patched; on servers enable **unattended security upgrades** (`sudo apt install unattended-upgrades`) or `dnf-automatic`.

## 3.16 systemd and services

**systemd** is the init system (PID 1) and service manager on almost all modern distros. It starts services in parallel, restarts them on failure, manages logs (journald), timers, mounts and targets.

### Unit types

| Unit | Purpose |
|---|---|
| `.service` | A daemon or one-off task |
| `.socket` | Socket activation (start service on first connection) |
| `.timer` | Scheduled activation (cron alternative) |
| `.target` | Group of units / system state (`multi-user.target`) |
| `.mount` / `.automount` | File system mounts |
| `.path` | Trigger on file changes |

**Unit file locations:** `/usr/lib/systemd/system/` or `/lib/systemd/system/` (from packages: **don't edit**) · `/etc/systemd/system/` (yours and overrides: **takes priority**).

### systemctl

| Command | Purpose |
|---|---|
| `systemctl status nginx` | State, PID, memory, last log lines |
| `sudo systemctl start / stop / restart nginx` | Control now |
| `sudo systemctl reload nginx` | Reload config without dropping connections (if supported) |
| `sudo systemctl enable --now nginx` | Start at boot **and** now |
| `sudo systemctl disable nginx` | Don't start at boot |
| `sudo systemctl mask nginx` | Make it impossible to start |
| `systemctl is-active nginx`, `is-enabled nginx` | Scriptable checks |
| `systemctl --failed` | Failed units |
| `systemctl list-units --type=service --state=running` | Running services |
| `systemctl list-unit-files` | Installed units and enabled state |
| `systemctl cat nginx` | Show the unit file in use (including overrides) |
| `sudo systemctl edit nginx` | Create an override (drop-in) without touching the package file |

| Command | Purpose |
|---|---|
| `sudo systemctl daemon-reload` | Reload unit files after editing |
| `systemctl list-dependencies multi-user.target` | Dependency tree |

## Writing a service unit

`/etc/systemd/system/shop-api.service`:

```
[Unit]
Description=Shop API
Documentation=https://github.com/<owner>/shop-api
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=shop
Group=shop
WorkingDirectory=/opt/shop-api
EnvironmentFile=/etc/shop-api/env
ExecStart=/opt/shop-api/.venv/bin/gunicorn -b 127.0.0.1:8000 app:app
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
# Hardening
NoNewPrivileges=true
ProtectSystem=full
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

| Directive | Meaning |
|---|---|
| `After` / `Wants` / `Requires` | Start order / soft dependency / hard dependency |
| `Type` | `simple` (default, process stays in foreground), `forking` (daemonizes), `oneshot` (runs and exits), `notify` (tells systemd when ready) |
| `User`, `Group` | Run as an unprivileged account (create with `useradd -r -s /usr/sbin/nologin shop`) |
| `EnvironmentFile` | Variables from a file (permissions 600; keeps secrets out of the unit) |
| `Restart` | `on-failure`, `always`, `no` |
| `LimitNOFILE` | Max open files (raise for busy servers) |
| `WantedBy=multi-user.target` | `enable` links it into the normal boot |

Then: `sudo systemctl daemon-reload && sudo systemctl enable --now shop-api && systemctl status shop-api`.

## systemd timers (cron alternative)

`/etc/systemd/system/backup.service` (Type=oneshot running the script) + `/etc/systemd/system/backup.timer`:

```
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
```

`sudo systemctl enable --now backup.timer` · `systemctl list-timers`. Advantages over cron: logs in the journal, `Persistent=true` runs missed jobs after downtime, dependencies, resource limits.

### Targets

| Target | Meaning |
|---|---|
| `multi-user.target` | Normal server mode (network, no GUI) |
| `graphical.target` | With desktop |
| `rescue.target` | Single-user maintenance |
| `emergency.target` | Minimal root shell |

`systemctl get-default`, `sudo systemctl set-default multi-user.target`.

## 3.17 Logs

### Where logs live

| Source | Location |
|---|---|
| systemd journal (all services, kernel) | `journalctl` (binary; persistent if `/var/log/journal/` exists) |
| General system log | `/var/log/syslog` (Ubuntu) · `/var/log/messages` (RHEL) |
| Authentication, sudo, SSH | `/var/log/auth.log` (Ubuntu) · `/var/log/secure` (RHEL) |
| Kernel | `dmesg`, `/var/log/kern.log` |
| Package installs | `/var/log/apt/history.log` · `/var/log/dnf.log` |
| Applications | `/var/log/nginx/`, `/var/log/mysql/`, app-specific |
| Boot/cloud-init | `/var/log/cloud-init-output.log` |

### journalctl

| Command | Purpose |
|---|---|
| `journalctl -u nginx` | Logs of one unit |
| `journalctl -u nginx -f` | Follow live |
| `journalctl -u nginx -n 100 --no-pager` | Last 100 lines |
| `journalctl --since "1 hour ago"` / `--since "2026-10-17 09:00" --until "10:00"` | Time window |

| Command | Purpose |
|---|---|
| `journalctl -p err -b` | Errors (priority ≤ err) since the current boot |
| `journalctl -b -1` | Previous boot (needs persistent journal) |
| `journalctl -k` | Kernel messages |
| `journalctl _PID=1234` / `_UID=1001` | By process / user |
| `journalctl -o json-pretty -u app` | Structured output |
| `journalctl --disk-usage`, `sudo journalctl --vacuum-size=500M` | Size / shrink |

**Log levels (syslog priorities):** 0 emerg · 1 alert · 2 crit · 3 err · 4 warning · 5 notice · 6 info · 7 debug.

### Log rotation

`logrotate` (run daily) rotates, compresses and deletes old logs. Config: `/etc/logrotate.conf` and `/etc/logrotate.d/`.

```
/var/log/shop-api/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 shop shop
    postrotate
        systemctl reload shop-api > /dev/null 2>&1 || true
    endscript
}
```

Test: `sudo logrotate -d /etc/logrotate.d/shop-api` (dry run) · `sudo logrotate -f ...` (force).

**In production:** ship logs off the server (CloudWatch agent, Fluent Bit, Vector, Promtail/Alloy → Loki, Elastic) so they survive server loss and can be searched centrally.

## 3.18 Storage, file systems and LVM

### Concepts

```
Physical disk (/dev/sda, /dev/nvme0n1)
   └── Partition table (GPT or MBR)
         ├── Partition 1 (/dev/sda1)  → file system (ext4/xfs) → mounted at /
         └── Partition 2 (/dev/sda2)  → LVM PV → volume group → logical volumes → file systems
```

| Term | Meaning |
|---|---|
| Block device | Disk or partition ( `/dev/sda`, `/dev/nvme0n1p1`, `/dev/xvdf` ) |
| Partition table | **GPT** (modern, large disks, many partitions) or **MBR** (legacy, 2 TB limit, 4 primary) |
| File system | How data is organized: **ext4** (Ubuntu default), **xfs** (RHEL/Amazon Linux default, can grow but not shrink), btrfs, vfat (EFI) |
| Mount | Attach a file system to a directory (mount point) |

| Term | Meaning |
|---|---|
| UUID | Unique ID of a file system: use it in `/etc/fstab` (device names can change) |
| Inode | Metadata record per file; a file system can run out of inodes before space |

## Inspecting storage

| Command | Shows |
|---|---|
| `lsblk -f` | Disks, partitions, file systems, UUIDs, mount points |
| `df -hT` | Used/free space per mounted file system with type |
| `df -i` | Inode usage |
| `du -sh /var/*` | Size of each item ( `--max-depth=1` ) |
| `du -ah /var/log \| sort -rh \| head -20` | 20 biggest files/dirs |
| `blkid` | UUIDs and types |
| `mount \| column -t` , `findmnt` | Current mounts |
| `sudo fdisk -l` | Partition tables |

## Partition, format, mount (new disk)

```
sudo parted /dev/sdb --script mklabel gpt mkpart primary xfs 0% 100%
sudo mkfs.xfs /dev/sdb1               # or: sudo mkfs.ext4 /dev/sdb1   (ERASES data)
sudo mkdir -p /data
sudo mount /dev/sdb1 /data
sudo blkid /dev/sdb1                  # copy the UUID
echo 'UUID=<uuid>  /data  xfs  defaults,nofail  0  2' | sudo tee -a /etc/fstab
sudo umount /data && sudo mount -a    # test fstab BEFORE rebooting
```

## /etc/fstab fields

```
UUID=1234-abcd   /data   xfs    defaults,nofail   0   2
|                |       |      |                 |   └ fsck order (0 skip, 1 root, 2 others)
|                |       |      |                 └ dump (0)
|                |       |      └ options (defaults, noatime, ro, nofail, _netdev for network FS)
|                |       └ file system type
|                └ mount point
└ device (UUID preferred)
```

A broken fstab line can stop the server from booting: always run `sudo mount -a` after editing and use `nofail` for non-critical disks.

## LVM (Logical Volume Manager)

LVM adds a flexible layer: combine disks and resize volumes without repartitioning.

```
 /dev/sdb   /dev/sdc           ← PV  (physical volumes)
     └─────┬─────┘
        vg_data               ← VG  (volume group = pool of space)
     ┌─────┴─────┐
  lv_app     lv_logs          ← LV  (logical volumes = flexible "partitions")
  /srv/app   /var/log/app     ← file systems mounted
```

```
sudo pvcreate /dev/sdb /dev/sdc
sudo vgcreate vg_data /dev/sdb /dev/sdc
sudo lvcreate -n lv_app -L 20G vg_data
sudo mkfs.xfs /dev/vg_data/lv_app
sudo mount /dev/vg_data/lv_app /srv/app

# Grow later: add a disk, extend the VG, extend the LV and the file system in one step (-r)
sudo pvcreate /dev/sdd && sudo vgextend vg_data /dev/sdd
sudo lvextend -r -L +10G /dev/vg_data/lv_app

pvs; vgs; lvs                               # summaries
```

**Snapshots:** `lvcreate -s` creates point-in-time copies (useful before risky changes).

### Growing a cloud disk (e.g. AWS EBS)

```
lsblk                                       # disk bigger than the partition?
sudo growpart /dev/nvme0n1 1                # grow partition 1
sudo xfs_growfs /                           # XFS (takes the mount point)
# ext4 instead: sudo resize2fs /dev/nvme0n1p1
df -h /
```

### Swap

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
swapon --show; free -h
```

`vm.swappiness` (0–100, `sysctl vm.swappiness`) controls how eagerly the kernel swaps.

### Other storage topics

| Topic | Note |
|---|---|
| RAID (`mdadm`) | Software RAID 0 (speed), 1 (mirror), 5/6 (parity), 10 |
| NFS client | `sudo mount -t nfs server:/export /mnt/share` (fstab option `_netdev`) |
| fsck | Check/repair **unmounted** file systems: `sudo fsck /dev/sdb1` (xfs: `xfs_repair`) |
| `noatime` mount option | Don't update access times: less disk writes |

## 3.19 Archiving, compression and file transfer

| Command | Purpose |
|---|---|
| `tar -czvf backup.tar.gz /etc/nginx` | Create gzip archive (c create, z gzip, v verbose, f file) |
| `tar -cJvf backup.tar.xz dir/` | xz compression (smaller, slower); `-j` = bzip2 |
| `tar -tzvf backup.tar.gz` | List contents |
| `tar -xzvf backup.tar.gz -C /restore` | Extract into a directory |
| `tar --exclude='*.log' -czf app.tgz app/` | Exclude patterns |
| `gzip file` / `gunzip file.gz` / `zcat file.gz` | Compress / decompress / read compressed |
| `zip -r site.zip site/` / `unzip site.zip -d out/` | Zip format |

| Command | Purpose |
|---|---|
| `rsync -avz --delete src/ user@host:/dst/` | Sync (a archive, v verbose, z compress, delete extra files at destination) |
| `rsync -avzn ...` | Dry run first ( `-n` ) |
| `scp file user@host:/tmp/` / `scp -r dir host:` | Copy over SSH |
| `sftp user@host` | Interactive SSH file transfer |
| `curl -LO https://example.com/file.tgz` / `wget URL` | Download |
| `dd if=/dev/zero of=test bs=1M count=100` | Low-level copy (disk images, tests). Careful: `of=` a wrong disk destroys it |

**Trailing slash in rsync:** `src/` copies the *contents* of src; `src` copies the directory itself.

## 3.20 Environment and shell configuration

### Variables

| Type | Example | Visible to child processes? |
|---|---|---|
| Shell variable | `NAME=web` | No |
| Environment variable | `export NAME=web` | Yes |

| Command | Purpose |
|---|---|
| `env` / `printenv` | Environment variables |
| `echo "$PATH"` | Directories searched for commands (left to right) |
| `export PATH="$HOME/bin:$PATH"` | Add a directory to PATH |
| `unset NAME` | Remove |
| `NAME=value command` | Set only for that one command |
| `set -o` / `shopt` | Shell options |

Important variables: `PATH` , `HOME` , `USER` , `SHELL` , `PWD` , `LANG` / `LC_ALL` (locale), `EDITOR` , `HISTSIZE` , `PS1` (prompt), `TZ` .

### Startup files

| File | When it runs |
|---|---|
| `/etc/profile` , `/etc/profile.d/*.sh` | Login shells, all users |
| `~/.bash_profile` or `~/.profile` | Your login shells (SSH login) |
| `~/.bashrc` | Every interactive non-login shell (new terminal). Usually sourced from `.bash_profile` |
| `/etc/environment` | System-wide variables (simple `KEY=value` lines) |

Reload after editing: `source ~/.bashrc` .

```
# ~/.bashrc additions
alias ll='ls -lah'
alias k='kubectl'
export EDITOR=vim
export HISTSIZE=10000
export HISTTIMEFORMAT='%F %T '
mkcd() { mkdir -p "$1" && cd "$1"; }
```

**Secrets:** never put passwords or keys in `.bashrc` or scripts committed to Git. Use a secrets manager, or a `chmod 600` env file loaded by the service.

## 3.21 Networking on Linux

### Interfaces and addresses (`ip`)

| Command | Purpose |
|---|---|
| `ip a` (`ip addr show`) | Interfaces and IPs (`lo` loopback, `eth0` / `ens5` / `enp0s3` network) |
| `ip -br a` | Brief view |
| `ip link set eth0 up/down` | Enable/disable interface |
| `sudo ip addr add 10.0.0.50/24 dev eth0` | Add an IP (temporary, lost at reboot) |
| `ip route` / `ip r` | Routing table (`default via 10.0.0.1` = gateway) |
| `ip route get 8.8.8.8` | Which route/interface is used for a destination |
| `sudo ip route add 10.20.0.0/16 via 10.0.0.254` | Add a static route (temporary) |
| `ip neigh` | ARP/neighbour table |
| `ip -s link` | Packet counters, errors, drops |

`ifconfig`, `route`, `netstat`, `arp` are older (net-tools) equivalents you'll still see in docs.

### Persistent network configuration

**Ubuntu (netplan)** `/etc/netplan/01-netcfg.yaml`, apply with `sudo netplan try` / `sudo netplan apply`:

```
network:
  version: 2
  ethernets:
    ens5:
      dhcp4: false
      addresses: [10.0.1.50/24]
      routes:
        - to: default
          via: 10.0.1.1
      nameservers:
        addresses: [10.0.0.2, 1.1.1.1]
```

**RHEL family (NetworkManager):** `nmcli con show`, `nmcli con mod ens5 ipv4.addresses 10.0.1.50/24 ipv4.gateway 10.0.1.1 ipv4.method manual`, `nmcli con up ens5`. On cloud instances, networking is normally DHCP-managed by the platform: don't hard-code IPs.

## Name resolution

| Item | Detail |
|---|---|
| `/etc/hosts` | Static name → IP entries, checked first ( `10.0.1.20 db.internal` ) |
| `/etc/resolv.conf` | DNS servers and search domains (often managed by systemd-resolved: `nameserver 127.0.0.53` ) |
| `/etc/nsswitch.conf` | Order of lookups ( `hosts: files dns` ) |
| `resolvectl status` | Real upstream DNS servers with systemd-resolved |
| `hostnamectl set-hostname web01` | Set hostname |
| `getent hosts name` | Resolve exactly as applications do (honours /etc/hosts) |

## Diagnostic tools

| Tool | Use |
|---|---|
| `ping -c 4 host` | Reachability and latency (ICMP may be blocked by firewalls) |
| `traceroute host` / `tracepath host` / `mtr host` | Path and where packets stop/slow down |
| `dig name` , `dig +short name` , `dig @8.8.8.8 name` , `dig name MX` | DNS queries (answer, TTL, server used) |
| `nslookup name` , `host name` | Simpler DNS lookups |
| `curl -v https://site` | Full HTTP(S) exchange: DNS, TCP, TLS, headers |
| `curl -I URL` / `curl -o /dev/null -s -w "%{http_code} %{time_total}\n" URL` | Headers only / status code and timing |
| `wget URL` | Download |
| `ss -tulpn` | Listening TCP/UDP ports with process ( `t` TCP, `u` UDP, `l` listening, `p` process, `n` numeric) |
| `ss -tan` | All TCP connections and states (ESTAB, TIME-WAIT, SYN-SENT…) |
| `nc -zv host 443` | Test whether a TCP port is open |
| `nc -l 9000` | Quick listener for testing |
| `sudo tcpdump -i any -nn port 443` | Capture packets ( `-w file.pcap` to save for Wireshark) |
| `openssl s_client -connect site:443 -servername site` | Inspect TLS certificate and handshake |

**`ss` output example:** `LISTEN 0 511 0.0.0.0:80 0.0.0.0:* users:(("nginx",pid=812,fd=6))` → nginx listens on port 80 on all IPv4 addresses. `127.0.0.1:8000` would mean **local only**: not reachable from other machines.

### Firewalls

| Tool | Where | Examples |
|---|---|---|
| `ufw` | Ubuntu (simple front-end) | `sudo ufw default deny incoming`, `sudo ufw allow 22/tcp`, `sudo ufw allow from 10.0.0.0/16 to any port 5432`, `sudo ufw enable`, `sudo ufw status numbered` |
| `firewalld` | RHEL family | `sudo firewall-cmd --add-service=http --permanent`, `--add-port=8080/tcp --permanent`, `sudo firewall-cmd --reload`, `--list-all` |
| `nftables` / `iptables` | Underlying kernel firewall | `sudo nft list ruleset`, `sudo iptables -L -n -v` (Docker and Kubernetes write rules here) |

**Before enabling a firewall over SSH, allow port 22 first** or you lock yourself out. In the cloud, security groups are the main firewall; host firewalls add defense in depth.

### Kernel network settings (sysctl)

`sysctl net.ipv4.ip_forward` · enable routing: `sudo sysctl -w net.ipv4.ip_forward=1` · persist in `/etc/sysctl.d/99-custom.conf` · reload with `sudo sysctl --system`. Common tuning: `net.core.somaxconn`, `fs.file-max`, `net.ipv4.ip_local_port_range`.

## 3.22 SSH in depth

**SSH (Secure Shell)** gives encrypted remote login, command execution, file transfer and tunnels (TCP port 22). Server daemon: `sshd`; client: `ssh`.

### Key-based authentication

```
Your laptop                                    Server
~/.ssh/id_ed25519      (PRIVATE, never share)
~/.ssh/id_ed25519.pub  ── copied to ──►  ~/.ssh/authorized_keys (one public key per line)
Login: server sends a challenge → your private key signs it → server verifies with the public key
```

```
ssh-keygen -t ed25519 -C "thirupathi@laptop"      # create key pair (set a passphrase)
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host     # install the public key on the server
ssh user@host                                      # log in
ssh -i ~/.ssh/aws-lab.pem ec2-user@203.0.113.10   # use a specific key
ssh user@host 'df -h; uptime'                      # run remote commands
```

**Required permissions** (SSH refuses keys that are too open): `~/.ssh` → 700 · private key → 600 · `authorized_keys` → 600 · home directory not group/world-writable.

### Client configuration `~/.ssh/config`

```
Host bastion
    HostName 203.0.113.10
    User ubuntu
    IdentityFile ~/.ssh/aws-lab.pem

Host app-*
    User ec2-user
    IdentityFile ~/.ssh/aws-lab.pem
    ProxyJump bastion

Host *
    ServerAliveInterval 60
    IdentitiesOnly yes
```

Now `ssh app-1` jumps through the bastion automatically (`ssh -J bastion user@10.0.11.5` does the same on the command line).

### ssh-agent

Holds decrypted keys in memory so you type the passphrase once: `eval "$(ssh-agent -s)"` · `ssh-add ~/.ssh/id_ed25519` · `ssh-add -l`. Agent forwarding (`-A`) is convenient but risky on untrusted servers; prefer `ProxyJump`.

### Port forwarding (tunnels)

| Type | Command | Use |
|---|---|---|
| Local | `ssh -L 5433:db.internal:5432 bastion` | Reach a private database at `localhost:5433` through the bastion |
| Remote | `ssh -R 8080:localhost:3000 server` | Expose your local port 3000 on the server's port 8080 |
| Dynamic (SOCKS) | `ssh -D 1080 bastion` | Browser proxy through the server |

### Server hardening (`/etc/ssh/sshd_config` or `/etc/ssh/sshd_config.d/*.conf`)

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowGroups ssh-users
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
```

Validate before restarting: `sudo sshd -t` → then `sudo systemctl reload ssh` (Ubuntu) / `sshd` (RHEL). **Keep your current session open** and test a new login before closing it.

Extra protection: firewall/security group allowing 22 only from known IPs or a VPN, `fail2ban` (bans IPs after failed attempts), or no SSH at all (AWS SSM Session Manager).

### known_hosts and fingerprints

The first connection asks you to trust the server's **host key** (saved in `~/.ssh/known_hosts`). `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED` means the key changed: expected if the server was rebuilt (remove the old entry with `ssh-keygen -R host`), dangerous otherwise (possible man-in-the-middle).

### SSH troubleshooting

| Symptom | Check |
|---|---|
| `Connection timed out` | Network path/firewall/security group/route; server running; correct IP |
| `Connection refused` | Host reachable but nothing on port 22: sshd stopped or different port |
| `Permission denied (publickey)` | Wrong user/key; key not in `authorized_keys`; permissions too open; `PubkeyAuthentication` |
| `Host key verification failed` | Changed host key (see above) |
| Slow login | Reverse DNS lookups (`UseDNS no`), GSSAPI (`GSSAPIAuthentication no`) |
| Debug | `ssh -vvv user@host` on the client; `sudo journalctl -u ssh -f` on the server |

## 3.23 Scheduling: cron, at and timers

### cron syntax

```
┌──────────── minute (0-59)
│ ┌────────── hour (0-23)
│ │ ┌──────── day of month (1-31)
│ │ │ ┌────── month (1-12)
│ │ │ │ ┌──── day of week (0-7, Sunday = 0 or 7)
│ │ │ │ │
* * * * *  command
```

| Expression | Runs |
|---|---|
| `*/5 * * * *` | Every 5 minutes |
| `0 2 * * *` | Daily at 02:00 |
| `30 9 * * 1-5` | 09:30 Monday–Friday |
| `0 0 1 * *` | Midnight on the 1st of each month |
| `0 */6 * * *` | Every 6 hours |
| `@reboot` / `@daily` / `@weekly` | Special strings |

| Where | Notes |
|---|---|
| `crontab -e` / `crontab -l` / `crontab -r` | Per-user crontab (edit / list / remove all!) |
| `/etc/crontab`, `/etc/cron.d/*` | System crontabs: have an extra **user** field: `0 2 * * * root /opt/backup.sh` |
| `/etc/cron.daily/` etc. | Scripts run daily/weekly/monthly |

**Cron gotchas:** minimal environment (set `PATH` in the crontab or use full paths), no terminal, `%` must be escaped as `\%`, output is lost unless redirected: `0 2 * * * /opt/backup.sh >> /var/log/backup.log 2>&1`. Check runs in `grep CRON /var/log/syslog` or `journalctl -u cron`.

`at`: one-off jobs: `echo "systemctl restart app" | at 23:00` · `atq` · `atrm`. Prefer **systemd timers** for important jobs (3.16).

## 3.24 Time synchronization

Correct time matters for logs, TLS certificates, Kerberos, distributed databases and AWS API signatures. Modern distros use **chrony** or **systemd-timesyncd**.

`timedatectl` (time zone, NTP sync status) · `sudo timedatectl set-timezone Asia/Kolkata` · `chronyc tracking` / `chronyc sources -v`. On AWS, the Amazon Time Sync Service is available at `169.254.169.123`. Servers usually run in **UTC**.

## 3.25 Linux security and hardening

### Hardening checklist (servers)

| Area | Actions |
|---|---|
| Updates | Patch regularly; automatic security updates ( `unattended-upgrades` / `dnf-automatic` ); reboot for kernel updates (or live patching) |
| Accounts | Named accounts (no shared root), least privilege via sudo, lock unused accounts, service accounts with `nologin` shell, strong password policy (PAM `pam_pwquality` , lockout with `faillock` ) |
| SSH | Keys only, no root login, restricted source IPs, fail2ban, or replace with SSM/VPN |
| Network | Firewall default-deny inbound; close unused ports ( `ss -tulpn` ); disable unused services ( `systemctl disable --now` ) |
| Files | Correct permissions on secrets (600), audit SUID/SGID and world-writable files, `/tmp` mounted `noexec` where possible |
| Mandatory access control | Keep **SELinux** (RHEL) or **AppArmor** (Ubuntu) **enforcing** |
| Logging/audit | Persistent journal, central log shipping, `auditd` rules for sensitive files |
| Integrity | File integrity monitoring (AIDE), malware/rootkit checks |
| Baselines | CIS Benchmarks; scan with **Lynis** ( `sudo lynis audit system` ) or OpenSCAP |

### SELinux (RHEL, Rocky, Amazon Linux)

Labels every file and process with a **context**; policy decides which process types may access which file types, even if normal permissions allow it.

| Command | Purpose |
|---|---|
| `getenforce` / `sestatus` | Mode: **Enforcing**, Permissive (log only), Disabled |
| `sudo setenforce 0` | Temporarily permissive (troubleshooting only) |
| `ls -Z /var/www/html` | File contexts (e.g. `httpd_sys_content_t` ) |
| `ps -eZ \| grep nginx` | Process contexts |
| `sudo restorecon -Rv /var/www/html` | Reset contexts to policy defaults (after moving files) |
| `sudo semanage fcontext -a -t httpd_sys_content_t "/srv/site(/.*)?"` | Define context for a custom path, then `restorecon` |
| `sudo semanage port -a -t http_port_t -p tcp 8081` | Allow a service on a non-standard port |

| Command | Purpose |
|---|---|
| `getsebool -a \| grep httpd`, `sudo setsebool -P httpd_can_network_connect on` | Booleans (e.g. let nginx proxy to backends) |
| `sudo ausearch -m avc -ts recent`, `audit2why` | Read denials and why they happened |

**Never "fix" problems by disabling SELinux permanently**: find the right context/boolean.

### AppArmor (Ubuntu)

Per-program profiles (`/etc/apparmor.d/`). `sudo aa-status` shows loaded profiles; denials appear in `journalctl -k | grep apparmor`. Modes: enforce / complain (`aa-complain`).

### auditd

Kernel-level auditing: `sudo auditctl -w /etc/passwd -p wa -k identity` watches writes/attribute changes; search with `ausearch -k identity`. Rules persist in `/etc/audit/rules.d/`.

## 3.26 Hands-on lab: nginx web server and reverse proxy

```
# 1) Install and start
sudo apt update && sudo apt install -y nginx          # RHEL: sudo dnf install -y nginx
sudo systemctl enable --now nginx
curl -I http://localhost                               # expect HTTP/1.1 200 OK

# 2) Inspect
ss -tulpn | grep :80
ls /etc/nginx/        # nginx.conf, sites-available/ + sites-enabled/ (Ubuntu), conf.d/
tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

**Site config** `/etc/nginx/sites-available/shop` (Ubuntu; on RHEL use `/etc/nginx/conf.d/shop.conf`):

```
upstream shop_backend {
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
}

server {
    listen 80;
    server_name shop.example.com;

    location /static/ {
        root /srv/shop;                    # serves /srv/shop/static/...
        expires 7d;
    }

    location / {
        proxy_pass http://shop_backend;     # round-robin load balancing
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    access_log /var/log/nginx/shop.access.log;
    error_log  /var/log/nginx/shop.error.log;
}
```

```
sudo ln -s /etc/nginx/sites-available/shop /etc/nginx/sites-enabled/shop
sudo nginx -t                        # ALWAYS test the configuration
sudo systemctl reload nginx

# Two test backends
mkdir -p /tmp/b1 /tmp/b2
echo backend-1 > /tmp/b1/index.html; echo backend-2 > /tmp/b2/index.html
(cd /tmp/b1 && python3 -m http.server 8001 &) ; (cd /tmp/b2 && python3 -m http.server 8002 &)
for i in 1 2 3 4; do curl -s -H "Host: shop.example.com" http://localhost/; done   # alternates
```

**HTTPS:** obtain a free certificate with Let's Encrypt ( `sudo certbot --nginx -d shop.example.com` ), or terminate TLS at a cloud load balancer.

**Common nginx errors:** `502 Bad Gateway` (backend down/wrong port/SELinux blocking `httpd_can_network_connect` ), `504 Gateway Timeout` (backend slow), `403 Forbidden` (file permissions/SELinux context, missing index), `address already in use` (another process on port 80: `ss -tulpn` ).

## 3.27 Troubleshooting scenarios

**Method:** 1) define the symptom exactly · 2) what changed? · 3) check status → logs → resources → network → configuration/permissions · 4) one change at a time · 5) verify and document.

### Server is slow / high CPU

```
uptime                                  # load vs core count (nproc)
top                                     # press P: which process? us vs sy vs wa vs st?
ps -eo pid,user,%cpu,%mem,cmd --sort=-%cpu | head
pidstat 1 5                             # is it constant or spiky?
```

Causes: runaway loop/bug, traffic spike, cron job, crypto-miner (unknown process!), too many workers. Fix: restart/scale/limit ( `systemd` `CPUQuota=` ), investigate the code, block abusive traffic.

### Out of memory / OOM kills

```
free -h; vmstat 1 5                     # available memory, swap activity (si/so)
ps -eo pid,user,%mem,rss,cmd --sort=-%mem | head
dmesg -T | grep -i -E "out of memory|killed process"
```

Causes: memory leak, too many workers/connections, JVM heap too large, cache without limits. Fix: restart the leaking service, set limits ( `MemoryMax=` in the unit), right-size, add swap as a buffer, fix the leak.

### Disk full

```
df -h                                   # which file system?
df -i                                   # inodes full? (millions of tiny files)
sudo du -xh / --max-depth=1 2>/dev/null | sort -rh | head
sudo du -xh /var --max-depth=2 2>/dev/null | sort -rh | head
sudo lsof +L1                           # deleted files still held open (space not freed)
```

Typical culprits: `/var/log` (logs without rotation), journal ( `journalctl --vacuum-size=500M` ), Docker ( `docker system df` , `docker system prune` ), core dumps, old kernels, application caches, backups written locally. If a deleted log is still open, **restart or reload the process** to release the space.

### High I/O wait

`iostat -xz 1` (device `%util`, `await`), `iotop -o` (which process). Causes: heavy logging, database without enough IOPS, swapping, backups during peak hours. Fix: faster disk (gp3 IOPS), move logs, schedule heavy jobs, add RAM.

### Service won't start

```
systemctl status app                      # exit code, last log lines
journalctl -u app -n 100 --no-pager       # full error
sudo -u appuser /opt/app/start.sh         # run it manually as the service user
ss -tulpn | grep :8000                    # port already in use?
namei -l /opt/app/config.yaml             # permissions along the path
```

Common causes: syntax error in config (`nginx -t`, `sshd -t`, `apachectl configtest`), port in use, missing environment variable/secret, wrong file ownership, SELinux denial, dependency (database) unavailable.

### Cannot connect to a service on the server

Server side: is it running? listening on the right **address** (`0.0.0.0` vs `127.0.0.1`) and port (`ss -tulpn`)? local test (`curl localhost:port`)? host firewall (`ufw status`, `firewall-cmd --list-all`)? Then network side: security group/NACL, routing, DNS.

### DNS not resolving

```
getent hosts api.example.com              # as applications see it
dig api.example.com                       # status NOERROR / NXDOMAIN / SERVFAIL, answer, TTL
dig @8.8.8.8 api.example.com              # compare with a public resolver
resolvectl status; cat /etc/resolv.conf   # which resolver is used?
grep api /etc/hosts                       # stale override?
```

### "Too many open files"

`ulimit -n` (shell), `cat /proc/<PID>/limits` (process), `ls /proc/<PID>/fd | wc -l` (current usage). Raise with `LimitNOFILE=` in the systemd unit or `/etc/security/limits.conf` for login sessions; also check for file/socket leaks.

### Time is wrong

`timedatectl` (NTP synchronized?), `chronyc tracking`; TLS errors "certificate not yet valid" often come from clock drift.

## 3.28 Command reference by task

| Task | Commands |
|---|---|
| Where am I / what's here | `pwd`, `ls -lah`, `tree -L 2` |
| Find files | `find`, `locate`, `which`, `type` |
| Read files | `cat`, `less`, `head`, `tail -f`, `wc -l` |
| Search text | `grep -rni`, `awk`, `sed -n`, `cut`, `sort`, `uniq -c` |
| Edit | `nano`, `vim`, `sed -i` |
| Users | `id`, `useradd -m`, `usermod -aG`, `passwd`, `sudo -l`, `visudo` |

| Task | Commands |
|---|---|
| Permissions | `chmod`, `chown`, `umask`, `getfacl/setfacl`, `namei -l` |
| Processes | `ps aux`, `top`/`htop`, `pgrep`, `kill`, `nice`, `lsof -p`, `strace -p` |
| Services | `systemctl status/start/enable/restart/daemon-reload`, `journalctl -u -f` |
| Packages | `apt update/install/remove`, `dnf install/upgrade`, `dpkg -L`, `rpm -qf` |
| Resources | `uptime`, `free -h`, `vmstat 1`, `iostat -xz 1`, `df -h`, `du -sh` |
| Storage | `lsblk -f`, `blkid`, `mkfs`, `mount`, `/etc/fstab`, `pvs/vgs/lvs`, `growpart`, `xfs_growfs/resize2fs` |
| Network | `ip a`, `ip r`, `ss -tulpn`, `ping`, `mtr`, `dig`, `curl -v`, `nc -zv`, `tcpdump` |
| Remote | `ssh`, `ssh-keygen`, `ssh-copy-id`, `scp`, `rsync -avz` |
| Archive | `tar -czvf`, `tar -xzvf -C`, `gzip`, `zip` |
| Schedule | `crontab -e`, `systemctl list-timers`, `at` |
| Logs | `journalctl`, `/var/log/*`, `dmesg -T`, `logrotate -d` |
| Security | `ufw`, `firewall-cmd`, `getenforce`, `ausearch`, `lynis`, `find -perm -4000` |
| System info | `uname -a`, `cat /etc/os-release`, `hostnamectl`, `lscpu`, `lsblk`, `timedatectl` |

## 3.29 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | What is the Linux kernel? | The core of the OS that manages CPU scheduling, memory, devices, file systems and networking; programs use it through system calls |
| 2 | Describe the Linux boot process | Firmware (BIOS/UEFI) → GRUB → kernel + initramfs → systemd (PID 1) → default target → services and login |
| 3 | Difference between a process and a thread? | A process has its own memory space; threads share the memory of their process |
| 4 | What is a zombie process? | A finished process whose parent hasn't read its exit status; fix by restarting/fixing the parent |
| 5 | `kill` vs `kill -9`? | SIGTERM lets the process clean up; SIGKILL kills immediately without cleanup (last resort) |
| 6 | What does load average mean? | Average number of runnable + uninterruptible processes over 1/5/15 minutes; compare to core count |
| 7 | How do you find which process uses a port? | `ss -tulpn \| grep :8080` or `lsof -i :8080` |
| 8 | Explain `chmod 755` and `644` | 755: owner rwx, group/others r-x (scripts/dirs). 644: owner rw, others read (files) |
| 9 | What are SUID, SGID and sticky bit? | SUID runs as file owner; SGID runs as group / dirs pass group to new files; sticky lets only owners delete files in a shared dir (`/tmp`) |
| 10 | Hard link vs soft link? | Hard link = another name for the same inode (same FS, no dirs); symlink = pointer to a path, can break, crosses file systems |
| 11 | What is an inode? Can a disk be full with free space? | Metadata record per file; yes, when inodes run out (`df -i`) |

| # | Question | Answer |
|---|---|---|
| 12 | Disk is full: what do you do? | `df -h` → `du` to find big dirs → logs/journal/docker/old files → check deleted-open files ( `lsof +L1` ) → add rotation/monitoring |
| 13 | `df` shows full but `du` shows less. Why? | Deleted files still held open by a process; restart/reload it |
| 14 | How do you make a mount permanent? | Add it to `/etc/fstab` with the UUID, then test with `mount -a` |
| 15 | What is LVM and why use it? | Abstraction over disks (PV → VG → LV) to resize/extend volumes and add disks without downtime |
| 16 | What does `free -h` "available" mean? | Memory that can be given to applications, including reclaimable cache |
| 17 | What is the OOM killer? | Kernel mechanism that kills a process when memory is exhausted; seen in `dmesg` /journal |
| 18 | `systemctl enable` vs `start` ? | `start` runs it now; `enable` starts it at boot ( `enable --now` does both) |
| 19 | How do you create a custom service? | Unit file in `/etc/systemd/system/` with [Unit]/[Service]/[Install], `daemon-reload` , `enable --now` |
| 20 | Where do you look when a service fails? | `systemctl status` , `journalctl -u name` , app logs, config test commands |
| 21 | What does `2>&1` do? | Redirects stderr to wherever stdout goes |
| 22 | Difference between `>` and `>>` ? | Overwrite vs append |
| 23 | How do you schedule a job every 5 minutes? | Cron `*/5 * * * * /path/script.sh >> /var/log/x.log 2>&1` or a systemd timer |
| 24 | Why do cron jobs work manually but fail in cron? | Different environment: PATH, working directory, no TTY, permissions; use full paths and log output |
| 25 | `su` vs `sudo` ? | `su` switches user with the target's password; `sudo` runs commands with your password, per-command rights and auditing |
| 26 | How do you give a user admin rights? | Add to `sudo` / `wheel` group or a sudoers.d rule (via `visudo` ) |
| 27 | How do you harden SSH? | Keys only, no root login, restricted IPs/groups, fail2ban, non-default exposure via VPN/SSM, keep updated |
| 28 | SSH says permission denied (publickey) | Wrong user/key, key not in authorized_keys, wrong permissions on ~/.ssh files |
| 29 | How do you check open ports and connections? | `ss -tulpn` , `ss -tan` , `lsof -i` |
| 30 | How do you troubleshoot DNS on Linux? | `getent hosts` , `dig` (and `dig @8.8.8.8` ), `/etc/resolv.conf` , `resolvectl status` , `/etc/hosts` |
| 31 | What is `/proc` ? | Virtual file system exposing kernel and process information (e.g. `/proc/cpuinfo` , `/proc/<pid>/status` ) |
| 32 | What is SELinux and what do you do if it blocks an app? | Mandatory access control; check `ausearch` / `audit2why` , fix contexts ( `restorecon` / `semanage` ) or booleans, don't disable it |
| 33 | How do you find files larger than 1 GB? | `find / -type f -size +1G 2>/dev/null` |

| # | Question | Answer |
|---|---|---|
| 34 | How do you see the last 100 lines of a log and follow it? | `tail -n 100 -f file` or `journalctl -u name -n 100 -f` |
| 35 | Explain `ps aux` columns RSS and VSZ | RSS = physical memory in use; VSZ = virtual memory reserved |
| 36 | How do you check the Linux distribution and kernel version? | `cat /etc/os-release`, `uname -r` |
| 37 | What is swap and is using it bad? | Disk used as overflow memory; light use is fine, heavy swapping means not enough RAM |
| 38 | How do you change the hostname permanently? | `hostnamectl set-hostname name` (and update `/etc/hosts` if needed) |
| 39 | What is `umask 022`? | Removes write for group/others: new files 644, new directories 755 |
| 40 | A website on the server is down: your steps? | Service status → logs → `ss` listening → local `curl` → firewall → resources (disk/memory) → recent changes → fix and verify |

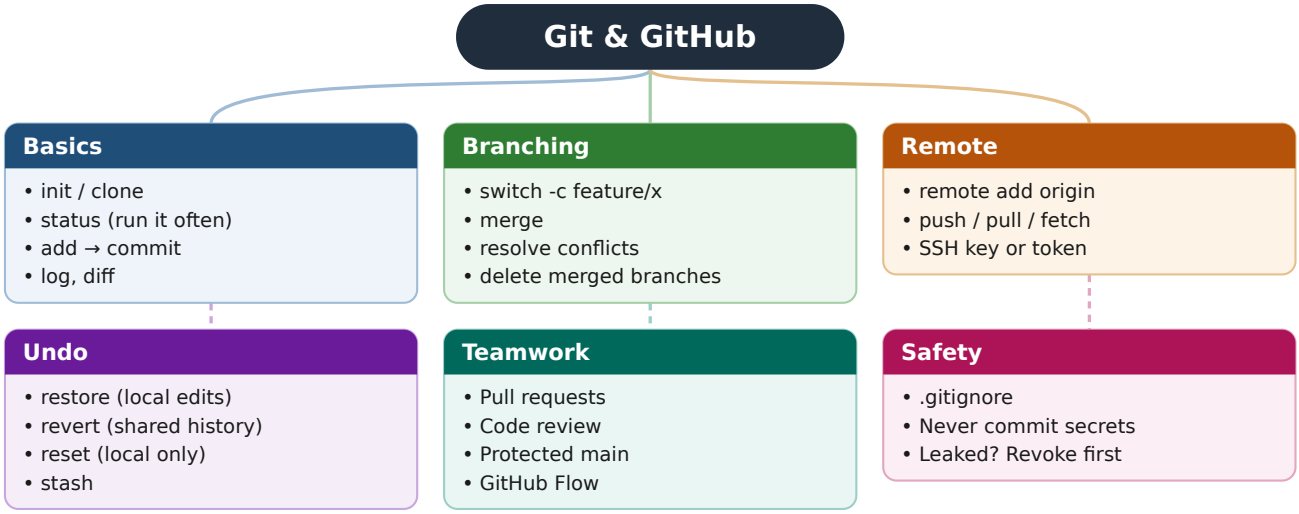
## 3.30 Self-check: I can…

- [ ] Explain kernel vs shell vs distribution and describe the boot process.
- [ ] Navigate the file system confidently and explain what `/etc`, `/var`, `/proc`, `/home`, `/opt` contain.
- [ ] Create, copy, move, link, find and safely delete files; use wildcards and quoting correctly.
- [ ] Search and analyze logs with `grep`, `awk`, `sed`, `sort`, `uniq`, `cut` and pipes.
- [ ] Edit files in vim and nano.
- [ ] Use redirection (`>`, `>>`, `2>&1`), pipes and exit codes.
- [ ] Create users and groups, configure sudo safely, and explain `/etc/passwd` and `/etc/shadow`.
- [ ] Set permissions numerically and symbolically; explain umask, SUID/SGID/sticky, ACLs.
- [ ] Inspect and control processes; send the right signals; use jobs and tmux.
- [ ] Diagnose CPU, memory, disk I/O and network usage with standard tools.
- [ ] Install, update, pin and remove packages with apt and dnf; add a signed repository.
- [ ] Write a systemd service and timer; read logs with journalctl; configure logrotate.
- [ ] Partition, format, mount (fstab), extend with LVM and grow a cloud disk.
- [ ] Archive, compress and transfer files with tar, rsync and scp.
- [ ] Configure environment variables, PATH, aliases and shell startup files.
- [ ] Configure and troubleshoot networking: IPs, routes, DNS, ports, firewalls.
- [ ] Use SSH keys, config files, jump hosts, tunnels; harden sshd.
- [ ] Schedule jobs with cron and systemd timers.
- [ ] Harden a server and work with SELinux/AppArmor instead of disabling them.
- [ ] Set up nginx as a web server and reverse proxy.
- [ ] Troubleshoot high CPU, OOM, full disk, failed services, connectivity and DNS step by step.

**REMEMBER**

- Permissions: **r = 4, w = 2, x = 1** → 7 = rwx, 6 = rw-, 5 = r-x, 4 = r--. Private keys: **600**.
- Cron fields in order: **minute hour day month weekday** ("**M**y **H**amster **D**oesn't **M**ind **W**aiting").
- Service broken? **status → logs → resources → network → config**.
- Linux has no recycle bin: double-check before `rm -rf`.

# 4. Git & GitHub




## 4.0 Why Git matters

Every piece of modern infrastructure and software lives in Git: application code, Dockerfiles, Terraform, Kubernetes manifests, CI pipelines, documentation. GitOps tools deploy **from** Git. Code review, audit trails and rollbacks all depend on it. You will use Git every working day.

## 4.1 Version control concepts

**Version control** records changes to files over time so you can see history, compare versions, undo mistakes, and work in parallel with others.

| Model | How | Examples |
|---|---|---|
| Centralized | One central server holds the history; clients check out files | SVN, Perforce |
| **Distributed** | Every clone has the **full history**; work offline; sync with remotes | **Git**, Mercurial |

**Git** = the version control tool (created by Linus Torvalds in 2005 for the Linux kernel). **GitHub / GitLab / Bitbucket / Azure Repos** = hosting platforms for Git repositories plus collaboration features: pull requests, code review, issues, CI/CD (GitHub Actions, GitLab CI), security scanning, permissions.

## 4.2 Installing and configuring Git

```
git --version
git config --global user.name "Thirupathi Keloth"
git config --global user.email "you@example.com"          # use your GitHub (or noreply) email
git config --global init.defaultBranch main
git config --global core.editor "code --wait"              # or vim / nano
git config --global pull.rebase false                      # pull = fetch + merge (team convention)
git config --global core.autocrlf input                    # Linux/WSL/macOS; on Windows use: true
git config --list --show-origin                            # where each setting comes from
```

| Level | File | Scope |
|---|---|---|
| `--system` | `/etc/gitconfig` | All users on the machine |
| `--global` | `~/.gitconfig` | Your user |
| `--local` (default) | `.git/config` | One repository (overrides global) |

**Useful aliases:**

```
git config --global alias.st status
git config --global alias.lg "log --oneline --graph --decorate --all"
```

## 4.3 How Git works internally

Git stores **snapshots**, not differences. Each commit records the complete state of the project (unchanged files are reused, so it's efficient).

### Objects

| Object | Contains |
|---|---|
| **blob** | File content |
| **tree** | A directory listing: names → blobs/trees |
| **commit** | Pointer to a root tree + parent commit(s) + author, committer, date, message |
| **tag** (annotated) | Name + message + pointer to a commit |

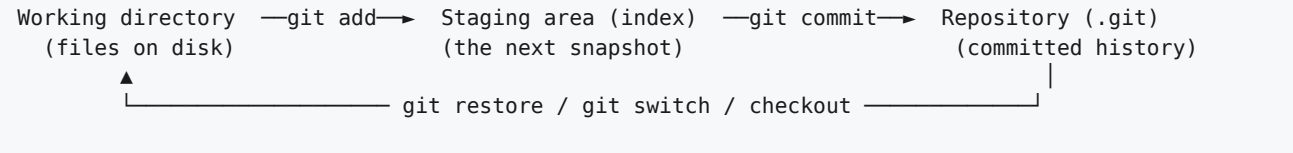
Every object is identified by a **SHA-1 hash** of its content (e.g. `3f9a1c2…`); newer Git also supports SHA-256. Change one byte → new hash. That's why history can't be altered silently.

```
commit C3 ──parent──► commit C2 ──parent──► commit C1
   │ tree               │ tree               │ tree
   ▼                    ▼                    ▼
 snapshot             snapshot             snapshot
main ──► C3      HEAD ──► main   (HEAD = where you are now)
```

### References

| Ref | Meaning |
|---|---|
| Branch ( `main` , `feature/login` ) | A movable **pointer to a commit**; moves forward when you commit |
| `HEAD` | Points to the current branch (or directly to a commit = **detached HEAD**) |
| Tag ( `v1.2.0` ) | Fixed pointer to a commit (releases) |
| Remote-tracking branch ( `origin/main` ) | Your copy of where `main` was on the remote at the last fetch |

### The three areas and file states




The `.git/` directory **is** the repository: objects, refs, config, hooks. Delete it and the folder is no longer a repo.

## 4.4 Everyday commands

| Command | Purpose |
|---|---|
| `git init` | Create a new repository in the current folder |
| `git clone <url> [dir]` | Copy a remote repository (full history) |
| `git status` / `git status -sb` | What changed, what's staged, current branch (short form) |
| `git add file` / `git add .` / `git add -p` | Stage a file / everything / **interactively choose hunks** |
| `git commit -m "Add health endpoint"` | Commit staged changes |
| `git commit -am "msg"` | Stage tracked modified files and commit (not new files) |
| `git commit --amend` | Change the last commit (message or content). Only if not pushed |
| `git rm file` / `git mv old new` | Delete / rename and stage it |
| `git restore file` | Discard working-directory changes |
| `git restore --staged file` | Unstage (keep changes in the working directory) |

### Viewing history

| Command | Shows |
|---|---|
| `git log` | Full history |
| `git log --oneline --graph --decorate --all` | Compact branch graph |
| `git log -p file` | History with patches for one file |
| `git log --stat` | Files changed per commit |
| `git log --author="Thiru" --since="2 weeks ago"` | Filter by author/date |
| `git log -S "password"` | Commits that added/removed a string (pickaxe) |
| `git log main..feature` | Commits in `feature` not in `main` |
| `git show <commit>` | One commit's details and diff |
| `git show <commit>:path/file` | A file as it was in that commit |

### Comparing

| Command | Compares |
|---|---|
| `git diff` | Working directory vs staging |
| `git diff --staged` | Staging vs last commit (what will be committed) |
| `git diff HEAD` | Working directory vs last commit |
| `git diff main...feature` | What the feature branch changed since it split from main |
| `git diff --name-only` | Just file names |

## 4.5 .gitignore

Tells Git which **untracked** files to ignore.

```
# OS / editor
.DS_Store
.vscode/
.idea/

# Secrets and local config - never commit
.env
*.pem
*.key
credentials*

# Build outputs and dependencies
node_modules/
__pycache__/
*.pyc
dist/
build/
.venv/

# Terraform
.terraform/
*.tfstate
*.tfstate.*
*.tfvars          # if they contain environment-specific or secret values
crash.log

# Logs
*.log
!keep-this.log    # ! = exception
```

| Pattern | Matches |
|---|---|
| `*.log` | Any `.log` file in any folder |
| `/build` | Only `build` at the repo root |
| `logs/` | Any directory named logs |
| `**/temp` | `temp` at any depth |
| `!file` | Do not ignore this (exception) |

- Templates: github.com/github/gitignore.
- Global ignore for your machine: `git config --global core.excludesFile ~/.gitignore_global`.
- `git check-ignore -v file` explains which rule ignores a file.

- **Already committed a file?** `.gitignore` won't affect it. Stop tracking: `git rm --cached file` then commit (the file stays on disk). If it was a secret: see 4.17.

## 4.6 Branches

A branch is a lightweight pointer to a commit; creating one is instant. Use a branch for every change.

| Command | Purpose |
|---|---|
| `git branch` / `git branch -a` / `-vv` | List local / all incl. remote / with upstream and last commit |
| `git switch -c feature/login` | Create and switch (older: `git checkout -b`) |
| `git switch main` / `git switch -` | Switch / back to previous branch |
| `git branch -d feature/login` | Delete a merged branch (`-D` force) |
| `git branch -m old new` | Rename |
| `git switch --detach <commit>` | Look at an old commit (detached HEAD) |

**Detached HEAD:** HEAD points to a commit, not a branch. Commits made here are easy to lose; to keep them: `git switch -c rescue-branch`.

**Branch naming conventions:** `feature/JIRA-123-login-page`, `fix/null-pointer-cart`, `hotfix/1.4.1`, `chore/update-deps`, `docs/readme-install`.

## 4.7 Merging

### Fast-forward vs three-way merge

```
FAST-FORWARD (main has no new commits):          THREE-WAY MERGE (both moved on):
main:    A──B                                     main:    A──B──C─────M   (M = merge commit,
feature:      └──D──E                                            └──D──E──┘     two parents)
after:   A──B──D──E   (main just moves forward)
```

| Command | Effect |
|---|---|
| `git switch main && git merge feature/login` | Merge feature into main |
| `git merge --no-ff feature/login` | Always create a merge commit (keeps the branch visible in history) |
| `git merge --squash feature/login` + `git commit` | Combine all feature commits into one new commit |
| `git merge --abort` | Cancel a merge with conflicts |

### Resolving conflicts

A conflict happens when both sides changed the **same lines** (or one deleted a file the other changed).

```
<<<<<<< HEAD
timeout: 30
=======
timeout: 60
>>>>>>> feature/longer-timeout
```

1. `git status` lists conflicted files ("both modified").
2. Edit each file: keep the correct content, **remove all markers**.
3. `git add <file>` for each resolved file.
4. `git commit` (merge) or `git rebase --continue` (rebase).
5. Run tests before pushing.

Helpers: VS Code's merge editor, `git mergetool`, `git checkout --ours/--theirs file` (take one side entirely), `git log --merge`. **Prevent conflicts:** small PRs, merge `main` into your branch often, agree on formatting tools (auto-formatters reduce noise).

## 4.8 Rebasing

**Rebase** replays your commits on top of another base, producing a **linear history**.

```
before:  main:    A──B──C              after `git rebase main` on feature:
         feature:    └──D──E           main:    A──B──C
                                       feature:          └──D'──E'   (new commits, new hashes)
```

| Command | Purpose |
|---|---|
| `git rebase main` | Move your branch onto the latest main |
| `git pull --rebase` | Update your branch without merge commits |
| `git rebase -i HEAD~4` | **Interactive**: clean up the last 4 commits before opening a PR |
| `git rebase --continue / --abort / --skip` | During conflicts |

**Interactive rebase actions:** `pick` (keep), `reword` (change message), `squash` (combine with previous, keep messages), `fixup` (combine, discard message), `drop` (remove), `edit` (stop to amend). Tip: `git commit --fixup <sha>` + `git rebase -i --autosquash`.

**Golden rule: never rebase commits that others already use** (shared branches like `main`). Rebasing rewrites hashes; pushing it requires a force-push and breaks collaborators' history. On your own feature branch it's fine; push with `--force-with-lease`.

| | Merge | Rebase |
|---|---|---|
| History | True history, merge commits | Linear, cleaner |
| Rewrites commits | No | Yes (new hashes) |
| Safe on shared branches | Yes | No |
| Conflict resolution | Once | Possibly per replayed commit |

## 4.9 Undoing changes and recovering work

| Situation | Command |
|---|---|
| Discard uncommitted edits to a file | `git restore file` |
| Unstage a file | `git restore --staged file` |
| Fix the last commit (not pushed) | `git commit --amend` |

| Situation | Command |
|---|---|
| Undo a commit that is **already pushed / shared** | `git revert <sha>` (creates a new commit that reverses it) |
| Undo a merge commit that is pushed | `git revert -m 1 <merge-sha>` |
| Move the branch back, keep changes staged | `git reset --soft HEAD~1` |
| Move back, keep changes unstaged (default) | `git reset HEAD~1` ( `--mixed` ) |
| Move back and **throw away** changes | `git reset --hard HEAD~1` (dangerous) |
| Copy one commit to the current branch | `git cherry-pick <sha>` |
| Find "lost" commits | `git reflog` |

## reset modes

```
                     branch pointer   staging area   working directory
git reset --soft X   moves to X       unchanged      unchanged      → changes appear staged
git reset --mixed X  moves to X       reset to X     unchanged      → changes appear unstaged
git reset --hard X   moves to X       reset to X     reset to X     → changes DELETED
```

## reflog: the safety net

`git reflog` records every position HEAD has had (local only, kept ~90 days). After a bad reset or deleted branch:

```
git reflog                       # find the commit, e.g. a1b2c3d HEAD@{3}: commit: Add cart
git switch -c recovered a1b2c3d  # or: git reset --hard a1b2c3d
```

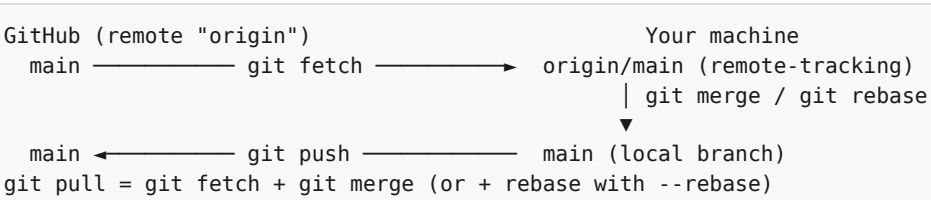
## Stash: put work aside temporarily

| Command | Purpose |
|---|---|
| `git stash push -m "wip login"` | Save uncommitted changes and clean the working directory |
| `git stash -u` | Include untracked files |
| `git stash list` | List stashes |
| `git stash pop` / `git stash apply stash@{1}` | Restore and drop / restore and keep |
| `git stash drop stash@{0}` | Delete a stash |

## Common "oops" recipes

| Oops | Fix |
|---|---|
| Committed to `main` instead of a branch (not pushed) | `git switch -c feature/x` (keeps the commit) → `git switch main` → `git reset --hard origin/main` |
| Wrong commit message (not pushed) | `git commit --amend -m "Better message"` |
| Forgot a file in the last commit | `git add file && git commit --amend --no-edit` |
| Deleted a branch | `git reflog` → `git switch -c name <sha>` |
| Need a fix from another branch | `git cherry-pick <sha>` |
| Pushed a bad commit to `main` | `git revert <sha>` → push (never rewrite shared history) |

## 4.10 Remotes: fetch, pull, push




| Command | Purpose |
|---|---|
| `git remote -v` | List remotes |
| `git remote add origin git@github.com:<user>/<repo>.git` | Connect a local repo |
| `git remote add upstream <url>` | Original repo when you work on a fork |
| `git fetch` / `git fetch --all --prune` | Download changes (doesn't touch your branches) / remove deleted remote branches |
| `git pull` | Fetch + integrate into the current branch |
| `git push -u origin feature/login` | Push and set upstream tracking (first push) |
| `git push` | Push current branch to its upstream |
| `git push --force-with-lease` | Force-push **only if** nobody else pushed meanwhile (after rebasing your own branch) |
| `git push origin --delete feature/login` | Delete a remote branch |
| `git branch -u origin/main` | Set upstream for the current branch |

**Never use plain `--force` on shared branches.** Branch protection on GitHub blocks it for `main`.

### HTTPS vs SSH authentication

| | HTTPS | SSH |
|---|---|---|
| URL | `https://github.com/user/repo.git` | `git@github.com:user/repo.git` |
| Auth | Personal access token (fine-grained, expiring) via credential manager; passwords not accepted | SSH key added to your GitHub account |
| Test | `git ls-remote <url>` | `ssh -T git@github.com` |

**SSH setup:** `ssh-keygen -t ed25519 -C "you@example.com"` → copy `~/.ssh/id_ed25519.pub` → GitHub → Settings → SSH and GPG keys → New SSH key → `ssh -T git@github.com` → "Hi ! You've successfully authenticated".

### Fork workflow (open source / restricted repos)

1. Fork the repo on GitHub → 2. clone **your fork** → 3. `git remote add upstream <original-url>` → 4. branch, commit, push to your fork → 5. open a PR to the original → 6. keep in sync: `git fetch upstream && git rebase upstream/main`.

## 4.11 Tags and releases

| Command | Purpose |
|---|---|
| `git tag` / `git tag -l "v1.*"` | List tags |
| `git tag v1.2.0` | Lightweight tag (just a pointer) |
| `git tag -a v1.2.0 -m "Release 1.2.0"` | **Annotated** tag (author, date, message): use for releases |
| `git push origin v1.2.0` / `git push --tags` | Push tags (not pushed by default) |
| `git tag -d v1.2.0` + `git push origin --delete v1.2.0` | Delete locally and remotely |
| `git describe --tags` | Nearest tag + commits since (useful build versions) |

**Semantic versioning:** `MAJOR.MINOR.PATCH`: increment MAJOR for breaking changes, MINOR for backward-compatible features, PATCH for fixes. Pre-releases: `2.0.0-rc.1`. Tags often trigger release pipelines (build image `app:1.2.0`, publish a GitHub Release with notes).

## 4.12 GitHub in depth

### Account security

- Enable **two-factor authentication** (authenticator app or passkey/security key).
- Use SSH keys or **fine-grained personal access tokens** with expiry and minimal scopes; never classic tokens with full access.
- Review authorized OAuth apps and SSH keys periodically.
- Commit email privacy: use the `@users.noreply.github.com` address if you don't want your email public.

### Repository essentials

| File / feature | Purpose |
|---|---|
| `README.md` | What the project is, how to install/run/test, architecture, screenshots |
| `LICENSE` | Usage rights (MIT/Apache-2.0 for open source) |
| `.gitignore` | Ignored files |
| `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md` | How to contribute |
| `.github/CODEOWNERS` | Who must review which paths |
| `.github/pull_request_template.md`, issue templates | Consistent PRs and issues |
| `.github/workflows/*.yml` | GitHub Actions pipelines |
| `.github/dependabot.yml` | Automated dependency update PRs |
| Visibility | Public / private / internal (organizations) |
| Topics, description, website | Discoverability (portfolio!) |

## Pull requests (PRs)

```
branch → push → open PR (description, linked issue) → CI checks run → review (comments, suggestions)
      → author pushes fixes → approvals → all checks green → merge → delete branch → deploy
```

| Feature | Detail |
|---|---|
| Draft PR | Work in progress; not ready for review |
| Reviewers / CODEOWNERS | Requested automatically for owned paths |
| Review types | Comment, Approve, Request changes; **suggested changes** can be applied with one click |
| Checks | CI status (tests, lint, security scans) shown on the PR |
| Linking issues | "Closes #42" in the description closes the issue on merge |

**Merge methods:**

| Method | Result | When |
|---|---|---|
| Create a merge commit | All commits + merge commit | Keep full history |
| **Squash and merge** | One commit on main per PR | Clean history; most common |
| Rebase and merge | Commits replayed linearly | Linear history with individual commits |

## Branch protection and rulesets (for `main`)

Require a pull request before merging · require **N approvals** · require review from **code owners** · dismiss stale approvals on new commits · require **status checks** to pass (CI) · require branches to be up to date · require **signed commits** · require linear history · block force-pushes and deletions · restrict who can push. **Rulesets** apply these across many repos/branches in an organization.

## Organizations and permissions

Organizations hold repos and **teams**; permission levels per repo: Read, Triage, Write, Maintain, Admin. Use teams (not individual grants), least privilege, and SSO enforcement in companies.

## Other GitHub features

| Feature | Use |
|---|---|
| Issues, labels, milestones | Track bugs and tasks |
| Projects | Kanban boards / planning |
| Actions | CI/CD (see CI/CD notes) |
| Packages / GHCR | Container and package registry ( `ghcr.io/<owner>/<image>` ) |
| Releases | Versioned downloads and notes from tags |
| Pages | Host static sites from a repo |
| Wiki / Discussions | Documentation / Q&A |
| Security: Dependabot alerts & updates, **secret scanning + push protection**, code scanning (CodeQL) | Find vulnerable dependencies, block leaked secrets, find code vulnerabilities |

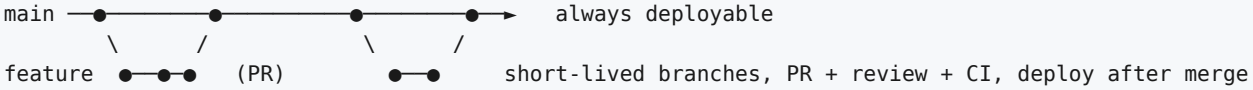
| Feature | Use |
|---|---|
| Codespaces | Cloud development environments |

### GitHub CLI ( `gh` )

```
gh auth login
gh repo create my-project --private --source=. --push
gh pr create --fill --base main
gh pr checks
gh pr merge --squash --delete-branch
gh issue list --label bug
gh run list && gh run watch          # GitHub Actions runs
```

## 4.13 Branching strategies

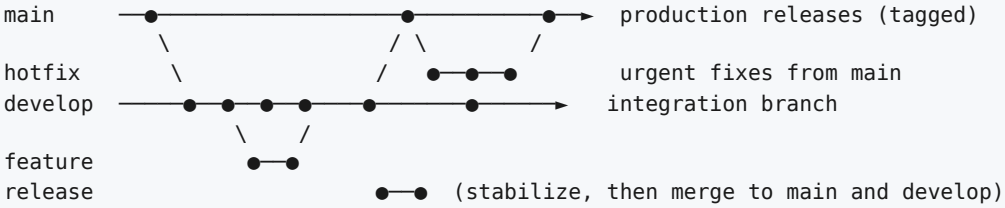
### GitHub Flow (simple, most common)




### Trunk-based development

Everyone integrates into `main` (the trunk) at least daily with very small changes; unfinished features are hidden behind **feature flags**; strong automated tests are required. Used by high-performing teams practicing continuous delivery.

### GitFlow (heavier, release-based)




| Strategy | Pros | Cons | Fits |
|---|---|---|---|
| GitHub Flow | Simple, fast, continuous deployment | Needs good CI and quick reviews | Web apps, most teams |
| Trunk-based | Fastest integration, fewest conflicts | Requires discipline, feature flags, excellent tests | Mature CI/CD teams |
| GitFlow | Structured releases, supports multiple versions | Complex, long-lived branches, more merge conflicts | Versioned products, mobile/desktop releases, regulated releases |
| Release branches ( `release/1.4` ) | Maintain older versions | Backporting work | Libraries, enterprise software |

**Anti-pattern:** long-lived environment branches ( `dev` , `qa` , `prod` ) for application code: they drift apart. Promote the **same artifact** (image tag) through environments instead. (GitOps config repos may use folders per environment.)

## 4.14 Commits, PRs and code review practices

### Good commits

- **Small and focused:** one logical change per commit.
- **Message:** imperative summary ≤ ~50 characters, blank line, then *why* (body wrapped ~72 chars).

```
Add retry with backoff to payment client

The payment provider returns 503 during deploys. Retry up to 3 times
with exponential backoff so checkout doesn't fail for users.

Closes #142
```

### Conventional Commits

`type(scope): description` → `feat(api): add order history endpoint`, `fix(auth): handle expired tokens`, `docs: update install steps`, `chore(deps): bump requests to 2.32.3`, `refactor`, `test`, `ci`, `perf`, `build`. `feat!:` or a `BREAKING CHANGE:` footer marks a major change. Tools can generate changelogs and version numbers from them (semantic-release, release-please).

### Pull request best practices

- Keep PRs small (a few hundred lines) and single-purpose.
- Clear description: what, why, how tested, screenshots, risk/rollback notes, linked issue.
- CI must be green before review.
- Respond to every review comment; resolve conversations.
- **Reviewers:** check correctness, tests, security (secrets, injection, permissions), readability, operability (logs, metrics, config); be specific and kind; approve when it's good enough, not perfect.

## 4.15 Repository structure choices

| Approach | Description | Trade-offs |
|---|---|---|
| Polyrepo | One repo per service/component | Clear ownership, independent pipelines; harder cross-repo changes |
| Monorepo | Many projects in one repo | Atomic cross-project changes, shared tooling; needs path-based CI and good tooling |
| App repo + config repo | Code and its deployment config (Helm values, Kustomize) separated | Common with GitOps (Argo CD watches the config repo) |

**Submodules** embed another repo at a fixed commit (`git submodule add <url> path`, clone with `--recurse-submodules`): powerful but often confusing. **Git LFS** stores large binary files outside normal history (`git lfs track "*.zip"`).

## 4.16 Hooks, pre-commit and commit signing

### Hooks

Scripts in `.git/hooks/` that run on events (`pre-commit`, `commit-msg`, `pre-push`). They're local and not shared by default, so teams use the **pre-commit framework**:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.4
    hooks:
      - id: gitleaks
```

`pip install pre-commit && pre-commit install` → checks run before every commit; run on all files with `pre-commit run --all-files`. The same checks should also run in CI (hooks can be skipped with `--no-verify`).

### Signed commits

Signing proves a commit really came from you ("Verified" badge on GitHub). Simplest with an SSH key:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
# then add the same public key on GitHub as a "Signing key"
git log --show-signature -1
```

## 4.17 Secrets and Git security

**Never commit:** passwords, API keys, tokens, private keys, `.env` files, kubeconfigs, Terraform state, cloud credentials.

**Prevention:** `.gitignore` · pre-commit **gitleaks**/trufflehog · GitHub **secret scanning push protection** · secrets in a manager (Vault, AWS Secrets Manager, GitHub Actions secrets) · code review.

**If a secret is committed (even if you "delete" it in a later commit, it stays in history):**

1. **Revoke / rotate the secret immediately.** Assume it is compromised. This is the most important step.
2. Remove it from the code; load it from a secret store / environment instead.
3. Clean history if needed: `git filter-repo --path secrets.env --invert-paths` (or BFG Repo-Cleaner), then force-push and tell collaborators to re-clone. Contact GitHub support to purge cached views if the repo is public.
4. Check logs for misuse (e.g. AWS CloudTrail for leaked AWS keys).
5. Add prevention (gitignore + scanning) and write a short incident note.

## 4.18 Advanced and investigative commands

| Command | Use |
|---|---|
| `git bisect start` → `git bisect bad` → `git bisect good v1.2.0` → test → `good` / `bad` … → `git bisect reset` | **Binary search** history to find the commit that introduced a bug (`git bisect run ./test.sh` automates it) |
| `git blame -L 10,30 file` | Who last changed lines 10–30 and in which commit |
| `git grep "TODO"` | Fast search in tracked files |

| Command | Use |
|---|---|
| `git shortlog -sn` | Commits per author |
| `git worktree add ../hotfix main` | Second working directory for another branch (no stash needed) |
| `git clean -fdn` → `-fd` | Preview / delete untracked files and directories |
| `git archive -o release.zip HEAD` | Export a snapshot without `.git` |
| `git gc`, `git count-objects -vH` | Housekeeping / repo size |
| `git log --follow file` | History across renames |
| `git range-diff` | Compare two versions of a rebased branch |

## 4.19 Troubleshooting common Git errors

| Error / situation | Cause | Fix |
|---|---|---|
| `! [rejected] main -> main (non-fast-forward)` / "Updates were rejected because the remote contains work" | Remote has commits you don't have | `git pull --rebase` (or `git pull`), resolve conflicts, push again |
| `Permission denied (publickey)` | SSH key not added to GitHub / not loaded | `ssh -T git@github.com`, add key, `ssh-add`, check `~/.ssh/config` |
| `remote: Support for password authentication was removed` | Using a password over HTTPS | Use a personal access token or SSH |
| `fatal: not a git repository` | Not inside a repo folder | `cd` into the repo or `git init` |
| `You are in 'detached HEAD' state` | Checked out a commit/tag | `git switch -c new-branch` to keep work, or `git switch main` |
| Merge conflicts | Same lines changed | Resolve markers, `git add`, commit / `rebase --continue` |
| `refusing to merge unrelated histories` | Two repos with separate roots (e.g. GitHub repo created with README + local init) | `git pull origin main --allow-unrelated-histories` (once), resolve, commit |
| `error: Your local changes would be overwritten` | Uncommitted changes conflict with switch/pull | Commit or `git stash`, then retry |
| File still tracked after adding to `.gitignore` | It was committed earlier | `git rm --cached file` and commit |
| Scripts fail with `\r` errors after editing on Windows | CRLF line endings | `core.autocrlf` settings, `.gitattributes` with `*.sh text eol=lf`, `dos2unix` |
| Push rejected: file > 100 MB | GitHub file size limit | Remove from history (`filter-repo`), use Git LFS or artifact storage |
| Pushed a secret | See 4.17 | Rotate first |
| Branch protection blocks push to `main` | Working as designed | Open a PR |

## 4.20 Hands-on labs

### Lab 1: local workflow

```
mkdir git-lab && cd git-lab && git init
echo "# Git Lab" > README.md && git add README.md && git commit -m "Add README"
git switch -c feature/greeting
echo 'echo "Hello"' > hello.sh && git add hello.sh && git commit -m "Add greeting script"
git switch main && git merge feature/greeting       # fast-forward
git log --oneline --graph --all
```

### Lab 2: create and resolve a conflict

```
git switch -c change-a && echo "timeout: 30" > app.yaml && git add . && git commit -m "Timeout 30"
git switch main && echo "timeout: 60" > app.yaml && git add . && git commit -m "Timeout 60"
git merge change-a          # CONFLICT → edit app.yaml, keep the right value, remove markers
git add app.yaml && git commit
```

**Lab 3: GitHub PR flow** Create a repo on GitHub → add the remote → push `main` → protect `main` (require PR + 1 approval) → push a feature branch → open a PR → review → squash-merge → pull `main` locally and delete the branch.

**Lab 4: recover from mistakes** — `git reset --hard HEAD~1` a commit, then recover it with `git reflog`; delete a branch and restore it.

**Lab 5: rebase interactively** — make 3 messy commits on a branch, run `git rebase -i HEAD~3`, squash them into one with a good message.

## 4.21 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | Git vs GitHub? | Git is the distributed version control tool; GitHub hosts Git repositories with collaboration features (PRs, Actions, issues, security) |
| 2 | What is a commit? | A snapshot of the project with metadata and parent pointer(s), identified by a hash |
| 3 | Working directory vs staging area vs repository? | Edited files / the next snapshot being prepared / committed history |
| 4 | `git fetch` vs `git pull`? | Fetch downloads without changing your branch; pull = fetch + merge (or rebase) |
| 5 | Merge vs rebase? | Merge preserves history with merge commits; rebase rewrites commits into a linear history; don't rebase shared branches |
| 6 | Fast-forward merge? | The branch pointer simply moves forward because the target has no new commits |
| 7 | How do you resolve a merge conflict? | Edit files to the correct content, remove markers, `git add`, commit/continue, run tests |
| 8 | `reset` vs `revert`? | Reset moves the branch pointer (rewrites local history); revert adds a new commit undoing changes (safe for shared branches) |
| 9 | `reset --soft` vs `--mixed` vs `--hard`? | Keep changes staged / keep unstaged / discard |

| # | Question | Answer |
|---|---|---|
| 10 | How do you recover a deleted branch or lost commit? | `git reflog`, then create a branch at that commit |
| 11 | What is `git stash`? | Temporarily saves uncommitted changes so you can switch context |
| 12 | What is `cherry-pick`? | Applies a specific commit onto the current branch |
| 13 | What is detached HEAD? | HEAD points to a commit instead of a branch; create a branch to keep work |
| 14 | What does `.gitignore` do and why doesn't it remove a tracked file? | Ignores untracked files only; use `git rm --cached` for tracked ones |
| 15 | What is `--force-with-lease`? | A safer force-push that fails if the remote changed since you last fetched |
| 16 | Explain GitHub Flow vs GitFlow vs trunk-based | Short branches + PRs to main / develop+release+hotfix branches / tiny frequent commits to main with feature flags |
| 17 | What is a pull request? | A request to merge a branch, with review, discussion and CI checks |
| 18 | What branch protection rules would you set on main? | Require PR + approvals + code owners, passing status checks, up-to-date branch, signed commits, no force-push/deletion |
| 19 | Squash merge vs merge commit? | One commit per PR (clean history) vs keep all commits plus a merge commit |
| 20 | Annotated vs lightweight tag? | Annotated has author/date/message (releases); lightweight is just a pointer |
| 21 | How do you find which commit introduced a bug? | `git bisect` (binary search), plus `git log -S` / `git blame` |
| 22 | What if someone committed a secret? | Rotate it immediately, remove from code, clean history (filter-repo), add scanning/push protection |
| 23 | What is a Git hook? | Script run on Git events (pre-commit, commit-msg, pre-push); shared via the pre-commit framework |
| 24 | Monorepo vs polyrepo? | One repo for many projects (atomic changes, shared tooling) vs one per service (independence, clear ownership) |
| 25 | How do you keep a fork up to date? | Add `upstream` remote, fetch it, rebase/merge `upstream/main` into your branch |

## 4.22 Self-check: I can…

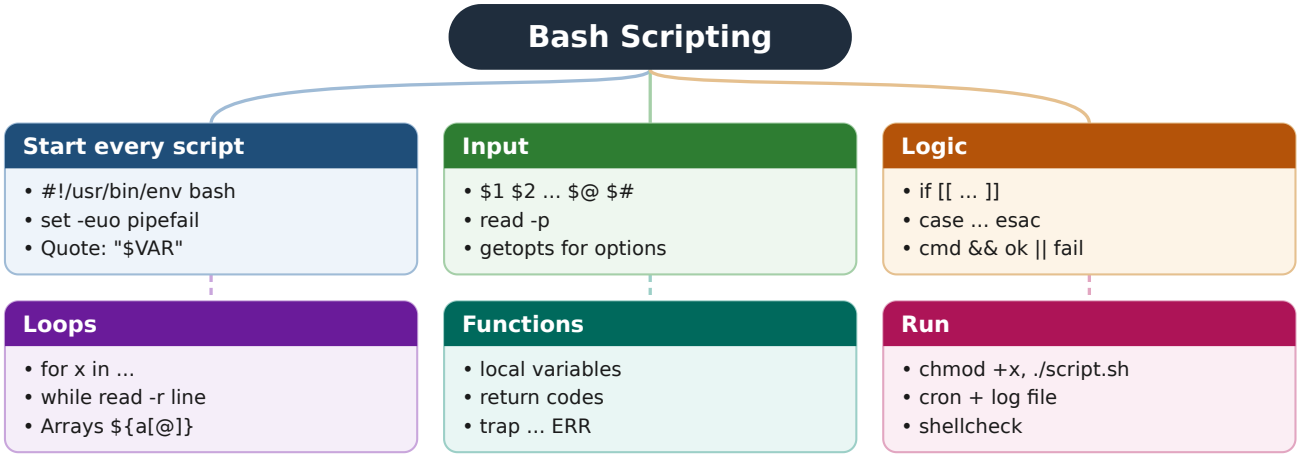
- [ ] Explain distributed version control and how Git stores data (objects, hashes, refs, HEAD).
- [ ] Configure Git (identity, editor, line endings) at the right level.
- [ ] Use the full basic cycle: status, add (incl. `-p`), commit, log, diff, show.
- [ ] Write a good `.gitignore` and untrack accidentally committed files.
- [ ] Create, switch, rename and delete branches; handle detached HEAD.
- [ ] Merge (fast-forward, three-way, squash) and resolve conflicts confidently.
- [ ] Rebase (including interactive) and explain when not to.
- [ ] Undo anything safely: restore, reset modes, revert, reflog, stash, cherry-pick.
- [ ] Work with remotes over SSH/HTTPS, fetch/pull/push, `--force-with-lease`, forks.

- [ ] Tag releases with semantic versions.
- [ ] Use GitHub PRs, reviews, merge methods, CODEOWNERS, branch protection and security features.
- [ ] Choose and explain a branching strategy for a team.
- [ ] Write good commit messages (Conventional Commits) and PR descriptions.
- [ ] Set up pre-commit hooks and signed commits.
- [ ] Respond correctly to a leaked secret.
- [ ] Use bisect, blame, worktree and fix common Git errors.

**REMEMBER**

- The flow: **edit → add → commit → push** (working dir → staging → repo → GitHub).
- Shared branch? Use **revert**, never reset or force-push.
- Small commits, clear messages: "Add health check endpoint".

# 5. Bash Scripting




## 5.0 Why Bash scripting matters

Bash glues the DevOps world together: CI pipeline steps, Docker entrypoints, server bootstrap (EC2 user data), cron jobs, quick automation around `aws`, `kubectl`, `git` and `curl`. You'll read Bash every day and write it often. For larger programs (complex logic, APIs, data processing), switch to Python.

## 5.1 Scripts, shebang and execution

```
#!/usr/bin/env bash
# deploy.sh - short description of what the script does
echo "Hello from $(hostname)"
```

| Topic | Detail |
|---|---|
| Shebang `#!` | First line: which interpreter runs the file. `#!/usr/bin/env bash` finds bash in `PATH` (portable); `#!/bin/bash` is also common |
| Make executable | `chmod +x deploy.sh`, then `./deploy.sh` |
| Run without +x | `bash deploy.sh` |
| `source deploy.sh` / `. deploy.sh` | Runs in the **current** shell (variables and `cd` persist): used for env files, not for normal scripts |
| `sh` vs `bash` | `sh` may be dash/POSIX (no arrays, no `[[ ]]`). If you use Bash features, run with bash |
| Line endings | Scripts edited on Windows may contain `\r` → `bad interpreter: /bin/bash^M`. Fix: `sed -i 's/\r$//' script.sh` |

## 5.2 Variables and parameter expansion

```
name="web01"                # no spaces around =
count=3
readonly APP_DIR="/opt/app"  # constant
echo "$name"                # always quote when using
echo "${name}-backup"       # braces separate the name from following text
unset name
```

| Expansion | Result | Example ( `file=/var/log/app.log.gz` , `v` unset) |
|---|---|---|
| `${v:-default}` | Value, or default if unset/empty | `${ENV:-dev}` → `dev` |
| `${v:=default}` | Same, and assigns it | |
| `${v:?message}` | Exit with error if unset/empty | `${DB_HOST:?DB_HOST is required}` |
| `${v:+alt}` | `alt` if set, else empty | `${DEBUG:+-x}` |
| `${#file}` | Length | `19` |
| `${file:0:4}` | Substring (offset, length) | `/var` |
| `${file#*/}` | Remove shortest prefix match | `var/log/app.log.gz` |
| `${file##*/}` | Remove longest prefix match → basename | `app.log.gz` |
| `${file%.*}` | Remove shortest suffix match | `/var/log/app.log` |
| `${file%%.*}` | Remove longest suffix match | `/var/log/app` |
| `${file%/*}` | Directory part | `/var/log` |
| `${file/log/LOG}` / `${file//log/LOG}` | Replace first / all | `/var/LOG/app.log.gz` / `/var/LOG/app.LOG.gz` |
| `${name^^}` / `${name,,}` | Uppercase / lowercase | `WEB01` |

**Environment vs shell variables:** `export VAR=value` makes it visible to child processes (commands you start). Scripts receive the environment of their caller; configuration for scripts usually comes from environment variables.

## 5.3 Quoting, word splitting and globbing

The shell processes a command line in order: expansions ( `$var` , `$(cmd)` , `$((...))` ) → **word splitting** on spaces → **glob** expansion ( `*` ). Unquoted variables get split and globbed: the source of many bugs.

```
file="my report.txt"
rm $file        # WRONG: tries to delete "my" and "report.txt"
rm "$file"      # RIGHT
```

| Form | Use |
|---|---|
| `"double quotes"` | Default for variables and command substitutions: `"$var"` , `"$(date)"` |
| `'single quotes'` | Literal text ( `'$HOME'` stays `$HOME` ), e.g. awk/jq programs |
| `"$@"` | All arguments, each kept as a separate word: **always use this to pass arguments on** |

| Form | Use |
|---|---|
| `$(command)` | Command substitution (prefer over backticks) |
| `$'...'` | ANSI-C strings: `$'line1\nline2'` |

## 5.4 Input: arguments, read and options

| Variable | Meaning |
|---|---|
| `$0` | Script name |
| `$1` … `$9`, `${10}` | Positional arguments |
| `$#` | Number of arguments |
| `"$@"` | All arguments as separate words |
| `"$*"` | All arguments as one string |
| `$?` | Exit code of the last command |
| `$$` | PID of the script |
| `$!` | PID of the last background job |
| `shift` | Drop `$1`, move others down |

```
read -r -p "Environment (dev/prod): " env       # -r keeps backslashes literal
read -r -s -p "Password: " pass; echo           # -s hides input (prefer env vars/secret stores)
read -r -t 10 -p "Continue? [y/N] " ans || ans="n"   # -t timeout
```

**Options with getopts:**

```
usage() { echo "Usage: $0 -e <env> [-n] [-v]" >&2; exit 2; }

env="" dry_run=false verbose=false
while getopts ":e:nvh" opt; do
  case "$opt" in
    e) env="$OPTARG" ;;
    n) dry_run=true ;;
    v) verbose=true ;;
    h) usage ;;
    \?) echo "Unknown option -$OPTARG" >&2; usage ;;
    :)  echo "Option -$OPTARG needs a value" >&2; usage ;;
  esac
done
shift $((OPTIND - 1))                 # remaining positional arguments in "$@"
[[ -n "$env" ]] || usage
echo "env=$env dry_run=$dry_run verbose=$verbose rest=$*"
```

## 5.5 Arithmetic

```
a=7; b=2
echo $(( a + b )) $(( a - b )) $(( a * b ))     # 9 5 14
echo $(( a / b )) $(( a % b )) $(( a ** b ))     # 3 1 49
(( count++ ))                           # increment
(( a > b )) && echo "a is bigger"       # arithmetic test
echo "scale=2; 7/2" | bc                # decimals need bc/awk → 3.50
awk 'BEGIN { printf "%.1f\n", 7/2 }'  # 3.5
```

Bash arithmetic is **integer only**.

## 5.6 Conditions

```
if [[ -f "$config" && -r "$config" ]]; then
  echo "config readable"
elif [[ "$env" == "prod" ]]; then
  echo "production without config!" >&2
  exit 1
else
  echo "using defaults"
fi
```

| Test | True when |
|---|---|
| `-e path` / `-f file` / `-d dir` / `-L link` | Exists / regular file / directory / symlink |
| `-r` / `-w` / `-x` | Readable / writable / executable |
| `-s file` | Exists and not empty |
| `file1 -nt file2` | file1 newer than file2 |
| `-z "$s"` / `-n "$s"` | String empty / not empty |
| `"$a" == "$b"` / `!=` | String equal / not equal |
| `"$s" == web*` | Pattern match (inside `[[ ]]`, unquoted pattern) |
| `"$s" =~ ^[0-9]+$` | Regex match ( `BASH_REMATCH` holds groups) |
| `$a -eq $b` · `-ne` `-lt` `-le` `-gt` `-ge` | Integer comparisons (or use `(( a < b ))` ) |
| `!` · `&&` · `\|\|` | Not / and / or |

`[[ ... ]]` (Bash) is safer than `[ ... ]` (POSIX `test`): no word splitting, supports `&&`, patterns and regex.

**A command is a condition:** `if grep -q "ERROR" app.log; then ...` (uses the exit code; `-q` = quiet).

### case

```
case "$1" in
  start)          systemctl start app ;;
  stop)           systemctl stop app ;;
  restart|reload) systemctl restart app ;;
  *.tar.gz)       tar -xzf "$1" ;;
  *)              echo "Usage: $0 {start|stop|restart}" >&2; exit 2 ;;
esac
```

## 5.7 Loops

```
for host in web1 web2 db1; do
  ping -c1 -W1 "$host" &>/dev/null && echo "$host up" || echo "$host DOWN"
done

for i in {1..5}; do echo "attempt $i"; done
for (( i=0; i<3; i++ )); do echo "$i"; done

for f in /var/log/*.log; do              # globs are safe with spaces when you quote "$f"
  [[ -e "$f" ]] || continue              # no match → the pattern itself would be returned
  echo "$(wc -l < "$f") lines in $f"
done

while IFS= read -r line; do              # read a file line by line, safely
  echo "user: $line"
done < users.txt

while IFS=, read -r name email team; do  # CSV fields
  echo "$name <$email> -> $team"
done < <(tail -n +2 users.csv)           # skip the header line

until curl -fs http://localhost:8000/health >/dev/null; do   # wait until healthy
  echo "waiting..."; sleep 2
done
```

`break` exits a loop; `continue` skips to the next iteration. **Don't** loop over `$(ls)` or `$(cat file)`: names with spaces break; use globs and `while read`.

## 5.8 Arrays

```
servers=("web1" "web2" "db 1")
servers+=("cache1")                      # append
echo "${servers[0]}"                     # first element
echo "${#servers[@]}"                    # count
for s in "${servers[@]}"; do echo "$s"; done
echo "${servers[@]:1:2}"                 # slice

declare -A port=( [web]=80 [db]=5432 )  # associative array (Bash 4+)
port[redis]=6379
for svc in "${!port[@]}"; do echo "$svc -> ${port[$svc]}"; done

mapfile -t lines < hosts.txt             # read a file into an array
```

## 5.9 Functions

```
log() {
  local level="$1"; shift
  printf '%s [%s] %s\n' "$(date '+%F %T')" "$level" "$*" >&2
}

is_port_open() {                        # returns status: 0 = yes, 1 = no
  local host="$1" port="$2"
  timeout 3 bash -c ">/dev/tcp/$host/$port" 2>/dev/null
}

get_free_disk_pct() {                   # "returns" data by printing it
  df --output=pcent / | tail -1 | tr -dc '0-9'
}

if is_port_open db.internal 5432; then log INFO "db reachable"; fi
used=$(get_free_disk_pct)
log INFO "root disk used: ${used}%"
```

- `local` keeps variables inside the function (otherwise variables are global).
- `return N` sets a status code (0–255); to return data, **print it** and capture with `$(...)`.
- Functions receive their own `$1`, `$@`, `$#`.

## 5.10 Exit codes and error handling

### Strict mode

```
set -euo pipefail
IFS=$'\n\t'
```

| Option | Effect | Watch out |
|---|---|---|
| `-e` | Exit when a command fails | Doesn't trigger inside `if`, `&&`, `\|\|` conditions; use `cmd \|\| true` when failure is acceptable |
| `-u` | Error on unset variables | Use `${VAR:-}` for optional variables |
| `-o pipefail` | A pipeline fails if any part fails | `grep` with no match returns 1: handle it |
| `IFS=$'\n\t'` | Split only on newlines/tabs | Safer loops over command output |

### trap

```
tmpdir=$(mktemp -d)                                  # safe temporary directory
cleanup() { rm -rf "$tmpdir"; }
trap cleanup EXIT                                     # always runs at exit (success or failure)
trap 'echo "Error on line $LINENO (exit $?)" >&2' ERR  # report failures
trap 'echo "Interrupted"; exit 130' INT TERM          # Ctrl+C / kill
```

### Exit code conventions

`0` success · `1` general error · `2` misuse/usage error · `126` not executable · `127` command not found · `128+N` killed by signal N (130 = Ctrl+C, 137 = SIGKILL/OOM, 143 = SIGTERM). Always `exit` with a meaningful code so CI/cron/systemd can detect failures.

## 5.11 Output, logging and here-documents

```
echo "message"                              # simple output
printf '%-10s %5d %s\n' "web1" 80 "ok"      # formatted output (preferred for exact formats)
echo "error" >&2                            # to stderr
exec > >(tee -a /var/log/myscript.log) 2>&1 # send all further output to screen + log file

cat > /etc/myapp/app.conf <<EOF             # here-document (variables expand)
port=${PORT:-8080}
env=${ENV:-dev}
EOF

cat <<'EOF' > script.txt                    # quoted delimiter: no expansion
literal $HOME and $(date)
EOF
```

Use `logger -t myscript "message"` to write to the system journal/syslog.

## 5.12 Working with text, JSON and APIs in scripts

```
# Text: grep / awk / sed / cut (see Linux notes)
errors=$(grep -c "ERROR" app.log || true)

# JSON with jq
curl -fsS https://api.github.com/repos/kubernetes/kubernetes | jq -r '.stargazers_count'
aws ec2 describe-instances --output json \
  | jq -r '.Reservations[].Instances[] | [.InstanceId, .State.Name] | @tsv'
kubectl get pods -o json | jq -r '.items[] | select(.status.phase != "Running") | .metadata.name'

# HTTP status checks
code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "$url")
[[ "$code" == "200" ]] || { echo "unhealthy: $code" >&2; exit 1; }

# YAML with yq
yq '.spec.replicas' deployment.yaml
```

`curl -f` makes curl return a failure exit code on HTTP errors (≥400); add `--retry 3` for flaky networks.

## 5.13 Debugging

| Tool | Use |
|---|---|
| `bash -n script.sh` | Syntax check without running |
| `bash -x script.sh` / `set -x` … `set +x` | Trace every command as it runs |
| `PS4='+ ${BASH_SOURCE}:${LINENO}: '` | Show file and line in traces |
| **shellcheck** | Static analysis: catches quoting bugs, unused variables, common mistakes ( `shellcheck script.sh` ; run it in CI) |
| `echo "DEBUG: var=$var" >&2` | Quick prints to stderr |

## 5.14 Production script template

```
#!/usr/bin/env bash
# backup-dir.sh - archive a directory with retention
# Usage: backup-dir.sh -s <source_dir> -d <dest_dir> [-k <keep_days>] [-n]
set -euo pipefail

readonly SCRIPT_NAME="${0##*/}"
KEEP_DAYS=7
DRY_RUN=false

log()  { printf '%s [%s] %s\n' "$(date '+%F %T')" "$1" "${*:2}" >&2; }
die()  { log ERROR "$*"; exit 1; }
usage() { echo "Usage: $SCRIPT_NAME -s <src> -d <dest> [-k days] [-n]" >&2; exit 2; }
run()  { if $DRY_RUN; then log DRY "$*"; else "$@"; fi; }

while getopts ":s:d:k:nh" opt; do
  case "$opt" in
    s) SRC="$OPTARG" ;;
    d) DEST="$OPTARG" ;;
    k) KEEP_DAYS="$OPTARG" ;;
    n) DRY_RUN=true ;;
    *) usage ;;
  esac
done
[[ -n "${SRC:-}" && -n "${DEST:-}" ]] || usage
[[ -d "$SRC" ]] || die "source $SRC does not exist"
[[ "$KEEP_DAYS" =~ ^[0-9]+$ ]] || die "keep days must be a number"

exec 9>"/tmp/${SCRIPT_NAME}.lock"           # prevent two runs at the same time
flock -n 9 || die "another instance is running"

run mkdir -p "$DEST"
archive="$DEST/$(basename "$SRC")-$(date +%Y%m%d-%H%M%S).tar.gz"
log INFO "creating $archive"
run tar -czf "$archive" -C "$(dirname "$SRC")" "$(basename "$SRC")"

log INFO "removing archives older than $KEEP_DAYS days"
run find "$DEST" -name "*.tar.gz" -type f -mtime +"$KEEP_DAYS" -delete
log INFO "done"
```

**Checklist for real scripts:** usage/help · input validation · strict mode · clear logging to stderr · meaningful exit codes · dry-run option for destructive actions · lock to avoid parallel runs · **idempotent** (safe to run twice) · no hard-coded secrets (read from environment/secret manager) · `shellcheck` clean · stored in Git.

## 5.15 Real-world scripts

### Disk usage alert with a webhook notification

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD="${THRESHOLD:-80}"
: "${ALERT_WEBHOOK_URL:?set ALERT_WEBHOOK_URL in the environment}"   # secret from env, not code

df --output=pcent,target -x tmpfs -x devtmpfs | tail -n +2 | while read -r pct mount; do
  used="${pct%\%}"
  if (( used >= THRESHOLD )); then
    msg="$(hostname): $mount is ${used}% full"
    curl -fsS -X POST -H 'Content-Type: application/json' \
      -d "{\"text\": \"$msg\"}" "$ALERT_WEBHOOK_URL" >/dev/null
    echo "ALERT: $msg"
  fi
done
```

### Service health check with automatic restart

```
#!/usr/bin/env bash
set -euo pipefail
SERVICE="${1:?usage: $0 <service> <url>}"
URL="${2:?usage: $0 <service> <url>}"

if ! curl -fs --max-time 5 "$URL" >/dev/null; then
  logger -t healthcheck "$SERVICE unhealthy, restarting"
  systemctl restart "$SERVICE"
  sleep 5
  if ! curl -fs --max-time 5 "$URL" >/dev/null; then
    logger -t healthcheck "$SERVICE still down"; exit 1
  fi
fi
```

### Bulk user creation from CSV

```
#!/usr/bin/env bash
set -euo pipefail
# users.csv: username,group
while IFS=, read -r user group; do
  [[ -z "$user" || "$user" == "username" ]] && continue
  getent group "$group" >/dev/null || sudo groupadd "$group"
  if id "$user" &>/dev/null; then
    echo "exists: $user"
  else
    sudo useradd -m -s /bin/bash -G "$group" "$user"
    echo "created: $user ($group)"
  fi
done < users.csv
```

### Wait for a dependency (container entrypoints, CI)

```
#!/usr/bin/env bash
set -euo pipefail
host="${1:?host}" port="${2:?port}" timeout="${3:-60}"
for (( i=0; i<timeout; i++ )); do
  if (exec 3<>"/dev/tcp/$host/$port") 2>/dev/null; then echo "$host:$port is up"; exit 0; fi
  sleep 1
done
echo "timeout waiting for $host:$port" >&2; exit 1
```

### AWS automation: list running instances without an Owner tag

```
#!/usr/bin/env bash
set -euo pipefail
REGION="${AWS_REGION:-ap-south-1}"
aws ec2 describe-instances --region "$REGION" \
  --filters Name=instance-state-name,Values=running \
  --query 'Reservations[].Instances[?!not_null(Tags[?Key==`Owner`].Value|[0])].InstanceId' \
  --output text | tr '\t' '\n' | sed '/^$/d'
```

### Simple deployment with rollback

```
#!/usr/bin/env bash
set -euo pipefail
APP_DIR=/opt/shop
NEW_VERSION="${1:?usage: $0 <git-tag>}"
previous=$(git -C "$APP_DIR" describe --tags --abbrev=0)

rollback() {
  echo "Deploy failed, rolling back to $previous" >&2
  git -C "$APP_DIR" checkout -q "$previous"
  sudo systemctl restart shop
}
trap rollback ERR

git -C "$APP_DIR" fetch --tags -q
git -C "$APP_DIR" checkout -q "$NEW_VERSION"
"$APP_DIR/.venv/bin/pip" install -q -r "$APP_DIR/requirements.txt"
sudo systemctl restart shop
sleep 3
curl -fs --max-time 5 http://localhost:8000/health >/dev/null
trap - ERR
echo "Deployed $NEW_VERSION"
```

(In real teams, deployments go through CI/CD pipelines and immutable artifacts; scripts like this are for simple servers and learning.)

## 5.16 Common mistakes

| Mistake | Fix |
|---|---|
| `var = value` (spaces) | `var=value` |
| Unquoted `$var` with spaces/globs | `"$var"` |
| `for f in $(ls)` | `for f in ./*` |
| `cat file \| while read line` loses variables (subshell) | `while read -r line; do ...; done < file` |
| Missing `-r` in `read` | `read -r` |
| `[ $a == $b ]` with empty variables | `[[ "$a" == "$b" ]]` |
| Using `==` for numbers | `-eq` or `(( a == b ))` |
| Ignoring failures | `set -euo pipefail`, check exit codes |
| Hard-coded passwords/tokens | Environment variables / secret managers |
| No logs from cron jobs | Redirect output `>> file 2>&1` |
| CRLF line endings | `dos2unix`, `.gitattributes` |

| Mistake | Fix |
|---|---|
| Relying on the current directory | `cd "$(dirname "$0")"` or absolute paths |

## 5.17 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | What is a shebang? | First line `#!` telling the OS which interpreter runs the script |
| 2 | `source script.sh` vs `./script.sh`? | Source runs in the current shell (changes persist); `./` runs in a new process |
| 3 | `$@` vs `$*`? | `"$@"` keeps each argument separate; `"$*"` joins them into one string |
| 4 | What does `set -euo pipefail` do? | Exit on errors, on unset variables, and on failures inside pipelines |
| 5 | How do you check the exit status of a command? | `$?` or use the command directly in `if` / `&&` / `\|\|` |
| 6 | `[ ]` vs `[[ ]]`? | `[[ ]]` is Bash: safer (no word splitting), supports patterns/regex/ `&&` |
| 7 | How do you read a file line by line? | `while IFS= read -r line; do ...; done < file` |
| 8 | How do you pass and parse options? | Positional args or `getopts` |
| 9 | How do you return a value from a function? | Print it and capture with `$(...)`; `return` only sets the status code |
| 10 | What is `trap` used for? | Run cleanup or handlers on EXIT, ERR, INT, TERM |
| 11 | How do you debug a script? | `bash -x`, `set -x`, `bash -n`, shellcheck |
| 12 | How do you make a script idempotent? | Check state before changing it (`id user`, `mkdir -p`, `grep -q` before appending) |
| 13 | How do you avoid hard-coding secrets? | Read them from environment variables or a secrets manager at runtime |
| 14 | Exit code 127 / 137? | Command not found / killed by SIGKILL (often OOM) |
| 15 | How do you prevent two copies of a cron script running? | `flock` lock file |
| 16 | How do you process JSON in Bash? | `jq` |
| 17 | Why quote variables? | To prevent word splitting and glob expansion |
| 18 | When would you use Python instead of Bash? | Complex logic, data structures, APIs/JSON heavy work, testing, cross-platform tools |

## 5.18 Self-check: I can…

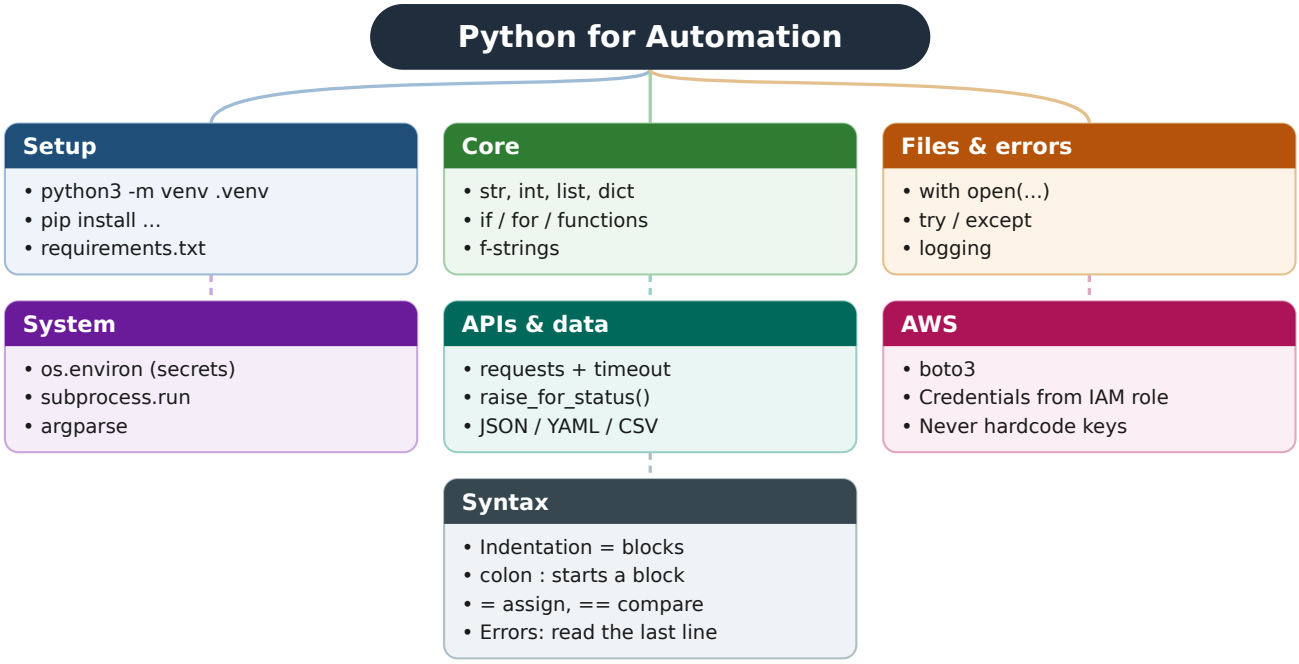
- [ ] Write, make executable and run scripts; explain shebang and `source`.
- [ ] Use variables and parameter expansion (defaults, required, trimming, replacing).
- [ ] Quote correctly and explain word splitting and globbing.
- [ ] Handle arguments, `read`, and options with `getopts`.
- [ ] Do integer arithmetic and know when to use `bc` / `awk`.
- [ ] Write conditions with `[[ ]]`, file/string/number tests, regex and `case`.

- [ ] Write `for`, `while read`, `until` loops safely.
- [ ] Use indexed and associative arrays.
- [ ] Write functions with local variables and proper return codes.
- [ ] Use strict mode, `trap`, `mktemp` and meaningful exit codes.
- [ ] Log properly, use here-documents, process JSON with `jq` and call APIs with `curl`.
- [ ] Debug with `bash -x` and keep scripts shellcheck-clean.
- [ ] Build production-quality scripts (validation, dry-run, locking, idempotency, no secrets).
- [ ] Automate real tasks: alerts, health checks, user management, waiting for services, AWS/kubectl automation.

**REMEMBER**

- Exit code **0 = success**, anything else = failure ( `$?` ).
- Tests: `-f` file · `-d` directory · `-z` empty · `-eq/-gt/-lt` numbers · `==` strings.
- Never put passwords in scripts: read them from the environment.

# 6. Python for DevOps Automation




## 6.0 Why Python for DevOps

Python is the most used language for automation: AWS (boto3), Kubernetes and Docker SDKs, Ansible (written in Python), API integrations, data processing, CLIs, Lambda functions, monitoring checks. It's readable, has a huge standard library and packages for everything. Use **Bash** for short command glue; use **Python** when you need logic, data structures, error handling, APIs, tests or cross-platform tools.

## 6.1 Setup and environment

| Task | Command |
|---|---|
| Check version | `python3 --version` (use 3.10+) |
| Interactive shell (REPL) | `python3` (exit with `exit()` or Ctrl+D) |
| Run a script | `python3 script.py` |
| Create a virtual environment | `python3 -m venv .venv` |
| Activate it | `source .venv/bin/activate` (Windows: `.venv\Scripts\activate`) |
| Install packages | `pip install requests boto3 pyyaml` |
| Freeze exact versions | `pip freeze > requirements.txt` |
| Reproduce | `pip install -r requirements.txt` |
| Install CLI tools globally, isolated | `pipx install <tool>` |
| Deactivate | `deactivate` |

**Why virtual environments:** each project gets its own packages and versions; no conflicts with system Python (which the OS itself uses: never `sudo pip install` into it). Add `.venv/` to `.gitignore`. Modern

projects describe dependencies in `pyproject.toml`; tools like **uv**, Poetry or pip-tools manage lock files.

## 6.2 Syntax basics

```
# Comments start with #
name = "web01"            # variables: no type declaration, snake_case names
MAX_RETRIES = 3           # constants: UPPER_CASE by convention

if name.startswith("web"):
    print("web server")   # indentation (4 spaces) defines blocks; a colon starts a block
else:
    print("other")

print(f"Server {name} allows {MAX_RETRIES} retries")   # f-string formatting
```

- **Indentation is syntax**: mixing tabs/spaces or wrong indentation → `IndentationError`.
- Everything is an object; variables are names pointing to objects.
- Style guide: **PEP 8** (4-space indents, snake_case functions/variables, CapWords classes, max ~88–100 chars per line with tools like black/ruff).

## 6.3 Data types

| Type | Example | Mutable? | Notes |
|---|---|---|---|
| `int` | `42`, `1_000_000` | No | Unlimited size |
| `float` | `3.14`, `1e-3` | No | Binary floating point (`0.1 + 0.2 != 0.3` exactly) |
| `bool` | `True`, `False` | No | Subclass of int |
| `str` | `"text"`, `'text'`, `"""multi-line"""` | No | Unicode text |
| `bytes` | `b"raw"` | No | Binary data (`.encode()` / `.decode()`) |
| `None` | `None` | — | "No value" |
| `list` | `[1, 2, 3]` | **Yes** | Ordered |
| `tuple` | `(1, 2)` | No | Ordered, fixed |
| `dict` | `{"env": "dev"}` | **Yes** | Key → value, insertion-ordered |
| `set` | `{"a", "b"}` | **Yes** | Unique items, fast membership |

**Truthiness:** `False`, `None`, `0`, `""`, `[]`, `{}`, `set()` are false in conditions; everything else is true. `if not items:` = "list is empty".

**Conversions:** `int("80")`, `float("1.5")`, `str(80)`, `bool("")`, `list("abc")`, `tuple(lst)`, `set(lst)`, `dict(pairs)`. Check type: `type(x)`, `isinstance(x, str)`.

### Strings

```
s = "  Web-Server-01  "
s.strip()                       # 'Web-Server-01'
s.strip().lower()               # 'web-server-01'
"a,b,c".split(",")              # ['a', 'b', 'c']
"-".join(["2026", "10", "17"]) # '2026-10-17'
"error" in "fatal error"        # True
"app.log".endswith(".log")      # True
"web01".replace("01", "02")     # 'web02'
"web01"[0:3], "web01"[-2:]      # ('web', '01')  slicing [start:stop]
len("web01")                    # 5
"Port: {}".format(80)           # older formatting
f"{3.14159:.2f} | {42:>6} | {'ok':<4}|"   # '3.14 |     42 | ok  |'
r"C:\new\folder"                # raw string: backslashes not escapes (regex, Windows paths)
```

## 6.4 Collections in depth

### Lists

```
servers = ["web1", "web2"]
servers.append("web3")              # add one
servers.extend(["db1", "db2"])      # add many
servers.insert(0, "lb1")            # insert at position
servers.remove("db2")               # remove by value (ValueError if missing)
last = servers.pop()                # remove and return last
servers.sort()                      # sort in place
sorted(servers, reverse=True)       # new sorted list
servers.index("web2")               # position
servers.count("web1")               # occurrences
servers[0], servers[-1], servers[1:3]   # first, last, slice
```

### Dictionaries

```
server = {"name": "web1", "port": 80, "tags": ["prod"]}
server["port"]                        # 80 (KeyError if missing)
server.get("env", "dev")              # 'dev' (safe default)
server["env"] = "prod"                # add/update
server.update({"port": 8080, "az": "a"})
del server["az"]
"name" in server                      # key exists?
for key, value in server.items():     # iterate pairs
    print(key, value)
list(server.keys()), list(server.values())
config = {**server, "port": 443}      # merge (new dict)
```

### Sets and tuples

```
allowed = {"dev", "staging", "prod"}
"qa" in allowed                     # fast membership → False
{"a", "b"} | {"b", "c"}             # union {'a','b','c'}
{"a", "b"} & {"b", "c"}             # intersection {'b'}
{"a", "b"} - {"b"}                  # difference {'a'}
host, port = ("db.internal", 5432) # tuple unpacking
```

### Comprehensions and useful built-ins

```
ports = [80, 443, 8080, 22]
web_ports = [p for p in ports if p != 22]                # list comprehension
squares = {p: p * p for p in range(3)}                   # dict comprehension
unique_envs = {s.split("-")[0] for s in ["prod-a", "prod-b", "dev-a"]}   # set → {'prod', 'dev'}

for i, name in enumerate(["web1", "web2"], start=1):      # index + value
    print(i, name)
for name, ip in zip(["web1", "web2"], ["10.0.1.5", "10.0.1.6"]):   # pair two lists
    print(name, ip)

instances = [{"id": "i-1", "cpu": 70}, {"id": "i-2", "cpu": 95}]
busiest = max(instances, key=lambda i: i["cpu"])           # {'id': 'i-2', 'cpu': 95}
by_cpu = sorted(instances, key=lambda i: i["cpu"], reverse=True)
any(i["cpu"] > 90 for i in instances), all(i["cpu"] > 50 for i in instances)   # (True, True)
sum(i["cpu"] for i in instances) / len(instances)          # 82.5
```

## 6.5 Control flow

```
cpu = 85
if cpu >= 90:
    level = "critical"
elif cpu >= 70:
    level = "warning"
else:
    level = "ok"

status = "up" if cpu < 100 else "down"     # conditional expression

for attempt in range(1, 4):                # 1, 2, 3
    if attempt == 2:
        continue                           # skip to next iteration
    print("attempt", attempt)
else:
    print("loop finished without break")   # for-else: runs if no break

retries = 0
while retries < 3:
    retries += 1

match level:                               # structural pattern matching (Python 3.10+)
    case "critical":
        print("page on-call")
    case "warning":
        print("create ticket")
    case _:
        print("nothing to do")
```

## 6.6 Functions

```
def check_port(host: str, port: int = 22, *, timeout: float = 3.0) -> bool:
    """Return True if a TCP connection to host:port succeeds.

    timeout is keyword-only (after the *), e.g. check_port("db", 5432, timeout=1).
    """
    import socket
    try:
        with socket.create_connection((host, port), timeout=timeout):
            return True
    except OSError:
        return False


def summarize(*hosts: str, **labels: str) -> dict:
    """*args collects positional args into a tuple; **kwargs collects keyword args into a dict."""
    return {"count": len(hosts), "hosts": hosts, "labels": labels}


print(summarize("web1", "web2", env="prod", team="shop"))
# {'count': 2, 'hosts': ('web1', 'web2'), 'labels': {'env': 'prod', 'team': 'shop'}}


def min_max(values: list[int]) -> tuple[int, int]:
    return min(values), max(values)        # returns a tuple


low, high = min_max([3, 9, 1])             # 1, 9
```

| Concept | Detail |
|---|---|
| Default values | `port: int = 22`. Never use a mutable default ( `def f(items=[])` ): use `None` and create inside |
| Type hints | Documentation + checking with mypy; not enforced at runtime |
| Docstring | First string in the function: shown by `help()` |
| Scope | Variables assigned inside a function are local; avoid `global` |
| Lambda | Small anonymous function: `key=lambda x: x["cpu"]` |
| Return | No `return` → returns `None` |

## 6.7 Modules, packages and the standard library

```
import os                                  # whole module
import json as js                          # alias
from pathlib import Path                   # one name
from datetime import datetime, timedelta, timezone
```

A **module** is a `.py` file; a **package** is a folder of modules (with `__init__.py` ). Your own `utils.py` can be imported with `import utils` from the same folder. **Don't name your files after standard modules** ( `json.py` , `logging.py` ): they shadow the real ones.

| Standard library module | Use in DevOps |
|---|---|
| `os` , `sys` | Environment variables, exit codes, arguments, platform |
| `pathlib` , `shutil` , `glob` , `tempfile` | Paths, copying/moving, patterns, temp files |

| Standard library module | Use in DevOps |
|---|---|
| `subprocess` | Run commands |
| `json`, `csv`, `configparser`, `tomllib` | Data formats |
| `re` | Regular expressions |
| `datetime`, `time` | Timestamps, durations, sleeping |
| `logging` | Proper logs |
| `argparse` | Command-line interfaces |
| `socket`, `ssl`, `urllib` | Networking basics |
| `hashlib`, `secrets`, `base64` | Checksums, secure random tokens, encoding |
| `collections` (`Counter`, `defaultdict`, `deque`) | Counting, grouping |
| `concurrent.futures` | Parallel work |
| `platform`, `getpass`, `shlex` | System info, hidden input, safe shell splitting |

```
from collections import Counter, defaultdict

codes = ["200", "500", "200", "404", "200"]
Counter(codes).most_common(2)                  # [('200', 3), ('500', 1)]

by_team = defaultdict(list)
for svc, team in [("api", "shop"), ("web", "shop"), ("auth", "core")]:
    by_team[team].append(svc)                  # {'shop': ['api', 'web'], 'core': ['auth']}

from datetime import datetime, timedelta, timezone
now = datetime.now(timezone.utc)
cutoff = now - timedelta(days=7)
print(now.isoformat(), now.strftime("%Y-%m-%d %H:%M"))
datetime.fromisoformat("2026-10-17T09:30:00+00:00") < now     # True
```

## 6.8 Files, paths and data formats

### Files and paths

```
from pathlib import Path

base = Path("/tmp/demo")
base.mkdir(parents=True, exist_ok=True)          # like mkdir -p
report = base / "report.txt"                     # join paths with /
report.write_text("line 1\nline 2\n")
print(report.read_text())
print(report.exists(), report.is_file(), report.suffix, report.stem, report.parent)

with open(report, "a", encoding="utf-8") as f:   # 'with' closes the file automatically
    f.write("line 3\n")                          # modes: r, w (truncate), a (append), b (binary)

with open(report, encoding="utf-8") as f:        # stream large files line by line (low memory)
    for line in f:
        print(line.rstrip())

for log in base.glob("*.txt"):                   # rglob("*.log") searches recursively
    print(log.name, log.stat().st_size)
```

`shutil.copy2(src, dst)`, `shutil.move`, `shutil.rmtree(dir)`, `shutil.disk_usage("/")`, `tempfile.TemporaryDirectory()`.

### JSON

```
import json

data = {"service": "api", "replicas": 3, "ports": [80, 443]}
text = json.dumps(data, indent=2)        # dict → JSON string
again = json.loads(text)                 # JSON string → dict

with open("/tmp/demo/config.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)         # write to file
with open("/tmp/demo/config.json", encoding="utf-8") as f:
    cfg = json.load(f)                   # read from file
print(cfg["ports"][1])                   # 443
```

### YAML, CSV, INI/TOML

```
import csv
import yaml                              # pip install pyyaml

doc = yaml.safe_load("""
service: api
replicas: 3
env:
  - name: LOG_LEVEL
    value: info
""")                                     # ALWAYS safe_load (yaml.load can execute code)
print(doc["env"][0]["value"])            # info
print(yaml.safe_dump(doc, sort_keys=False))

with open("/tmp/demo/users.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["user", "team"])
    w.writeheader()
    w.writerows([{"user": "alice", "team": "ops"}, {"user": "bob", "team": "dev"}])

with open("/tmp/demo/users.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["user"], row["team"])
```

`configparser` reads INI files; `tomllib` (3.11+) reads TOML (`pyproject.toml`).

### Environment variables and configuration

```
import os

region = os.environ.get("AWS_REGION", "ap-south-1")    # optional with default
token = os.environ["API_TOKEN"]                        # required: KeyError if missing
debug = os.getenv("DEBUG", "false").lower() == "true"  # env vars are always strings
```

**Configuration order of precedence (common pattern):** defaults in code → config file (YAML) → environment variables → command-line arguments. **Secrets** come from environment variables injected by the platform (CI secrets, Kubernetes Secrets, ECS task definitions) or directly from a secrets manager (AWS Secrets Manager, Vault). Never commit them; `python-dotenv` can load a local, git-ignored `.env` file for development.

## 6.9 Errors and exceptions

```
import sys


class ConfigError(Exception):
    """Raised when configuration is invalid."""


def load_port(value: str) -> int:
    try:
        port = int(value)
    except ValueError as e:
        raise ConfigError(f"port must be a number, got {value!r}") from e
    if not 1 <= port <= 65535:
        raise ConfigError(f"port out of range: {port}")
    return port


try:
    port = load_port("80a")
except ConfigError as e:
    print(f"configuration error: {e}", file=sys.stderr)
    port = 8080
else:
    print("valid port")                    # runs only if no exception
finally:
    print("always runs (cleanup)")
```

| Practice | Why |
|---|---|
| Catch **specific** exceptions ( `FileNotFoundError` , `KeyError` , `requests.Timeout` ) | A bare `except:` hides bugs and even Ctrl+C |
| Add context, re-raise with `from e` | Keeps the original traceback |
| Fail fast on bad config | Clear error at startup instead of strange behaviour later |
| Exit with codes: `sys.exit(0)` success, `sys.exit(1)` failure | CI, cron and systemd rely on them |
| Read tracebacks **bottom-up** | The last line is the error; the lines above show where it happened |

## 6.10 Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,                    # DEBUG < INFO < WARNING < ERROR < CRITICAL
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
)
log = logging.getLogger("healthcheck")

log.debug("not shown at INFO level")
log.info("checking %s", "web1")                          # lazy formatting
log.warning("slow response: %.2fs", 2.5)
try:
    1 / 0
except ZeroDivisionError:
    log.exception("calculation failed")                  # logs the traceback
```

- Use `logging`, not `print`, in tools that run unattended.
- Log to **stdout/stderr** in containers (the platform collects it); to files with `RotatingFileHandler` on servers.
- **Structured (JSON) logs** are easier to search in CloudWatch/Loki/Elastic (libraries: `python-json-logger`, `structlog`).
- Never log secrets or full tokens.

## 6.11 Running system commands: subprocess

```
import shlex
import subprocess

result = subprocess.run(
    ["df", "-h", "/"],          # list of arguments: no shell, no injection risk
    capture_output=True,        # capture stdout/stderr
    text=True,                  # str instead of bytes
    check=True,                 # raise CalledProcessError on non-zero exit
    timeout=30,                 # raise TimeoutExpired if it hangs
)
print(result.stdout)

r = subprocess.run(["systemctl", "is-active", "nginx"], capture_output=True, text=True)
print("nginx:", r.stdout.strip(), "exit", r.returncode)   # no check=True → inspect yourself

cmd = shlex.split("git log --oneline -n 3")               # split a string safely
print(subprocess.run(cmd, capture_output=True, text=True).stdout)
```

**Avoid `shell=True` with any user input** (command injection). If you need shell features, build the pipeline in Python or pass a fixed string you fully control.

## 6.12 HTTP and REST APIs with requests

```
import os

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()                                    # reuses connections
retries = Retry(total=3, backoff_factor=1, status_forcelist=[429, 500, 502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))
session.headers.update({"Accept": "application/vnd.github+json"})
token = os.getenv("GITHUB_TOKEN")                               # optional; raises rate limits
if token:
    session.headers["Authorization"] = f"Bearer {token}"


def list_repos(org: str) -> list[dict]:
    """Return all repositories of an organization, following pagination."""
    repos, page = [], 1
    while True:
        resp = session.get(
            f"https://api.github.com/orgs/{org}/repos",
            params={"per_page": 100, "page": page},
            timeout=10,                                         # ALWAYS set a timeout
        )
        resp.raise_for_status()                                 # error on 4xx/5xx
        batch = resp.json()
        if not batch:
            return repos
        repos.extend(batch)
        page += 1


if __name__ == "__main__":
    repos = sorted(list_repos("kubernetes"), key=lambda r: r["stargazers_count"], reverse=True)
    for repo in repos[:5]:
        print(f'{repo["name"]:<30} {repo["stargazers_count"]:>7}')
```

| Call | Use |
|---|---|
| `session.get(url, params=..., timeout=10)` | Read |
| `session.post(url, json={...})` | Create (sends JSON body + header) |
| `session.put / patch / delete` | Update / delete |
| `resp.status_code`, `resp.json()`, `resp.text`, `resp.headers` | Read the response |
| `resp.raise_for_status()` | Turn HTTP errors into exceptions |
| Auth | `headers={"Authorization": f"Bearer {token}"}`, `auth=(user, password)` |

**API etiquette:** handle **pagination**, respect **rate limits** (`429`, `Retry-After`/rate-limit headers), retry only idempotent calls or use idempotency keys, set timeouts, keep tokens in env/secret stores.

## 6.13 AWS automation with boto3

```
import boto3
from botocore.exceptions import ClientError

# credentials come from env vars, a profile/SSO, or an IAM role - never from code
session = boto3.Session(region_name="ap-south-1")
ec2 = session.client("ec2")

# Paginators handle pages for you
paginator = ec2.get_paginator("describe_instances")
pages = paginator.paginate(Filters=[{"Name": "instance-state-name", "Values": ["running"]}])
for page in pages:
    for reservation in page["Reservations"]:
        for inst in reservation["Instances"]:
            tags = {t["Key"]: t["Value"] for t in inst.get("Tags", [])}
            print(inst["InstanceId"], inst["InstanceType"],
                  tags.get("Name", "-"), tags.get("Owner", "MISSING"))

# Error handling
s3 = session.client("s3")
try:
    s3.head_bucket(Bucket="my-bucket-that-may-not-exist")
except ClientError as e:
    code = e.response["Error"]["Code"]                  # e.g. '404', '403', 'AccessDenied'
    print(f"bucket check failed: {code}")

# Upload / download
s3.upload_file("/tmp/demo/report.txt", "my-reports-bucket", "reports/report.txt")

# Read a secret parameter (decrypted)
ssm = session.client("ssm")
param = ssm.get_parameter(Name="/shop/prod/db/password", WithDecryption=True)
db_password = param["Parameter"]["Value"]

# Waiters: block until a state is reached
ec2.get_waiter("instance_stopped").wait(InstanceIds=["i-0123456789abcdef0"])
```

| Concept | Detail |
|---|---|
| Client vs resource | `client` = low-level, maps 1:1 to API calls (recommended); `resource` = older object-oriented API |
| Credentials chain | Same as the AWS CLI: env vars → shared config/SSO profile → container/instance role. **Never hard-code keys** |
| Paginators | Many list/describe calls return pages; use `get_paginator` |
| Waiters | Wait for states (instance running, stack complete) |
| Errors | `botocore.exceptions.ClientError`; check `e.response["Error"]["Code"]`; throttling → retries (configurable via `botocore.config.Config(retries=...)`) |
| Dry run | EC2 calls accept `DryRun=True` to test permissions |
| Testing | `moto` library mocks AWS for unit tests |

## 6.14 Other libraries DevOps engineers use

| Library | Use |
|---|---|
| `kubernetes` | Kubernetes API client (list pods, patch deployments) |

| Library | Use |
|---|---|
| `docker` | Docker Engine API |
| `paramiko`, `fabric` | SSH automation |
| `jinja2` | Templates (config files; Ansible uses it) |
| `click` / `typer` | Friendly CLIs with subcommands |
| `rich` | Pretty terminal tables/progress |
| `pydantic` | Validate config/data with type hints |
| `prometheus_client` | Expose custom metrics |
| `pytest`, `moto`, `responses` | Testing, AWS mocks, HTTP mocks |
| `black`, `ruff`, `mypy` | Formatting, linting, type checking |

## 6.15 Classes and dataclasses (enough to read and write tools)

```
from dataclasses import dataclass, field


@dataclass
class Service:
    name: str
    url: str
    timeout: float = 5.0
    tags: list[str] = field(default_factory=list)   # mutable default done right

    def label(self) -> str:
        return f"{self.name} ({self.url})"


svc = Service("api", "https://api.example.com/health", tags=["prod"])
print(svc.label(), svc.timeout)                      # api (https://api.example.com/health) 5.0
```

A **class** bundles data (attributes) and behaviour (methods); `self` is the instance. `@dataclass` generates `__init__`, `__repr__` and comparisons automatically: ideal for config and result objects.

## 6.16 Concurrency: checking many things fast

Network checks spend most of their time waiting, so threads speed them up a lot.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import socket


def port_open(host: str, port: int) -> tuple[str, int, bool]:
    try:
        with socket.create_connection((host, port), timeout=2):
            return host, port, True
    except OSError:
        return host, port, False


targets = [("github.com", 443), ("example.com", 80), ("10.255.255.1", 22)]
with ThreadPoolExecutor(max_workers=10) as pool:
    futures = [pool.submit(port_open, h, p) for h, p in targets]
    for fut in as_completed(futures):
        host, port, ok = fut.result()
        print(f"{host}:{port} {'open' if ok else 'closed/unreachable'}")
```

Use `ProcessPoolExecutor` for CPU-heavy work; `asyncio` (+ `httpx` / `aiohttp`) for thousands of concurrent I/O tasks.

## 6.17 Testing with pytest

```
# file: checks.py
def classify_cpu(cpu: float) -> str:
    if cpu >= 90:
        return "critical"
    if cpu >= 70:
        return "warning"
    return "ok"
```

```
# file: test_checks.py   (run: pytest -q)
import pytest
from checks import classify_cpu


@pytest.mark.parametrize("cpu, expected", [(10, "ok"), (70, "warning"), (95, "critical")])
def test_classify_cpu(cpu, expected):
    assert classify_cpu(cpu) == expected
```

**Mocking external calls** so tests don't hit real APIs:

```
from unittest.mock import patch, MagicMock
import requests


def get_status(url: str) -> int:
    return requests.get(url, timeout=5).status_code


def test_get_status():
    fake = MagicMock(status_code=503)
    with patch("requests.get", return_value=fake) as mocked:
        assert get_status("https://example.com") == 503
        mocked.assert_called_once()
```

Run tests in CI on every pull request ( `pytest -q --maxfail=1` ), together with `ruff check .` and `black --check .`.

## 6.18 Code quality and packaging

| Practice | Tool / how |
|---|---|
| Formatting | `black .` (or `ruff format`) |
| Linting | `ruff check .` (fast, replaces flake8/isort/many plugins) |
| Type checking | `mypy .` with type hints |
| Dependencies pinned | `requirements.txt` with versions or a lock file (uv/poetry/pip-tools) |
| Security scanning | `pip-audit`, Dependabot |
| Entry point | `if __name__ == "__main__": main()` and `sys.exit(main())` |
| CLI | `argparse` (stdlib) or `click` / `typer` |
| Distribution | Container image ( `python:3.12-slim` ) or a package with a console script in `pyproject.toml` |
| Project layout | `src/<pkg>/`, `tests/`, `pyproject.toml`, `README.md`, `.gitignore` |

## 6.19 Project: service health checker CLI

A complete, realistic tool combining everything: YAML config, environment overrides, parallel HTTP checks, logging, JSON report and exit codes for CI/cron.

```
# services.yaml
timeout: 5
services:
  - name: github
    url: https://api.github.com
  - name: example
    url: https://example.com
    expect_status: 200
```

```
#!/usr/bin/env python3
"""healthcheck.py - check HTTP services listed in a YAML file.

Usage: python3 healthcheck.py services.yaml [--report out.json] [--workers 8]
Exit code: 0 all healthy, 1 at least one unhealthy, 2 configuration error.
"""
import argparse
import json
import logging
import os
import sys
import time
from concurrent.futures import ThreadPoolExecutor
from dataclasses import asdict, dataclass

import requests
import yaml

log = logging.getLogger("healthcheck")


@dataclass
class Result:
    name: str
    url: str
    ok: bool
    status: int | None
    seconds: float
    error: str = ""


def check(name: str, url: str, expect: int, timeout: float) -> Result:
    start = time.monotonic()
    try:
        resp = requests.get(url, timeout=timeout)
        ok = resp.status_code == expect
        return Result(name, url, ok, resp.status_code, round(time.monotonic() - start, 3))
    except requests.RequestException as e:
        return Result(name, url, False, None, round(time.monotonic() - start, 3), str(e))


def load_config(path: str) -> dict:
    try:
        with open(path, encoding="utf-8") as f:
            cfg = yaml.safe_load(f) or {}
    except (OSError, yaml.YAMLError) as e:
        raise SystemExit(f"config error: {e}") from e
    if not cfg.get("services"):
        raise SystemExit("config error: no services defined")
    cfg["timeout"] = float(os.getenv("HC_TIMEOUT", cfg.get("timeout", 5)))
    return cfg


def main() -> int:
    parser = argparse.ArgumentParser(description="HTTP health checker")
    parser.add_argument("config")
    parser.add_argument("--report", help="write a JSON report to this file")
    parser.add_argument("--workers", type=int, default=8)
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")

    try:
        cfg = load_config(args.config)
    except SystemExit as e:
```

```
        log.error("%s", e)
        return 2

    with ThreadPoolExecutor(max_workers=args.workers) as pool:
        results = list(pool.map(
            lambda s: check(s["name"], s["url"], s.get("expect_status", 200), cfg["timeout"]),
            cfg["services"]))

    for r in results:
        level = logging.INFO if r.ok else logging.ERROR
        log.log(level, "%-10s %-5s status=%s time=%.3fs %s",
                r.name, "UP" if r.ok else "DOWN", r.status, r.seconds, r.error)

    if args.report:
        with open(args.report, "w", encoding="utf-8") as f:
            json.dump([asdict(r) for r in results], f, indent=2)

    return 0 if all(r.ok for r in results) else 1


if __name__ == "__main__":
    sys.exit(main())
```

**Extend it:** send a Slack/Teams message on failure (webhook URL from an env var), expose results as Prometheus metrics, run it from cron/a systemd timer/a Kubernetes CronJob, package it in a Docker image, add tests with `responses` to mock HTTP.

## 6.20 Common errors and what they mean

| Error | Meaning / fix |
|---|---|
| `IndentationError` / `TabError` | Inconsistent indentation: use 4 spaces |
| `SyntaxError` | Missing colon, bracket or quote; check the line shown (or the line before) |
| `NameError: name 'x' is not defined` | Typo, or used before assignment/import |
| `TypeError: can only concatenate str (not "int") to str` | Mixing types: use f-strings or `str()` |
| `KeyError: 'env'` | Missing dict key: use `.get()` or check `in` |
| `IndexError: list index out of range` | Accessing past the end of a list |
| `AttributeError: 'NoneType' object has no attribute ...` | A function returned `None` (often a failed lookup) |
| `ModuleNotFoundError` | Package not installed in **this** venv (activate it, `pip install`), or file name shadows a module |
| `FileNotFoundError` / `PermissionError` | Wrong path (relative to the current directory!) / permissions |
| `json.decoder.JSONDecodeError` | Response wasn't JSON (check status code / HTML error page) |
| `requests.exceptions.ConnectionError` / `Timeout` | DNS/network/firewall problem / server too slow |
| `botocore.exceptions.NoCredentialsError` | No AWS credentials found (profile, env, role) |
| `ClientError: AccessDenied` | IAM permissions (see AWS notes) |

| Error | Meaning / fix |
|---|---|
| `UnicodeDecodeError` | Wrong encoding: open files with `encoding="utf-8"` (or `errors="replace"`) |

## 6.21 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | Why use Python instead of Bash? | Better for complex logic, data structures, APIs/JSON, error handling, tests and portability |
| 2 | What is a virtual environment? | An isolated Python environment per project with its own packages |
| 3 | List vs tuple vs set vs dict? | Ordered mutable / ordered immutable / unique unordered / key-value mapping |
| 4 | Mutable vs immutable? | Mutable objects (list, dict, set) can change in place; immutable (str, int, tuple) can't |
| 5 | What are `*args` and `**kwargs`? | Collect extra positional arguments (tuple) and keyword arguments (dict) |
| 6 | Why avoid mutable default arguments? | The default is created once and shared between calls |
| 7 | What does `if __name__ == "__main__":` do? | Runs code only when the file is executed directly, not when imported |
| 8 | How do you handle exceptions? | try/except with specific exceptions, else/finally, raise with context, proper exit codes |
| 9 | How do you run shell commands safely? | `subprocess.run([...], check=True, capture_output=True, text=True, timeout=...)` without `shell=True` |
| 10 | How do you call a REST API reliably? | requests with timeouts, raise_for_status, retries with backoff, pagination, auth from env |
| 11 | How does boto3 get credentials? | Same chain as the CLI: env → profile/SSO → container/instance role; never hard-coded |
| 12 | What are paginators and waiters in boto3? | Helpers that iterate all pages of results / wait until a resource reaches a state |
| 13 | `json.load` vs `json.loads`? | From a file object vs from a string |
| 14 | Why `yaml.safe_load`? | `yaml.load` can construct arbitrary objects (security risk) |
| 15 | How do you speed up many network checks? | Threads (`ThreadPoolExecutor`) or asyncio |
| 16 | How do you test code that calls external APIs? | Mock them (`unittest.mock`, `responses`, `moto` for AWS) |
| 17 | What is a list comprehension? | Compact syntax to build a list from an iterable with an optional filter |
| 18 | How do you manage dependencies reproducibly? | Pinned requirements/lock file in a venv or container image |
| 19 | How should a script report failure to CI/cron? | Non-zero exit code via `sys.exit(1)` plus clear logs |
| 20 | What is a dataclass? | A class decorator that generates init/repr/eq for data containers |

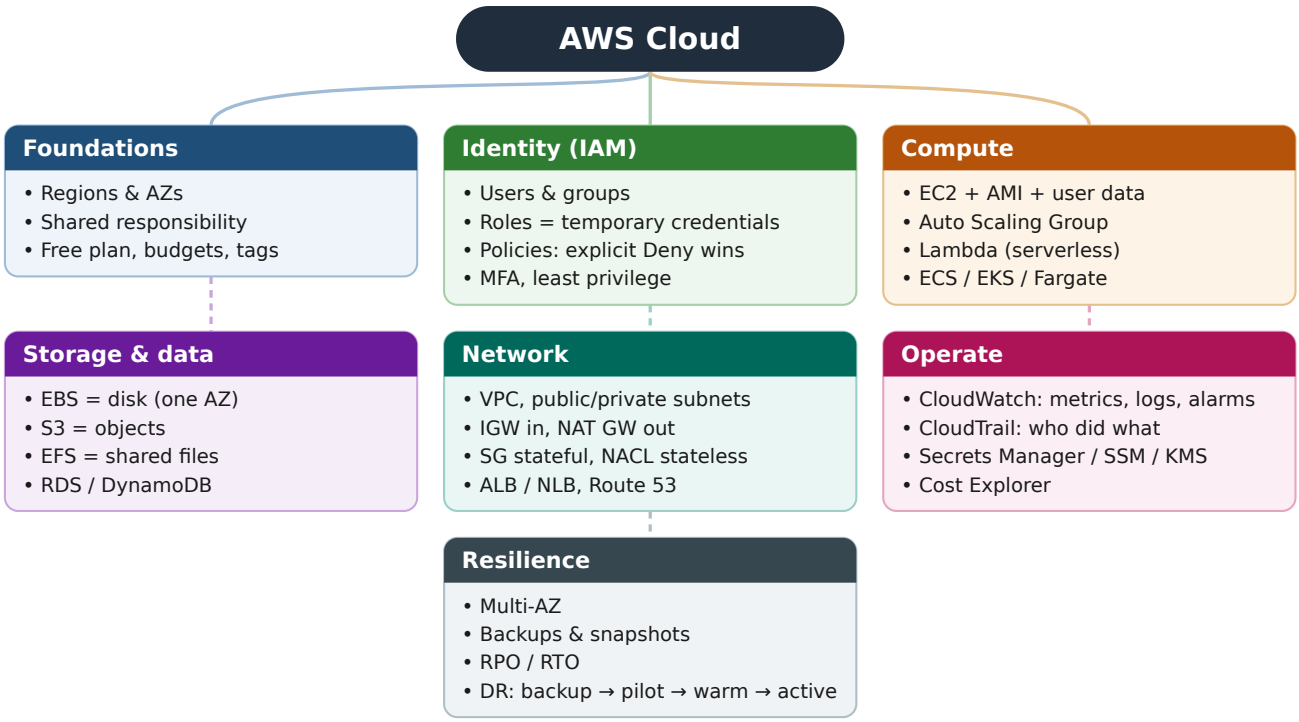
## 6.22 Self-check: I can…

- [ ] Set up a project with a virtual environment and pinned requirements.
- [ ] Use all core data types, string methods, slicing and f-strings.
- [ ] Use lists, dicts, sets, tuples, comprehensions, `sorted` / `max` with `key`, `enumerate`, `zip`.
- [ ] Write control flow including `match`, and functions with defaults, keyword-only args, `*args/**kwargs`, type hints.
- [ ] Use the standard library: pathlib, json, csv, datetime, collections, subprocess, logging, argparse.
- [ ] Read/write files and JSON/YAML/CSV safely; read configuration and secrets from the environment.
- [ ] Handle exceptions properly and return meaningful exit codes.
- [ ] Call REST APIs with timeouts, retries, pagination and token auth.
- [ ] Automate AWS with boto3 (paginators, waiters, error codes, no hard-coded keys).
- [ ] Write dataclasses and read simple classes.
- [ ] Parallelize I/O-bound checks with threads.
- [ ] Write pytest tests, including mocks for external services.
- [ ] Keep code formatted, linted, typed and packaged as a CLI or container.
- [ ] Build and extend the health checker project.

**REMEMBER**

- Three habits: **virtual environment**, **timeout on every network call**, **secrets from the environment**.
- Return a non-zero exit code on failure so cron/CI notices.

# 7. AWS Cloud




## 7.0 How to use these notes

- **Prerequisites:** Networking (IP, subnets, DNS, ports, TCP/HTTP) and Linux basics. AWS builds on both.
- **Order:** read the sections in order the first time. Later, use them as reference.
- **Each service is explained the same way:** what it is → why we use it → how it works → key settings → commands → best practices → common mistakes.
- **Labs** are marked ⚠️ when they can use credits. Every lab ends with **cleanup**. Always check **Billing → Bills** after a lab.
- **Region used in examples:** `ap-south-1` (Mumbai). Replace it with your own region.
- Placeholders look like `<your-bucket-name>` or `i-0123456789abcdef0`: replace them with your real values.

## 7.1 Cloud computing fundamentals

### What cloud computing is

**What:** renting IT resources (servers, storage, databases, networks, software) over the internet, **on demand**, and **paying only for what you use**.

**Why companies use it:**

| Traditional data center | Cloud |
|---|---|
| Buy servers months in advance (big upfront cost, CapEx) | Pay monthly for what you use (operating cost, OpEx) |
| Guess capacity: too much = waste, too little = outage | Scale up and down in minutes |

| Traditional data center | Cloud |
|---|---|
| Weeks to get a new server | Minutes to get a new server |
| You run the building, power, cooling, hardware | The provider does |
| Expanding to another country = building a data center | Deploy to another region in minutes |

AWS lists six benefits: trade upfront expense for variable expense · benefit from massive economies of scale · stop guessing capacity · increase speed and agility · stop spending money running data centers · go global in minutes.

**Real-world example:** an online shop needs 10 servers normally and 100 during a sale. In the cloud it runs 10, automatically grows to 100 for the sale, and goes back to 10 afterwards, paying for the extra servers only for those hours.

## Service models

```
                Applications   Data   Runtime   OS   Virtualization   Servers   Storage   Network
On-premises     YOU            YOU    YOU       YOU  YOU              YOU       YOU       YOU
IaaS  (EC2)     YOU            YOU    YOU       YOU  AWS              AWS       AWS       AWS
PaaS  (RDS)     YOU            YOU    AWS       AWS  AWS              AWS       AWS       AWS
SaaS  (Gmail)   provider       YOU*   provider  ...  provider         provider  provider  provider
                               * you still own your data and user access
```

| Model | You get | You manage | AWS examples |
|---|---|---|---|
| IaaS (Infrastructure as a Service) | Virtual servers, networks, disks | OS patches, software, data, security config | EC2, EBS, VPC |
| PaaS (Platform as a Service) | A platform to run code or databases | Your code and data | Elastic Beanstalk, RDS, Lambda (serverless) |
| SaaS (Software as a Service) | Finished software | Users and data | Amazon WorkMail, Chime (and non-AWS: Gmail) |

## Deployment models

| Model | Meaning | Example |
|---|---|---|
| Public cloud | Resources run on a provider's shared infrastructure | A startup fully on AWS |
| Private cloud | Cloud-style infrastructure dedicated to one organization (often on-premises) | A bank running OpenStack/VMware in its own data center |
| Hybrid cloud | On-premises connected to public cloud | Core banking on-premises, website and analytics on AWS, connected by VPN/Direct Connect |
| Multi-cloud | More than one public cloud provider | AWS for apps, Google Cloud for analytics |

## 7.2 AWS global infrastructure

```
AWS GLOBAL NETWORK
│
├── Region  ap-south-1 (Mumbai)              ← a geographic area, fully independent
│     ├── Availability Zone ap-south-1a       ← one or more data centers
│     ├── Availability Zone ap-south-1b       ← separate power, cooling, network
│     └── Availability Zone ap-south-1c       ← connected by fast, low-latency private links
│
├── Region  us-east-1 (N. Virginia) ...
│
├── Edge locations (hundreds of cities)       ← CloudFront CDN, Route 53 DNS
├── Local Zones                               ← compute close to big cities (very low latency)
├── Wavelength Zones                          ← compute inside 5G telecom networks
└── Outposts                                  ← AWS hardware installed in YOUR data center
```

| Component | What it is | Why it matters |
|---|---|---|
| **Region** | A geographic area with multiple AZs (e.g. `ap-south-1`) | Data stays in the region you choose (laws, latency). Most services are **regional** |
| **Availability Zone (AZ)** | One or more physically separate data centers in a region (e.g. `ap-south-1a`) | Spread your app across **2+ AZs** so one data center failure doesn't take you down |
| **Edge location** | Small sites near users | CloudFront caches content and Route 53 answers DNS close to users |
| **Local Zone / Wavelength / Outposts** | Extensions of a region | Very low latency or on-premises needs |

**How to choose a region:**

1. **Compliance / data laws:** e.g. data must stay in India → `ap-south-1` or `ap-south-2`.
2. **Latency:** close to your users.
3. **Service availability:** new services don't launch in every region at once.
4. **Price:** prices differ between regions (`us-east-1` is often cheapest).

**Global vs regional vs zonal services:**

| Scope | Examples |
|---|---|
| Global | IAM, Route 53, CloudFront, Organizations |
| Regional | S3 buckets, VPC, Lambda, DynamoDB, most services |
| Zonal (AZ-specific) | EC2 instances, EBS volumes, subnets |

**Common mistake:** "My EC2 instances disappeared!" → you are looking at a different region in the console (top-right region selector).

## 7.3 Shared responsibility model

```
┌──────────────────────────────────────────────────────────────┐
│ CUSTOMER: security IN the cloud                              │
│  Your data · IAM users/permissions · OS patching (EC2) · app code  │
│  Security groups / firewall rules · encryption choices · backups   │
├──────────────────────────────────────────────────────────────┤
│ AWS: security OF the cloud                                   │
│  Data centers · hardware · global network · hypervisor       │
│  Managed-service software (e.g. the RDS database engine patching)  │
└──────────────────────────────────────────────────────────────┘
```

The line moves depending on the service:

| Task | EC2 (IaaS) | RDS (managed DB) | Lambda (serverless) | S3 |
|---|---|---|---|---|
| Physical security, hardware | AWS | AWS | AWS | AWS |
| Operating system patches | **You** | AWS | AWS | AWS |
| Database engine patches | **You** (if installed) | AWS (you choose the window) | — | — |
| Network firewall rules (SGs) | **You** | **You** | **You** (if in a VPC) | — |
| Access permissions (IAM, policies) | **You** | **You** | **You** | **You** |
| Data encryption & backups config | **You** | **You** | **You** | **You** |

**Interview line:** "AWS is responsible for security *of* the cloud; the customer is responsible for security *in* the cloud. The more managed the service, the more AWS takes on, but data and access control are always the customer's."

## 7.4 How AWS pricing works

**Three main cost drivers: compute** (per second/hour), **storage** (per GB-month), **data transfer** (per GB).

| Data transfer | Typical cost |
|---|---|
| Into AWS from the internet | Free |
| Out of AWS to the internet | Charged per GB (the first 100 GB/month is free across all services) |
| Between AZs in the same region | Charged (small, per GB, each direction) |
| Between regions | Charged per GB |
| Within the same AZ using private IPs | Free |

**Purchasing options (compute):**

| Option | How it works | Saving vs On-Demand | Use for |
|---|---|---|---|
| On-Demand | Pay per second, no commitment | — | Dev/test, unpredictable workloads, short labs |

| Option | How it works | Saving vs On-Demand | Use for |
|---|---|---|---|
| Savings Plans | Commit to $X/hour for 1 or 3 years | Up to ~72% | Steady production usage |
| Reserved Instances | Commit to a specific instance type/region | Up to ~72% | Older model, still common |
| Spot Instances | Use spare capacity; AWS can reclaim with a **2-minute warning** | Up to ~90% | Batch jobs, CI runners, stateless workers |
| Dedicated Hosts/Instances | Hardware not shared with other customers | More expensive | Licensing or compliance |

**AWS Free plan (accounts created since July 2025):** $100 credits at sign-up + up to $100 more for completing onboarding activities; the plan lasts **6 months or until credits run out**. Some services always have a free amount (e.g. Lambda requests, DynamoDB storage). At the end you choose to upgrade to a paid plan; otherwise the account is closed. Always confirm the current rules on the AWS Free Tier page.

**Always-on cost traps:** NAT Gateways, load balancers, public IPv4 addresses (billed per hour), EKS control planes, RDS instances, unattached EBS volumes, old snapshots, interface VPC endpoints, resources left in another region.

## 7.5 Ways to use AWS

| Method | What it is | Used for |
|---|---|---|
| Management Console | Web interface | Learning, exploring, one-off checks |
| AWS CLI | Command-line tool ( `aws ...` ) | Scripts, automation, fast admin work |
| SDKs | Libraries for Python (boto3), JavaScript, Go, Java... | Applications and tools that call AWS |
| Infrastructure as Code | Terraform, CloudFormation, CDK | **How companies build production infrastructure** |
| CloudShell | Browser terminal with the CLI pre-installed and already logged in | Quick CLI work without local setup |

### ARN (Amazon Resource Name)

Every AWS resource has a unique ID called an **ARN**:

```
arn:partition:service:region:account-id:resource
arn:aws:s3:::my-bucket                                  ← S3 (global name: no region/account)
arn:aws:ec2:ap-south-1:123456789012:instance/i-0abc123  ← EC2 instance
arn:aws:iam::123456789012:role/web-server-role          ← IAM (global: no region)
```

ARNs are used in IAM policies, CLI commands and Terraform outputs.

### Tags

**Tags** are key-value labels on resources: `Project=shop`, `Env=prod`, `Owner=thirupathi`, `CostCenter=1234`. **Why:** find resources, split costs per team/project (cost allocation tags), automate (e.g. "stop all instances with `AutoStop=true` at night"), and control access with tag-based IAM conditions. **Best practice:** agree on a tagging standard and enforce it (Terraform `default_tags`, AWS Organizations tag policies).

## 7.6 Account setup and security baseline

Do this **once, on day one** of any AWS account.

### Step-by-step

1. **Create the account** at aws.amazon.com (email, account name, card for verification, phone check, choose the **Free plan**).
2. **Protect the root user** (the email you signed up with = full, unlimited access): - Enable **MFA** on root (IAM → Security credentials → Assign MFA; use an authenticator app or hardware key). - Never create access keys for root. - Store the root password safely and **stop using root** for daily work.
3. **Create your daily admin identity:** - Recommended: **IAM Identity Center** (single sign-on) user with the `AdministratorAccess` permission set, then log in through the access portal URL. - Simpler for a personal lab: an **IAM user** in an `Admins` group with `AdministratorAccess`, console password and **MFA**.
4. **Billing safety:** - Billing → **Budgets** → create a monthly cost budget (e.g. $10) with email alerts at 50%, 80%, 100%. - Turn on **Free plan usage alerts** and check the credit balance page. - Enable **IAM user access to billing** (Account settings) if you use an IAM admin user.
5. **Visibility:** CloudTrail **Event history** is on by default (90 days of management events). For long-term audit, create a **trail** that writes to S3.
6. **Pick one region** and use it consistently.
7. **Set the account alternate contacts** (billing, operations, security).

### Tasks only the root user can do

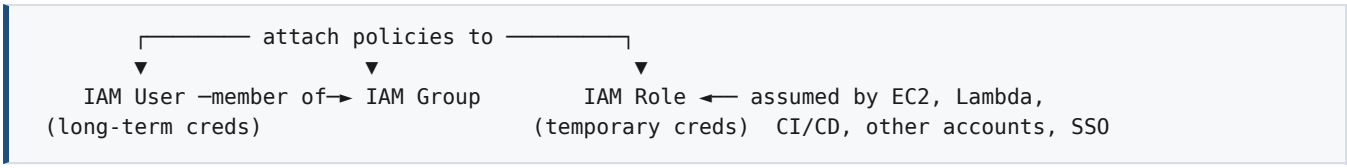
Change account settings (name, email, root password), close the account, change the support plan, restore IAM admin permissions if they were removed, some billing tasks. Everything else: use your admin identity.

## 7.7 IAM: Identity and Access Management

**What:** the AWS service that controls **who** can do **what** on **which** resources. **Why:** every API call to AWS is checked by IAM. Most cloud breaches come from leaked credentials or permissions that are too broad, so IAM is the most important security service. **Scope:** global (not per region), free.

### Core concepts

| Concept | What it is | Example |
|---|---|---|
| **Principal** | Who is making the request | A user, a role session, an AWS service |
| **Authentication** | Proving who you are | Password + MFA, access keys, temporary credentials |
| **Authorization** | Checking what you are allowed to do | IAM evaluates policies |
| **IAM user** | A long-term identity for a person or application, with a password and/or access keys | `thirupathi-admin` |
| **IAM group** | A collection of users that share permissions (groups can't contain groups) | `Developers`, `Admins` |
| **IAM role** | An identity with permissions but **no long-term credentials**. It is **assumed** and gives **temporary credentials** | `web-server-role` used by EC2, `github-deploy-role` used by CI |
| **Policy** | A JSON document that allows or denies actions | `AmazonS3ReadOnlyAccess` |




## Policy anatomy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadAppBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::shop-assets",
        "arn:aws:s3:::shop-assets/*"
      ],
      "Condition": {
        "IpAddress": { "aws:SourceIp": "203.0.113.0/24" }
      }
    }
  ]
}
```

| Element | Meaning |
|---|---|
| `Version` | Policy language version. Always `"2012-10-17"` |
| `Statement` | List of permission rules |
| `Sid` | Optional label for the statement |
| `Effect` | `Allow` or `Deny` |
| `Action` | API actions, format `service:Action`. Wildcards allowed: `s3:Get*`, `ec2:*` |
| `Resource` | ARNs the actions apply to. `*` = all |
| `Condition` | Optional extra checks: source IP, MFA, tags, time, region… |
| `Principal` | Only in **resource-based** policies (who is allowed), e.g. in bucket policies and role trust policies |

**Note:** `s3:ListBucket` applies to the bucket ARN; `s3:GetObject` applies to objects (`bucket/*`). Forgetting one of them is a classic mistake.

## Policy types

| Type | Attached to | Purpose |
|---|---|---|
| AWS managed policy | Users/groups/roles | Ready-made by AWS (e.g. `ReadOnlyAccess`). Broad; good for starting |
| Customer managed policy | Users/groups/roles | Your own reusable policy. **Preferred for least privilege** |
| Inline policy | One user/group/role only | Strict one-to-one; harder to manage |
| Resource-based policy | A resource (S3 bucket, SQS queue, KMS key, role trust policy) | Says **who** may access this resource, including other accounts |
| Permissions boundary | A user or role | The **maximum** permissions it can ever have |

| Type | Attached to | Purpose |
|---|---|---|
| Service Control Policy (SCP) | AWS Organizations account/OU | Guardrail for entire accounts (never grants, only limits) |
| Session policy | A temporary session | Further limits a role session |

## How AWS decides: policy evaluation

```
Request arrives
   │
   ├─ Is there an explicit DENY in any applicable policy? ── yes ──► DENIED
   │
   ├─ Do SCPs (if any) allow it? ──────────────────────────── no ──► DENIED
   │
   ├─ Is there an ALLOW in identity or resource policy? ──── no ──► DENIED (default deny)
   │
   ├─ Do permission boundary / session policy (if any) allow it? ── no ──► DENIED
   │
   └─ ALLOWED
```

**Remember:** everything is **denied by default** → an **Allow** is needed → an explicit **Deny always wins**.

## IAM roles and STS in depth

A role has **two** policies:

| Policy | Question it answers | Example |
|---|---|---|
| **Trust policy** | **Who** may assume this role? | "The EC2 service may assume me" |
| **Permissions policy** | **What** can the role do once assumed? | "Read objects from bucket `shop-assets`" |

**Trust policy for EC2:**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

**Trust policy for GitHub Actions (OIDC, no stored keys):**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:<owner>/<repo>:ref:refs/heads/main"
      }
    }
  }]
}
```

**How a role is used (STS = Security Token Service):**

```
EC2 instance / Lambda / CI job / user
        │ 1. "I want to assume role X"  (sts:AssumeRole)
        ▼
       STS ── 2. checks the role's TRUST policy
        │ 3. returns temporary credentials (key + secret + session token; 1 hour by default)
        ▼
  App calls AWS APIs with those credentials → checked against the role's PERMISSIONS policy
```

| Role use case | How |
|---|---|
| App on EC2 needs S3 | Role attached through an **instance profile**; SDK/CLI on the instance gets credentials automatically from instance metadata |
| Lambda function | **Execution role** |
| ECS task | **Task role** (app permissions) + **task execution role** (pull image, write logs) |
| EKS pod | IRSA or EKS Pod Identity |
| CI/CD pipeline | OIDC federation → `AssumeRoleWithWebIdentity` |
| Another AWS account | Cross-account role trusting that account |
| People (SSO) | IAM Identity Center permission sets (roles behind the scenes) |
| AWS services themselves | **Service-linked roles** (created automatically, e.g. for Auto Scaling) |

**Why roles beat access keys:** temporary credentials expire automatically, nothing is stored in code or on disk, rotation is automatic, every session is logged in CloudTrail.

## Credentials and authentication hygiene

| Item | Practice |
|---|---|
| MFA | Required for root and all human users |
| Access keys | Avoid. If required: one per user, rotate, never in code/Git, delete unused ones |
| Password policy | Length, complexity, rotation as company policy requires |

| Item | Practice |
|---|---|
| Credential report | IAM → Credential report: lists all users, key ages, MFA status |
| Last accessed info | Shows which services a role/user actually used → remove the rest |
| IAM Access Analyzer | Finds resources shared publicly or with other accounts; generates least-privilege policies from CloudTrail activity |

## IAM Identity Center (SSO)

Central login for people across one or many AWS accounts. Users sign in to an **access portal**, pick an account and a **permission set**, and get temporary credentials for the console and CLI ( `aws configure sso` ). Can connect to an external identity provider (Microsoft Entra ID, Okta, Google). **This is how most companies give engineers AWS access today.**

## Best practices

1. Lock down root (MFA, no keys, don't use it).
2. Humans use **SSO/federation**; workloads use **roles**. Avoid long-term access keys.
3. **Least privilege:** start small, add permissions when needed; use Access Analyzer to trim.
4. Use **groups** (or permission sets), not per-user policies.
5. Use **conditions** (MFA required, source IP, tags, region).
6. Separate environments with **separate accounts** (dev/prod) under Organizations.
7. Monitor with CloudTrail; alert on root logins and IAM changes.

## Hands-on: least-privilege access (free)

**Goal:** a `Developers` group that can only read one S3 bucket, and an EC2 role that can read the same bucket.

1. **Policy:** IAM → Policies → Create policy → JSON → paste the "Policy anatomy" example above (without the `Condition` ) using your bucket name → name it `ShopAssetsRead` .
2. **Group:** IAM → User groups → Create `Developers` → attach `ShopAssetsRead` .
3. **User:** IAM → Users → Create `dev-test` → console access → add to `Developers` .
4. **Test:** log in as `dev-test` (private browser window) → S3 → you can open `shop-assets` but get **Access Denied** on other buckets and on EC2. This is least privilege working.
5. **Role:** IAM → Roles → Create role → Trusted entity **AWS service → EC2** → attach `ShopAssetsRead` → name `web-server-role` . (You'll attach it to an instance in the EC2 lab.)

**Same with the CLI:**

```
aws iam create-policy --policy-name ShopAssetsRead --policy-document file://shop-read.json
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::<account-id>:policy/ShopAssetsRead
aws iam create-role --role-name web-server-role --assume-role-policy-document file://ec2-trust.json
aws iam attach-role-policy --role-name web-server-role \
  --policy-arn arn:aws:iam::<account-id>:policy/ShopAssetsRead
aws iam create-instance-profile --instance-profile-name web-server-role
aws iam add-role-to-instance-profile --instance-profile-name web-server-role \
  --role-name web-server-role
```

**Cleanup:** delete the user, group, role, instance profile and policy when finished.

## Troubleshooting "AccessDenied"

1. **Who am I?** `aws sts get-caller-identity` (right account? right role?).

2. **Read the error message:** it names the action and resource (e.g. `not authorized to perform: s3:GetObject on resource: arn:aws:s3:::bucket/file`).
3. Is there an **Allow** for that exact action **and** resource ARN (bucket vs `bucket/*`)?
4. Is there an explicit **Deny** anywhere (identity policy, bucket policy, SCP, permission boundary)?
5. **Resource policy:** bucket/KMS key policies can block access even when IAM allows it (e.g. encrypted object without `kms:Decrypt`).
6. **Conditions:** MFA required? Wrong source IP or region? Missing tag?
7. Use the **IAM policy simulator** and **CloudTrail** (the event shows the exact denied call).

## 7.8 AWS CLI in depth

### Install and configure

- Install AWS CLI v2 (Windows MSI / `curl` installer on Linux / `brew` on macOS). Check: `aws --version`.
- **Recommended login (SSO):** `aws configure sso` → enter the SSO start URL and region → pick account and role → profile name. Later: `aws sso login --profile dev`.
- **Access-key login (labs only):** `aws configure` → access key, secret key, default region, output format.

**Where settings live:**

| File | Contains |
|---|---|
| `~/.aws/config` | Profiles, regions, output format, SSO settings |
| `~/.aws/credentials` | Access keys (if used). **Never commit or share this file** |

**Which credentials win (simplified order):** command-line options → environment variables (`AWS_ACCESS_KEY_ID`, `AWS_PROFILE`, `AWS_REGION`) → SSO / `credentials` file → container credentials → EC2 instance role.

### Command structure

```
aws  ec2  describe-instances  --region ap-south-1  --output table
|    |    |                   └──────────── options ────────────┘
|    |    └ operation (verb-noun)
|    └ service
└ the CLI
```

### Output and filtering

| Option | Use |
|---|---|
| `--output json \| table \| text \| yaml` | Output format (`text` is best for scripts) |
| `--query '<JMESPath>'` | Filter the output **on your side** |
| `--filters Name=...,Values=...` | Filter **on the AWS side** (supported by many describe commands) |
| `--profile`, `--region` | Choose account/role and region |
| `--dry-run` | EC2: check permissions without doing anything |
| `--no-cli-pager` | Don't open output in a pager |

**JMESPath examples:**

```
# IDs of all instances
aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId' --output text

# Table of ID, state and Name tag
aws ec2 describe-instances --output table --query \
  'Reservations[].Instances[].{ID:InstanceId,State:State.Name,Name:Tags[?Key==`Name`]|[0].Value}'

# Only running instances (filter on the AWS side)
aws ec2 describe-instances --filters Name=instance-state-name,Values=running \
  --query 'Reservations[].Instances[].[InstanceId,PrivateIpAddress]' --output text

# Wait until an instance is running
aws ec2 wait instance-running --instance-ids i-0123456789abcdef0
```

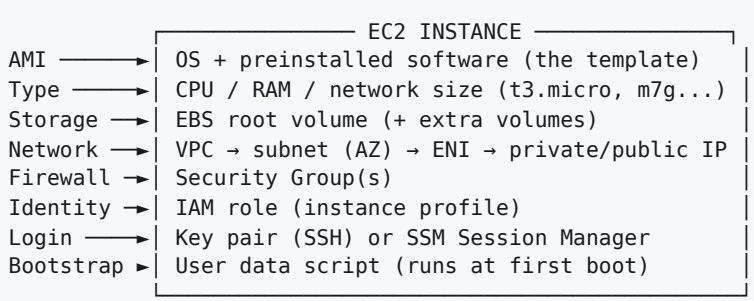
### Commands you'll use constantly

| Command | Purpose |
|---|---|
| `aws sts get-caller-identity` | Which account and identity am I using? |
| `aws configure list` / `aws configure list-profiles` | Current settings / available profiles |
| `aws ec2 describe-regions --output table` | List regions |
| `aws ec2 describe-instances` | List instances |
| `aws s3 ls` | List buckets |
| `aws iam list-roles --query 'Roles[].RoleName'` | List roles |
| `aws logs tail /aws/lambda/my-fn --follow` | Follow CloudWatch logs live |
| `aws ssm start-session --target i-0123...` | Shell on an instance without SSH (needs the Session Manager plugin) |
| `aws <service> help` | Built-in help for any service |

## 7.9 EC2: Elastic Compute Cloud

**What:** virtual servers ("instances") in the cloud. **Why:** full control over the OS and software, any size from tiny to huge, pay per second, start/stop in minutes. **In companies:** web and app servers, CI runners, bastion hosts, legacy apps, Kubernetes worker nodes (EKS nodes are EC2).

### Building blocks of an instance




### Instance types

**Reading a name:** `m7g.xlarge`

| Part | Meaning |
|---|---|
| `m` | **Family** (purpose) |
| `7` | **Generation** (newer = better price/performance) |
| `g` | **Attributes**: `g` = AWS Graviton (ARM), `a` = AMD, `i` = Intel, `d` = local NVMe disk, `n` = more network, `e` = extra memory |
| `xlarge` | **Size**: nano, micro, small, medium, large, xlarge, 2xlarge… (each step roughly doubles CPU/RAM) |

| Family | Optimized for | Examples | Typical use |
|---|---|---|---|
| T (burstable) | Low baseline CPU with bursts (CPU credits) | t3.micro, t4g.small | Dev/test, small websites, labs |
| M (general purpose) | Balanced CPU/RAM | m7i.large, m7g.large | Most app servers |
| C (compute) | High CPU | c7g.xlarge | Batch processing, CI builds, high-traffic web |
| R / X (memory) | High RAM | r7g.2xlarge | Databases, caches, in-memory analytics |
| I / D (storage) | Fast local disks | i4i.xlarge | NoSQL databases, data warehousing |
| P / G / Inf / Trn (accelerated) | GPUs / ML chips | g5.xlarge | Machine learning, video |

**Burstable (T) instances:** earn CPU credits while idle and spend them during bursts. T3/T4g default to **unlimited** mode (extra bursting is charged). Check `CPUCreditBalance` in CloudWatch if a T instance is slow.

**Graviton (ARM)** instances are usually cheaper for the same performance, but your software and container images must support ARM64.

## AMIs (Amazon Machine Images)

| Item | Detail |
|---|---|
| What | Template containing the OS, settings and software for the root volume |
| Sources | AWS (Amazon Linux 2023, Ubuntu, Windows), Marketplace (vendor images, may cost extra), community, **your own** |
| Regional | An AMI ID exists in one region only; copy it to use elsewhere |
| Custom AMI ("golden image") | Install and harden once → create image → launch identical servers fast. Built with tools like Packer or EC2 Image Builder |
| Find latest Amazon Linux | `aws ssm get-parameters --names /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 --query 'Parameters[0].Value' --output text` |

**Default login users:** Amazon Linux / RHEL → `ec2-user` · Ubuntu → `ubuntu` · Debian → `admin`.

## Launch settings explained

| Setting | What to choose and why |
|---|---|
| Name & tags | `Name=web-1`, plus `Project`, `Env`, `Owner` |

| Setting | What to choose and why |
|---|---|
| AMI | Amazon Linux 2023 or Ubuntu LTS |
| Instance type | `t3.micro` / `t4g.micro` for labs |
| Key pair | ED25519 key; download the `.pem` once and `chmod 400` it. Or skip keys and use SSM |
| Network | VPC + subnet (public subnet if it must be reachable from the internet) |
| Auto-assign public IP | Enable only for public servers |
| Security group | Allow only needed ports (80/443 for web; 22 from your IP only, or none with SSM) |
| Storage | gp3 root volume, size as needed, **encrypted** |
| IAM instance profile | A role with only the permissions the app needs (and `AmazonSSMManagedInstanceCore` for Session Manager) |
| Metadata options | **IMDSv2 required** (token-based; protects against SSRF credential theft) |
| User data | Bootstrap script (install packages, start services) |
| Termination protection | Enable for important servers |

## Connecting to an instance

| Method | Needs | Notes |
|---|---|---|
| SSH | Key pair, port 22 open to your IP, public IP (or bastion/VPN) | `ssh -i key.pem ubuntu@<public-ip>` |
| EC2 Instance Connect | Supported AMI, port 22 open to the Instance Connect service range | Browser-based SSH, temporary key pushed for 60 s |
| **SSM Session Manager** | SSM agent (preinstalled on Amazon Linux/Ubuntu AMIs), instance role with `AmazonSSMManagedInstanceCore`, outbound access to SSM (internet/NAT or VPC endpoints) | **No open ports, no keys, sessions logged.** Preferred in companies |
| RDP | Windows instances, port 3389 from your IP | Get the password with the key pair |

## User data and instance metadata

**User data** runs **once, as root, at the first boot** (via cloud-init). Logs: `/var/log/cloud-init-output.log`.

```
#!/bin/bash
dnf install -y nginx                        # Amazon Linux 2023 (use apt-get on Ubuntu)
echo "<h1>Hello from $(hostname)</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

**Instance metadata (IMDS)** at `169.254.169.254` lets software on the instance learn about itself (ID, AZ, IPs) and get role credentials. **IMDSv2** requires a session token:

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
MD=http://169.254.169.254/latest/meta-data
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/instance-id
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/placement/availability-zone
```

## Instance lifecycle and billing

```
launch → pending → RUNNING ⇄ stopping → STOPPED → (start) → pending → RUNNING
                          │
                          └──────── terminate ──────► shutting-down → TERMINATED (gone)
```

| State | Compute billed? | EBS billed? | Notes |
|---|---|---|---|
| Running | Yes | Yes | |
| Stopped | No | **Yes** | Public IP is released (unless Elastic IP); instance store data is lost |
| Hibernated | No | Yes | RAM saved to EBS; resumes faster |
| Terminated | No | Root volume deleted by default ( `DeleteOnTermination` ) | Cannot be undone |

**Reboot** keeps the same host, IPs and data. **Stop/start** usually moves the instance to new hardware (fixes some hardware issues).

## Placement groups

| Type | What | Use |
|---|---|---|
| Cluster | Instances close together in one AZ | Lowest latency, HPC |
| Spread | Each instance on different hardware (max 7 per AZ per group) | A few critical instances |
| Partition | Groups of instances on separate racks | Big distributed systems (Kafka, HDFS, Cassandra) |

## EC2 networking

| Item | Detail |
|---|---|
| ENI (Elastic Network Interface) | Virtual network card. Has private IP(s), SGs, MAC. Primary ENI can't be detached |
| Private IP | Stays for the instance's life |
| Public IP | Assigned at launch if enabled; **changes after stop/start** |
| Elastic IP | Static public IPv4 you own until released. Billed hourly like all public IPv4 |
| Source/destination check | Must be **disabled** on NAT instances / appliances that forward traffic |

## Monitoring an instance

| Check | Meaning | Typical fix |
|---|---|---|
| System status check failed | AWS hardware/host problem | Stop and start (moves host) |
| Instance status check failed | Problem inside the OS (boot failure, full disk, bad network config) | Check the **system log / screenshot**, fix OS config, restore from snapshot |
| Attached EBS status check failed | Volume problem | Check volume health, restore from snapshot |

CloudWatch EC2 metrics by default: CPU, network, disk I/O for instance store, status checks — **not memory or disk space** (install the **CloudWatch agent** for those). Basic monitoring = 5-minute data; detailed = 1-minute (paid).

### ⚠️ Lab: launch a web server with the CLI

Cost: `t3.micro` is small but **not free** outside credits; public IPv4 is billed per hour. Delete everything at the end.

```
REGION=ap-south-1
AMI=$(aws ssm get-parameters --region $REGION \
  --names /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --query 'Parameters[0].Value' --output text)

MYIP=$(curl -s https://checkip.amazonaws.com)/32
VPC=$(aws ec2 describe-vpcs --region $REGION --filters Name=isDefault,Values=true \
  --query 'Vpcs[0].VpcId' --output text)

SG=$(aws ec2 create-security-group --region $REGION --group-name web-lab-sg \
  --description "web lab" --vpc-id $VPC --query GroupId --output text)
aws ec2 authorize-security-group-ingress --region $REGION --group-id $SG \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --region $REGION --group-id $SG \
  --protocol tcp --port 22 --cidr $MYIP

ID=$(aws ec2 run-instances --region $REGION --image-id $AMI --instance-type t3.micro \
  --security-group-ids $SG --user-data file://userdata.sh \
  --metadata-options HttpTokens=required \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-lab}]' \
  --query 'Instances[0].InstanceId' --output text)

aws ec2 wait instance-running --region $REGION --instance-ids $ID
aws ec2 describe-instances --region $REGION --instance-ids $ID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text
# open http://<public-ip> in the browser (give user data ~1 minute)
```

**Cleanup:**

```
aws ec2 terminate-instances --region $REGION --instance-ids $ID
aws ec2 wait instance-terminated --region $REGION --instance-ids $ID
aws ec2 delete-security-group --region $REGION --group-id $SG
```

### EC2 troubleshooting

| Symptom | Check |
|---|---|
| SSH times out | Instance running? Public IP? SG allows 22 from **your current IP**? NACL? Route table has `0.0.0.0/0 → IGW`? Company network blocking 22? |
| `Permission denied (publickey)` | Wrong username (`ec2-user` vs `ubuntu`), wrong key, key file permissions not `400` |
| `UNPROTECTED PRIVATE KEY FILE` | `chmod 400 key.pem` (on Windows: remove other users' access in file properties) |
| Website not loading | SG port 80/443? nginx running (`systemctl status nginx`)? Listening on `0.0.0.0` (`ss -tulpn`)? User data failed (`/var/log/cloud-init-output.log`)? |
| Instance status check failed | Get system log (Actions → Monitor → Get system log), check full disk / fstab errors |

| Symptom | Check |
|---|---|
| Can't launch: `InsufficientInstanceCapacity` | Try another AZ or instance type |
| Can't launch: `VcpuLimitExceeded` | Service Quotas → request an increase |
| Lost key pair | Use SSM Session Manager or EC2 Instance Connect, or attach the root volume to another instance to add a new public key |

## 7.10 Storage for EC2: EBS, instance store, EFS

### EBS (Elastic Block Store)

**What:** network-attached **block** disks for EC2 (like a virtual hard drive). **Key facts:** lives in **one AZ** (attach only to instances in the same AZ) · persists independently of the instance · can be resized online · encrypted with KMS · backed up with **snapshots**.

| Volume type | Kind | Performance model | Use |
|---|---|---|---|
| **gp3** | SSD | Baseline 3,000 IOPS and 125 MB/s at any size; more IOPS/throughput can be provisioned separately | **Default choice**: boot volumes, most apps |
| gp2 | SSD (older) | IOPS grow with size (3 IOPS/GB, burst) | Legacy; migrate to gp3 (cheaper) |
| io2 / io2 Block Express | SSD | Provisioned IOPS, highest durability | Critical databases |
| st1 | HDD | Throughput-optimized | Big sequential data, logs |
| sc1 | HDD | Cheapest, cold | Rarely accessed data |

- **IOPS** = input/output operations per second (many small reads/writes, e.g. databases). **Throughput** = MB/s (large sequential reads/writes).
- **Multi-Attach** (io1/io2 only): attach one volume to several instances in the same AZ (needs a cluster-aware file system).
- **Encryption:** turn on **EBS encryption by default** per region (EC2 settings). Snapshots of encrypted volumes are encrypted.

**Snapshots:**

- Point-in-time backups stored in S3 (managed by AWS), **incremental** (only changed blocks are stored after the first).
- Restore = create a new volume from the snapshot (in **any AZ** of the region), or copy the snapshot to another region (DR).
- Automate with **Data Lifecycle Manager** or **AWS Backup**.
- Old snapshots cost money: set retention.

### ⚠️ Lab: add, format, mount and grow a volume

```
# 1) Create a 10 GiB gp3 volume in the SAME AZ as the instance, then attach it
aws ec2 create-volume --availability-zone ap-south-1a --size 10 --volume-type gp3 --encrypted \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=data-lab}]'
aws ec2 attach-volume --volume-id vol-0123456789abcdef0 \
  --instance-id i-0123456789abcdef0 --device /dev/sdf

# 2) On the instance
lsblk                                  # new disk appears, e.g. nvme1n1 (Nitro names it nvme*)
sudo mkfs -t xfs /dev/nvme1n1          # format (ONLY on a new, empty disk!)
sudo mkdir /data && sudo mount /dev/nvme1n1 /data
df -h /data

# 3) Mount automatically at boot, using the UUID (device names can change)
sudo blkid /dev/nvme1n1                # copy the UUID
echo 'UUID=<uuid> /data xfs defaults,nofail 0 2' | sudo tee -a /etc/fstab
sudo umount /data && sudo mount -a && df -h /data   # test fstab before rebooting

# 4) Grow the volume later (no downtime)
aws ec2 modify-volume --volume-id vol-0123456789abcdef0 --size 20
sudo xfs_growfs /data                  # ext4 would be: sudo resize2fs /dev/nvme1n1
```

**Snapshot:** `aws ec2 create-snapshot --volume-id vol-0123456789abcdef0 --description "data-lab backup"` . **Cleanup:** unmount, remove the fstab line, detach, delete the volume and snapshot.

`nofail` in fstab prevents the instance from failing to boot if the volume is missing.

### Instance store

Physical disks on the host machine (instance types with `d`, e.g. `m6id`). Very fast, **but data is lost on stop, hibernate or terminate** (it survives a reboot). Use only for caches, buffers, scratch data, or replicated data.

### EFS (Elastic File System)

**What:** managed **NFS** file system shared by many instances (and containers/Lambda), across **multiple AZs**, grows and shrinks automatically.

| Setting | Options |
|---|---|
| Storage classes | Standard, Infrequent Access, Archive (lifecycle policies move files automatically) |
| Availability | Regional (multi-AZ) or One Zone (cheaper) |
| Throughput mode | Elastic (recommended), Provisioned, Bursting |
| Access | Mount targets in each AZ subnet; SG must allow **NFS port 2049** from the clients |

Mount: `sudo mount -t efs fs-0123456789abcdef0:/ /mnt/efs` (with `amazon-efs-utils`) or standard NFS mount. Linux only.

| | EBS | Instance store | EFS | S3 |
|---|---|---|---|---|
| Type | Block | Block | File (NFS) | Object |
| Shared by many instances | No (except Multi-Attach) | No | **Yes** | Yes (via API) |
| Survives stop | Yes | **No** | Yes | Yes |
| Scope | One AZ | One host | Region (multi-AZ) | Region |

| | EBS | Instance store | EFS | S3 |
|---|---|---|---|---|
| Typical use | OS, databases | Cache | Shared content, home dirs | Files, backups, static sites, data lakes |

**FSx** (awareness): managed Windows File Server, Lustre (HPC), NetApp ONTAP, OpenZFS.

## 7.11 S3: Simple Storage Service

**What: object storage**: store any number of files ("objects") in containers ("buckets") and access them over HTTPS/API. **Why:** practically unlimited, very durable (designed for 99.999999999% = "11 nines"), cheap, integrated with almost every AWS service. **In companies:** static websites and assets, backups, logs, data lakes, Terraform state, CI artifacts, file uploads from apps.

### Core concepts

| Term | Meaning |
|---|---|
| Bucket | Container for objects. Name is **globally unique**, 3–63 characters, lowercase letters, numbers, hyphens, dots. Created in one region |
| Object | A file + metadata. Up to 5 TB each |
| Key | The object's full name: `logs/2026/10/app.log`. "Folders" are just key **prefixes** |
| Version ID | Present when versioning is on |
| Consistency | **Strong read-after-write** consistency for all operations |

**Uploads:** single PUT up to 5 GB; use **multipart upload** for large files (recommended above ~100 MB; the CLI does it automatically).

### Storage classes

| Class | Availability zones | Min. storage duration | Retrieval | Use |
|---|---|---|---|---|
| Standard | 3+ AZs | — | Instant | Frequently accessed data |
| Intelligent-Tiering | 3+ AZs | — | Instant (archive tiers optional) | Unknown or changing access patterns |
| Standard-IA | 3+ AZs | 30 days | Instant, per-GB retrieval fee | Infrequent but needs fast access (backups) |
| One Zone-IA | **1 AZ** | 30 days | Instant, retrieval fee | Re-creatable infrequent data |
| Glacier Instant Retrieval | 3+ AZs | 90 days | Milliseconds | Archives accessed ~quarterly |
| Glacier Flexible Retrieval | 3+ AZs | 90 days | Minutes to hours | Archives, DR copies |
| Glacier Deep Archive | 3+ AZs | 180 days | Within ~12–48 hours | Long-term compliance archives (cheapest) |

### Lifecycle rules

Automatically **transition** objects to cheaper classes or **expire** (delete) them after N days.

```
{
  "Rules": [{
    "ID": "logs-retention",
    "Filter": { "Prefix": "logs/" },
    "Status": "Enabled",
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" }
    ],
    "Expiration": { "Days": 365 },
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 }
  }]
}
```

Apply: `aws s3api put-bucket-lifecycle-configuration --bucket <bucket> --lifecycle-configuration file://lifecycle.json`.

## Versioning, Object Lock, replication

| Feature | What it does | Use |
|---|---|---|
| Versioning | Keeps every version; a delete adds a "delete marker" instead of removing data | Protect against accidental overwrite/delete. **Turn on for important buckets** (and Terraform state) |
| MFA Delete | Requires MFA to permanently delete versions or change versioning | Extra protection (root only) |
| Object Lock | Write-once-read-many (WORM): objects can't be deleted until a retention date | Compliance, ransomware protection |
| Replication (CRR / SRR) | Copy new objects automatically to another bucket in another (CRR) or same (SRR) region | DR, compliance, log aggregation. Requires versioning on both buckets |

## Security and access control

**Layers that decide access:**

```
Block Public Access (account + bucket)  ── overrides anything that would make data public
        |
IAM policies (identity-based)            ── what users/roles can do
Bucket policy (resource-based)           ── who can access this bucket (other accounts, services)
ACLs                                     ── legacy; disabled by default (bucket owner enforced)
        |
Encryption (SSE-S3 default, SSE-KMS)     ── KMS key policy must also allow decrypt for SSE-KMS
```

| Control | Detail |
|---|---|
| Block Public Access | **On by default** for new buckets. Leave it on unless the bucket truly must be public |
| Bucket policy | JSON with `Principal`. E.g. allow only CloudFront, enforce HTTPS, enforce encryption |
| Default encryption | All new objects are encrypted (SSE-S3 by default). Use **SSE-KMS** for audit and key control; **DSSE-KMS** for dual-layer; SSE-C = your own key per request |
| Presigned URL | Temporary link to one private object: `aws s3 presign s3://<bucket>/report.pdf --expires-in 3600` |
| Access points | Named entry points with their own policy for large shared buckets |

| Control | Detail |
|---|---|
| VPC gateway endpoint | Private S3 access from a VPC without internet/NAT |

**Bucket policy: deny any request that is not HTTPS:**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyInsecureTransport",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": ["arn:aws:s3:::<bucket>", "arn:aws:s3:::<bucket>/*"],
    "Condition": { "Bool": { "aws:SecureTransport": "false" } }
  }]
}
```

## Static website hosting

| Approach | How | When |
|---|---|---|
| S3 website endpoint | Enable static website hosting, make objects public (Block Public Access off for that bucket + public-read bucket policy) | Quick labs only. HTTP only |
| **S3 + CloudFront (recommended)** | Bucket stays **private**; CloudFront with **Origin Access Control (OAC)** reads it; HTTPS with ACM certificate; custom domain via Route 53 | Production static sites |

## Other features

| Feature | Use |
|---|---|
| Event notifications | Trigger Lambda/SQS/SNS/EventBridge on upload/delete (e.g. make thumbnails) |
| Transfer Acceleration | Faster long-distance uploads via edge locations |
| Server access logs / CloudTrail data events | Who accessed what |
| S3 Inventory / Storage Lens | Reports and analytics about your storage |
| Requester Pays | The downloader pays transfer costs |

**Pricing factors:** storage (GB-month by class) + requests (PUT/GET) + data transfer out + retrieval fees (IA/Glacier) + features (replication, Intelligent-Tiering monitoring).

## S3 CLI

| Command | Purpose |
|---|---|
| `aws s3 mb s3://<bucket> --region ap-south-1` | Make bucket |
| `aws s3 ls` / `aws s3 ls s3://<bucket>/logs/` | List buckets / objects under a prefix |
| `aws s3 cp file.txt s3://<bucket>/docs/` | Upload ( `cp s3://... .` to download) |
| `aws s3 cp dir s3://<bucket>/dir --recursive` | Upload a folder |

| Command | Purpose |
|---|---|
| `aws s3 sync ./site s3://<bucket> --delete` | Make the bucket match a local folder |
| `aws s3 mv`, `aws s3 rm s3://<bucket>/file` | Move / delete |
| `aws s3 rb s3://<bucket> --force` | Delete bucket and all objects (not old versions) |
| `aws s3api put-bucket-versioning --bucket <bucket> --versioning-configuration Status=Enabled` | Turn on versioning |
| `aws s3api get-bucket-policy --bucket <bucket>` | Show bucket policy |
| `aws s3 presign s3://<bucket>/file --expires-in 600` | Temporary download link |

`aws s3` = high-level, file-like commands. `aws s3api` = every S3 API operation (settings, policies, versions).

### ⚠️ Lab: private bucket, versioning, lifecycle (almost free)

1. `aws s3 mb s3://<yourname>-lab-<random>` (Block Public Access stays on).
2. Enable versioning (command above). Upload `notes.txt` twice with different content.
3. `aws s3api list-object-versions --bucket <bucket>` → see two versions. Delete the object and list again → see the **delete marker**. Restore by deleting the delete marker version.
4. Apply the lifecycle JSON above.
5. Create a presigned URL and open it in a browser; open the plain object URL → **AccessDenied** (private).
6. **Cleanup:** delete all versions (console: Empty bucket) then `aws s3 rb s3://<bucket>`.

### S3 troubleshooting

| Symptom | Likely cause |
|---|---|
| `AccessDenied` reading an object | IAM missing `s3:GetObject` on `bucket/*`; bucket policy Deny; object encrypted with KMS and no `kms:Decrypt`; object owned by another account (old ACL setups) |
| `AccessDenied` listing | Missing `s3:ListBucket` on the **bucket** ARN (not `/*`) |
| Website 403 | Block Public Access on / no public bucket policy (or use CloudFront OAC) / missing `index.html` |
| `BucketAlreadyExists` | Name taken by someone else globally: choose a unique name |
| Bucket won't delete | Not empty, or old versions/delete markers remain |
| Unexpected costs | Old versions without lifecycle rules, Glacier minimum duration fees, data transfer out |

## 7.12 VPC: Virtual Private Cloud

**What:** your own private, isolated network inside an AWS region. You choose the IP range, subnets, routing and firewalls. **Why:** every EC2 instance, RDS database, load balancer, Lambda-in-VPC and EKS cluster lives in a VPC. Network design decides security, availability and cost. **In companies:** each environment (dev/staging/prod) usually has its own VPC, often in its own account, connected through Transit Gateway.

## Reference design (2 AZs, public + private + database tiers)

```
Region ap-south-1 — VPC 10.0.0.0/16
│
├─ Internet Gateway (IGW) ◄──────────────── internet
│
├─ AZ a                                    AZ b
│  ├─ Public  10.0.1.0/24                 ├─ Public  10.0.2.0/24
│  │   ALB node, NAT Gateway a             │   ALB node, NAT Gateway b
│  │   route: 0.0.0.0/0 → IGW               │   route: 0.0.0.0/0 → IGW
│  ├─ Private 10.0.11.0/24 (app)           ├─ Private 10.0.12.0/24 (app)
│  │   EC2 / ECS / EKS nodes                │   EC2 / ECS / EKS nodes
│  │   route: 0.0.0.0/0 → NAT a             │   route: 0.0.0.0/0 → NAT b
│  └─ Private 10.0.21.0/24 (data)           └─ Private 10.0.22.0/24 (data)
│      RDS primary                             RDS standby
│      no internet route                       no internet route
│
└─ Gateway endpoint for S3 (private S3 access, free)
```

## CIDR planning

| Rule | Detail |
|---|---|
| VPC size | Between `/16` (65,536 IPs) and `/28` (16 IPs). `/16` is common |
| Private ranges | Use RFC 1918: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` |
| **No overlaps** | Never overlap with other VPCs, on-premises networks or VPNs you may connect later. Peering/TGW/VPN can't route overlapping ranges |
| Secondary CIDRs | Extra ranges can be added to a VPC later |
| Reserved IPs | AWS reserves **5 IPs per subnet**: `.0` network, `.1` VPC router, `.2` DNS, `.3` future use, last = broadcast. A `/24` gives **251** usable |
| Leave room | Allocate bigger blocks than you need now; EKS consumes many IPs (one per pod with the VPC CNI) |

**Default VPC:** every region has one (`172.31.0.0/16`, a public `/20` subnet per AZ, IGW attached). Fine for quick labs; build your own VPCs for real work.

## Subnets

- A subnet is a slice of the VPC CIDR in **exactly one AZ**.
- **Public subnet** = its route table has `0.0.0.0/0 → Internet Gateway`. Instances also need a public IP to be reachable.
- **Private subnet** = no route to an IGW. Outbound internet (updates, APIs) goes through a **NAT Gateway**.
- **Isolated subnet** = no internet route at all (databases). Use VPC endpoints for AWS services.
- Setting `map-public-ip-on-launch` gives instances a public IP automatically (public subnets only).

## Route tables

| Concept | Detail |
|---|---|
| Route table | Rules: destination CIDR → target |
| `local` route | Automatic route for the VPC CIDR; all subnets can reach each other (SGs/NACLs still apply) |

| Concept | Detail |
|---|---|
| Main route table | Used by subnets without an explicit association. Keep it private; create separate public tables |
| Association | Each subnet uses exactly one route table |
| Longest prefix wins | `10.0.5.0/24` beats `10.0.0.0/16` beats `0.0.0.0/0` |
| Targets | `igw-`, `nat-`, `pcx-` (peering), `tgw-`, `vpce-` (gateway endpoint), `eni-`, virtual private gateway ( `vgw-` ) |

## Internet Gateway and NAT Gateway

| | Internet Gateway | NAT Gateway |
|---|---|---|
| Purpose | Two-way internet access for resources with public IPs | **Outbound-only** internet for private subnets |
| Placement | Attached to the VPC (one per VPC) | Inside a **public** subnet, with an Elastic IP |
| Cost | Free (data transfer charged) | **Hourly + per GB processed**: one of the most common surprise bills |
| HA | Built-in | Zonal: create **one per AZ** for HA, each private subnet routes to the NAT in its own AZ |

A **NAT instance** (EC2 doing NAT, source/dest check disabled) is cheaper but you manage HA, patching and scaling: only for labs or very low budgets. For IPv6, use an **egress-only internet gateway** for outbound-only access.

## Security Groups (SG) in depth

| Property | Detail |
|---|---|
| Level | Attached to network interfaces (instances, ALBs, RDS, Lambda ENIs…) |
| Stateful | Return traffic is automatically allowed |
| Rules | **Allow only.** Default: no inbound, all outbound |
| Sources | CIDR, prefix list, or **another security group** (best practice for tiers) |
| Evaluation | All rules are combined; no order |

**Tiered SG design (SG references instead of IPs):**

| SG | Inbound rule |
|---|---|
| `alb-sg` | 443 (and 80 for redirect) from `0.0.0.0/0` |
| `app-sg` | 8080 from **`alb-sg`** |
| `db-sg` | 5432 from **`app-sg`** |
| `bastion` | Not needed: use SSM Session Manager |

## Network ACLs (NACL) in depth

| Property | Detail |
|---|---|
| Level | Subnet boundary |

| Property | Detail |
|---|---|
| Stateless | Return traffic needs its own rule |
| Rules | Allow **and** Deny, numbered; **lowest number first, first match wins**; final `*` rule denies |
| Default NACL | Allows everything in and out |
| Custom NACL | Denies everything until you add rules |
| Ephemeral ports | Return traffic to clients uses ports **1024–65535**, so outbound (for servers) and inbound (for clients) rules must allow them |

**Example NACL for a public web subnet:**

| Direction | Rule # | Protocol | Port | Source/Dest | Action |
|---|---|---|---|---|---|
| Inbound | 100 | TCP | 443 | 0.0.0.0/0 | Allow |
| Inbound | 110 | TCP | 1024–65535 | 0.0.0.0/0 | Allow (responses to outbound calls) |
| Inbound | 120 | TCP | 22 | 198.51.100.7/32 | Deny (block a bad actor before broad allows) |
| Outbound | 100 | TCP | 1024–65535 | 0.0.0.0/0 | Allow (responses to clients) |
| Outbound | 110 | TCP | 443 | 0.0.0.0/0 | Allow (calls to the internet) |

**Use NACLs** for coarse subnet-wide blocks (deny an IP range); do day-to-day filtering with SGs.

## VPC endpoints (private access to AWS services)

| Type | Services | How | Cost |
|---|---|---|---|
| **Gateway endpoint** | **S3 and DynamoDB only** | Adds a route in the route table | Free |
| **Interface endpoint (PrivateLink)** | Most services (ECR, SSM, Secrets Manager, STS, CloudWatch, KMS…) and your own services | Creates ENIs with private IPs in your subnets; private DNS makes the normal service name resolve to them | Hourly per AZ + per GB |

**Why:** keep traffic off the internet, reduce NAT Gateway costs, allow fully isolated subnets. Endpoint policies can restrict what's reachable (e.g. only your buckets).

## Connecting VPCs and networks

| Option | What | Notes |
|---|---|---|
| VPC peering | Private connection between two VPCs (same/different account or region) | **Not transitive** (A↔B and B↔C does not give A↔C); CIDRs must not overlap; update route tables on both sides |
| Transit Gateway (TGW) | Regional hub connecting many VPCs, VPNs and Direct Connect | Hub-and-spoke, transitive routing, route tables for segmentation. Standard for multi-account networks |
| PrivateLink | Expose one service privately to other VPCs/accounts via an NLB | Consumer creates an interface endpoint; no routing between whole networks |

| Option | What | Notes |
|---|---|---|
| Site-to-Site VPN | IPsec tunnels over the internet to on-premises (2 tunnels per connection) | Quick to set up; terminates on a virtual private gateway or TGW; static or BGP routing |
| Direct Connect (DX) | Dedicated private fibre link to AWS | Consistent latency, high bandwidth, weeks to provision; combine with VPN for encryption/backup |
| Client VPN | Managed OpenVPN for users | Remote access to VPCs |
| Cloud WAN | Managed global network across regions | Large enterprises |

## DNS inside a VPC

- The **Route 53 Resolver** at VPC base + 2 (e.g. `10.0.0.2`) answers DNS for instances.
- VPC settings: `enableDnsSupport` (use the resolver) and `enableDnsHostnames` (give instances DNS names). Both must be on for private hosted zones and interface endpoint private DNS.
- **Private hosted zones**: internal names like `db.internal.shop` visible only in associated VPCs.
- **Resolver endpoints** (inbound/outbound) + forwarding rules connect DNS between AWS and on-premises.

## IPv6 in a VPC

Add an Amazon-provided IPv6 `/56` to the VPC and `/64` to subnets. All IPv6 addresses are public, so use **security groups** and an **egress-only IGW** for outbound-only access. Dual-stack (IPv4 + IPv6) is common.

## Visibility and troubleshooting tools

| Tool | Use |
|---|---|
| **VPC Flow Logs** | Records accepted/rejected traffic per ENI/subnet/VPC to CloudWatch Logs or S3. Fields include srcaddr, dstaddr, srcport, dstport, protocol, action (ACCEPT/REJECT) |
| **Reachability Analyzer** | Tells you whether A can reach B and which component blocks it (SG, NACL, route) |
| Network Access Analyzer | Finds unintended network paths (e.g. databases reachable from the internet) |
| Traffic Mirroring | Copy packets to an analysis appliance (IDS, Wireshark) |

**Flow log line example:** `2 123456789012 eni-0a1b2c3d 10.0.1.25 10.0.11.40 49152 8080 6 10 840 1700000000 1700000060 REJECT OK` → TCP (6) from 10.0.1.25 to port 8080 on 10.0.11.40 was **rejected** → check `app-sg` allows 8080 from the ALB.

### ⚠️ Lab: build a VPC with the CLI (free except NAT if you add it)

```
R=ap-south-1
VPC=$(aws ec2 create-vpc --region $R --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=lab-vpc}]' \
  --query Vpc.VpcId --output text)
aws ec2 modify-vpc-attribute --region $R --vpc-id $VPC --enable-dns-hostnames '{"Value":true}'

PUB=$(aws ec2 create-subnet --region $R --vpc-id $VPC --cidr-block 10.0.1.0/24 \
  --availability-zone ${R}a --query Subnet.SubnetId --output text)
PRIV=$(aws ec2 create-subnet --region $R --vpc-id $VPC --cidr-block 10.0.11.0/24 \
  --availability-zone ${R}a --query Subnet.SubnetId --output text)
aws ec2 modify-subnet-attribute --region $R --subnet-id $PUB --map-public-ip-on-launch

IGW=$(aws ec2 create-internet-gateway --region $R \
  --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --region $R --internet-gateway-id $IGW --vpc-id $VPC

RT=$(aws ec2 create-route-table --region $R --vpc-id $VPC \
  --query RouteTable.RouteTableId --output text)
aws ec2 create-route --region $R --route-table-id $RT \
  --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
aws ec2 associate-route-table --region $R --route-table-id $RT --subnet-id $PUB
```

**Test:** launch one instance in `$PUB` (gets a public IP, reachable) and one in `$PRIV` (no public IP; reach it from the public one by private IP). From the private instance, `curl https://example.com` fails → no NAT. Optionally add a NAT Gateway (⚠️ hourly cost) in `$PUB`, route `0.0.0.0/0` from the private route table to it, test, and delete it.

**Cleanup (reverse order):** terminate instances → delete NAT Gateway and release its EIP → disassociate/delete the route table → detach and delete the IGW → delete subnets → delete the VPC.

### VPC troubleshooting

| Symptom | Check in order |
|---|---|
| Instance not reachable from internet | Public subnet route to IGW · public IP/EIP · SG inbound · NACL inbound **and** outbound ephemeral · OS firewall · service listening |
| Private instance can't reach internet | Route `0.0.0.0/0 → nat-` in **its** route table · NAT in a **public** subnet · NAT subnet routes to IGW · SG/NACL outbound · NAT state "available" |
| Peered VPCs can't talk | Routes on **both** sides · SGs allow the peer CIDR · no overlapping CIDRs · DNS resolution option for peering |
| Can't reach S3 privately | Gateway endpoint route on that subnet's route table · endpoint policy · bucket policy conditions ( `aws:sourceVpce` ) |
| Interface endpoint not used | Private DNS enabled? VPC DNS settings on? Endpoint SG allows 443 from clients? |
| Out of IPs | Subnet too small (remember 5 reserved, EKS uses many) → add a secondary CIDR / bigger subnets |

## 7.13 Elastic Load Balancing (ELB)

**What:** managed load balancers that spread traffic across many targets and send traffic only to **healthy** ones. **Why:** high availability (multi-AZ), scaling, TLS termination, a single stable entry point.

## Types

| Type | Layer | Protocols | Key features | Use |
|---|---|---|---|---|
| **ALB** (Application) | 7 | HTTP, HTTPS, gRPC, WebSocket | Path/host/header/query routing, redirects, fixed responses, auth (Cognito/OIDC), Lambda targets, WAF | Web apps, APIs, microservices, Kubernetes Ingress |
| **NLB** (Network) | 4 | TCP, UDP, TLS | Millions of requests/s, very low latency, **static IP per AZ** / Elastic IPs, preserves client IP, PrivateLink | Non-HTTP protocols, extreme performance, fixed IPs |
| GWLB (Gateway) | 3 | IP (GENEVE 6081) | Inserts firewall/IDS appliances transparently | Security appliances |
| CLB (Classic) | 4/7 | Legacy | — | Migrate away |

## Components

```
Client ──► DNS name (my-alb-123.ap-south-1.elb.amazonaws.com)
              │
         LISTENER  (HTTPS :443, certificate from ACM)
            │ RULES (evaluated by priority)
            │   host = api.shop.com      → TG-api
            │   path = /images/*         → TG-images
            │   default                  → TG-web
            ▼
       TARGET GROUP (protocol/port, health check) ──► targets: instances | IPs | Lambda
```

| Component | Detail |
|---|---|
| Listener | Port + protocol (+ TLS certificate & security policy for HTTPS) |
| Rules (ALB) | Conditions (host, path, header, method, query, source IP) → actions (forward, redirect, fixed-response, authenticate) |
| Target group | Group of targets + health check. Target types: **instance**, **ip** (containers, on-prem), **lambda**, **alb** (for NLB) |
| Health check | Protocol, path (e.g. `/health`), port, interval (default 30 s), timeout, healthy/unhealthy thresholds, success codes |
| Deregistration delay | Time to finish in-flight requests before removing a target (default 300 s; often lowered to 30–60 s) |
| Stickiness | Sends a user to the same target (cookie). Avoid if the app can be stateless |
| Cross-zone load balancing | Spread evenly across targets in all AZs (on by default for ALB) |
| Access logs | Per-request logs to S3 |

**Client IP behind an ALB:** the app sees the ALB's IP; the real client IP is in the `X-Forwarded-For` header (and `X-Forwarded-Proto` tells HTTP vs HTTPS).

**HTTPS best practice:** listener 443 with an **ACM** certificate, listener 80 with a **redirect to 443**, modern TLS security policy, SG of targets allows traffic only from the ALB SG.

### ALB error codes

| Code | Meaning | Usual cause |
|---|---|---|
| 502 Bad Gateway | Target sent an invalid response or closed the connection | App crashed, wrong port, keep-alive timeout shorter than ALB idle timeout |
| 503 Service Unavailable | **No healthy targets** in the target group | All targets failing health checks, or none registered |
| 504 Gateway Timeout | Target didn't answer in time | Slow app/database, idle timeout (default 60 s) too short, SG/NACL dropping traffic |
| 460 / 463 | Client closed connection / too many IPs in X-Forwarded-For | Client side |

### ⚠️ Lab: ALB in front of two instances (hourly cost: delete after)

1. Launch two web instances (user data from 7.9) in **two different AZs**, SG `app-sg` allowing 80 from `alb-sg`.
2. Create target group `web-tg` (HTTP 80, health check `/`) and register both instances.
3. Create an internet-facing ALB in two public subnets with `alb-sg` (80 from anywhere), listener 80 → `web-tg`.
4. Open the ALB DNS name and refresh: the hostname alternates between the two instances.
5. Stop nginx on one instance → after the unhealthy threshold, all traffic goes to the other. Start it again.
6. **Cleanup:** delete ALB → target group → instances → security groups.

## 7.14 EC2 Auto Scaling

**What:** keeps the right number of instances running: replaces unhealthy ones and adds/removes instances based on demand. **Why:** self-healing, high availability, cost saving (scale in when quiet).

| Component | Detail |
|---|---|
| **Launch template** | Versioned recipe: AMI, instance type, key, SGs, IAM profile, user data, metadata options, tags |
| **Auto Scaling Group (ASG)** | Min / desired / max capacity, subnets (spread across AZs), target groups, health check type |
| Health check type | `EC2` (status checks only) or `ELB` (also the load balancer health check: recommended with an ALB) |
| Health check grace period | Seconds to wait after launch before health checks count (time for user data to finish) |
| Instance warmup / cooldown | Prevents scaling again before new instances are useful |
| Termination policy | Which instance to remove first (default: balance AZs, then oldest launch template…) |
| Lifecycle hooks | Pause instances on launch/terminate to run scripts (e.g. drain, copy logs) |
| Instance refresh | Rolling replacement of all instances with a new launch template version (deployments) |
| Warm pools | Pre-initialized stopped instances for faster scale-out |
| Mixed instances policy | Several instance types + On-Demand/Spot mix |

### Scaling policies

| Policy | How | Example |
|---|---|---|
| **Target tracking** (recommended) | Keep a metric at a target value | Average CPU = 50%, or ALB requests per target = 1,000 |
| Step scaling | Different adjustments by alarm breach size | +1 at 70% CPU, +3 at 90% |
| Simple scaling | One adjustment per alarm + cooldown | Legacy |
| Scheduled | At specific times | Scale to 10 at 08:00 weekdays, back to 2 at 20:00 |
| Predictive | ML forecast from history | Daily traffic patterns |

```
aws autoscaling put-scaling-policy --auto-scaling-group-name web-asg \
  --policy-name cpu50 --policy-type TargetTrackingScaling \
  --target-tracking-configuration file://cpu50.json

# cpu50.json
# {
#   "PredefinedMetricSpecification": { "PredefinedMetricType": "ASGAverageCPUUtilization" },
#   "TargetValue": 50.0
# }
```

**Golden rule:** instances in an ASG must be **disposable**: no data stored only on the instance (use RDS, S3, EFS), configuration from user data / AMI, logs shipped to CloudWatch.

### ASG troubleshooting

| Symptom | Cause |
|---|---|
| Instances launch and terminate in a loop | Failing ELB health checks (app not ready, wrong health check path/port, grace period too short) |
| Not scaling out | Max capacity reached, metric not breaching, cooldown, vCPU quota, no capacity for the type |
| Launch failures | Check **Activity history**: invalid AMI, missing IAM permission for the instance profile, subnet out of IPs |

## 7.15 Route 53 (DNS)

**What:** AWS's highly available DNS service, domain registrar, and health checker (port 53 → "Route 53").

| Concept | Detail |
|---|---|
| Hosted zone | Records for a domain. **Public** (internet) or **private** (inside associated VPCs) |
| NS / SOA | Created automatically; set the NS records at your registrar if the domain is registered elsewhere |
| Record types | A, AAAA, CNAME, MX, TXT, NS, SOA, PTR, SRV, CAA |
| **Alias record** | AWS extension: points to AWS resources (ALB, CloudFront, S3 website, API Gateway, another record). Works at the **zone apex** ( `shop.com` ), free queries, no TTL to set |
| CNAME | Alias name → another name. **Not allowed at the zone apex** |

| Concept | Detail |
|---|---|
| TTL | Seconds resolvers cache the answer. Lower it (e.g. 60) before migrations |

### Routing policies

| Policy | Behaviour | Use |
|---|---|---|
| Simple | One record, one or more values | Single resource |
| Weighted | Split by weights (e.g. 90/10) | Canary releases, A/B tests |
| Latency | Lowest-latency region for the user | Multi-region apps |
| Failover | Primary/secondary with health checks | Active-passive DR |
| Geolocation | By user's country/continent | Localized content, legal restrictions |
| Geoproximity | By distance with a bias | Shift traffic between regions |
| Multivalue answer | Up to 8 healthy random records | Simple client-side load spreading |
| IP-based | By client IP ranges (CIDR) | ISP-specific routing |

**Health checks** monitor an endpoint (HTTP/HTTPS/TCP), another health check, or a CloudWatch alarm; failover policies use them.

```
aws route53 list-hosted-zones
aws route53 list-resource-record-sets --hosted-zone-id Z0123456789ABC
dig +short app.shop.com            # verify from your machine
```

## 7.16 CloudFront and Global Accelerator

### CloudFront (CDN)

**What:** content delivery network caching content at hundreds of edge locations. **Why:** faster for users worldwide, less load on origins, HTTPS everywhere, DDoS protection (Shield Standard), WAF integration.

| Concept | Detail |
|---|---|
| Distribution | The CDN configuration with a domain like `d111111abcdef8.cloudfront.net` (+ your own domain via ACM certificate in **us-east-1**) |
| Origin | Where content comes from: S3 bucket, ALB, EC2, API Gateway, any HTTP server |
| Origin Access Control (OAC) | Lets CloudFront read a **private** S3 bucket (bucket policy allows only that distribution) |
| Behaviors | Path patterns ( `/api/*` , `/static/*` ) → origin + cache settings |
| Cache policy / TTL | What's part of the cache key (headers, cookies, query strings) and how long to cache |
| Invalidation | Remove cached files early: `aws cloudfront create-invalidation --distribution-id E123 --paths "/index.html"` (prefer versioned file names) |
| Signed URLs / cookies | Private content for paying/logged-in users |
| Edge functions | CloudFront Functions (lightweight header/URL logic) and Lambda@Edge (heavier logic) |

### Global Accelerator

Two static **anycast** IPs that route users over the AWS global network to the best healthy regional endpoint (ALB, NLB, EC2). For non-HTTP traffic, fixed IPs, and fast regional failover. CloudFront = caching for HTTP; Global Accelerator = network path optimization, no caching.

## 7.17 Databases on AWS

### Choosing a database

| Need | Service | Type |
|---|---|---|
| Relational (SQL), standard engines | **RDS** (MySQL, PostgreSQL, MariaDB, SQL Server, Oracle, Db2) | Managed relational |
| Relational, highest performance/availability | **Aurora** (MySQL/PostgreSQL compatible) | Cloud-native relational |
| Key-value / document at any scale, serverless | **DynamoDB** | NoSQL |
| In-memory cache, sessions, leaderboards | **ElastiCache** (Redis OSS / Valkey / Memcached), **MemoryDB** (durable Redis) | In-memory |
| MongoDB-compatible documents | DocumentDB | Document |
| Graph (relationships, fraud, social) | Neptune | Graph |
| Cassandra-compatible wide-column | Keyspaces | Wide-column |
| Time series (IoT, metrics) | Timestream | Time series |
| Search and log analytics | OpenSearch Service | Search engine |
| Data warehouse (analytics on huge data) | Redshift | Columnar warehouse |

**Managed vs self-managed:** you *can* install a database on EC2, but managed services handle patching, backups, failover, replication and monitoring. Choose self-managed only for special requirements.

### RDS (Relational Database Service)

| Topic | Detail |
|---|---|
| Instance class | `db.t4g.micro` (labs), `db.m7g` (general), `db.r7g` (memory-heavy) |
| Storage | gp3 / io2, **storage autoscaling** up to a max you set |
| Network | **DB subnet group** (private subnets in 2+ AZs), SG allowing the DB port only from the app SG, **Publicly accessible = No** |
| **Multi-AZ (instance)** | Synchronous **standby** in another AZ, **automatic failover** (typically 1–2 minutes); the standby is **not readable**. For **availability** |
| Multi-AZ DB cluster | Writer + two **readable** standbys in 3 AZs, faster failover (MySQL/PostgreSQL) |
| **Read replicas** | Asynchronous copies (same or other region) for **read scaling** and DR; can be promoted to standalone |
| Automated backups | Daily snapshot + transaction logs → **point-in-time restore (PITR)** to any second in the retention window (up to 35 days) |

| Topic | Detail |
|---|---|
| Manual snapshots | Kept until you delete them; copy across regions/accounts |
| Restore | Always creates a **new** DB instance (new endpoint) |
| Parameter group | Engine settings (e.g. `max_connections`) |
| Option group | Engine features (Oracle/SQL Server options) |
| Maintenance window | When AWS applies patches/upgrades |
| Encryption | At rest with KMS (choose at creation; to encrypt an existing DB: snapshot → copy encrypted → restore); in transit with TLS |
| Authentication | Password (store in **Secrets Manager**, with rotation), or **IAM database authentication** |
| RDS Proxy | Connection pooling: protects the DB from too many connections (Lambda, spiky apps); faster failover |
| Monitoring | CloudWatch metrics, **Performance Insights / Database Insights** (top SQL, waits), Enhanced Monitoring (OS level), engine logs to CloudWatch |

**Endpoint usage:** the app connects to the **DB endpoint DNS name** (e.g. `shop-db.abc123.ap-south-1.rds.amazonaws.com:5432`), never an IP: the name follows failover.

| Multi-AZ vs read replica | Multi-AZ | Read replica |
|---|---|---|
| Purpose | High availability | Read scaling (and DR) |
| Replication | Synchronous | Asynchronous (replica lag) |
| Readable | No (instance deployment) | Yes |
| Failover | Automatic | Manual promotion |

## Aurora

- Storage is a shared cluster volume with **6 copies across 3 AZs**, grows automatically.
- Up to **15 Aurora Replicas** sharing the same storage (low lag); automatic failover to a replica (usually under ~30 s).
- Endpoints: **cluster (writer)** endpoint, **reader** endpoint (load-balances replicas), custom endpoints.
- **Aurora Serverless v2:** capacity scales in fine-grained steps automatically (ACUs).
- **Global Database:** cross-region replication with fast cross-region recovery.
- Backtrack (MySQL), fast cloning.

## DynamoDB

| Concept | Detail |
|---|---|
| Table / item / attribute | Item = row (max 400 KB), schemaless except the key |
| **Partition key** | Required; decides which partition stores the item. Choose a key with **many distinct values** (e.g. `userId`) to avoid hot partitions |
| Sort key | Optional; items with the same partition key are sorted by it (e.g. `orderDate`) |
| Capacity modes | **On-demand** (pay per request, no planning) or **Provisioned** (RCU/WCU with auto scaling, cheaper for steady traffic) |

| Concept | Detail |
|---|---|
| Reads | `GetItem` (one item), `Query` (by partition key, efficient), `Scan` (whole table, expensive: avoid) |
| Consistency | Eventually consistent reads by default; strongly consistent option |
| Indexes | **GSI** (different partition/sort key, any time) · **LSI** (same partition key, different sort key, only at creation) |
| Features | TTL (auto-delete expired items), **Streams** (change events → Lambda), PITR backups, Global Tables (multi-region active-active), transactions, DAX (in-memory cache) |

**Design rule:** model DynamoDB around your **access patterns** (the queries you need), not around normalized tables.

## ElastiCache

| | Redis OSS / Valkey | Memcached |
|---|---|---|
| Data structures | Strings, hashes, lists, sets, sorted sets, streams | Simple key-value |
| Persistence / replication | Yes (replicas, Multi-AZ failover, snapshots) | No |
| Use | Sessions, leaderboards, queues, caching, pub/sub | Simple, multi-threaded caching |

**Caching patterns:** *lazy loading / cache-aside* (read cache → on miss read DB and fill cache) and *write-through* (write DB and cache together). Always set a **TTL**.

## ⚠️ Lab: private RDS PostgreSQL reachable only from an app instance

1. Create SG `db-sg` allowing **5432 from** `app-sg` (not from the internet).
2. Create a DB subnet group with two **private** subnets in different AZs.
3. Create RDS PostgreSQL: `db.t4g.micro`, 20 GB gp3, single-AZ (lab), **Publicly accessible: No**, `db-sg`, manage the master password in **Secrets Manager**, backups 1 day.
4. From the app instance: `sudo dnf install -y postgresql16` then `psql -h <db-endpoint> -U postgres -d postgres` → `SELECT version();`.
5. From your laptop the endpoint does **not** connect: correct, it's private.
6. **Cleanup:** delete the DB (skip final snapshot for labs), delete its automated backups, the subnet group, the secret and the SG.

## Database connectivity troubleshooting

| Symptom | Check |
|---|---|
| Connection timeout | SG on the DB allows the port from the **app's SG**? Same VPC (or peering/TGW routes)? NACLs? DB "Available"? |
| `password authentication failed` | Wrong user/password; secret rotated but app cached the old one |
| `could not translate host name` | Wrong endpoint, VPC DNS settings off, private hosted zone not associated |
| `too many connections` | Connection leaks / no pooling → use RDS Proxy or a pool; raise `max_connections` carefully |
| Slow queries | Performance Insights → top SQL; missing indexes; instance too small (CPU/memory/IOPS) |
| Replica data is stale | Replica lag (`ReplicaLag` metric); read your own writes from the writer |

## 7.18 Application integration (decoupling services)

**Why decouple:** if service A calls service B directly and B is slow or down, A fails too. A queue or event bus in between absorbs spikes, enables retries and lets each side scale independently.

### SQS (Simple Queue Service)

```
Producer ──send──► [ SQS queue ] ──poll/receive──► Consumer ──delete when done
                          │ after N failed receives
                          ▼
                  [ Dead-letter queue ]   ← inspect and replay failed messages
```

| Concept | Detail |
|---|---|
| Standard queue | Nearly unlimited throughput, **at-least-once** delivery, best-effort ordering → consumers must be **idempotent** (safe to process twice) |
| FIFO queue | Exactly-once processing, strict order per message group; name ends with `.fifo` |
| Visibility timeout | After a consumer receives a message it's hidden for this time (default 30 s). Not deleted in time → becomes visible again. Set it longer than processing time |
| Retention | Default 4 days, max 14 days |
| Long polling | `WaitTimeSeconds` up to 20 s: fewer empty responses, lower cost |
| Dead-letter queue (DLQ) | Messages that failed `maxReceiveCount` times go here |
| Delay queues | Postpone delivery |
| Common pairing | SQS → Lambda (event source mapping), SQS → ECS/EC2 workers scaled on queue depth |

### SNS (Simple Notification Service)

**Pub/sub:** a publisher sends to a **topic**; every subscriber gets a copy (SQS, Lambda, HTTPS, email, SMS, mobile push). **Fan-out pattern:** SNS topic → several SQS queues so each consumer processes every event independently. Message filtering policies let subscribers receive only matching messages.

### EventBridge

**Serverless event bus:** AWS services, SaaS apps and your own apps publish **events** (JSON); **rules** match patterns and route to **targets** (Lambda, SQS, Step Functions, ECS tasks, API destinations).

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": { "state": ["stopped", "terminated"] }
}
```

Also: **EventBridge Scheduler** (cron/rate schedules, one-time schedules), archive & replay, schema registry, Pipes (point-to-point integrations).

### Step Functions

**Workflow orchestration** as a state machine (JSON/visual): steps, branches, retries with backoff, parallel work, waits, human approval, error handling. Calls Lambda and 200+ AWS services directly. Standard (long-running, exactly-once) vs Express (high-volume, short).

### Streaming

| Service | What | Use |
|---|---|---|
| Kinesis Data Streams | Real-time data streams with shards, ordered per shard, replay | Clickstreams, IoT, logs |
| Amazon Data Firehose | Load streaming data into S3, Redshift, OpenSearch, Splunk with no code | Log/metric delivery |
| MSK | Managed Apache Kafka | Kafka-based event platforms |
| Amazon MQ | Managed ActiveMQ / RabbitMQ | Migrating apps that use standard brokers (JMS, AMQP) |

| Choose | When |
|---|---|
| SQS | One consumer group processes work items (task queue) |
| SNS | Push the same message to many subscribers |
| EventBridge | Route events by content between many services/SaaS; schedules |
| Kinesis / MSK | High-volume ordered streams, multiple readers, replay |
| Step Functions | Multi-step business workflows with state and retries |

## 7.19 Analytics services (awareness level)

| Service | What it does |
|---|---|
| Athena | Run SQL directly on files in S3 (CSV, JSON, Parquet); pay per data scanned. Great for querying logs (ALB, CloudTrail, VPC Flow Logs) |
| Glue | Serverless ETL + **Data Catalog** (table definitions used by Athena/Redshift/EMR) + crawlers |
| EMR | Managed Hadoop/Spark clusters for big data processing |
| Redshift | Data warehouse; Redshift Serverless; Spectrum queries S3 |
| QuickSight | Business intelligence dashboards |
| Lake Formation | Build and secure data lakes on S3 with fine-grained permissions |
| OpenSearch Service | Search and log analytics (ELK-style), dashboards |

**Data lake pattern:** raw data lands in S3 → Glue catalogs/transforms → Athena/Redshift query → QuickSight dashboards.

## 7.20 Serverless: Lambda and API Gateway

### Lambda

**What:** run code **without managing servers**. AWS runs your function when an event arrives and bills per request and per millisecond of execution. **Why:** no patching or capacity planning, scales automatically from zero to thousands of parallel executions, cheap for spiky or low traffic. **In companies:** APIs, file processing (S3 upload → thumbnail), scheduled automation (cleanup, reports, auto-stop instances), event processing (SQS, DynamoDB Streams, EventBridge), glue between services.

| Concept | Detail |
|---|---|
| Runtime | Python, Node.js, Java, .NET, Ruby, Go/Rust (custom runtime), or a **container image** (up to 10 GB) |
| Handler | Function entry point, e.g. `app.handler` → file `app.py`, function `handler(event, context)` |
| Event | JSON input describing what happened (S3 upload, API request, SQS batch…) |
| Execution role | IAM role the function uses to call AWS (least privilege!) |
| Memory | 128 MB to 10,240 MB; **CPU scales with memory** |
| Timeout | Default 3 s, **max 15 minutes** |
| Ephemeral storage | `/tmp`, 512 MB default, up to 10 GB |
| Environment variables | Configuration (not secrets: read secrets from Secrets Manager/Parameter Store) |
| Concurrency | Parallel executions; regional account limit (default 1,000, can be raised). **Reserved concurrency** caps a function; **provisioned concurrency** keeps instances warm |
| Cold start | First request on a new execution environment is slower (init). Reduce with smaller packages, provisioned concurrency, SnapStart (Java/Python/.NET) |
| Versions & aliases | Immutable versions; aliases ( `prod` → v7) enable safe deployments and weighted traffic shifting |
| Layers | Shared libraries packaged separately |
| VPC access | Attach to private subnets to reach RDS etc. (then internet needs a NAT or VPC endpoints) |
| Invocation types | Synchronous (API Gateway), asynchronous (S3, SNS, EventBridge: built-in retries + on-failure destination/DLQ), poll-based (SQS, Kinesis, DynamoDB Streams via event source mappings) |
| Logs | Automatically to CloudWatch Logs group `/aws/lambda/<name>` |

**Example: stop running instances tagged `AutoStop=true` (scheduled nightly by EventBridge Scheduler):**

```
import boto3

ec2 = boto3.client("ec2")

def handler(event, context):
    resp = ec2.describe_instances(Filters=[
        {"Name": "tag:AutoStop", "Values": ["true"]},
        {"Name": "instance-state-name", "Values": ["running"]},
    ])
    ids = [i["InstanceId"] for r in resp["Reservations"] for i in r["Instances"]]
    if ids:
        ec2.stop_instances(InstanceIds=ids)
    print(f"Stopped: {ids}")
    return {"stopped": ids}
```

Execution role needs `ec2:DescribeInstances`, `ec2:StopInstances`, plus `AWSLambdaBasicExecutionRole` for logs.

**Good for Lambda:** short, event-driven, stateless work. **Not good:** long jobs (>15 min), steady very high load (containers may be cheaper), workloads needing local state.

### API Gateway

| Type | Use |
|---|---|
| **HTTP API** | Simple, cheaper, faster; Lambda/HTTP backends, JWT auth. **Default choice** |
| REST API | More features: API keys & usage plans, request validation, caching, WAF, private APIs |
| WebSocket API | Real-time two-way apps (chat) |

Features: stages ( `dev` , `prod` ), custom domains (ACM), throttling, CORS, authorizers (IAM, Cognito, JWT, Lambda), access logs.

**Classic serverless API:** `Client → API Gateway → Lambda → DynamoDB` (+ Cognito for user login).

### Other serverless building blocks

Cognito (user sign-up/sign-in, JWT tokens) · AppSync (managed GraphQL) · SQS/SNS/EventBridge/Step Functions (7.18) · Fargate (serverless containers) · Aurora Serverless v2 · DynamoDB on-demand.

## 7.21 Containers on AWS

| Service | What | When |
|---|---|---|
| **ECR** | Private container image registry | Always: where your images live |
| **ECS** | AWS's own container orchestrator (simpler than Kubernetes) | Most teams that just want to run containers on AWS |
| **EKS** | Managed Kubernetes | Teams standardizing on Kubernetes, multi-cloud skills, rich ecosystem |
| **Fargate** | Serverless compute for ECS and EKS (no EC2 nodes to manage) | Default for small/medium workloads |
| App Runner | Give it a container image or repo → running HTTPS service | Simplest web apps/APIs |
| Lambda (container images) | Run a container image as a function | Event-driven work packaged as images |
| AWS Batch | Batch jobs on managed compute | Large batch processing |

### ECR

| Feature | Detail |
|---|---|
| Repository | One per image name, e.g. `shop/api` |
| Login | `aws ecr get-login-password --region R \| docker login --username AWS --password-stdin <acct>.dkr.ecr.R.amazonaws.com` |
| Push | `docker tag api:1.4.2 <acct>.dkr.ecr.R.amazonaws.com/shop/api:1.4.2` → `docker push ...` |
| **Tag immutability** | Prevents overwriting an existing tag (turn it on) |
| Image scanning | Basic or Enhanced (Inspector) vulnerability scanning on push |
| Lifecycle policies | Delete untagged/old images automatically (saves storage cost) |
| Replication / pull-through cache | Copy images across regions/accounts; cache public images (Docker Hub, ECR Public) |

## ECS in depth

```
ECS CLUSTER
 └─ SERVICE "api" (desired count 3, rolling deploy, behind ALB target group)
      └─ TASKS (running copies of the task definition)
           └─ containers: api:1.4.2 (+ sidecars)
Runs on: FARGATE (serverless)  or  EC2 instances (capacity providers / ASG)
```

| Concept | Detail |
|---|---|
| Task definition | JSON blueprint: image, CPU/memory, ports, env vars, **secrets** (from Secrets Manager/SSM), log config, IAM roles. Versioned (revisions) |
| Task | A running instance of a task definition |
| Service | Keeps N tasks running, replaces failed ones, integrates with ALB/NLB, scales with Application Auto Scaling |
| Launch type / capacity provider | **Fargate** (no servers) or **EC2** (you manage the instances; more control, can be cheaper at scale) |
| **Task role** | Permissions for **your app** inside the container (e.g. read S3) |
| **Task execution role** | Permissions for **ECS itself** to pull the image from ECR, fetch secrets, write logs |
| Networking | `awsvpc` mode: each task gets its own ENI and private IP and its own security group |
| Deployments | Rolling update (min healthy % / max %), deployment circuit breaker with automatic rollback, blue/green via CodeDeploy |
| Service discovery | Cloud Map / Service Connect for service-to-service names |
| Logs | `awslogs` driver → CloudWatch Logs; or FireLens |
| ECS Exec | `aws ecs execute-command` → shell into a running container (debugging) |

**Minimal Fargate task definition (excerpt):**

```
{
  "family": "shop-api",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "256",
  "memory": "512",
  "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::123456789012:role/shop-api-task-role",
  "containerDefinitions": [{
    "name": "api",
    "image": "123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2",
    "portMappings": [{ "containerPort": 8000 }],
    "secrets": [{
      "name": "DB_PASSWORD",
      "valueFrom": "arn:aws:secretsmanager:ap-south-1:123456789012:secret:shop/db-AbCdEf"
    }],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/shop-api",
        "awslogs-region": "ap-south-1",
        "awslogs-stream-prefix": "api"
      }
    }
  }]
}
```

### ECS troubleshooting

| Symptom | Cause |
|---|---|
| Task stops immediately | Read **stopped reason** in the console/ `describe-tasks` : `CannotPullContainerError` (image name/tag, ECR permissions on execution role, no route to ECR → NAT or VPC endpoints), `ResourceInitializationError` (can't fetch secrets/logs), app exit code (check logs) |
| Tasks keep restarting | Failing ALB health checks (path/port/SG), container health check failing, out of memory (exit 137) |
| Service stuck "deploying" | New tasks never become healthy → circuit breaker rolls back |

### EKS (overview; details in the Kubernetes notes)

| Topic | Detail |
|---|---|
| Control plane | Managed by AWS across AZs; billed per cluster-hour |
| Data plane | **Managed node groups** (EC2), self-managed nodes, **Fargate profiles**, or **EKS Auto Mode** (AWS manages nodes, scaling and core add-ons) |
| Networking | **VPC CNI**: pods get VPC IPs (plan subnet sizes!) |
| Load balancing | AWS Load Balancer Controller: Ingress → ALB, Service type LoadBalancer → NLB |
| Pod permissions | **EKS Pod Identity** or IRSA (IAM roles for service accounts) |
| Access | Access entries map IAM principals to Kubernetes RBAC; `aws eks update-kubeconfig --name <cluster>` |
| Add-ons | VPC CNI, CoreDNS, kube-proxy, EBS CSI driver, metrics-server; Karpenter for node autoscaling |
| Upgrades | One minor Kubernetes version at a time: control plane → add-ons → nodes |

| ECS vs EKS | ECS | EKS |
|---|---|---|
| Learning curve | Lower | Higher (Kubernetes) |
| Portability | AWS only | Kubernetes anywhere |
| Ecosystem | AWS integrations | Huge Kubernetes ecosystem (Helm, Argo CD, operators) |
| Control-plane cost | Free | Per cluster-hour |

## 7.22 Other compute options

| Service | What | Use |
|---|---|---|
| Elastic Beanstalk | Upload code; it creates EC2, ASG, ALB, monitoring for you (PaaS) | Simple web apps without managing infrastructure details |
| Lightsail | Simple VPS bundles with fixed monthly price | Small websites, blogs, learners |
| AWS Batch | Queue and run batch jobs on EC2/Fargate/Spot | Data processing, rendering |
| Outposts | AWS racks in your data center | Low latency to on-prem systems, data residency |

## 7.23 Management, operations and Infrastructure as Code

### AWS Systems Manager (SSM)

A toolbox for operating fleets of servers (EC2 and on-premises). Requires the **SSM agent** and an instance role with `AmazonSSMManagedInstanceCore`.

| Capability | Use |
|---|---|
| **Session Manager** | Shell access with no SSH keys or open ports; sessions logged to S3/CloudWatch |
| **Run Command** | Run a script on many instances at once (by tag) |
| **Patch Manager** | Scan and install OS patches on a schedule (patch baselines, maintenance windows) |
| **Parameter Store** | Configuration and secrets ( `SecureString` encrypted with KMS); hierarchical names like `/shop/prod/db/host` |
| Automation | Runbooks for common tasks (restart, create AMI, remediate) |
| State Manager | Keep instances in a defined state (install agent, settings) |
| Inventory | Installed software and configuration per instance |
| Fleet Manager | Console view of managed nodes |

```
aws ssm start-session --target i-0123456789abcdef0
aws ssm send-command --document-name AWS-RunShellScript \
  --targets Key=tag:Env,Values=dev --parameters 'commands=["uptime","df -h"]'
aws ssm put-parameter --name /shop/dev/db/password --type SecureString --value '<secret>'
aws ssm get-parameter --name /shop/dev/db/password --with-decryption \
  --query Parameter.Value --output text
```

### CloudFormation and CDK

**CloudFormation** = AWS's native IaC service. You write a **template** (YAML/JSON); CloudFormation creates a **stack** and manages updates, dependencies and rollbacks.

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Versioned S3 bucket
Parameters:
  Env:
    Type: String
    AllowedValues: [dev, prod]
Resources:
  AssetsBucket:
    Type: AWS::S3::Bucket
    Properties:
      VersioningConfiguration:
        Status: Enabled
      Tags:
        - Key: Env
          Value: !Ref Env
Outputs:
  BucketName:
    Value: !Ref AssetsBucket
```

| Concept | Detail |
|---|---|
| Template sections | `Parameters`, `Mappings`, `Conditions`, `Resources` (required), `Outputs` |

| Concept | Detail |
|---|---|
| Intrinsic functions | `!Ref`, `!GetAtt`, `!Sub`, `!Join`, `!If`, `!ImportValue` |
| Stack | Deployed instance of a template; failed creates/updates **roll back** automatically |
| Change set | Preview of changes before updating (like `terraform plan`) |
| Drift detection | Find manual changes made outside CloudFormation |
| StackSets | Deploy a stack to many accounts/regions |
| Nested stacks / modules | Reuse |

**CDK (Cloud Development Kit):** write infrastructure in TypeScript/Python/Java/Go → it generates CloudFormation.

| | Terraform | CloudFormation | CDK |
|---|---|---|---|
| Clouds | Multi-cloud + SaaS providers | AWS only | AWS only (generates CloudFormation) |
| Language | HCL | YAML/JSON | Programming languages |
| State | State file (you manage: S3 + locking) | Managed by AWS | Managed by AWS |
| Typical use | Most DevOps teams, multi-cloud | AWS-centric orgs, AWS service templates | Developer-heavy teams |

### AWS Config

Records the **configuration history** of resources and evaluates **rules** (e.g. "S3 buckets must block public access", "EBS volumes must be encrypted"). Non-compliant resources can be **auto-remediated** with SSM Automation. Conformance packs bundle rules for standards (CIS, PCI).

### Other management services

| Service | Use |
|---|---|
| Service Catalog | Approved, self-service products (pre-built CloudFormation/Terraform) for teams |
| Trusted Advisor | Checks for cost, security, fault tolerance, performance, service limits |
| Service Quotas | View and request increases for limits |
| Health Dashboard | AWS service events affecting your account |
| Resource Groups & Tag Editor | Find and manage resources by tag |
| Compute Optimizer | Right-sizing recommendations for EC2, ASG, EBS, Lambda, ECS |
| License Manager | Track software licenses |

## 7.24 Monitoring and auditing

### CloudWatch

| Feature | Detail |
|---|---|
| **Metrics** | Time-series numbers grouped by **namespace** (`AWS/EC2`, `AWS/RDS`, custom `Shop/App`) and **dimensions** (e.g. `InstanceId`). Standard 1-minute (or 5-min basic for EC2), high-resolution down to 1 s for custom metrics |

| Feature | Detail |
|---|---|
| Statistics | Average, Sum, Minimum, Maximum, SampleCount, percentiles (p90, p99) |
| **Alarms** | Watch a metric over N periods → states `OK`, `ALARM`, `INSUFFICIENT_DATA` → actions: SNS notification, Auto Scaling, EC2 stop/reboot/recover, SSM. **Composite alarms** combine alarms to reduce noise. Anomaly detection bands |
| Dashboards | Graphs and numbers across services/regions |
| **Logs** | Log **groups** (per app) → log **streams** (per instance/container). **Set retention** (default: never expire = growing cost) |
| Metric filters | Turn log patterns into metrics (count `ERROR` lines → alarm) |
| **Logs Insights** | Query language for logs |
| Subscription filters | Stream logs to Lambda, Kinesis, OpenSearch |
| CloudWatch agent | Sends **memory, disk space**, processes and log files from EC2/on-prem (not collected by default) |
| Container Insights / Lambda Insights | Metrics for ECS, EKS, Lambda |
| Synthetics | Canary scripts that test your website/API every few minutes |
| Application Signals / X-Ray | Application performance monitoring and distributed **tracing** |

**Logs Insights examples:**

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

```
stats count(*) as errors by bin(5m)
| filter @message like /Exception/
```

**Create a CPU alarm with email notification:**

```
TOPIC=$(aws sns create-topic --name ops-alerts --query TopicArn --output text)
aws sns subscribe --topic-arn $TOPIC --protocol email \
  --notification-endpoint you@example.com      # then confirm the email
aws cloudwatch put-metric-alarm --alarm-name web-cpu-high \
  --namespace AWS/EC2 --metric-name CPUUtilization --statistic Average \
  --dimensions Name=InstanceId,Value=i-0123456789abcdef0 \
  --period 300 --evaluation-periods 2 --threshold 80 \
  --comparison-operator GreaterThanThreshold --alarm-actions $TOPIC
```

## CloudTrail

| Topic | Detail |
|---|---|
| What | Records **API calls**: who, what, when, from where (user/role, action, source IP, request parameters, result) |
| Event history | Last **90 days** of management events, free, on by default |

| Topic | Detail |
|---|---|
| Trail | Delivers events to S3 (and optionally CloudWatch Logs) for long-term retention; **organization trail** covers all accounts |
| Event types | **Management events** (create/delete/modify resources), **data events** (S3 object reads/writes, Lambda invokes: extra cost), Insights events (unusual API activity) |
| CloudTrail Lake | Store and query events with SQL |
| Protection | Log file integrity validation, S3 bucket in a separate log-archive account, Object Lock |

**Typical questions CloudTrail answers:** "Who deleted this security group?", "Who changed the bucket policy?", "Was the root user used?". Search: CloudTrail → Event history → filter by *Event name* (`DeleteSecurityGroup`) or *Resource name*.

### EventBridge for operations

Rules on events like *EC2 state change*, *GuardDuty finding*, *CloudTrail API call* (e.g. `ConsoleLogin` by root) → SNS/Slack/Lambda for alerts and auto-remediation.

## 7.25 Security services and data protection

### Encryption and secrets

| Service | What | Key points |
|---|---|---|
| **KMS** | Create and control encryption keys | AWS managed keys vs **customer managed keys** (you control key policy, rotation, deletion). **Envelope encryption**: a data key encrypts data; KMS encrypts the data key. Every use is logged in CloudTrail. Key policy + IAM decide who can use a key |
| CloudHSM | Dedicated hardware security modules | Strict compliance needing single-tenant HSMs |
| **Secrets Manager** | Store secrets (DB passwords, API keys) with **automatic rotation** | Apps fetch at runtime via SDK/ECS/EKS integration; paid per secret |
| **Parameter Store** | Config + `SecureString` secrets | Standard tier free; no built-in rotation |
| **ACM** | Free public TLS certificates for ALB, CloudFront, API Gateway, with automatic renewal | Certificates for CloudFront must be in `us-east-1` |

**Encryption at rest:** turn on for EBS (default encryption), S3 (default SSE-S3, or SSE-KMS), RDS, DynamoDB, EFS, backups. **In transit:** HTTPS/TLS everywhere; enforce with bucket policies (`aws:SecureTransport`), ALB HTTPS listeners, RDS `require SSL`.

### Threat detection and protection

| Service | What it does |
|---|---|
| **GuardDuty** | Threat detection using CloudTrail, VPC Flow Logs, DNS logs (+ S3, EKS, RDS, Lambda, runtime monitoring): compromised instances, crypto-mining, credential misuse, unusual API calls |

| Service | What it does |
|---|---|
| **Inspector** | Continuous vulnerability scanning of EC2, ECR images and Lambda (CVEs, network exposure) |
| Macie | Finds sensitive data (personal data, credentials) in S3 |
| **Security Hub** | Central view of findings + automated checks against standards (AWS Foundational Security Best Practices, CIS) |
| Detective | Investigates the root cause of findings (graph of activity) |
| **WAF** | Web application firewall for CloudFront, ALB, API Gateway, AppSync: managed rule groups (OWASP top risks, SQL injection, XSS, bad IPs, bots), rate-based rules |
| **Shield** | DDoS protection: Standard (free, automatic) and Advanced (paid, 24/7 response team, cost protection) |
| Network Firewall | Managed stateful firewall/IPS for VPC traffic (domain filtering, Suricata rules) |
| Firewall Manager | Central management of WAF, Shield, SGs, Network Firewall across accounts |
| IAM Access Analyzer | External access and unused access findings, policy generation and validation |
| Audit Manager / Artifact | Compliance evidence / AWS compliance reports (SOC, ISO, PCI) |

**Incident response basics (compromised instance):** isolate (replace SG with a no-access SG), snapshot volumes for forensics, revoke/rotate credentials (role sessions, keys), investigate with CloudTrail/GuardDuty/Detective, rebuild from a clean AMI, write a postmortem.

## 7.26 Multi-account governance

```
AWS Organizations (management account: billing + governance only, no workloads)
├── OU Security        ── Log Archive account (CloudTrail, Config logs)
│                         Security Tooling account (GuardDuty, Security Hub admin)
├── OU Infrastructure  ── Network account (Transit Gateway, shared VPCs, DNS)
│                         Shared Services account (CI/CD, ECR)
├── OU Workloads
│     ├── OU Dev   ── shop-dev account
│     └── OU Prod  ── shop-prod account
└── OU Sandbox     ── experiment accounts (strict budgets)
```

| Service / concept | Detail |
|---|---|
| **Organizations** | Groups accounts, **consolidated billing** (one bill, combined volume discounts), organizational units (OUs) |
| **SCPs** | Maximum permissions for accounts/OUs: e.g. deny leaving the organization, deny disabling CloudTrail/GuardDuty, allow only approved regions. SCPs never grant permissions |
| RCPs | Resource control policies: org-wide limits on who can access your resources |
| Tag / backup / AI opt-out policies | Org-wide standards |
| **Control Tower** | Automated landing zone: account factory, guardrails (preventive SCPs + detective Config rules), central logging |
| **IAM Identity Center** | Single sign-on across all accounts with permission sets |

| Service / concept | Detail |
|---|---|
| RAM (Resource Access Manager) | Share subnets, TGWs, Route 53 Resolver rules, etc. across accounts |

**Why multi-account:** strongest isolation boundary (blast radius), clear cost per team/environment, separate quotas, simpler permissions.

**Example SCP: allow only two regions (global services exempted):**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyOtherRegions",
    "Effect": "Deny",
    "NotAction": [
      "iam:*", "organizations:*", "route53:*", "cloudfront:*",
      "support:*", "sts:*", "budgets:*"
    ],
    "Resource": "*",
    "Condition": { "StringNotEquals": { "aws:RequestedRegion": ["ap-south-1", "us-east-1"] } }
  }]
}
```

## 7.27 Cost management (FinOps basics)

| Tool | Use |
|---|---|
| Billing dashboard / Bills | What you were charged, per service and region |
| **Cost Explorer** | Explore and forecast cost by service, account, tag, usage type; RI/Savings Plans recommendations |
| **Budgets** | Alerts (and actions) when cost/usage/coverage crosses thresholds |
| Cost Anomaly Detection | ML alerts for unusual spend |
| Cost and Usage Report (Data Exports) | Line-item billing data to S3 for Athena/QuickSight analysis |
| Cost allocation tags | Activate tags (e.g. `Project`) to split costs |
| Compute Optimizer / Trusted Advisor | Right-sizing and idle-resource findings |
| Pricing Calculator | Estimate before you build |

**Cost optimization checklist:**

1. **Delete idle resources:** unattached EBS volumes, old snapshots/AMIs, unused EIPs/public IPv4, idle load balancers, stopped instances you don't need, forgotten NAT Gateways.
2. **Right-size** instances (Compute Optimizer), move to **Graviton**, gp2 → gp3.
3. **Schedule** dev/test to stop nights and weekends.
4. **Commit** for steady usage (Savings Plans); **Spot** for flexible work.
5. **Storage lifecycle:** S3 lifecycle rules/Intelligent-Tiering, log retention in CloudWatch.
6. **Network:** VPC gateway endpoints for S3/DynamoDB instead of NAT, keep chatty traffic in one AZ, use CloudFront to cut data transfer out.
7. **Tag everything** and review Cost Explorer weekly.

## 7.28 High availability, backup and disaster recovery

### Availability building blocks

| Layer | HA approach |
|---|---|
| DNS | Route 53 (100% SLA), health checks, failover routing |
| Edge | CloudFront, Global Accelerator |
| Load balancing | ALB/NLB across ≥2 AZs |
| Compute | Auto Scaling Group / ECS service / EKS across ≥2 AZs, stateless instances |
| Database | RDS Multi-AZ / Aurora replicas / DynamoDB (multi-AZ by default) |
| Storage | S3 (multi-AZ by default), EFS regional, EBS snapshots |
| Network | One NAT Gateway per AZ, redundant VPN tunnels / DX + VPN backup |

**Design for failure:** assume any instance, AZ, or dependency can fail; use health checks, retries with backoff, timeouts, idempotency and graceful degradation.

### Backup

**AWS Backup:** central backup **plans** (schedule + retention + lifecycle to cold storage), **vaults** (with Vault Lock for immutability), cross-region and cross-account copies, for EC2/EBS, RDS/Aurora, DynamoDB, EFS, FSx, S3 and more. **Test restores regularly**: an untested backup is not a backup.

### Disaster recovery strategies

| Strategy | Idea | RTO | RPO | Cost |
|---|---|---|---|---|
| Backup & restore | Back up to another region; rebuild with IaC when needed | Hours | Hours (last backup) | $ |
| Pilot light | Core data replicated live (DB replica); servers off/minimal | Tens of minutes | Minutes | $$ |
| Warm standby | Scaled-down full copy always running in the DR region | Minutes | Seconds–minutes | $$$ |
| Multi-site active/active | Full capacity in 2+ regions serving traffic | Near zero | Near zero | $$$$ |

- **RPO (Recovery Point Objective):** how much data loss is acceptable (time since last good copy).
- **RTO (Recovery Time Objective):** how long the system may be down.
- **Elastic Disaster Recovery (DRS):** continuous block-level replication of servers into AWS for fast recovery.
- IaC (Terraform/CloudFormation) makes rebuilding another region repeatable.

## 7.29 Migration to AWS (awareness)

**The 7 Rs:** Retire · Retain · **Rehost** (lift-and-shift) · **Relocate** (e.g. VMware Cloud on AWS) · **Replatform** (lift-tinker-shift, e.g. to RDS) · Repurchase (move to SaaS) · **Refactor** (re-architect, e.g. to containers/serverless).

| Service | Use |
|---|---|
| Migration Hub / Application Discovery Service | Inventory and track migrations |

| Service | Use |
|---|---|
| Application Migration Service (MGN) | Rehost servers into EC2 with continuous replication |
| Database Migration Service (DMS) + Schema Conversion | Migrate databases (same or different engine) with minimal downtime |
| DataSync | Copy large file data (NFS/SMB/S3) online |
| Storage Gateway | Hybrid storage: on-prem apps use S3/backups through a local gateway |
| Snow family | Physical devices to ship large data offline |
| Transfer Family | Managed SFTP/FTPS/FTP into S3/EFS |

## 7.30 Well-Architected Framework

| Pillar | Key practices |
|---|---|
| **Operational Excellence** | IaC, small frequent reversible changes, runbooks, observability, learn from failures |
| **Security** | Strong identity foundation (least privilege, SSO, roles), traceability (CloudTrail), protect data (encryption), security at all layers, automate, prepare for incidents |
| **Reliability** | Multi-AZ, auto-recovery, scale horizontally, test recovery, manage quotas, backups |
| **Performance Efficiency** | Right resource types, serverless/managed services, measure and experiment, caching |
| **Cost Optimization** | Pay only for what you need, measure (tags, Cost Explorer), right-size, commit/Spot, managed services |
| **Sustainability** | Maximize utilization, efficient hardware (Graviton), reduce data movement and storage |

The **Well-Architected Tool** in the console asks review questions per workload and tracks improvement plans.

## 7.31 Reference architectures

### A. Highly available 3-tier web application

```
Users → Route 53 → CloudFront (+ WAF) → ALB (public subnets, 2 AZs)
                                          |
                      ASG / ECS service (private app subnets, 2 AZs)
                        |             |              |
            ElastiCache (sessions)  S3 (assets)   Secrets Manager (DB creds)
                        |
                RDS Multi-AZ / Aurora (private data subnets)
Logs/metrics → CloudWatch · audit → CloudTrail · backups → AWS Backup
```

### B. Static website

`Route 53 → CloudFront (ACM certificate, OAC) → private S3 bucket`. Cheap, fast, no servers. Deploy with `aws s3 sync` + CloudFront invalidation from CI.

### C. Serverless API

`Route 53 → API Gateway (HTTP API, JWT authorizer with Cognito) → Lambda → DynamoDB`, async work through SQS/EventBridge → Lambda, monitoring with CloudWatch + X-Ray.

### D. Containers platform

`GitHub Actions (OIDC) → ECR → ECS Fargate or EKS (private subnets) behind ALB`, secrets from Secrets Manager, logs to CloudWatch, Terraform for all infrastructure.

### E. Event-driven processing

`S3 upload → EventBridge rule → Step Functions (validate → process with Lambda/Fargate → store in DynamoDB → notify with SNS)`, failures to a DLQ.

## 7.32 AWS troubleshooting playbook

| Problem | Investigation order |
|---|---|
| Anything returns **AccessDenied** | `aws sts get-caller-identity` → error message (action + resource) → identity policies → resource policies (bucket/KMS) → SCP/boundary → conditions → CloudTrail event |
| **EC2 unreachable** | Status checks → public IP/EIP → SG → NACL → route table (IGW) → OS firewall/service → key/username → use SSM / serial console |
| **Website returns 5xx through ALB** | Target group health (reason codes) → app logs → target SG allows ALB SG → health check path → idle timeouts (504) |
| **Private instance can't reach the internet / AWS APIs** | Route to NAT (in a public subnet) or VPC endpoints → SG/NACL outbound → DNS settings |
| **App can't connect to RDS** | DB SG allows app SG on DB port → same VPC/routes → endpoint name → credentials/secret → DB status/connections |
| **Lambda failing** | CloudWatch Logs for the function → timeout/memory → execution role permissions → VPC access (NAT/endpoints) → throttling (concurrency) |
| **ECS task stopped** | Stopped reason → image pull (execution role, network to ECR) → secrets → health checks → logs → exit code (137 = OOM) |
| **S3 403** | IAM vs bucket policy vs Block Public Access vs KMS key policy → `ListBucket` on bucket ARN vs `GetObject` on `/*` |
| **Unexpected bill** | Cost Explorer by service and **region** → usage type (NAT bytes, data transfer, EBS snapshots, public IPv4) → find and delete; add budgets |
| **Can't launch instances** | Activity/error message → service quotas (vCPU) → capacity in AZ → subnet free IPs → AMI/permissions |

## 7.33 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | What is the difference between a Region and an AZ? | A region is a geographic area; an AZ is one or more isolated data centers inside a region. Deploy across multiple AZs for HA |
| 2 | Explain the shared responsibility model | AWS secures the cloud (hardware, facilities, managed-service software); customers secure what they put in it (data, IAM, OS patching on EC2, network rules) |
| 3 | IAM user vs role? | User = long-term identity with password/keys. Role = assumed identity with temporary credentials (STS); used by services, CI, federation, cross-account |
| 4 | How are IAM policies evaluated? | Default deny → explicit allow needed → explicit deny always wins; SCPs and permission boundaries limit the maximum |

| # | Question | Answer |
|---|---|---|
| 5 | How does an application on EC2 get AWS credentials safely? | IAM role via instance profile; SDK reads temporary credentials from IMDSv2; never store keys on the instance |
| 6 | Security group vs NACL? | SG: stateful, instance-level, allow-only, all rules evaluated. NACL: stateless, subnet-level, allow+deny, ordered rules, needs ephemeral ports |
| 7 | Public vs private subnet? | Public has a route `0.0.0.0/0 → IGW`; private doesn't (uses NAT for outbound) |
| 8 | How does a private instance download updates? | Through a NAT Gateway in a public subnet (or VPC endpoints for AWS services) |
| 9 | Internet Gateway vs NAT Gateway? | IGW enables two-way internet for public IPs; NAT GW gives outbound-only internet to private subnets |
| 10 | What are VPC endpoints? | Private access to AWS services without internet: gateway (S3/DynamoDB, free, route table) and interface (PrivateLink ENIs, paid) |
| 11 | VPC peering vs Transit Gateway? | Peering: 1-to-1, non-transitive. TGW: hub for many VPCs/VPNs with transitive routing |
| 12 | ALB vs NLB? | ALB: layer 7 HTTP routing (paths/hosts), WAF. NLB: layer 4 TCP/UDP, extreme performance, static IPs, preserves client IP |
| 13 | ALB returns 503. Why? | No healthy targets registered in the target group |
| 14 | How does Auto Scaling know when to scale? | Scaling policies on CloudWatch metrics (target tracking e.g. CPU 50%), schedules, or predictive scaling |
| 15 | EBS vs EFS vs S3 vs instance store? | Block disk for one instance (AZ) / shared NFS across AZs / object storage via API / ephemeral host disk |
| 16 | How do you back up EBS? | Snapshots (incremental, stored in S3), automated by DLM or AWS Backup, copied cross-region for DR |
| 17 | S3 storage classes and lifecycle? | Standard → IA → Glacier tiers by access frequency; lifecycle rules transition/expire automatically |
| 18 | How do you secure an S3 bucket? | Block Public Access, least-privilege IAM, bucket policy (HTTPS only, specific principals), encryption (SSE-KMS), versioning, access logs, CloudFront OAC for public content |
| 19 | RDS Multi-AZ vs read replica? | Multi-AZ = synchronous standby for automatic failover (HA); read replica = async copy for read scaling |
| 20 | When would you use DynamoDB instead of RDS? | Key-value access at massive scale, predictable single-digit-ms latency, serverless scaling, flexible schema |
| 21 | SQS vs SNS vs EventBridge? | Queue for work items (pull) / pub-sub fan-out (push) / content-based event routing with schedules and SaaS sources |
| 22 | What are Lambda limits you must remember? | 15-min max timeout, up to 10 GB memory, concurrency limits, cold starts, stateless |
| 23 | ECS task role vs execution role? | Task role = permissions for the app; execution role = permissions for ECS to pull images, fetch secrets, write logs |
| 24 | ECS vs EKS? | ECS simpler and AWS-native; EKS is Kubernetes with its ecosystem and portability, more complex and costs per cluster |
| 25 | CloudWatch vs CloudTrail? | CloudWatch = performance metrics, logs, alarms. CloudTrail = audit log of API calls (who did what) |

| # | Question | Answer |
|---|---|---|
| 26 | How do you store secrets on AWS? | Secrets Manager (rotation) or SSM Parameter Store SecureString, accessed by IAM roles at runtime, encrypted with KMS |
| 27 | What is KMS envelope encryption? | Data encrypted with a data key; the data key is encrypted by a KMS key; only the encrypted data key is stored |
| 28 | How do you design for high availability? | Multi-AZ for every tier, load balancer + auto scaling, managed multi-AZ databases, health checks, stateless apps, backups |
| 29 | RPO vs RTO; DR strategies? | Data-loss tolerance vs downtime tolerance. Backup/restore → pilot light → warm standby → active/active (cost rises, RTO/RPO fall) |
| 30 | How do you reduce AWS costs? | Delete idle resources, right-size, Graviton, Savings Plans/Spot, schedules, S3 lifecycle, VPC endpoints vs NAT, tags + budgets |
| 31 | Why use multiple AWS accounts? | Isolation/blast radius, separate billing and quotas, simpler permissions; managed with Organizations, SCPs, Control Tower, SSO |
| 32 | What is an SCP? | Organization policy that sets the maximum permissions for accounts; it never grants access |
| 33 | How does CI/CD deploy to AWS without stored keys? | OIDC federation: the pipeline assumes an IAM role via `AssumeRoleWithWebIdentity` with conditions on repo/branch |
| 34 | What's IMDSv2 and why require it? | Session-token based metadata access; blocks SSRF attacks from stealing role credentials |
| 35 | An EC2 instance's public IP changed after restart. Why, and fix? | Auto-assigned public IPs are released on stop; use an Elastic IP or (better) a load balancer/DNS name |
| 36 | CloudFront with a private S3 bucket? | Origin Access Control: bucket policy allows only the distribution; Block Public Access stays on |
| 37 | Terraform vs CloudFormation? | Terraform multi-cloud with its own state file; CloudFormation AWS-native with managed state, change sets and drift detection |
| 38 | Someone deleted a production resource. How do you find out who? | CloudTrail event history / trail: filter by event name and resource; then add guardrails (SCP, IAM, deletion protection) |
| 39 | What happens to data on instance store when you stop the instance? | It is lost (survives only reboots) |
| 40 | How do you give on-prem servers private access to AWS? | Site-to-Site VPN (quick) or Direct Connect (dedicated, consistent), usually via Transit Gateway; DNS via Route 53 Resolver endpoints |

## 7.34 Self-check: I can…

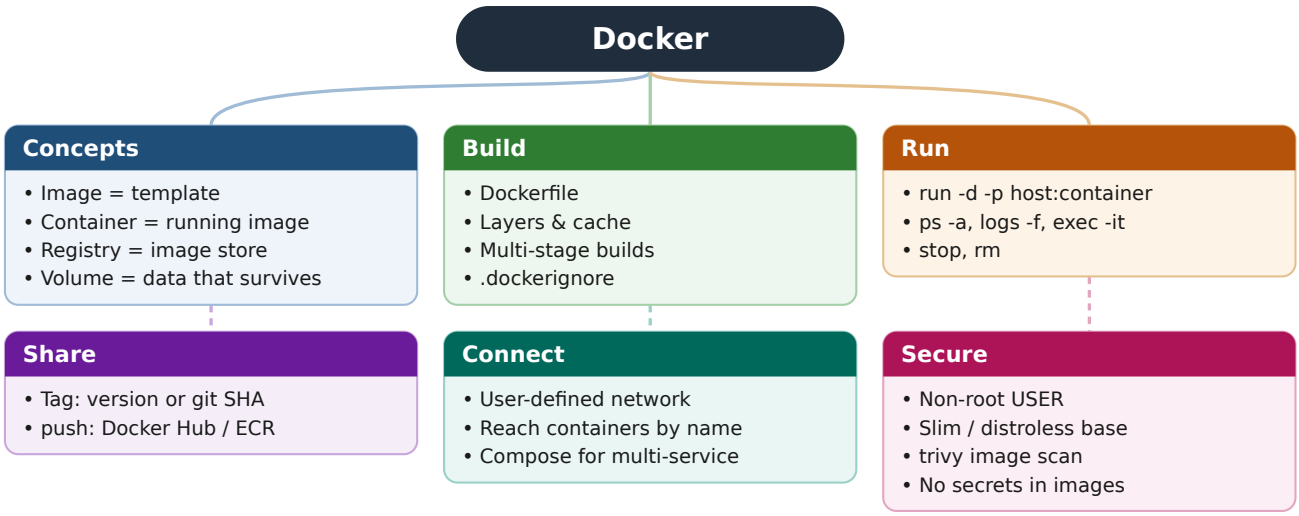
- [ ] Explain regions, AZs, edge locations and the shared responsibility model.
- [ ] Secure a new account: root MFA, admin via SSO/IAM, budgets, CloudTrail.
- [ ] Read and write IAM policies; explain roles, trust policies and STS.
- [ ] Use the AWS CLI with profiles, `--query` and `--filters`.
- [ ] Launch, connect to (SSH and SSM), bootstrap and troubleshoot an EC2 instance.
- [ ] Choose between EBS, EFS, instance store and S3; attach, mount and snapshot a volume.
- [ ] Create a secure S3 bucket with versioning, lifecycle and least-privilege access.
- [ ] Design and build a multi-AZ VPC with public/private subnets, IGW, NAT and endpoints.
- [ ] Explain SGs vs NACLs and debug connectivity with Flow Logs / Reachability Analyzer.

- [ ] Put an ALB in front of an Auto Scaling Group and explain 502/503/504.
- [ ] Configure Route 53 records and routing policies; explain alias vs CNAME.
- [ ] Create a private RDS database with Multi-AZ and backups, and connect from an app.
- [ ] Explain DynamoDB keys, SQS/SNS/EventBridge, Lambda and API Gateway.
- [ ] Explain ECR, ECS (task/service/roles) and EKS at a high level.
- [ ] Set up CloudWatch alarms, log retention, Logs Insights queries and CloudTrail investigations.
- [ ] Explain KMS, Secrets Manager, GuardDuty, WAF and Security Hub.
- [ ] Describe a multi-account setup with Organizations, SCPs and IAM Identity Center.
- [ ] Find and cut costs; set budgets; clean up after every lab.
- [ ] Explain HA patterns, AWS Backup, RPO/RTO and the four DR strategies.
- [ ] Draw and explain the 3-tier, static site, serverless and container reference architectures.

**REMEMBER**

- **Public subnet** = route to an Internet Gateway. Private servers reach the internet through a **NAT Gateway**.
- **Security Group**: stateful, allow-only, per instance. **NACL**: stateless, allow + deny, per subnet.
- First command when anything fails: `aws sts get-caller-identity` (who am I, which account?).
- Well-Architected pillars: Operations, Security, Reliability, Performance, Cost, Sustainability.
- After every lab: **delete resources and check Billing**.

# 8. Docker



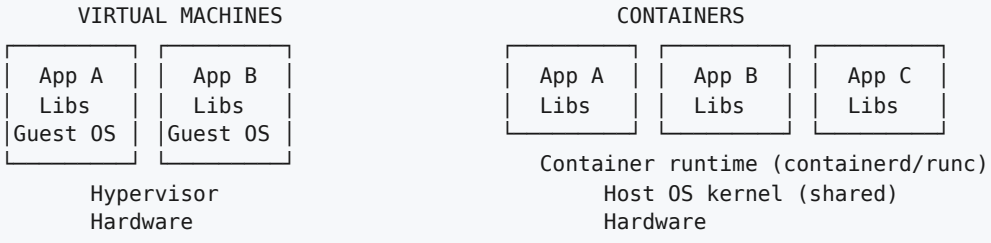

## 8.0 Why containers

**The problem:** "It works on my machine." Apps depend on specific language versions, libraries and OS packages; servers differ; deployments break. **The solution:** package the application **with everything it needs** into an **image**, and run it as an isolated **container** that behaves the same on a laptop, a CI runner, a VM or Kubernetes.

**Where companies use containers:** microservices, CI/CD build environments, Kubernetes/ECS/EKS deployments, local development stacks (Compose), batch jobs, serverless containers (Fargate, Cloud Run, Lambda images).

## 8.1 How containers work

### Containers vs virtual machines




| | VM | Container |
|---|---|---|
| Isolation | Separate kernel per VM (strong) | Shares the host kernel (process-level) |
| Size | GBs | MBs |
| Start time | Seconds–minutes | Milliseconds–seconds |
| Density | Tens per host | Hundreds per host |
| Best for | Different OSes, strong isolation | Packaging and running applications |

In practice they're combined: containers run **inside** cloud VMs.

### Linux building blocks

| Feature | What it provides |
|---|---|
| **Namespaces** | Isolated *view* of the system: `pid` (own process tree; app is PID 1), `net` (own interfaces, IP, ports), `mnt` (own file system), `uts` (own hostname), `ipc`, `user` (UID mapping, rootless) |
| **cgroups** | *Limits* and accounting for CPU, memory, I/O, number of processes |
| **Union / overlay file system** (OverlayFS) | Read-only image layers stacked with a thin writable layer per container |
| Capabilities, seccomp, AppArmor/SELinux | Restrict what processes can do even as "root" |

A container is just **a normal Linux process** with namespaces, cgroups and a layered file system. (On Windows/macOS, Docker Desktop runs a small Linux VM for this.)

### Standards and runtimes

| Component | Role |
|---|---|
| **OCI** (Open Container Initiative) | Standards for image format, runtime and distribution: images built by Docker run on containerd, CRI-O, Podman, Kubernetes |
| `runc` | Low-level runtime: creates namespaces/cgroups and starts the process |
| `containerd` | Manages images and container lifecycles (used by Docker and most Kubernetes clusters) |
| Docker Engine (`dockerd`) | Daemon providing the Docker API, builds (BuildKit), networking, volumes |
| Docker CLI (`docker`) | Client that talks to the daemon's API |
| Podman | Daemonless, rootless-friendly alternative with Docker-compatible commands |
| Registry | Stores and distributes images (Docker Hub, ECR, GHCR, Harbor) |

```
docker CLI ──REST API──► dockerd ──► containerd ──► runc ──► container process
                            │
                            └── pulls/pushes images ◄──► registry
```

## 8.2 Installing and verifying Docker

| Platform | How |
|---|---|
| Windows / macOS | **Docker Desktop** (on Windows enable the WSL2 backend) |
| Ubuntu server | Docker Engine from Docker's apt repository (`docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin`, `docker-compose-plugin`) |
| Amazon Linux 2023 | `sudo dnf install -y docker && sudo systemctl enable --now docker` |

```
docker version            # client + server versions (no server = daemon down / no permission)
docker info               # storage driver, cgroup version, number of containers/images
docker run --rm hello-world
sudo usermod -aG docker $USER   # run docker without sudo (log out/in after)
```

**Security note:** membership in the `docker` group is **equivalent to root** on that machine (you can mount the host file system into a container). Grant it only to trusted users; consider **rootless Docker** or Podman.

## 8.3 Images

### Naming

```
registry.example.com:5000 / team / shop-api : 1.4.2
└──────── registry ─────────┘ └ namespace ┘└ repo ┘ └ tag ┘
docker.io/library/nginx:1.27        (Docker Hub official image; "docker.io/library/" is implied)
123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2   (Amazon ECR)
ghcr.io/thirupathi/shop-api:sha-3f9a1c2                        (GitHub Container Registry)
nginx@sha256:4c0fdaa8b6...          (by DIGEST: immutable content hash)
```

- **Tag** = movable label (`latest` is just a default tag name, not "the newest").
- **Digest** = content hash; pinning by digest guarantees the exact image.

### Layers

Each Dockerfile instruction that changes files creates a **read-only layer**. Layers are cached and shared between images (pulling a second image based on the same base is fast). A container adds one thin **writable layer** on top; it disappears when the container is removed.

### Image commands

| Command | Purpose |
|---|---|
| `docker pull nginx:1.27` | Download an image |
| `docker images` / `docker image ls` | List local images |
| `docker image inspect nginx:1.27` | Full metadata (layers, env, entrypoint, digest) |
| `docker history nginx:1.27` | Layers with sizes and the instructions that created them |
| `docker tag shop-api:1.4.2 ghcr.io/<user>/shop-api:1.4.2` | Add another name |
| `docker rmi image` / `docker image prune -a` | Remove one / all unused images |
| `docker save -o img.tar image` / `docker load -i img.tar` | Export/import as a file |

### Choosing a base image

| Base | Size | Notes |
|---|---|---|
| `ubuntu`, `debian` | Larger | Full package manager and tools; easy debugging |
| `*-slim` (e.g. `python:3.12-slim`) | Medium-small | Debian with fewer packages: **good default** |

| Base | Size | Notes |
|---|---|---|
| `alpine` | Very small | Uses **musl** libc instead of glibc: some Python wheels/binaries misbehave or need compiling |
| **distroless** (`gcr.io/distroless/...`) | Tiny | No shell, no package manager: minimal attack surface; debug with ephemeral containers |
| `scratch` | Empty | For static binaries (Go, Rust) |
| Chainguard / hardened images | Small | Minimal, frequently patched images with few CVEs |

Prefer **official** or verified images, pin versions (`python:3.12.7-slim`, not `python:latest`), and rebuild regularly to pick up security patches.

## 8.4 Running containers

### docker run options

```
docker run -d --name web \
  -p 8080:80 \
  -e APP_ENV=prod --env-file ./app.env \
  -v web-data:/usr/share/nginx/html \
  --restart unless-stopped \
  --memory 256m --cpus 0.5 \
  --read-only --tmpfs /tmp --tmpfs /var/cache/nginx --tmpfs /var/run \
  nginx:1.27
```

| Option | Meaning |
|---|---|
| `-d` | Detached (background) |
| `-it` | Interactive with a terminal (`docker run -it ubuntu bash`) |
| `--rm` | Delete the container when it exits |
| `--name web` | Name (otherwise random) |
| `-p 8080:80` | Publish **host port 8080** → container port 80 (`-p 127.0.0.1:8080:80` = local only) |
| `-P` | Publish all `EXPOSE`d ports to random host ports |
| `-e KEY=value`, `--env-file f` | Environment variables |
| `-v name:/path` / `-v /host/path:/path` / `--mount` | Named volume / bind mount |
| `--network mynet` | Attach to a network |
| `-w /app` | Working directory |
| `--user 1000:1000` | Run as a non-root UID/GID |
| `--entrypoint sh` | Override the image's ENTRYPOINT |
| `--restart no \| on-failure \| always \| unless-stopped` | Restart policy |
| `--memory`, `--cpus`, `--pids-limit` | Resource limits |

| Option | Meaning |
|---|---|
| `--read-only`, `--cap-drop ALL`, `--security-opt no-new-privileges` | Hardening |
| `--init` | Small init process as PID 1 (signal handling, reaping zombies) |
| `--health-cmd`, `--health-interval` | Health check at run time |

### Lifecycle commands

| Command | Purpose |
|---|---|
| `docker ps` / `docker ps -a` | Running / all containers (with exit codes) |
| `docker logs -f --tail 100 web` | Follow logs (stdout/stderr of the main process) |
| `docker exec -it web sh` | Shell inside a running container ( `bash` if present) |
| `docker stop web` | SIGTERM, then SIGKILL after 10 s ( `-t 30` to wait longer) |
| `docker kill web` | SIGKILL immediately |
| `docker start / restart web` | Start a stopped container / restart |
| `docker rm web` / `docker rm -f web` | Remove (force = stop first) |
| `docker inspect web` | Full JSON: state, IP, mounts, env, restart count, OOMKilled |
| `docker inspect -f '{{.State.Status}} {{.State.ExitCode}}' web` | Pick fields |
| `docker stats` | Live CPU/memory/network per container |
| `docker top web` | Processes inside |
| `docker cp web:/etc/nginx/nginx.conf .` | Copy files out/in |
| `docker port web` | Port mappings |
| `docker container prune` | Remove all stopped containers |

**Container states:** created → running → (paused) → exited → removed. The container lives as long as its **main process (PID 1)** runs: if the command finishes, the container exits.

**Avoid** `docker commit` (snapshotting a modified container into an image): images must be reproducible from a Dockerfile.

## 8.5 Dockerfile in depth

### Instructions

| Instruction | Purpose | Notes |
|---|---|---|
| `FROM image:tag [AS name]` | Base image (and stage name) | First instruction (after optional `ARG`); pin versions |
| `ARG NAME=default` | Build-time variable | Not available at runtime; visible in image history (not for secrets) |

| Instruction | Purpose | Notes |
|---|---|---|
| `ENV KEY=value` | Runtime environment variable | Persisted in the image; defaults only, no secrets |
| `WORKDIR /app` | Working directory (created if missing) | Use instead of `RUN cd ...` |
| `COPY src dest` | Copy from the build context | `--chown=app:app`, `--from=build` (multi-stage) |
| `ADD src dest` | Like COPY + extracts tar archives + URLs | Prefer `COPY` unless you need tar extraction |
| `RUN command` | Execute at **build time**, creates a layer | Combine related commands, clean caches in the same RUN |
| `USER app` | User for following instructions and at runtime | Run as non-root |
| `EXPOSE 8000` | Documents the listening port | Doesn't publish anything |
| `VOLUME /data` | Marks a mount point | Data there isn't kept in image layers |
| `CMD [...]` | Default command / default arguments | Overridden by arguments to `docker run` |
| `ENTRYPOINT [...]` | Fixed executable | `CMD` becomes its default args |
| `HEALTHCHECK CMD ...` | Container health status | `--interval`, `--timeout`, `--retries`, `--start-period` |
| `LABEL key=value` | Metadata | OCI labels: `org.opencontainers.image.source`, `.version`, `.revision` |
| `STOPSIGNAL SIGTERM` | Signal sent by `docker stop` | |
| `SHELL ["/bin/bash", "-c"]` | Shell used by shell-form RUN | |

## Shell form vs exec form

```
CMD python app.py              # shell form: runs "/bin/sh -c 'python app.py'": sh is PID 1,
                               # SIGTERM goes to sh, app may not shut down gracefully
CMD ["python", "app.py"]       # exec form: python is PID 1 and receives signals → PREFERRED
```

## CMD vs ENTRYPOINT

| Dockerfile | `docker run image` runs | `docker run image --debug` runs |
|---|---|---|
| `CMD ["python", "app.py"]` | `python app.py` | `--debug` (replaces CMD!) |
| `ENTRYPOINT ["python", "app.py"]` | `python app.py` | `python app.py --debug` |
| `ENTRYPOINT ["python", "app.py"]` + `CMD ["--port", "8000"]` | `python app.py --port 8000` | `python app.py --debug` |

Pattern: `ENTRYPOINT` = the program, `CMD` = default arguments. Entrypoint **scripts** (`docker-entrypoint.sh`) do setup then `exec "$@"` so the app becomes PID 1.

## Build context and .dockerignore

`docker build -t app:1.0 .` sends the **build context** (the `.` folder) to the builder. `.dockerignore` excludes files: smaller/faster builds, no secrets or junk in images.

```
.git
.venv
__pycache__
*.pyc
node_modules
.env
*.pem
Dockerfile*
docker-compose*.yml
tests/
docs/
```

## Layer caching rules

The builder reuses a cached layer if the instruction **and** its inputs didn't change; once one layer changes, **all following layers rebuild**. Therefore: put things that change rarely first (base, OS packages, dependency files), and your source code **last**.

```
# GOOD: dependencies cached until requirements.txt changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# BAD: any code change reinstalls all dependencies
# COPY . .
# RUN pip install -r requirements.txt
```

## Production Dockerfile: Python web app

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*

RUN useradd --create-home --uid 10001 appuser

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY --chown=appuser:appuser . .

USER 10001
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD ["curl", "-fsS", "http://localhost:8000/health"]

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "app:app"]
```

Key points: pinned slim base · `PYTHONUNBUFFERED` so logs appear immediately · OS packages installed and apt lists removed **in the same layer** · non-root user with a **numeric UID** (lets Kubernetes verify `runAsNonRoot`) · dependencies before code · exec-form CMD · app binds to `0.0.0.0` (binding to `127.0.0.1` inside a container makes it unreachable from outside). Linters such as hadolint also suggest pinning apt package versions (`curl=8.5.0-2`); strict teams do, others rely on pinned base images + regular rebuilds.

### Node.js and Java examples

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
USER 1000
EXPOSE 3000
CMD ["node", "server.js"]
```

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar app.jar
RUN useradd -r -u 10001 app
USER 10001
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

`-XX:MaxRAMPercentage` makes the JVM respect the container's memory limit.

## 8.6 Multi-stage builds, BuildKit and tagging

### Multi-stage build

```
# syntax=docker/dockerfile:1
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o /out/api ./cmd/api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/api /api
USER 65532:65532
EXPOSE 8080
ENTRYPOINT ["/api"]
```

Compilers, source code and build tools stay in the first stage; the final image contains **only the binary** (a few MB, almost no CVEs). Python/Node apps use the same idea: build wheels or bundles in a "builder" stage, copy only results.

### BuildKit features

```
# Cache package downloads between builds (not stored in the image)
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt

# Use a secret during build WITHOUT leaving it in any layer
RUN --mount=type=secret,id=npm_token \
    NPM_TOKEN=$(cat /run/secrets/npm_token) npm ci
```

```
docker build --secret id=npm_token,env=NPM_TOKEN -t app:1.0 .
docker buildx build --platform linux/amd64,linux/arm64 -t ghcr.io/<user>/app:1.0 --push .
docker build --target build -t app:builder .        # build only up to a named stage
docker build --no-cache --pull -t app:1.0 .         # ignore cache, refresh base image
```

**Never** pass secrets with `ARG` / `ENV` or `COPY` them in: they end up in the image history.

### Tagging strategy

| Tag | Use |
|---|---|
| `1.4.2` | Release version (semantic) |
| `sha-3f9a1c2` (git commit) | Exact traceability from image to code; CI default |
| `1.4`, `1` | Floating "latest patch / latest minor" (convenience) |
| `latest` | Avoid in deployments (ambiguous); OK for local use |

Deploy the **same immutable image** to dev → staging → prod (promote, don't rebuild). Turn on **tag immutability** in registries.

## 8.7 Registries

| Registry | Notes |
|---|---|
| Docker Hub | Default public registry; **pull rate limits** for anonymous/free users: authenticate or mirror |
| Amazon ECR | Private per AWS account/region; IAM auth; scanning; lifecycle policies |
| GitHub Container Registry | `ghcr.io`; auth with `GITHUB_TOKEN` in Actions |
| Google Artifact Registry, Azure ACR | Other clouds |
| Harbor / Nexus / Artifactory | Self-hosted, with scanning and replication |

```
docker login                                        # Docker Hub
echo "$GHCR_TOKEN" | docker login ghcr.io -u <user> --password-stdin
aws ecr get-login-password --region ap-south-1 \
  | docker login --username AWS --password-stdin 123456789012.dkr.ecr.ap-south-1.amazonaws.com

docker tag shop-api:1.4.2 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
docker logout ghcr.io
```

`--password-stdin` keeps tokens out of shell history and process lists. Credentials are stored in `~/.docker/config.json` (use a credential helper).

## 8.8 Storage: volumes, bind mounts, tmpfs

```
container writable layer   → deleted with the container (don't keep data here)
named volume  (-v pgdata:/var/lib/postgresql/data)  → managed by Docker in /var/lib/docker/volumes
bind mount    (-v $(pwd)/src:/app)                  → a host directory/file mounted in
tmpfs         (--tmpfs /tmp)                        → in memory, gone at stop
```

| Type | Best for | Watch out |
|---|---|---|
| **Named volume** | Databases and persistent app data | Back it up; survives `docker rm` (removed with `docker volume rm` / `compose down -v`) |
| **Bind mount** | Development (live code), config files from host | Host path must exist; UID/permission mismatches; ties container to one host |
| tmpfs | Secrets/temp data that shouldn't touch disk | Lost on stop; uses RAM |

```
docker volume create pgdata
docker volume ls
docker volume inspect pgdata
docker run --rm -v pgdata:/data -v "$(pwd)":/backup alpine \
  tar -czf /backup/pgdata-backup.tgz -C /data .          # back up a volume
docker volume prune                                       # remove unused volumes (careful!)
```

**Permission problems** ("permission denied" writing to a mount): the container user's UID doesn't match the owner of the host directory. Fix by `chown`-ing the host path to that UID, running with `--user $(id -u):$(id -g)`, or using named volumes.

## 8.9 Networking

| Driver | Behaviour | Use |
|---|---|---|
| `bridge` (default `docker0`) | Private network on the host; NAT to the outside | Single containers |
| **User-defined bridge** | Like bridge + **automatic DNS by container name** + better isolation | Multi-container apps on one host (Compose creates one) |
| `host` | Container uses the host network stack directly (no `-p`, no isolation) | Performance-sensitive tools, Linux only |
| `none` | No network | Batch jobs that must be offline |
| `overlay` | Spans multiple hosts | Docker Swarm |
| `macvlan` / `ipvlan` | Container gets its own address on the physical network | Legacy apps needing a LAN IP |

```
docker network create shopnet
docker run -d --name db --network shopnet --env-file db.env \
  -v pgdata:/var/lib/postgresql/data postgres:16   # db.env (git-ignored): POSTGRES_PASSWORD
docker run -d --name api --network shopnet -p 8000:8000 -e DB_HOST=db shop-api:1.4.2
docker network inspect shopnet            # connected containers and their IPs
docker network connect othernet api       # attach to a second network
docker exec api getent hosts db           # name resolution via Docker's embedded DNS (127.0.0.11)
```

**Port publishing:** `-p 8080:80` creates NAT rules (iptables) from host:8080 to container:80. `-p 127.0.0.1:8080:80` exposes only on the host's loopback (safer for admin UIs/databases). Note that Docker's iptables rules can bypass `ufw`: don't publish database ports publicly by accident.

**Remember:** `localhost` inside a container = the container itself. To reach another container use its **name** on a shared network; to reach the host use `host.docker.internal` (Docker Desktop, or `--add-host=host.docker.internal:host-gateway` on Linux).

## 8.10 Configuration and secrets

| Need | Approach |
|---|---|
| Non-secret config | Environment variables ( `-e` , `--env-file` , Compose `environment` ), or a mounted config file |
| Secrets in development | `.env` file **excluded from Git**, loaded by Compose |
| Secrets in production | Orchestrator secrets injected at runtime: Kubernetes Secrets (+ External Secrets), ECS `secrets` from Secrets Manager/SSM, Docker/Compose secrets mounted as files in `/run/secrets/` |
| Build-time secrets | BuildKit `--mount=type=secret` |

**Twelve-factor app principle:** store config in the environment; the same image runs everywhere with different config.

Prefer **secret files** ( `*_FILE` variables, many official images support them, e.g. `POSTGRES_PASSWORD_FILE` ) over plain environment variables, because env vars show up in `docker inspect` , process listings and crash dumps.

## 8.11 Docker Compose in depth

**Compose** defines and runs multi-container applications from one YAML file ( `compose.yaml` ). Used for local development, integration tests in CI, and small single-host deployments.

### Full example: web app + PostgreSQL + Redis + nginx

```
# compose.yaml
name: shop

services:
  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    image: ghcr.io/thirupathi/shop-api:${API_TAG:-dev}
    environment:
      APP_ENV: ${APP_ENV:-dev}
      DB_HOST: db
      DB_NAME: shop
      DB_USER: shop
      REDIS_URL: redis://cache:6379/0
    env_file:
      - .env                          # secrets like DB_PASSWORD, not committed
    depends_on:
      db:
        condition: service_healthy    # wait for the DB health check, not just "started"
      cache:
        condition: service_started
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8000/health"]
      interval: 15s
      timeout: 3s
      retries: 5
      start_period: 20s
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
    networks: [backend, frontend]

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD must be set in .env}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped
    networks: [backend]

  cache:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redisdata:/data
    networks: [backend]

  proxy:
    image: nginx:1.27-alpine
    ports:
      - "80:80"
```

```
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [api]
    networks: [frontend]

  adminer:
    image: adminer
    ports:
      - "127.0.0.1:8081:8080"        # local-only admin UI
    profiles: [debug]                # only started with --profile debug
    networks: [backend]

volumes:
  pgdata:
  redisdata:

networks:
  frontend:
  backend:
```

| Key | Meaning |
|---|---|
| `build` / `image` | Build from a Dockerfile and/or the image name to use or tag |
| `environment`, `env_file` | Variables ( `${VAR:-default}` and `${VAR:?error}` interpolate from the shell or a `.env` file next to compose.yaml) |
| `ports` | `"host:container"`; quote them (YAML can misread `22:22`-style values) |
| `volumes` | Named volumes (declared at the bottom) or bind mounts ( `./path:/path:ro` ) |
| `depends_on` + `condition: service_healthy` | Start order **and** readiness |
| `healthcheck` | Health status shown in `docker compose ps` |
| `restart` | Restart policy |
| `deploy.resources.limits` | CPU/memory limits |
| `networks` | Isolation: `proxy` can't reach `db` directly here |
| `profiles` | Optional services |
| `command` / `entrypoint` | Override the image defaults |

## Compose commands

| Command | Purpose |
|---|---|
| `docker compose up -d` | Create/start everything in the background |
| `docker compose up -d --build` | Rebuild images first |
| `docker compose ps` | Status and health |
| `docker compose logs -f api` | Follow one service's logs |
| `docker compose exec api sh` | Shell in a running service |
| `docker compose run --rm api pytest` | One-off command in a new container |
| `docker compose restart api` / `stop` / `start` | Control services |

| Command | Purpose |
|---|---|
| `docker compose down` | Stop and remove containers and networks |
| `docker compose down -v` | ...and **delete volumes** (data loss!) |
| `docker compose config` | Show the final merged configuration (validate files) |
| `docker compose pull` / `build` | Update images / build |
| `docker compose up -d --scale api=3` | Several replicas (without fixed host ports) |
| `docker compose --profile debug up -d` | Include profile services |
| `docker compose -f compose.yaml -f compose.prod.yaml up -d` | Layer override files |

**Override files:** `compose.override.yaml` is applied automatically (dev settings like bind mounts and debug ports); production uses explicit `-f` files.

## 8.12 Resource limits, logging and restart behaviour

### Limits

| Limit | Option | When exceeded |
|---|---|---|
| Memory | `--memory 512m` | Kernel OOM-kills the process → exit code **137**, `OOMKilled: true` |
| CPU | `--cpus 1.5` | Throttled (slower), not killed |
| Processes | `--pids-limit 200` | Can't fork more (protects against fork bombs) |
| Open files | `--ulimit nofile=65536:65536` | "Too many open files" |

Languages must respect container limits: modern JVMs do (`MaxRAMPercentage`), Node may need `--max-old-space-size`, Python worker counts should match CPU limits.

### Logging

Containers should log to **stdout/stderr**; Docker captures it with a **logging driver**. Default `json-file` grows forever unless you rotate it:

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

(`/etc/docker/daemon.json`, then `sudo systemctl restart docker`.) Other drivers: `local`, `journald`, `awslogs` (CloudWatch), `fluentd`, `gelf`. In Kubernetes, node agents (Fluent Bit, Alloy) collect container logs.

### Restart policies

`no` (default) · `on-failure[:max]` (restart on non-zero exit) · `always` (also after daemon restart) · `unless-stopped` (like always unless you stopped it). For real high availability use an orchestrator (Kubernetes, ECS), not restart policies.

## 8.13 Container security

| Practice | How |
|---|---|
| Minimal, trusted, pinned base images | slim/distroless/hardened images; pin version (or digest); rebuild regularly |
| **Run as non-root** | `USER` in Dockerfile; `--user`; Kubernetes `runAsNonRoot` |
| Least privileges | `--cap-drop ALL` (add back only what's needed, e.g. `NET_BIND_SERVICE`), `--security-opt no-new-privileges` |
| Never `--privileged` | Gives the container nearly full host access |
| **Never mount** `/var/run/docker.sock` into untrusted containers | Equals root on the host |
| Read-only root file system | `--read-only` + `tmpfs` for writable paths |
| No secrets in images | BuildKit secrets, runtime secret injection; scan images for secrets |
| **Scan images** | `trivy image app:1.0`, Docker Scout, Grype, ECR/Inspector scanning; fail CI on CRITICAL |
| SBOM | `syft app:1.0` / `trivy image --format cyclonedx` lists everything inside |
| Sign and verify images | `cosign sign` / `cosign verify`; enforce in Kubernetes admission policies |
| Limit resources | Memory/CPU/PIDs limits prevent noisy neighbours and DoS |
| Kernel protections | Default seccomp and AppArmor/SELinux profiles stay enabled |
| Rootless Docker / Podman | Daemon and containers without root on the host |
| Lint Dockerfiles | `hadolint Dockerfile` |
| Benchmarks | CIS Docker Benchmark ( `docker-bench-security` ) |

```
trivy image --severity HIGH,CRITICAL --exit-code 1 shop-api:1.4.2   # fail if serious CVEs
trivy fs --scanners vuln,secret,misconfig .                         # scan source/Dockerfiles
```

## 8.14 Troubleshooting

### Exit codes

| Exit code | Meaning | Typical cause |
|---|---|---|
| 0 | Process finished successfully | Command isn't a long-running server (container "exits immediately" by design) |
| 1 | Application error | Read `docker logs` |
| 126 | Command not executable | Missing `chmod +x` on entrypoint script |
| 127 | Command not found | Wrong CMD/ENTRYPOINT path, binary missing in the image (e.g. `bash` in Alpine/distroless) |
| 137 | Killed (SIGKILL) | **OOM** ( `docker inspect` → `OOMKilled: true` ) or `docker kill` |

| Exit code | Meaning | Typical cause |
|---|---|---|
| 139 | Segmentation fault | Native crash, architecture/library mismatch |
| 143 | SIGTERM | Normal `docker stop` |

## Common problems

| Symptom | Investigation / fix |
|---|---|
| Container exits immediately | `docker ps -a` (exit code) → `docker logs` → run interactively: `docker run -it --entrypoint sh image` |
| Restart loop | Logs from the previous run (`docker logs`), missing env/secret, dependency not ready (add health checks/retries), OOM |
| App unreachable from host | App listening on `127.0.0.1` inside the container (must be `0.0.0.0`) · missing/wrong `-p` · firewall · wrong port |
| `port is already allocated` | Another container/process uses the host port (`docker ps`, `ss -tulpn`) |
| Containers can't reach each other | Not on the same user-defined network · using `localhost` · wrong service name/port |
| `exec format error` | Image built for another CPU architecture (arm64 vs amd64) → `--platform` / buildx multi-arch |
| `permission denied` on volume | UID mismatch between container user and host files |
| `no space left on device` | `docker system df` → `docker system prune` (careful with `-a --volumes`), log rotation, remove old images |
| Build is slow / always rebuilds | Cache busted early (COPY . before deps), huge build context (missing `.dockerignore`) |
| `toomanyrequests` pulling images | Docker Hub rate limit → log in, use a mirror/ECR pull-through cache |
| `Cannot connect to the Docker daemon` | Daemon not running (`systemctl status docker`) or user not in `docker` group |
| Debugging a distroless image | `docker debug` (Docker Desktop) or run a debug sidecar sharing the namespace: `docker run -it --pid=container:app --net=container:app busybox` |

## Diagnostic commands

```
docker ps -a
docker logs --tail 200 -f app
docker inspect app --format '{{json .State}}' | jq
docker exec -it app sh -c 'env | sort; ss -tulpn 2>/dev/null || netstat -tulpn'
docker events --since 10m             # what the daemon did (starts, dies, OOM)
docker system df                      # disk usage by images/containers/volumes/cache
```

## 8.15 Docker in CI/CD

```
# GitHub Actions excerpt: build, scan, push
- uses: docker/setup-buildx-action@v3
- uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}
- uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ghcr.io/${{ github.repository }}:sha-${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

| Topic | Practice |
|---|---|
| Tags | Commit SHA for every build, semantic version on release tags |
| Cache | BuildKit cache (registry or CI cache) for fast builds |
| Scan | Trivy/Scout step that fails on critical vulnerabilities |
| Build without a Docker daemon | Kaniko, Buildah, BuildKit rootless (common inside Kubernetes runners) |
| Docker-in-Docker vs socket mounting | Both have security trade-offs; prefer rootless builders or managed runners |
| Promotion | Build once, push, deploy the same digest to every environment |

## 8.16 Docker vs orchestration

Docker runs containers on **one host**. Production systems need scheduling across many hosts, self-healing, scaling, rolling updates, service discovery and secrets: that's **orchestration**: **Kubernetes** (industry standard), **Amazon ECS**, Docker Swarm (simple, less used), Nomad. Compose is for local/dev and small single-host setups.

## 8.17 Lab: containerize a Python web app

```
# api/app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)


@app.get("/health")
def health():
    return jsonify(status="ok")


@app.get("/")
def index():
    return jsonify(message="Hello from shop-api", env=os.getenv("APP_ENV", "dev"))
```

```
# api/requirements.txt
flask==3.0.3
gunicorn==23.0.0
```

1. Write the Dockerfile from 8.5 (Python) in `api/`, and a `.dockerignore`.
2. `docker build -t shop-api:1.0 api/` → `docker images shop-api` (note the size).
3. `docker run -d --name api -p 8000:8000 -e APP_ENV=lab shop-api:1.0` → `curl localhost:8000/` and `/health`.
4. `docker exec -it api sh -c 'id; ps aux'` → confirm it runs as `appuser`, not root.
5. Change a line in `app.py`, rebuild: only the last layers rebuild (cache).
6. `trivy image shop-api:1.0` → read the findings; try a different base image and compare.
7. Write a `compose.yaml` with the API + PostgreSQL (health check, volume, `.env` for the password) → `docker compose up -d` → `docker compose ps`.
8. Stop the DB container and watch the API's behaviour; read logs; start it again.
9. **Cleanup:** `docker compose down -v`, `docker rm -f api`, `docker image prune`.

## 8.18 Command reference

| Area | Commands |
|---|---|
| Images | `pull`, `images`, `build -t`, `tag`, `push`, `rmi`, `history`, `image inspect`, `image prune` |
| Containers | `run`, `ps -a`, `logs -f`, `exec -it`, `stop`, `start`, `restart`, `rm -f`, `inspect`, `stats`, `top`, `cp`, `container prune` |
| Volumes | `volume create/ls/inspect/rm/prune` |
| Networks | `network create/ls/inspect/connect/rm` |
| Compose | `compose up -d --build`, `ps`, `logs -f`, `exec`, `run --rm`, `down [-v]`, `config` |
| Build | `buildx build --platform ... --push`, `build --target`, `--secret`, `--no-cache` |
| System | `version`, `info`, `system df`, `system prune`, `events`, `login/logout` |

## 8.19 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | Container vs VM? | Containers share the host kernel and isolate processes (namespaces/cgroups); VMs virtualize hardware with separate kernels |
| 2 | Image vs container? | Image = read-only layered template; container = running instance with a writable layer |
| 3 | What are namespaces and cgroups? | Namespaces isolate what a process sees (PIDs, network, mounts…); cgroups limit resources |
| 4 | What is a Docker layer and how does caching work? | Each instruction creates a layer; unchanged instructions reuse cache until the first changed one |
| 5 | CMD vs ENTRYPOINT? | ENTRYPOINT is the fixed executable; CMD gives default args (or the default command), easily overridden |
| 6 | COPY vs ADD? | COPY copies files; ADD also extracts tars and fetches URLs. Prefer COPY |

| # | Question | Answer |
|---|---|---|
| 7 | Shell form vs exec form? | Exec form runs the process as PID 1 so it receives signals; shell form wraps it in `sh -c` |
| 8 | How do you reduce image size? | Slim/distroless bases, multi-stage builds, `.dockerignore`, clean caches in the same RUN, fewer dependencies |
| 9 | What is a multi-stage build? | Several FROM stages; build tools in early stages, only artifacts copied to the final image |
| 10 | Volume vs bind mount? | Docker-managed storage for persistent data vs a host path mounted into the container |
| 11 | How do containers communicate? | On a user-defined network by container/service name via embedded DNS |
| 12 | What does `-p 8080:80` do? | Maps host port 8080 to container port 80 |
| 13 | Why does my app work in the container but not from the host? | It's bound to 127.0.0.1 inside the container, or the port isn't published |
| 14 | Exit code 137? | SIGKILL, usually OOM because of the memory limit |
| 15 | How do you handle secrets in Docker? | Never bake into images; inject at runtime (orchestrator secrets, files in /run/secrets), BuildKit secrets for builds |
| 16 | How do you secure containers? | Non-root, minimal images, scanning, drop capabilities, read-only FS, no privileged/docker.sock, resource limits, signing |
| 17 | What is Docker Compose for? | Defining and running multi-container apps on one host (dev, testing, small deployments) |
| 18 | `depends_on` guarantees the DB is ready? | Only start order; use `condition: service_healthy` with a health check (and retries in the app) |
| 19 | Why not use the `latest` tag in production? | It's mutable and ambiguous; use version/SHA tags or digests for reproducibility and rollback |
| 20 | How do you debug a container that keeps restarting? | Exit code + `docker logs` + `docker inspect` (OOMKilled, env) + run it interactively with a shell entrypoint |
| 21 | What is OCI? | Open standards for container images and runtimes so images work across tools (Docker, containerd, Podman, Kubernetes) |
| 22 | Docker vs Kubernetes? | Docker builds/runs containers on a host; Kubernetes orchestrates containers across a cluster |

## 8.20 Self-check: I can…

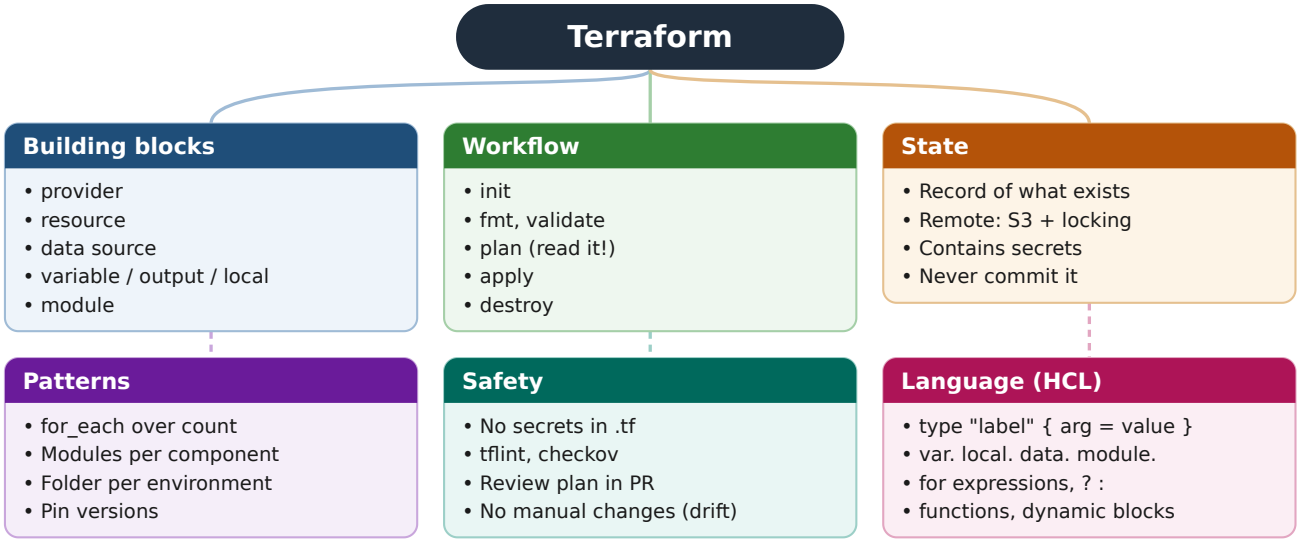
- [ ] Explain containers vs VMs, namespaces, cgroups, overlay file systems and OCI/containerd/runc.
- [ ] Install Docker and explain the security impact of the `docker` group.
- [ ] Name, pull, inspect, tag and remove images; choose a suitable base image.
- [ ] Run containers with ports, env, volumes, limits, restart policies and hardening options.
- [ ] Write production Dockerfiles: caching order, exec form, non-root, health checks, `.dockerignore`.
- [ ] Explain CMD vs ENTRYPOINT behaviour precisely.
- [ ] Build multi-stage and multi-architecture images; use BuildKit cache and secrets.
- [ ] Tag and push images to Docker Hub, GHCR and ECR.
- [ ] Use volumes, bind mounts and tmpfs correctly and back up a volume.
- [ ] Configure container networking and explain port publishing and container DNS.
- [ ] Write a full Compose stack with health checks, profiles, overrides and limits.

- [ ] Configure log rotation and understand OOM behaviour.
- [ ] Harden and scan images; generate an SBOM.
- [ ] Troubleshoot by exit code, logs, inspect, events and disk usage.
- [ ] Build and push images in CI with caching and scanning.

**REMEMBER**

- Port mapping is **-p HOST:CONTAINER**.
- Dockerfile order: **base → dependencies → code** (code changes most, so it goes last).
- Exit code **137** = killed (usually out of memory). Exit **1** = application error → read the logs.
- Inside a container, `localhost` means the container itself.

# 9. Terraform (Infrastructure as Code)




## 9.0 Why Infrastructure as Code

Clicking resources together in a cloud console is slow, error-prone, undocumented and impossible to reproduce exactly. **Infrastructure as Code (IaC)** describes infrastructure in text files that are versioned in Git, reviewed in pull requests, tested in CI and applied automatically.

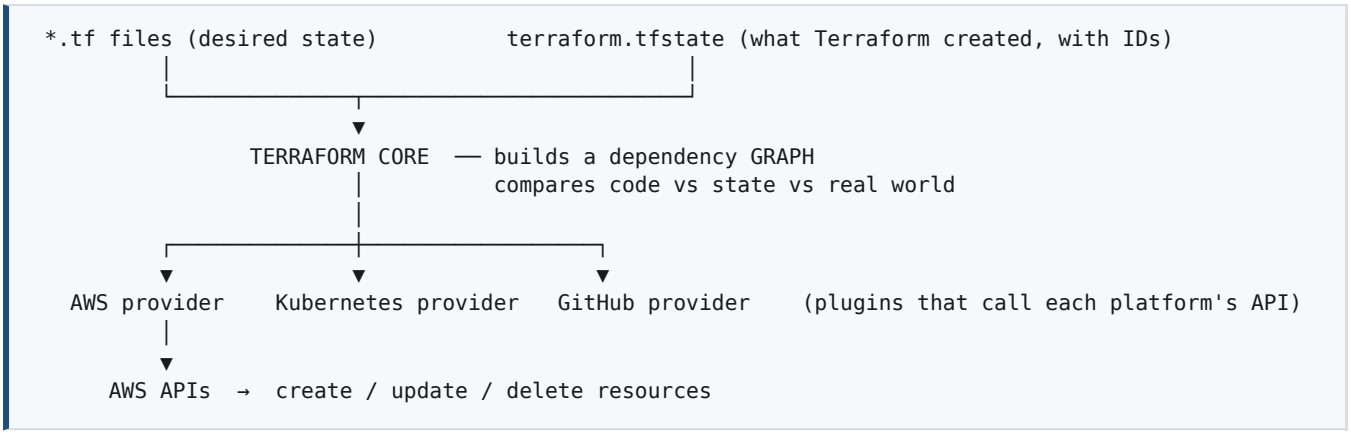
| Benefit | What it means in practice |
|---|---|
| Repeatable | Build identical dev, staging and prod; rebuild a region for DR |
| Reviewable | Changes go through pull requests and `plan` output |
| Auditable | Git history shows who changed what and why |
| Fast | Whole environments in minutes |
| Self-documenting | The code *is* the description of the infrastructure |
| Safer | Drift detection, policy checks, no "snowflake" servers |

## 9.1 IaC concepts and the tool landscape

| Concept | Meaning |
|---|---|
| **Declarative** | You describe the desired end state ("3 subnets exist"); the tool works out the steps (Terraform, CloudFormation, Kubernetes YAML) |
| Imperative | You write the steps ("create subnet, then…") (scripts, AWS CLI) |
| **Idempotent** | Applying the same code again changes nothing if reality already matches |
| **Drift** | Real infrastructure differs from code (someone changed it manually) |
| Immutable infrastructure | Replace resources instead of modifying them in place (new AMI/image → new instances) |
| Provisioning vs configuration | Creating infrastructure (Terraform) vs configuring inside servers (Ansible) |

| Tool | Type | Notes |
|---|---|---|
| **Terraform** | Declarative, multi-cloud, HCL | Most popular IaC tool; huge provider ecosystem; Business Source License since 2023 |
| **OpenTofu** | Open-source fork of Terraform (Linux Foundation) | Drop-in compatible CLI ( `tofu` ), same HCL |
| CloudFormation / CDK | AWS-native | Managed state; CDK uses programming languages |
| Pulumi | Real programming languages | TypeScript/Python/Go for infrastructure |
| Crossplane | Kubernetes-native IaC | Cloud resources as Kubernetes objects |
| Ansible | Configuration management (can provision too) | Procedural playbooks; complements Terraform |

## 9.2 How Terraform works




**The core workflow:**

```
write code → terraform init → terraform plan → review → terraform apply → (later) destroy
```

| Step | What happens |
|---|---|
| `init` | Downloads providers and modules, configures the backend (state location), writes `.terraform.lock.hcl` |
| `plan` | Refreshes current state from the real APIs, compares with the code, prints the change set |
| `apply` | Executes the plan in dependency order (in parallel where possible), updates the state |
| `destroy` | Deletes everything the configuration manages |

**Plan symbols:** `+` create · `~` update in place · `-` destroy · `-/+` destroy then create (**replace**) · `+/-` create then destroy (with `create_before_destroy` ) · `<=` read (data source). Always read *why* something will be replaced ("forces replacement").

## 9.3 Installation and CLI

**Install:** download the binary from HashiCorp (or `apt` / `dnf` from the HashiCorp repository, `brew install terraform` ), or use a version manager ( `tfenv` ) so each project uses its pinned version. Check: `terraform version` .

| Command | Purpose |
|---|---|
| `terraform init` | Initialize (run after cloning, adding providers/modules, or changing the backend). `-upgrade` updates providers within constraints; `-reconfigure` / `-migrate-state` for backend changes |
| `terraform fmt -recursive` | Format code to the standard style |
| `terraform validate` | Syntax and internal consistency check |
| `terraform plan` | Preview changes; `-out=tfplan` saves the exact plan |
| `terraform apply` / `apply tfplan` | Apply (asks for confirmation) / apply a saved plan exactly |
| `terraform destroy` | Delete everything in this configuration |
| `terraform output [-json] [name]` | Show outputs |
| `terraform show` | Show state or a saved plan in human-readable form |
| `terraform state list / show / mv / rm / pull` | Inspect and manipulate state (see 9.6) |
| `terraform import ADDRESS ID` | Bring an existing resource under management (prefer `import` blocks, 9.15) |
| `terraform console` | Evaluate expressions interactively |
| `terraform graph` | Dependency graph (DOT format) |
| `terraform providers` | Providers used by the configuration and modules |
| `terraform workspace list/new/select` | Workspaces (9.10) |
| `terraform force-unlock LOCK_ID` | Remove a stuck state lock (only when sure nobody is running) |
| `terraform test` | Run `.tftest.hcl` tests (9.14) |
| `terraform login` | Authenticate to HCP Terraform / a private registry |

| Useful flags | Meaning |
|---|---|
| `-var="env=prod"`, `-var-file=prod.tfvars` | Set variables |
| `-replace=aws_instance.web` | Force replacement of one resource |
| `-refresh-only` | Update state to match reality without changing infrastructure (drift review) |
| `-target=module.vpc` | Limit to some resources: **emergency use only**, can leave state inconsistent |
| `-auto-approve` | Skip confirmation (CI only, after a reviewed plan) |
| `-lock-timeout=5m` | Wait for a state lock |
| `-parallelism=10` | Concurrent operations (default 10) |
| `-input=false` | Never prompt (CI) |

| Environment variable | Use |
|---|---|
| `TF_VAR_name` | Set input variable `name` (good for secrets in CI) |
| `TF_LOG=DEBUG`, `TF_LOG_PATH=tf.log` | Debug logging (provider API calls) |
| `TF_CLI_ARGS_plan="-parallelism=20"` | Default flags per command |
| `TF_IN_AUTOMATION=1` | Cleaner output in CI |
| `AWS_PROFILE`, `AWS_REGION` | Credentials/region for the AWS provider |

**Files Terraform creates:**

| File/folder | Commit to Git? |
|---|---|
| `*.tf`, `*.tftest.hcl` | Yes |
| `.terraform.lock.hcl` (provider versions + checksums) | **Yes** |
| `.terraform/` (downloaded providers/modules) | No |
| `terraform.tfstate`, `*.tfstate.backup` | **Never** (remote backend instead) |
| `*.tfvars` with secrets/env specifics | Usually no (or only non-secret ones) |
| `tfplan` files | No (they can contain secrets) |

## 9.4 The Terraform language (HCL)

### Anatomy of a block

```
# block type = resource | label 1 = resource type | label 2 = your name for it
resource "aws_instance" "web" {
  ami           = "ami-0abc123"     # ARGUMENT: name = value
  instance_type = var.instance_type # a value can reference other things
  tags = {                          # map value
    Name = "web-1"
  }
  root_block_device { # NESTED BLOCK (no "=")
    volume_size = 20
  }
}
```

- **Block** = `type "labels" { body }`. **Argument** = `name = value`. **Nested block** = `name { ... }` without `=`.
- The resource's **address** is `aws_instance.web`. Other code reads its attributes, e.g. `aws_instance.web.public_ip`.
- Terraform reads **all** `.tf` **files in the folder** together; file names are only for humans.

### Block types

| Block | Purpose | Example |
|---|---|---|
| `terraform` | Terraform version, required providers, backend (state location) | `terraform { required_version = ">= 1.6" }` |
| `provider` | Configures a platform's API (region, credentials from environment/role) | `provider "aws" { region = "ap-south-1" }` |

| Block | Purpose | Example |
|---|---|---|
| `resource` | Something Terraform **creates and manages** | `resource "aws_s3_bucket" "logs" { ... }` |
| `data` | **Reads** something that already exists | `data "aws_availability_zones" "available" {}` |
| `variable` | An **input** | `variable "env" { type = string }` |
| `locals` | **Computed helper values** inside the module | `locals { name = "${var.env}-web" }` |
| `output` | A value printed after apply / used by other modules | `output "ip" { value = aws_instance.web.public_ip }` |
| `module` | Uses a reusable folder of Terraform code | `module "vpc" { source = "./modules/vpc" }` |
| `moved` / `import` | Refactor without destroying / adopt existing resources | `import` block with `to = aws_s3_bucket.logs` and `id = "my-bucket"` |

## How to reference things

| You write | It means |
|---|---|
| `var.env` | Input variable `env` |
| `local.name` | Local value `name` |
| `aws_vpc.main.id` | Attribute `id` of resource `aws_vpc` named `main` |
| `data.aws_ami.ubuntu.id` | Attribute of a data source |
| `module.vpc.vpc_id` | Output `vpc_id` of module `vpc` |
| `aws_subnet.private[0].id` | Item 0 when the resource uses `count` |
| `aws_subnet.this["public-a"].id` | Item by key when the resource uses `for_each` |
| `each.key` / `each.value` | Current item inside a `for_each` resource |
| `count.index` | Current number (0, 1, 2…) inside a `count` resource |
| `path.module` | Folder of the current module (for files/templates) |

**Implicit dependencies:** referencing `aws_vpc.main.id` inside a subnet tells Terraform to create the VPC first. Use `depends_on` only when there is no reference.

## Data types

| Type | Example |
|---|---|
| `string` | `"t3.micro"` |
| `number` | `3` |
| `bool` | `true` |
| `list(string)` | `["a", "b"]` — ordered, accessed by index `[0]` |
| `set(string)` | `toset(["a", "b"])` — unique, no order |
| `map(string)` | `{ env = "dev", team = "ops" }` — accessed by key `["env"]` |
| `object({...})` | `{ name = string, size = number }` — fixed structure |

## Variables in detail

```
variable "instance_type" {
  description = "EC2 size for the web tier" # documents the input
  type        = string                      # type constraint
  default     = "t3.micro"                  # optional: no default = required input

  validation { # reject bad values early
    condition     = contains(["t3.micro", "t3.small"], var.instance_type)
    error_message = "Use t3.micro or t3.small."
  }
}

variable "db_password" {
  type      = string
  sensitive = true # hidden in plan/apply output (still stored in state!)
}
```

**Ways to set variables (later ones win):** `default` → `terraform.tfvars` / `*.auto.tfvars` → `-var-file=prod.tfvars` → `-var="env=prod"`. Environment variable form: `TF_VAR_db_password`.

## Expressions

```
locals {
  name        = "${var.project}-${var.env}"                    # string interpolation
  is_prod     = var.env == "prod"                              # comparison → bool
  size        = local.is_prod ? "t3.small" : "t3.micro"        # conditional: cond ? a : b
  upper_names = [for s in var.servers : upper(s)]              # for expression → list
  name_map    = { for s in var.servers : s => "${s}.internal" } # for expression → map
  subnet_ids  = aws_subnet.private[*].id                       # splat: every id
}
```

## Useful built-in functions

| Function | Example | Result |
|---|---|---|
| `length` | `length(["a","b"])` | `2` |
| `upper` / `lower` | `upper("dev")` | `"DEV"` |
| `join` / `split` | `join(",", ["a","b"])` | `"a,b"` |
| `merge` | `merge(local.tags, { Name = "web" })` | Combined map |
| `concat` | `concat(["a"], ["b"])` | `["a","b"]` |
| `lookup` | `lookup(var.sizes, "prod", "t3.micro")` | Value or default |
| `contains` | `contains(["a","b"], "a")` | `true` |
| `cidrsubnet` | `cidrsubnet("10.0.0.0/16", 8, 1)` | `"10.0.1.0/24"` |
| `file` / `templatefile` | `templatefile("${path.module}/init.sh.tftpl", { port = 80 })` | File content (with variables) |
| `try` / `coalesce` | `try(var.cfg.size, "small")` | Fallback when missing |
| `toset` / `tolist` | `toset(var.names)` | Type conversion |

Try any function in `terraform console`.

## Meta-arguments (work on every resource)

| Meta-argument | Purpose |
|---|---|
| `count = 3` | Create N copies ( `count.index` ) |
| `for_each = toset([...])` or a map | One copy per item ( `each.key` , `each.value` ) — preferred |
| `depends_on = [aws_iam_role.x]` | Force an order when there is no reference |
| `provider = aws.us_east` | Use an aliased provider (e.g. second region) |
| `lifecycle { ... }` | `create_before_destroy` , `prevent_destroy` , `ignore_changes` |

**Dynamic block** (repeat a nested block from a list):

```
variable "ingress_ports" {
  type    = list(number)
  default = [80, 443]
}

resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

## Complete annotated example: VPC + public subnet + web server (single file)

```
# ---------- versions.tf ----------
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws" # where to download the provider
      version = "~> 5.0"        # any 5.x version
    }
  }
}

provider "aws" {
  region = var.region # credentials come from your AWS CLI profile / IAM role
  default_tags {
    tags = local.common_tags # added to every resource automatically
  }
}

# ---------- variables.tf ----------
variable "region" {
  type    = string
  default = "ap-south-1"
}

variable "env" {
  type        = string
  description = "Environment name, e.g. dev or prod"
}

variable "my_ip_cidr" {
  type        = string
  description = "Your public IP for SSH, e.g. 203.0.113.10/32"
}

# ---------- main.tf ----------
locals {
  name        = "lab-${var.env}"
  common_tags = { Project = "devops-lab", Env = var.env, ManagedBy = "terraform" }
}

data "aws_availability_zones" "available" {
  state = "available" # read the AZ names of this region
}

data "aws_ami" "ubuntu" { # find the newest Ubuntu 24.04 image
  most_recent = true
  owners      = ["099720109477"] # Canonical's account
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags                 = { Name = "${local.name}-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id # reference → VPC is created first
}

resource "aws_subnet" "public" {
```

```
  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(aws_vpc.main.cidr_block, 8, 1) # 10.0.1.0/24
  availability_zone       = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true
  tags                    = { Name = "${local.name}-public-a" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"                 # everything else ...
    gateway_id = aws_internet_gateway.igw.id # ... goes to the internet gateway
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "web" {
  name   = "${local.name}-web"
  vpc_id = aws_vpc.main.id

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "SSH only from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip_cidr]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1" # all protocols
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data              = <<-EOT
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
  EOT
  tags                   = { Name = "${local.name}-web" }
}

# ---------- outputs.tf ----------
output "web_url" {
  description = "Open this in a browser"
  value       = "http://${aws_instance.web.public_ip}"
}
```

Run it: `terraform init` → `terraform plan -var="env=dev" -var="my_ip_cidr=<your-ip>/32"` → `terraform apply ...` → open the URL → **`terraform destroy ...`** when finished.

**Naming conventions:** snake_case names (`aws_instance.web_server`), one purpose per file (`versions.tf`, `variables.tf`, `main.tf`, `outputs.tf`), `this` as the name when a module has only one resource of a type, run `terraform fmt` before every commit.

## 9.5 Providers in depth

A **provider** is a plugin that knows one platform's API: `hashicorp/aws`, `hashicorp/azurerm`, `hashicorp/google`, `hashicorp/kubernetes`, `hashicorp/helm`, `integrations/github`, `hashicorp/random`, `hashicorp/tls`, `cloudflare/cloudflare`, `grafana/grafana`, `datadog/datadog`… thousands are on registry.terraform.io.

### Version constraints and the lock file

```
terraform {
  required_version = ">= 1.6, < 2.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.70" # >= 5.70, < 6.0
    }
    random = {
      source  = "hashicorp/random"
      version = ">= 3.6"
    }
  }
}
```

| Constraint | Means |
|---|---|
| `= 5.70.0` | Exactly |
| `>= 5.70` | At least |
| `~> 5.70` | Any 5.x from 5.70 (no 6.0) |
| `~> 5.70.0` | Any 5.70.x patch |

`.terraform.lock.hcl` records the exact versions and checksums chosen by `init`: commit it so every machine and CI uses identical providers; update deliberately with `terraform init -upgrade`.

### Configuring the AWS provider

```
provider "aws" {
  region = "ap-south-1"

  default_tags {
    tags = {
      Project   = "shop"
      ManagedBy = "terraform"
    }
  }
}

# Second provider configuration for another region (alias)
provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"
}

resource "aws_acm_certificate" "cdn" {
  provider          = aws.us_east_1 # CloudFront certificates must live in us-east-1
  domain_name       = "shop.example.com"
  validation_method = "DNS"
}

# Deploy into another account by assuming a role
provider "aws" {
  alias  = "prod"
  region = "ap-south-1"
  assume_role {
    role_arn     = "arn:aws:iam::222222222222:role/terraform-deploy"
    session_name = "terraform"
  }
}
```

**Authentication (never hard-code keys in `.tf` files):** environment variables, `AWS_PROFILE` / SSO, an instance/container role, or **OIDC from CI** (GitHub Actions assumes a deploy role). Terraform uses the same credential chain as the AWS CLI.

## 9.6 State in depth

### What state is and why it matters

`terraform.tfstate` is a JSON file mapping each resource address in your code ( `aws_instance.web` ) to the **real object ID** ( `i-0abc…` ) and its last known attributes. Terraform needs it to know what it manages, compute diffs, and track dependencies and metadata.

**State is sensitive:** it can contain passwords, private keys and connection strings in plain text (even for `sensitive` values). Protect it like a secret.

### Local vs remote state

| | Local ( `terraform.tfstate` on disk) | Remote backend |
|---|---|---|
| Teamwork | Impossible (everyone has a different copy) | Shared, single source of truth |
| Locking | None | Prevents two applies at once |
| Safety | Lost laptop = lost state | Versioned, encrypted, backed up |
| Use | Personal experiments only | **Everything real** |

### S3 backend (AWS)

```
terraform {
  backend "s3" {
    bucket       = "acme-terraform-state-123456789012"
    key          = "prod/network/terraform.tfstate" # one key per stack/environment
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true # native S3 locking (Terraform 1.10+); older setups used a DynamoDB table
  }
}
```

**State bucket requirements:** versioning on (recover old states), encryption (SSE-KMS), Block Public Access, a bucket policy limiting access to the Terraform roles, ideally in a separate "tooling" account. Create the bucket once (bootstrap) with a small separate configuration or the CLI. Backend blocks can't use variables; pass values with `terraform init -backend-config=backend-prod.hcl` for multiple environments.

Other backends: HCP Terraform / Terraform Enterprise (remote runs, policy, UI), `gcs`, `azurerm`, `kubernetes`, `pg`.

### State commands

| Command | Use |
|---|---|
| `terraform state list` | All managed resource addresses |
| `terraform state show aws_vpc.main` | Attributes of one resource |
| `terraform state mv aws_instance.web aws_instance.app` | Rename/move in state (prefer `moved` blocks in code) |
| `terraform state rm aws_s3_bucket.old` | Stop managing a resource **without deleting it** (prefer `removed` blocks) |
| `terraform state pull > backup.tfstate` | Download the current state (backup before surgery) |
| `terraform state push` | Upload a state file (dangerous; last resort) |
| `terraform apply -refresh-only` | Accept real-world changes into state after review |

### Drift

Someone changed a security group in the console → next `plan` shows Terraform wants to revert it. Decide: **revert** (apply the code) or **adopt** (change the code to match). Prevent drift with least-privilege console access, scheduled `plan` jobs that alert on drift, and AWS Config.

### Splitting state

One giant state = slow plans and a huge blast radius. Split by **environment** and **component/layer** (network, data, compute, DNS), each with its own state key and pipeline. Share values between stacks with outputs read through `terraform_remote_state`, data sources (look resources up by tags), or SSM parameters.

```
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "acme-terraform-state-123456789012"
    key    = "prod/network/terraform.tfstate"
    region = "ap-south-1"
  }
}

locals {
  private_subnet_ids = data.terraform_remote_state.network.outputs.private_subnet_ids
}
```

## 9.7 Data sources, outputs and locals

| Construct | Purpose | Example |
|---|---|---|
| **Data source** | Read existing information (not managed by this config) | Latest AMI, account ID, existing VPC by tag, IAM policy documents, secrets |
| **Output** | Expose values: CLI display, CI, other stacks, parent modules | VPC ID, ALB DNS name |
| **Local** | Name an expression to avoid repetition | Common tags, name prefixes |

```
data "aws_caller_identity" "current" {}
data "aws_region" "current" {}

data "aws_vpc" "shared" {
  tags = { Name = "shared-vpc" }
}

data "aws_iam_policy_document" "read_assets" {
  statement {
    actions   = ["s3:GetObject"]
    resources = ["arn:aws:s3:::shop-assets/*"]
  }
}

data "aws_secretsmanager_secret_version" "db" {
  secret_id = "shop/prod/db"
}

locals {
  account_id = data.aws_caller_identity.current.account_id
  name       = "shop-${var.env}"
}

output "account_id" {
  value = local.account_id
}

output "db_connection" {
  value     = jsondecode(data.aws_secretsmanager_secret_version.db.secret_string)["host"]
  sensitive = true # hidden in CLI output (still stored in state)
}
```

`aws_iam_policy_document` builds IAM JSON in HCL (validated, composable) instead of hand-written JSON strings.

## 9.8 Modules in depth

A **module** is a folder of `.tf` files with inputs (variables) and outputs. Every configuration is a module (the **root module**); modules it calls are **child modules**.

```
modules/network/
├── main.tf         # resources
├── variables.tf    # inputs (typed, documented, validated)
├── outputs.tf      # outputs
├── versions.tf     # required_providers (no provider blocks inside reusable modules)
└── README.md       # usage example, inputs/outputs table
```

### Calling modules

```
module "network" {
  source = "./modules/network" # local path

  name       = "shop-dev"
  cidr_block = "10.10.0.0/16"
  azs        = ["ap-south-1a", "ap-south-1b"]
}

module "vpc_community" {
  source  = "terraform-aws-modules/vpc/aws" # public registry
  version = "~> 5.13"                       # ALWAYS pin module versions

  name            = "shop-prod"
  cidr            = "10.20.0.0/16"
  azs             = ["ap-south-1a", "ap-south-1b"]
  public_subnets  = ["10.20.1.0/24", "10.20.2.0/24"]
  private_subnets = ["10.20.11.0/24", "10.20.12.0/24"]
}

# Git source pinned to a tag:
# source = "git::https://github.com/acme/terraform-modules.git//network?ref=v1.4.0"

output "vpc_id" {
  value = module.network.vpc_id
}
```

| Source type | Example |
|---|---|
| Local path | `./modules/network` |
| Terraform Registry | `terraform-aws-modules/vpc/aws` + `version` |
| Git | `git::https://...//subdir?ref=v1.4.0` (pin a tag/commit) |
| S3 / HTTP archive | `s3::https://...module.zip` |
| Private registry | `app.terraform.io/acme/network/aws` |

**Good module design:** one clear purpose · minimal required inputs with sensible defaults · typed and validated variables · useful outputs · no hard-coded regions/accounts · no `provider` blocks (the caller passes providers) · semantic versioning with a changelog · examples and tests. Don't wrap a single resource in a module without adding value.

## 9.9 Lifecycle, refactoring and checks

### lifecycle

```
resource "aws_db_instance" "main" {
  # ...
  lifecycle {
    prevent_destroy = true # plan fails if anything would destroy it
  }
}

resource "aws_launch_template" "web" {
  # ...
  lifecycle {
    create_before_destroy = true # new one first, then delete the old (no gap)
  }
}

resource "aws_autoscaling_group" "web" {
  # ...
  lifecycle {
    ignore_changes = [desired_capacity] # changed by autoscaling at runtime: don't fight it
  }
}

resource "aws_instance" "app" {
  # ...
  lifecycle {
    replace_triggered_by = [aws_launch_template.web.latest_version]
    precondition {
      condition     = var.instance_type != "t2.micro"
      error_message = "t2 instances are not allowed; use t3/t4g."
    }
  }
}
```

### Refactoring without destroying: moved, import, removed

```
moved {
  from = aws_instance.web
  to   = aws_instance.app # renamed in code: state follows, nothing is recreated
}

moved {
  from = aws_subnet.public[0]
  to   = aws_subnet.public["a"] # switched from count to for_each
}

import {
  to = aws_s3_bucket.legacy
  id = "acme-legacy-bucket" # adopt an existing bucket (see 9.15)
}

removed {
  from = aws_s3_bucket.archive # stop managing it, keep the real bucket
  lifecycle {
    destroy = false
  }
}
```

### check blocks (continuous assertions)

```
check "website_up" {
  data "http" "site" {
    url = "https://shop.example.com/health"
  }
  assert {
    condition     = data.http.site.status_code == 200
    error_message = "Health endpoint is not returning 200."
  }
}
```

Checks produce **warnings** on plan/apply without blocking: useful for post-deployment validation.

## 9.10 Environments: workspaces vs directories

| Approach | How | Pros | Cons |
|---|---|---|---|
| **Directory per environment** | `envs/dev`, `envs/prod` each with its own backend key and tfvars, calling shared modules | Explicit, different settings/versions per env, separate permissions | Some duplication |
| Workspaces | `terraform workspace new prod`; same code, separate state per workspace; `terraform.workspace` in expressions | Little duplication | Easy to apply to the wrong workspace; same backend/permissions; hidden differences |
| Wrapper tools | Terragrunt (DRY backends and inputs), Terramate | Large estates | Extra tooling |

Most teams use **directories (or separate stacks) per environment** with shared versioned modules; workspaces suit short-lived copies (feature environments).

## 9.11 Project structure and conventions

```
infrastructure/
├── modules/
│   ├── network/          (VPC, subnets, routes, NAT, endpoints)
│   ├── web-service/      (ALB, target group, ASG, launch template, security groups)
│   └── database/         (RDS, subnet group, parameter group, secret)
├── envs/
│   ├── dev/
│   │   ├── backend.tf    (state key: dev/...)
│   │   ├── main.tf       (module calls)
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
│   │   └── outputs.tf
│   └── prod/ ...
├── .tflint.hcl
├── .pre-commit-config.yaml
└── README.md
```

**Conventions:** `snake_case` names · resource name describes the role (`aws_security_group.web`, not `sg1`) · `this` for a module's single main resource · one purpose per file (`versions.tf`, `providers.tf`, `variables.tf`, `outputs.tf`, `main.tf`, `data.tf`) · descriptions on every variable and output · consistent

tags via `default_tags` · `terraform fmt` + `validate` + `tflint` before every commit · generate module docs with `terraform-docs`.

## 9.12 Team workflow and Terraform in CI/CD

```
feature branch → PR → CI: fmt-check · validate · tflint · security scan
                          · terraform plan (posted to the PR)
              → review code AND plan → merge to main
              → CD: terraform apply of the reviewed plan (prod behind an approval gate)
```

| Rule | Why |
|---|---|
| Nobody applies from a laptop to shared environments | Consistent credentials, logs, reviewed plans |
| Plan on every PR, apply only after merge | Review sees the exact impact |
| Apply the **saved plan** ( `-out` ) or re-plan and compare | No surprises between review and apply |
| Separate roles for plan (read-only) and apply (write) | Least privilege |
| Approval gates for production | Human check on destructive changes |
| Scheduled drift detection ( `plan -detailed-exitcode` : exit 2 = changes) | Catch manual changes |

**GitHub Actions example (OIDC, no stored AWS keys):**

```yaml
name: terraform
on:
  pull_request:
    paths: ["envs/dev/**", "modules/**"]
  push:
    branches: [main]
    paths: ["envs/dev/**", "modules/**"]

permissions:
  id-token: write
  contents: read
  pull-requests: write

jobs:
  terraform:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: envs/dev
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.9.8
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-terraform
          aws-region: ap-south-1
      - run: terraform fmt -check -recursive
      - run: terraform init -input=false
      - run: terraform validate
      - run: terraform plan -input=false -out=tfplan
      - name: Apply (main branch only)
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: terraform apply -input=false tfplan
```

Dedicated tools: **Atlantis** (plan/apply via PR comments), **HCP Terraform/Terraform Enterprise**, Spacelift, env0, Scalr (remote runs, policy, cost estimation, drift detection).

## 9.13 Terraform security

| Area | Practice |
|---|---|
| State | Remote, encrypted, versioned, access-restricted; never in Git; separate state per environment |
| Secrets | Don't write secrets in `.tf` / `.tfvars`; read them with data sources (Secrets Manager/SSM), generate with `random_password` and store in Secrets Manager, or use managed secrets (`manage_master_user_password = true` on RDS); mark outputs/variables `sensitive`; newer versions support **ephemeral** values that never reach state |
| Credentials | OIDC/assumed roles, no long-lived keys; separate plan and apply roles; least privilege |
| Supply chain | Pin provider and module versions; commit the lock file; use trusted module sources |
| Static analysis | `tflint` (provider-aware lint), **checkov** / **trivy config** / tfsec (misconfigurations: public buckets, open SGs, unencrypted disks) |
| Policy as code | **OPA/Conftest** on the JSON plan (`terraform show -json tfplan`), HCP Sentinel, AWS Config rules after deployment |
| Reviews | Every change via PR with plan output; CODEOWNERS for sensitive modules (IAM, networking) |

| Area | Practice |
|---|---|
| Protection | `prevent_destroy` on critical resources; deletion protection flags (RDS, ALB) |

```
tflint --init && tflint --recursive
checkov -d . --quiet
trivy config .
terraform show -json tfplan > plan.json && conftest test plan.json -p policy/
```

## 9.14 Testing Terraform

| Level | Tool | What it checks |
|---|---|---|
| Format/syntax | `terraform fmt -check`, `terraform validate` | Style and internal consistency |
| Lint | tflint | Invalid instance types, deprecated syntax, naming rules |
| Security/policy | checkov, trivy, Conftest | Misconfigurations, company rules |
| Unit/integration | `terraform test` (`.tftest.hcl`) | Assertions on plans or real short-lived deployments |
| End-to-end | Terratest (Go) | Deploy, call endpoints, destroy |

```
# tests/network.tftest.hcl
mock_provider "aws" {} # fake AWS provider: runs offline, no credentials or cost (Terraform 1.7+)

variables {
  name       = "test"
  cidr_block = "10.99.0.0/16"
  azs        = ["ap-south-1a", "ap-south-1b"]
}

run "plan_creates_two_public_subnets" {
  command = plan

  assert {
    condition     = length(aws_subnet.public) == 2
    error_message = "Expected one public subnet per AZ."
  }
}
```

`terraform test` (inside the module folder) runs each `run` block. With `mock_provider` it runs offline in seconds (ideal for CI); without it, `command = apply` creates **real** resources and destroys them afterwards.

## 9.15 Importing existing infrastructure

Many companies start with resources created by hand. Bring them under Terraform without recreating them:

```
import {
  to = aws_s3_bucket.legacy
  id = "acme-legacy-bucket"
}
```

```
terraform plan -generate-config-out=generated.tf   # Terraform writes matching resource code
# review/clean generated.tf, move it into your files
terraform plan                                      # should show "1 to import, 0 to change"
terraform apply
```

After import, `plan` must show **no changes** before you start modifying. Import IDs differ per resource type (bucket name, instance ID, ARN): check each resource's documentation ("Import" section).

## 9.16 Complete project: modular VPC + load-balanced web tier

Structure:

```
project/
├── modules/network/   main.tf  variables.tf  outputs.tf  versions.tf  tests/
└── envs/dev/          main.tf  (providers, module calls, web tier)
```

### modules/network

```
# modules/network/variables.tf
variable "name" {
  type        = string
  description = "Name prefix for all resources"
}

variable "cidr_block" {
  type        = string
  description = "VPC CIDR, e.g. 10.10.0.0/16"
  validation {
    condition     = can(cidrhost(var.cidr_block, 0))
    error_message = "cidr_block must be a valid CIDR."
  }
}

variable "azs" {
  type        = list(string)
  description = "Availability zones, one public and one private subnet each"
}
```

```
# modules/network/main.tf
resource "aws_vpc" "this" {
  cidr_block           = var.cidr_block
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags                 = { Name = "${var.name}-vpc" }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
  tags   = { Name = "${var.name}-igw" }
}

resource "aws_subnet" "public" {
  for_each                = { for i, az in var.azs : az => i }
  vpc_id                  = aws_vpc.this.id
  availability_zone       = each.key
  cidr_block              = cidrsubnet(var.cidr_block, 8, each.value + 1) # .1.0/24, .2.0/24
  map_public_ip_on_launch = true
  tags                    = { Name = "${var.name}-public-${each.key}", Tier = "public" }
}

resource "aws_subnet" "private" {
  for_each          = { for i, az in var.azs : az => i }
  vpc_id            = aws_vpc.this.id
  availability_zone = each.key
  cidr_block        = cidrsubnet(var.cidr_block, 8, each.value + 11) # .11.0/24, .12.0/24
  tags              = { Name = "${var.name}-private-${each.key}", Tier = "private" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
  tags = { Name = "${var.name}-public-rt" }
}

resource "aws_route_table_association" "public" {
  for_each       = aws_subnet.public
  subnet_id      = each.value.id
  route_table_id = aws_route_table.public.id
}
```

```
# modules/network/outputs.tf
output "vpc_id" {
  value = aws_vpc.this.id
}

output "public_subnet_ids" {
  value = [for s in aws_subnet.public : s.id]
}

output "private_subnet_ids" {
  value = [for s in aws_subnet.private : s.id]
}
```

```
# modules/network/versions.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0" # modules use minimum versions; the root module pins exactly
    }
  }
}
```

### envs/dev

```
# envs/dev/main.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.70"
    }
  }
  # backend "s3" { ... }  # remote state in real use (9.6)
}

provider "aws" {
  region = "ap-south-1"
  default_tags {
    tags = { Project = "shop", Env = "dev", ManagedBy = "terraform" }
  }
}

locals {
  name = "shop-dev"
}

module "network" {
  source     = "../../modules/network"
  name       = local.name
  cidr_block = "10.10.0.0/16"
  azs        = ["ap-south-1a", "ap-south-1b"]
}

data "aws_ssm_parameter" "al2023" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

resource "aws_security_group" "alb" {
  name   = "${local.name}-alb"
  vpc_id = module.network.vpc_id
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "web" {
  name   = "${local.name}-web"
  vpc_id = module.network.vpc_id
  ingress {
    description     = "HTTP only from the ALB"
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
  egress {
    from_port   = 0
    to_port     = 0
```

```
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_lb" "web" {
  name               = "${local.name}-alb"
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb.id]
  subnets            = module.network.public_subnet_ids
}

resource "aws_lb_target_group" "web" {
  name     = "${local.name}-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = module.network.vpc_id
  health_check {
    path                = "/"
    healthy_threshold   = 2
    unhealthy_threshold = 2
    interval            = 15
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.web.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.web.arn
  }
}

resource "aws_launch_template" "web" {
  name_prefix            = "${local.name}-web-"
  image_id               = data.aws_ssm_parameter.al2023.value
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data = base64encode(<<-EOT
    #!/bin/bash
    dnf install -y nginx
    echo "<h1>$(hostname)</h1>" > /usr/share/nginx/html/index.html
    systemctl enable --now nginx
  EOT
  )
  metadata_options {
    http_tokens = "required" # IMDSv2
  }
}

resource "aws_autoscaling_group" "web" {
  name             = "${local.name}-web"
  min_size         = 2
  max_size         = 4
  desired_capacity = 2
  # labs: public subnets (no NAT cost); production: private subnets + NAT/endpoints
  vpc_zone_identifier       = module.network.public_subnet_ids
  target_group_arns         = [aws_lb_target_group.web.arn]
  health_check_type         = "ELB"
  health_check_grace_period = 120

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
```

```
  lifecycle {
    ignore_changes = [desired_capacity]
  }
}

output "url" {
  value = "http://${aws_lb.web.dns_name}"
}
```

⚠️ **Cost:** the ALB and two instances are billed hourly. Run `terraform apply`, open the URL (refresh to see both hostnames), then `terraform destroy` the same day. **Production upgrades:** private subnets + one NAT Gateway per AZ (or VPC endpoints), HTTPS listener with ACM, remote state, RDS module, scaling policies, WAF.

## 9.17 Troubleshooting

| Error / symptom | Cause | Fix |
|---|---|---|
| `Error acquiring the state lock` | Another run in progress, or a crashed run left the lock | Wait; verify nobody is running; `terraform force-unlock <ID>` |
| `No valid credential sources found` / `ExpiredToken` | No/expired AWS credentials | `aws sts get-caller-identity`, SSO login, correct profile/role |
| `AccessDenied` / `UnauthorizedOperation` | Terraform role lacks permissions | Read the action in the error; add least-privilege permission |
| `resource already exists` | Created outside Terraform or in another state | `import` it, or rename |
| Plan wants to **replace** something unexpectedly | Changed an attribute that "forces replacement" (e.g. subnet AZ, DB engine) | Read the plan reason; use `moved`, `lifecycle`, or plan a migration |
| Perpetual diff (changes every plan) | Attribute normalized by the API, or modified by another system | Match the API format; `ignore_changes` for externally managed attributes |
| `Inconsistent dependency lock file` | Providers changed without updating the lock | `terraform init -upgrade` and commit the lock file |
| `Provider produced inconsistent result` | Provider bug or eventual consistency | Retry; upgrade provider; check open issues |
| `Cycle` error | Resources reference each other circularly (e.g. two SGs referencing each other inline) | Use separate `aws_vpc_security_group_ingress_rule` resources |
| `Invalid count argument` / `for_each` unknown | Count/for_each depends on values known only after apply | Use static keys; split into two applies; derive from inputs |
| Timeout creating a resource | Slow service (RDS, NAT), health checks failing | Increase `timeouts`; check the resource's events in AWS |
| Destroy fails (dependency violation) | Resources created outside Terraform still attached (ENIs, objects in a bucket) | Remove them manually or `force_destroy` (careful) |

Debug: `TF_LOG=DEBUG terraform plan 2> debug.log`, `terraform console`, `terraform state show`, and CloudTrail for the failing API call.

## 9.18 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | What is IaC and why use it? | Infrastructure defined as versioned code: repeatable, reviewable, auditable, fast, less drift |
| 2 | Declarative vs imperative? | Describe the end state vs write the steps |
| 3 | What is a provider? | Plugin translating Terraform resources into a platform's API calls |
| 4 | Resource vs data source? | Managed object Terraform creates/updates/deletes vs read-only lookup |
| 5 | What is Terraform state and why is it needed? | Mapping between code and real resources (IDs, attributes, dependencies) used to compute changes |
| 6 | Why use remote state with locking? | Team sharing, single source of truth, prevents concurrent applies, versioned backups |
| 7 | Why is state sensitive? | Contains attribute values including secrets in plain text |
| 8 | `terraform plan` vs `apply`? | Preview vs execute changes |
| 9 | What does `init` do? | Downloads providers/modules, configures backend, creates the lock file |
| 10 | `count` vs `for_each`? | Index-based copies vs keyed copies; for_each is stable when items are added/removed |
| 11 | What are modules? | Reusable packages of resources with inputs/outputs; pin versions |
| 12 | How do you manage multiple environments? | Separate directories/state per environment with shared modules (or workspaces for simple cases) |
| 13 | How do you handle secrets? | Never in code; secrets manager data sources or managed secrets; sensitive/ephemeral values; protect state |
| 14 | What is drift and how do you detect it? | Real infra differs from code; `plan` / `-refresh-only` / scheduled drift checks |
| 15 | How do you import existing resources? | `import` blocks (+ `-generate-config-out`) or `terraform import`, then plan must show no changes |
| 16 | How do you rename a resource without recreating it? | `moved` block (or `terraform state mv`) |
| 17 | `prevent_destroy`, `create_before_destroy`, `ignore_changes`? | Block deletion / replace with no downtime / ignore attributes changed outside Terraform |
| 18 | What is the lock file? | `.terraform.lock.hcl`: exact provider versions and checksums; commit it |
| 19 | How do you pass values between stacks? | Outputs + `terraform_remote_state`, data sources by tags, or SSM parameters |
| 20 | How do you run Terraform in CI/CD safely? | fmt/validate/lint/scan + plan on PR, apply after merge with approval, OIDC roles, saved plans |

| # | Question | Answer |
|---|---|---|
| 21 | What is `depends_on` for? | Explicit ordering when there is no reference between resources |
| 22 | What happens if two people apply at once? | With locking, the second waits/fails; without it, state can be corrupted |
| 23 | State lock is stuck. What do you do? | Confirm no running apply, then `force-unlock` with the lock ID |
| 24 | Terraform vs Ansible? | Terraform provisions infrastructure declaratively with state; Ansible configures systems procedurally/agentlessly |
| 25 | Terraform vs CloudFormation? | Multi-cloud with self-managed state vs AWS-only with AWS-managed state |
| 26 | What is `terraform test`? | Built-in testing framework with `.tftest.hcl` run blocks and assertions |
| 27 | How do you enforce company rules? | Policy as code (OPA/Conftest, Sentinel), checkov/trivy in CI, code owners |
| 28 | What causes "forces replacement"? | Changing immutable attributes; Terraform must destroy and recreate |
| 29 | How do you structure a large Terraform codebase? | Versioned modules + small per-environment, per-layer stacks with separate states and pipelines |
| 30 | What is OpenTofu? | Open-source, community-governed fork of Terraform with a compatible CLI and language |

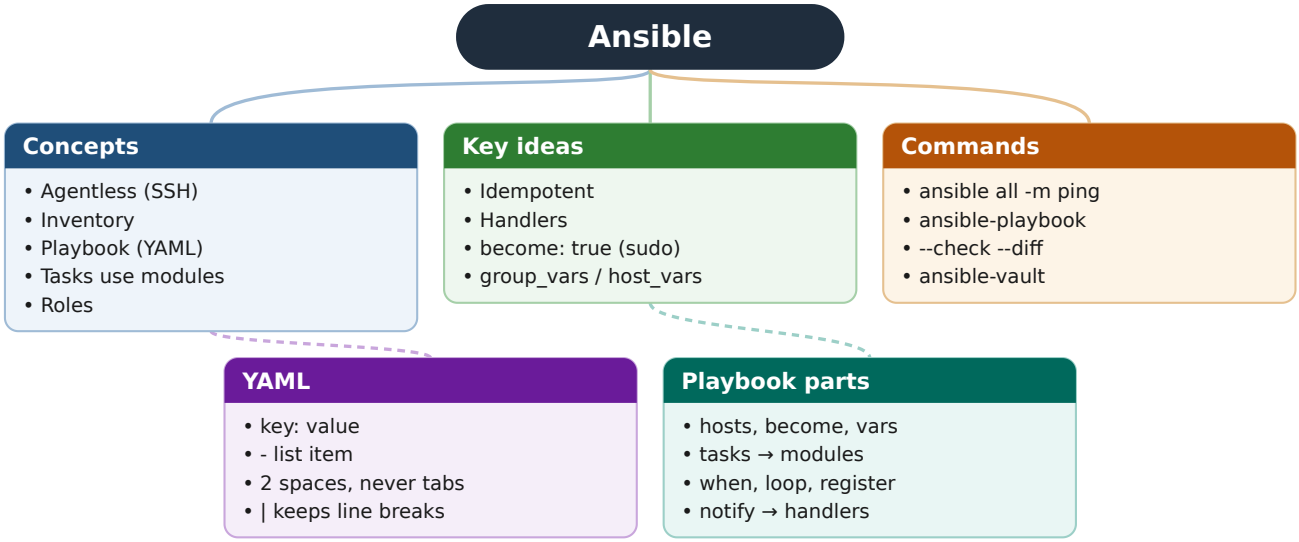
## 9.19 Self-check: I can…

- [ ] Explain IaC benefits, declarative vs imperative, idempotency, drift and immutable infrastructure.
- [ ] Explain how Terraform core, providers, the graph and state work together.
- [ ] Use every common CLI command and flag; know which files to commit.
- [ ] Write HCL fluently: blocks, references, types, variables with validation, expressions, functions.
- [ ] Configure providers with version constraints, aliases, assume-role and default tags.
- [ ] Set up an S3 remote backend with locking and explain why state is sensitive.
- [ ] Use state commands, refresh-only, and split state by environment/layer.
- [ ] Use data sources, outputs, locals and `terraform_remote_state`.
- [ ] Write and call versioned modules from local, registry and Git sources.
- [ ] Use `count` / `for_each`, `lifecycle`, `moved`, `import`, `removed` and `check` blocks.
- [ ] Structure multi-environment projects and run Terraform safely in CI/CD with OIDC.
- [ ] Secure Terraform: secrets, least privilege, scanning, policy as code.
- [ ] Test with validate, tflint, checkov and `terraform test`.
- [ ] Import existing infrastructure and generate configuration.
- [ ] Build, verify and destroy the VPC + ALB + ASG project.
- [ ] Troubleshoot locks, credentials, replacements, cycles and perpetual diffs.

**REMEMBER**

- The loop: **write → init → plan → apply**.
- Plan symbols: **+** create · **~** update · **-** destroy · **-/+** replace (check why!).

# 10. Ansible (Configuration Management)




## 10.0 Why configuration management

Terraform creates servers; something must **configure** them: packages, users, files, services, security settings, application deployments. Doing that by hand over SSH on 50 servers is slow and produces servers that drift apart. **Configuration management** describes the desired configuration as code and applies it consistently and repeatably.

**Where companies use Ansible:** server configuration and hardening (CIS baselines), application deployment to VMs, patching, user management, network device configuration (Cisco, Juniper, Arista), cloud provisioning tasks, building golden images with Packer, orchestrating multi-step operations (rolling restarts).

## 10.1 Concepts and architecture

| Concept | Meaning |
|---|---|
| **Agentless** | Nothing to install on targets: Ansible connects over **SSH** (Linux) or **WinRM/SSH** (Windows) and runs small Python modules there |
| **Push model** | The control node pushes changes when you run a playbook (Puppet/Chef classically **pull** via agents) |
| **Idempotent** | Modules describe a *state* ("package present"); running twice changes nothing the second time |
| **Declarative-ish** | Tasks run in order (procedural), but each module is declarative about its result |

```
CONTROL NODE (your laptop / CI runner / AWX)
 ├── ansible.cfg            settings
 ├── inventory              which hosts, in which groups
 ├── playbooks (YAML)       what to do
 ├── roles / collections    reusable content and modules
 │
 └── SSH ──► managed node 1 (Python runs the module, returns JSON: ok / changed / failed)
     SSH ──► managed node 2
     SSH ──► managed node N   (in parallel: "forks")
```

| Tool | Model | Language | Notes |
|---|---|---|---|
| **Ansible** | Agentless, push | YAML + Jinja2 | Easiest start; most popular for config mgmt |
| Puppet | Agent, pull | Puppet DSL | Strong for large, long-lived fleets |
| Chef | Agent, pull | Ruby | Developer-oriented |
| Salt | Agent (or SSH), push/pull, event-driven | YAML + Jinja2 | Very fast at scale |
| **Terraform** | Provisioning (API-driven) | HCL | Creates infrastructure; pairs with Ansible |

**Typical split:** Terraform builds the VPC, instances, load balancer → Ansible configures the OS and deploys the app (or Ansible + Packer bake a golden AMI; containers often replace in-server configuration).

## 10.2 Installation and ansible.cfg

```
pipx install --include-deps ansible     # full package with community collections
# or a minimal core + only what you need:
pipx install ansible-core
ansible-galaxy collection install community.general ansible.posix amazon.aws
ansible --version                       # shows config file in use, Python, collection paths
```

Managed nodes need **Python 3** and SSH access (key-based) with a user that can `sudo`.

**ansible.cfg** is searched in this order (first found wins): `ANSIBLE_CONFIG` env var → `./ansible.cfg` (project directory) → `~/.ansible.cfg` → `/etc/ansible/ansible.cfg`. Keep one per project, committed to Git.

```
[defaults]
inventory = inventory/hosts.yml
remote_user = ubuntu
private_key_file = ~/.ssh/lab.pem
roles_path = roles
host_key_checking = True        ; keep True in production; False only for throwaway labs
forks = 20
stdout_callback = yaml
retry_files_enabled = False

[privilege_escalation]
become = False
become_method = sudo

[ssh_connection]
pipelining = True               ; fewer SSH round trips, faster runs
```

## 10.3 Inventory

The inventory lists managed hosts and organizes them into **groups**.

### INI format

```
[web]
web1.example.com
web2.example.com ansible_host=10.0.1.21

[db]
db1.example.com ansible_user=ec2-user

[prod:children]
web
db

[web:vars]
http_port=80
```

### YAML format (preferred for structure)

```
all:
  vars:
    ansible_user: ubuntu
  children:
    web:
      hosts:
        web1:
          ansible_host: 10.0.1.20
        web2:
          ansible_host: 10.0.1.21
      vars:
        http_port: 80
    db:
      hosts:
        db1:
          ansible_host: 10.0.21.10
    prod:
      children:
        web:
        db:
```

### group_vars and host_vars

```
inventory/
├── hosts.yml
├── group_vars/
│   ├── all.yml          # variables for every host
│   ├── web.yml          # variables for group web
│   └── prod/
│       ├── vars.yml
│       └── vault.yml    # encrypted secrets (10.11)
└── host_vars/
    └── web1.yml         # variables for one host
```

### Useful connection variables

`ansible_host` (IP/DNS to connect to) · `ansible_user` · `ansible_port` · `ansible_ssh_private_key_file` · `ansible_python_interpreter` · `ansible_connection` (`ssh`, `local`, `winrm`, `community.aws.aws_ssm`) · `ansible_become`.

### Host patterns

| Pattern | Hosts |
|---|---|
| `all` | Everything |
| `web` | Group web |
| `web:db` | Web **or** db |
| `web:&prod` | In web **and** prod |
| `prod:!db` | prod **except** db |
| `web1*` | Wildcard |

### Dynamic inventory (AWS)

Cloud hosts come and go, so build the inventory from the cloud API by tags:

```
# inventory/aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions:
  - ap-south-1
filters:
  tag:Project: shop
  instance-state-name: running
keyed_groups:
  - key: tags.Role          # tag Role=web → group role_web
    prefix: role
  - key: tags.Env
    prefix: env
hostnames:
  - private-ip-address
compose:
  ansible_host: private_ip_address
```

```
ansible-inventory -i inventory/aws_ec2.yml --graph     # show groups and hosts
ansible-inventory -i inventory/ --list                 # full JSON with variables
```

## 10.4 Ad-hoc commands

One-off tasks without a playbook: `ansible <pattern> -m <module> -a "<args>"`.

```
ansible all -m ansible.builtin.ping                          # connectivity + Python check (not ICMP)
ansible web -m ansible.builtin.command -a "uptime"
ansible web -m ansible.builtin.shell -a "df -h | grep /dev/root"   # shell = pipes/redirects allowed
ansible web -b -m ansible.builtin.apt -a "name=htop state=present update_cache=true"
ansible web -b -m ansible.builtin.service -a "name=nginx state=restarted"
ansible web -m ansible.builtin.setup -a "filter=ansible_distribution*"   # facts
ansible db -m ansible.builtin.copy -a "src=./motd dest=/etc/motd" -b
ansible all -m ansible.builtin.ping -l web1 -f 10          # limit to web1, 10 forks
```

| Flag | Meaning |
|---|---|
| `-i inventory` | Inventory file/directory |
| `-m module` / `-a "args"` | Module and its arguments |
| `-b` / `--become` | Run with sudo |

| Flag | Meaning |
|---|---|
| `-K` | Ask for the sudo password |
| `-u user`, `--private-key` | Connection user/key |
| `-l pattern` | Limit hosts |
| `-f N` | Parallel forks |
| `-C` / `--check`, `-D` / `--diff` | Dry run / show changes |
| `-v`, `-vvv`, `-vvvv` | Verbosity (connection debugging) |

`command` is safer than `shell` (no shell interpretation); both are **not idempotent**: prefer real modules.

## 10.5 YAML for Ansible (and Kubernetes, Compose, Actions)

YAML is used by Ansible, Kubernetes, Docker Compose, GitHub Actions and Helm, so learn it once.

```
# Comment
name: web-server          # key: value   (a space after the colon is required)
port: 8080                # number
enabled: true             # boolean (true/false)
version: "1.10"           # quote values that look like numbers but are text
empty: null               # no value

servers:                  # LIST: each item starts with "- "
  - web1
  - web2

database:                 # MAP (dictionary): nested keys, indented
  host: db.internal
  port: 5432

users:                    # list of maps
  - name: alice
    groups: [admin, dev]  # inline list
  - name: bob
    groups: [dev]

script: |                 # | keeps line breaks (multi-line text, scripts)
  echo "line 1"
  echo "line 2"
description: >            # > folds lines into one long line
  This is a long
  sentence.
```

**Rules:** indentation = structure, **spaces only (never tabs)**, usually 2 spaces per level, items in the same list must line up. Check a file with `yamllint file.yml` or `python3 -c "import yaml,sys; yaml.safe_load(open(sys.argv[1]))" file.yml`.

## 10.6 Playbook anatomy

```
---                                  # optional start-of-document marker
- name: Configure web servers        # PLAY: a name for humans
  hosts: web                         # which inventory group to run on
  become: true                       # use sudo for tasks
  vars:                              # VARIABLES for this play
    app_port: 8080
    packages:
      - nginx
      - curl

  tasks:                             # TASKS run in order, top to bottom
    - name: Install packages         # always name tasks: shown in the output
      ansible.builtin.apt:           # MODULE (fully qualified name)
        name: "{{ packages }}"       # {{ }} = Jinja2 variable (quote it)
        state: present               # desired state, not a command → idempotent
        update_cache: true

    - name: Deploy nginx site config
      ansible.builtin.template:      # renders a Jinja2 template file with variables
        src: templates/site.conf.j2
        dest: /etc/nginx/sites-available/default
        mode: "0644"
      notify: Reload nginx           # trigger the HANDLER only if this task changed something

    - name: Check if the app answers
      ansible.builtin.uri:
        url: "http://localhost:{{ app_port }}/health"
        status_code: 200
      register: health               # save the result in a variable
      ignore_errors: true

    - name: Show the result
      ansible.builtin.debug:
        msg: "Health status: {{ health.status | default('unknown') }}"   # | = Jinja2 filter

    - name: Create users
      ansible.builtin.user:
        name: "{{ item }}"
        groups: www-data
        append: true
      loop:                          # LOOP: run the task once per item
        - alice
        - bob

    - name: Only on Ubuntu
      ansible.builtin.debug:
        msg: "This is {{ ansible_facts['distribution'] }}"   # FACTS gathered from the host
      when: ansible_facts['distribution'] == "Ubuntu"         # CONDITION

  handlers:                          # HANDLERS run once at the end, only when notified
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

**Template file** `templates/site.conf.j2` (Jinja2 = text with `{{ variables }}` and `{% logic %}`):

```
server {
    listen 80;
    location / {
        proxy_pass http://127.0.0.1:{{ app_port }};
    }
}
```

| Element | Meaning |
|---|---|
| Play | Maps a group of hosts to a list of tasks |
| Task | One step, using one module |
| Module | The tool that does the work ( `apt` , `copy` , `template` , `service` , `user` , `file` , `uri` , `command` ) |
| `state` | Desired result: `present` / `absent` , `started` / `stopped` / `restarted` |
| `{{ var }}` | Insert a variable (Jinja2) |
| `register` | Save a task's result |
| `when` | Run the task only if the condition is true |
| `loop` / `item` | Repeat a task for each item |
| `notify` / `handlers` | Run a follow-up (e.g. restart) only when something changed |
| `become` | Run with sudo |
| Facts ( `ansible_facts` ) | Information collected from each host (OS, IPs, memory) |

**Role layout** (reusable unit, created with `ansible-galaxy init nginx` ):

```
roles/nginx/
├── tasks/main.yml        # the tasks
├── handlers/main.yml     # handlers
├── templates/            # .j2 files
├── defaults/main.yml     # default variables (easy to override)
└── vars/main.yml         # fixed variables
```

Use it in a playbook with `roles: [nginx]` . **Avoid `command` / `shell` modules** when a real module exists: real modules are idempotent and report "changed" correctly.

## 10.7 Most-used modules

Always use **fully qualified collection names (FQCN)**, e.g. `ansible.builtin.copy` , to avoid ambiguity. Docs: `ansible-doc ansible.builtin.copy` (examples at the bottom).

| Module | Purpose | Example arguments |
|---|---|---|
| `ansible.builtin.package` / `apt` / `dnf` | Install/remove packages | `name: [nginx, curl]` , `state: present/latest/absent` , `update_cache: true` |
| `ansible.builtin.service` / `systemd_service` | Start/stop/enable services | `name: nginx` , `state: started` , `enabled: true` , `daemon_reload: true` |
| `ansible.builtin.copy` | Copy a file or inline content | `src` , `dest` , `owner` , `group` , `mode: "0644"` , `content` , `backup: true` |

| Module | Purpose | Example arguments |
|---|---|---|
| `ansible.builtin.template` | Render a Jinja2 template | `src: app.conf.j2`, `dest`, `mode`, `validate: "nginx -t -c %s"` |
| `ansible.builtin.file` | Directories, links, permissions, delete | `path`, `state: directory/link/absent/touch`, `mode`, `owner` |
| `ansible.builtin.lineinfile` | Ensure one line in a file | `path`, `regexp`, `line`, `state` |
| `ansible.builtin.blockinfile` | Ensure a block of lines | `path`, `block`, `marker` |
| `ansible.builtin.user` / `group` | Accounts | `name`, `groups`, `append: true`, `shell`, `state` |
| `ansible.posix.authorized_key` | SSH public keys | `user`, `key: "{{ lookup('file', 'alice.pub') }}"` |
| `ansible.builtin.git` | Clone/update a repo | `repo`, `dest`, `version: v1.4.0` |
| `ansible.builtin.get_url` / `unarchive` | Download / extract | `url`, `dest`, `checksum`; `src`, `remote_src: true` |
| `ansible.builtin.uri` | HTTP requests/health checks | `url`, `status_code: 200`, `return_content` |
| `ansible.builtin.command` / `shell` | Run commands (last resort) | `cmd`, `creates:` / `removes:` to make them idempotent, `changed_when` |
| `ansible.builtin.cron` | Cron entries | `name`, `minute`, `hour`, `job` |
| `ansible.posix.mount` | Mounts + fstab | `path`, `src`, `fstype`, `state: mounted` |
| `ansible.posix.sysctl` | Kernel parameters | `name: net.ipv4.ip_forward`, `value: "1"` |
| `community.general.ufw` / `ansible.posix.firewalld` | Firewalls | `rule: allow`, `port: "443"` |
| `ansible.builtin.wait_for` | Wait for a port/file | `port: 8000`, `timeout: 60` |
| `ansible.builtin.reboot` | Reboot and wait | `reboot_timeout: 600` |
| `ansible.builtin.assert` / `fail` | Validate assumptions | `that: [...]`, `fail_msg` |
| `ansible.builtin.debug` | Print messages/variables | `msg`, `var` |
| `community.docker.docker_container` | Run containers | `name`, `image`, `ports`, `restart_policy` |
| `kubernetes.core.k8s` | Apply Kubernetes manifests | `state: present`, `src: deploy.yaml` |
| `amazon.aws.ec2_instance`, `amazon.aws.s3_object` | AWS resources | (prefer Terraform for long-lived infrastructure) |

## Return values

Every task returns JSON: `changed`, `failed`, `msg`, module-specific fields (`stdout`, `rc`, `status`...). Status colours in output: **ok** (green, nothing to do), **changed** (yellow), **failed** (red), **skipped** (cyan), **unreachable**. The play recap summarizes them per host.

## 10.8 Variables, facts and precedence

### Where variables come from

| Source | Example |
|---|---|
| Role `defaults/main.yml` | Lowest priority: sensible defaults users override |
| Inventory `group_vars` / `host_vars` | Per environment/group/host settings |
| Play `vars:` / `vars_files:` | Play-level values |
| `register:` / `set_fact:` | Values created during the run |
| Role `vars/main.yml` | High priority: internal constants of a role |
| Extra vars `-e "version=1.4.2"` / `-e @vars.yml` | **Highest priority**: always win |

**Simplified precedence (low → high):** role defaults → inventory group_vars (`all` → specific groups) → host_vars → play vars / vars_files → role vars → block/task vars → set_fact/registered vars → **extra vars (`-e`)**. Rule of thumb: put user-tunable values in **role defaults** or **group_vars**; use `-e` only for one-off overrides.

### Facts

Facts are gathered automatically from each host at the start of a play (`gather_facts: true`):

```
- name: Show some facts
  ansible.builtin.debug:
    msg: >-
      {{ inventory_hostname }} runs {{ ansible_facts['distribution'] }}
      {{ ansible_facts['distribution_version'] }} with
      {{ ansible_facts['memtotal_mb'] }} MB RAM, IP {{ ansible_facts['default_ipv4']['address'] }}
```

Disable with `gather_facts: false` for speed when you don't need them; cache them with fact caching (redis/jsonfile) on large fleets.

### Magic variables

| Variable | Meaning |
|---|---|
| `inventory_hostname` | Host name as in the inventory |
| `groups['web']` | List of hosts in group web |
| `hostvars['db1']['ansible_host']` | Variables of another host |
| `group_names` | Groups the current host belongs to |
| `ansible_play_hosts` | Hosts in the current play |
| `playbook_dir`, `role_path` | Paths |

### register and set_fact

```
- name: Check app version
  ansible.builtin.command: /opt/app/bin/app --version
  register: app_version
  changed_when: false               # a read-only command never "changes" anything

- name: Remember major version
  ansible.builtin.set_fact:
    app_major: "{{ app_version.stdout.split('.')[0] }}"
```

## 10.9 Jinja2: templates, filters and tests

Ansible evaluates `{{ expressions }}` and `{% statements %}` with **Jinja2**, in playbooks and `.j2` template files.

| Filter | Example | Result |
|---|---|---|
| `default` | `{{ port \| default(8080) }}` | Fallback value |
| `upper` / `lower` / `replace` | `{{ env \| upper }}` | `PROD` |
| `join` | `{{ ['a','b'] \| join(',') }}` | `a,b` |
| `to_json` / `to_nice_yaml` / `from_json` | Convert data | |
| `int` / `bool` / `string` | Type conversion | |
| `length` | `{{ servers \| length }}` | Count |
| `map` / `select` / `selectattr` | `{{ users \| selectattr('admin') \| map(attribute='name') \| list }}` | Filter lists of dicts |
| `regex_replace` / `regex_search` | `{{ url \| regex_replace('^https?://', '') }}` | |
| `b64encode` / `password_hash('sha512')` | Encoding / password hashes | |
| `ipaddr` (ansible.utils) | Network calculations | |

**Template example** `templates/nginx-site.conf.j2`:

```
# Managed by Ansible - manual changes will be overwritten
upstream app {
{% for host in groups['app'] %}
    server {{ hostvars[host]['ansible_host'] }}:{{ app_port }};
{% endfor %}
}

server {
    listen 80;
    server_name {{ server_name }};
{% if enable_gzip | default(true) %}
    gzip on;
{% endif %}
    location / {
        proxy_pass http://app;
    }
}
```

**Lookups** read data on the **control node**: `{{ lookup('env', 'HOME') }}`, `{{ lookup('file', 'files/key.pub') }}`, `{{ lookup('amazon.aws.aws_secret', 'shop/db') }}`, `{{ lookup('ansible.builtin.password', '/dev/null length=24') }}`.

## 10.10 Controlling execution

### Conditionals and loops

```
- name: Install packages on Debian family only
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop: [nginx, curl, jq]
  when: ansible_facts['os_family'] == "Debian"

- name: Create users with their groups
  ansible.builtin.user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
    append: true
  loop:
    - { name: alice, groups: "sudo,devops" }
    - { name: bob, groups: "devops" }
  loop_control:
    label: "{{ item.name }}"        # cleaner output
```

### Blocks, error handling and handlers

```
- name: Deploy with rollback
  block:
    - name: Install new release
      ansible.builtin.unarchive:
        src: "releases/app-{{ version }}.tar.gz"
        dest: /opt/app
    - name: Health check
      ansible.builtin.uri:
        url: http://localhost:8000/health
        status_code: 200
      retries: 5
      delay: 3
      register: health
      until: health.status == 200
  rescue:
    - name: Roll back to previous release
      ansible.builtin.command: /opt/app/bin/rollback.sh
      changed_when: true
  always:
    - name: Report
      ansible.builtin.debug:
        msg: "Deployment of {{ version }} finished on {{ inventory_hostname }}"
```

**Handlers** run once at the end of the play, only if notified by a task that **changed**. Use `listen:` to group handlers; force them earlier with `- ansible.builtin.meta: flush_handlers`.

### Rolling updates and orchestration

```
- name: Rolling deploy to web servers
  hosts: web
  become: true
  serial: 2                          # 2 hosts at a time (or "25%")
  max_fail_percentage: 0             # stop everything if any host fails
  pre_tasks:
    - name: Remove host from the load balancer
      ansible.builtin.debug:
        msg: "deregister {{ inventory_hostname }}"   # e.g. an AWS ELB module call
      delegate_to: localhost
  roles:
    - app
  post_tasks:
    - name: Add host back to the load balancer
      ansible.builtin.debug:
        msg: "register {{ inventory_hostname }}"
      delegate_to: localhost
```

| Keyword | Meaning |
|---|---|
| `serial` | Batch size for rolling changes |
| `delegate_to: host` | Run this task on another host (e.g. localhost, a load balancer) |
| `run_once: true` | Run once for the whole batch (DB migration) |
| `tags: [nginx]` | Run/skip parts: `--tags nginx`, `--skip-tags slow` |
| `check_mode` / `diff` per task | Force dry-run or diff behaviour |
| `ignore_errors` / `failed_when` / `changed_when` | Control task status |
| `async` + `poll` | Long-running tasks in the background |
| `strategy: free` | Hosts proceed independently (default `linear` waits per task) |
| `throttle: 1` | Limit concurrency of one task |

## 10.11 Roles and collections

### Role structure

```
roles/nginx/
├── defaults/main.yml      # tunable defaults (lowest precedence)
├── vars/main.yml          # internal constants (high precedence)
├── tasks/main.yml         # entry point (can include other task files)
├── handlers/main.yml
├── templates/site.conf.j2
├── files/                 # static files for copy
├── meta/main.yml          # metadata, dependencies on other roles
└── README.md
```

`ansible-galaxy role init roles/nginx` creates the skeleton.

```
# site.yml
- name: Web tier
  hosts: web
  become: true
  roles:
    - role: common
    - role: nginx
      vars:
        server_name: shop.example.com
```

Other ways to use roles: `ansible.builtin.import_role` (static, at parse time) and `include_role` (dynamic, at runtime, can be in loops/conditions).

### Collections and dependencies

Collections bundle modules, plugins and roles (`amazon.aws`, `community.general`, `kubernetes.core`, `cisco.ios`). Pin them in `requirements.yml`:

```
collections:
  - name: amazon.aws
    version: ">=8.0.0,<9.0.0"
  - name: community.general
    version: ">=9.0.0"
roles:
  - name: geerlingguy.docker
    version: "7.4.1"
```

`ansible-galaxy install -r requirements.yml` (also in CI). Ansible Galaxy hosts community content; review roles before trusting them.

## 10.12 Secrets with Ansible Vault

```
ansible-vault create group_vars/prod/vault.yml      # new encrypted file (opens $EDITOR)
ansible-vault edit group_vars/prod/vault.yml
ansible-vault view group_vars/prod/vault.yml
ansible-vault encrypt secrets.yml / decrypt secrets.yml
ansible-vault encrypt_string 'S3cr3t!' --name 'db_password'   # inline encrypted variable
ansible-playbook site.yml --ask-vault-pass
ansible-playbook site.yml --vault-password-file ~/.vault_pass  # file chmod 600, NOT in Git
ansible-playbook site.yml --vault-id prod@prompt --vault-id dev@~/.vault_dev
```

**Pattern:** `group_vars/prod/vars.yml` contains `db_password: "{{ vault_db_password }}"` and `vault.yml` (encrypted) contains `vault_db_password: ...`: you can grep variable names without decrypting. Mark tasks handling secrets with `no_log: true` so values don't appear in output/logs. Alternatively fetch secrets at runtime from **AWS Secrets Manager / HashiCorp Vault** lookups so nothing is stored in Git at all.

## 10.13 Project layout and best practices

```
ansible/
├── ansible.cfg
├── requirements.yml
├── inventory/
│   ├── dev/  (hosts.yml or aws_ec2.yml, group_vars/, host_vars/)
│   └── prod/
├── playbooks/
│   ├── site.yml          # everything
│   ├── web.yml
│   └── patch.yml
├── roles/
│   ├── common/           # users, SSH hardening, time sync, packages
│   ├── nginx/
│   └── app/
└── .ansible-lint
```

**Best practices:** idempotent tasks only (avoid `command` / `shell`, or use `creates` / `changed_when`) · name every task · FQCN modules · variables with role-name prefixes (`nginx_port`) · defaults in roles, environment differences in inventory · `--check --diff` before real runs · small, single-purpose roles · secrets in Vault/secret managers with `no_log` · pin collection/role versions · run from CI or AWX, not laptops, for production · keep playbooks in Git with reviews.

## 10.14 Testing and quality

| Tool | Checks |
|---|---|
| `ansible-playbook site.yml --syntax-check` | YAML/playbook syntax |
| `yamllint .` | YAML style |
| **ansible-lint** | Best practices (FQCN, names, idempotency, risky shell usage) |
| `--check --diff` | Dry run showing what would change |
| **Molecule** | Role testing: creates test instances/containers, runs the role, runs it **again to verify idempotency**, verifies, destroys |
| CI | Lint + syntax + Molecule on every PR |

## 10.15 Ansible in the bigger picture

| Integration | How |
|---|---|
| Terraform + Ansible | Terraform outputs/tags → dynamic inventory → Ansible configures (or Terraform `local-exec` triggers a playbook) |
| Packer + Ansible | Ansible provisions a temporary VM → Packer saves it as a golden AMI → ASGs launch identical instances |
| AWS SSM | Connect through Session Manager (`community.aws.aws_ssm` connection) without SSH |
| Kubernetes | `kubernetes.core.k8s` / `helm` modules; but Kubernetes config is usually managed by GitOps (Argo CD) |
| AWX / Ansible Automation Platform | Web UI, RBAC, credentials store, schedules, job history, REST API |

| Integration | How |
|---|---|
| Semaphore UI, Rundeck | Lightweight alternatives |
| Network automation | `cisco.ios`, `arista.eos`, `junipernetworks.junos` collections (`network_cli` connection) |

**Performance tips:** raise `forks`, enable `pipelining`, disable fact gathering when unneeded or cache facts, use `strategy: free` where order doesn't matter, avoid looping a module where it accepts a list (`apt name: [a, b, c]`).

## 10.16 Project: web server role

**Goal:** an idempotent role that installs nginx, deploys a site from a template, opens the firewall, and reloads nginx only when the config changes.

```
# roles/web/defaults/main.yml
web_server_name: "{{ inventory_hostname }}"
web_root: /var/www/site
web_port: 80
web_packages:
  - nginx
```

```
# roles/web/tasks/main.yml
- name: Install web packages
  ansible.builtin.package:
    name: "{{ web_packages }}"
    state: present

- name: Create web root
  ansible.builtin.file:
    path: "{{ web_root }}"
    state: directory
    owner: root
    group: root
    mode: "0755"

- name: Deploy index page
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ web_root }}/index.html"
    mode: "0644"

- name: Remove the distribution's default site (only our config should serve traffic)
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
    state: absent
  notify: Reload nginx

- name: Deploy nginx site configuration
  ansible.builtin.template:
    src: site.conf.j2
    dest: /etc/nginx/conf.d/site.conf
    mode: "0644"
  notify: Reload nginx

- name: Validate the complete nginx configuration
  ansible.builtin.command: nginx -t
  changed_when: false             # read-only check: never reports "changed"

- name: Ensure nginx is enabled and running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

```
# roles/web/handlers/main.yml
- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

```
# roles/web/templates/site.conf.j2
server {
    listen {{ web_port }};
    server_name {{ web_server_name }};
    root {{ web_root }};
    index index.html;
}
```

```
# roles/web/templates/index.html.j2
<h1>{{ web_server_name }}</h1>
<p>Deployed by Ansible on
   {{ ansible_facts['distribution'] }} {{ ansible_facts['distribution_version'] }}</p>
```

```
# web.yml
- name: Configure web servers
  hosts: web
  become: true
  roles:
    - web
```

**Run and verify idempotency:**

```
ansible-playbook -i inventory/hosts.yml web.yml --check --diff   # preview
ansible-playbook -i inventory/hosts.yml web.yml                  # first run: changed > 0
ansible-playbook -i inventory/hosts.yml web.yml                  # second run: changed=0
curl http://<web-host>/
```

The second run reporting `changed=0` proves the role is idempotent.

## 10.17 Troubleshooting

| Error | Cause | Fix |
|---|---|---|
| `UNREACHABLE! ... Permission denied (publickey)` | Wrong user/key, key not on host | Check `ansible_user`, `private_key_file`; `ssh -i key user@host` manually; `-vvvv` |
| `UNREACHABLE! ... Connection timed out` | Network/firewall/security group/bastion | Test SSH directly; configure `ProxyJump` in `ansible_ssh_common_args` |
| `Missing sudo password` | `become` needs a password | `-K`, or passwordless sudo for the automation user |
| `/usr/bin/python: not found` | Python missing or in another path | Install python3; set `ansible_python_interpreter=/usr/bin/python3` |
| `Could not import the python3-apt module` | Ansible uses a Python that lacks the OS apt bindings (e.g. a newer Python than the system one) | Set `ansible_python_interpreter` to the system Python (`/usr/bin/python3`), or install `python3-apt` for it |
| `couldn't resolve module/action` | Collection not installed | `ansible-galaxy collection install ...` / `requirements.yml` |
| `'x' is undefined` | Variable missing for that host/group | Check precedence, group membership (`ansible-inventory --host h`), typo; use `default()` |
| `Syntax Error while loading YAML` | Indentation, tabs, unquoted `{{` at value start, colons in strings | `--syntax-check`, yamllint, quote values |
| Task always shows `changed` | `command` / `shell` without `changed_when` / `creates` | Use proper modules or set `changed_when` |
| Handler didn't run | Notifying task didn't change, name mismatch, play failed before the end | Check names; `--force-handlers`; `meta: flush_handlers` |
| Very slow runs | Fact gathering, serial loops, low forks | Pipelining, more forks, fact caching, list arguments |
| Host key verification failed | New/changed host keys | Add to `known_hosts` (ssh-keyscan) or manage keys properly |

Debug helpers: `-vvv`, `ansible.builtin.debug: var=...`, `ansible-inventory --host <h>` (all variables for a host), `ansible-playbook --step` (confirm each task), `--start-at-task "name"`.

## 10.18 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | What is Ansible and why agentless? | Configuration management tool that connects over SSH/WinRM and runs modules; no agents to install or maintain |
| 2 | Ansible vs Terraform? | Ansible configures systems (procedural tasks, no state file); Terraform provisions infrastructure declaratively with state |
| 3 | What is idempotency in Ansible? | Running a playbook again produces no changes if the system is already in the desired state |
| 4 | Inventory: static vs dynamic? | Fixed list of hosts vs generated from a cloud API by tags (aws_ec2 plugin) |
| 5 | Playbook vs play vs task vs role? | File of plays; a play maps hosts to tasks; a task calls a module; a role packages tasks/handlers/templates/vars |
| 6 | What are handlers? | Tasks triggered by `notify` from changed tasks, run once at the end (e.g. restart a service) |
| 7 | `command` vs `shell`? | `command` runs without a shell (safer); `shell` allows pipes/redirects; both not idempotent by default |
| 8 | How do you make a command task idempotent? | `creates` / `removes`, `changed_when`, or use a dedicated module |
| 9 | Explain variable precedence briefly | Role defaults lowest → inventory vars → play vars → role vars → set_fact/register → extra vars (`-e`) highest |
| 10 | What are facts? | System information gathered from hosts (OS, IPs, memory) available as `ansible_facts` |
| 11 | How do you handle secrets? | Ansible Vault (encrypted files/strings) or runtime lookups from secret managers, with `no_log` |
| 12 | How do you do a rolling update? | `serial`, `max_fail_percentage`, pre/post tasks with `delegate_to` for load balancer (de)registration |
| 13 | What is Jinja2 used for? | Templating variables, filters, loops and conditions in playbooks and config templates |
| 14 | `import_*` vs `include_*`? | Static (parsed upfront, tags inherited) vs dynamic (evaluated at runtime, can loop/condition) |
| 15 | How do you test roles? | ansible-lint, syntax-check, check/diff mode, Molecule (converge + idempotency test) |
| 16 | How do you run only part of a playbook? | Tags (`--tags` / `--skip-tags`), `--limit`, `--start-at-task` |
| 17 | What is `delegate_to`? | Runs a task on a different host than the current target (e.g. localhost or a load balancer) |
| 18 | What is AWX / AAP? | Web UI/API for Ansible: RBAC, credentials, schedules, job history |
| 19 | How do you speed up Ansible? | More forks, pipelining, fact caching or no facts, `strategy: free`, list arguments instead of loops |
| 20 | A host is unreachable. How do you debug? | `ansible host -m ping -vvvv`, manual SSH with the same user/key, network/SG/bastion checks |

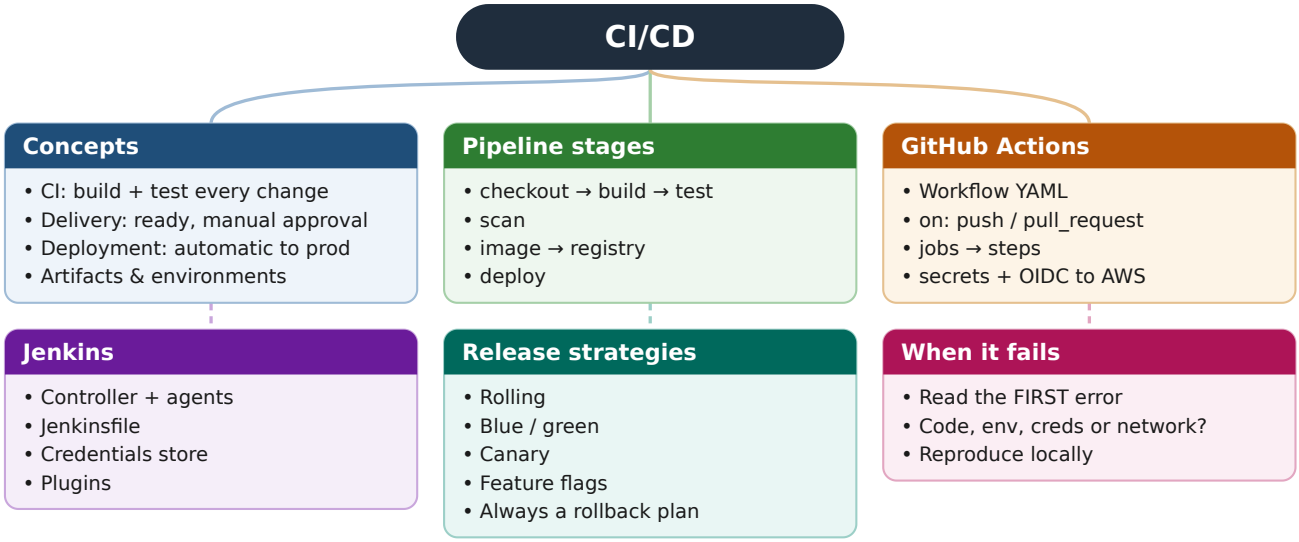
## 10.19 Self-check: I can…

- [ ] Explain configuration management, agentless push, idempotency and how Ansible complements Terraform.
- [ ] Install Ansible and collections; write a project `ansible.cfg`.
- [ ] Write static inventories (INI/YAML) with groups, group_vars/host_vars, and an AWS dynamic inventory.
- [ ] Run ad-hoc commands with the right flags.
- [ ] Write correct YAML and complete playbooks with handlers.
- [ ] Use the most common modules with FQCN and read return values.
- [ ] Explain variable precedence and use facts, magic variables, register and set_fact.
- [ ] Use Jinja2 filters, templates with loops/conditions, and lookups.
- [ ] Use loops, conditions, blocks/rescue/always, tags, delegate_to, serial rolling updates.
- [ ] Build roles, use collections and pin dependencies with requirements.yml.
- [ ] Protect secrets with Vault or secret manager lookups and `no_log`.
- [ ] Lint and test with ansible-lint, check/diff and Molecule.
- [ ] Build the web role and prove idempotency (`changed=0` on the second run).
- [ ] Troubleshoot connection, privilege, variable, YAML and idempotency problems.

**REMEMBER**

- **Terraform builds the house; Ansible furnishes it.**
- Idempotent = running it twice changes nothing the second time.

# 11. CI/CD




## 11.0 Why CI/CD

Without automation, releases are big, risky, manual events: "merge everything on Friday, build on someone's laptop, copy files to servers, hope". CI/CD turns delivery into a **repeatable, automated, fast and safe** pipeline: every change is built, tested, scanned and deployable within minutes. It is the core of DevOps work.

## 11.1 Core concepts

| Term | Meaning |
|---|---|
| **Continuous Integration (CI)** | Developers merge small changes to the main branch frequently; every change triggers an automatic build and tests, so integration problems are found within minutes |
| **Continuous Delivery** | Every change that passes the pipeline is **releasable**; deploying to production is a business decision (one click / approval) |
| **Continuous Deployment** | Every passing change goes to production **automatically**, no human gate |
| Pipeline | Automated sequence of stages defined as code ( `.github/workflows/*.yml` , `Jenkinsfile` , `.gitlab-ci.yml` ) |
| Stage / job / step | Phase (build, test, deploy) / unit of work on one runner / single command or action |
| Runner / agent | Machine/container that executes jobs |
| Artifact | Output of a build kept for later stages (container image, JAR, zip, test reports) |
| Environment | Deployment target (dev, staging, prod) with its own config, secrets and protection rules |
| Trigger | What starts a pipeline: push, pull request, tag, schedule, manual, another pipeline |
| Quality gate | Condition that must pass to continue (tests, coverage, no critical vulnerabilities, approval) |

```
CONTINUOUS INTEGRATION                    CONTINUOUS DELIVERY / DEPLOYMENT
commit → build → unit tests → lint →     publish artifact → deploy dev → integration/e2e tests →
SAST/deps scan → package (image) →       deploy staging → (manual approval = Delivery) → deploy prod
                                          → verify (smoke tests, metrics) → (auto = Deployment)
```

### Principles

1. **Everything in version control:** app code, pipeline definitions, infrastructure, configuration.
2. **Small, frequent changes:** smaller blast radius, easier reviews and rollbacks.
3. **Fast feedback:** CI on a PR should finish in minutes (parallelism, caching, test selection).
4. **Build once, deploy many:** one immutable artifact (image digest) promoted through all environments; only configuration differs.
5. **Fail fast:** cheap checks first (lint, unit tests) before expensive ones (e2e, deployments).
6. **Keep main green:** a broken main branch is everyone's top priority.
7. **Automate rollback** and make deployments boring.
8. **Shift left:** test and scan for security early, in the PR.

### DORA metrics (how teams measure delivery performance)

| Metric | Measures | Elite teams (indicative) |
|---|---|---|
| Deployment frequency | How often you deploy to production | On demand (many per day) |
| Lead time for changes | Commit → running in production | Less than a day |
| Change failure rate | % of deployments causing incidents/rollbacks | Low (around 0–15%) |
| Time to restore service (MTTR) | How quickly you recover from failures | Less than an hour |

## 11.2 Anatomy of a pipeline

| Stage | Typical steps | Fails when |
|---|---|---|
| Source | Checkout, determine version (git SHA/tag) | — |
| Static checks | Formatting, lint (ruff, eslint, hadolint, tflint), type checks | Style/lint errors |
| Build | Compile, install dependencies (with cache) | Compilation/dependency errors |
| Unit tests | Fast isolated tests + coverage report | Test failures, coverage below threshold |
| Security (shift-left) | Secret scan (gitleaks), SAST (CodeQL/Semgrep), dependency scan (SCA), IaC scan | Leaked secrets, critical findings |
| Package | Build container image / artifact, SBOM, sign | Build errors |
| Image scan | Trivy/Grype on the image | Critical/high CVEs |
| Publish | Push to registry/artifact repo with immutable tags | Auth/registry errors |
| Deploy to dev/test | Helm/kubectl/ECS/Terraform/GitOps update | Rollout fails, health checks fail |
| Integration / e2e / contract tests | Tests against deployed services | Broken integrations |
| Approval (Delivery) | Human gate for production | Rejected |

| Stage | Typical steps | Fails when |
|---|---|---|
| Deploy to prod | Progressive strategy (rolling/canary/blue-green) | Health/metrics checks fail → rollback |
| Verify | Smoke tests, synthetic checks, error-rate/latency analysis | Regressions |
| Notify | Chat/email, update tickets, release notes | — |

### Test types (the testing pyramid)

```
        /\        End-to-end / UI tests      few, slow, brittle, most realistic
       /  \       Integration / contract     some: services + DB/APIs together
      /    \      Unit tests                 many, fast, isolated
     /______\
```

Plus: **smoke tests** (quick "is it alive" after deploy), **performance/load tests** (k6, JMeter, Locust), **security tests** (DAST: OWASP ZAP), **chaos tests**.

### Caches vs artifacts

| | Cache | Artifact |
|---|---|---|
| Purpose | Speed up builds (dependencies, build cache) | Pass outputs between jobs / keep results |
| Correctness | Must be safe to lose | Part of the pipeline's result |
| Examples | `~/.cache/pip`, `node_modules` keyed by lockfile hash, Docker layer cache | Images, binaries, test/coverage reports, SBOMs, Terraform plans |

## 11.3 Versioning, artifacts and supply chain

| Topic | Practice |
|---|---|
| Version scheme | Semantic versions for releases ( `v1.4.2` tag) + git SHA for every build ( `sha-3f9a1c2` ) |
| Artifact repositories | Container registries (ECR, GHCR, Docker Hub, Harbor), package repos (Nexus, Artifactory, CodeArtifact, PyPI/npm private), S3 for files |
| Immutability | A version is never overwritten (registry tag immutability); deploy by **digest** for certainty |
| Traceability | Labels/annotations linking artifact → commit → pipeline run → PR |
| SBOM | Software bill of materials generated at build (Syft, Trivy) |
| Signing and provenance | Sign images (cosign/Sigstore), produce build provenance (**SLSA** levels, GitHub artifact attestations); verify before deploy |
| Retention | Lifecycle policies to delete old/untagged artifacts |

## 11.4 Environments, configuration and promotion

```
          same image digest sha256:ab12...
build ──► DEV (auto) ──► STAGING (auto + e2e) ──► PROD (approval / automated checks)
           config: dev     config: staging          config: prod (secrets from a secret manager)
```

| Practice | Why |
|---|---|
| Promote artifacts, don't rebuild per environment | What you tested is exactly what you ship |
| Config outside the artifact (env vars, ConfigMaps, parameter stores) | Same image everywhere |
| Environment-specific secrets in the CI secret store / secret manager, scoped per environment | Dev pipelines can't touch prod secrets |
| Protection rules (required reviewers, wait timers, branch restrictions) on prod | Controlled releases |
| Production-like staging | Find problems before prod |
| Ephemeral preview environments per PR | Test features in isolation |

### Database migrations

Schema changes are the hardest part of continuous delivery. Use migration tools (Flyway, Liquibase, Alembic, Django/Rails migrations) run by the pipeline, and the **expand → migrate → contract** pattern: (1) add new columns/tables in a backward-compatible way, (2) deploy code using both, (3) backfill data, (4) remove old columns in a later release. This keeps every deployment **rollback-safe**.

### Feature flags

Deploy code "dark" and switch features on per user/percentage at runtime (LaunchDarkly, Unleash, Flagsmith, AWS AppConfig). Separates **deploy** from **release**, enables trunk-based development and instant "rollback" by turning a flag off.

## 11.5 Deployment strategies in depth

```
ROLLING          v1 v1 v1 v1 → v2 v1 v1 v1 → v2 v2 v1 v1 → v2 v2 v2 v2
BLUE/GREEN       [blue v1 ← 100% traffic]   [green v2 ready] → switch → [green v2 ← 100%]
CANARY           v1 95% / v2 5% → watch metrics → 25% → 50% → 100% (or roll back)
```

| Strategy | How | Downtime | Rollback | Extra cost | Risk |
|---|---|---|---|---|---|
| Recreate | Stop all old, start new | Yes | Redeploy old | None | High |
| **Rolling** | Replace instances in batches ( `maxSurge` / `maxUnavailable` ) | No | Roll back gradually | Small | Medium: mixed versions during rollout |
| **Blue/green** | Full new environment, switch traffic (LB/DNS) | No | Instant switch back | 2× during release | Low |
| **Canary** | Small % of traffic first, analyze metrics, increase | No | Shift traffic back | Small | Lowest (limited blast radius) |
| A/B testing | Route specific users to versions to compare business metrics | No | Route back | Small | Product experiment |
| Shadow (mirroring) | Copy real traffic to new version, discard responses | No | n/a | Extra capacity | Tests with real traffic safely |
| Feature flags | Code deployed off, enabled gradually | No | Turn flag off | Flag platform | Low |

**Tools:** Kubernetes Deployments (rolling), **Argo Rollouts** / Flagger (canary, blue/green with automated metric analysis), AWS CodeDeploy (EC2/ECS/Lambda blue/green and canary), ALB weighted target groups, Route 53 weighted records, service meshes (Istio traffic splitting).

**Automated verification:** after each step, check error rate, latency (p95/p99), saturation and business KPIs; roll back automatically when thresholds are breached.

## 11.6 GitHub Actions in depth

### Building blocks

| Concept | Meaning |
|---|---|
| Workflow | YAML file in `.github/workflows/`; a repo can have many |
| Event (`on`) | Trigger: `push`, `pull_request`, `workflow_dispatch` (manual, with inputs), `schedule` (cron, UTC), `release`, `workflow_call` (reusable), `workflow_run`, `merge_group`, `repository_dispatch` |
| Job | Group of steps on one runner; jobs run **in parallel** unless linked with `needs` |
| Step | `run:` a shell command or `uses:` an action |
| Action | Reusable unit (`actions/checkout@v4`, `docker/build-push-action@v6`) from the Marketplace or your repo |
| Runner | GitHub-hosted (`ubuntu-latest`, `windows-latest`, `macos-latest`, larger/ARM runners) or **self-hosted** (your machines, e.g. in your VPC; ARC runs them on Kubernetes) |
| Secrets / variables | Encrypted secrets and plain variables at repo, **environment** or organization level |
| Environment | Deployment target with protection rules (required reviewers, wait timer, branch restrictions) and its own secrets |
| Artifacts / cache | Files passed between jobs / dependency caches between runs |

## Syntax reference

```
name: ci                                    # shown in the Actions tab
run-name: CI for ${{ github.ref_name }}     # optional per-run title

on:
  push:
    branches: [main]
    tags: ["v*"]
    paths-ignore: ["docs/**", "*.md"]
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      environment:
        description: Target environment
        type: choice
        options: [dev, staging]
        default: dev
  schedule:
    - cron: "0 3 * * 1"                     # Mondays 03:00 UTC

permissions:                                # least privilege for GITHUB_TOKEN (default for all jobs)
  contents: read

concurrency:                                # cancel older runs of the same branch
  group: ci-${{ github.ref }}
  cancel-in-progress: true

env:                                        # workflow-level environment variables
  PYTHON_VERSION: "3.12"

defaults:
  run:
    shell: bash

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    strategy:
      fail-fast: false
      matrix:
        python: ["3.11", "3.12"]
    services:                               # sidecar containers for tests
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-timeout 3s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: pip
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest -q --junitxml=report-${{ matrix.python }}.xml
        env:
          DATABASE_URL: postgresql://postgres:test@localhost:5432/postgres
      - uses: actions/upload-artifact@v4
        if: always()                        # upload even if tests failed
        with:
```

```
          name: test-report-${{ matrix.python }}
          path: report-${{ matrix.python }}.xml
```

| Keyword | Purpose |
|---|---|
| `needs: [test, lint]` | Wait for other jobs; access their outputs via `needs.<job>.outputs.<name>` |
| `if:` | Conditions: `github.ref == 'refs/heads/main'`, `github.event_name == 'push'`, `success()`, `failure()`, `always()`, `cancelled()` |
| `strategy.matrix` | Run a job for combinations (versions, OSes); `include` / `exclude` |
| `outputs:` + `$GITHUB_OUTPUT` | Pass values between steps/jobs: `echo "tag=v1" >> "$GITHUB_OUTPUT"` |
| `$GITHUB_ENV`, `$GITHUB_STEP_SUMMARY`, `$GITHUB_PATH` | Set env vars for later steps / write a Markdown summary / add to PATH |
| `environment: production` | Use environment protection rules and secrets |
| `continue-on-error`, `timeout-minutes` | Allow failure / prevent hung jobs |
| `working-directory` | Directory for `run` steps |

## Expressions and contexts

`${{ <expression> }}` evaluates contexts: `github` (event, ref, sha, actor, repository), `env`, `vars`, `secrets`, `matrix`, `steps.<id>.outputs`, `needs`, `inputs`, `runner`, `job`. Functions: `contains()`, `startsWith()`, `format()`, `toJSON()`, `fromJSON()`, `hashFiles('**/requirements.txt')`.

## GITHUB_TOKEN and permissions

Every run gets an automatic short-lived `GITHUB_TOKEN`. Set **minimal** `permissions:` at workflow or job level (e.g. `contents: read`, `packages: write` to push to GHCR, `id-token: write` for OIDC, `pull-requests: write` to comment). Default organization settings should be read-only.

## Secrets

- Add in Settings → Secrets and variables → Actions (repo / environment / org). Reference as `${{ secrets.NAME }}`; values are masked in logs.
- Secrets are **not** passed to workflows triggered by pull requests from forks.
- Prefer **OIDC** for cloud access instead of storing cloud keys:

```
permissions:
  id-token: write
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/github-deploy
      aws-region: ap-south-1
```

The IAM role's trust policy restricts which repo/branch/environment may assume it (see AWS notes, IAM roles).

## Caching

`actions/setup-python|node|java|go` have built-in `cache:` options. General cache:

```
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: pip-${{ runner.os }}-${{ hashFiles('**/requirements*.txt') }}
    restore-keys: pip-${{ runner.os }}-
```

Docker layer caching: `docker/build-push-action` with `cache-from: type=gha` / `cache-to: type=gha,mode=max`.

## Reusable workflows and composite actions

```
# .github/workflows/build-image.yml (in a central "platform" repo)
on:
  workflow_call:
    inputs:
      image-name:
        type: string
        required: true
    secrets:
      registry-token:
        required: false
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t "${{ inputs.image-name }}" .
```

```
# Caller in an application repo
jobs:
  image:
    uses: acme/platform/.github/workflows/build-image.yml@v1
    with:
      image-name: shop-api
```

**Reusable workflows** share whole jobs (golden-path pipelines across many repos). **Composite actions** (`action.yml` with `runs: using: composite`) bundle several steps into one reusable step.

## Security hardening for Actions

| Risk | Mitigation |
|---|---|
| Compromised third-party action | Pin actions to a full **commit SHA** (`uses: actions/checkout@<sha> # v4`), prefer verified creators, Dependabot for action updates |
| Over-privileged token | Minimal `permissions:`; org default read-only |
| Script injection | Never put untrusted input (`github.event.pull_request.title`, issue bodies, branch names) directly into `run:`; pass via `env:` and quote |
| `pull_request_target` / `workflow_run` misuse | These run with secrets in the base repo context: never check out and run untrusted PR code in them |
| Secrets exposure | Environment-scoped secrets, OIDC, no secrets printed, `::add-mask::` for dynamic values |
| Self-hosted runner compromise | Use ephemeral runners, never on public repos, isolate in their own network/accounts |
| Supply chain | Dependency review, CodeQL, artifact attestations / signed images |

Lint workflows with **actionlint**; scan with zizmor/OpenSSF Scorecard.

### Complete pipeline: test → build → scan → push → deploy (GitOps)

```
name: delivery
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: delivery-${{ github.ref }}
  cancel-in-progress: true

env:
  REGISTRY: 123456789012.dkr.ecr.ap-south-1.amazonaws.com
  IMAGE: shop/api

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: ruff check .
      - run: pytest -q

  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

  image:
    needs: [test, security]
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write
    outputs:
      tag: ${{ steps.meta.outputs.tag }}
    steps:
      - uses: actions/checkout@v4
      - id: meta
        run: echo "tag=sha-${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-ecr-push
          aws-region: ap-south-1
      - uses: aws-actions/amazon-ecr-login@v2
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v6
        with:
          context: .
          load: true
```

```
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ steps.meta.outputs.tag }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ steps.meta.outputs.tag }}
          severity: CRITICAL,HIGH
          exit-code: "1"
          ignore-unfixed: true
      - run: docker push "${REGISTRY}/${IMAGE}:${{ steps.meta.outputs.tag }}"

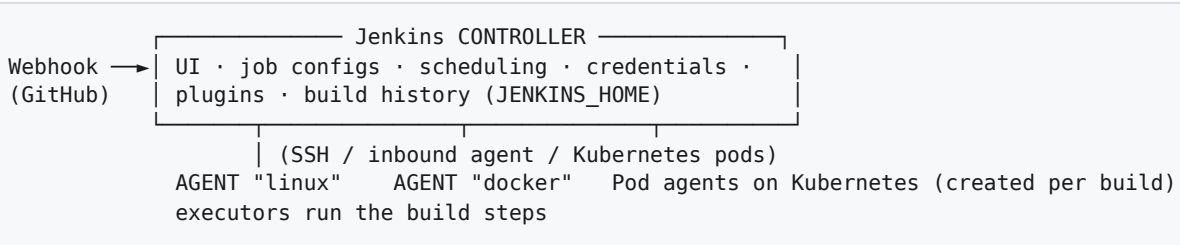
  deploy-dev:
    needs: image
    runs-on: ubuntu-latest
    environment: dev
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
        with:
          repository: acme/shop-config            # GitOps config repo
          token: ${{ secrets.CONFIG_REPO_TOKEN }}
      - name: Bump image tag for dev
        env:
          TAG: ${{ needs.image.outputs.tag }}
        run: |
          yq -i ".image.tag = \"${TAG}\"" apps/shop-api/values-dev.yaml
          git config user.name "ci-bot"
          git config user.email "ci-bot@users.noreply.github.com"
          git commit -am "shop-api: deploy ${TAG} to dev"
          git push
```

Argo CD watches the config repo and deploys the new tag (see GitOps notes). Production promotion is a pull request to `values-prod.yaml`, protected by review: an auditable, revertible release.

## 11.7 Jenkins in depth

**Jenkins** is the open-source, self-hosted automation server: extremely flexible through ~1,800 plugins, still very common in enterprises (often alongside newer tools).

### Architecture




| Term | Meaning |
|---|---|
| Controller | Central server: UI, configuration, scheduling. **Don't run builds on it** (set its executors to 0) |
| Agent | Machine/container executing builds; selected by **labels** ( `linux` , `docker` , `arm64` ) |
| Executor | A build slot on an agent |
| Job / project types | Freestyle (UI-configured, legacy), **Pipeline** (Jenkinsfile), **Multibranch Pipeline** (one pipeline per branch/PR automatically), Organization Folder (all repos of a GitHub org) |

| Term | Meaning |
|---|---|
| Plugins | Git, GitHub Branch Source, Pipeline, Credentials Binding, Docker Pipeline, Kubernetes, JUnit, Configuration as Code, Role-based Authorization |
| `JENKINS_HOME` | All configuration and build data: back it up |

## Installing (lab)

```
docker network create jenkins
docker run -d --name jenkins --network jenkins --restart unless-stopped \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts-jdk17
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Open `http://localhost:8080`, unlock, install suggested plugins, create an admin user. Port 50000 is for inbound agents.

## Declarative Jenkinsfile reference

```
pipeline {
  // where to run: agent any / none / docker {...} / kubernetes {...} / a label
  agent { label 'docker' }

  options {
    timeout(time: 30, unit: 'MINUTES')
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '30'))
    timestamps()
  }

  parameters {
    choice(name: 'TARGET_ENV', choices: ['dev', 'staging'], description: 'Deploy target')
    booleanParam(name: 'RUN_E2E', defaultValue: true, description: 'Run end-to-end tests')
  }

  triggers {
    pollSCM('H/15 * * * *')                          // prefer webhooks; polling as fallback
  }

  environment {
    REGISTRY = '123456789012.dkr.ecr.ap-south-1.amazonaws.com'
    IMAGE    = "${REGISTRY}/shop/api"
    TAG      = "sha-${env.GIT_COMMIT.take(7)}"
  }

  stages {
    stage('Checkout') {
      steps { checkout scm }
    }

    stage('Quality') {
      parallel {                                     // run in parallel
        stage('Unit tests') {
          steps {
            sh 'pip install -r requirements.txt -r requirements-dev.txt'
            sh 'pytest -q --junitxml=reports/junit.xml'
          }
          post { always { junit 'reports/junit.xml' } }
        }
        stage('Lint') {
          steps { sh 'ruff check .' }
        }
      }
    }

    stage('Build image') {
      steps { sh 'docker build -t "$IMAGE:$TAG" .' }
    }

    stage('Scan image') {
      steps { sh 'trivy image --severity HIGH,CRITICAL --exit-code 1 "$IMAGE:$TAG"' }
    }

    stage('Push') {
      when { branch 'main' }
      steps {
        withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
                          credentialsId: 'aws-ecr-push']]) {
          sh '''
            aws ecr get-login-password --region ap-south-1 \
              | docker login --username AWS --password-stdin "$REGISTRY"
            docker push "$IMAGE:$TAG"
          '''
```

```
        }
      }
    }

    stage('Deploy to dev') {
      when { branch 'main' }
      steps { sh './deploy.sh dev "$TAG"' }
    }

    stage('Approve production') {
      when { branch 'main' }
      steps {
        timeout(time: 1, unit: 'HOURS') {
          input message: "Deploy ${TAG} to production?", ok: 'Deploy', submitter: 'release-managers'
        }
      }
    }

    stage('Deploy to production') {
      when { branch 'main' }
      steps { sh './deploy.sh prod "$TAG"' }
    }
  }

  post {
    success { echo "Build ${env.BUILD_NUMBER} OK" }
    failure { echo 'Build failed - notify the team (Slack/email plugin)' }
    always  { cleanWs() }
  }
}
```

| Section / directive | Purpose |
|---|---|
| `agent` | Where the pipeline/stage runs ( `label` , `docker { image '...' }` , `kubernetes { yaml '...' }` ) |
| `options` | Timeouts, concurrency, retention, timestamps, retries |
| `parameters` | Inputs for manual/parameterized builds ( `params.TARGET_ENV` ) |
| `triggers` | `cron` , `pollSCM` , upstream; webhooks via the GitHub plugin are preferred |
| `environment` | Variables (and `credentials('id')` bindings) |
| `stages` / `stage` / `steps` | Pipeline structure |
| `when` | Conditions: `branch` , `tag` , `changeset` , `expression { ... }` , `environment` |
| `parallel` | Run stages concurrently |
| `input` | Manual approval with allowed submitters |
| `post` | `always` , `success` , `failure` , `unstable` , `changed` , `cleanup` |

**Scripted pipelines** ( `node { ... }` , pure Groovy) are older and more flexible but harder to read; prefer declarative.

## Credentials

Stored encrypted in Jenkins (or fetched from Vault/AWS Secrets Manager via plugins). Types: username/password, secret text, SSH key, secret file, certificate, AWS credentials. Use them only inside `withCredentials` / `credentials()` bindings; Jenkins masks them in logs. **Never** echo them or put them in the Jenkinsfile.

```
environment {
  SONAR_TOKEN = credentials('sonar-token')          // secret text → env var
}
steps {
  withCredentials([usernamePassword(credentialsId: 'dockerhub',
                                    usernameVariable: 'U', passwordVariable: 'P')]) {
    // single quotes: the shell expands $P, not Groovy (keeps the secret out of the command string)
    sh 'echo "$P" | docker login -u "$U" --password-stdin'
  }
}
```

**Groovy string tip:** use **single quotes** in `sh '...'` for secrets so Groovy doesn't interpolate them into the command string (which can leak them).

## Shared libraries

Reusable Groovy code in a separate repo (`vars/buildPythonApp.groovy`), loaded with `@Library('acme-pipelines@v2') _`: teams then write a 3-line Jenkinsfile like `buildPythonApp(image: 'shop/api')`. This is Jenkins' version of golden-path pipelines.

## Agents on Kubernetes

The Kubernetes plugin creates a **pod per build** with the containers you need (e.g. `python`, `docker` / `kaniko`, `kubectl`), then deletes it: elastic capacity, clean environments.

## Operating Jenkins well

| Area | Practice |
|---|---|
| Configuration as code | **JCasC** plugin (`jenkins.yaml`) + job DSL / multibranch; no manual click-configuration |
| Security | Disable builds on controller, enable CSRF protection, role-based authorization (Matrix/Role Strategy), SSO, least-privilege credentials, restrict script approvals |
| Updates | Keep core and plugins updated (plugins are the main source of vulnerabilities); test upgrades |
| Backups | Back up `JENKINS_HOME` (or rebuild from JCasC + Git) |
| Scale | Ephemeral agents (Kubernetes/cloud), build discarders, artifact storage outside Jenkins (S3/Artifactory) |
| Webhooks | GitHub → Jenkins `/github-webhook/` for instant builds |

## 11.8 Other CI/CD platforms

### GitLab CI

```
# .gitlab-ci.yml
stages: [test, build, deploy]

variables:
  IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

test:
  stage: test
  image: python:3.12-slim
  script:
    - pip install -r requirements.txt -r requirements-dev.txt
    - pytest -q
  cache:
    key:
      files: [requirements.txt]
    paths: [.cache/pip]

build:
  stage: build
  image: docker:27
  services: [docker:27-dind]
  script:
    - echo "$CI_REGISTRY_PASSWORD"
      | docker login -u "$CI_REGISTRY_USER" --password-stdin "$CI_REGISTRY"
    - docker build -t "$IMAGE" .
    - docker push "$IMAGE"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

deploy_prod:
  stage: deploy
  script: ./deploy.sh prod "$CI_COMMIT_SHORT_SHA"
  environment: production
  when: manual
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

### AWS CodeBuild buildspec

```
version: 0.2
phases:
  install:
    runtime-versions:
      python: 3.12
  pre_build:
    commands:
      - pip install -r requirements.txt
      - aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY
  build:
    commands:
      - pytest -q
      - docker build -t $REGISTRY/shop/api:$CODEBUILD_RESOLVED_SOURCE_VERSION .
  post_build:
    commands:
      - docker push $REGISTRY/shop/api:$CODEBUILD_RESOLVED_SOURCE_VERSION
artifacts:
  files: [imagedefinitions.json]
```

(CodePipeline orchestrates source → CodeBuild → CodeDeploy/ECS deploy.)

### Comparison

| | GitHub Actions | Jenkins | GitLab CI | Azure DevOps | AWS CodePipeline |
|---|---|---|---|---|---|
| Hosting | SaaS (+ self-hosted runners) | Self-hosted | SaaS or self-hosted | SaaS or server | AWS managed |
| Config | YAML | Groovy Jenkinsfile | YAML | YAML / classic | JSON/console + buildspec YAML |
| Strength | Native to GitHub, huge marketplace | Ultimate flexibility, plugins, on-prem | All-in-one DevOps platform | Microsoft/.NET enterprises | AWS-native integration |
| Weakness | GitHub-centric | Maintenance and plugin sprawl | GitLab-centric | Complexity | AWS-only, less flexible |

Kubernetes-native options: **Tekton** (CI pipelines as CRDs), **Argo Workflows**; CD: **Argo CD** / Flux (GitOps), Spinnaker.

## 11.9 Quality gates and testing in pipelines

| Gate | Tool examples |
|---|---|
| Tests pass | pytest, JUnit, Jest; publish JUnit XML for reports |
| Coverage threshold | coverage.py / JaCoCo + `--cov-fail-under=80` |
| Code quality | SonarQube/SonarCloud quality gate, linters |
| Security | No critical SAST/SCA/image findings; no secrets |
| Performance | k6 thresholds (p95 < 300 ms) on staging |
| Manual approval | Production environments |

**Flaky tests** (pass/fail randomly) destroy trust in CI: quarantine and fix them, never just re-run until green. Keep pipelines fast: run unit tests in parallel, cache dependencies, use test impact analysis, run long e2e suites on merge rather than every commit.

## 11.10 Securing the pipeline (DevSecOps in CI/CD)

| Threat | Control |
|---|---|
| Leaked credentials in code/logs | Secret scanning + push protection, masked secrets, OIDC instead of static keys |
| Over-privileged pipelines | Least-privilege tokens/roles per job and environment; separate build and deploy permissions |
| Malicious dependencies / actions / plugins | Pin versions (SHAs/lock files), dependency review, SCA scanning, trusted registries/mirrors |
| Tampered artifacts | Sign images (cosign), generate provenance (SLSA), verify signatures at deploy (admission policies) |
| Poisoned pipeline execution (malicious PR changes the pipeline) | Protected branches, required reviews for workflow changes (CODEOWNERS on `.github/`), no secrets for fork PRs |
| Compromised runners | Ephemeral, isolated runners; no long-lived credentials on them |

| Threat | Control |
|---|---|
| Unreviewed production changes | Branch protection, required checks, environment approvals, audit logs |

## 11.11 Troubleshooting pipelines

| Symptom | Likely cause | Fix |
|---|---|---|
| Works locally, fails in CI | Different versions, missing env vars/secrets, OS differences, uncommitted files | Pin tool versions, reproduce in the same container image, check env |
| `permission denied` pushing image / deploying | Token/role missing permission, wrong OIDC trust condition, `permissions:` block too strict | Read the exact error; adjust least-privilege policy |
| `Error: Credentials could not be loaded` (OIDC) | Missing `id-token: write`, role trust policy `sub` doesn't match repo/branch/environment | Fix permissions and trust policy conditions |
| Tests pass/fail randomly | Flaky tests: timing, ordering, shared state, external dependencies | Isolate, mock, fix waits; quarantine |
| Pipeline very slow | No caching, serial jobs, huge Docker context, unnecessary e2e on every commit | Cache, parallelize, `.dockerignore`, path filters |
| Out of disk on runner | Large images/builds | Clean up, prune Docker, bigger runners |
| YAML error / workflow not triggering | Indentation, wrong event/branch/path filters, workflow on a different branch | actionlint, check `on:` filters and default branch |
| Secret empty in a PR workflow | PRs from forks don't receive secrets | Use `pull_request` safely; run privileged steps after merge |
| Deployment succeeded but app broken | No post-deploy verification | Smoke tests, health/metric checks, automated rollback |
| Jenkins build stuck "waiting for executor" | No agent with the label available, executors busy | Check labels/agents, add capacity |

**Approach:** read the **first** error in the log (later errors are often consequences), re-run with debug logging (`ACTIONS_STEP_DEBUG=true` secret / Jenkins `set -x`), reproduce locally in the same container image, check what changed (dependency versions, secrets expiry, base images).

## 11.12 Labs

1. **CI for a Python app (GitHub Actions):** lint + pytest matrix on 2 Python versions, cache pip, upload test reports; protect `main` requiring the check.
2. **Container pipeline:** build the Docker image, scan with Trivy (fail on CRITICAL), push to GHCR with `sha-<commit>` tags using `GITHUB_TOKEN`.
3. **OIDC to AWS:** create an IAM OIDC provider + role trusted only for your repo's `main` branch; push to ECR without any stored AWS keys.
4. **Environments:** add `dev` (automatic) and `production` (required reviewer) environments; deploy jobs use environment-scoped secrets.
5. **Jenkins:** run Jenkins in Docker, create a Multibranch Pipeline from your repo, add the Jenkinsfile above (simplified), configure a GitHub webhook, store credentials, add an approval step.

6. **Break it on purpose:** introduce a failing test, a leaked fake secret (gitleaks should catch it), and a vulnerable base image (Trivy should fail): fix each.

## 11.13 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | CI vs continuous delivery vs continuous deployment? | Auto build/test on every change / always releasable with manual prod approval / automatic to prod |
| 2 | What stages would your pipeline have? | Checkout → lint → unit tests → security scans → build image → scan → push → deploy dev → integration tests → approval → prod → verify |
| 3 | What is "build once, deploy many"? | One immutable artifact promoted through environments; only config changes |
| 4 | How do you handle secrets in pipelines? | CI secret stores scoped per environment, OIDC for cloud, masking, no secrets in code/logs/images |
| 5 | Blue/green vs canary vs rolling? | Switch between two full environments / gradual traffic shift with metric checks / batch replacement |
| 6 | How do you roll back? | Redeploy previous artifact/digest ( `helm rollback` , `kubectl rollout undo` , revert in GitOps repo), traffic switch for blue/green, feature flag off |
| 7 | How do you make pipelines fast? | Caching, parallel jobs, matrix, path filters, small images, test selection, fail-fast ordering |
| 8 | Artifacts vs cache? | Outputs you keep/pass on vs speed-up data safe to lose |
| 9 | How do you deploy DB schema changes safely? | Migration tools + expand/migrate/contract so each release is backward compatible |
| 10 | What are DORA metrics? | Deployment frequency, lead time, change failure rate, time to restore |
| 11 | GitHub Actions: job vs step vs action? | Job runs on a runner; steps inside run commands or actions; actions are reusable steps |
| 12 | How do you share pipeline logic across repos? | Reusable workflows/composite actions (Actions), shared libraries (Jenkins), templates/includes (GitLab) |
| 13 | How do you secure GitHub Actions? | Minimal permissions, pin actions by SHA, OIDC, environment protections, avoid untrusted input in `run` , careful with `pull_request_target` |
| 14 | Jenkins controller vs agent? | Controller schedules/configures; agents execute builds (no builds on the controller) |
| 15 | Declarative vs scripted Jenkins pipeline? | Structured opinionated syntax vs free-form Groovy |
| 16 | How do you store credentials in Jenkins? | Credentials store/Vault plugin, used via `withCredentials` / `credentials()` , single-quoted shell strings |
| 17 | A pipeline fails only in CI. Approach? | Read the first error, compare versions/env, reproduce in the CI image, check secrets/permissions, recent changes |
| 18 | What is a quality gate? | A required condition (tests, coverage, scans, approvals) a change must pass to progress |
| 19 | What is a feature flag and why use it? | Runtime toggle to enable features gradually; decouples deploy from release and allows instant disable |
| 20 | How do you verify a deployment? | Health checks, smoke tests, synthetic monitoring, error rate/latency analysis, automated rollback |

| # | Question | Answer |
|---|---|---|
| 21 | Self-hosted vs hosted runners? | Your machines (network access, custom tools, cost control, more security responsibility) vs managed, clean, ephemeral |
| 22 | What is GitOps-based delivery? | CI updates the desired version in a Git config repo; an in-cluster agent (Argo CD) applies it |

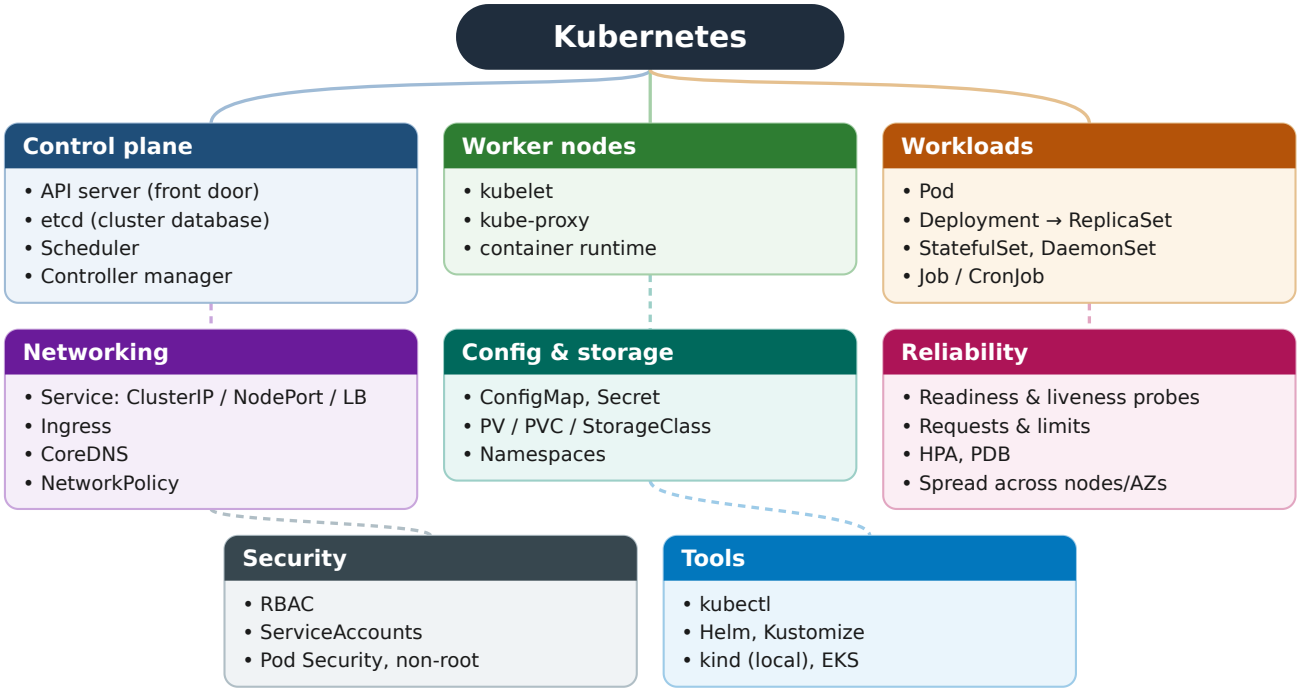
## 11.14 Self-check: I can…

- [ ] Explain CI, continuous delivery and continuous deployment, and the core principles.
- [ ] Design a complete pipeline with the right stages, gates and test types.
- [ ] Explain artifacts vs caches, versioning, immutability, SBOMs and signing.
- [ ] Design environment promotion with config separation, approvals and safe DB migrations.
- [ ] Compare and choose deployment strategies and explain automated rollback.
- [ ] Write GitHub Actions workflows: triggers, matrix, services, caching, artifacts, outputs, environments, concurrency.
- [ ] Use OIDC to deploy to AWS without stored keys and harden workflows.
- [ ] Build reusable workflows/composite actions.
- [ ] Run Jenkins, write a declarative Jenkinsfile with parallel stages, credentials and approvals.
- [ ] Explain shared libraries, Kubernetes agents, JCasC and Jenkins security.
- [ ] Read GitLab CI and CodeBuild configurations.
- [ ] Measure delivery with DORA metrics.
- [ ] Secure the software supply chain in pipelines.
- [ ] Troubleshoot failing, slow and flaky pipelines.

**REMEMBER**

- **Code → Build → Test → Scan → Image → Registry → Deploy.**
- Prefer **OIDC** (short-lived credentials) over stored cloud keys.
- Never disable tests to make a pipeline green.

# 12. Kubernetes




## 12.0 Why Kubernetes

Docker runs containers on one machine. Real systems run hundreds of containers across many machines and need: placing containers on machines with free capacity, restarting failed ones, scaling up and down, rolling out new versions without downtime, service discovery and load balancing, configuration and secrets, storage, and access control. **Kubernetes (K8s)** is the industry-standard platform that does all of this, on any cloud or on-premises.

**Where you meet it:** managed clusters (Amazon EKS, Google GKE, Azure AKS), on-prem (OpenShift, Rancher RKE2, vanilla kubeadm), edge (k3s), local development (kind, minikube, Docker Desktop, k3d).

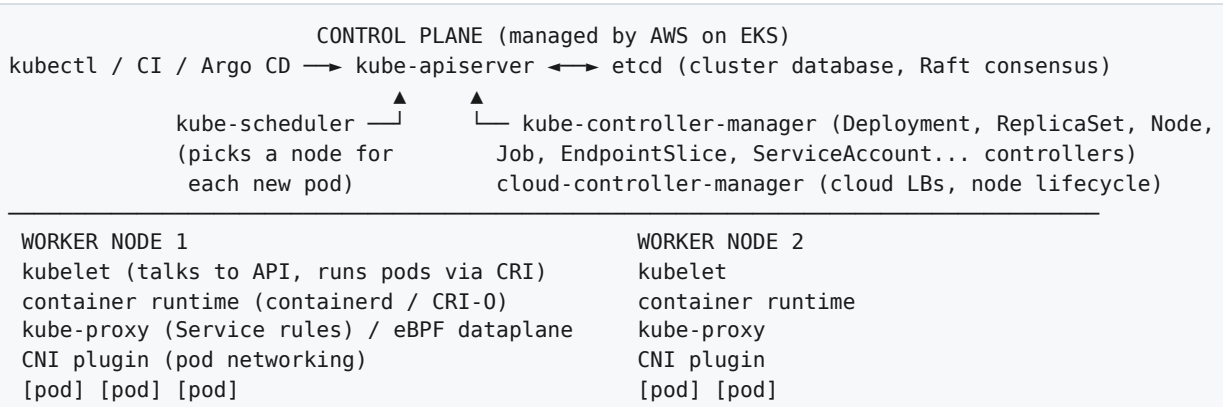
## 12.1 Core idea: desired state and reconciliation

Kubernetes is **declarative**. You store the **desired state** ("3 replicas of image `shop-api:1.4.2`") in the API; **controllers** constantly compare it with the **actual state** and act to close the gap.

```
        ┌──────────── reconciliation loop (forever) ────────────┐
        ▼                                                       │
observe actual state ──► compare with desired spec ──► act (create/delete/update) ─┘
e.g. 2 pods running        spec.replicas = 3              start 1 more pod
```

Every object has a `spec` (what you want, written by you) and a `status` (what exists, written by Kubernetes). This is why Kubernetes self-heals: delete a pod and the ReplicaSet controller creates a new one.

## 12.2 Architecture




| Component | Responsibility |
|---|---|
| **kube-apiserver** | Front door: REST API, authentication, authorization (RBAC), admission control, validation; the only component that talks to etcd |
| **etcd** | Distributed key-value store holding all cluster state. **Back it up** (self-managed clusters) |
| **kube-scheduler** | Assigns pending pods to nodes based on resource requests, affinity, taints, spread constraints |
| **kube-controller-manager** | Runs the built-in control loops |
| cloud-controller-manager | Integrates with the cloud: load balancers for Services, node addresses/lifecycle |
| **kubelet** | Agent on each node: ensures the pods assigned to it are running, runs probes, reports status |
| **Container runtime** | Pulls images and runs containers via the CRI (containerd, CRI-O). Docker Engine itself is no longer used directly by Kubernetes, but Docker-built images work fine |
| **kube-proxy** | Programs iptables/IPVS rules so Service IPs reach pods (some CNIs like Cilium replace it with eBPF) |
| **CNI plugin** | Gives pods IPs and connectivity (AWS VPC CNI, Calico, Cilium, Flannel) |
| Add-ons | CoreDNS (DNS), metrics-server (resource metrics), ingress controller, CSI storage drivers |

### What happens when you `kubectl apply` a Deployment

1. `kubectl` sends the Deployment to the **API server** → authenticated, authorized (RBAC), admission policies applied, stored in **etcd**.
2. **Deployment controller** sees it → creates a **ReplicaSet**.
3. **ReplicaSet controller** creates 3 **Pod** objects (no node yet: `Pending`).
4. **Scheduler** picks a node for each pod (writes `spec.nodeName`).
5. **kubelet** on that node sees the pod → asks the runtime to pull the image and start containers; CNI assigns an IP.
6. kubelet runs **probes**; when the readiness probe passes, the pod becomes **Ready**.
7. **EndpointSlice controller** adds the pod IP to the Service's endpoints → traffic flows.

High-availability control planes run 3+ API server/etcd members across AZs (managed services do this for you).

## 12.3 Tools and kubectl mastery

### Local clusters

| Tool | Notes |
|---|---|
| **kind** | Kubernetes in Docker containers; multi-node; great for labs/CI (`kind create cluster --name lab`) |
| minikube | VM/container-based single node with add-ons |
| k3d / k3s | Lightweight Kubernetes (k3s) in Docker |
| Docker Desktop | Built-in single-node cluster |

```
# kind-config.yaml: 1 control plane + 2 workers, port 80 mapped for an ingress controller
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 80
        hostPort: 8080
  - role: worker
  - role: worker
```

`kind create cluster --name lab --config kind-config.yaml` · `kind delete cluster --name lab`.

### kubeconfig

`~/.kube/config` (or `$KUBECONFIG`) holds **clusters** (API URL + CA), **users** (credentials: certs, tokens, exec plugins like `aws eks get-token`), and **contexts** (cluster + user + default namespace).

```
kubectl config get-contexts
kubectl config use-context kind-lab
kubectl config set-context --current --namespace=shop
kubectl config view --minify            # current context only
```

Always check **which cluster** you're on before destructive commands (tools: `kubectx` / `kubens`, a prompt that shows the context).

### Essential commands

| Command | Purpose |
|---|---|
| `kubectl get pods [-n ns] [-A] [-o wide]` | List (`-A` all namespaces; wide = node, IP) |
| `kubectl get deploy,svc,ingress,pvc -n shop` | Several types at once |
| `kubectl get pod web-7d9 -o yaml` | Full live object |
| `kubectl describe pod web-7d9` | Details + **Events** (scheduling, image pulls, probe failures) |
| `kubectl apply -f file.yaml` / `-f dir/` / `-k overlay/` | Create or update declaratively (Kustomize with `-k`) |
| `kubectl diff -f file.yaml` | Preview changes |

| Command | Purpose |
|---|---|
| `kubectl delete -f file.yaml` / `kubectl delete pod web-7d9` | Delete |
| `kubectl logs pod [-c container] [-f] [--previous] [--tail=100]` | Container logs; `--previous` = crashed instance |
| `kubectl logs deploy/web --all-containers` | Logs via a controller |
| `kubectl exec -it pod -- sh` | Shell inside a container |
| `kubectl debug -it pod --image=busybox --target=app` | Ephemeral debug container (distroless images) |
| `kubectl port-forward svc/web 8080:80` | Reach a Service/pod from your machine |
| `kubectl cp pod:/path ./local` | Copy files |
| `kubectl top pods / nodes` | CPU/memory usage (needs metrics-server) |
| `kubectl get events -A --sort-by=.lastTimestamp` | Recent events |
| `kubectl explain deployment.spec.strategy` | Built-in field documentation |
| `kubectl api-resources` | All resource types (and short names: po, deploy, svc, ns, cm…) |
| `kubectl auth can-i create deployments -n shop --as alice` | Check permissions |
| `kubectl rollout status/history/undo/restart deploy/web` | Manage rollouts |
| `kubectl scale deploy/web --replicas=5` | Scale manually |
| `kubectl label pod web-7d9 tier=frontend` / `kubectl annotate ...` | Metadata |
| `kubectl cordon/drain/uncordon node` | Node maintenance |

## Output and selection

```
kubectl get pods -l app=web,env!=prod                        # label selectors
kubectl get pods --field-selector status.phase!=Running      # field selectors
kubectl get pods -o jsonpath='{.items[*].metadata.name}'
kubectl get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName,IP:.status.podIP
kubectl get pod web-7d9 -o json | jq '.status.containerStatuses[].restartCount'
kubectl create deployment web --image=nginx:1.27 --replicas=2 --dry-run=client -o yaml > web.yaml
```

**Imperative vs declarative:** `kubectl create/run/expose/scale` are fine for experiments and generating YAML; production uses **declarative YAML in Git** applied by CI or GitOps (`kubectl apply`, Helm, Argo CD). `kubectl apply --server-side` lets multiple managers own different fields.

Useful extras: **k9s** (terminal UI), **krew** plugin manager (`kubectl krew install ctx ns neat tree`), **stern** (multi-pod log tailing).

## 12.4 Objects, YAML and metadata

```
apiVersion: apps/v1          # API group/version (core resources use "v1")
kind: Deployment             # object type
metadata:
  name: web                  # unique per kind per namespace
  namespace: shop
  labels:                    # identifying key/values: used by SELECTORS
    app.kubernetes.io/name: web
    app.kubernetes.io/part-of: shop
  annotations:               # non-identifying metadata: tools, descriptions, configs
    owner: team-shop@example.com
spec: {}                     # desired state (structure depends on kind)
# status: written by Kubernetes, never by you
```

| Concept | Meaning |
|---|---|
| **Labels** | Key/value tags used to **select** objects (Services → pods, Deployments → pods, NetworkPolicies, `kubectl -l`). Recommended keys: `app.kubernetes.io/name`, `/instance`, `/version`, `/component`, `/part-of`, `/managed-by` |
| Annotations | Arbitrary metadata, not for selection (ingress settings, checksums, build info) |
| **Namespaces** | Virtual partitions for teams/apps/environments: scope names, RBAC, quotas, policies. System namespaces: `kube-system`, `kube-public`, `default`. Nodes, PVs, ClusterRoles, StorageClasses are **cluster-scoped** |
| Owner references | Pods are owned by ReplicaSets, owned by Deployments: deleting the owner garbage-collects children |
| Finalizers | Keys that block deletion until a controller cleans up external resources (stuck "Terminating" objects often have finalizers) |
| `resourceVersion`, `generation` | Optimistic concurrency / spec change counter |

## 12.5 Pods in depth

A **Pod** is the smallest deployable unit: one or more containers that share a **network namespace** (same IP, talk over `localhost`), can share **volumes**, and are always scheduled together on one node. Pods are **disposable**: you almost never create them directly; controllers (Deployment, StatefulSet, Job) create and replace them.

## Complete pod spec (annotated)

```
apiVersion: v1
kind: Pod
metadata:
  name: api
  labels:
    app.kubernetes.io/name: api
spec:
  serviceAccountName: api                 # identity for RBAC / cloud IAM
  automountServiceAccountToken: false     # only mount a token if the app calls the API
  terminationGracePeriodSeconds: 30       # time between SIGTERM and SIGKILL
  securityContext:                        # pod-level security
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001                        # group owner of mounted volumes
    seccompProfile:
      type: RuntimeDefault
  initContainers:                         # run to completion, in order, before app containers
    - name: wait-for-db
      image: busybox:1.36
      command: ["sh", "-c", "until nc -z db 5432; do echo waiting; sleep 2; done"]
  containers:
    - name: app
      image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
      imagePullPolicy: IfNotPresent
      ports:
        - name: http
          containerPort: 8080
      env:
        - name: LOG_LEVEL
          value: info
        - name: POD_NAME                  # Downward API: pod facts as env vars
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
      resources:
        requests:                         # used by the scheduler (guaranteed share)
          cpu: 250m
          memory: 256Mi
        limits:                           # hard ceiling
          memory: 256Mi
      startupProbe:                       # protects slow starters
        httpGet:
          path: /healthz
          port: http
        failureThreshold: 30
        periodSeconds: 2
      readinessProbe:                     # in or out of Service endpoints
        httpGet:
          path: /ready
          port: http
        periodSeconds: 5
      livenessProbe:                      # restart if stuck
        httpGet:
          path: /healthz
          port: http
        periodSeconds: 10
        failureThreshold: 3
      lifecycle:
        preStop:                          # give load balancers time to drain
          exec:
            command: ["sleep", "5"]
      securityContext:                    # container-level security
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
```

```
        capabilities:
          drop: ["ALL"]
      volumeMounts:
        - name: tmp
          mountPath: /tmp
  volumes:
    - name: tmp
      emptyDir: {}
```

## Pod lifecycle

| Phase | Meaning |
|---|---|
| `Pending` | Accepted, but not running yet: waiting for scheduling, image pull, or volumes |
| `Running` | Bound to a node, at least one container running |
| `Succeeded` | All containers exited 0 (Jobs) |
| `Failed` | All containers terminated, at least one non-zero |
| `Unknown` | Node not reporting (network or node failure) |

Container **states**: `Waiting` (with a reason such as `ContainerCreating`, `ImagePullBackOff`, `CrashLoopBackOff`), `Running`, `Terminated` (with exit code and reason such as `OOMKilled`, `Error`, `Completed`).

**Restart policy** (`Always` default, `OnFailure`, `Never`): the kubelet restarts failed containers in place with exponential back-off (10s, 20s, 40s … capped at 5 minutes). That back-off is what `CrashLoopBackOff` means.

**Conditions** (`kubectl describe`): `PodScheduled`, `Initialized`, `ContainersReady`, `Ready`. Only `Ready` pods receive Service traffic.

## Probes

| Probe | Question | On failure | Typical use |
|---|---|---|---|
| **startupProbe** | Has the app finished starting? | Restart after `failureThreshold × periodSeconds` | Slow-starting apps (JVM); other probes wait until it succeeds |
| **readinessProbe** | Can it take traffic right now? | Removed from Service endpoints (no restart) | Warming caches, dependency briefly down, overload |
| **livenessProbe** | Is it alive or deadlocked? | Container **restarted** | Recover from hangs |

Probe types: `httpGet` (2xx/3xx = success), `tcpSocket`, `exec` (exit 0), `grpc`. Settings: `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `successThreshold`, `failureThreshold`.

**Senior rules:**

- Liveness must check **only the process itself**, never databases or other services. Otherwise one DB outage restarts every pod and turns a partial outage into a total one.
- Readiness may check critical dependencies, but carefully: if all pods go unready at once, the Service has no endpoints.
- Use a `startupProbe` instead of a large `initialDelaySeconds`.
- No liveness probe is better than a wrong one.

## Graceful termination

```
kubectl delete / rollout / drain
  │
  ├─► pod marked Terminating ──► removed from EndpointSlices (in parallel, takes a moment)
  ├─► preStop hook runs (e.g. sleep 5 so load balancers stop sending traffic)
  ├─► SIGTERM sent to PID 1 of each container ──► app stops accepting, finishes in-flight requests
  └─► after terminationGracePeriodSeconds (default 30s) ──► SIGKILL
```

The app must handle **SIGTERM** (and run as PID 1 correctly: use exec-form `CMD` / `ENTRYPOINT` or an init like `tini`). Without that you get dropped requests on every deployment.

## Multi-container patterns

| Pattern | Example |
|---|---|
| **Init container** | Wait for dependencies, run migrations, fetch config, fix permissions |
| **Sidecar** | Log shipper, service-mesh proxy (Envoy), secrets agent. Native sidecars: an init container with `restartPolicy: Always` starts before and stops after the app (stable since Kubernetes 1.33) |
| Ambassador | Local proxy to an external service (e.g. cloud SQL proxy) |
| Adapter | Converts the app's output (metrics format, logs) for a standard system |

## Resources and QoS

| Term | Meaning |
|---|---|
| CPU units | `1` = one vCPU/core; `250m` = 0.25 core (millicores) |
| Memory units | `Mi` / `Gi` (powers of 2: 1Mi = 1,048,576 bytes); `M` / `G` are powers of 10 |
| **requests** | Reserved amount; the scheduler places pods only where requests fit |
| **limits** | Ceiling. CPU over the limit is **throttled**; memory over the limit is **OOMKilled** (exit code 137) |

| QoS class | Rule | Evicted under node memory pressure |
|---|---|---|
| **Guaranteed** | Every container has requests = limits for CPU and memory | Last |
| **Burstable** | At least one request or limit set, not Guaranteed | Middle |
| **BestEffort** | No requests or limits at all | First |

**Production practice:** always set CPU and memory **requests**; set a memory **limit** (commonly equal to the request for predictability); many teams **omit CPU limits** for latency-sensitive services because throttling causes latency spikes, while requests still guarantee a fair share. Right-size from real usage (`kubectl top`, Prometheus, VPA recommendations).

## Security context essentials

| Setting | Effect |
|---|---|
| `runAsNonRoot: true`, `runAsUser: 10001` | Refuse to run as root |
| `allowPrivilegeEscalation: false` | Block setuid/sudo-style escalation |

| Setting | Effect |
|---|---|
| `readOnlyRootFilesystem: true` | Image filesystem cannot be modified (mount `emptyDir` for `/tmp`) |
| `capabilities: drop: ["ALL"]` | Remove Linux capabilities (add back only what is needed, e.g. `NET_BIND_SERVICE`) |
| `seccompProfile: RuntimeDefault` | Restrict system calls |
| `privileged: true`, `hostNetwork`, `hostPID`, `hostPath` | Effectively node access: forbid for applications |

These match the Pod Security Standard **restricted** profile (12.12).

### Debugging a pod

```
kubectl describe pod api-7d9f-x2k              # events: scheduling, pulls, probe failures, OOM
kubectl logs api-7d9f-x2k -c app --previous    # logs of the crashed container
kubectl get pod api-7d9f-x2k -o jsonpath='{.status.containerStatuses[0].lastState}'
kubectl exec -it api-7d9f-x2k -- sh            # only if the image has a shell
kubectl debug -it api-7d9f-x2k --image=busybox:1.36 --target=app   # ephemeral container
kubectl debug node/worker-1 -it --image=busybox:1.36               # shell on a node (host at /host)
```

## 12.6 Workload controllers

| Controller | Use for | Key behavior |
|---|---|---|
| **Deployment** | Stateless apps (APIs, web) | Manages ReplicaSets; rolling updates and rollbacks; pods are interchangeable |
| ReplicaSet | (Used by Deployments) | Keeps N pods matching a selector; don't use directly |
| **StatefulSet** | Databases, Kafka, ZooKeeper, Elasticsearch | Stable names (`db-0`, `db-1`), stable per-pod storage, ordered start/stop |
| **DaemonSet** | One pod per node: log agents, monitoring agents, CNI, CSI node plugins | New nodes get the pod automatically |
| **Job** | Run to completion: migrations, batch processing | Retries (`backoffLimit`), parallelism, completions |
| **CronJob** | Scheduled Jobs: backups, reports | Cron schedule, concurrency policy, history limits |

### Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: shop
spec:
  replicas: 3
  revisionHistoryLimit: 5                # old ReplicaSets kept for rollback
  selector:
    matchLabels:                         # immutable after creation; must match template labels
      app.kubernetes.io/name: api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%                      # extra pods allowed above replicas during rollout
      maxUnavailable: 0                  # never drop below desired capacity
  minReadySeconds: 10                    # pod must stay Ready this long to count as available
  progressDeadlineSeconds: 600           # mark rollout failed after 10 minutes without progress
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
    spec:
      containers:
        - name: app
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
```

**How a rolling update works:** changing anything in `spec.template` (image, env, resources) creates a **new ReplicaSet**. The controller scales the new one up and the old one down, step by step, respecting `maxSurge` / `maxUnavailable` and waiting for readiness. Changing only `replicas` does not create a new ReplicaSet.

```
kubectl set image deploy/api app=registry/shop/api:1.4.3 -n shop    # (or change YAML in Git)
kubectl rollout status deploy/api -n shop                           # waits; non-zero on failure
kubectl rollout history deploy/api -n shop
kubectl rollout undo deploy/api -n shop [--to-revision=3]
kubectl rollout restart deploy/api -n shop                          # re-create pods (new config)
kubectl rollout pause / resume deploy/api -n shop
```

`strategy.type: Recreate` stops all old pods before starting new ones (downtime; only for apps that cannot run two versions at once). Canary and blue/green need Argo Rollouts, Flagger, or a service mesh.

**Note:** pods do **not** restart when a ConfigMap or Secret they use changes. Use a checksum annotation in the template (Helm pattern), `kubectl rollout restart`, Kustomize's hashed generator names, or a reloader controller.

### StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
  namespace: shop
spec:
  serviceName: db                        # headless Service giving stable DNS names
  replicas: 3
  podManagementPolicy: OrderedReady      # db-0, then db-1, then db-2 (Parallel also possible)
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: db-credentials
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:                  # one PVC per pod: data-db-0, data-db-1, data-db-2
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: gp3
        resources:
          requests:
            storage: 20Gi
```

Guarantees: pod names `db-0..db-2`, DNS `db-0.db.shop.svc.cluster.local`, each pod re-attaches **its own** PVC after rescheduling, ordered scale-down from the highest ordinal. PVCs are **kept** when the StatefulSet is deleted (data safety; configurable with `persistentVolumeClaimRetentionPolicy`).

**Senior advice:** running databases on Kubernetes is possible (use an **operator**: CloudNativePG, Strimzi for Kafka, ECK for Elasticsearch) but managed services (RDS, Aurora, MSK) are usually the better choice on AWS: backups, patching and failover are handled for you.

## DaemonSet

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-agent
  updateStrategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: node-agent
    spec:
      tolerations:                      # also run on tainted nodes (e.g. control plane)
        - operator: Exists
      containers:
        - name: agent
          image: prom/node-exporter:v1.8.2
          ports:
            - containerPort: 9100
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 128Mi
```

### Job and CronJob

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate-1-4-2
  namespace: shop
spec:
  backoffLimit: 3                        # retries before the Job is Failed
  activeDeadlineSeconds: 600             # hard timeout
  ttlSecondsAfterFinished: 3600          # auto-delete one hour after finishing
  template:
    spec:
      restartPolicy: Never               # Jobs need Never or OnFailure
      containers:
        - name: migrate
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          command: ["python", "-m", "app.migrate"]
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
  namespace: shop
spec:
  schedule: "30 1 * * *"                 # 01:30 every day
  timeZone: Asia/Kolkata                 # without it, the controller's time zone (usually UTC)
  concurrencyPolicy: Forbid              # skip a run if the previous one is still running
  startingDeadlineSeconds: 300
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/report:2.0.1
```

Parallel Jobs: `completions: 10` + `parallelism: 3` runs 10 successful pods, 3 at a time; `completionMode: Indexed` gives each pod an index (`JOB_COMPLETION_INDEX`) to process a shard. Jobs must be **idempotent**: a pod can run twice after a node failure.

### Choosing a controller

```
Does it run forever?
├── No → once: Job · on a schedule: CronJob
└── Yes → one copy on every node? → DaemonSet
          └── needs stable identity or its own disk per replica? → StatefulSet (or operator)
               └── otherwise → Deployment
```

## 12.7 Services and networking

### The Kubernetes network model

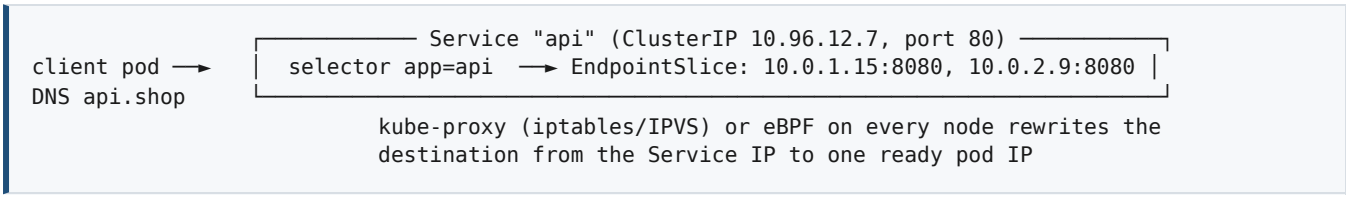
1. Every pod gets its **own IP**.
2. Pods can reach all other pods **without NAT**, across nodes.
3. Agents on a node (kubelet) can reach all pods on that node.
4. Pod IPs are **ephemeral**: a replaced pod gets a new IP. A **Service** gives a stable name and IP.

The **CNI plugin** implements this:

| CNI | How it works | Notes |
|---|---|---|
| **AWS VPC CNI** (EKS default) | Pods get real VPC IPs from node ENIs | Native VPC routing and security groups; pod count per node limited by ENIs/IPs (use prefix delegation); plan subnet sizes |
| **Calico** | BGP routing or overlay (VXLAN/IP-in-IP) | Strong NetworkPolicy support |
| **Cilium** | eBPF dataplane | Can replace kube-proxy; L7 policy; Hubble observability |
| Flannel | Simple VXLAN overlay | No NetworkPolicy by itself |

Three separate IP ranges must never overlap: **node/VPC CIDR**, **pod CIDR** (overlay CNIs), **Service CIDR** (e.g. `10.96.0.0/12`; EKS default `10.100.0.0/16` or `172.20.0.0/16`).

## Service types




| Type | Reachable from | How |
|---|---|---|
| **ClusterIP** (default) | Inside the cluster | Virtual IP + DNS name |
| **NodePort** | `<any node IP>:30000-32767` | ClusterIP plus a port opened on every node |
| **LoadBalancer** | Internet or VPC | NodePort/ClusterIP plus a cloud load balancer (on EKS: NLB via AWS Load Balancer Controller) |
| ExternalName | Inside the cluster | DNS CNAME to an external name (e.g. an RDS endpoint) |
| **Headless** (`clusterIP: None`) | Inside the cluster | No virtual IP; DNS returns the pod IPs directly (StatefulSets, client-side load balancing) |

```
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: shop
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: api          # sends traffic to READY pods with this label
  ports:
    - name: http
      port: 80                           # Service port (what clients use)
      targetPort: http                   # container port by name or number
      protocol: TCP
```

```
# Internet-facing NLB on EKS (AWS Load Balancer Controller installed)
apiVersion: v1
kind: Service
metadata:
  name: api-public
  namespace: shop
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
spec:
  type: LoadBalancer
  loadBalancerClass: service.k8s.aws/nlb
  selector:
    app.kubernetes.io/name: api
  ports:
    - port: 443
      targetPort: 8080
```

Other fields: `sessionAffinity: ClientIP` (sticky by client IP), `externalTrafficPolicy: Local` (keep the client source IP, only nodes with local pods receive traffic), `internalTrafficPolicy`, `trafficDistribution: PreferClose` (prefer same-zone endpoints, saving cross-AZ cost).

**EndpointSlices** list the ready pod IPs and ports behind a Service. `kubectl get endpointslices -l kubernetes.io/service-name=api -n shop`. **Empty endpoints = the selector matches no ready pod**: the most common Service problem.

**kube-proxy modes:** `iptables` (default; rules per Service/endpoint), `ipvs` (kernel load balancer, better for very many Services), `nftables` (newer Linux backend). eBPF CNIs (Cilium) can replace kube-proxy entirely.

## DNS (CoreDNS)

| Name | Resolves to |
|---|---|
| `api` | Service `api` in the **same** namespace |
| `api.shop` | Service `api` in namespace `shop` |
| `api.shop.svc.cluster.local` | Fully qualified Service name |
| `db-0.db.shop.svc.cluster.local` | One StatefulSet pod through its headless Service |

Pods get `/etc/resolv.conf` with `nameserver <CoreDNS ClusterIP>`, search domains (`shop.svc.cluster.local svc.cluster.local cluster.local`) and `ndots:5`. With `ndots:5`, a name like `api.example.com` (fewer than 5 dots) is first tried with every search suffix: extra lookups. For external names in hot paths, use a trailing dot (`api.example.com.`) or lower `ndots` via `dnsConfig`. **NodeLocal DNSCache** reduces latency and CoreDNS load in large clusters.

```
kubectl run -it --rm dnstest --image=busybox:1.36 --restart=Never -- nslookup api.shop
kubectl -n kube-system logs deploy/coredns
```

## Ingress

An **Ingress** is an HTTP(S) routing rule (host and path → Service). It does nothing without an **ingress controller**: ingress-nginx, Traefik, HAProxy, **AWS Load Balancer Controller** (creates an ALB), Istio, Kong.

```
Internet ──► Load balancer ──► Ingress controller ─┬─► shop.example.com/api ──► Service api ──► pods
                                                   └─► shop.example.com/    ──► Service web ──► pods
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'
    alb.ingress.kubernetes.io/certificate-arn: >-
      arn:aws:acm:ap-south-1:123456789012:certificate/1111-2222
spec:
  ingressClassName: alb                  # which controller handles it
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

With ingress-nginx, TLS comes from a Secret (`spec.tls` with `secretName`) usually issued by **cert-manager** (Let's Encrypt); with the ALB controller, from ACM. **ExternalDNS** creates Route 53 records from Ingress/Service hostnames.

Note: the community **ingress-nginx** project announced its retirement (maintenance ends March 2026). New platforms choose the **Gateway API** with a maintained implementation, or a supported controller.

## Gateway API (successor to Ingress)

Role-oriented and more expressive (header matching, traffic splitting, TCP/UDP/gRPC routes, cross-namespace delegation):

| Resource | Owned by | Purpose |
|---|---|---|
| `GatewayClass` | Infrastructure provider | Which implementation (Envoy Gateway, Cilium, Istio, AWS Gateway API Controller, NGINX Gateway Fabric) |
| `Gateway` | Platform team | Listeners: ports, protocols, TLS, which namespaces may attach routes |
| `HTTPRoute` / `GRPCRoute` / `TLSRoute` | App teams | Routing rules to Services |

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api
  namespace: shop
spec:
  parentRefs:
    - name: public-gateway
      namespace: gateway-system
  hostnames: ["shop.example.com"]
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:                        # 90/10 canary split
        - name: api-stable
          port: 80
          weight: 90
        - name: api-canary
          port: 80
          weight: 10
```

## NetworkPolicy

By default **every pod can talk to every pod** in the cluster. NetworkPolicies are pod-level firewalls, enforced by the CNI (Calico, Cilium, AWS VPC CNI with network policy enabled; Flannel alone does **not** enforce them).

Rules:

- Policies are **additive allow lists**. Once any policy selects a pod for a direction (Ingress/Egress), everything not allowed in that direction is denied.
- `podSelector: {}` selects all pods in the namespace.
- Selectors in `from` / `to`: `podSelector`, `namespaceSelector`, both in one item (AND), or `ipBlock`.

```
# 1. Default deny all traffic in the namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
# 2. Allow DNS for every pod (otherwise nothing can resolve names)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Egress"]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
---
# 3. Only the ingress controller may reach the API pods on 8080
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-from-ingress
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress
      ports:
        - protocol: TCP
          port: 8080
---
# 4. API pods may reach the database (port 5432) only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-to-db
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: ["Egress"]
  egress:
    - to:
```

```
        - ipBlock:
            cidr: 10.0.20.0/24          # database subnet (RDS)
      ports:
        - protocol: TCP
          port: 5432
```

Common mistake: a namespace-and-pod selector written as **two list items** (`- namespaceSelector` and `- podSelector`) means **OR** (much wider) instead of AND.

### Service mesh (when you need it)

A mesh (Istio, Linkerd, Cilium service mesh) adds, without app changes: **mutual TLS** between services, retries/timeouts/circuit breaking, fine-grained traffic splitting, and per-request metrics/tracing. Cost: more components, more latency and operational complexity. Adopt it when you have many services and a concrete need (zero-trust mTLS, advanced traffic control), not by default. Istio **ambient mode** removes per-pod sidecars.

## 12.8 Configuration and secrets

### ConfigMap

Non-secret configuration, kept out of the image so the **same image** runs in every environment.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
  namespace: shop
data:
  LOG_LEVEL: info
  FEATURE_CHECKOUT_V2: "true"           # values are always strings: quote booleans/numbers
  app.yaml: |
    cache:
      ttl_seconds: 300
```

```
# Using it in a container (fragment of a pod spec)
containers:
  - name: app
    image: shop/api:1.4.2
    envFrom:
      - configMapRef:
          name: api-config              # every key becomes an env var
    env:
      - name: LOG_LEVEL                 # or pick single keys
        valueFrom:
          configMapKeyRef:
            name: api-config
            key: LOG_LEVEL
    volumeMounts:
      - name: config
        mountPath: /etc/api             # each key becomes a file: /etc/api/app.yaml
        readOnly: true
volumes:
  - name: config
    configMap:
      name: api-config
      items:
        - key: app.yaml
          path: app.yaml
```

| Consumed as | Updates when the ConfigMap changes? |
|---|---|
| Environment variables | **No**: only at container start (restart needed) |
| Mounted volume | Yes, eventually (kubelet sync, up to about a minute); the app must re-read the file. Not with `subPath` mounts |

`immutable: true` on a ConfigMap/Secret prevents accidental changes and reduces API server load. Limit: 1 MiB per object.

## Secret

```
kubectl create secret generic db-credentials -n shop \
  --from-literal=DB_USER=shop --from-literal=DB_PASSWORD="$(openssl rand -base64 24)"
kubectl create secret tls shop-tls --cert=tls.crt --key=tls.key -n shop
kubectl create secret docker-registry regcred --docker-server=REGISTRY \
  --docker-username=USER --docker-password="$TOKEN" -n shop
```

| Fact | Consequence |
|---|---|
| Values are **base64-encoded, not encrypted** | Anyone who can read the Secret (RBAC `get` / `list` secrets) sees the value. `base64 -d` is not security |
| Stored in etcd | Enable **encryption at rest** (EKS: KMS envelope encryption is on by default for new clusters since 2024) |
| Never commit Secret YAML to Git | Use one of the patterns below |
| Mounted secrets are tmpfs (in memory) | Not written to node disk |
| Types | `Opaque`, `kubernetes.io/tls`, `kubernetes.io/dockerconfigjson`, `kubernetes.io/service-account-token` |

Prefer mounting secrets as **files** over environment variables: env vars leak more easily (crash dumps, `env` in debug sessions, child processes).

## Production secret management patterns

| Pattern | How it works | Good for |
|---|---|---|
| **External Secrets Operator (ESO)** | `ExternalSecret` CR references a key in AWS Secrets Manager / SSM Parameter Store / Vault; ESO creates and refreshes the Kubernetes Secret | Most cloud teams; GitOps-friendly (only the reference is in Git) |
| Secrets Store CSI Driver | Mounts secrets from the provider directly as files (optionally syncs to a Secret) | Avoid storing in etcd |
| **Sealed Secrets** | Encrypt with the cluster's public key (`kubeseal`); only the in-cluster controller can decrypt; the encrypted YAML is safe in Git | Small teams, no external store |
| SOPS (with KMS/age) | Encrypt values in files; decrypted by Argo CD/Flux plugins or CI | GitOps with existing KMS |
| Vault Agent / Vault Secrets Operator | Dynamic, short-lived credentials (e.g. per-pod DB users) | Strict security requirements |

```
# External Secrets Operator: pull "prod/shop/db" from AWS Secrets Manager
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: db-credentials
  namespace: shop
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager           # a ClusterSecretStore using IRSA/Pod Identity
    kind: ClusterSecretStore
  target:
    name: db-credentials                # Kubernetes Secret ESO creates
  data:
    - secretKey: DB_PASSWORD
      remoteRef:
        key: prod/shop/db
        property: password
```

## 12.9 Storage

Container filesystems are **ephemeral**: when a container restarts, written files are gone.

### Volume types

| Volume | Lifetime | Use |
|---|---|---|
| `emptyDir` | Pod lifetime ( `medium: Memory` for tmpfs) | Scratch space, sharing files between containers, `/tmp` with read-only root |
| `configMap` / `secret` / `projected` / `downwardAPI` | Pod | Config files, tokens, pod metadata |
| `hostPath` | Node | Node agents only; a security risk for apps (forbidden by Pod Security baseline) |
| **persistentVolumeClaim** | Independent of pods | Databases, uploads, anything that must survive |
| Ephemeral CSI / generic ephemeral volumes | Pod | Per-pod scratch disks from a StorageClass |

### PV, PVC and StorageClass

```
Developer: PersistentVolumeClaim "I need 20Gi, ReadWriteOnce, class gp3"
   │
   ▼  dynamic provisioning
StorageClass gp3 (provisioner ebs.csi.aws.com, parameters type=gp3, encrypted=true)
   │
   ▼  CSI driver calls the cloud API
PersistentVolume (an actual 20Gi EBS volume) ── bound 1:1 ── PVC ── mounted by a pod
```

| Object | Who creates it | What it is |
|---|---|---|
| **StorageClass** | Platform team | A "type" of storage: provisioner + parameters + policies |
| **PersistentVolumeClaim (PVC)** | App team (namespaced) | A request for storage |

| Object | Who creates it | What it is |
|---|---|---|
| **PersistentVolume (PV)** | Usually auto-created by the CSI driver (cluster-scoped) | The real disk/share |

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
reclaimPolicy: Delete                    # Retain for important data
volumeBindingMode: WaitForFirstConsumer  # create the disk in the AZ where the pod is scheduled
allowVolumeExpansion: true
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: uploads
  namespace: shop
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: gp3
  resources:
    requests:
      storage: 20Gi
```

## Access modes and reclaim policies

| Access mode | Meaning | Examples |
|---|---|---|
| **ReadWriteOnce (RWO)** | Read-write by pods on **one node** | AWS EBS, Azure Disk, GCE PD |
| ReadWriteOncePod | Read-write by **one pod** only | CSI volumes that need strict single writer |
| ReadOnlyMany (ROX) | Read-only by many nodes | Shared static content |
| **ReadWriteMany (RWX)** | Read-write by many nodes | AWS **EFS**, NFS, CephFS, FSx |

| Reclaim policy | When the PVC is deleted |
|---|---|
| `Delete` | The PV and the cloud disk are deleted (default for dynamic provisioning) |
| `Retain` | PV and disk are kept for manual recovery; use for production data |

## AWS storage on EKS

| Driver | Backing | Notes |
|---|---|---|
| **EBS CSI driver** (EKS add-on) | EBS volumes (gp3, io2) | RWO; a volume lives in **one AZ**, so its pod can only run in that AZ (use `WaitForFirstConsumer`; spread StatefulSet replicas across AZs) |
| **EFS CSI driver** | EFS file systems | RWX across AZs; good for shared files, slower per operation than EBS |
| FSx for Lustre / NetApp ONTAP CSI | FSx | High-performance or enterprise NAS needs |

| Driver | Backing | Notes |
|---|---|---|
| Mountpoint for S3 CSI | S3 bucket as a file system | Read-heavy large datasets (ML) |

### Operations

- **Expansion:** with `allowVolumeExpansion: true`, edit the PVC's `storage` request upwards; the CSI driver grows the disk and file system (shrinking is not supported).
- **Snapshots:** `VolumeSnapshotClass` + `VolumeSnapshot` (CSI snapshot controller); restore by creating a PVC with `dataSource` pointing at the snapshot.
- **Backups:** Velero (12.15) or AWS Backup for EBS/EFS.
- **Stuck PVC `Pending`:** no default StorageClass, wrong class name, CSI driver not installed or lacking IAM permissions, `WaitForFirstConsumer` (normal until a pod uses it), quota exceeded.
- **Pod stuck `ContainerCreating` with "Multi-Attach error":** an RWO volume is still attached to another node (old pod not yet gone); usually resolves after the old node detaches it.

## 12.10 Scheduling and availability

The **scheduler** places each pending pod in two phases: **filtering** (which nodes *can* run it: enough allocatable CPU/memory for the *requests*, node selectors, affinity, taints, volume zone, ports) then **scoring** (which feasible node is *best*: spreading, balanced usage, image locality). No feasible node = pod stays `Pending` with a `FailedScheduling` event explaining why.

### Controls

| Tool | Use |
|---|---|
| `nodeSelector` | Simple: only nodes with these labels ( `kubernetes.io/arch: arm64` ) |
| **Node affinity** | Expressive rules; `required...` (hard) or `preferred...` (soft) |
| **Pod affinity / anti-affinity** | Place near / away from other pods (e.g. no two replicas on one node) |
| **Taints and tolerations** | Nodes repel pods unless the pod tolerates the taint (dedicated GPU nodes, spot nodes, control plane) |
| **Topology spread constraints** | Spread replicas evenly across zones and nodes |
| PriorityClass | Important pods schedule first and can **preempt** (evict) lower-priority pods |

```
# Fragment of a Deployment pod template: spread across AZs and nodes, prefer on-demand nodes
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule   # hard rule for zones
      labelSelector:
        matchLabels:
          app.kubernetes.io/name: api
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: ScheduleAnyway  # soft rule for nodes
      labelSelector:
        matchLabels:
          app.kubernetes.io/name: api
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 50
          preference:
            matchExpressions:
              - key: karpenter.sh/capacity-type
                operator: In
                values: ["on-demand"]
  tolerations:
    - key: dedicated
      operator: Equal
      value: shop
      effect: NoSchedule
```

Taint effects: `NoSchedule` (new pods without the toleration are not placed), `PreferNoSchedule` (avoid if possible), `NoExecute` (also evicts running pods without the toleration). Example: `kubectl taint nodes gpu-1 dedicated=ml:NoSchedule`; remove with a trailing `-`. Note: a toleration only **allows** a pod on a tainted node; to **force** it there, combine it with node affinity.

## PodDisruptionBudget (PDB)

Limits **voluntary** disruptions (drain, upgrades, autoscaler scale-down), not crashes.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api
  namespace: shop
spec:
  minAvailable: 2                       # or maxUnavailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
```

A PDB that allows zero disruptions (e.g. `minAvailable: 1` with 1 replica) **blocks node drains and cluster upgrades** forever. Run at least 2 replicas when you add a PDB.

## Node maintenance

```
kubectl cordon worker-2                                  # no new pods
kubectl drain worker-2 --ignore-daemonsets --delete-emptydir-data --timeout=10m
#   evicts pods respecting PDBs and graceful termination
# ... patch / replace the node ...
kubectl uncordon worker-2
```

### High availability checklist for one service

- ≥ 2 replicas (3 for critical services), spread across **zones** and nodes.
- Readiness probes, graceful shutdown (SIGTERM + `preStop`), `maxUnavailable: 0` rollouts.
- A PDB.
- Requests set so the scheduler and autoscalers can do their job.
- No single-AZ dependency (EBS volume, single NAT gateway) hiding underneath.

## 12.11 Autoscaling

```
            traffic grows
                 │
HPA ──► more pod replicas ──► pods Pending (no room) ──► Cluster Autoscaler / Karpenter
                                                              ──► new nodes
VPA ──► right-sizes each pod's requests
KEDA ──► scales on events (queue length, Kafka lag, cron), including to zero
```

### Horizontal Pod Autoscaler (HPA)

Needs **metrics-server** for CPU/memory (custom metrics need an adapter such as Prometheus Adapter or KEDA). Formula: `desired = ceil(current × currentMetric / target)`.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70         # % of the CPU *request*
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300    # avoid flapping
      policies:
        - type: Percent
          value: 50
          periodSeconds: 60
```

Rules: CPU utilization is relative to **requests** (no requests = HPA cannot work). Remove `spec.replicas` from the Deployment in Git once an HPA owns it, otherwise every deploy resets the count (or have GitOps ignore that field).

### VPA, Cluster Autoscaler, Karpenter, KEDA

| Tool | Scales | Notes |
|---|---|---|
| **VPA** | Pod requests/limits | Modes `Off` (recommendations only: safest and very useful), `Initial`, `Recreate` / `InPlaceOrRecreate`. Don't let VPA and HPA both act on CPU/memory of the same workload |

| Tool | Scales | Notes |
|---|---|---|
| **Cluster Autoscaler** | Node groups (ASGs) | Adds nodes when pods are Pending; removes underused nodes; works per node group with fixed instance types |
| **Karpenter** | Nodes directly (EC2 Fleet) | Picks instance types per pending pods, mixes spot/on-demand, consolidates to cheaper nodes, fast; configured with `NodePool` and `EC2NodeClass`. The common choice on EKS (EKS Auto Mode runs it for you) |
| **KEDA** | Deployments/Jobs on event sources | SQS queue depth, Kafka lag, Prometheus queries, cron; scale to zero |

```
# Karpenter NodePool (v1 API): spot + on-demand, amd64/arm64, consolidation
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: general
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64", "arm64"]
      expireAfter: 720h                 # replace nodes every 30 days (fresh AMIs)
  limits:
    cpu: "200"                          # cost guard: never more than 200 vCPUs
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 1m
```

## 12.12 Security

### Authentication and authorization

```
request ──► Authentication (who?) ──► Authorization (RBAC) ──► Admission (mutate+validate) ──► etcd
            certs, OIDC, cloud IAM      Role bindings              PSA, Kyverno, webhooks
```

Kubernetes has no user database: **humans** authenticate via certificates, OIDC (SSO) or cloud IAM (EKS access entries); **workloads** use **ServiceAccounts** (short-lived projected tokens).

### RBAC

| Object | Scope | Contains |
|---|---|---|
| **Role** | One namespace | Rules: apiGroups + resources + verbs |
| **ClusterRole** | Cluster-wide (or reusable in namespaces) | Same, plus cluster-scoped resources (nodes, PVs) |
| **RoleBinding** | One namespace | Grants a Role or ClusterRole to subjects in that namespace |

| Object | Scope | Contains |
|---|---|---|
| ClusterRoleBinding | Whole cluster | Grants a ClusterRole everywhere: use sparingly |

Verbs: `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`, `deletecollection`, plus special ones (`impersonate`, `escalate`, `bind`). Built-in ClusterRoles: `view`, `edit`, `admin`, `cluster-admin`.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: deployer
  namespace: shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "patch", "update"]
  - apiGroups: [""]                       # "" = core API group
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: deployer
  namespace: shop
subjects:
  - kind: ServiceAccount
    name: deployer
    namespace: shop
  - kind: Group                           # e.g. an SSO group mapped via EKS access entries
    name: shop-developers
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io
```

```
kubectl auth can-i delete pods -n shop --as system:serviceaccount:shop:deployer
kubectl auth can-i --list -n shop --as alice
```

**Dangerous permissions** (treat as cluster-admin-equivalent): `secrets` get/list, `pods/exec`, creating pods in privileged namespaces, `escalate` / `bind` / `impersonate`, writing RBAC objects, `nodes/proxy`, wildcard `*` rules.

## Pod Security Admission (PSA)

Built-in admission controller enforcing the **Pod Security Standards** per namespace via labels:

| Level | Allows |
|---|---|
| `privileged` | Everything (system namespaces only) |
| `baseline` | Blocks known escalations: privileged, hostNetwork/hostPID, hostPath, added dangerous capabilities |

| Level | Allows |
|---|---|
| `restricted` | Baseline + must run as non-root, drop ALL capabilities, no privilege escalation, seccomp RuntimeDefault |

```
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/audit: restricted
```

## Policy engines and admission control

| Tool | Language | Typical rules |
|---|---|---|
| **ValidatingAdmissionPolicy** (built-in, GA 1.30) | CEL expressions | Simple validations without webhooks |
| **Kyverno** | YAML policies | Require labels/requests/probes, disallow `:latest`, only allowed registries, verify image signatures, **mutate** (add defaults) and **generate** (default NetworkPolicy per namespace) |
| OPA **Gatekeeper** | Rego | Same goals, powerful for complex logic |

```
# Kyverno: images must come from our ECR and must not use ":latest"
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: trusted-images
spec:
  validationFailureAction: Enforce
  rules:
    - name: only-our-registry
      match:
        any:
          - resources:
              kinds: ["Pod"]
      validate:
        message: "Images must come from 123456789012.dkr.ecr.ap-south-1.amazonaws.com"
        pattern:
          spec:
            containers:
              - image: "123456789012.dkr.ecr.ap-south-1.amazonaws.com/*"
    - name: no-latest-tag
      match:
        any:
          - resources:
              kinds: ["Pod"]
      validate:
        message: "Use an immutable tag, not :latest"
        pattern:
          spec:
            containers:
              - image: "!*:latest"
```

Roll out policies in **audit** mode first, fix violations, then enforce.

### Cluster security checklist (senior view)

| Area | Controls |
|---|---|
| API access | Private or IP-restricted endpoint, SSO/IAM identities, no shared kubeconfigs, short-lived credentials |
| RBAC | Least privilege per team/namespace, no `cluster-admin` for humans day to day, audit bindings regularly |
| Workloads | PSA `restricted`, non-root, read-only root FS, no privileged/host access, resource limits |
| Network | Default-deny NetworkPolicies, private nodes, security groups, egress control |
| Supply chain | Trusted registries, image scanning, signed images (cosign) verified by admission (Kyverno/Sigstore policy-controller), SBOMs |
| Secrets | External secret store, encryption at rest, no secrets in Git or env dumps |
| Workload identity | IRSA / EKS Pod Identity per ServiceAccount; block pod access to the node's instance metadata (IMDSv2 hop limit 1) |
| Detection | API **audit logs** (EKS control plane logging), runtime detection (**Falco**, GuardDuty EKS runtime monitoring) |
| Benchmarks | **kube-bench** (CIS Kubernetes Benchmark), Kubescape/Trivy cluster scans |
| Lifecycle | Upgrade Kubernetes and nodes regularly (patches), minimal node OS (Bottlerocket) |

## 12.13 Packaging: Helm and Kustomize

Raw YAML repeats itself across environments. Two standard tools solve that:

| | Helm | Kustomize |
|---|---|---|
| Approach | Templates (Go templates) + values files → YAML | Plain YAML **base** + **overlays** with patches (no templating) |
| Packaging | Charts, versioned, published to repositories (OCI registries) | Directories in Git |
| Release tracking | Yes: install/upgrade/rollback history (stored as Secrets) | No: just `kubectl apply -k` |
| Best for | Distributing software (third-party apps), complex parameterization | Your own apps, environment differences |
| Built in | Separate CLI | `kubectl apply -k` / `kustomize build` |

Many teams use **both**: Helm for third-party components (ingress controllers, Prometheus, cert-manager) and Kustomize or a small internal chart for their own services, all deployed by Argo CD.

## Helm

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm search repo kube-prometheus-stack --versions | head
helm show values prometheus-community/kube-prometheus-stack > default-values.yaml
helm upgrade --install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace --version 65.1.0 -f values-prod.yaml \
  --atomic --timeout 10m                # roll back automatically if the upgrade fails
helm list -A
helm history monitoring -n monitoring
helm rollback monitoring 3 -n monitoring
helm uninstall monitoring -n monitoring
helm template ./charts/api -f values-dev.yaml   # render locally (CI checks, GitOps)
helm lint ./charts/api
helm install api oci://123456789012.dkr.ecr.ap-south-1.amazonaws.com/charts/api --version 1.2.0
```

Always **pin chart versions** ( `--version` ) and keep values files in Git.

Chart structure:

```
charts/api/
├── Chart.yaml            # name, version (chart), appVersion (app), dependencies
├── values.yaml           # defaults
├── values.schema.json    # optional: validates values
├── templates/
│   ├── _helpers.tpl      # named templates (labels, names)
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── hpa.yaml
│   └── NOTES.txt         # printed after install
└── charts/               # dependency charts
```

```
# Chart.yaml
apiVersion: v2
name: api
description: Shop API
type: application
version: 1.2.0                        # chart version: bump on every chart change
appVersion: "1.4.2"                   # default image tag
```

```
# values.yaml
replicaCount: 2
image:
  repository: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api
  tag: ""                             # empty = use appVersion
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    memory: 256Mi
autoscaling:
  enabled: false
  maxReplicas: 10
```

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "api.fullname" . }}
  labels:
    {{- include "api.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "api.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "api.selectorLabels" . | nindent 8 }}
      annotations:                      # restart pods when the ConfigMap changes
        checksum/config: >-
          {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
        - name: app
          {{- $tag := .Values.image.tag | default .Chart.AppVersion }}
          image: "{{ .Values.image.repository }}:{{ $tag }}"
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

Template essentials: `.Values`, `.Release.Name`, `.Release.Namespace`, `.Chart.Version`; functions `default`, `quote`, `toYaml`, `nindent`, `required "msg"`, `include`, `tpl`; control `if/else`, `range`, `with`. **Hooks** (`helm.sh/hook: pre-upgrade`) run Jobs such as migrations. Inspect what will change with the `helm diff` plugin before upgrading.

## Kustomize

```
k8s/
├── base/
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   └── service.yaml
└── overlays/
    ├── dev/kustomization.yaml
    └── prod/
        ├── kustomization.yaml
        └── replicas-patch.yaml
```

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
labels:
  - pairs:
      app.kubernetes.io/part-of: shop
    includeSelectors: false
```

```
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-prod
resources:
  - ../../base
images:
  - name: shop/api                        # image name used in base/deployment.yaml
    newName: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api
    newTag: "1.4.2"                       # CI/GitOps bumps this line
patches:
  - path: replicas-patch.yaml
configMapGenerator:
  - name: api-config
    literals:
      - LOG_LEVEL=warn                    # generated name gets a hash: change = rolling restart
```

```
# overlays/prod/replicas-patch.yaml (strategic merge patch)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 4
```

```
kubectl kustomize overlays/prod          # render
kubectl diff -k overlays/prod            # compare with the cluster
kubectl apply -k overlays/prod
kustomize edit set image shop/api=REGISTRY/shop/api:1.4.3   # what CI runs to bump a tag
```

### Validating manifests in CI

| Tool | Checks |
|---|---|
| `kubeconform` | Schema validity against the Kubernetes version (and CRD schemas) |
| `kube-linter` / Polaris / Datree-style rules | Best practices: probes, requests, non-root, no `latest` |
| `helm lint`, `helm template \| kubeconform` | Chart correctness |
| Kyverno CLI / `conftest` (OPA) | Your organization's policies before they reach the cluster |
| `pluto` / `kubent` | Deprecated/removed API versions before upgrades |

## 12.14 Observability on Kubernetes

| Signal | Standard tools |
|---|---|
| Resource metrics | **metrics-server** ( `kubectl top` , HPA) |
| Metrics | **Prometheus** (kube-prometheus-stack: Prometheus Operator, Alertmanager, Grafana, node-exporter, **kube-state-metrics**) or Amazon Managed Prometheus; `ServiceMonitor` / `PodMonitor` CRs select what to scrape |
| Logs | Containers log to **stdout/stderr**; kubelet stores them under `/var/log/pods` ; a DaemonSet (Fluent Bit, Vector, Grafana Alloy) ships them to Loki, OpenSearch or CloudWatch |

| Signal | Standard tools |
|---|---|
| Traces | OpenTelemetry SDKs + OpenTelemetry Collector → Tempo, Jaeger, X-Ray |
| Events | `kubectl get events` (kept about one hour by default): export them (event exporter) for history |
| AWS | **CloudWatch Container Insights** (EKS add-on), control plane logs (api, audit, authenticator) |

Key metrics to know: pod restarts (`kube_pod_container_status_restarts_total`), pods not ready, Deployment replicas available vs desired, container CPU throttling (`container_cpu_cfs_throttled_periods_total`), memory working set vs limit, OOM kills, node pressure conditions, PVC usage, API server latency/errors. Full coverage in the Monitoring & SRE notes.

## 12.15 Cluster operations

### Namespaces as tenancy units: quotas and defaults

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: shop-quota
  namespace: shop
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 40Gi
    limits.memory: 60Gi
    pods: "100"
    persistentvolumeclaims: "20"
    services.loadbalancers: "2"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: shop
spec:
  limits:
    - type: Container
      defaultRequest:                    # applied when a container sets no requests
        cpu: 100m
        memory: 128Mi
      default:                           # applied when a container sets no limits
        memory: 256Mi
      max:
        memory: 4Gi
```

With a quota on `requests.cpu`, pods **without** requests are rejected; a LimitRange supplies defaults so they are accepted.

### Upgrades

Kubernetes releases a minor version roughly every **4 months**; each is supported about 14 months upstream (EKS: 14 months standard support, then paid extended support for 12 more).

1. Read the release notes and the **deprecated/removed APIs** list; scan manifests and Helm releases (`pluto`, `kubent`, EKS upgrade insights).
2. Upgrade in **dev/staging first**.
3. Control plane first, **one minor version at a time** (1.32 → 1.33 → 1.34).
4. Then add-ons (CoreDNS, kube-proxy, VPC CNI, CSI drivers, ingress controller) to compatible versions.

5. Then nodes: new node group/AMI or Karpenter drift replaces nodes gradually; PDBs protect availability. Nodes (kubelet) may lag the control plane by up to three minor versions, never be newer.
6. Verify workloads, dashboards and alerts.

Alternative for big jumps or high risk: **blue/green clusters** (build a new cluster, move traffic).

### Backup and disaster recovery

| What | How |
|---|---|
| Cluster objects | **GitOps** is the primary backup: everything can be re-applied from Git. **Velero** backs up API objects (and PV snapshots) to S3 for the rest |
| etcd (self-managed) | `etcdctl snapshot save` on a schedule, stored off-cluster; test restores. Managed control planes (EKS) handle etcd |
| Persistent data | CSI snapshots, Velero, AWS Backup, database-native backups (prefer managed DBs) |
| Rebuild | Terraform for the cluster, GitOps for workloads: measure how long a full rebuild takes (RTO) |

```
velero backup create shop-daily --include-namespaces shop --ttl 720h
velero schedule create shop-nightly --schedule "0 2 * * *" --include-namespaces shop
velero restore create --from-backup shop-daily
```

### Extending Kubernetes: CRDs and operators

A **CustomResourceDefinition (CRD)** adds a new object type to the API (`Certificate`, `ExternalSecret`, `Application`, `NodePool`). An **operator** is a controller that reconciles those objects with domain knowledge: "a `Postgres` object with 3 instances" becomes StatefulSets, Services, backups and failover. You build them with Kubebuilder / Operator SDK (Go) or Kopf (Python). Much of the platform you run (cert-manager, ESO, Argo CD, Karpenter, Prometheus Operator) is operators.

```
kubectl get crds
kubectl explain certificate.spec          # CRDs with schemas are documented too
```

## 12.16 Amazon EKS in practice

| Area | What to know |
|---|---|
| Control plane | Managed by AWS across 3 AZs; you pay a per-cluster hourly fee (about $0.10/h standard support, more for extended support) plus nodes |
| Compute options | **Managed node groups** (EC2 ASGs AWS manages), **Karpenter** (self-managed), **Fargate** (serverless pods, no DaemonSets), **EKS Auto Mode** (AWS runs Karpenter, VPC CNI, EBS CSI, load balancer controller for you) |
| Networking | **VPC CNI**: pods use VPC IPs, so size subnets generously (e.g. /19 per AZ for pods) or enable prefix delegation; nodes in **private subnets**; public/private API endpoint options |
| Human access | **EKS access entries** map IAM principals to Kubernetes permissions (access policies or groups for RBAC); the old `aws-auth` ConfigMap is deprecated |
| Pod → AWS access | **EKS Pod Identity** (agent add-on, association: namespace + service account → IAM role) or **IRSA** (OIDC provider + annotated ServiceAccount). Never use node instance-role permissions for apps |
| Load balancing | **AWS Load Balancer Controller**: Ingress → ALB, Service `LoadBalancer` → NLB, `ip` target type for direct pod targets |

| Area | What to know |
|---|---|
| Storage | EBS CSI and EFS CSI drivers as EKS add-ons (with IAM via Pod Identity) |
| Add-ons | Managed versions of VPC CNI, CoreDNS, kube-proxy, EBS CSI, Pod Identity Agent, metrics-server, CloudWatch observability |
| Logging | Enable control plane logs (api, audit, authenticator) to CloudWatch |
| Cost | Right-size requests, Karpenter consolidation, spot for stateless workloads, Graviton (arm64) nodes, avoid cross-AZ chatter, delete idle LBs/volumes |

```
aws eks update-kubeconfig --region ap-south-1 --name shop-prod   # writes kubeconfig context
aws eks list-access-entries --cluster-name shop-prod
aws eks create-pod-identity-association --cluster-name shop-prod \
  --namespace shop --service-account api --role-arn arn:aws:iam::123456789012:role/shop-api
```

Creating a cluster is normally **Terraform** ( `terraform-aws-modules/eks/aws` ) or `eksctl` :

```
# eksctl cluster config (lab use; production via Terraform)
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: lab
  region: ap-south-1
  version: "1.33"
managedNodeGroups:
  - name: default
    instanceType: t3.medium
    desiredCapacity: 2
    minSize: 1
    maxSize: 3
    privateNetworking: true
addons:
  - name: vpc-cni
  - name: coredns
  - name: kube-proxy
  - name: eks-pod-identity-agent
```

**Cost warning:** an EKS cluster costs about $0.10 per hour (≈ $73/month) **before** nodes, NAT gateways and load balancers. It is not covered by "always free" offers. Labs: create, practice, and **delete the same day** ( `eksctl delete cluster --name lab --region ap-south-1` , then check for leftover load balancers, EBS volumes and NAT gateways). Do most practice on **kind** for free.

## 12.17 Complete production application manifests

One service, `api` in namespace `shop` , with everything a production review expects. Apply with `kubectl apply -f shop-api.yaml` (in real teams: through Kustomize/Helm and Argo CD).

```
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: restricted
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
  namespace: shop
automountServiceAccountToken: false
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
  namespace: shop
data:
  LOG_LEVEL: info
  DB_HOST: shop-db.abc123.ap-south-1.rds.amazonaws.com
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: shop
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/part-of: shop
spec:
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app.kubernetes.io/name: api
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
        app.kubernetes.io/part-of: shop
    spec:
      serviceAccountName: api
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        seccompProfile:
          type: RuntimeDefault
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app.kubernetes.io/name: api
      containers:
        - name: app
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          ports:
            - name: http
              containerPort: 8080
```

```
          envFrom:
            - configMapRef:
                name: api-config
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials   # created by External Secrets, never stored in Git
                  key: DB_PASSWORD
          resources:
            requests:
              cpu: 250m
              memory: 256Mi
            limits:
              memory: 256Mi
          startupProbe:
            httpGet:
              path: /healthz
              port: http
            failureThreshold: 30
            periodSeconds: 2
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 10
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"]
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: shop
spec:
  selector:
    app.kubernetes.io/name: api
  ports:
    - name: http
      port: 80
      targetPort: http
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api
  namespace: shop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
```

```
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/healthcheck-path: /ready
spec:
  ingressClassName: alb
  rules:
    - host: api.shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  name: http
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 3
  maxReplicas: 15
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api
  namespace: shop
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: ["Ingress"]
  ingress:
    - ports:                              # ALB (ip targets) reaches pods from VPC addresses
        - protocol: TCP
          port: 8080
      from:
        - ipBlock:
            cidr: 10.0.0.0/16
```

**Review checklist this file satisfies:** no `replicas` field (HPA owns it) · immutable image tag · requests and memory limit · three probes · graceful shutdown · non-root, read-only, no capabilities · dedicated ServiceAccount without a token · secret from an external store · zone spread · PDB · NetworkPolicy · Pod Security `restricted` namespace.

## 12.18 Troubleshooting guide

### The method

```
1. What is broken, for whom, since when?    kubectl get pods -n shop -o wide; recent deploys?
2. Look at the object and its EVENTS         kubectl describe <pod|deploy|svc|node|pvc>
3. Look at logs                              kubectl logs <pod> [-c c] [--previous]
4. Follow the chain                          Ingress → Service → EndpointSlice → Pod → container
5. Check the node/cluster level              kubectl get nodes; describe node; kube-system pods
6. Change one thing, verify, write it down
```

### Pod status reference

| Status / reason | Meaning | Check | Typical fix |
|---|---|---|---|
| `Pending` + `FailedScheduling` "Insufficient cpu/memory" | No node has room for the **requests** | `describe pod`, `kubectl describe nodes` (Allocated resources) | Lower requests, add nodes / check autoscaler |
| `Pending` "didn't match node affinity/selector", "untolerated taint" | Placement rules exclude all nodes | Pod spec vs node labels/taints | Fix labels, selectors, tolerations |
| `Pending` "unbound PersistentVolumeClaim" | Storage not provisioned | `describe pvc`, CSI driver pods/logs | StorageClass name, CSI driver/IAM, AZ |
| `ContainerCreating` (long) | Volume mount, CNI or secret problem | Events: `FailedMount`, `FailedCreatePodSandBox` | Missing ConfigMap/Secret, CNI IPs exhausted (VPC CNI), Multi-Attach |
| `ErrImagePull` / `ImagePullBackOff` | Image can't be pulled | Events: "not found", "unauthorized", "timeout" | Tag/name typo, registry auth (`imagePullSecrets`, node IAM for ECR), network/NAT to registry |
| `CrashLoopBackOff` | Container starts and exits repeatedly | `logs --previous`, exit code in `describe` | App error, missing config/env, wrong command, failing dependency, liveness too aggressive |
| `OOMKilled` (exit 137) | Memory above **limit** | `describe` Last State, memory metrics | Fix leak or raise limit; tune runtime heap (JVM `-XX:MaxRAMPercentage`) |
| Exit code 1 / 2 | Application error / misuse of shell | Logs | Fix the app/config |
| Exit code 126 / 127 | Command not executable / not found | Image contents, `command` | Correct path, add binary, `chmod +x` |

| Status / reason | Meaning | Check | Typical fix |
|---|---|---|---|
| Exit code 143 | SIGTERM (normal at shutdown) | — | If during start: probes killing it |
| `CreateContainerConfigError` | Referenced ConfigMap/Secret/key missing | Events | Create it or fix the key name |
| `Running` but `0/1 READY` | Readiness probe failing | `describe` (probe failure messages), app logs | Probe path/port wrong, app not ready, dependency down |
| `Terminating` forever | Finalizer or unreachable node | `get pod -o yaml` (finalizers), node status | Fix the controller / node; last resort `--grace-period=0 --force` |
| `Evicted` | Node pressure (memory/disk) | `describe node` conditions | Set requests/limits, clean disk, bigger nodes |

## Service and networking

```
kubectl get svc api -n shop -o wide                          # selector, ports
kubectl get endpointslices -n shop -l kubernetes.io/service-name=api   # empty = no ready pods
kubectl get pods -n shop -l app.kubernetes.io/name=api --show-labels  # do labels match?
kubectl run -it --rm net --image=nicolaka/netshoot --restart=Never -n shop -- bash
#   inside: curl -sv http://api/ready ; nslookup api ; nc -zv api 80
kubectl port-forward -n shop deploy/api 8080:8080            # bypass Service: test the pod
```

| Symptom | Likely cause |
|---|---|
| Service has no endpoints | Selector/label mismatch, pods not Ready, wrong namespace |
| Connection refused via Service | `targetPort` ≠ container port, app listening on `127.0.0.1` instead of `0.0.0.0` |
| Timeout between pods | NetworkPolicy denying, security groups (VPC CNI / security groups for pods), node firewall |
| DNS fails ( `could not resolve` ) | CoreDNS pods down, NetworkPolicy blocking port 53, wrong name/namespace |
| Ingress 404 | Host/path rule not matching, wrong `ingressClassName`, controller not running |
| Ingress/ALB 502/503/504 | No healthy targets (readiness/health check path), target port wrong, app timeouts, security groups |
| LoadBalancer `EXTERNAL-IP` stays `<pending>` | No load balancer controller, missing subnet tags ( `kubernetes.io/role/elb` ), IAM permissions |

## Deployments and rollouts

| Symptom | Check |
|---|---|
| `rollout status` hangs | New pods not Ready: `describe` new pods; `progressDeadlineSeconds` exceeded → `ProgressDeadlineExceeded` |
| Change "did nothing" | Edited only a ConfigMap (pods don't restart) or the image tag is the same mutable tag |

| Symptom | Check |
|---|---|
| Old version back after a deploy | GitOps reverted a manual `kubectl` edit (Git is the source of truth) |
| Replicas jump back | HPA owns `replicas` and Git also sets it |

### Nodes

```
kubectl get nodes -o wide
kubectl describe node worker-2   # Conditions: Ready, Memory/Disk/PIDPressure
kubectl get pods -A --field-selector spec.nodeName=worker-2
kubectl debug node/worker-2 -it --image=busybox:1.36    # then: chroot /host; journalctl -u kubelet
```

`NotReady` causes: kubelet stopped or can't reach the API, container runtime down, CNI failing, disk full, expired certificates, instance problems (check EC2 status checks).

### Access (RBAC) errors

`Error from server (Forbidden): pods is forbidden: User "alice" cannot list resource "pods" in the namespace "shop"` → the identity is authenticated but not authorized. Check: `kubectl auth can-i list pods -n shop --as alice`, RoleBindings in the namespace, EKS access entries. `error: You must be logged in to the server (Unauthorized)` → authentication: expired token, wrong AWS profile, missing access entry.

## 12.19 Hands-on labs (kind on your laptop: free)

| # | Lab | You prove that you can |
|---|---|---|
| 1 | Create a 3-node kind cluster (12.3); explore with `get nodes`, `get pods -A`, `api-resources` | Run and inspect a cluster |
| 2 | Deploy nginx with 3 replicas; delete a pod and watch it come back ( `-w` ) | Explain reconciliation |
| 3 | Roll out a new image; break it with a bad tag; read events; `rollout undo` | Debug and roll back |
| 4 | Expose with a ClusterIP Service; test from a netshoot pod; break the selector and find it via EndpointSlices | Debug Services |
| 5 | Add probes; make readiness fail; observe `0/1 READY` and empty endpoints | Use probes correctly |
| 6 | Set a 64Mi memory limit and run a memory hog ( `polinux/stress` ); see `OOMKilled` | Understand limits and QoS |
| 7 | ConfigMap + Secret as env and files; change them and observe what updates | Manage configuration |
| 8 | StatefulSet with a PVC (kind's default `standard` class); delete pod 0 and confirm data survives | Use persistent storage |
| 9 | Install ingress-nginx or Envoy Gateway; route two paths to two Services | Route HTTP traffic |
| 10 | Install metrics-server (with `--kubelet-insecure-tls` on kind only) + HPA; generate load; watch scaling | Autoscale |
| 11 | Create a namespace with PSA `restricted`; try to run a root pod; fix the spec | Apply Pod Security |
| 12 | Role + RoleBinding for a ServiceAccount; test with `auth can-i --as` | Configure RBAC |
| 13 | Package lab 2–5 as a Helm chart and as Kustomize base/overlays (dev/prod) | Package apps |
| 14 | Default-deny + allow rules with Calico or Cilium installed in kind | Use NetworkPolicy |

| # | Lab | You prove that you can |
|---|---|---|
| 15 | Troubleshooting drill: a partner (or script) breaks image, selector, probe, ConfigMap key, quota; fix each from events | Troubleshoot methodically |

Clean up: `kind delete cluster --name lab`.

## 12.20 Interview questions

1. What problem does Kubernetes solve compared with running Docker on servers?
2. Explain the control plane components and what each does.
3. Walk through what happens after `kubectl apply -f deployment.yaml` until traffic reaches the pod.
4. What is reconciliation / a controller? Give an example.
5. Pod vs container vs Deployment vs ReplicaSet.
6. Why shouldn't you create bare pods in production?
7. Liveness vs readiness vs startup probes; what goes wrong when liveness checks the database?
8. Requests vs limits; what happens when a container exceeds each? QoS classes.
9. Why do many teams avoid CPU limits?
10. How does a rolling update work? Explain `maxSurge` and `maxUnavailable`. How do you roll back?
11. How do you achieve zero-downtime deployments (probes, preStop, SIGTERM, PDB)?
12. Deployment vs StatefulSet vs DaemonSet vs Job/CronJob: when each?
13. Service types; what is a headless Service?
14. How does a Service find its pods? What does empty EndpointSlices tell you?
15. Explain Kubernetes DNS names and `ndots`.
16. Ingress vs LoadBalancer Service vs Gateway API.
17. How do NetworkPolicies work? Why add a DNS egress rule after default-deny?
18. ConfigMap vs Secret; are Secrets secure? How do you manage secrets in production?
19. PV, PVC and StorageClass; RWO vs RWX; what is `WaitForFirstConsumer` for on AWS?
20. Taints/tolerations vs node affinity; topology spread constraints.
21. What is a PodDisruptionBudget, and how can it block an upgrade?
22. HPA vs VPA vs Cluster Autoscaler vs Karpenter vs KEDA.
23. Explain RBAC objects and how to grant a CI pipeline deploy rights to one namespace.
24. What are Pod Security Standards; what does `restricted` require?
25. How would you enforce "only images from our registry, no `latest`"?
26. Helm vs Kustomize; how do you make pods restart when config changes?
27. How do you upgrade an EKS cluster safely?
28. How do pods get AWS permissions on EKS (Pod Identity / IRSA) and why not the node role?
29. VPC CNI: why can pods run out of IP addresses, and how do you fix it?
30. What are CRDs and operators? Name ones you've used.
31. How do you back up and restore a cluster?
32. A pod is `CrashLoopBackOff`: your exact steps.
33. A pod is `Pending`: list the possible causes.
34. Users get 502 from the ALB after a deploy: how do you investigate?
35. How do you make a multi-tenant cluster safe for several teams (namespaces, RBAC, quotas, policies, NetworkPolicies)?

## 12.21 Self-check

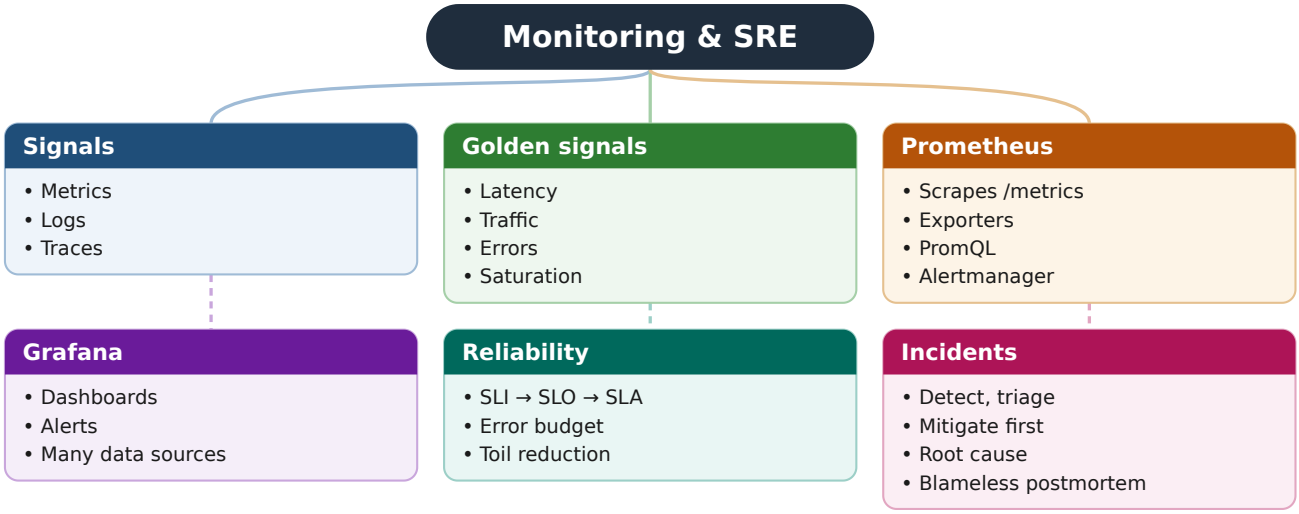
- [ ] I can explain every control plane and node component and the apply → running flow.

- [ ] I can write a Deployment, Service, Ingress, ConfigMap, Secret reference, HPA, PDB and NetworkPolicy from memory.
- [ ] I can choose between Deployment, StatefulSet, DaemonSet, Job and CronJob and justify it.
- [ ] I can configure probes, resources and graceful shutdown correctly and explain QoS.
- [ ] I can trace traffic Ingress → Service → EndpointSlice → Pod and debug each hop.
- [ ] I can set up storage with StorageClass/PVC and explain AZ limits of EBS.
- [ ] I can control placement (affinity, taints, spread) and availability (PDB, drain).
- [ ] I can configure HPA and explain Karpenter/Cluster Autoscaler.
- [ ] I can create least-privilege RBAC and apply Pod Security `restricted`.
- [ ] I can package an app with Helm and with Kustomize overlays.
- [ ] I can explain an EKS setup: access entries, Pod Identity, LB controller, VPC CNI, add-ons, upgrades, costs.
- [ ] I can diagnose every pod status in 12.18 from events and logs.

**REMEMBER**

- Debug order: **get → describe (Events) → logs --previous → exec → get endpoints**.
- **Labels connect everything**: Deployment selector ↔ pod labels ↔ Service selector.
- **Readiness** = send traffic? **Liveness** = restart? **Requests** = scheduling, **limits** = hard cap.

# 13. Monitoring & SRE




## 13.0 Why this matters

Shipping software is half the job; knowing **whether it works for users right now** is the other half. Without observability, you learn about outages from customers, you debug by guessing, and you can't tell whether a release made things better or worse. Site Reliability Engineering (SRE) adds the discipline on top: measurable reliability targets, alerting on what users feel, and a structured way to handle and learn from incidents.

## 13.1 Fundamentals

| Term | Meaning |
|---|---|
| **Monitoring** | Collecting predefined measurements and alerting on known failure modes ("is CPU > 90%?", "is the site up?") |
| **Observability** | How well you can understand a system's internal state from its outputs, including questions you didn't predict ("why are only Android users in Mumbai slow since 14:05?") |
| Telemetry | The data a system emits: metrics, logs, traces (plus profiles and events) |
| Instrumentation | Code or agents that produce telemetry |

### The signals

| Signal | What it is | Strength | Weakness | Tools |
|---|---|---|---|---|
| **Metrics** | Numeric time series with labels (`http_requests_total{status="500"}`) | Cheap, fast, ideal for dashboards and alerts, long retention | Little detail per event; label cardinality limits | Prometheus, CloudWatch, Datadog |
| **Logs** | Timestamped event records | Rich detail, exact errors | Expensive at volume, slow to aggregate | Loki, OpenSearch/ELK, CloudWatch Logs, Splunk |

| Signal | What it is | Strength | Weakness | Tools |
|---|---|---|---|---|
| **Traces** | The path of one request across services (spans with timings) | Shows where latency/errors happen in distributed systems | Need instrumentation and sampling | OpenTelemetry, Jaeger, Tempo, X-Ray |
| Profiles | Where CPU/memory is spent in code over time | Finds hot code paths | Specialized | Pyroscope, Parca |
| Events | Deploys, config changes, scaling | Correlate changes with symptoms | Often missing | Annotations, change logs |

**Correlation is the goal:** a dashboard spike (metric) → the traces for slow requests in that minute → the logs of the failing span (trace ID in the log line). Use the same labels everywhere (`service`, `env`, `version`, `region`).

### What to measure: three standard methods

| Method | For | Signals |
|---|---|---|
| **Four Golden Signals** (Google SRE) | Any user-facing service | **Latency** (separate successful from failed requests), **Traffic** (requests/s), **Errors** (rate of failures), **Saturation** (how "full": CPU, memory, queue depth, connection pools) |
| **RED** | Request-driven services (APIs) | **R**ate, **E**rrors, **D**uration |
| **USE** (Brendan Gregg) | Resources (CPU, disk, network, pools) | **U**tilization, **S**aturation, **E**rrors |

Monitor **symptoms** (what users feel: errors, latency) for paging; use **causes** (CPU, disk, a pod restart) for dashboards and diagnosis.

### Averages lie: percentiles

If 99 requests take 100 ms and 1 takes 10 s, the average is ~200 ms, which describes nobody. Use percentiles: **p50** (median, typical user), **p95**, **p99** (tail: your most-affected users, often the most valuable ones with the biggest carts). Percentiles cannot be averaged across instances; compute them from **histograms**.

## 13.2 Metrics in depth

### Metric types (Prometheus model)

| Type | Behavior | Example | Query with |
|---|---|---|---|
| **Counter** | Only goes up (resets to 0 on restart) | `http_requests_total`, `errors_total` | `rate()`, `increase()`: never graph a raw counter |
| **Gauge** | Goes up and down | `memory_bytes`, `queue_depth`, `temperature` | Directly, `avg_over_time()`, `max()` |
| **Histogram** | Counts observations into buckets (`_bucket{le="0.5"}`), plus `_sum` and `_count` | Request duration, response size | `histogram_quantile()`; aggregatable across instances |

| Type | Behavior | Example | Query with |
|---|---|---|---|
| Summary | Client-side quantiles | Rarely used now | Cannot aggregate across instances: prefer histograms |

Native histograms (Prometheus 2.40+, stable in 3.x) store high-resolution buckets automatically; classic histograms need sensible bucket boundaries around your SLO (e.g. 0.1, 0.25, 0.5, 1, 2.5 s).

### Labels and cardinality

Each unique combination of metric name + label values is a separate **time series**. `http_requests_total{method, route, status}` with 5 methods × 50 routes × 10 statuses = 2,500 series per instance: fine. Adding `user_id` or `request_id` creates millions: **cardinality explosion**, which kills Prometheus memory and makes managed services expensive.

**Rules:** labels have bounded values (route template `/orders/{id}`, never the raw URL); never put IDs, emails, timestamps or error messages in labels; put high-cardinality detail in logs/traces.

### Naming conventions

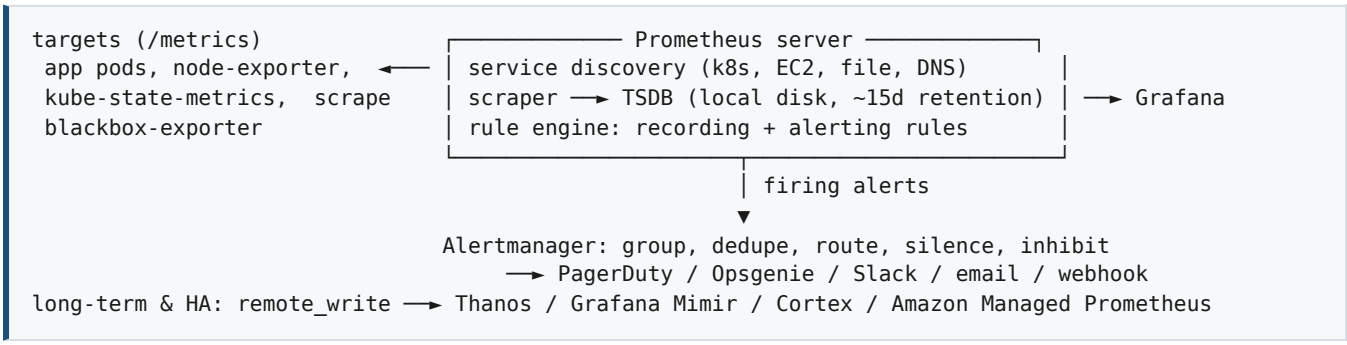
`<namespace>_<name>_<unit>_<suffix>`: `http_server_request_duration_seconds`, `shop_orders_created_total`. Base units (seconds, bytes), `_total` for counters, snake_case.

### Pull vs push

| | Pull (Prometheus) | Push (StatsD, CloudWatch PutMetricData, OTLP push) |
|---|---|---|
| How | Server scrapes `/metrics` endpoints every N seconds | Apps send metrics to a collector |
| Pros | Server controls rate; "target down" is itself a signal ( `up == 0` ); simple to debug (curl `/metrics` ) | Works for short-lived jobs and behind firewalls |
| Short-lived jobs | Pushgateway (only for batch jobs) | Native |

## 13.3 Prometheus

### Architecture




### Common exporters

| Exporter | Exposes |
|---|---|
| **node-exporter** | Linux host metrics: CPU, memory, disk, filesystem, network |
| **kube-state-metrics** | Kubernetes object state: deployments, pod status, restarts, replicas |
| cAdvisor (in kubelet) | Per-container CPU/memory/network |

| Exporter | Exposes |
|---|---|
| **blackbox-exporter** | Probes from outside: HTTP, TCP, ICMP, DNS, TLS certificate expiry |
| postgres/mysql/redis exporters | Database internals |
| CloudWatch exporter / YACE | AWS service metrics into Prometheus |

## Configuration

```
# prometheus.yml
global:
  scrape_interval: 30s
  evaluation_interval: 30s
  external_labels:
    cluster: shop-prod                  # identifies this Prometheus in federated/remote storage

rule_files:
  - rules/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: node
    static_configs:
      - targets: ["10.0.1.10:9100", "10.0.1.11:9100"]
        labels:
          env: prod

  - job_name: ec2-nodes                 # discover EC2 instances by tag
    ec2_sd_configs:
      - region: ap-south-1
        port: 9100
        filters:
          - name: tag:Monitoring
            values: ["enabled"]
    relabel_configs:
      - source_labels: [__meta_ec2_tag_Name]
        target_label: instance_name

  - job_name: blackbox-http             # check the public site from outside
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets: ["https://shop.example.com/healthz"]
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: blackbox-exporter:9115
```

**Relabeling** rewrites labels before scraping ( `relabel_configs` : choose targets, set labels from discovery metadata) or after ( `metric_relabel_configs` : drop expensive series). It is the main tool for controlling cost and cardinality.

### On Kubernetes: kube-prometheus-stack

The Helm chart installs the **Prometheus Operator**, Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics and default dashboards/alerts. You declare scrape targets and rules as Kubernetes objects:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: api
  namespace: shop
  labels:
    release: monitoring                 # must match the operator's serviceMonitorSelector
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: api       # Services with this label
  endpoints:
    - port: http                        # named Service port
      path: /metrics
      interval: 30s
```

The most common "my target isn't scraped" cause: the `release` label (selector) doesn't match, or the Service port name is wrong. Check **Status → Targets** in the Prometheus UI.

### Scaling and long-term storage

One Prometheus handles millions of active series but keeps data locally (default retention 15 days) and is a single point of failure. Production patterns: **two identical replicas** (HA; Alertmanager deduplicates) + `remote_write` to **Thanos**, **Grafana Mimir**, or **Amazon Managed Service for Prometheus** for long retention and a global view. Grafana **Alloy** or the OpenTelemetry Collector can scrape and forward instead of a full Prometheus ("agent mode").

## 13.4 PromQL in depth

### Building blocks

| Concept | Example | Meaning |
|---|---|---|
| Instant vector | `up` | Latest value of every series named `up` |
| Label matchers | `http_requests_total{job="api", status=~"5..", method!="GET"}` | `=`, `!=`, `=~` regex, `!~` |
| Range vector | `http_requests_total[5m]` | All samples in the last 5 minutes (input to functions) |
| Offset | `rate(http_requests_total[5m] offset 1w)` | Same time last week |
| Scalar math | `node_memory_MemAvailable_bytes / 1024 / 1024` | MiB |

### Essential functions

| Function | Use |
|---|---|
| `rate(counter[5m])` | Per-second average increase over 5 min; handles counter resets. **Use for alerts and graphs** |
| `irate(counter[5m])` | Rate from the last two samples: spiky, for fast-moving graphs only |

| Function | Use |
|---|---|
| `increase(counter[1h])` | Total increase over the window ( `rate × seconds` ) |
| `sum by (label) (...)` / `without (...)` | Aggregate, keeping or dropping labels. Also `avg` , `min` , `max` , `count` , `topk(5, ...)` , `quantile` |
| `histogram_quantile(0.99, sum by (le) (rate(x_bucket[5m])))` | p99 from a histogram (keep `le` in the `by` ) |
| `avg_over_time(gauge[1h])` , `max_over_time` , `quantile_over_time` | Aggregate one series over time |
| `absent(up{job="api"})` | Fires when the series doesn't exist (target disappeared completely) |
| `predict_linear(gauge[6h], 4*3600)` | Linear forecast (disk full in 4 hours?) |
| `changes()` , `resets()` , `delta()` , `deriv()` | Change detection on gauges |
| `label_replace()` , `clamp_min()` | Label and value manipulation |

**Rule:** `rate` first, then `sum` . Never `rate(sum(...))` , because summing counters breaks reset detection. Range window ≥ 4 × scrape interval (30s scrape → `[2m]` minimum, `[5m]` common).

## Query library (memorize the patterns)

```
# Request rate per service (RED: Rate)
sum by (service) (rate(http_requests_total[5m]))

# Error ratio (RED: Errors): 5xx / all
sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
  /
sum by (service) (rate(http_requests_total[5m]))

# p95 and p99 latency (RED: Duration)
histogram_quantile(0.95,
  sum by (le, service) (rate(http_request_duration_seconds_bucket[5m])))

# Share of requests faster than 300 ms (latency SLI); needs a 0.3 bucket
sum(rate(http_request_duration_seconds_bucket{le="0.3"}[5m]))
  /
sum(rate(http_request_duration_seconds_count[5m]))

# Targets that are down
up == 0

# CPU usage % per node (USE: Utilization)
100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))

# Memory used %
100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)

# Filesystem will be full within 4 hours
predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"}[6h], 4 * 3600) < 0

# Pods restarting in the last hour
increase(kube_pod_container_status_restarts_total[1h]) > 3

# Deployments without all replicas available
kube_deployment_status_replicas_available < kube_deployment_spec_replicas

# Container memory as % of its limit
sum by (namespace, pod, container) (container_memory_working_set_bytes{container!=""})
  /
sum by (namespace, pod, container) (kube_pod_container_resource_limits{resource="memory"})

# CPU throttling ratio per container
sum by (pod) (rate(container_cpu_cfs_throttled_periods_total[5m]))
  /
sum by (pod) (rate(container_cpu_cfs_periods_total[5m]))

# Top 5 pods by CPU
topk(5, sum by (pod) (rate(container_cpu_usage_seconds_total{container!=""}[5m])))

# Traffic now vs one week ago (ratio)
sum(rate(http_requests_total[5m])) / sum(rate(http_requests_total[5m] offset 1w))

# TLS certificate expires within 14 days (blackbox exporter)
probe_ssl_earliest_cert_expiry - time() < 14 * 86400
```

## Vector matching (joins)

Binary operators match series with **identical label sets**. When they differ, use `on(...)` / `ignoring(...)`, and `group_left` / `group_right` for many-to-one:

```
# Add the "team" label from a metadata metric to the error rate
sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
  * on (service) group_left (team)
service_owner_info
```

## 13.5 Instrumenting an application

```
"""app.py: minimal HTTP service exposing RED metrics for Prometheus."""
import random
import time
from http.server import BaseHTTPRequestHandler, HTTPServer

from prometheus_client import Counter, Histogram, start_http_server

REQUESTS = Counter(
    "http_requests_total", "HTTP requests", ["method", "route", "status"]
)
LATENCY = Histogram(
    "http_request_duration_seconds", "Request duration", ["route"],
    buckets=(0.05, 0.1, 0.25, 0.3, 0.5, 1.0, 2.5, 5.0),
)


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        route = "/orders" if self.path.startswith("/orders") else "other"
        start = time.perf_counter()
        time.sleep(random.uniform(0.01, 0.4))            # simulated work
        status = 500 if random.random() < 0.02 else 200  # 2% errors
        self.send_response(status)
        self.end_headers()
        self.wfile.write(b"ok\n")
        LATENCY.labels(route=route).observe(time.perf_counter() - start)
        REQUESTS.labels(method="GET", route=route, status=str(status)).inc()

    def log_message(self, *args):                        # silence default access log
        pass


if __name__ == "__main__":
    start_http_server(9000)                              # metrics on :9000/metrics
    HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

Run: `pip install prometheus-client`, `python3 app.py`, send traffic (`for i in $(seq 200); do curl -s localhost:8000/orders >/dev/null; done`), then `curl -s localhost:9000/metrics | grep http_`. In real services use the framework integrations (Flask/FastAPI middleware, Spring Boot Actuator/Micrometer, Go `promhttp`) or **OpenTelemetry** SDKs (13.9).

## 13.6 Alerting

### Alerting principles (what separates good on-call from burnout)

1. **Page only on symptoms that hurt users now** or will very soon (SLO burn, site down, data loss risk). Everything else is a **ticket** or dashboard.
2. Every page must be **actionable**, and must link a **runbook**.
3. Alert on **rates and ratios over windows**, not single samples; use `for:` to ignore blips.
4. Fewer, better alerts. A noisy alert trains people to ignore the real one (**alert fatigue**).
5. Review alerts after every incident and regularly: delete ones nobody acts on.

| Severity | Response | Example |
|---|---|---|
| `critical` (page) | Wake someone up, immediately | Error budget burning 14× fast, checkout down |
| `warning` (ticket) | Next business day | Disk full in 3 days, certificate expires in 14 days |
| `info` | No notification (dashboards) | Deployment happened |

## Recording and alerting rules

**Recording rules** precompute expensive queries into new series (faster dashboards, consistent SLO math). Naming: `level:metric:operations`.

```
# rules/api.yml
groups:
  - name: api-recording
    interval: 30s
    rules:
      - record: service:http_requests:rate5m
        expr: sum by (service) (rate(http_requests_total[5m]))
      - record: service:http_errors:ratio_rate5m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
            /
          sum by (service) (rate(http_requests_total[5m]))

  - name: api-alerts
    rules:
      - alert: ApiHighErrorRate
        expr: service:http_errors:ratio_rate5m{service="api"} > 0.05
        for: 5m                           # must be true for 5 minutes before firing
        labels:
          severity: critical
          team: shop
        annotations:
          summary: "API error rate {{ $value | humanizePercentage }} (> 5%)"
          description: "5xx ratio above 5% for 5 minutes on {{ $labels.service }}."
          runbook_url: https://wiki.example.com/runbooks/api-high-error-rate
          dashboard: https://grafana.example.com/d/api-red

      - alert: TargetDown
        expr: up{job="api"} == 0
        for: 3m
        labels:
          severity: warning
        annotations:
          summary: "{{ $labels.instance }} is not being scraped"

      - alert: DiskWillFillIn4Hours
        expr: |
          predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"}[6h], 4 * 3600) < 0
        for: 30m
        labels:
          severity: warning
        annotations:
          summary: "Filesystem {{ $labels.mountpoint }} on {{ $labels.instance }} fills in < 4h"

      - alert: CertificateExpiresSoon
        expr: probe_ssl_earliest_cert_expiry - time() < 14 * 86400
        for: 1h
        labels:
          severity: warning
        annotations:
          summary: "TLS certificate for {{ $labels.instance }} expires in < 14 days"
```

Alert states: **inactive → pending** (condition true, waiting for `for`) **→ firing**. Validate rules in CI: `promtool check rules rules/*.yml`; unit-test them with `promtool test rules`.

### Alertmanager

Receives alerts and decides **who** gets notified, **how often**, and **in what groups**.

```
# alertmanager.yml
route:
  receiver: default-slack
  group_by: ["alertname", "service"]     # one notification per group, not per series
  group_wait: 30s                        # wait to collect alerts of a new group
  group_interval: 5m                     # wait before sending updates for a group
  repeat_interval: 4h                    # re-send if still firing
  routes:
    - matchers: ['severity="critical"']
      receiver: pagerduty-shop
      continue: true                     # also evaluate the next routes
    - matchers: ['severity="critical"']
      receiver: shop-slack
    - matchers: ['severity="warning"']
      receiver: shop-slack

inhibit_rules:                           # suppress symptoms when the cause is known
  - source_matchers: ['alertname="ClusterUnreachable"']
    target_matchers: ['severity=~"warning|critical"']
    equal: ["cluster"]

receivers:
  - name: default-slack
    slack_configs:
      - api_url_file: /etc/alertmanager/secrets/slack-webhook   # secret from a file
        channel: "#alerts"
        send_resolved: true
  - name: shop-slack
    slack_configs:
      - api_url_file: /etc/alertmanager/secrets/slack-webhook
        channel: "#shop-alerts"
        send_resolved: true
  - name: pagerduty-shop
    pagerduty_configs:
      - routing_key_file: /etc/alertmanager/secrets/pagerduty-key
```

| Feature | Purpose |
|---|---|
| Grouping | 50 pods failing = one notification |
| Deduplication | HA Prometheus pairs send the same alert twice; Alertmanager sends once |
| **Silences** | Mute matching alerts for a time window (planned maintenance): `amtool silence add alertname=TargetDown --duration=2h --comment="patching"` |
| **Inhibition** | Mute dependent alerts when a root-cause alert fires |

Validate: `amtool check-config alertmanager.yml`. Webhook URLs and keys come from files mounted from a secret store, never written in the config in Git.

## 13.7 Grafana

| Concept | Meaning |
|---|---|
| Data source | Prometheus, Loki, Tempo, CloudWatch, Elasticsearch, PostgreSQL… |

| Concept | Meaning |
|---|---|
| Dashboard | Panels (time series, stat, gauge, table, heatmap, logs) built from queries |
| Variables | Drop-downs ( `$namespace` , `$service` ) from queries like `label_values(up, namespace)` |
| Annotations | Mark deploys/incidents on graphs |
| Alerting | Grafana can also evaluate alert rules (unified alerting) |
| Explore | Ad-hoc querying; jump from metrics to logs to traces |

**Dashboard design:**

- One **overview** per service following RED/golden signals: top row = SLO status, request rate, error ratio, p95/p99 latency; below = saturation (CPU, memory, pool usage); bottom = dependencies.
- Consistent units and colors; titles say what "good" looks like.
- Link to drill-down dashboards and runbooks; avoid 40-panel "wall of graphs".

**Dashboards as code:** provisioning files (YAML + JSON) in Git, the Grafana Terraform provider, or the Grafana Operator. With kube-prometheus-stack, a ConfigMap labeled `grafana_dashboard: "1"` is loaded automatically. Don't hand-edit production dashboards without saving them back to Git.

## 13.8 Logging

### Good logs

| Practice | Why |
|---|---|
| **Structured** (JSON) with consistent fields: `timestamp` , `level` , `service` , `env` , `version` , `trace_id` , `message` | Machine-searchable, joinable with traces |
| Correct levels: `DEBUG` (off in prod), `INFO` (business events), `WARN` (unexpected but handled), `ERROR` (failed operation), `FATAL` | Filtering and alerting |
| Log to **stdout/stderr** in containers | The platform collects them; no log files inside containers |
| **Never log secrets or personal data** (passwords, tokens, card numbers, full emails); mask at source | Security and privacy law (GDPR, DPDP Act in India) |
| Include context (order ID, user ID hash, route) but not huge payloads | Debuggable and affordable |
| Sample or rate-limit noisy logs | Cost |

```
{"timestamp":"2026-10-18T09:14:03.512Z","level":"ERROR","service":"api","env":"prod",
 "version":"1.4.2","trace_id":"4bf92f3577b34da6a3ce929d0e0e4736","route":"/orders",
 "message":"payment provider timeout","duration_ms":3002}
```

### Log pipeline

```
app stdout ──► node: /var/log/pods/... ──► agent (DaemonSet): Fluent Bit / Vector / Grafana Alloy
                                                parse, add k8s metadata, drop noise, mask PII
                                                              |
                                  ┌───────────────────────────┼──────────────────────────┐
                                  ▼                           ▼                          ▼
                  Loki (labels index, cheap        OpenSearch / Elasticsearch      CloudWatch Logs
                  object storage on S3)            (full-text index, powerful,      (managed, AWS-native)
                                                    expensive at scale)             → S3 for archive
```

| Backend | Trade-off |
|---|---|
| **Grafana Loki** | Indexes only labels; stores compressed chunks on S3: cheap, scales; queries scan text (keep label sets small) |
| OpenSearch / ELK | Full-text indexing: fast arbitrary search, costly storage/compute |
| CloudWatch Logs | Zero-ops on AWS, Logs Insights queries; ingestion cost adds up: set retention per log group |
| Splunk / Datadog | Commercial, feature-rich, priced by volume |

**Retention and cost:** decide retention per class (debug 7 days, app 30 days, audit/security 1+ years in cheap storage). Ingestion volume is the cost driver: drop health-check and debug noise at the agent.

### LogQL (Loki) essentials

```
{namespace="shop", app="api"} |= "error"                          # filter lines containing "error"
{namespace="shop", app="api"} | json | level="ERROR" | route="/orders"
sum by (route) (count_over_time({app="api"} | json | level="ERROR" [5m]))   # errors/5m by route
{app="api"} | json | duration_ms > 1000 | line_format "{{.route}} {{.duration_ms}}ms"
```

## 13.9 Distributed tracing and OpenTelemetry

```
trace 4bf92f35... (one user request)
├── span: GET /checkout                api-gateway    480 ms
│   ├── span: POST /orders             orders-svc     430 ms
│   │   ├── span: SELECT orders        postgres        12 ms
│   │   └── span: POST /charge         payments-svc   400 ms  ◄── the slow part
│   └── span: GET /cart                cart-svc        25 ms
```

| Term | Meaning |
|---|---|
| Trace | All spans of one request, sharing a **trace ID** |
| Span | One operation: name, start, duration, status, attributes ( `http.route` , `db.system` ), parent span |
| **Context propagation** | Passing trace/span IDs between services in headers: W3C `traceparent` standard |
| **Sampling** | Keep a fraction of traces: head-based (decide at start, e.g. 10%) or **tail-based** (decide after completion: keep all errors and slow ones) in the Collector |

### OpenTelemetry (OTel)

The CNCF **vendor-neutral standard** for metrics, logs and traces: APIs/SDKs for every major language, **auto-instrumentation** (Java agent, Python `opentelemetry-instrument` , Kubernetes operator injection), the **OTLP** protocol, and the **Collector**. Instrument once, send anywhere (Tempo, Jaeger, X-Ray, Datadog, Prometheus).

```
# otel-collector.yaml: receive OTLP, batch, tail-sample, export traces and metrics
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
processors:
  memory_limiter:
    check_interval: 1s
    limit_percentage: 80
  k8sattributes: {}                     # add pod/namespace/deployment attributes
  tail_sampling:
    decision_wait: 10s
    policies:
      - name: keep-errors
        type: status_code
        status_code:
          status_codes: [ERROR]
      - name: keep-slow
        type: latency
        latency:
          threshold_ms: 1000
      - name: sample-rest
        type: probabilistic
        probabilistic:
          sampling_percentage: 10
  batch: {}
exporters:
  otlp/tempo:
    endpoint: tempo.monitoring:4317
    tls:
      insecure: true                    # in-cluster only; use TLS across networks
  prometheusremotewrite:
    endpoint: http://mimir.monitoring/api/v1/push
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, k8sattributes, tail_sampling, batch]
      exporters: [otlp/tempo]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, k8sattributes, batch]
      exporters: [prometheusremotewrite]
```

Python auto-instrumentation: `pip install opentelemetry-distro opentelemetry-exporter-otlp`, `opentelemetry-bootstrap -a install`, then run with `OTEL_SERVICE_NAME=api OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317 opentelemetry-instrument python app.py`.

## 13.10 AWS CloudWatch and native observability

| Service | Use |
|---|---|
| **CloudWatch Metrics** | AWS services publish automatically (EC2 CPU, ALB `HTTPCode_Target_5XX_Count`, `TargetResponseTime`, RDS `FreeStorageSpace`, Lambda `Errors` / `Duration`). Standard resolution 1 min (EC2 basic: 5 min; detailed monitoring: 1 min) |
| **CloudWatch agent** | OS-level metrics (memory, disk: **not** collected by default on EC2) and log files |
| **CloudWatch Alarms** | Threshold or anomaly-detection alarms → SNS (email, chat, PagerDuty), Auto Scaling, EC2 actions. **Composite alarms** combine alarms to cut noise |

| Service | Use |
|---|---|
| **CloudWatch Logs** | Log groups/streams, retention, metric filters (turn log patterns into metrics), subscription filters (stream to Lambda/Firehose/OpenSearch) |
| **Logs Insights** | Query language over logs |
| Container Insights / Application Signals | EKS/ECS metrics and logs; service-level SLOs |
| **X-Ray** / CloudWatch traces | Distributed tracing (accepts OpenTelemetry via ADOT) |
| Synthetics canaries | Scripted checks of URLs and user journeys |
| Amazon Managed Prometheus / Managed Grafana | Hosted Prometheus-compatible storage and Grafana |
| AWS Health, CloudTrail | Provider incidents; API audit trail (who changed what) |

```
# Alarm: ALB target 5xx > 10 per minute for 3 consecutive minutes -> SNS topic
aws cloudwatch put-metric-alarm --alarm-name shop-alb-5xx \
  --namespace AWS/ApplicationELB --metric-name HTTPCode_Target_5XX_Count \
  --dimensions Name=LoadBalancer,Value=app/shop-alb/0123456789abcdef \
  --statistic Sum --period 60 --evaluation-periods 3 --threshold 10 \
  --comparison-operator GreaterThanThreshold --treat-missing-data notBreaching \
  --alarm-actions arn:aws:sns:ap-south-1:123456789012:shop-alerts
```

```
# Logs Insights: top error messages in the last hour
fields @timestamp, level, message
| filter level = "ERROR"
| stats count(*) as errors by message
| sort errors desc
| limit 20
```

**Cost awareness:** custom metrics, high-resolution metrics, log ingestion (per GB) and long retention cost money. Set **retention on every log group** (default is "never expire"), avoid per-request custom metrics, and use metric filters or EMF (Embedded Metric Format) instead of `PutMetricData` loops.

## 13.11 Site Reliability Engineering

SRE (from Google) is "what happens when you treat operations as a software problem". Core ideas:

| Principle | In practice |
|---|---|
| Reliability is a **feature with a target** | 100% is the wrong target: users can't tell 99.99% from 100%, and it costs exponentially more. Pick a target the business needs |
| **Error budgets** balance speed and stability | Budget left → ship faster; budget spent → focus on reliability |
| Eliminate **toil** | Manual, repetitive, automatable work that scales with load. Cap it (Google: under 50% of SRE time) and automate it |
| Monitoring on symptoms, with SLOs | Page on user impact, not on every CPU spike |
| **Blameless postmortems** | Fix systems and processes, not people |

| Principle | In practice |
|---|---|
| Simplicity, gradual rollouts, automation | Most outages come from changes: make changes small, progressive, reversible |

## SLI, SLO, SLA

| Term | Definition | Example |
|---|---|---|
| **SLI** (indicator) | A measured ratio of good events to valid events | Successful (non-5xx) requests / all requests; requests faster than 300 ms / all requests |
| **SLO** (objective) | Target for an SLI over a window | 99.9% of requests succeed over a rolling 30 days |
| **SLA** (agreement) | Contract with customers, with penalties (credits) | 99.5% monthly availability or 10% credit. SLA is **looser** than the SLO so you're warned first |
| **Error budget** | `1 − SLO`: the failure you're allowed | 0.1% of requests may fail in 30 days |

**Good SLIs** are measured as close to the user as possible (load balancer or client, not CPU), and cover what users care about: availability, latency, correctness, freshness (data pipelines), durability (storage).

## Error budget math (30-day window)

| SLO | Allowed bad time per 30 days | Per day |
|---|---|---|
| 99% | 7 h 12 min | 14 min 24 s |
| 99.5% | 3 h 36 min | 7 min 12 s |
| **99.9%** ("three nines") | **43 min 12 s** | 1 min 26 s |
| 99.95% | 21 min 36 s | 43 s |
| 99.99% | 4 min 19 s | 8.6 s |
| 99.999% | 26 s | 0.9 s |

Request-based example: 10,000,000 requests per 30 days at 99.9% → budget = **10,000 failed requests**. If an incident fails 4,000 requests, 40% of the budget is gone.

**Dependencies multiply:** a service calling three dependencies each at 99.9% (in series) can be at most ~99.7% available. You can't promise more than your critical dependencies allow, unless you add redundancy or graceful degradation.

## Error budget policy (agreed in writing with product owners)

- Budget remaining: releases proceed normally.
- Budget below 25%: extra review for risky changes, prioritize reliability items.
- Budget exhausted: **freeze feature releases** (except fixes and security) until the SLO recovers; postmortem required for any incident consuming more than 20% of the budget.

## Alerting on SLOs: multi-window, multi-burn-rate

**Burn rate** = how fast the budget is consumed relative to "exactly on budget". Burn rate 1 uses the whole budget in 30 days; burn rate 14.4 uses it in about 2 days.

| Severity | Burn rate | Long window | Short window | Budget consumed when it fires |
|---|---|---|---|---|
| Page | 14.4 | 1 h | 5 min | 2% |

| Severity | Burn rate | Long window | Short window | Budget consumed when it fires |
|---|---|---|---|---|
| Page | 6 | 6 h | 30 min | 5% |
| Ticket | 1 | 3 days | 6 h | 10% |

Both windows must exceed the threshold: the long window proves it's significant, the short window proves it's **still happening** (so the alert resolves quickly after recovery).

```
# SLO 99.9% availability -> error budget 0.001
groups:
  - name: api-slo
    rules:
      - record: service:sli_errors:ratio_rate5m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
            / sum by (service) (rate(http_requests_total[5m]))
      - record: service:sli_errors:ratio_rate30m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[30m]))
            / sum by (service) (rate(http_requests_total[30m]))
      - record: service:sli_errors:ratio_rate1h
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[1h]))
            / sum by (service) (rate(http_requests_total[1h]))
      - record: service:sli_errors:ratio_rate6h
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[6h]))
            / sum by (service) (rate(http_requests_total[6h]))
      - alert: ApiErrorBudgetFastBurn
        expr: |
          (
            service:sli_errors:ratio_rate1h{service="api"} > (14.4 * 0.001)
            and
            service:sli_errors:ratio_rate5m{service="api"} > (14.4 * 0.001)
          )
          or
          (
            service:sli_errors:ratio_rate6h{service="api"} > (6 * 0.001)
            and
            service:sli_errors:ratio_rate30m{service="api"} > (6 * 0.001)
          )
        labels:
          severity: critical
        annotations:
          summary: "API is burning its 30-day error budget too fast"
          runbook_url: https://wiki.example.com/runbooks/api-error-budget
```

Tools generate these rules from a short SLO definition: **Sloth**, **Pyrra**, OpenSLO spec, Grafana SLO, CloudWatch Application Signals SLOs.

## 13.12 Incident management

### Severity levels (example)

| Sev | Definition | Response |
|---|---|---|
| SEV1 | Critical user-facing outage or data loss/security breach | All hands, incident commander, status page, exec updates every 30 min |
| SEV2 | Major degradation, key feature broken for many users | On-call + team, updates every hour |

| Sev | Definition | Response |
|---|---|---|
| SEV3 | Minor impact, workaround exists | Normal working hours |
| SEV4 | No user impact | Ticket |

## Roles

| Role | Responsibility |
|---|---|
| **Incident Commander (IC)** | Coordinates, decides, delegates. Does **not** debug |
| Operations / technical lead | Investigates and applies fixes |
| Communications lead | Status page, stakeholders, support teams |
| Scribe | Timeline of actions and findings |
| Subject-matter experts | Pulled in as needed |

## Lifecycle

```
Detect ⟶ Triage (severity, declare, open channel) ⟶ Mitigate (stop the bleeding) ⟶
Resolve (root fix) ⟶ Postmortem (within ~5 days) ⟶ Follow-up actions tracked to completion
```

**Mitigate first, investigate later:** roll back the last deploy, shift traffic away from a bad region/AZ, scale up, disable a feature flag, fail over. Root cause can wait until users are fine.

**Key metrics:** MTTD (time to detect), MTTA (to acknowledge), **MTTR** (to restore), incident count by severity, % of incidents detected by monitoring (vs by customers).

## On-call that works

- Rotations with primary and secondary, handover notes, fair schedules across time zones (follow-the-sun when possible).
- Every alert has a **runbook**: what it means, how to check impact, mitigation steps, escalation contacts, dashboards.
- Track page volume; more than ~2 incidents per 12-hour shift means the system or alerts need work.
- Tools: PagerDuty, Opsgenie, Grafana OnCall/IRM, incident.io, FireHydrant.

## Runbook template

```
Alert: ApiHighErrorRate (critical)
Meaning:     >5% of API requests return 5xx for 5 minutes. Users see failed checkouts.
Check:       Grafana "API RED" dashboard; recent deploys (Argo CD history); dependency status.
Mitigate:    1. If a deploy happened in the last 30 min: roll back (argocd app rollback shop-api).
             2. If one AZ/pod set is bad: cordon/drain or scale; check ALB target health.
             3. If a dependency (payments) is failing: enable feature flag "payments-degraded".
Escalate:    After 15 min without mitigation: page shop secondary + payments on-call.
Verify:      Error ratio < 1% for 10 min; synthetic checkout canary green.
```

## Blameless postmortem template

| Section | Content |
|---|---|
| Summary | What happened, impact in numbers (duration, users/requests affected, revenue, budget consumed) |
| Timeline | UTC timestamps: trigger, detection, key decisions, mitigation, resolution |

| Section | Content |
|---|---|
| Root cause(s) and contributing factors | Technical and process causes (use "5 whys"; there is rarely one root cause) |
| Detection | How we found out; could monitoring have caught it sooner? |
| What went well / what went badly / where we got lucky | Honest learning |
| Action items | Specific, owned, dated, prioritized (prevent, detect, mitigate) |

"Blameless" means people describe what they did and why it made sense at the time, without fear. Human error is a starting point for asking **why the system allowed it**.

## 13.13 Capacity, load testing and chaos engineering

| Practice | How |
|---|---|
| **Capacity planning** | Track saturation trends (CPU, memory, connections, queue depth) and traffic growth; forecast ( `predict_linear` , seasonality); keep headroom (e.g. survive losing one AZ at peak) |
| **Load testing** | k6, Locust, JMeter, Gatling: find the breaking point and the latency curve before users do; run in a production-like environment; test autoscaling |
| **Chaos engineering** | Inject failures deliberately to prove resilience: kill pods/nodes, add latency, fail an AZ, exhaust CPU. Tools: Chaos Mesh, LitmusChaos, AWS Fault Injection Service. Start with a **hypothesis**, small blast radius, in staging, with a stop condition |
| **Game days** | Scheduled drills where the team practices incident response (and restores from backup) |

```
// k6 load test: ramp to 200 virtual users, fail the run if SLO-like thresholds break
import http from "k6/http";
import { check, sleep } from "k6";

export const options = {
  stages: [
    { duration: "2m", target: 50 },
    { duration: "5m", target: 200 },
    { duration: "2m", target: 0 },
  ],
  thresholds: {
    http_req_failed: ["rate<0.001"],     // < 0.1% errors
    http_req_duration: ["p(95)<300"],    // p95 < 300 ms
  },
};

export default function () {
  const res = http.get("https://staging.shop.example.com/api/products");
  check(res, { "status is 200": (r) => r.status === 200 });
  sleep(1);
}
```

## 13.14 A production monitoring stack (reference design)

```
                          ┌──────────────── Grafana (dashboards, Explore, SLOs) ────────────────┐
                          │                       │                     │                      │
Metrics:  apps ─OTLP/scrape─► Prometheus (x2, HA) ─remote_write─► Mimir / AMP (long-term)
Logs:     stdout ─► Fluent Bit / Alloy DaemonSet ─► Loki (S3)        or CloudWatch Logs
Traces:   OTel SDK ─► OTel Collector (tail sampling) ─► Tempo (S3)   or X-Ray
Probes:   blackbox exporter + synthetic canaries (outside-in, from several regions)
Alerts:   Prometheus rules (SLO burn, golden signals) ─► Alertmanager ─► PagerDuty / Slack
Changes:  Argo CD / CI deploy events ─► Grafana annotations
```

**Must-haves before go-live:** golden-signal dashboard per service, SLO + burn-rate alerts, runbooks linked from alerts, external synthetic check, log retention policy, trace IDs in logs, on-call rotation, alert routing tested end-to-end (fire a test alert).

**Who watches the watcher?** A **dead man's switch**: an always-firing `Watchdog` alert sent to an external service (Healthchecks.io, PagerDuty heartbeat, Cronitor) that pages you when it **stops** arriving. kube-prometheus-stack ships this alert by default.

## 13.15 Troubleshooting the monitoring system

| Symptom | Check |
|---|---|
| Target missing in Prometheus | Status → Targets / Service Discovery; ServiceMonitor `release` label; Service port name; NetworkPolicy blocking the scrape |
| Target `DOWN` | `curl` the `/metrics` endpoint from the Prometheus pod; TLS/auth; wrong port |
| Query returns nothing | Label names/values (use Explore autocomplete); `rate()` window shorter than 2 scrapes; metric renamed |
| Graph spikes or drops at restarts | Using `irate` or raw counters; use `rate` |
| Prometheus OOM / slow | Cardinality: `topk(10, count by (__name__) ({__name__=~".+"}))`, TSDB status page; drop series with `metric_relabel_configs` |
| Alert never fires | `promtool test rules`; alert pending (check `for`); Alertmanager routing (`amtool config routes test`); silences/inhibitions |
| Alert floods | Group by better labels, add `for`, inhibition, fix the threshold or delete the alert |
| No logs in Loki/CloudWatch | Agent DaemonSet running on that node? Parsing errors in agent logs; permissions (IAM for CloudWatch); rate limits on ingestion |
| Traces broken into pieces | Context not propagated (missing instrumentation on an HTTP client or queue); mixed propagation formats |

## 13.16 Hands-on labs

| # | Lab |
|---|---|
| 1 | Run Prometheus + node-exporter + Grafana with Docker Compose; add Prometheus as a data source; import the "Node Exporter Full" dashboard |
| 2 | Run the 13.5 app; scrape it; write the RED queries from 13.4 |
| 3 | Add the recording and alerting rules from 13.6; validate with `promtool check rules`; trigger the error alert by raising the error rate |

| # | Lab |
|---|---|
| 4 | Add Alertmanager with a Slack/email receiver (webhook from a local file); test grouping and a silence |
| 5 | Add blackbox-exporter; monitor a public URL and its certificate expiry |
| 6 | On kind: install kube-prometheus-stack with Helm; add a ServiceMonitor for your app; break the label and debug |
| 7 | Add Loki + Alloy/Fluent Bit; query JSON logs with LogQL; build a logs panel next to metrics |
| 8 | Add OpenTelemetry auto-instrumentation and Tempo; follow one slow request from metric to trace to log |
| 9 | Define a 99.9% SLO; implement the burn-rate alerts from 13.11; calculate the remaining budget after a simulated incident |
| 10 | Run the k6 test against your app; record the breaking point; write a one-page postmortem for the "incident" you caused |
| 11 | (AWS, small cost: watch the free plan credits) CloudWatch agent on an EC2 instance, a CPU alarm to SNS email, a Logs Insights query; delete everything afterwards |

## 13.17 Interview questions

1. Monitoring vs observability; metrics vs logs vs traces: when do you use each?
2. Explain the four golden signals, RED and USE.
3. Why are averages misleading? How do you compute p99 in Prometheus?
4. Counter vs gauge vs histogram vs summary.
5. What is cardinality and how do you prevent a cardinality explosion?
6. Explain Prometheus architecture and the pull model; how is a short-lived job monitored?
7. Write PromQL for request rate, error ratio and p95 latency per service.
8. Why `rate()` before `sum()`? `rate` vs `irate` vs `increase`?
9. How does Prometheus discover targets on Kubernetes (ServiceMonitor)? A target is missing: steps?
10. How do you make Prometheus highly available and keep a year of data?
11. Recording rules: why? Alerting rule states (`pending`, `firing`) and `for:`.
12. What makes a good alert? How do you fight alert fatigue?
13. Alertmanager grouping, inhibition, silences, routing.
14. Structured logging: what fields, and what must never be logged?
15. Loki vs Elasticsearch vs CloudWatch Logs trade-offs.
16. Explain a trace, spans, context propagation and sampling (head vs tail).
17. What is OpenTelemetry and why are companies standardizing on it?
18. Which CloudWatch metrics are not collected by default on EC2? How do you alarm on ALB 5xx?
19. SLI vs SLO vs SLA, with examples. How do you choose an SLO?
20. Calculate the monthly error budget for 99.9% and 99.95%.
21. What is burn-rate alerting and why multi-window?
22. What is an error budget policy? What happens when the budget is exhausted?
23. What is toil? Give examples you've automated.
24. Walk through how you'd run a SEV1 incident. What does the incident commander do?
25. What is a blameless postmortem? What sections does it have?
26. MTTD, MTTA, MTTR: how would you reduce each?
27. How do you monitor the monitoring system (dead man's switch)?
28. Load testing vs chaos engineering: how and when?
29. Design observability for a new microservice platform on EKS.

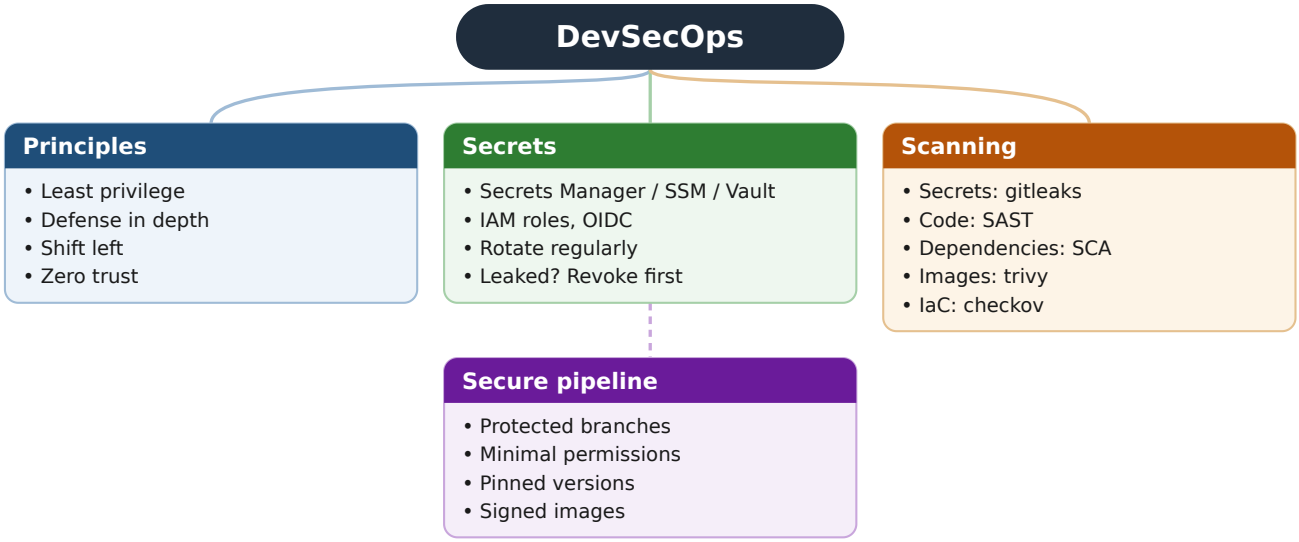
## 13.18 Self-check

- [ ] I can explain the signals, golden signals, RED and USE, and why percentiles matter.
- [ ] I can choose metric types and keep labels low-cardinality.
- [ ] I can configure Prometheus scraping (static, EC2, Kubernetes ServiceMonitor) and debug missing targets.
- [ ] I can write the PromQL query library in 13.4 from memory.
- [ ] I can write recording/alerting rules and an Alertmanager routing tree, and validate them with promtool/amtool.
- [ ] I can design a Grafana service dashboard and keep it as code.
- [ ] I can set up structured logging, a log pipeline and retention, and query with LogQL / Logs Insights.
- [ ] I can explain tracing and configure an OpenTelemetry Collector.
- [ ] I can define SLIs/SLOs, compute error budgets and implement burn-rate alerts.
- [ ] I can run an incident (roles, mitigation first) and write a blameless postmortem.

**REMEMBER**

- **SLI** = what you measure · **SLO** = your target · **SLA** = promise to customers.
- Alert on what users feel; every alert must be actionable.
- During an incident: **restore service first, investigate second**.

# 14. DevSecOps




## 14.0 Why DevSecOps

Traditional security was a gate at the end: a security team reviewed a release weeks after the code was written, found problems that were expensive to fix, and slowed everything down. Modern teams deploy many times a day, so security must be **built into every step**, automated in the pipeline, and owned by the engineers who build the system. That is DevSecOps: **security as code, continuously, by everyone**, with the security team providing tools, guardrails and expertise.

Real breaches DevOps engineers prevent: leaked cloud keys in public repositories (crypto-mining bills within minutes), vulnerable dependencies (Log4Shell, 2021), compromised build systems and packages (SolarWinds 2020, Codecov 2021, the xz backdoor 2024, malicious npm/PyPI packages), public S3 buckets, over-privileged IAM roles, exposed Kubernetes dashboards.

## 14.1 Principles

| Principle | Meaning |
|---|---|
| **Shift left** | Find issues as early as possible: IDE, pre-commit, pull request. A bug found in a PR costs minutes; in production, an incident |
| **Shield right** | Also protect and detect at runtime: WAF, runtime detection, audit logs, anomaly detection |
| **Defense in depth** | Several independent layers (network, identity, workload, data), so one failure is not a breach |
| **Least privilege** | Every human, service and pipeline gets only the permissions it needs, for only as long as it needs them |
| **Zero trust** | Never trust by network location; authenticate and authorize every request (identity-based access, mTLS) |
| Secure by default | Templates, modules and golden paths ship with security already on (encryption, private, non-root) |

| Principle | Meaning |
|---|---|
| Automate guardrails, not gates | Policies in code that block only real risks, with fast feedback and a documented exception process |
| **Shared responsibility** | Cloud provider secures the cloud; you secure what you put in it (IAM, data, network config, OS, apps) |

**CIA triad** (what security protects): **Confidentiality** (only authorized access), **Integrity** (no unauthorized change), **Availability** (accessible when needed).

### Threat modeling

Before building, ask four questions (Shostack): *What are we building? What can go wrong? What are we going to do about it? Did we do a good job?* Draw a data-flow diagram with trust boundaries and apply **STRIDE**:

| Threat | Violates | Example | Typical control |
|---|---|---|---|
| **S**poofing | Authentication | Stolen token used to call the API | MFA, short-lived tokens, mTLS |
| **T**ampering | Integrity | Modified container image | Image signing, immutable tags, checksums |
| **R**epudiation | Non-repudiation | "I didn't delete that bucket" | CloudTrail / audit logs, signed commits |
| **I**nformation disclosure | Confidentiality | Public S3 bucket, secrets in logs | Encryption, block public access, masking |
| **D**enial of service | Availability | Traffic flood, resource exhaustion | WAF, rate limits, autoscaling, quotas |
| **E**levation of privilege | Authorization | Container escape to node, IAM `PassRole` abuse | Non-root, Pod Security, least-privilege IAM |

## 14.2 Security across the delivery lifecycle

```
PLAN            CODE              BUILD / CI              DEPLOY              RUN
threat model,   IDE plugins,      secret scan, SAST,      admission policy,   WAF, runtime detect,
security        pre-commit hooks, SCA, IaC + image scan,  verify signatures,  CSPM, audit logs,
stories         signed commits    SBOM, sign, DAST        least-priv identity rescans, response
```

| Category | What it finds | Tools (open source / common) | When |
|---|---|---|---|
| **Secret scanning** | Keys, tokens, passwords in code and history | gitleaks, TruffleHog, GitHub secret scanning + push protection | Pre-commit, PR, whole history |
| **SAST** | Insecure code patterns (injection, unsafe deserialization) | Semgrep, CodeQL, SonarQube, Bandit (Python) | PR |
| **SCA** | Known-vulnerable or malicious dependencies, licenses | Dependabot, Renovate, Trivy, Grype, OSV-Scanner, Snyk | PR + daily |
| **IaC scanning** | Misconfigurations in Terraform, CloudFormation, K8s YAML, Dockerfiles | Checkov, Trivy (config), KICS, tflint, hadolint, kube-linter | PR |

| Category | What it finds | Tools (open source / common) | When |
|---|---|---|---|
| **Container image scanning** | OS package and library CVEs in images | Trivy, Grype, ECR scanning (Inspector), Docker Scout | Build + registry rescans |
| **SBOM** | Inventory of everything inside an artifact | Syft, Trivy, CycloneDX/SPDX formats | Build |
| **Signing / provenance** | Proves who built an artifact, from what | cosign (Sigstore), SLSA provenance, GitHub attestations | Build; verify at deploy |
| **Policy as code** | Organization rules on config and deployments | OPA/Conftest, Kyverno, Gatekeeper, Sentinel, AWS SCPs | PR + admission |
| **DAST** | Vulnerabilities in the running app (black-box) | OWASP ZAP, Burp Suite, Nuclei | Staging |
| **Runtime security** | Suspicious behavior in hosts/containers | Falco, GuardDuty, Tetragon | Production |
| **CSPM** | Cloud misconfigurations across accounts | AWS Security Hub, Prowler, ScoutSuite | Continuous |

**Senior advice:** start with the highest return (secret scanning, dependency updates, image scanning, IaC scanning, least-privilege IAM), make findings **actionable** (one owner, clear fix), tune false positives aggressively, and fail builds only on what you'll really fix (e.g. critical/high with a fix available).

## 14.3 Secrets management

### Rules

1. **Never** hardcode secrets in code, Dockerfiles, images, Terraform, Ansible, CI files, or Kubernetes YAML in Git.
2. Store them in a **secret manager**; fetch at runtime with an identity (IAM role), not another secret.
3. Prefer **no long-lived secrets at all**: OIDC federation for CI → cloud, IAM roles for workloads, short-lived database credentials.
4. **Rotate** automatically; scope per environment and service.
5. Detect leaks before they land (pre-commit, push protection) and after (history scans).

| Store | Notes |
|---|---|
| **AWS Secrets Manager** | Versioning, automatic rotation (Lambda; built-in for RDS), resource policies, cross-account; per-secret monthly price |
| SSM Parameter Store (SecureString) | KMS-encrypted parameters; standard tier free; no built-in rotation |
| **HashiCorp Vault / OpenBao** | Dynamic secrets (per-request DB users, AWS keys), PKI, transit encryption, leases |
| GitHub/GitLab CI secrets | For values the pipeline itself needs (prefer OIDC for cloud) |
| Kubernetes | External Secrets Operator / CSI driver / Sealed Secrets (Kubernetes notes 12.8) |

```
# Create, read and rotate-ready usage of a secret (value generated, never typed in history)
aws secretsmanager create-secret --name prod/shop/db \
  --secret-string "{\"username\":\"shop\",\"password\":\"$(openssl rand -base64 24)\"}"
aws secretsmanager get-secret-value --secret-id prod/shop/db \
  --query SecretString --output text | jq -r .username
```

```
"""Read a secret at runtime with the instance/pod IAM role (no keys in code)."""
import json

import boto3


def get_db_credentials(secret_id: str = "prod/shop/db") -> dict:
    client = boto3.client("secretsmanager")
    response = client.get_secret_value(SecretId=secret_id)
    return json.loads(response["SecretString"])
```

### Detecting secrets

```
# .pre-commit-config.yaml: scan staged changes before every commit
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.24.2
    hooks:
      - id: gitleaks
```

```
pip install pre-commit && pre-commit install       # once per clone
gitleaks git --redact -v .                          # scan the whole Git history
gitleaks dir --redact .                             # scan files in a directory
```

Enable **GitHub secret scanning and push protection** on every repository (blocks pushes containing known token formats).

### A secret leaked: the playbook

1. **Revoke/rotate immediately.** Assume it was stolen the moment it was pushed (bots scan public GitHub within minutes). Deleting the commit is **not** enough: forks, caches and clones keep it.
2. Check for misuse: CloudTrail for the key's activity ( `aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIA...` ), billing anomalies, new IAM users/roles, unusual regions.
3. Contain: remove anything created by the attacker, tighten policies.
4. Clean history if needed ( `git filter-repo` ), but only **after** rotation.
5. Postmortem: why was a long-lived key there at all? Move to OIDC/roles; add pre-commit and push protection.

## 14.4 Static application security testing (SAST)

SAST analyzes source code without running it. It catches injection (SQL, command), path traversal, unsafe deserialization, weak crypto, hardcoded credentials, SSRF patterns.

```
# VULNERABLE: SQL injection and shell injection
def find_user(cursor, name):
    cursor.execute("SELECT * FROM users WHERE name = '" + name + "'")  # ' OR '1'='1

def ping(host):
    import os
    os.system("ping -c 1 " + host)              # attacker: 8.8.8.8; rm -rf /
```

```
# FIXED: parameterized query, no shell, input validation
import ipaddress
import subprocess


def find_user(cursor, name):
    cursor.execute("SELECT * FROM users WHERE name = %s", (name,))


def ping(host):
    ipaddress.ip_address(host)                      # raises ValueError if not an IP
    subprocess.run(["ping", "-c", "1", host], check=True, timeout=5)
```

```
semgrep scan --config p/python --config p/secrets --error .   # rulesets from the registry
bandit -r src/ -ll                                              # Python, medium+ severity
```

Custom rules encode your organization's lessons:

```
# semgrep-rules/no-shell-true.yml
rules:
  - id: subprocess-shell-true
    languages: [python]
    severity: ERROR
    message: "subprocess with shell=True allows command injection; pass a list instead"
    pattern: subprocess.$FUNC(..., shell=True, ...)
```

**CodeQL** (GitHub code scanning) builds a database of the code and runs deep data-flow queries (source → sink); free for public repositories, part of GitHub Advanced Security for private ones.

## 14.5 Dependencies and software composition analysis (SCA)

Modern apps are 80–90% third-party code. SCA matches your dependencies (including transitive ones) against vulnerability databases.

| Term | Meaning |
|---|---|
| **CVE** | Unique ID for a public vulnerability ( `CVE-2021-44228` = Log4Shell) |
| **CVSS** | Severity score 0–10 (Critical ≥ 9.0, High 7.0–8.9, Medium 4.0–6.9, Low < 4) |
| **EPSS** | Probability a CVE will be exploited in the next 30 days |
| **CISA KEV** | Catalog of vulnerabilities **known to be exploited**: fix these first |
| Reachability | Whether your code actually calls the vulnerable function |
| GHSA / OSV | GitHub advisories / open vulnerability database across ecosystems |

**Prioritize by risk, not only CVSS:** known exploited (KEV) or high EPSS + internet-facing + reachable + fix available = fix now; low-severity in a dev-only tool = backlog.

### Keep dependencies current automatically

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: pip
    directory: /
    schedule:
      interval: weekly
    groups:
      minor-and-patch:
        update-types: ["minor", "patch"]   # one PR for all small updates
  - package-ecosystem: docker
    directory: /
    schedule:
      interval: weekly
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
  - package-ecosystem: terraform
    directory: /infra
    schedule:
      interval: weekly
```

**Renovate** is the more configurable alternative (auto-merge rules, dashboards, monorepos).

### Dependency hygiene

- **Lockfiles** committed ( `requirements.txt` with pinned versions/hashes, `poetry.lock` , `package-lock.json` , `go.sum` ); install with `npm ci` / `pip install --require-hashes` .
- Use a **private proxy registry** (Artifactory, Nexus, CodeArtifact) to cache and control packages.
- Watch for **typosquatting** ( `reqeusts` instead of `requests` ) and **dependency confusion** (a public package with your internal package's name): scope internal packages, configure registries explicitly.
- Remove unused dependencies: less code, fewer CVEs.
- License compliance: flag copyleft licenses (GPL/AGPL) where policy forbids them.

```
trivy fs --scanners vuln,secret,misconfig --severity HIGH,CRITICAL .
osv-scanner scan source -r .
pip-audit -r requirements.txt
```

## 14.6 Container security

| Layer | Controls |
|---|---|
| Base image | Minimal (distroless, Chainguard/Wolfi, Alpine, `-slim` ), from trusted sources, **pinned by digest**, rebuilt regularly to pick up patches |
| Build | Multi-stage (no compilers/tools in the final image), no secrets in layers (BuildKit `--mount=type=secret` ), `.dockerignore` , non-root `USER` , lint with hadolint |
| Scan | Trivy/Grype in CI (fail on fixable critical/high), registry rescans for new CVEs (ECR enhanced scanning with Inspector) |
| Registry | Private, immutable tags, lifecycle policies, access via IAM, pull only from approved registries |
| Integrity | **Sign** images (cosign), verify signatures at admission |
| Runtime | Non-root, read-only root FS, drop capabilities, seccomp, no privileged/host mounts, resource limits (Kubernetes 12.5, 12.12) |

| Layer | Controls |
|---|---|
| Detection | Falco/GuardDuty runtime monitoring for shells in containers, unexpected network connections, crypto miners |

```
trivy image --severity CRITICAL,HIGH --ignore-unfixed --exit-code 1 shop/api:1.4.2
trivy image --format cyclonedx --output sbom.cdx.json shop/api:1.4.2     # SBOM from the image
syft shop/api:1.4.2 -o spdx-json > sbom.spdx.json                        # alternative SBOM tool
grype sbom:sbom.spdx.json                                                # scan an SBOM later
```

### Signing and verifying images (Sigstore cosign)

```
# Keyless signing in CI: identity comes from the CI's OIDC token, recorded in a transparency log
cosign sign --yes 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api@sha256:3f9a...

# Verify: only accept images signed by our repository's release workflow
cosign verify 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api@sha256:3f9a... \
  --certificate-identity-regexp '^https://github.com/acme/shop-api/.github/workflows/release.yml@' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com

# Attach the SBOM as a signed attestation
cosign attest --yes --type cyclonedx --predicate sbom.cdx.json \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api@sha256:3f9a...
```

Always sign and verify **digests** (`@sha256:...`), not tags: tags can be moved. Enforce verification in the cluster with Kyverno `verifyImages` or the Sigstore policy-controller.

## 14.7 Infrastructure as Code security

IaC lets you find misconfigurations **before** they exist: public buckets, open security groups, unencrypted storage, wildcard IAM.

```
checkov -d infra/ --framework terraform --compact          # 1000+ built-in checks
trivy config infra/                                        # Terraform, K8s, Dockerfile, Helm
tflint --recursive                                         # provider-aware lint
terraform plan -out tf.plan && terraform show -json tf.plan > plan.json
checkov -f plan.json                                       # scan the plan: resolved values
```

```
# Example findings and their fixes
resource "aws_security_group_rule" "ssh" {
  type              = "ingress"
  from_port         = 22
  to_port           = 22
  protocol          = "tcp"
  cidr_blocks       = ["0.0.0.0/0"] # FINDING: SSH open to the internet -> use SSM Session Manager
  security_group_id = aws_security_group.app.id
}

resource "aws_s3_bucket_public_access_block" "logs" {
  bucket                  = aws_s3_bucket.logs.id
  block_public_acls       = true # FIX: all four settings true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Suppress a finding only with a reason, in code, reviewed like any change:
`#checkov:skip=CKV_AWS_144:Cross-region replication not required for dev logs`.

### Policy as code with OPA / Conftest

Write your organization's rules once and run them in CI against Terraform plans, Kubernetes YAML, Dockerfiles.

```
# policy/terraform.rego: deny security groups open to the world on non-web ports
package main

import rego.v1

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_security_group_rule"
    rc.change.after.type == "ingress"
    "0.0.0.0/0" in rc.change.after.cidr_blocks
    not rc.change.after.from_port in {80, 443}
    msg := sprintf("%s opens port %v to the internet", [rc.address, rc.change.after.from_port])
}

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_s3_bucket"
    not rc.change.after.tags.owner
    msg := sprintf("%s must have an 'owner' tag", [rc.address])
}
```

```
conftest test plan.json --policy policy/        # exit code 1 when any deny rule matches
```

At the cloud-account level, **AWS Service Control Policies** (Organizations) are the hardest guardrail: e.g. deny leaving the organization, deny disabling CloudTrail/GuardDuty, deny regions you don't use. HashiCorp **Sentinel** plays the same role in Terraform Cloud/Enterprise.

## 14.8 Software supply chain security

Attackers increasingly target the **build path** instead of the app: compromised dependencies, build servers, CI tokens and registries.

```
source ──► build ──► artifact ──► registry ──► deploy
  ▲          ▲          ▲           ▲           ▲
protected  hardened,  SBOM +     immutable,    verify signature
branches,  ephemeral  signature  private,      and provenance
reviews,   runners,   + prove-   scanned       before running
signed     pinned     nance
commits    actions
```

| Control | Detail |
|---|---|
| **SLSA** framework | Levels for build integrity: provenance exists (L1), generated by a hosted build platform and signed (L2), hardened, isolated builds (L3) |
| Provenance | Signed statement: which source commit, which workflow, which builder produced this artifact (GitHub artifact attestations, SLSA generator) |
| **SBOM** | Inventory (CycloneDX/SPDX) so you can answer "are we affected by the new CVE?" in minutes; required by many customers and regulations |
| **VEX** | Statement that a CVE in the SBOM does/doesn't affect the product (reduces noise) |
| Pin everything | Actions by commit SHA, base images by digest, dependencies by lockfile/hash |

| Control | Detail |
|---|---|
| CI hardening | Minimal `GITHUB_TOKEN` permissions, OIDC to cloud, environment protection, no untrusted code in privileged workflows, ephemeral runners (CI/CD notes 11.6) |
| Source protection | Branch protection/rulesets, required reviews, CODEOWNERS, signed commits, MFA for all org members |
| Scorecards | **OpenSSF Scorecard** rates repositories (yours and dependencies) on these practices |

## 14.9 Dynamic testing and application security

**DAST** attacks the running application from outside, like an attacker, finding issues that static tools can't see (misconfigured headers, auth flaws, server issues).

```
# OWASP ZAP baseline scan against staging (passive, safe); report as HTML
docker run --rm -v "$PWD:/zap/wrk" -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t https://staging.shop.example.com -r zap-report.html
```

Only scan systems you own or are authorized to test. Full active scans ( `zap-full-scan.py` ) send attack payloads: staging only.

### OWASP Top 10 (2025 edition)

| # | Category | DevOps-side controls |
|---|---|---|
| A01 | Broken Access Control | Authorization on every request, deny by default, tests for IDOR |
| A02 | Security Misconfiguration | IaC scanning, hardened baselines, no default credentials, security headers |
| A03 | Software Supply Chain Failures | SCA, SBOM, signing, pinned dependencies, hardened CI |
| A04 | Cryptographic Failures | TLS everywhere, KMS-managed keys, no weak algorithms, encrypt at rest |
| A05 | Injection | Parameterized queries, input validation, SAST |
| A06 | Insecure Design | Threat modeling, secure design patterns, rate limits |
| A07 | Authentication Failures | MFA, SSO/OIDC, session management, credential stuffing protection |
| A08 | Software or Data Integrity Failures | Signed artifacts/updates, verified deserialization, CI integrity |
| A09 | Security Logging and Alerting Failures | Audit logs, centralized logging, alerting on auth anomalies |
| A10 | Mishandling of Exceptional Conditions | Fail securely (closed), no stack traces to users, handle errors consistently |

**API security:** authentication on every endpoint, object-level authorization, rate limiting/throttling (API Gateway, WAF), schema validation, no sensitive data in URLs (they end up in logs).

**Web Application Firewall** (AWS WAF on ALB/CloudFront/API Gateway): managed rule groups (OWASP core rules, known bad inputs, IP reputation, bot control), rate-based rules. A WAF reduces risk; it doesn't replace fixing the code.

## 14.10 Cloud security on AWS

| Area | Controls |
|---|---|
| **Account structure** | AWS Organizations with separate accounts per environment/workload; Control Tower landing zone; dedicated security and log-archive accounts |
| **Identity** | IAM Identity Center (SSO) for humans, no IAM users with long-lived keys, MFA everywhere, root account locked away with MFA and no keys |
| **Least privilege** | Roles per workload, scoped resources and conditions, IAM Access Analyzer (unused access, policy validation), permission boundaries for delegated admin |
| **Guardrails** | SCPs (deny dangerous actions, restrict regions), AWS Config rules (detect non-compliant resources), resource control policies |
| **Logging** | CloudTrail organization trail (all regions, log file validation) to a locked S3 bucket in the log-archive account; VPC Flow Logs; ALB/CloudFront/S3 access logs |
| **Detection** | **GuardDuty** (threat detection: compromised instances, unusual API calls, crypto mining, EKS/runtime), **Security Hub** (aggregates findings, CIS / AWS Foundational Security Best Practices checks), Inspector (EC2/ECR/Lambda vulnerabilities), Macie (sensitive data in S3), Detective (investigation) |
| **Data protection** | KMS encryption at rest (customer-managed keys for sensitive data, key policies), TLS in transit (ACM), S3 Block Public Access at account level, backups with AWS Backup (vault lock for ransomware protection) |
| **Network** | Private subnets, security groups as allow lists, no SSH from the internet (SSM Session Manager), VPC endpoints, WAF/Shield on the edge |
| **Response** | Automated remediation (EventBridge + Lambda / Config remediation), runbooks for compromised keys and instances |

```
prowler aws --compliance cis_3.0_aws        # open-source assessment against CIS benchmarks
aws accessanalyzer list-findings \
  --analyzer-arn arn:aws:access-analyzer:ap-south-1:123456789012:analyzer/org
aws guardduty list-detectors
```

### Example: least-privilege policy with conditions

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadShopUploadsOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::acme-shop-uploads-prod/uploads/*"
    },
    {
      "Sid": "DecryptWithShopKeyViaS3Only",
      "Effect": "Allow",
      "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
      "Resource": "arn:aws:kms:ap-south-1:123456789012:key/1111-2222-3333",
      "Condition": {
        "StringEquals": {"kms:ViaService": "s3.ap-south-1.amazonaws.com"}
      }
    }
  ]
}
```

### Example SCP guardrail

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyUnusedRegions",
      "Effect": "Deny",
      "NotAction": ["iam:*", "organizations:*", "sts:*", "support:*", "cloudfront:*",
                    "route53:*", "budgets:*", "waf:*", "health:*"],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {"aws:RequestedRegion": ["ap-south-1", "us-east-1"]}
      }
    },
    {
      "Sid": "ProtectSecurityServices",
      "Effect": "Deny",
      "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
                 "guardduty:DeleteDetector", "config:StopConfigurationRecorder"],
      "Resource": "*"
    }
  ]
}
```

## 14.11 Kubernetes and runtime security

Kubernetes controls are covered in the Kubernetes notes (RBAC, Pod Security, NetworkPolicy, admission policies, secrets: 12.8, 12.12). Runtime detection adds the last layer: **Falco** watches system calls via eBPF and alerts on suspicious behavior.

```
# Falco custom rule: alert when someone opens a shell in a production container
- rule: Shell in production container
  desc: Interactive shell started in a container in a prod namespace
  condition: >
    spawned_process and container and proc.name in (bash, sh, zsh)
    and proc.tty != 0 and k8s.ns.name startswith "prod"
  output: >
    Shell in prod container (user=%user.name pod=%k8s.pod.name ns=%k8s.ns.name
    cmd=%proc.cmdline image=%container.image.repository)
  priority: WARNING
  tags: [container, shell, mitre_execution]
```

Other runtime tools: Tetragon (eBPF enforcement), GuardDuty EKS Runtime Monitoring, KubeArmor. Audit: **kube-bench** (CIS), Kubescape, Trivy `k8s` scans.

## 14.12 Vulnerability management as a process

Scanners produce thousands of findings; the process decides what gets fixed.

| Step | Practice |
|---|---|
| Inventory | Know every asset: images in use (not just built), hosts, functions, dependencies (SBOMs) |
| Detect | Continuous scanning in CI **and** of running workloads/registries (new CVEs appear for old images) |
| Prioritize | Severity + exploitability (KEV, EPSS) + exposure (internet-facing?) + reachability + data sensitivity |
| Assign | Route to the owning team automatically (tags, CODEOWNERS, service catalog) |

| Step | Practice |
|---|---|
| Fix within SLA | Example: critical 7 days, high 30 days, medium 90 days; KEV-listed critical internet-facing: 48 hours |
| Exceptions | Time-limited, approved risk acceptance with compensating controls, recorded (e.g. `.trivyignore` entries with expiry and ticket) |
| Measure | Open findings by age/severity, % within SLA, mean time to remediate, base image freshness |

The best fix is often **rebuilding on a fresh base image**: automate weekly rebuilds and redeploys so patching is routine, not a project.

## 14.13 Compliance and audit

| Framework | What it is | DevOps evidence |
|---|---|---|
| **SOC 2** | US audit of security/availability/confidentiality controls | Access reviews, change management (PRs, approvals), monitoring, incident process, backups |
| **ISO/IEC 27001** | International information security management standard | Risk register, policies, asset inventory, controls evidence |
| **PCI DSS** | Card payment data | Network segmentation, encryption, vulnerability scans, logging, quarterly reviews |
| HIPAA | US health data | Encryption, access logs, BAAs |
| **GDPR / India DPDP Act 2023** | Personal data protection | Data minimization, consent, retention limits, breach notification, data residency |
| CIS Benchmarks | Hardening baselines for OS, cloud, Kubernetes | Automated benchmark scans |

**Compliance as code:** controls implemented as policies (SCPs, Config rules, Kyverno, OPA), evidence collected automatically (CloudTrail, Git history, pipeline logs, AWS Audit Manager), continuous instead of yearly. Git-based change management (PR review + CI + automated deploy) is itself strong audit evidence.

## 14.14 A complete secure pipeline

All controls together in GitHub Actions. Cheap checks run in parallel on every PR; the image is built, scanned, given an SBOM, pushed, signed and attested only on `main`.

```yaml
name: secure-delivery
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

env:
  REGISTRY: 123456789012.dkr.ecr.ap-south-1.amazonaws.com
  IMAGE: shop/api

jobs:
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0                 # full history for secret scanning
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

  sast:
    runs-on: ubuntu-latest
    container:
      image: semgrep/semgrep:1.95.0
    steps:
      - uses: actions/checkout@v4
      - run: semgrep scan --config p/python --config semgrep-rules/ --error

  dependencies-and-iac:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aquasecurity/trivy-action@0.28.0
        with:
          scan-type: fs
          scanners: vuln,misconfig
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      - name: Policy checks on Kubernetes manifests
        env:
          VER: "0.56.0"
        run: |
          BASE=https://github.com/open-policy-agent/conftest/releases/download
          curl -sSL -o conftest.tgz "${BASE}/v${VER}/conftest_${VER}_Linux_x86_64.tar.gz"
          tar xzf conftest.tgz conftest
          ./conftest test k8s/ --policy policy/k8s/

  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest -q

  image:
    needs: [secrets, sast, dependencies-and-iac, tests]
```

```
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write                    # OIDC for AWS and keyless signing
      attestations: write
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-ecr-push
          aws-region: ap-south-1
      - uses: aws-actions/amazon-ecr-login@v2
      - uses: docker/setup-buildx-action@v3
      - id: build
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE }}:sha-${{ github.sha }}
          sbom: true                     # BuildKit SBOM attestation
          provenance: mode=max           # BuildKit SLSA provenance
      - name: Scan the pushed image
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.REGISTRY }}/${{ env.IMAGE }}@${{ steps.build.outputs.digest }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      - uses: sigstore/cosign-installer@v3
      - name: Sign the image digest (keyless)
        env:
          DIGEST: ${{ steps.build.outputs.digest }}
        run: cosign sign --yes "${REGISTRY}/${IMAGE}@${DIGEST}"
      - uses: actions/attest-build-provenance@v2
        with:
          subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE }}
          subject-digest: ${{ steps.build.outputs.digest }}
          push-to-registry: true
```

Then: the GitOps step bumps the **digest** in the config repo (CI/CD notes 11.6); Kyverno in the cluster verifies the cosign signature before admitting the pod; DAST runs against staging after deployment; production deploys need approval and pass the same verification.

Pipeline hygiene: pin third-party actions to commit SHAs in real repositories (tags shown here for readability), keep the scanners' versions pinned and updated by Dependabot, and publish results as SARIF to the code-scanning tab so developers see findings in the PR.

## 14.15 Security incident response for DevOps engineers

| Scenario | First actions |
|---|---|
| **Leaked AWS access key** | Deactivate/delete the key → CloudTrail review of its activity → remove attacker resources (check all regions: EC2, Lambda, IAM users/roles/keys) → billing check → postmortem and move to roles/OIDC |
| Compromised EC2 instance (GuardDuty finding) | Isolate (replace security group with a deny-all/forensics SG), **don't terminate** before snapshotting EBS volumes and capturing memory if needed → revoke the instance role sessions → rebuild from a clean image |
| Malicious dependency published | Check SBOMs: which artifacts include it → pin to a safe version → rebuild/redeploy → rotate secrets the build or app could reach |

| Scenario | First actions |
|---|---|
| Exposed S3 bucket | Enable Block Public Access → review access logs/CloudTrail data events for downloads → assess data exposed → legal/privacy notification process |
| Suspicious shell in a pod (Falco) | Capture details (pod spec, image digest, events) → isolate with a deny-all NetworkPolicy → snapshot → delete and redeploy → investigate image and entry point |

Principles: **contain first, preserve evidence, rotate credentials, rebuild from known-good, then learn**. Involve the security team early; follow your organization's legal/notification process for data incidents.

## 14.16 Hands-on labs

| # | Lab |
|---|---|
| 1 | Install pre-commit + gitleaks in a test repo; commit a fake key ( `AKIAIOSFODNN7EXAMPLE` is AWS's documented example) and watch it get blocked |
| 2 | Run `gitleaks git` over one of your repositories' full history |
| 3 | Write the vulnerable Python from 14.4; scan with Semgrep and Bandit; fix it and re-scan; add the custom rule |
| 4 | Add Dependabot to your GitHub repo; review and merge the PRs it opens |
| 5 | Scan an old image ( `python:3.8` ) vs `python:3.12-slim` vs a distroless image with Trivy; compare CVE counts |
| 6 | Generate an SBOM with Syft; then scan the SBOM with Grype |
| 7 | Sign an image in a local registry ( `registry:2` ) with a cosign key pair; verify it; tamper (push a new image to the same tag) and see verification by digest protect you |
| 8 | Run Checkov and Trivy config on your Terraform project; fix two findings; skip one with a written reason |
| 9 | Write the Conftest policy from 14.7; test it against a Terraform plan JSON |
| 10 | Run the ZAP baseline scan against a local app (e.g. OWASP Juice Shop in Docker on your own machine) |
| 11 | Build the 14.14 pipeline in your GitHub repo (without the AWS parts at first: push to GHCR instead) |
| 12 | (AWS, mostly free within limits; check pricing) Enable GuardDuty and Security Hub in a lab account for a few days, read the findings, then disable them |

## 14.17 Interview questions

1. What is DevSecOps, and how is it different from a security review at the end?
2. Shift left vs shield right; give tool examples for each.
3. Walk through threat modeling with STRIDE for a simple web app on AWS.
4. Where do secrets belong? How do CI pipelines get cloud access without stored keys?
5. A developer pushed an AWS key to a public repository. What do you do, in order?
6. SAST vs DAST vs SCA vs IaC scanning: what does each find, and where in the pipeline?
7. CVE, CVSS, EPSS, KEV: how do you prioritize hundreds of vulnerabilities?
8. How do you keep dependencies and base images patched continuously?
9. How do you secure a Docker image from build to runtime?
10. What is an SBOM and why do organizations require it?
11. Explain image signing with cosign; why sign digests instead of tags?

12. What is SLSA? What is build provenance?
13. Describe supply-chain attacks you know (SolarWinds, Codecov, xz) and the controls against them.
14. How would you prevent a Terraform change that opens SSH to the world from being applied?
15. What is policy as code? OPA/Conftest vs Kyverno vs SCPs.
16. Name OWASP Top 10 categories and a DevOps control for three of them.
17. How do you secure GitHub Actions workflows?
18. AWS security services: GuardDuty vs Security Hub vs Inspector vs Config vs CloudTrail vs Macie.
19. How do you design least-privilege IAM? What tools help?
20. Design guardrails for a multi-account AWS organization.
21. How do you handle a critical CVE in a base image used by 50 services?
22. What is runtime security (Falco), and what would you alert on?
23. How do you balance security gates against developer speed (false positives, exceptions)?
24. What evidence does DevOps provide for a SOC 2 or ISO 27001 audit?

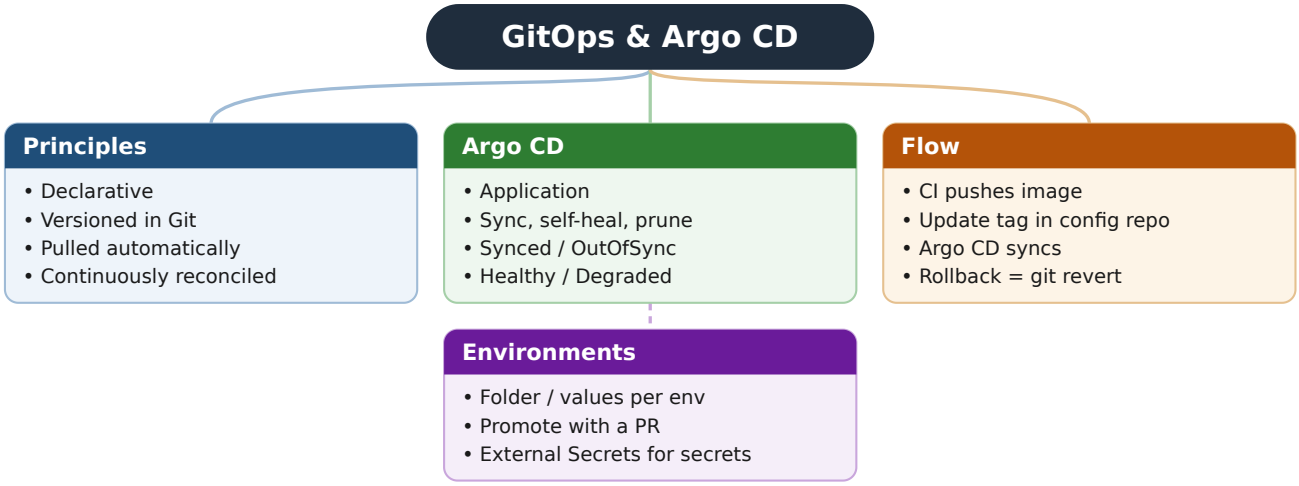
## 14.18 Self-check

- [ ] I can explain DevSecOps principles, the CIA triad and STRIDE threat modeling.
- [ ] I can place secret scanning, SAST, SCA, IaC scanning, image scanning, SBOM, signing, DAST and runtime detection in the right pipeline stage.
- [ ] I never store secrets in code and can set up a secret manager, OIDC and leak detection.
- [ ] I can run the leaked-credential playbook from memory.
- [ ] I can prioritize vulnerabilities using severity, exploitability, exposure and reachability.
- [ ] I can harden, scan, sign and verify container images.
- [ ] I can scan Terraform with Checkov/Trivy and write an OPA policy.
- [ ] I can explain supply chain security (SLSA, provenance, pinning, CI hardening).
- [ ] I can describe the AWS security service stack and multi-account guardrails.
- [ ] I can build the complete secure pipeline in 14.14.

**REMEMBER**

- Never in Git: **passwords, keys, tokens, .env files, tfstate**.
- Security is every layer: IAM → network → server → container → Kubernetes → data.

# 15. GitOps & Argo CD




## 15.0 Why GitOps

With classic CI/CD, the pipeline **pushes** changes into the cluster (`kubectl apply` or `helm upgrade` from CI). That means CI holds powerful cluster credentials, nobody knows whether the cluster still matches what was deployed (someone ran `kubectl edit` at 2 a.m.), and rebuilding a cluster means re-running dozens of pipelines.

**GitOps** makes Git the single source of truth for the desired state of the system, and an agent **inside** the cluster continuously pulls that state and makes reality match it. Every change is a commit: reviewed, audited, and revertible.

## 15.1 Principles

The OpenGitOps project (CNCF) defines four principles:

| Principle | Meaning |
|---|---|
| 1. **Declarative** | The whole desired state is expressed declaratively (Kubernetes YAML, Helm values, Kustomize) |
| 2. **Versioned and immutable** | Stored in Git (or OCI), with full history; every version can be recovered |
| 3. **Pulled automatically** | Software agents pull the desired state from the source; no external system pushes |
| 4. **Continuously reconciled** | Agents constantly compare desired vs actual state and correct drift |

### Push vs pull deployment

```
PUSH (classic CI/CD)                              PULL (GitOps)
CI ──kubectl/helm (cluster-admin creds)──► K8s    CI ──commit new tag──► config repo (Git)
                                                                          ▲  pulls / watches
cluster drift goes unnoticed                      Argo CD / Flux in the cluster ──applies──► K8s
                                                  drift detected and corrected automatically
```

| Benefit | Why |
|---|---|
| **Audit trail** | Who changed what, when, and who approved it: the Git log and PRs |
| **Rollback** | `git revert` and the cluster follows |
| **Security** | CI never needs cluster credentials; the cluster pulls outbound only |
| **Drift detection** | Manual changes are visible (OutOfSync) and can be auto-reverted (self-heal) |
| **Disaster recovery** | A new cluster + the GitOps agent pointed at the repo = everything restored |
| **Consistency** | Same process for every team, environment and cluster |

**Limits to know:** GitOps manages *desired state*, not one-off actions (database migrations need hooks or Jobs); secrets need special handling (15.8); fast "click to rollback" becomes "revert a commit", so the review flow must be quick for emergencies.

## 15.2 Repository design

### Separate application code from deployment configuration

| Repository | Contains | Changed by |
|---|---|---|
| App repo ( `shop-api` ) | Source code, Dockerfile, tests, CI workflow, maybe a Helm chart | Developers |
| **Config repo** ( `shop-gitops` ) | Kubernetes manifests / Helm values / Kustomize overlays per environment, Argo CD Applications | CI (image tag bumps), platform and app teams via PRs |

Why separate: CI writing tag bumps into the app repo would trigger more CI runs; config changes (replicas, env vars) shouldn't require an app build; access control differs (prod config needs stricter review).

### Environments: folders, not branches

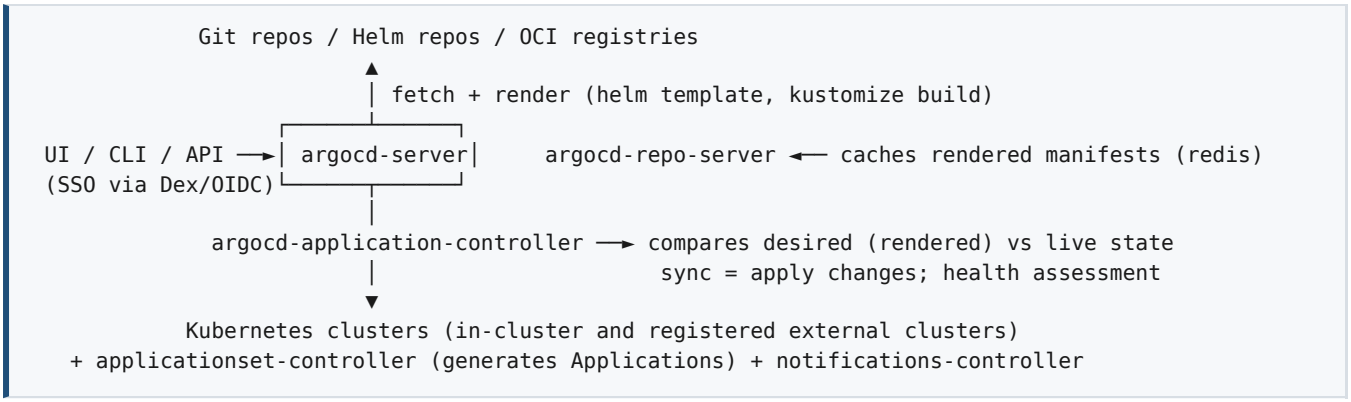
```
shop-gitops/
├── apps/
│   └── shop-api/
│       ├── base/                    # Deployment, Service, HPA... (Kustomize base)
│       └── overlays/
│           ├── dev/                 # image tag, replicas, config for dev
│           ├── staging/
│           └── prod/
├── platform/                        # cluster add-ons: ingress, cert-manager, ESO, monitoring
│   ├── cert-manager/
│   └── external-secrets/
└── clusters/
    ├── dev/                         # what runs in the dev cluster (Applications / AppSets)
    └── prod/
```

**Avoid one branch per environment** ( `dev` , `staging` , `prod` branches): merges between branches drift, cherry-picks get lost, and you can't see all environments' differences in one place. Use **one main branch with a folder per environment**; promotion = a PR changing the next environment's folder.

Monorepo (one config repo for everything) is simpler to start; split per team or per domain when access control or scale requires it.

## 15.3 Argo CD architecture

**Argo CD** (CNCF graduated) is the most widely used GitOps tool for Kubernetes, with a web UI, CLI and API.




| Component | Role |
|---|---|
| `argocd-server` | API, web UI, authentication (local users, SSO through Dex or direct OIDC) |
| `argocd-repo-server` | Clones repos and renders manifests (plain YAML, Helm, Kustomize, Jsonnet, config management plugins) |
| `argocd-application-controller` | Watches Applications, diffs desired vs live, syncs, computes health |
| `argocd-applicationset-controller` | Templates many Applications from generators |
| `argocd-notifications-controller` | Sends events to Slack, email, webhooks |
| Redis | Cache |

### Install and first login

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo          # initial admin password
kubectl -n argocd port-forward svc/argocd-server 8443:443   # UI at https://localhost:8443
argocd login localhost:8443 --username admin --insecure     # lab only: self-signed cert
argocd account update-password
kubectl -n argocd delete secret argocd-initial-admin-secret # after changing the password
```

Production installs use the **Helm chart** (`argo/argo-cd`) with HA mode, SSO, ingress with TLS, and the Argo CD configuration itself managed by Argo CD (self-management).

## 15.4 The Application resource

An **Application** connects a **source** (repo + path + revision) to a **destination** (cluster + namespace), with a sync policy.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api-prod
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io   # deleting the App deletes its resources
spec:
  project: shop
  source:
    repoURL: https://github.com/acme/shop-gitops.git
    targetRevision: main
    path: apps/shop-api/overlays/prod          # Kustomize detected automatically
  destination:
    server: https://kubernetes.default.svc     # the cluster Argo CD runs in
    namespace: shop
  syncPolicy:
    automated:
      prune: true                              # delete resources removed from Git
      selfHeal: true                           # revert manual changes in the cluster
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
      - PruneLast=true                         # prune after everything else is healthy
    retry:
      limit: 5
      backoff:
        duration: 10s
        factor: 2
        maxDuration: 3m
  ignoreDifferences:                           # fields owned by other controllers
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas                       # HPA manages replicas
  revisionHistoryLimit: 10
```

```
# A Helm chart from a chart repository, values from the config repo (multiple sources)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: cert-manager
  namespace: argocd
spec:
  project: platform
  sources:
    - repoURL: https://charts.jetstack.io
      chart: cert-manager
      targetRevision: v1.16.1                  # pinned chart version
      helm:
        valueFiles:
          - $values/platform/cert-manager/values-prod.yaml
    - repoURL: https://github.com/acme/shop-gitops.git
      targetRevision: main
      ref: values                              # referenced above as $values
  destination:
    server: https://kubernetes.default.svc
    namespace: cert-manager
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

### Status: sync and health

| Sync status | Meaning |
|---|---|
| `Synced` | Live state matches Git at the target revision |
| `OutOfSync` | Differences exist (new commit, or drift) |
| Unknown | Couldn't compare (repo error, rendering error) |

| Health status | Meaning |
|---|---|
| `Healthy` | Resources are working (Deployment rolled out, pods ready) |
| `Progressing` | Not yet healthy but making progress (rollout in progress) |
| `Degraded` | Failed (rollout failed, CrashLoopBackOff, failed Job) |
| `Suspended` | Paused (suspended CronJob, paused Rollout) |
| `Missing` | Resource in Git but not in the cluster |

Argo CD has built-in health checks for standard kinds and many CRDs; custom ones are written in Lua in `argocd-cm`.

### Sync policy choices

| Setting | Trade-off |
|---|---|
| Manual sync | A human clicks "Sync": control, but drift accumulates and it's easy to forget |
| `automated` | Commits deploy within minutes (default poll 3 minutes; Git webhooks make it near-instant) |
| `prune: true` | Removing YAML removes resources (otherwise orphans stay) |
| `selfHeal: true` | Manual `kubectl` changes are reverted; Git is enforced. Emergency change = commit (or temporarily disable auto-sync) |

```
argocd app list
argocd app get shop-api-prod                  # status, health, resources, sync result
argocd app diff shop-api-prod                 # what would change
argocd app sync shop-api-prod [--prune] [--dry-run]
argocd app history shop-api-prod
argocd app rollback shop-api-prod 12          # only with auto-sync disabled; prefer git revert
argocd app set shop-api-prod --sync-policy none   # pause automation (emergency)
argocd app delete shop-api-prod
```

**Rollback in GitOps** = `git revert <commit>` in the config repo. `argocd app rollback` is a temporary measure: with automated sync on, Argo CD would re-apply Git again, so it requires disabling auto-sync, and Git must be fixed afterwards.

## 15.5 Sync waves, phases and hooks

Order resources within a sync with the annotation `argocd.argoproj.io/sync-wave` (lower numbers first, can be negative; Argo CD waits for each wave to be healthy). **Hooks** run at specific phases.

| Phase | Use |
|---|---|
| `PreSync` | Database migration, backup before deploying |
| `Sync` | Normal resources (default) |

| Phase | Use |
|---|---|
| `PostSync` | Smoke tests, notifications after everything is healthy |
| `SyncFail` | Cleanup or alert when a sync fails |

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
    argocd.argoproj.io/sync-wave: "-1"
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          command: ["python", "-m", "app.migrate"]
```

Typical waves: `-2` namespaces and CRDs → `-1` config, secrets (ExternalSecrets) → `0` workloads → `1` ingress/routes. Migrations must be **backward-compatible** (expand → migrate → contract, CI/CD notes 11.4) because the old version keeps running during rollout.

## 15.6 Projects, RBAC and multi-tenancy

An **AppProject** limits what a team's Applications can do: which repos, which clusters/namespaces, which resource kinds.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: shop
  namespace: argocd
spec:
  description: Shop team applications
  sourceRepos:
    - https://github.com/acme/shop-gitops.git
  destinations:
    - server: https://kubernetes.default.svc
      namespace: shop
    - server: https://kubernetes.default.svc
      namespace: shop-*
  clusterResourceWhitelist: []                 # no cluster-scoped resources
  namespaceResourceBlacklist:
    - group: ""
      kind: ResourceQuota                      # quotas are owned by the platform team
  roles:
    - name: developers
      description: Shop developers can sync their apps
      policies:
        - p, proj:shop:developers, applications, get, shop/*, allow
        - p, proj:shop:developers, applications, sync, shop/*, allow
      groups:
        - shop-developers                      # SSO group
  syncWindows:
    - kind: deny                               # no automatic prod syncs during peak sale hours
      schedule: "0 18 * * *"
      duration: 4h
      applications: ["*-prod"]
```

Global RBAC lives in the `argocd-rbac-cm` ConfigMap (Casbin policy lines: `p, <subject>, <resource>, <action>, <object>, allow|deny` and `g, <group>, <role>`), with `policy.default: role:readonly` or stricter. Use **SSO groups**, not local users; disable the `admin` user after setup.

Never let application teams use the built-in `default` project in production: it allows any repo and any destination.

## 15.7 Scaling: App of Apps and ApplicationSets

### App of Apps (bootstrap pattern)

One root Application points at a folder full of Application manifests; Argo CD creates them all. Bootstrapping a cluster becomes: install Argo CD, apply one root Application.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root-prod
  namespace: argocd
spec:
  project: default                              # platform-owned bootstrap only
  source:
    repoURL: https://github.com/acme/shop-gitops.git
    targetRevision: main
    path: clusters/prod                         # contains Application/ApplicationSet YAMLs
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

## ApplicationSet

Generates Applications from **generators**, replacing hand-written copies:

| Generator | Creates one Application per… |
|---|---|
| List | Item in a static list |
| **Cluster** | Registered cluster (optionally filtered by labels): deploy add-ons to every cluster |
| **Git directory / file** | Folder (or config file) in a repo: a new folder = a new app |
| Matrix / Merge | Combination of two generators (every app × every cluster) |
| **Pull request** | Open PR: ephemeral preview environments |
| SCM provider | Repository in a GitHub/GitLab organization |

```yaml
# Every folder under apps/*/overlays/dev becomes an Application in the dev cluster
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: dev-apps
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ["missingkey=error"]
  generators:
    - git:
        repoURL: https://github.com/acme/shop-gitops.git
        revision: main
        directories:
          - path: apps/*/overlays/dev
  template:
    metadata:
      name: "{{ index .path.segments 1 }}-dev"   # apps/<name>/overlays/dev -> <name>-dev
    spec:
      project: shop
      source:
        repoURL: https://github.com/acme/shop-gitops.git
        targetRevision: main
        path: "{{ .path.path }}"
      destination:
        name: dev-cluster
        namespace: "{{ index .path.segments 1 }}"
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

```yaml
# Install the monitoring stack on every cluster labeled env=prod
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: monitoring
  namespace: argocd
spec:
  goTemplate: true
  generators:
    - clusters:
        selector:
          matchLabels:
            env: prod
  template:
    metadata:
      name: "monitoring-{{ .name }}"
    spec:
      project: platform
      source:
        repoURL: https://github.com/acme/shop-gitops.git
        targetRevision: main
        path: platform/monitoring
      destination:
        server: "{{ .server }}"
        namespace: monitoring
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
          - ServerSideApply=true
```

### Multi-cluster

`argocd cluster add <kube-context> --label env=prod` registers a cluster (Argo CD creates a ServiceAccount there and stores credentials as a Secret in `argocd`). On EKS, prefer IAM-based cluster secrets (IRSA/Pod Identity for Argo CD) over long-lived tokens. Topologies: **hub-and-spoke** (one central Argo CD manages many clusters: one pane of glass, but a big blast radius) or **one Argo CD per cluster** (isolation, more instances to run). Many organizations use one per environment tier.

## 15.8 Secrets in GitOps

Plain Kubernetes Secrets must never be committed. Options (details in Kubernetes notes 12.8):

| Approach | What's in Git |
|---|---|
| **External Secrets Operator** (recommended on AWS) | `ExternalSecret` references to Secrets Manager / Parameter Store paths |
| Sealed Secrets | `SealedSecret` encrypted with the cluster's public key |
| SOPS + KMS/age | Encrypted values; decrypted by a plugin (Argo CD CMP or Flux native SOPS support) |
| Secrets Store CSI driver | Only `SecretProviderClass` references |

Avoid plugins that inject secret values into rendered manifests inside Argo CD (they end up in Argo CD's cache and UI diffs); prefer operators that create Secrets in the cluster.

## 15.9 Image updates and environment promotion

### The end-to-end flow

```
developer PR ─► merge to main (app repo) ─► CI: test, build, scan, push image sha-3f9a1c2, sign
                                                  │
                                                  ▼ CI commits (or opens a PR) in the config repo
              apps/shop-api/overlays/dev/kustomization.yaml: newTag: sha-3f9a1c2
                                                  │
                                                  ▼ Argo CD syncs dev automatically
              tests pass in dev ─► PR: copy the same tag to staging ─► merge ─► staging syncs
              staging verified  ─► PR: copy the same tag to prod (CODEOWNERS approval) ─► prod
```

The **same image digest** moves through environments; only configuration differs. Each promotion is a reviewed, revertible commit.

```
# What CI runs in the config repo (job with a token scoped to that repo only)
cd apps/shop-api/overlays/dev
kustomize edit set image shop/api="${REGISTRY}/shop/api:${TAG}"
git commit -am "shop-api: dev -> ${TAG}"
git push                      # or: push a branch and open a PR for staging/prod
```

### Automation options

| Tool | How |
|---|---|
| CI commit / PR (above) | Simple, explicit, works with any tool |
| **Argo CD Image Updater** | Watches the registry for new tags matching a rule (semver, latest build), writes the new tag back to Git (or parameter overrides) |
| **Kargo** | Promotion orchestration on top of Argo CD: "freight" (image + config versions) moves through stages with verification |
| Flux image automation | `ImageRepository` + `ImagePolicy` + `ImageUpdateAutomation` commit tag bumps to Git |
| Renovate | Opens PRs for new image/chart versions in the config repo |

## 15.10 Progressive delivery with Argo Rollouts

A Kubernetes Deployment only does rolling updates. **Argo Rollouts** replaces the Deployment with a `Rollout` resource that supports **canary** and **blue-green** with automated analysis of metrics, and automatic rollback.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: shop-api
  namespace: shop
spec:
  replicas: 5
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app.kubernetes.io/name: shop-api
  template:
    metadata:
      labels:
        app.kubernetes.io/name: shop-api
    spec:
      containers:
        - name: app
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          ports:
            - containerPort: 8080
  strategy:
    canary:
      canaryService: shop-api-canary           # Services managed by Rollouts
      stableService: shop-api-stable
      trafficRouting:
        nginx:
          stableIngress: shop-api               # or ALB, Istio, Gateway API plugin...
      analysis:                                 # runs in the background during the canary
        templates:
          - templateName: success-rate
        startingStep: 1
        args:
          - name: service
            value: shop-api-canary
      steps:
        - setWeight: 10
        - pause:
            duration: 5m
        - setWeight: 30
        - pause:
            duration: 5m
        - setWeight: 60
        - pause:
            duration: 10m
        # then 100%: promotion complete
---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
  namespace: shop
spec:
  args:
    - name: service
  metrics:
    - name: success-rate
      interval: 1m
      count: 5
      failureLimit: 1                           # more than one failed measurement = abort
      successCondition: result[0] >= 0.99
      provider:
        prometheus:
          address: http://prometheus-operated.monitoring:9090
          query: |
            sum(rate(http_requests_total{service="{{args.service}}",status!~"5.."}[2m]))
```

```
            /
            sum(rate(http_requests_total{service="{{args.service}}"}[2m]))
```

If the success rate drops below 99%, the Rollout **aborts** and shifts all traffic back to stable automatically. Blue-green: `strategy.blueGreen` with `activeService`, `previewService`, optional pre/post-promotion analysis and `autoPromotionEnabled: false` for a manual switch.

```
kubectl argo rollouts get rollout shop-api -n shop --watch
kubectl argo rollouts promote shop-api -n shop       # skip the current pause
kubectl argo rollouts abort shop-api -n shop         # back to stable
```

Alternative: **Flagger** (works with Flux or Argo CD, with service meshes and ingress controllers).

## 15.11 Notifications

The notifications controller sends events (sync succeeded/failed, health degraded, new deployment) to Slack, Teams, email, GitHub commit statuses, webhooks.

```
# Subscribe one Application to Slack messages (triggers and templates come from the catalog)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api-prod
  namespace: argocd
  annotations:
    notifications.argoproj.io/subscribe.on-sync-failed.slack: shop-deploys
    notifications.argoproj.io/subscribe.on-health-degraded.slack: shop-deploys
    notifications.argoproj.io/subscribe.on-deployed.slack: shop-deploys
spec:
  project: shop
  source:
    repoURL: https://github.com/acme/shop-gitops.git
    targetRevision: main
    path: apps/shop-api/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
```

The Slack token lives in the `argocd-notifications-secret` Secret (populated from your secret store), never in Git. Also send deploy events to Grafana as annotations so dashboards show "what changed".

## 15.12 Flux CD

**Flux** (CNCF graduated) is the other major GitOps tool: a set of Kubernetes controllers (the GitOps Toolkit), no built-in UI, very Kubernetes-native and composable.

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: shop-gitops
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/acme/shop-gitops.git
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: shop-api-prod
  namespace: flux-system
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: shop-gitops
  path: ./apps/shop-api/overlays/prod
  prune: true
  wait: true                                    # wait for resources to be ready
  timeout: 5m
---
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: jetstack
  namespace: flux-system
spec:
  interval: 1h
  url: https://charts.jetstack.io
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: cert-manager
  namespace: cert-manager
spec:
  interval: 30m
  chart:
    spec:
      chart: cert-manager
      version: v1.16.1
      sourceRef:
        kind: HelmRepository
        name: jetstack
        namespace: flux-system
  values:
    crds:
      enabled: true
```

```
flux bootstrap github --owner=acme --repository=shop-gitops --path=clusters/prod
flux get kustomizations
flux reconcile kustomization shop-api-prod --with-source
flux logs --level=error
```

| | **Argo CD** | **Flux** |
|---|---|---|
| UI | Rich web UI (big adoption driver) | None built in (Weave GitOps / Headlamp / Capacitor plugins) |

| | Argo CD | Flux |
|---|---|---|
| Model | `Application` / `ApplicationSet` | `GitRepository` / `OCIRepository` + `Kustomization` / `HelmRelease` |
| Helm | Renders with `helm template` (no Helm release records) | Real Helm releases (helm list works, Helm hooks native) |
| Multi-tenancy | AppProjects + RBAC + SSO in Argo CD | Kubernetes RBAC + service account impersonation per Kustomization |
| Multi-cluster | Hub-and-spoke from one instance is easy | Usually Flux per cluster |
| Secrets | Via operators/plugins | Native SOPS decryption |
| Image automation | Image Updater (separate) | Built-in controllers |
| Typical choice | Teams wanting visibility and self-service UI | Platform teams wanting composable, API-only GitOps |

Both are solid; pick one per organization. The concepts (repo layout, promotion, secrets, drift) are the same.

## 15.13 Operating GitOps in production

| Area | Practice |
|---|---|
| Availability | Argo CD HA manifests/Helm values (multiple replicas of server/repo-server, Redis HA); controller sharding for many clusters |
| Performance | Git webhooks instead of polling; resource exclusions for noisy CRDs; increase repo-server parallelism for large monorepos; `ApplyOutOfSyncOnly=true` for huge apps |
| Upgrades | Read release notes (Argo CD 3.x changed defaults: annotation-based resource tracking, stricter RBAC for logs and sub-resources); upgrade one minor at a time; manage Argo CD with Argo CD |
| Security | SSO only, `admin` disabled, AppProjects per team, restricted cluster-scoped resources, repo credentials as scoped deploy keys/GitHub App, network access to the API restricted, audit Git and Argo CD events |
| Disaster recovery | Everything is in Git: new cluster → install Argo CD → apply root app. Back up Argo CD's own config ( `argocd admin export` ) for credentials/cluster registrations not in Git |
| Break-glass | Documented emergency procedure: disable auto-sync for one app, apply the fix, then commit the same fix to Git and re-enable |
| Policy | Validate manifests in the config repo's CI (kubeconform, Kyverno CLI, conftest) so bad YAML never reaches Argo CD |
| Drift reporting | Alert on apps `OutOfSync` or `Degraded` for more than N minutes (Argo CD exposes Prometheus metrics: `argocd_app_info{sync_status,health_status}` ) |

```
# Applications degraded or out of sync (alert if this stays > 0 for 15 minutes)
count by (name, project) (argocd_app_info{health_status!="Healthy"})
count by (name, project) (argocd_app_info{sync_status="OutOfSync"})
```

## 15.14 Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| `ComparisonError` / `Unknown` | Repo unreachable or credentials wrong; Helm/Kustomize render failure; bad path | `argocd app get` message; test `kustomize build` / `helm template` locally; check repo-server logs |
| Always `OutOfSync` right after syncing | Fields mutated by controllers/webhooks (replicas by HPA, defaults added by admission, CRD defaulting) | `argocd app diff`; `ignoreDifferences` (+ `RespectIgnoreDifferences=true`), ServerSideApply, normalize manifests |
| Sync succeeded but app `Degraded` | Pods failing (image, config, probes) | Kubernetes troubleshooting (12.18) on the resources Argo CD shows |
| `Progressing` forever | Rollout waiting on readiness, a LoadBalancer Service waiting for an address, missing health check for a CRD | Inspect the resource; add a custom health check |
| Sync stuck "waiting for completion of hook" | PreSync Job failing or hanging | Job logs; `hook-delete-policy`; terminate the operation (`argocd app terminate-op`) |
| Resources not deleted | `prune` disabled, or resource has `Prune=false` annotation | Enable prune; sync with `--prune` |
| Manual changes keep reverting | `selfHeal: true` is working as designed | Change Git instead |
| Application won't delete | Finalizer waiting for resource deletion | Check the resources/their finalizers; remove the App finalizer only knowingly |
| `permission denied` / "not permitted in project" | AppProject `sourceRepos` / `destinations` /whitelists or RBAC | Update the project or RBAC policy |
| Changes take minutes to appear | Polling interval (3 min default) | Configure Git webhooks to `argocd-server` |
| Secret values missing | ExternalSecret not syncing (store auth, wrong key) | `kubectl describe externalsecret`; ESO logs |

## 15.15 Hands-on labs (kind: free)

| # | Lab |
|---|---|
| 1 | Install Argo CD in kind; log in via port-forward and CLI; change the admin password |
| 2 | Create your own config repo with a Kustomize base and dev/prod overlays for a small app; create an Application for dev (manual sync), sync it |
| 3 | Turn on automated sync with prune and self-heal; `kubectl scale` the Deployment manually and watch Argo CD revert it; delete a YAML file and watch prune |
| 4 | Promote: change the image tag in dev via commit, then promote to prod via a PR; roll back with `git revert` |
| 5 | Add a PreSync migration Job and sync waves; observe the order in the UI |
| 6 | Create an AppProject restricting repos and namespaces; try to deploy outside it and read the error |

| # | Lab |
|---|---|
| 7 | Replace individual Applications with an App of Apps, then with a Git-directory ApplicationSet; add a new app by adding a folder |
| 8 | Install a Helm chart (e.g. cert-manager) as a multi-source Application with values from your repo |
| 9 | Install Argo Rollouts; convert the Deployment to a canary Rollout; promote and abort manually |
| 10 | Add External Secrets with a fake/local provider or Sealed Secrets so no plain Secret is in Git |
| 11 | Repeat lab 2–3 with Flux (`flux bootstrap` against your GitHub repo) and compare the experience |
| 12 | Disaster drill: delete the kind cluster, recreate it, install Argo CD, apply the root app; time the full recovery |

## 15.16 Interview questions

1. What is GitOps? State the four OpenGitOps principles.
2. Push-based CI/CD vs pull-based GitOps: security and operational differences.
3. Why separate application and configuration repositories?
4. Why are environment branches an anti-pattern? How do you model environments?
5. Explain Argo CD's architecture and what each component does.
6. What does an Application define? Sync status vs health status.
7. What do `prune` and `selfHeal` do, and what are the risks of each?
8. How do you roll back in a GitOps world? Why is `argocd app rollback` not enough?
9. An app is always OutOfSync after sync: causes and fixes.
10. How do you run database migrations with Argo CD (hooks, waves, backward compatibility)?
11. What is an AppProject? How do you make Argo CD safe for many teams?
12. App of Apps vs ApplicationSet; name ApplicationSet generators and use cases.
13. How do you manage add-ons across 20 clusters?
14. How do you handle secrets with GitOps?
15. Describe the full flow from a developer's merge to production with GitOps, including promotion.
16. How does Argo Rollouts canary with analysis work? What happens when metrics are bad?
17. Argo CD vs Flux: differences and when you'd choose each.
18. How do you handle an emergency fix when self-heal reverts manual changes?
19. How do you monitor Argo CD and alert on failed deployments?
20. How would you rebuild a production cluster from scratch with GitOps, and how long should it take?

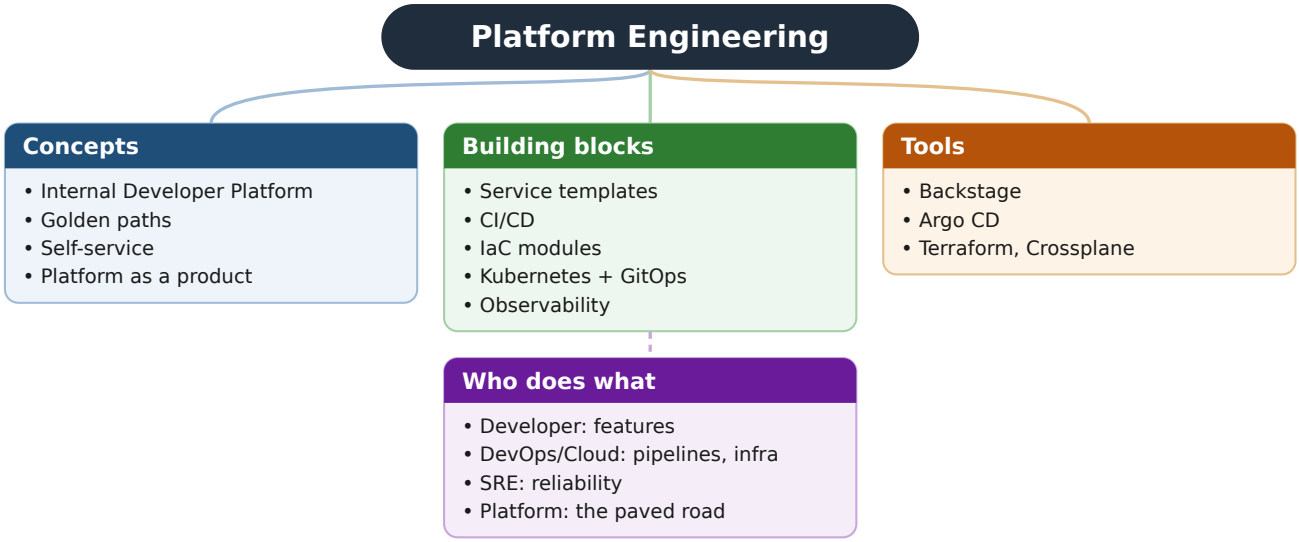
## 15.17 Self-check

- [ ] I can explain GitOps principles and the push vs pull model with its security benefits.
- [ ] I can design a config repository with environment folders and a promotion process.
- [ ] I can install Argo CD and write Applications for Kustomize, Helm and multi-source setups.
- [ ] I can choose sync options (automated, prune, self-heal, waves, hooks, ignoreDifferences) deliberately.
- [ ] I can secure multi-tenancy with AppProjects, RBAC and SSO.
- [ ] I can bootstrap clusters with App of Apps and scale with ApplicationSets.
- [ ] I can keep secrets out of Git with ESO or Sealed Secrets.
- [ ] I can implement canary releases with Argo Rollouts and metric analysis.
- [ ] I can compare Argo CD and Flux and troubleshoot common sync and health problems.

**REMEMBER**

- **Git is the source of truth.** Nobody runs `kubectl apply` on production by hand.

# 16. Platform Engineering




## 16.0 Why platform engineering

"You build it, you run it" gave teams ownership, but also handed every developer Kubernetes, Terraform, IAM, CI pipelines, observability, security scanners and cost management. The result in many companies: **cognitive overload**, every team reinventing the same pipelines slightly differently, inconsistent security, and senior engineers spending their time on plumbing.

**Platform engineering** builds an **Internal Developer Platform (IDP)**: a curated, self-service layer of tools, workflows and infrastructure that lets application teams ship and run software **without becoming infrastructure experts**, while the organization gets standardization, security and cost control by default. It is the natural career growth path for DevOps engineers.

## 16.1 Core concepts

| Term | Meaning |
|---|---|
| **Internal Developer Platform (IDP)** | The whole set of self-service capabilities (infrastructure, CI/CD, environments, observability, security) glued together for developers |
| **Internal developer portal** | The front door / UI of the platform (e.g. Backstage): catalog, templates, docs, links. A portal is **not** the platform; it's one interface to it |
| **Platform as a product** | The platform team treats developers as customers: user research, roadmap, documentation, support, adoption metrics. Not a ticket queue |
| **Golden path / paved road** | The opinionated, supported, well-documented way to do a common task (create a service, add a database), fast and compliant by default. Optional, but so convenient most teams choose it |
| **Self-service** | Developers get what they need through a portal, CLI, API or Git PR, in minutes, without filing tickets |
| **Cognitive load** | The mental effort a team spends; the platform absorbs the *extraneous* load (infra plumbing) so teams focus on their product |

| Term | Meaning |
|---|---|
| **Thinnest viable platform** | Start with the smallest platform that removes real pain (sometimes just docs + templates + one pipeline); grow from demand |
| Escape hatch | A supported way to go off the golden path when there's a real need, with clear responsibility |

### Team Topologies (the organizational model behind platforms)

| Team type | Purpose |
|---|---|
| **Stream-aligned** | Delivers business value end-to-end (checkout team, search team) |
| **Platform** | Provides internal services that reduce stream-aligned teams' cognitive load |
| **Enabling** | Experts who coach teams to adopt new skills (e.g. SRE practices, security), then move on |
| Complicated-subsystem | Owns a part needing deep specialist knowledge (video codec, ML model serving) |

Interaction modes: **X-as-a-Service** (the goal for a mature platform: consume without talking to the platform team), **Collaboration** (working closely for a period, e.g. building a new capability with a pilot team), **Facilitating** (enabling teams teaching).

### DevOps vs SRE vs Platform Engineering

| | DevOps | SRE | Platform Engineering |
|---|---|---|---|
| What | Culture and practices: collaboration, automation, ownership | Reliability engineering with SLOs and error budgets | Building internal products for developer self-service |
| Main output | Automated delivery, shared responsibility | Reliable production, incident practice | The IDP, golden paths |
| Relationship | The philosophy | One implementation of operations | One implementation of DevOps at scale |

They coexist: a platform team provides the paved road (often including SRE tooling), stream-aligned teams own their services on it.

## 16.2 Platform capabilities

Based on the CNCF Platforms white paper, a complete platform provides these capabilities through consistent interfaces:

| Capability | Examples of what developers get |
|---|---|
| Infrastructure provisioning | A database, queue, bucket or cache via template/PR/API (Terraform modules, Crossplane, ACK) |
| Compute and runtime | Kubernetes namespaces with quotas, or serverless/containers (EKS, ECS, Lambda) |
| Application environments | dev/staging/prod plus ephemeral preview environments per PR |
| Build, test, delivery | Reusable CI workflows, GitOps deployment, progressive delivery |
| Observability | Dashboards, logs, traces and SLO alerts automatically for every new service |

| Capability | Examples of what developers get |
|---|---|
| Security | Scanning in pipelines, signed images, policies, identity (Pod Identity/IRSA), secret store integration |
| Secrets and configuration | ExternalSecrets wired to Secrets Manager; config per environment |
| Messaging and data | Approved Kafka/SQS/RDS patterns with backup and encryption |
| Identity and access | SSO, team-based RBAC, least-privilege roles created automatically |
| Developer portal and docs | Service catalog, ownership, API docs, TechDocs, scorecards |
| Cost visibility | Cost per service/team (tags, Kubecost/OpenCost), budgets and alerts |

**Interfaces:** portal (UI forms), CLI, API, and **Git** (a PR to a config repo is often the best self-service API: reviewed, audited, GitOps-applied).

## 16.3 Golden paths

A golden path for "create a new backend service" produces, in minutes:

```
developer fills a template form (name, team, language, needs a database? public API?)
   │
   ▼ scaffolder
1. Git repository from a template: app skeleton, Dockerfile, tests, README, CODEOWNERS,
   catalog-info.yaml, reusable CI workflow reference, Dependabot config
2. Config repo PR: Kustomize/Helm values for dev/staging/prod, Argo CD Application
3. Infrastructure claim: e.g. PostgreSQL (Crossplane claim / Terraform module PR)
4. Secrets wiring: ExternalSecret pointing at the generated database credentials
5. Observability: ServiceMonitor, default dashboard, default SLO + burn-rate alerts
6. Security: image scanning, signing and policy checks inherited from the shared CI workflow
7. Catalog registration: owner, system, links (repo, CI, Argo CD, dashboards, runbooks)
   │
   ▼
first deployment to dev runs automatically: "time to first deploy" measured in minutes
```

**Properties of a good golden path:** optional but attractive; opinionated defaults (security, observability) built in; documented end to end; maintained (templates updated centrally, existing services upgraded through reusable workflows/modules rather than copy-paste); measured (adoption, time saved).

**Reusable building blocks** carry updates to all services: reusable CI workflows ( `uses: acme/platform/.github/workflows/build.yml@v3` ), Helm library charts, Terraform modules with versions, Kyverno policies, base images.

## 16.4 Backstage: the developer portal

**Backstage** (created by Spotify, CNCF incubating) is the most widely adopted open-source portal framework. Core features:

| Feature | What it does |
|---|---|
| **Software Catalog** | Inventory of all software: services, websites, libraries, APIs, resources, with owners and relationships, read from `catalog-info.yaml` files in repos |
| **Software Templates** (Scaffolder) | Forms that create repos, PRs, and call APIs: the golden path engine |

| Feature | What it does |
|---|---|
| **TechDocs** | Docs-as-code (Markdown in the repo with MkDocs) rendered in the portal |
| Search | Across catalog, docs, and plugins |
| **Plugins** | Kubernetes, Argo CD, GitHub Actions, PagerDuty, Grafana, SonarQube, cost, security… shown on each entity's page |
| Scorecards (via plugins) | Production readiness checks per service (has owner, runbook, SLO, no critical CVEs) |

## Catalog entities

```
# catalog-info.yaml in the shop-api repository
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: shop-api
  description: Orders and checkout API
  annotations:
    github.com/project-slug: acme/shop-api
    backstage.io/techdocs-ref: dir:.
    argocd/app-name: shop-api-prod
    backstage.io/kubernetes-id: shop-api
    pagerduty.com/service-id: P1234AB
  tags: [python, fastapi]
  links:
    - url: https://grafana.example.com/d/shop-api
      title: Dashboard
    - url: https://wiki.example.com/runbooks/shop-api
      title: Runbook
spec:
  type: service
  lifecycle: production
  owner: group:shop-team
  system: shop
  providesApis: [shop-orders-api]
  dependsOn:
    - resource:shop-orders-db
---
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: shop-orders-api
spec:
  type: openapi
  lifecycle: production
  owner: group:shop-team
  system: shop
  definition:
    $text: ./openapi.yaml
---
apiVersion: backstage.io/v1alpha1
kind: Resource
metadata:
  name: shop-orders-db
spec:
  type: database
  owner: group:shop-team
  system: shop
```

Entity kinds: `Component` (service, website, library), `API`, `Resource` (database, bucket), `System` (group of components), `Domain` (business area), `Group` and `User` (usually synced from the identity provider),

`Template`, `Location`.

### Software template (scaffolder)

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: python-service
  title: Python service (golden path)
  description: FastAPI service with CI, GitOps deployment, observability and security defaults
spec:
  owner: group:platform-team
  type: service
  parameters:
    - title: Service details
      required: [name, owner]
      properties:
        name:
          type: string
          pattern: "^[a-z][a-z0-9-]{2,30}$"
          description: Lowercase name, e.g. orders-api
        owner:
          type: string
          ui:field: OwnerPicker
          ui:options:
            catalogFilter:
              kind: Group
        database:
          type: boolean
          default: false
          title: Needs a PostgreSQL database
  steps:
    - id: fetch
      name: Generate the repository content
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
          owner: ${{ parameters.owner }}
          database: ${{ parameters.database }}
    - id: publish
      name: Create the GitHub repository
      action: publish:github
      input:
        repoUrl: github.com?owner=acme&repo=${{ parameters.name }}
        defaultBranch: main
        protectDefaultBranch: true
    - id: register
      name: Register in the catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps.publish.output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml
  output:
    links:
      - title: Repository
        url: ${{ steps.publish.output.remoteUrl }}
      - title: Open in catalog
        icon: catalog
        entityRef: ${{ steps.register.output.entityRef }}
```

Further steps typically open a PR in the GitOps repo (`publish:github:pull-request`) adding the service's Argo CD Application and, if `database` is true, a database claim.

**Running Backstage** is real engineering: it's a TypeScript/React app you build and deploy yourself (plugins are code). Alternatives: **Roadie** (hosted Backstage), **Port**, **Cortex**, **OpsLevel**, **Atlassian Compass**: less flexible, much less maintenance.

## 16.5 Self-service infrastructure

| Approach | How developers request | Notes |
|---|---|---|
| **Terraform modules + PR** | Add a module block (from the internal registry) in an infra repo; CI plans, reviewers approve, Atlantis/HCP Terraform/Spacelift applies | Most common; familiar; review-based |
| **Crossplane** | Apply a simple Kubernetes object (a claim/composite resource) in their namespace; Crossplane creates cloud resources and keeps reconciling them | Kubernetes-native; GitOps-friendly; drift corrected continuously |
| AWS Controllers for Kubernetes (ACK) | Kubernetes CRDs mapping 1:1 to AWS resources | Low abstraction; often used inside Crossplane-like compositions or by platform teams |
| Portal action calling an API | Template step triggers a pipeline | Good UX; the pipeline still uses one of the above |

### Crossplane in brief

```
Platform team defines:  CompositeResourceDefinition (XRD) = the API
                          ("PostgreSQLInstance" with size, version)
                        Composition = how to build it on AWS (RDS instance, subnet group,
                          security group, secret) using providers (provider-aws) and functions
Developer applies:      a small resource of that kind in their namespace ⟶ Crossplane creates
                        everything and writes connection details into a Secret for the app
```

```
# The platform API: what developers may set (size and version only)
apiVersion: apiextensions.crossplane.io/v1
kind: CompositeResourceDefinition
metadata:
  name: xpostgresqlinstances.platform.acme.io
spec:
  group: platform.acme.io
  names:
    kind: XPostgreSQLInstance
    plural: xpostgresqlinstances
  claimNames:
    kind: PostgreSQLInstance
    plural: postgresqlinstances
  versions:
    - name: v1alpha1
      served: true
      referenceable: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                size:
                  type: string
                  enum: [small, medium, large]   # mapped to instance classes by the platform
                version:
                  type: string
                  enum: ["16", "17"]
              required: [size]
```

```
# What a developer writes (in the GitOps repo, applied by Argo CD)
apiVersion: platform.acme.io/v1alpha1
kind: PostgreSQLInstance
metadata:
  name: orders-db
  namespace: shop
spec:
  size: small
  version: "16"
  writeConnectionSecretToRef:
    name: orders-db-conn                    # host, port, username, password for the app
```

The Composition (owned by the platform team) maps `small` to `db.t4g.medium`, forces encryption, private subnets, backups, deletion protection in prod and tags for cost: developers can't get it wrong. Crossplane **v2** (2025) makes composite resources namespaced directly (claims become optional/legacy) and builds compositions from **functions** pipelines; the concepts stay the same.

## 16.6 Environments and multi-tenancy

| Model | Isolation | Cost / overhead | Use |
|---|---|---|---|
| **Namespace per team/app** (with RBAC, quotas, NetworkPolicies, Pod Security) | Soft | Lowest | Most internal workloads |
| Virtual clusters (**vcluster**) | Each team gets its own API server inside a namespace | Low–medium | Teams needing CRDs/cluster-admin-like freedom, CI test clusters |

| Model | Isolation | Cost / overhead | Use |
|---|---|---|---|
| **Cluster per environment** | Strong between dev/staging/prod | Medium | Standard baseline |
| Cluster per team / per tenant | Strongest | Highest | Regulated or untrusted tenants |
| Account per environment/workload (AWS) | Hard boundary for IAM, billing, quotas | Managed by landing zone | AWS best practice |

**Ephemeral preview environments:** each PR gets a namespace (or vcluster) with the app deployed (ApplicationSet pull-request generator), a URL posted on the PR, and automatic deletion on merge/close. Seed data and stubbed dependencies keep them cheap; TTL controllers prevent leaks.

## 16.7 Designing platform abstractions

The art is choosing **what to hide and what to expose**:

| Too low (leaky) | Right level | Too high (rigid) |
|---|---|---|
| Developers write 400 lines of Kubernetes YAML and IAM JSON | Developers declare intent: image, port, CPU/memory size, public or internal, needs Postgres; platform generates the rest with secure defaults | A single "deploy" button that can't express health checks, scaling or a second container |

Patterns:

- **App specification** owned by developers, rendered by the platform: a Helm umbrella chart with a small values file, a Kustomize component library, a CUE/KCL schema, or **Score** (a workload spec translated to Compose/Kubernetes/Helm).
- **Sensible defaults + documented overrides** (escape hatches) for the 10% of cases that need them.
- **Version your platform APIs** (`v1alpha1` → `v1`) and give migration paths; never break hundreds of services silently.

```
# Example developer-facing app spec (rendered by the platform's chart/templates)
service: orders-api
team: shop
image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/orders-api
port: 8080
size: medium                  # platform maps to requests/limits and HPA bounds
exposure: public              # public -> Ingress + WAF + TLS; internal -> ClusterIP only
healthPath: /healthz
slo:
  availability: 99.9
dependencies:
  postgres: orders-db
```

## 16.8 Governance, security and cost by default

| Concern | Platform mechanism |
|---|---|
| Security | Shared pipelines with scanning/signing, Kyverno/OPA policies, Pod Security `restricted` namespaces, workload identity provisioned automatically |
| Compliance | Policies as code, audit trails from Git and CI, scorecards for production readiness |

| Concern | Platform mechanism |
|---|---|
| Reliability | Default probes, PDBs, spread constraints, SLOs and alerts in templates |
| **Cost (FinOps)** | Mandatory tags/labels ( `team` , `service` , `env` ) enforced by policy; showback dashboards (Kubecost/OpenCost, AWS Cost Explorer by tag); budgets and anomaly alerts; right-sizing recommendations (VPA, Compute Optimizer); auto-shutdown of non-prod at night; spot/Graviton by default where safe |
| Lifecycle | Deprecation of old runtimes and versions tracked in the catalog; automated upgrade PRs |

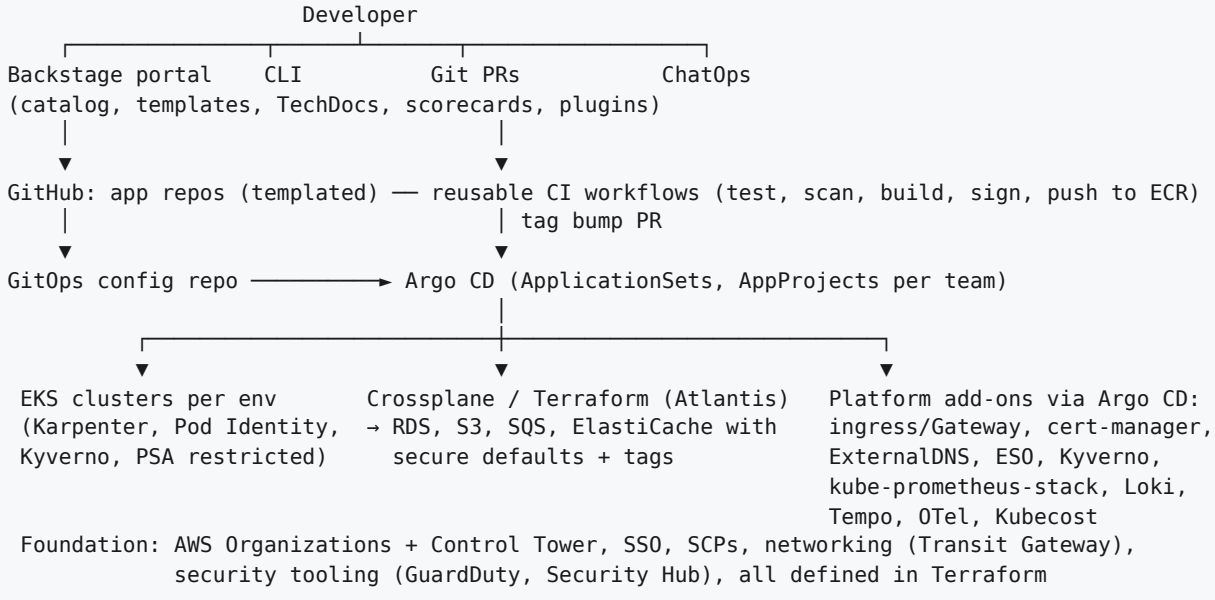
**Production readiness scorecard** example (shown per service in the portal): owner defined · runbook linked · SLO defined · alerts route to on-call · on supported base image · no critical CVEs older than 7 days · on the latest reusable CI workflow major version · resource requests set · PDB present.

## 16.9 Measuring the platform

| Category | Metrics |
|---|---|
| Delivery (DORA) | Deployment frequency, lead time for changes, change failure rate, time to restore |
| Developer experience | Time to first deploy for a new service, time to provision an environment/database, waiting time for tickets, onboarding time for new hires |
| Adoption | % of services on golden paths / reusable workflows, active portal users, template runs |
| Satisfaction | Developer surveys (NPS / satisfaction), qualitative interviews (SPACE and DevEx frameworks) |
| Reliability and efficiency | Platform availability (it's a production system with its own SLOs), cost per service, toil removed |

Run the platform like a product: a **roadmap** driven by developer pain points (interviews, surveys, ticket analysis), clear **documentation**, a support channel with SLAs, **release notes** for platform changes, and deprecation policies.

## 16.10 Reference architecture: an IDP on AWS



## 16.11 Building a platform: approach and anti-patterns

### A pragmatic roadmap

1. **Discover:** interview teams, map the current path to production, measure lead time and pain points (where do they wait? what do they copy-paste?).
2. **Thinnest viable platform:** one golden path for the most common service type: template + reusable pipeline + GitOps deploy + default observability.
3. **Pilot** with one or two friendly teams (collaboration mode); iterate on their feedback.
4. **Productize:** docs, portal/catalog, support model, versioning; measure adoption and time to first deploy.
5. **Expand capabilities** by demand: databases, preview environments, scorecards, cost views.
6. **Mature** (CNCF platform maturity model): from provisional/ad hoc → operational → scalable → optimizing, across investment, adoption, interfaces, operations and measurement.

### Anti-patterns

| Anti-pattern | Better |
|---|---|
| Building a portal first ("Backstage project") with nothing behind it | Build the paved road first; add a portal when there's something to expose |
| Platform as a ticket queue ("ask the platform team for a namespace") | Self-service by default |
| Mandating the platform without making it better than alternatives | Make the golden path the easiest path; earn adoption |
| Hiding everything behind a rigid abstraction | Right-level abstractions with escape hatches |
| Copy-paste templates that drift forever | Centrally versioned reusable workflows, modules and charts |
| No product management or metrics | Treat as a product: users, roadmap, feedback, adoption metrics |
| Big-bang migration of all teams | Incremental, starting with new services and willing teams |

## 16.12 Hands-on labs

| # | Lab |
|---|---|
| 1 | Write a one-page "path to production" map for a sample app: every step, tool and wait time; mark which steps a platform could remove |
| 2 | Create a GitHub **template repository** with app skeleton, Dockerfile, tests, CODEOWNERS, Dependabot and `catalog-info.yaml` |
| 3 | Turn your CI pipeline into a **reusable workflow** in a separate "platform" repo; call it from two app repos |
| 4 | Build a small Helm chart that takes the developer app spec from 16.7 and renders Deployment, Service, HPA, PDB, ServiceMonitor with secure defaults |
| 5 | Run Backstage locally ( `npx @backstage/create-app@latest` ); register your repo via `catalog-info.yaml` ; add TechDocs |
| 6 | Add the template from 16.4 to Backstage (with a simple skeleton) and create a new service from it |

| # | Lab |
|---|---|
| 7 | Install Crossplane in kind with a provider (e.g. provider-kubernetes or provider-aws against a lab account with strict cleanup); create an XRD + Composition; claim a resource |
| 8 | Add an ApplicationSet pull-request generator to create preview environments per PR in kind |
| 9 | Write Kyverno policies enforcing `team` labels and resource requests; show the "cost by team" with OpenCost |
| 10 | Define a production readiness scorecard and score three of your lab services manually |

## 16.13 Interview questions

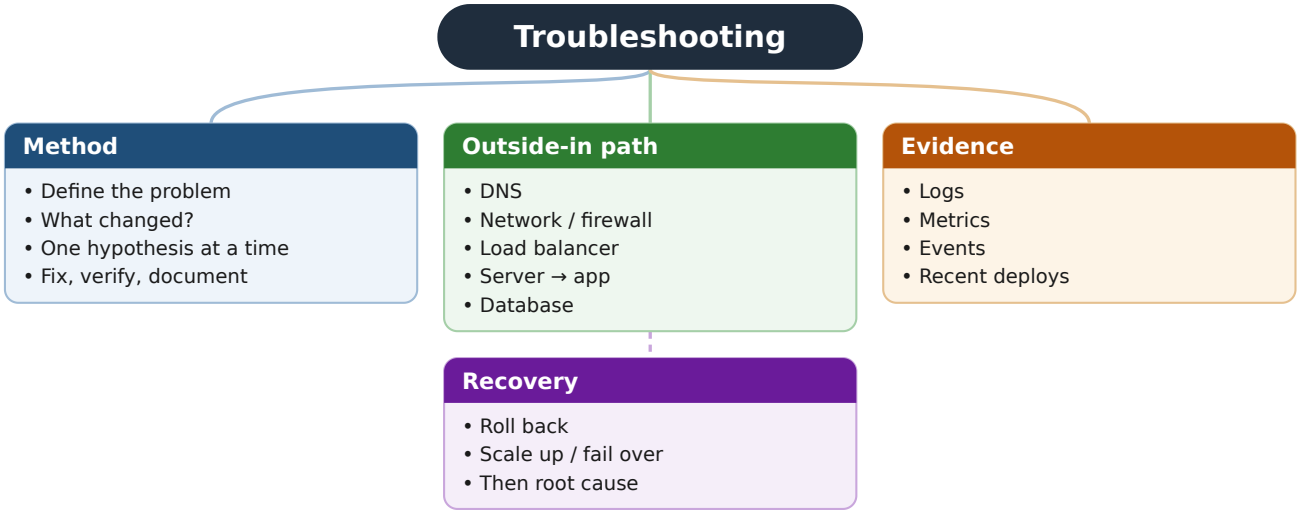
1. What is platform engineering and why has it grown? How does it relate to DevOps and SRE?
2. IDP vs developer portal: what's the difference?
3. What does "platform as a product" mean in daily work?
4. What is a golden path? What makes one successful?
5. Explain Team Topologies and the interaction modes for a platform team.
6. What is cognitive load and how does a platform reduce it?
7. What capabilities would you build first for a 30-team engineering organization, and why?
8. Explain Backstage's catalog, templates and TechDocs. What are the costs of running Backstage?
9. How do developers self-serve infrastructure safely? Terraform modules vs Crossplane vs ACK.
10. How do you choose the right abstraction level? How do you provide escape hatches?
11. How do you roll out updates (e.g. a new security scanner) to 200 services' pipelines?
12. Multi-tenancy options on Kubernetes: namespaces, vcluster, cluster per team: trade-offs.
13. How would you implement ephemeral preview environments per pull request?
14. How do you enforce security, compliance and cost controls without blocking developers?
15. Which metrics prove your platform is working?
16. Describe platform engineering anti-patterns you'd avoid.
17. Design an IDP on AWS/EKS end to end.
18. A team refuses to adopt the platform. What do you do?

## 16.14 Self-check

- [ ] I can explain IDP, portal, golden path, platform as a product and cognitive load.
- [ ] I can describe Team Topologies and how a platform team should interact with others.
- [ ] I can list the platform capabilities and the interfaces developers use (portal, CLI, API, Git).
- [ ] I can design a golden path for a new service end to end.
- [ ] I can write Backstage catalog entities and a scaffolder template.
- [ ] I can explain self-service infrastructure with Terraform modules and Crossplane.
- [ ] I can compare multi-tenancy models and design preview environments.
- [ ] I can define platform metrics and avoid the common anti-patterns.
- [ ] I can draw a reference IDP architecture on AWS and explain each component.

> **REMEMBER**
>
> - A good platform makes **the right way the easy way**.

# 17. Troubleshooting




## 17.0 Why a separate troubleshooting book

Each topic's notes end with their own troubleshooting table. Real incidents don't respect topic boundaries: "the website is slow" might be DNS, a full disk, a bad deploy, a database lock, an exhausted NAT gateway port range, or CPU throttling in Kubernetes. This book gives you **one method** and **cross-domain playbooks**, the way senior engineers actually work through problems under pressure.

## 17.1 The method

```
1. DEFINE      What exactly is wrong? For whom? Since when? How bad? (scope + impact + start time)
2. STABILIZE   If users are hurting: mitigate first (roll back, fail over, scale, disable feature)
3. WHAT CHANGED Deploys, config, infra, traffic, certificates, dependencies, time (cron, DST)
4. HYPOTHESIZE Based on evidence, list likely causes, most probable and cheapest to check first
5. TEST        One change or one check at a time; record what you did and saw (timestamps)
6. NARROW      Divide and conquer: which layer, which component, which hop
7. FIX + VERIFY Fix the cause, confirm with the same signal that showed the problem
8. PREVENT     Postmortem actions: alert, test, guardrail, runbook, automation
```

### Questions that solve half of all incidents

| Question | Why |
|---|---|
| **What changed?** | Most outages follow a change. Check deploy history, Git log of the config repo, Terraform applies, CloudTrail |
| Is it everyone or some users? | Some = one region/AZ/node/pod/version/browser/customer: compare good vs bad |
| Is it all requests or some endpoints? | One endpoint = code/dependency; all = infrastructure/network/capacity |
| Did it start suddenly or gradually? | Sudden = change or failure; gradual = leak, growth, saturation, certificate/disk countdown |
| Can I reproduce it? | Reproduction turns guessing into testing |

| Question | Why |
|---|---|
| What do the metrics, logs, traces and events say at the start time? | Evidence over intuition |

### Divide by layers (bottom up or top down)

```
User/browser ⟶ DNS ⟶ CDN/WAF ⟶ Load balancer ⟶ Ingress/Proxy ⟶ Service ⟶ Pod/Process
     ⟶ Application code ⟶ Dependencies (DB, cache, queue, external APIs)
Underneath everything: compute (CPU, memory, disk, network), OS, cloud quotas/limits, identity (IAM)
```

Test at a hop to cut the problem in half: if `curl` from inside the cluster to the Service works but from the internet it doesn't, everything from the Service inward is fine.

### Anti-patterns under pressure

- Changing several things at once (you'll never know what fixed or broke it).
- Restarting before capturing evidence (logs, heap dumps, `ps`, `ss` output). Capture first, then restart if needed.
- Trusting assumptions ("the config is definitely right"): verify.
- Debugging alone for hours during a SEV1: escalate, declare an incident, get an incident commander.
- Fixing in production by hand without putting the fix back into code (GitOps will revert it, or the next deploy will).

## 17.2 The toolkit by layer

| Layer | Commands / tools |
|---|---|
| Process / CPU / memory | `top` / `htop`, `ps aux --sort=-%cpu`, `free -h`, `vmstat 1`, `pidstat 1`, `/proc/<pid>/status` |
| Disk | `df -h`, `df -i`, `du -xh --max-depth=1 /`, `iostat -xz 1`, `lsof +L1` |
| Logs | `journalctl -u nginx --since "10 min ago"`, `journalctl -p err -b`, `dmesg -T`, `/var/log/*` |
| Services | `systemctl status/restart/cat nginx`, `systemctl --failed` |
| Network config | `ip a`, `ip r`, `ip neigh`, `resolvectl status`, `cat /etc/resolv.conf` |
| Connectivity | `ping`, `traceroute` / `mtr`, `nc -zv host port`, `curl -v`, `telnet` |
| Sockets | `ss -tulpn` (listening), `ss -tanp state established`, `ss -s` |
| DNS | `dig name`, `dig +trace name`, `dig @8.8.8.8 name`, `nslookup` |
| TLS | `openssl s_client -connect host:443 -servername host`, `curl -vI https://...` |
| Packets | `tcpdump -i any port 443 -nn`, Wireshark |
| HTTP | `curl -w '%{http_code} %{time_connect} %{time_starttransfer} %{time_total}\n' -o /dev/null -s URL` |
| Containers | `docker ps -a`, `docker logs --tail 100 -f`, `docker inspect`, `docker exec`, `docker stats` |
| Kubernetes | `kubectl get/describe/logs/events/top/debug` (Kubernetes notes 12.18) |
| AWS | Console health/status checks, CloudWatch metrics/logs, CloudTrail, VPC Reachability Analyzer, `aws sts get-caller-identity`, Systems Manager Session Manager |

| Layer | Commands / tools |
|---|---|
| Observability | Grafana dashboards, PromQL, LogQL/Logs Insights, traces |

### Linux performance in 60 seconds (Brendan Gregg's checklist)

```
uptime                  # load averages: trend over 1, 5, 15 minutes
dmesg -T | tail         # kernel errors: OOM killer, disk/network errors
vmstat 1 5              # r (run queue), free memory, si/so (swapping), us/sy/wa/st CPU
mpstat -P ALL 1 3       # per-CPU balance: one hot CPU = single-threaded bottleneck
pidstat 1 3             # which processes use CPU
iostat -xz 1 3          # disk: %util, await (latency), r/s w/s
free -m                 # memory and page cache
sar -n DEV 1 3          # network throughput per interface
sar -n TCP,ETCP 1 3     # TCP connections, retransmits
top                     # overview
```

## 17.3 Linux host playbooks

### High CPU

1. `top` (press `1` for per-CPU, `P` to sort): which process? `us` (user code), `sy` (kernel), `wa` (waiting for disk), `st` (stolen by hypervisor: noisy neighbor or burstable credits exhausted).
2. Process identified → is it expected (traffic growth) or abnormal (infinite loop, runaway job, crypto miner)?
3. Look inside: `pidstat -t -p <pid> 1` (threads), application profiler, `perf top`, Java `jstack`, Python `py-spy top --pid <pid>`.
4. On AWS t-family instances: CPU credit balance at zero → throttled to baseline: switch to unlimited mode or a non-burstable type.

### High load average but CPU idle

Load counts processes **running and waiting** (including uninterruptible disk I/O, state `D`). High load + low CPU = processes blocked on I/O: `vmstat` (`wa`, `b` column), `iostat -xz 1` (high `await`, `%util` near 100%), `ps -eo state,pid,cmd | grep '^D'`. Causes: slow/saturated disk (EBS throughput/IOPS limits, gp2 burst credits), NFS/EFS hangs.

### Memory pressure and OOM

```
free -h                                 # "available" is what matters, not "free"
ps aux --sort=-%mem | head
dmesg -T | grep -i -E "killed process|out of memory"
journalctl -k | grep -i oom
cat /proc/meminfo | grep -E "MemAvailable|Slab|Shmem"
```

Linux uses free RAM for page cache: low "free" is normal. Real trouble: low **available**, swapping (`si/so` in `vmstat`), the OOM killer in `dmesg`. Fix the leak (heap dumps, profiler), set limits (systemd `MemoryMax=`, container limits), right-size the instance, tune runtime heaps (JVM `-Xmx` must leave room for non-heap memory).

### Disk full

```
df -h                                     # which filesystem?
df -i                                     # inodes exhausted? (millions of tiny files)
sudo du -xh --max-depth=1 / 2>/dev/null | sort -h | tail -15   # biggest directories
sudo lsof +L1                             # deleted files still held open by a process
sudo journalctl --vacuum-size=500M        # shrink the systemd journal
docker system df && docker system prune   # container images/volumes (careful in prod)
```

Classic trap: you deleted a huge log but `df` still shows full, because the process still holds the file open. Restart/reload that process, or truncate instead of deleting: `: > /var/log/app/big.log`. Prevent with **logrotate**, journal limits, monitoring with `predict_linear` alerts, and growing EBS volumes online (`growpart` + `resize2fs` / `xfs_growfs`).

### A service won't start

```
systemctl status myapp                    # state, last log lines, exit code
journalctl -u myapp -n 100 --no-pager     # full recent logs
systemctl cat myapp                       # the unit file actually in use
sudo -u myapp /opt/myapp/bin/start        # run as the service user to see the real error
namei -l /opt/myapp/config.yml            # permissions along the whole path
ss -tulpn | grep :8080                    # port already in use?
```

Common causes: wrong path/permissions, missing environment file, port in use, config syntax error (`nginx -t`, `sshd -t`), dependency not started, SELinux/AppArmor denial (`ausearch -m avc -ts recent`, `getenforce`), `ExecStart` not absolute.

### "Too many open files"

`ulimit -n` (shell), `cat /proc/<pid>/limits` (actual process), `ls /proc/<pid>/fd | wc -l`. Raise with systemd `LimitNOFILE=65535` in the unit (not `/etc/security/limits.conf` for services). Also check for connection leaks in the app.

### Other host classics

| Symptom | Check |
|---|---|
| Clock wrong / TLS and auth failures ("certificate not yet valid", Kerberos/AWS signature errors) | `timedatectl`, chrony (`chronyc tracking`); AWS Time Sync at 169.254.169.123 |
| Can't SSH | Security group/NACL, sshd running, `~/.ssh` permissions (700, key 600), wrong user (`ec2-user` vs `ubuntu`), disk full, instance status checks; use **SSM Session Manager** instead |
| `Permission denied` despite correct mode | Ownership, parent directory `x` bit, ACLs (`getfacl`), SELinux context (`ls -Z`, `restorecon`), mount options (`noexec`) |
| Zombie processes | Parent isn't reaping children: fix/restart the parent; zombies themselves use no CPU/memory |
| Cron job didn't run | `grep CRON /var/log/syslog` or `journalctl -u cron`, PATH differences in cron, missing newline at the end of the crontab, timezone |

## 17.4 Network playbooks

### "I can't connect": walk the layers

```
# 1. Name resolution
dig +short api.example.com                    # nothing? -> DNS problem
# 2. Route to the network
ip route get 10.0.20.15                        # which interface / gateway?
traceroute -n 10.0.20.15                       # where does it stop?
# 3. Port reachable?
nc -zv -w 3 10.0.20.15 5432                    # succeeded / refused / timed out
# 4. Service listening on the server (run there)
ss -tulpn | grep 5432                          # listening on 0.0.0.0 or only 127.0.0.1?
# 5. Application protocol
curl -sv https://api.example.com/healthz       # TLS handshake, HTTP status, headers
```

### Read the error: refused vs timeout vs reset

| Error | Meaning | Typical causes |
|---|---|---|
| **Connection refused** | Packet reached the host; nothing listening on that port (TCP RST) | Service down, wrong port, listening on `127.0.0.1` only, container port not published |
| **Connection timed out** | No answer at all: packets dropped | Security group/NACL/firewall drop, wrong route, host down, wrong IP, no NAT/IGW for egress |
| **Connection reset by peer** | Connection was established, then killed | App crashed mid-request, proxy idle timeout, TLS mismatch, load balancer deregistration |
| **No route to host** | Routing or ARP failure (or ICMP reject by firewall) | Wrong subnet/route table, host down on the LAN |
| `Name or service not known` / NXDOMAIN | DNS has no answer | Typo, missing record, wrong DNS server, private zone not associated with the VPC |
| `SERVFAIL` | DNS server failed to resolve | DNSSEC problems, upstream DNS broken |

### AWS network checklist (instance A can't reach B)

| Check | Where |
|---|---|
| Security group on **B** allows inbound from A's SG/CIDR on the port | EC2 → Security groups (stateful: return traffic automatic) |
| Security group on **A** allows outbound | Usually all outbound allowed by default |
| NACLs on both subnets allow the port **and** ephemeral return ports (1024–65535) | VPC → Network ACLs (stateless) |
| Route tables: local route within VPC; peering/TGW routes both directions | VPC → Route tables |
| Internet egress: public subnet needs IGW route + public IP; private subnet needs NAT gateway route | Route tables, NAT status |
| Host firewall (iptables/firewalld/ufw) | On the instance |
| DNS: private hosted zone associated with VPC; `enableDnsSupport/Hostnames` on | Route 53, VPC settings |
| **VPC Reachability Analyzer**: tells you exactly which component blocks the path | VPC → Reachability Analyzer |

| Check | Where |
|---|---|
| VPC Flow Logs: `REJECT` entries show drops | CloudWatch Logs / S3 |

### DNS problems

```
dig api.example.com                          # answer, TTL, which server answered
dig @1.1.1.1 api.example.com                 # compare with a public resolver
dig +trace api.example.com                   # walk delegation from the root
dig NS example.com ; dig SOA example.com     # delegation and zone serial
getent hosts api.example.com                 # what the OS resolver (incl. /etc/hosts) returns
```

Traps: stale caches after a change (wait for the old **TTL**; lower TTL *before* migrations), `/etc/hosts` overrides, split-horizon DNS (different answers inside vs outside the VPC), CNAME at the zone apex (use Route 53 alias records), Kubernetes `ndots` lookups (12.7).

### TLS problems

```
openssl s_client -connect shop.example.com:443 -servername shop.example.com </dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
curl -vI https://shop.example.com 2>&1 | grep -E "SSL|subject|expire|issuer"
```

| Error | Cause |
|---|---|
| `certificate has expired` | Renewal failed (cert-manager, ACM DNS validation record removed) |
| `hostname mismatch` / `SAN` | Certificate doesn't cover this name (`www` vs apex, new subdomain) |
| `unable to get local issuer certificate` | Server doesn't send the intermediate chain, or client lacks the CA (corporate proxy CAs) |
| `handshake failure` / `protocol version` | No common TLS version/cipher; SNI missing |
| Wrong certificate served | Missing SNI (`-servername`), default certificate on the load balancer listener |

### Slow network

- Measure where time goes: `curl -w` (DNS, connect, TLS, first byte), `mtr` (per-hop loss and latency).
- Packet loss/retransmits: `sar -n ETCP 1`, `ss -ti` (retrans), `netstat -s | grep retrans`.
- **MTU problems** (VPN, Transit Gateway, jumbo frames): small requests work, large ones hang. Test with `ping -M do -s 1472 host` and lower until it passes.
- Cross-AZ or cross-region traffic adds latency and cost; NAT gateway throughput and **port exhaustion** (`ErrorPortAllocation` metric) for many outbound connections to one destination.

## 17.5 Web applications and HTTP

### Status codes as clues

| Code | Meaning | Where to look |
|---|---|---|
| 400 Bad Request | Malformed request | Client payload, headers too large, API validation |
| 401 Unauthorized | Not authenticated | Missing/expired token, clock skew, wrong auth header |

| Code | Meaning | Where to look |
|---|---|---|
| 403 Forbidden | Authenticated but not allowed | Authorization rules, WAF block, S3/IAM policy, CORS-related blocks in browsers |
| 404 Not Found | Path doesn't exist here | Wrong route/ingress path rule, missing file, wrong host header/virtual host |
| 405 / 413 / 429 | Method not allowed / body too large / rate limited | Proxy `client_max_body_size`, API Gateway/WAF throttling |
| 500 Internal Server Error | Application threw an unhandled error | **Application logs** and traces |
| **502 Bad Gateway** | Proxy/LB got an invalid response or connection error from the backend | Backend crashed or closed connection, wrong target port, keep-alive timeout mismatch (backend idle timeout shorter than LB's) |
| **503 Service Unavailable** | No healthy backend or service overloaded | Target group has no healthy targets, Kubernetes Service with no endpoints, maintenance mode, circuit breaker open |
| **504 Gateway Timeout** | Backend didn't answer in time | Slow queries, dependency timeouts, LB/ingress timeout too short for the operation, backend unreachable (security group) |

## 502/503/504 behind an ALB: decision path

```
Check ALB metrics: HTTPCode_ELB_5XX (generated by the ALB) vs HTTPCode_Target_5XX (from your app)
├── Target 5xx → application errors → app logs/traces
└── ELB 5xx
    ├── 503 → target group: healthy targets = 0? → health check path/port/codes, app startup,
    │                                               security group from ALB to targets
    ├── 502 → targets resetting connections → app crashes, keep-alive timeout < ALB idle timeout
    │                                          (60s default): set app keep-alive > 60s
    └── 504 → target too slow / unreachable → TargetResponseTime, DB latency, ALB idle timeout
```

## Latency spikes

1. Is it all endpoints or one? One → code path or its dependency (trace it). All → shared resource.
2. Saturation: CPU (and **CPU throttling** in containers), memory/GC pauses, thread or worker pool exhaustion, **database connection pool** exhaustion (requests wait for a connection).
3. Dependencies: database slow queries ( `pg_stat_statements`, slow query log, locks: `pg_locks` ), cache hit ratio drop (cold cache after deploy/flush), external API latency.
4. Infrastructure: noisy neighbor/steal time, EBS throttling, network retransmits, DNS resolution delays, cross-AZ hops.
5. Traffic shape: a retry storm (clients retrying without backoff multiply the load), one big customer, a bot.

## Memory leak pattern

Memory grows steadily after each deploy until OOMKilled/restart (sawtooth graph). Confirm with the memory metric over days, take heap dumps at two points and compare (Java: `jmap` /JFR; Python: `tracemalloc`, `memray`; Node: heap snapshots). Mitigate with restarts/limits while fixing; don't "fix" with bigger instances only.

### Database connection problems

| Symptom | Cause |
|---|---|
| `too many connections` | Each pod's pool × number of pods > DB `max_connections`. Use smaller pools, a pooler (PgBouncer, RDS Proxy) |
| Timeouts connecting | Security group, wrong endpoint (writer vs reader), DB failover in progress, DNS caching of old IP |
| Auth failed after rotation | App cached old password: reload secrets on rotation, use RDS Proxy/IAM auth |
| Slow after a deploy | New query without an index, ORM N+1 queries: check `EXPLAIN ANALYZE` |

## 17.6 Git problems

| Problem | Solution |
|---|---|
| Committed to the wrong branch (not pushed) | `git switch -c right-branch` (keeps the commit), then on the old branch `git reset --hard HEAD~1` |
| Undo the last commit, keep changes | `git reset --soft HEAD~1` |
| Undo a pushed commit safely | `git revert <sha>` (new commit; history unchanged) |
| "Lost" commits after a reset/rebase | `git reflog` → `git switch -c rescue <sha>` |
| Merge conflict | `git status` → edit the conflict markers → `git add` → `git commit` (or `git rebase --continue`); `git merge --abort` to back out |
| `rejected (non-fast-forward)` on push | Remote has new commits: `git pull --rebase` then push; never force-push shared branches (`--force-with-lease` only on your own) |
| Detached HEAD | You checked out a commit/tag: `git switch -c new-branch` to keep work, or `git switch main` |
| Secret committed | Rotate the secret **first**; then remove from history (`git filter-repo`) if needed (DevSecOps 14.3) |
| Huge file rejected by GitHub | Remove from history, use Git LFS or artifact storage |
| Line ending noise (Windows) | `git config --global core.autocrlf true` on Windows; `.gitattributes` with `* text=auto` |
| Authentication failed | HTTPS needs a token (not password); SSH: `ssh -T git@github.com`, correct key loaded (`ssh-add -l`) |

## 17.7 Docker problems

| Symptom | Check / fix |
|---|---|
| Container exits immediately | `docker logs <c>`, `docker inspect <c> --format '{{.State.ExitCode}}'`; the main process must run in the foreground |
| Exit code 137 | OOM-killed (`docker inspect` → `OOMKilled: true`) or `docker kill`; raise memory limit or fix the leak |
| Can't reach the app from the host | Port not published (`-p 8080:80`), app listening on `127.0.0.1` inside the container (must be `0.0.0.0`), wrong container port |

| Symptom | Check / fix |
|---|---|
| Containers can't reach each other by name | Use a user-defined network ( `docker network create` ); the default bridge has no DNS |
| `permission denied` on a bind mount | UID/GID mismatch between host files and the container user; SELinux `:z` label |
| Build is slow / always rebuilds everything | Layer order: copy dependency files and install before copying source; `.dockerignore` |
| `no space left on device` | `docker system df` , prune unused images, build cache ( `docker builder prune` ) |
| Image pull rate limited (429) | Authenticate to Docker Hub, use a mirror/ECR pull-through cache |
| `exec format error` | Image built for another CPU architecture (arm64 vs amd64): `docker buildx build --platform` |

## 17.8 Kubernetes quick decision tree

Full tables in the Kubernetes notes (12.18). The fastest path:

```
kubectl get pods -n <ns>
├── Pending ──────────────► describe pod → Events: resources / affinity / taints / PVC
├── ContainerCreating ────► describe pod → mount, secret/configmap missing, CNI IPs
├── ImagePullBackOff ─────► image name/tag, registry auth (ECR permissions), network to registry
├── CrashLoopBackOff ─────► logs --previous; exit code: 137 = OOM, 1 = app error, 127 = no command
├── Running 0/1 READY ────► readiness probe failing → probe path/port, app dependencies
└── Running 1/1 but broken ► Service selector ↔ labels, EndpointSlices, targetPort,
                             NetworkPolicy, Ingress rules/class, DNS name
Nodes: kubectl get nodes → NotReady → describe node (pressure conditions), kubelet logs
Recent change? kubectl rollout history; Argo CD history;
               kubectl get events -A --sort-by=.lastTimestamp
```

## 17.9 AWS playbooks

### IAM `AccessDenied`

Policy evaluation logic (in order):

```
1. Explicit DENY anywhere (identity policy, resource policy, SCP, RCP, boundary, session) ─► DENIED
2. Organizations SCPs / RCPs allow it?                          no ─► DENIED
3. Resource-based policy allows it (same account)?              may grant access by itself
4. Identity-based policy allows it?                             no (and no resource grant) ─► DENIED
5. Permissions boundary allows it? Session policy allows it?    no ─► DENIED
Default: implicit DENY. Cross-account: BOTH the identity policy AND the resource policy must allow.
```

```
aws sts get-caller-identity                    # who am I really? (wrong profile is very common)
aws sts decode-authorization-message --encoded-message "<message from the error>" \
  --query DecodedMessage --output text | jq .   # which statement denied it (needs permission)
aws iam simulate-principal-policy --policy-source-arn arn:aws:iam::123456789012:role/app \
  --action-names s3:GetObject --resource-arns arn:aws:s3:::acme-shop-uploads-prod/a.txt
```

Also check CloudTrail for the failed event ( `errorCode: AccessDenied` ): it shows the exact principal, action and resource. Frequent causes: wrong resource ARN (bucket ARN vs `bucket/*` object ARN), missing KMS key permission for encrypted objects, SCP region restriction, condition keys (source VPC endpoint, MFA, tags), and the instance/pod using a different role than you think.

## S3 403 checklist

Identity policy → bucket policy (explicit denies, `aws:SecureTransport`, VPC endpoint conditions) → Block Public Access (for public access) → **Object Ownership**/ACLs (objects uploaded by another account) → KMS key policy → the object actually exists (missing `s3:ListBucket` turns 404 into 403).

## EC2 instance unreachable

| Check | Tool |
|---|---|
| Instance state, **system** status check (AWS hardware/network) vs **instance** status check (your OS: kernel, disk full, network config) | EC2 console / `aws ec2 describe-instance-status` |
| Console output / screenshot | EC2 → Monitor and troubleshoot → Get system log / instance screenshot |
| Security group, NACL, route, public IP/EIP | Reachability Analyzer |
| SSH key/user, sshd | SSM Session Manager (requires SSM agent + instance profile + network path to SSM endpoints) |
| Recover a broken OS | EC2 Serial Console, or stop → detach root volume → attach to a rescue instance → fix → reattach |

## Other AWS classics

| Symptom | Likely cause |
|---|---|
| Private instance can't reach the internet / AWS APIs | No NAT gateway route, NAT in a subnet without IGW route, or missing VPC endpoints; NACL blocks ephemeral ports |
| Lambda timeout | Function in a VPC without NAT/endpoints for the service it calls; downstream slow; timeout too low; cold starts |
| Lambda `AccessDenied` / KMS errors | Execution role missing permission; environment variable encryption key policy |
| RDS connection refused/timeout | Security group from app SG, wrong endpoint, public access off (correct for prod), DB stopped/storage full |
| ALB health checks failing | Wrong path/port/success codes, security group from ALB to targets, app startup slower than the grace period |
| Auto Scaling keeps replacing instances | Health check failing during boot: increase grace period, fix user data errors ( `/var/log/cloud-init-output.log` ) |
| CloudFront 403/502 | Origin access control/bucket policy, origin protocol/certificate mismatch, origin timeout |
| **Sudden cost spike** | Cost Explorer by service/usage type → NAT gateway data, data transfer, forgotten instances/volumes/load balancers, logging volume, leaked keys used for mining. Set **budgets with alerts** |
| `LimitExceeded` / `InsufficientInstanceCapacity` | Service quotas (request increase); capacity: try other instance types/AZs |

## 17.10 Terraform problems

| Problem | Solution |
|---|---|
| `Error acquiring the state lock` | Another run is active, or a crashed run left the lock: confirm nobody is running, then `terraform force-unlock <LOCK_ID>` |
| Plan wants to recreate a resource unexpectedly | Read the `# forces replacement` line: an immutable argument changed; use `lifecycle { prevent_destroy = true }` for critical resources, `moved` blocks for renames |
| Drift (changes made in the console) | `terraform plan -refresh-only` to see it; fix in code or accept with `apply -refresh-only` |
| Resource exists but not in state | `import` block (or `terraform import`) then plan until no changes |
| `No valid credential sources found` | AWS profile/env vars/SSO login (`aws sso login`), `aws sts get-caller-identity` |
| Provider/version errors after an upgrade | Constrain versions, commit `.terraform.lock.hcl`, `terraform init -upgrade` deliberately |
| `Cycle` error | Two resources reference each other: split rules (e.g. separate `aws_security_group_rule`), remove the circular dependency |
| Destroy fails (dependency violation) | Resources created outside Terraform depend on it (ENIs from Lambda/EKS, objects in a bucket): find and remove them |
| State corruption/loss | Restore a previous state version (S3 versioning), never edit state by hand: use `terraform state mv/rm` |
| `count` / `for_each` "value unknown until apply" | Keys must be known at plan time: use static keys, not resource attributes |

## 17.11 Ansible problems

| Problem | Solution |
|---|---|
| `UNREACHABLE` | SSH: host, user, key, port, security group; test with `ssh -i key user@host`; `ansible all -m ping -vvv` |
| `Missing sudo password` / permission denied | `become: true`, `--ask-become-pass`, or passwordless sudo for the automation user |
| Module fails on Python | Wrong `ansible_python_interpreter`; missing Python libraries on the target (e.g. `python3-apt`) |
| Task always "changed" | Using `command` / `shell` without `creates` / `changed_when`; prefer real modules |
| Variable has an unexpected value | Variable precedence: `ansible -m debug -a "var=myvar"`, `ansible-inventory --host h1` |
| Handler didn't run | Task wasn't "changed", or play failed before handlers: `--force-handlers`, `meta: flush_handlers` |
| Slow runs | `gather_facts: false` when not needed, fact caching, `pipelining = True`, more `forks` |

## 17.12 CI/CD problems

| Problem | Check |
|---|---|
| Works locally, fails in CI | Different versions (pin tool versions), missing env vars/secrets, OS/architecture differences, cached state locally, test order dependence |
| Flaky tests | Time/order/network dependence; rerun once to confirm, then fix or quarantine *with a ticket* (never silently skip) |
| Secrets empty | Fork PRs don't receive secrets; environment-scoped secret used in a job without `environment:`; wrong name |
| OIDC `Not authorized to perform sts:AssumeRoleWithWebIdentity` | Trust policy `sub` condition doesn't match repo/branch/environment; missing `id-token: write` |
| Docker push denied | Registry login step, repository doesn't exist (ECR needs it created), IAM permissions |
| Pipeline slow | Cache dependencies and Docker layers, parallelize jobs, run only affected tests/paths |
| Deploy "succeeded" but old version runs | Same mutable tag ( `latest` ) with `IfNotPresent` , GitOps repo not updated, Argo CD not synced, wrong cluster/namespace |

## 17.13 Scenario drills

Practice talking through these out loud: hypothesis → check → finding → fix → prevention.

| # | Scenario | Key path to the answer |
|---|---|---|
| 1 | Website down right after a deploy | Roll back first; compare versions; check readiness and errors of new pods |
| 2 | Site slow every day at 02:00 | Scheduled job: backups, cron, batch jobs, log rotation; correlate timing |
| 3 | Disk 100% on a server, `du` shows little | Deleted-but-open files ( `lsof +L1` ), or another mount hidden under a mount point |
| 4 | Some users get errors, others don't | Compare: region/AZ, one bad node/pod, version skew, specific customer data, DNS cache |
| 5 | Pods `CrashLoopBackOff` after a config change | `logs --previous` , missing env/key; revert config in Git |
| 6 | `connection timed out` from app to RDS | Security group inbound from the app's SG, subnets/route, correct endpoint |
| 7 | 502s from the ALB a few times per minute | Keep-alive timeout mismatch, pods terminating without draining (preStop/SIGTERM) |
| 8 | AWS bill doubled this month | Cost Explorer by usage type; NAT data, idle resources, logs; budgets + tags |
| 9 | Terraform plan wants to destroy the production database | Stop; read why (forces replacement / renamed resource); `moved` block or fix argument; `prevent_destroy` |
| 10 | Certificate expired on Saturday | Renew/re-issue now; then automation + expiry alerts 14+ days ahead |

| # | Scenario | Key path to the answer |
|---|---|---|
| 11 | Memory grows until restart every 3 days | Leak: heap dumps, recent dependency changes; temporary restart schedule while fixing |
| 12 | Cannot SSH to an EC2 instance after a reboot | Status checks, console output: fstab error, disk full; Session Manager or rescue volume |
| 13 | Kubernetes node `NotReady` | Kubelet logs, disk/memory pressure, CNI, instance health; cordon/drain/replace |
| 14 | CI suddenly fails on all branches | External change: base image/tool version, expired token, registry outage, runner image update |
| 15 | DNS change not visible to users after an hour | Old TTL, resolver caches, wrong zone/delegation, private vs public zone |
| 16 | Latency up 3× but CPU low | Waiting: DB locks/slow queries, connection pool exhaustion, external API, CPU throttling, network |
| 17 | `AccessDenied` for a role that "has admin" | SCP, permission boundary, explicit deny in bucket/KMS policy, different role actually used |
| 18 | Queue backlog growing | Consumers down/slow, poison messages (DLQ), scaling not triggered, downstream throttling |
| 19 | Deploy succeeded, users still see the old version | CDN/browser cache, old pods still serving, wrong environment, mutable tag |
| 20 | Monitoring shows everything green, users complain | Monitoring the wrong thing (no user-facing SLI), synthetic checks missing, alerting broken (dead man's switch) |

## 17.14 Interview questions

1. Walk me through how you troubleshoot "the website is slow".
2. What do you do first in an outage: find the root cause or mitigate? Why?
3. High load average but low CPU usage: what's happening?
4. A disk is full, you deleted logs, still full: why?
5. Connection refused vs connection timed out: what does each tell you?
6. How do you troubleshoot DNS resolution problems?
7. 502 vs 503 vs 504 from a load balancer: causes of each.
8. An EC2 instance in a private subnet can't reach the internet: checks in order.
9. Explain IAM policy evaluation. How do you debug `AccessDenied`?
10. A pod is in `CrashLoopBackOff`: steps.
11. Terraform wants to replace a production database: what do you do?
12. How do you troubleshoot an intermittent problem you can't reproduce?
13. Tell me about the hardest production issue you solved (use: situation, impact, method, root cause, fix, prevention).
14. How do you avoid making an incident worse while troubleshooting?
15. What do you put in a runbook?

## 17.15 Self-check

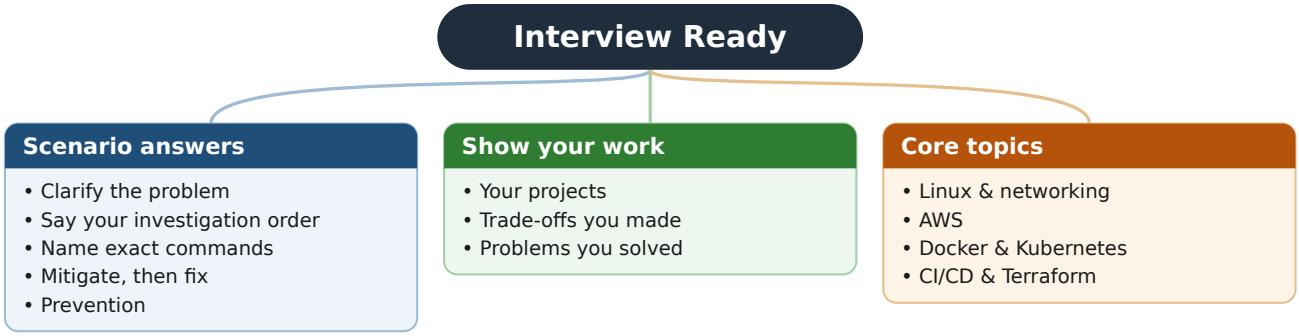
- [ ] I follow the method: define, stabilize, what changed, hypothesize, test, narrow, fix, prevent.
- [ ] I can run the 60-second Linux performance checklist and interpret it.
- [ ] I can solve the host playbooks: CPU, load, memory/OOM, disk full, services, file limits.
- [ ] I can walk a connection problem layer by layer and interpret refused/timeout/reset.

- [ ] I can debug DNS and TLS with dig and openssl.
- [ ] I can map HTTP 5xx errors behind a load balancer to causes.
- [ ] I can recover from common Git, Docker, Terraform, Ansible and CI/CD problems.
- [ ] I can debug IAM AccessDenied, S3 403 and EC2/VPC connectivity on AWS.
- [ ] I can talk through all 20 scenario drills confidently.

**REMEMBER**

- The first question is always **"What changed?"**
- Change one thing at a time and write down what you did.

# 18. Interview Preparation




## 18.0 How DevOps / Cloud interviews work

| Round | What they test | How to prepare |
|---|---|---|
| Recruiter screen | Fit, communication, salary range, notice period | 60-second introduction, clear story of why DevOps |
| Technical screen | Breadth of fundamentals (Linux, networking, Git, cloud, containers, CI/CD) | Rapid-fire answers in 18.3 |
| Hands-on / live task | Write a script, fix a Dockerfile/YAML, debug a broken environment, Terraform a resource | Labs from every topic; tasks in 18.6 |
| Scenario / troubleshooting | Your method under ambiguity | Troubleshooting notes (17) and 18.4 |
| System design | Architecture for availability, scale, security, cost | Framework and worked designs in 18.5 |
| Behavioral / hiring manager | Ownership, collaboration, incidents, learning | STAR stories in 18.7 |

Interviewers look for: **solid fundamentals**, a **structured way of thinking**, **real hands-on experience** (details only a doer knows), awareness of **trade-offs**, security and cost, and honest communication.

## 18.1 Resume, GitHub and LinkedIn

### Resume rules

- One or two pages; reverse chronological; a short summary line: *"DevOps engineer skilled in AWS, Terraform, Docker, Kubernetes and CI/CD, building automated, secure delivery pipelines."*
- Skills grouped: Cloud (AWS: EC2, VPC, IAM, S3, RDS, EKS) · IaC (Terraform, Ansible) · Containers (Docker, Kubernetes, Helm) · CI/CD (GitHub Actions, Jenkins, Argo CD) · Observability (Prometheus, Grafana, CloudWatch) · Scripting (Bash, Python) · OS (Linux).
- Every bullet = **action + technology + measurable result**:
- Weak: "Worked on CI/CD pipelines."
- Strong: "Built GitHub Actions pipelines with test, Trivy scanning and ECR push for 6 services, cutting release time from 2 hours to 12 minutes."
- Projects section (critical when you have no DevOps job yet): link each to a GitHub repo with a README, architecture diagram, and how to run it.

- Only list what you can defend in depth: every keyword invites questions.

### Portfolio from this program

| Project | Shows |
|---|---|
| Linux server automation (Bash + cron + logrotate) | Linux, scripting |
| Python automation (AWS boto3 reports, health checker with tests) | Python, AWS SDK, testing |
| AWS 3-tier architecture with Terraform (VPC, ALB, ASG, RDS) | Cloud architecture, IaC, cost awareness |
| Ansible role for web servers (idempotent, linted) | Configuration management |
| Dockerized app with Compose (multi-stage, non-root, healthchecks) | Containers |
| CI/CD pipeline (test, scan, build, push, deploy) | Delivery automation, DevSecOps |
| Kubernetes deployment with Helm/Kustomize + HPA + ingress | Kubernetes |
| Monitoring stack with SLO alerts | Observability, SRE |
| GitOps with Argo CD (dev → prod promotion) | Modern delivery |
| **Final project**: everything together on EKS | End-to-end ownership |

Each README: problem, architecture diagram, tools, how to deploy and **destroy**, security choices, costs, what you'd improve. Never commit secrets (scan with gitleaks before making repos public).

### LinkedIn

Headline with target role and stack, summary in first person, projects in Featured, posts about what you learned (short write-ups of labs and fixes build visibility). Certifications that help: **AWS Solutions Architect – Associate**, **CKA** (Kubernetes administrator), **HashiCorp Terraform Associate**; later AWS DevOps Engineer – Professional, CKS.

## 18.2 How to answer

### Technical questions: the 5-part answer

1. **Definition** in one sentence.
2. **How it works** (mechanism).
3. **Example** or command.
4. **Trade-offs / when not to use it.**
5. **Your experience** ("In my project I...").

Example: *"What is a Kubernetes readiness probe?"*

> "It's a periodic check the kubelet runs to decide whether a container can receive traffic (definition). If it fails, the pod is removed from the Service's endpoints but not restarted (mechanism). For example an HTTP GET on `/ready` every 5 seconds that checks the app has loaded its config (example). It differs from liveness, which restarts the container; using a dependency check in liveness can cause cascading restarts (trade-off). In my EKS project I added readiness on `/ready` and saw zero failed requests during rolling updates (experience)."

### Behavioral questions: STAR

**S**ituation (context, briefly) → **T**ask (your responsibility) → **A**ction (what *you* did, specifically: most of the answer) → **R**esult (measurable outcome + what you learned).

### Live problems: think aloud

State assumptions, ask clarifying questions, start simple, verify each step, mention what you'd do in production (tests, security, monitoring). Silence hurts more than a wrong first guess.

### When you don't know

"I haven't used X directly. My understanding is … because it's similar to Y which I've used. Here's how I'd find out: …" Never bluff; interviewers drill until bluffs collapse.

## 18.3 Rapid-fire fundamentals with model answers

Short, correct answers to the most-asked questions. Expand each with the 5-part method when the interviewer wants depth. Each topic's notes end with more questions.

### Linux

| Question | Model answer |
|---|---|
| What happens when you run a command in the shell? | The shell parses it, expands variables/globs, finds the program via `PATH`, `fork()`s a child and `exec()`s the program; the parent waits and gets the exit code (`$?`) |
| Hard link vs soft link? | Hard link = another name for the same inode (same filesystem, survives deleting the original). Symlink = a small file pointing to a path (can cross filesystems, breaks if target removed) |
| Explain `chmod 750 file` | Owner rwx (7), group r-x (5), others none (0) |
| What is a process vs a thread? Zombie vs orphan? | Process: own memory space; threads share it. Zombie: finished, parent hasn't read its exit status. Orphan: parent died; adopted by PID 1 |
| `kill` vs `kill -9`? | Default SIGTERM lets the process clean up; SIGKILL (9) can't be caught: use only as a last resort |
| What's load average? | Average number of processes running or waiting (CPU or uninterruptible I/O) over 1/5/15 min; compare with CPU count |
| How do you find what's using port 8080? | `ss -tulpn \| grep :8080` (or `lsof -i :8080`) |
| What is an inode? | Metadata structure for a file (owner, permissions, size, block pointers); filenames live in directories pointing to inodes. You can run out of inodes with free space left |
| How does systemd manage services? | Unit files define how to start/stop/restart; `systemctl enable` creates boot links; journald collects logs (`journalctl -u`) |
| How do you make a script run every day at 2 AM? | Cron `0 2 * * * /path/script.sh` (or a systemd timer), with logging and absolute paths |

### Networking

| Question | Model answer |
|---|---|
| What happens when you type a URL in a browser? | DNS resolution (cache → resolver → root → TLD → authoritative), TCP handshake to the IP on 443, TLS handshake (certificate check, keys), HTTP request, server/LB/app processing, response, rendering, more requests for assets |
| TCP vs UDP | TCP: connection-oriented, reliable, ordered, flow/congestion control (HTTP, SSH, DBs). UDP: connectionless, no guarantees, low overhead (DNS, video, VoIP, QUIC) |

| Question | Model answer |
|---|---|
| Explain the TCP three-way handshake | SYN → SYN-ACK → ACK; then data. Closing uses FIN/ACK in both directions |
| What is a subnet? How many hosts in a /26? | A range of IPs sharing a network prefix. /26 = 64 addresses, 62 usable hosts in classic networking (AWS reserves 5 per subnet → 59) |
| Public vs private IP; what is NAT? | Private ranges (10/8, 172.16/12, 192.168/16) aren't routed on the internet; NAT translates private addresses to a public one for outbound traffic |
| What does DNS TTL do? | How long resolvers cache a record; lower it before migrations |
| A record vs CNAME vs alias | A: name → IPv4. CNAME: name → another name (not at the zone apex). Route 53 alias: apex-capable pointer to AWS resources |
| Load balancer L4 vs L7 | L4 (NLB) routes on IP/port, very fast, preserves TCP; L7 (ALB) understands HTTP: host/path routing, headers, TLS termination, WAF |
| What is a reverse proxy? | A server in front of backends that receives client requests and forwards them (Nginx, ALB): TLS, routing, caching, load balancing |
| Ports to know | 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL, 6379 Redis, 6443 Kubernetes API, 9090 Prometheus |

## Git

| Question | Model answer |
|---|---|
| `git merge` vs `git rebase` | Merge keeps history and adds a merge commit; rebase replays commits onto a new base for linear history. Never rebase shared/pushed branches |
| `reset` vs `revert` | Reset moves the branch pointer (rewrites history; local use). Revert creates a new commit undoing an old one (safe on shared branches) |
| What is `git stash`? | Temporarily shelves uncommitted changes to switch context; `git stash pop` restores them |
| Branching strategy you prefer? | Trunk-based development with short-lived feature branches, PR reviews, CI on every PR, feature flags; GitFlow only for products with versioned releases |
| How do you recover a deleted branch? | `git reflog` to find the commit, then `git branch name <sha>` |

## Scripting (Bash / Python)

| Question | Model answer |
|---|---|
| What does `set -euo pipefail` do? | Exit on error ( `-e` ), error on unset variables ( `-u` ), fail a pipeline if any command fails ( `pipefail` ): safer scripts |
| `$@` vs `$*` ; `$?` ; `$$` | `"$@"` = all arguments as separate words (use this); `$?` = last exit code; `$$` = current PID |
| Why quote variables? | Unquoted variables undergo word splitting and globbing: `rm $file` with spaces deletes the wrong things. Use `"$file"` |
| How do you parse JSON in Bash? | `jq` , e.g. `aws ec2 describe-instances \| jq -r '.Reservations[].Instances[].InstanceId'` |
| Python: list vs tuple vs dict vs set | List: ordered, mutable; tuple: ordered, immutable; dict: key→value; set: unique unordered items with fast membership |

| Question | Model answer |
|---|---|
| How do you handle errors in Python? | `try/except` for specific exceptions, `finally`/context managers for cleanup, logging, non-zero exit code for scripts |
| How do you call AWS from Python? | `boto3` with credentials from the role/profile (never hardcoded), paginators for large lists, handle `ClientError` |

## AWS

| Question | Model answer |
|---|---|
| Region vs Availability Zone | Region = geographic area with multiple isolated AZs (data centers with independent power/network). Spread across AZs for HA, regions for DR/latency |
| Security group vs NACL | SG: instance/ENI level, stateful, allow rules only. NACL: subnet level, stateless (return traffic needs rules), allow and deny, ordered rules |
| Public vs private subnet | Public subnet's route table has a route to an Internet Gateway; private subnets reach the internet via a NAT gateway (outbound only) |
| IAM user vs role vs policy | User: long-term identity (avoid for apps). Role: assumable identity with temporary credentials (EC2, Lambda, pods, CI via OIDC, cross-account). Policy: JSON permissions document |
| How do you give an EC2 instance S3 access? | An IAM role with a least-privilege policy attached through an instance profile; never access keys on the instance |
| S3 storage classes | Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier Instant/Flexible/Deep Archive; lifecycle rules move/expire objects |
| How do you design a highly available web app? | Multi-AZ: ALB across ≥2 AZs, Auto Scaling group (or ECS/EKS) in private subnets, RDS Multi-AZ, S3/CloudFront for static assets, Route 53 health checks, backups |
| RDS Multi-AZ vs read replica | Multi-AZ: synchronous standby for automatic failover (HA). Read replica: asynchronous copy for read scaling (can be promoted; also cross-region DR) |
| What is the shared responsibility model? | AWS secures the cloud infrastructure; you secure your data, IAM, network config, OS patches and applications |
| How do you control AWS costs? | Tagging, budgets and alerts, right-sizing, Savings Plans/RIs, spot for flexible workloads, auto scaling, lifecycle policies, removing idle resources, watching NAT/data transfer |
| What is VPC peering vs Transit Gateway? | Peering: 1:1, non-transitive. Transit Gateway: hub connecting many VPCs and on-prem with central routing |
| CloudWatch vs CloudTrail | CloudWatch: metrics, logs, alarms (performance/operations). CloudTrail: audit log of API calls (who did what) |

## Docker

| Question | Model answer |
|---|---|
| Container vs virtual machine | Containers share the host kernel and isolate with namespaces/cgroups: lightweight, start in ms. VMs virtualize hardware with their own kernel: stronger isolation, heavier |
| Image vs container | Image: read-only layered template. Container: a running instance with a writable layer |
| `CMD` vs `ENTRYPOINT` | ENTRYPOINT sets the executable; CMD gives default arguments (overridable). Use exec (JSON) form so signals reach the process |

| Question | Model answer |
|---|---|
| How do you reduce image size? | Multi-stage builds, minimal base (slim/distroless/alpine), combine and clean package installs, `.dockerignore` |
| `COPY` vs `ADD` | COPY copies files; ADD also extracts archives and fetches URLs: prefer COPY |
| How do containers persist data? | Volumes (managed by Docker) or bind mounts; containers' writable layer is lost on removal |
| How do you secure containers? | Minimal pinned base images, non-root user, no secrets in images, scan (Trivy), read-only FS, drop capabilities, resource limits, signed images |

## Terraform

| Question | Model answer |
|---|---|
| What is Terraform state and why remote state? | State maps config to real resources and tracks attributes. Remote state (S3 with locking) enables teamwork, locking and encryption; never commit state (it contains secrets) |
| `terraform plan` vs `apply` | Plan shows the diff between config/state/reality; apply executes it. In CI: save the plan and apply exactly that plan after approval |
| `count` vs `for_each` | count: index-based (removing an item shifts indexes, causing recreation). for_each: keyed by map/set (stable): preferred for collections |
| What are modules? | Reusable packages of resources with inputs/outputs; version them; compose environments from them |
| How do you handle drift? | `plan -refresh-only` to detect, then either fix the code or revert the manual change; prevent console changes with permissions and pipelines |
| How do you manage secrets in Terraform? | Don't put them in code; read from Secrets Manager/SSM or let services manage them ( `manage_master_user_password` for RDS); mark outputs `sensitive` ; protect state |
| How do you import existing resources? | `import` blocks (or `terraform import` ), write matching config, plan until no changes |
| Workspaces vs separate directories for environments | Directories (or separate state/backends per env) are clearer and safer for prod; workspaces suit identical short-lived copies |

## Ansible

| Question | Model answer |
|---|---|
| What is Ansible and why agentless? | Configuration management over SSH (WinRM for Windows): no agent to install; push model from a control node |
| What is idempotency? | Running the same playbook again changes nothing if the system is already in the desired state; modules check before acting |
| Playbook vs role | Playbook maps hosts to tasks/roles; role is a reusable structured unit (tasks, handlers, templates, defaults, vars) |
| What are handlers? | Tasks run only when notified by a change, once at the end of the play (e.g. restart nginx after config change) |
| How do you store secrets? | Ansible Vault or lookups from a secret manager; `no_log: true` for sensitive tasks |

| Question | Model answer |
|---|---|
| Terraform vs Ansible | Terraform provisions infrastructure declaratively with state; Ansible configures servers/apps procedurally-declaratively without state. Commonly used together |

## CI/CD

| Question | Model answer |
|---|---|
| CI vs continuous delivery vs continuous deployment | CI: merge often, auto build/test. Delivery: always releasable, prod deploy by decision. Deployment: every passing change goes to prod automatically |
| What stages does your pipeline have? | Lint → unit tests → security scans (secrets, SAST, deps) → build image → scan image → push → deploy dev → integration tests → staging → approval → prod (progressive) → verify |
| How do you secure a pipeline? | Least-privilege tokens, OIDC to cloud, environment protections, pinned actions, no untrusted code with secrets, scanned artifacts, signed images |
| Blue/green vs canary vs rolling | Blue/green: two full environments, instant switch. Canary: small % first with metric analysis. Rolling: replace instances in batches |
| How do you roll back? | Redeploy the previous immutable artifact (or `git revert` in GitOps); keep DB migrations backward-compatible so rollbacks are safe |
| How do you speed up a slow pipeline? | Caching, parallel jobs, test splitting, running only affected parts, smaller images, faster runners |

## Kubernetes

| Question | Model answer |
|---|---|
| Control plane components | API server (front door), etcd (state), scheduler (places pods), controller manager (reconcile loops), cloud controller manager |
| Pod vs Deployment vs ReplicaSet | Pod: smallest unit (containers sharing network/storage). ReplicaSet keeps N pods. Deployment manages ReplicaSets for rolling updates/rollbacks |
| Liveness vs readiness vs startup probe | Liveness restarts a stuck container; readiness controls Service traffic; startup protects slow starts |
| Requests vs limits | Requests: what the scheduler reserves. Limits: ceiling; CPU throttled, memory over limit → OOMKilled |
| Service types | ClusterIP (internal), NodePort, LoadBalancer (cloud LB), ExternalName, headless |
| ConfigMap vs Secret | Both inject config; Secrets are for sensitive data (base64, not encrypted by default: use encryption at rest and external secret stores) |
| StatefulSet vs Deployment | StatefulSet gives stable names, ordered rollout and per-pod storage (databases, Kafka); Deployment for stateless apps |
| How does HPA work? | Scales replicas based on metrics (CPU % of requests, custom metrics) between min and max |
| How do you debug a CrashLoopBackOff? | `describe` (events, exit code), `logs --previous`, check config/secrets, probes, OOM, command |
| How do pods get AWS permissions on EKS? | EKS Pod Identity or IRSA: a ServiceAccount mapped to an IAM role; temporary credentials; never the node role |

### Monitoring and SRE

| Question | Model answer |
|---|---|
| Four golden signals | Latency, traffic, errors, saturation |
| SLI vs SLO vs SLA | SLI: measured ratio (good/valid events). SLO: internal target (99.9% over 30 days). SLA: contractual promise with penalties, looser than the SLO |
| Error budget for 99.9% per 30 days | 0.1% → 43.2 minutes of full downtime (or 0.1% of requests failing) |
| What should page someone? | Symptoms affecting users now (SLO burn, outage), always actionable with a runbook; everything else is a ticket |
| Prometheus counter queries | Use `rate()` over a window, then aggregate with `sum by (...)`; percentiles with `histogram_quantile` on bucket rates |
| What is a blameless postmortem? | A structured review of an incident focusing on systemic causes and action items, not on blaming individuals |

### Security and GitOps

| Question | Model answer |
|---|---|
| How do you avoid secrets in code? | Secret managers + IAM roles/OIDC, pre-commit and push-protection scanning, ExternalSecrets in Kubernetes, rotation |
| What is shift-left security? | Moving security checks early: IDE, pre-commit, PR (SAST, SCA, IaC scanning) where fixes are cheapest |
| What is least privilege? | Granting only the minimum permissions needed, scoped to specific resources and time |
| What is GitOps? | Git as the single source of truth for desired state; an in-cluster agent (Argo CD/Flux) pulls and continuously reconciles, giving audit trail, drift correction and easy rollback |
| Argo CD sync vs health | Sync: does live state match Git? Health: are the resources actually working? |

## 18.4 Scenario questions with model answers

**1. "Production is down after a deployment. What do you do?"**

> Confirm impact (dashboards, error rate, which services). Communicate: declare an incident, open a channel. Mitigate first: roll back to the previous version (redeploy the previous image or `git revert` in GitOps) since the timing points to the deploy. Verify recovery with the same metrics. Then investigate the diff (code, config, migrations), write a postmortem, and add the missing test or canary analysis that would have caught it.

**2. "An application is slow. How do you troubleshoot?"**

> Define: which endpoints, which users, since when. Check golden signals: latency percentiles, errors, traffic, saturation. Use traces to find the slow span. If it's the DB: slow query logs, locks, connection pool. If it's resources: CPU throttling, memory/GC, disk I/O. Check what changed (deploys, traffic, data volume). Fix, verify on p95/p99, add an alert/SLO if it wasn't caught early.

**3. "Your AWS bill doubled. What do you do?"**

Cost Explorer grouped by service, then usage type and tags, compared with last month. Typical culprits: NAT gateway data processing, inter-AZ/inter-region transfer, forgotten instances/EBS volumes/snapshots/load balancers, log ingestion, an autoscaler that didn't scale down, or a leaked key used for mining (check CloudTrail, unknown regions). Fix, then add budgets with alerts, tagging policies, and anomaly detection.

**4. "A developer needs access to production logs but not to change anything."**

Give a read-only role through SSO scoped to the log groups/namespaces they need (CloudWatch Logs read, or Grafana/Loki viewer role, or Kubernetes `view` role in the namespace without `secrets` and `pods/exec`), time-bound if possible, audited via CloudTrail.

**5. "How would you migrate an app from EC2 to Kubernetes?"**

Containerize (Dockerfile, config via env, logs to stdout, health endpoints, graceful shutdown). Write manifests/Helm chart with probes, resources, HPA, PDB. Externalize state (RDS/S3). Set up CI to build/scan/push and GitOps to deploy. Run in staging, load test, then shift traffic gradually (weighted DNS or ALB) with monitoring, keeping EC2 as a rollback until stable.

**6. "Terraform plan shows it will destroy and recreate the production database."**

Stop. Read which argument forces replacement. If it's a rename, use a `moved` block; if an immutable setting changed, revert it or plan a controlled migration (snapshot, maintenance window). Add `prevent_destroy` and deletion protection on critical resources; require plan review in CI.

**7. "A secret was committed to GitHub."**

Rotate/revoke it immediately, check usage logs for abuse, clean up, then remove from history if needed, and prevent recurrence with pre-commit secret scanning, push protection and moving to OIDC/roles.

**8. "Kubernetes pods keep getting OOMKilled."**

Confirm via `describe` (Last State: OOMKilled, exit 137). Compare memory usage to the limit over time: a leak (steady growth) vs undersized limit (spikes at load). Check runtime heap settings (JVM/Node). Fix the leak or right-size requests/limits using VPA recommendations; add an alert on memory vs limit.

**9. "Disk is filling up on a server every week."**

Find what grows ( `du` ), usually logs or temp files. Configure logrotate/journal limits, application log levels, ship logs centrally, clean temp files with a timer; alert with `predict_linear`. If data legitimately grows, move it to S3 or grow the volume and plan capacity.

**10. "How do you do a zero-downtime deployment?"**

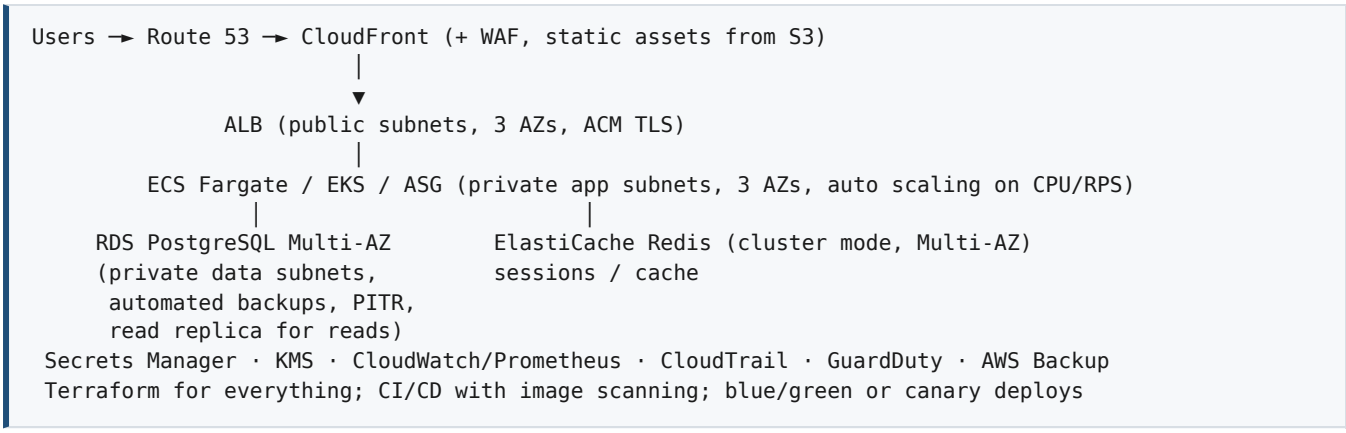
Rolling or blue/green with readiness probes, graceful shutdown (SIGTERM handling, preStop delay, connection draining), `maxUnavailable: 0`, backward-compatible DB migrations (expand/contract), enough capacity (PDBs), and automated verification with rollback.

## 18.5 System design for DevOps interviews

### The framework (spend the first minutes here)

1. **Clarify requirements:** users/traffic (requests/s, peak), latency, availability target (SLO), data size and consistency, regions, compliance, budget, team size.
2. **High-level architecture:** draw the request path and main components.
3. **Deep dive:** compute, data, networking, security, CI/CD, observability.
4. **Reliability:** failure modes (instance, AZ, region, dependency), scaling, backups and DR (RPO/RTO).
5. **Security and cost:** IAM, encryption, network isolation, secrets; cost drivers and optimizations.
6. **Trade-offs and evolution:** what you'd do first, what later, what you'd revisit.

### Design 1: highly available web application on AWS




Talking points: stateless app tier (sessions in Redis), Multi-AZ everywhere, health checks at every layer, autoscaling with headroom to survive losing one AZ, NAT gateway per AZ (availability vs cost), VPC endpoints for S3/ECR to cut NAT traffic, DR: backups copied cross-region (RPO hours) or warm standby in a second region (RPO minutes, higher cost).

### Design 2: CI/CD platform for 50 microservices

- Monorepo or polyrepo with a **reusable workflow** (test, scan, build, sign, push) versioned centrally; each service calls it.
- Ephemeral, autoscaled runners (GitHub-hosted or self-hosted on Kubernetes with ARC), OIDC to AWS, per-environment roles.
- Artifacts: ECR with immutable tags, scan on push, lifecycle policies; SBOM and signatures.
- Deployment: GitOps config repo, Argo CD ApplicationSets per environment, promotion by PR, Argo Rollouts canary with Prometheus analysis for critical services.
- Governance: branch protection, CODEOWNERS, required checks, Kyverno verifying signatures, audit trail in Git.
- Metrics: DORA dashboards, pipeline duration and flakiness, failed deploy rate.

### Design 3: observability for an EKS platform

- Metrics: kube-prometheus-stack per cluster (two replicas) → remote write to Amazon Managed Prometheus or Mimir; ServiceMonitors via golden-path templates.
- Logs: Fluent Bit/Alloy DaemonSet → Loki on S3 (or CloudWatch Logs) with retention tiers; JSON logs with trace IDs.
- Traces: OpenTelemetry SDK/auto-instrumentation → OTel Collector (tail sampling) → Tempo/X-Ray.
- Grafana with SSO and team folders; dashboards as code.
- Alerting: SLO burn-rate alerts per service, platform alerts (nodes, control plane, certificates), Alertmanager → PagerDuty with routing by team label; dead man's switch.

- Cost control: drop high-cardinality metrics, sample traces, log retention by class.

### Design 4: disaster recovery strategy

| Strategy | RPO / RTO | Cost | How |
|---|---|---|---|
| Backup and restore | Hours / hours–day | Lowest | Cross-region backups (AWS Backup, snapshots), IaC to rebuild |
| Pilot light | Minutes / tens of minutes | Low | Data replicated (cross-region read replica, S3 replication), minimal core running, scale up on failover |
| Warm standby | Seconds–minutes / minutes | Medium | Scaled-down full copy running in the second region |
| Multi-site active/active | Near zero / near zero | Highest | Both regions serve traffic (Route 53 latency/failover, global databases like Aurora Global/DynamoDB global tables) |

Always: define RPO/RTO with the business, automate the failover runbook, **test it regularly** (game days).

## 18.6 Hands-on tasks you may get (with solutions)

### Task 1: top 5 client IPs in an Nginx access log

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -5
```

Follow-ups: count 5xx responses per minute ( `awk '$9 ~ /^5/ {print substr($4, 2, 17)}' access.log | uniq -c` ), handle rotated `.gz` logs ( `zcat -f access.log*` ).

### Task 2: disk usage alert script

```
#!/usr/bin/env bash
# disk-alert.sh: warn when any filesystem is above a threshold (default 80%)
set -euo pipefail

threshold="${1:-80}"
if ! [[ "$threshold" =~ ^[0-9]+$ ]] || (( threshold < 1 || threshold > 99 )); then
  echo "usage: $0 [threshold 1-99]" >&2
  exit 2
fi

status=0
while read -r usage mount; do
  pct="${usage%\%}"
  if (( pct >= threshold )); then
    echo "WARNING: ${mount} is at ${pct}% (threshold ${threshold}%)"
    status=1
  fi
done < <(df -P -x tmpfs -x devtmpfs | awk 'NR > 1 {print $5, $6}')

if (( status == 0 )); then
  echo "OK: all filesystems below ${threshold}%"
fi
exit "$status"
```

Exit code 1 when any filesystem is over the threshold, so cron/monitoring can act on it.

## Task 3: check a list of URLs

```
#!/usr/bin/env bash
# check-urls.sh: print HTTP status and total time for each URL in a file
set -euo pipefail

file="${1:?usage: $0 urls.txt}"
failed=0
while IFS= read -r url; do
  [[ -z "$url" || "$url" == \#* ]] && continue
  result=$(curl -s -o /dev/null -m 10 -w '%{http_code} %{time_total}' "$url" || true)
  code="${result%% *}"
  if [[ "$code" =~ ^(2|3)[0-9][0-9]$ ]]; then
    echo "OK   ${result} ${url}"
  else
    echo "FAIL ${result:-000} ${url}"
    failed=$((failed + 1))
  fi
done < "$file"
echo "failed: ${failed}"
(( failed == 0 ))
```

## Task 4: summarize HTTP status codes from a log with Python

```
#!/usr/bin/env python3
"""Count HTTP status codes in an Nginx/Apache combined log and print JSON."""
import json
import re
import sys
from collections import Counter

# 1.2.3.4 - - [18/Oct/2026:10:00:00 +0530] "GET /x HTTP/1.1" 200 512 "-" "curl/8"
LINE = re.compile(r'^(\S+) \S+ \S+ \[[^\]]+\] "[^"]*" (\d{3}) ')


def summarize(lines):
    codes = Counter()
    ips = Counter()
    bad = 0
    for line in lines:
        match = LINE.match(line)
        if not match:
            bad += 1
            continue
        ip, status = match.groups()
        codes[status] += 1
        ips[ip] += 1
    total = sum(codes.values())
    errors = sum(n for code, n in codes.items() if code.startswith("5"))
    return {
        "total": total,
        "by_status": dict(sorted(codes.items())),
        "error_rate_5xx": round(errors / total, 4) if total else 0.0,
        "top_ips": ips.most_common(3),
        "unparsed_lines": bad,
    }


if __name__ == "__main__":
    path = sys.argv[1] if len(sys.argv) > 1 else "/var/log/nginx/access.log"
    with open(path, encoding="utf-8", errors="replace") as handle:
        print(json.dumps(summarize(handle), indent=2))
```

### Task 5: find EC2 instances missing an `owner` tag (boto3)

```
#!/usr/bin/env python3
"""List EC2 instances without an 'owner' tag in one region (read-only)."""
import sys

import boto3
from botocore.exceptions import BotoCoreError, ClientError


def untagged_instances(region: str, tag_key: str = "owner"):
    ec2 = boto3.client("ec2", region_name=region)
    paginator = ec2.get_paginator("describe_instances")
    for page in paginator.paginate():
        for reservation in page["Reservations"]:
            for inst in reservation["Instances"]:
                tags = {t["Key"]: t["Value"] for t in inst.get("Tags", [])}
                if tag_key not in tags:
                    yield inst["InstanceId"], inst["State"]["Name"], tags.get("Name", "-")


if __name__ == "__main__":
    region = sys.argv[1] if len(sys.argv) > 1 else "ap-south-1"
    try:
        for instance_id, state, name in untagged_instances(region):
            print(f"{instance_id}\t{state}\t{name}")
    except (BotoCoreError, ClientError) as err:
        print(f"error: {err}", file=sys.stderr)
        sys.exit(1)
```

### Task 6: fix this Dockerfile

```
# BEFORE: what's wrong?
FROM python:latest
COPY . /app
RUN pip install -r /app/requirements.txt
ENV DB_PASSWORD=supersecret
CMD python /app/app.py
```

Problems: unpinned `latest` base (unreproducible), huge full image, `COPY .` before dependency install (no layer caching, copies `.git` and secrets), secret baked into the image, runs as root, shell-form `CMD` (signals not delivered), no health check.

```
# AFTER
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD ["python", "-c", "import urllib.request as u; u.urlopen('http://localhost:8000/healthz')"]
CMD ["python", "-m", "src.app"]
```

The password comes from the runtime environment (secret manager → env var or mounted file), plus a `.dockerignore` excluding `.git`, `.env`, tests and caches.

### Task 7: why doesn't this Service get traffic?

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: frontend          # BUG 1: matches no pods (pods are labeled app=web)
  ports:
    - port: 80
      targetPort: 8080     # BUG 2: nginx listens on 80
```

Fix: `selector: {app: web}` and `targetPort: 80`. Then also mention what's missing for production: resources, probes, securityContext, PDB. Diagnose live with `kubectl get endpointslices -l kubernetes.io/service-name=web` (empty).

### Task 8: Terraform an S3 bucket the secure way

```
resource "aws_s3_bucket" "artifacts" {
  bucket = "acme-artifacts-${var.env}"
  tags = {
    owner = "platform"
    env   = var.env
  }
}

resource "aws_s3_bucket_versioning" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_public_access_block" "artifacts" {
  bucket                  = aws_s3_bucket.artifacts.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

variable "env" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.env)
    error_message = "env must be dev, staging or prod."
  }
}
```

## 18.7 Behavioral questions (STAR)

Prepare 6–8 stories from your projects, labs, studies or previous jobs; each story can answer several questions.

| Question | What they want |
|---|---|
| Tell me about yourself | 60–90 seconds: background → why DevOps → what you built (projects) → what you want next |
| A time you solved a hard technical problem | Method, persistence, learning |
| A failure or mistake you made | Ownership, honesty, what you changed afterwards |
| A conflict with a teammate / developer | Empathy, data-driven discussion, outcome |
| When you automated something | Initiative, impact in time/errors saved |
| How you learn new technology | Concrete approach: docs, labs, building projects, teaching others |

| Question | What they want |
|---|---|
| Working under pressure (incident) | Calm, structured, communication |
| Why this company / role? | You researched them: their stack, product, engineering blog |

**Example STAR (from this program's final project):**

> **S:** In my final project I deployed a microservice app on EKS, and every deployment caused a burst of 502 errors. **T:** I owned the delivery pipeline and wanted zero-downtime releases. **A:** I reproduced it with a load test during a rollout, saw pods were killed while the ALB still sent traffic, then added a readiness probe, a `preStop` sleep longer than the target deregistration delay, SIGTERM handling in the app, and `maxUnavailable: 0`. I added an alert on ALB 5xx during deployments. **R:** Rollouts now complete with zero failed requests under load; I documented it as a checklist in the repo's README.

## 18.8 Questions to ask the interviewer

- What does a typical week look like for this role? How much is project work vs operations?
- How are deployments done today, and how often? What's the biggest pain in the delivery process?
- How does on-call work, and how many pages does a typical shift get?
- How do you measure reliability (SLOs)? How are incidents reviewed?
- What would success look like in the first 90 days?
- How does the team learn (certifications budget, mentoring, internal tech talks)?

## 18.9 Job search strategy

| Target roles | Focus |
|---|---|
| Junior DevOps / Cloud Engineer | Linux, AWS core, Terraform, Docker, CI/CD, scripting |
| Cloud Support / Operations Engineer | Strong troubleshooting, networking, AWS services: a common entry door |
| Site Reliability Engineer (junior) | Linux, monitoring, incidents, scripting, Kubernetes |
| Platform Engineer (later) | Kubernetes, GitOps, IaC at scale, developer experience |

- Apply with a **tailored resume** per job description (mirror their stack where you truly have it).
- Referrals beat cold applications: engage with engineers on LinkedIn, local meetups (AWS User Groups, CNCF/Kubernetes community events), and open-source contributions (documentation fixes count).
- Track applications in a spreadsheet (company, role, date, stage, follow-up).
- After each interview, write down every question you got and your gaps; turn them into labs.
- Research the salary range for the role and city from several sources before the recruiter call; state a researched range, and negotiate on the full package (base, bonus, learning budget, remote policy).

## 18.10 Final two-week revision plan

| Days | Focus |
|---|---|
| 1–2 | Linux and networking: rapid-fire tables, 60-second checklist, subnetting drills |
| 3 | Git, Bash and Python: write tasks 1–5 from memory |

| Days | Focus |
|---|---|
| 4–5 | AWS: VPC design drawing from memory, IAM evaluation, HA architecture, costs |
| 6 | Docker and Terraform: Dockerfile and S3/VPC Terraform from memory |
| 7 | Ansible and CI/CD: pipeline stages, deployment strategies, security |
| 8–9 | Kubernetes: write all core manifests from memory; troubleshooting tree |
| 10 | Monitoring/SRE and DevSecOps: PromQL library, SLO math, leaked-secret playbook |
| 11 | GitOps and platform engineering: Argo CD flow, ApplicationSets, golden paths |
| 12 | System design: practice designs 1–4 out loud, 30 minutes each, on paper |
| 13 | Behavioral: rehearse 6–8 STAR stories; record yourself |
| 14 | Mock interview end to end; review weak spots; rest |

## Final checklist

- [ ] Resume with measurable project bullets; GitHub repos with READMEs and diagrams; no secrets anywhere.
- [ ] I can explain every project end to end, including why I chose each tool and what I'd improve.
- [ ] I can answer every rapid-fire question in 18.3 in under a minute.
- [ ] I can complete tasks 1–8 in 18.6 without notes.
- [ ] I can whiteboard designs 1–4 with trade-offs, security and cost.
- [ ] I have STAR stories ready and questions to ask the interviewer.

**REMEMBER**

- Behavioral questions: **STAR** = Situation, Task, Action, Result.
- "I don't know, but here is how I would find out" is a good answer.